MATHEUS FACURE ALVES

Causal Inference

for the Brave and True

A light-hearted yet rigorous approach to learning impact estimation and sensitivity analysis.

All in Python, and with as many memes as I could find.

OPEN-SOURCE EDITION

Causal Inference for the Brave and True

Causal Inference

for the Brave and True

A light-hearted yet rigorous approach to learning impact estimation and sensitivity analysis

Matheus Facure Alves

Causal Inference for the Brave and True
by Matheus Facure Alves

Copyright © 2020 Matheus Facure.

This book is open source. Its text and code are released under the MIT License:

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Notebooks, data and code: https://github.com/matheusfacure/python-causality-handbook
Online edition: https://matheusfacure.github.io/python-causality-handbook/

If you found this book valuable, you can support it on Patreon at
https://www.patreon.com/causal_inference_for_the_brave_and_true.

This edition was typeset on October 26, 2026 from the book's Jupyter notebooks (source revision `4df1445`).

What happens in a man's life is already written. A man must move through life as his destiny wills.

CAINE

Yes, yet each man is free to live as he chooses. Though they seem opposite, both are true.

OLD MAN

Brief Contents

Preface xv

PART I · The Yang 1

1 Introduction to Causality 3

2 Randomised Experiments 17

3 Stats Review: The Most Dangerous Equation 25

4 Graphical Causal Models 43

5 The Unreasonable Effectiveness of Linear Regression 61

6 Grouped and Dummy Regression 75

7 Beyond Confounders 89

8 Instrumental Variables 111

9 Non Compliance and LATE 129

10 Matching 141

11 Propensity Score 157

12 Doubly Robust Estimation 177

13 Difference-in-Differences 187

14 Panel Data and Fixed Effects 197

15 Synthetic Control 213

16 Regression Discontinuity Design 235

PART II · The Yin 253

17 Predictive Models 101 255

18 Heterogeneous Treatment Effects and Personalization 279

19 Evaluating Causal Models 295

20 Plug-and-Play Estimators 313

21 Meta Learners 327

22 Debiased/Orthogonal Machine Learning 341

23 Challenges with Effect Heterogeneity and Nonlinearity 369

24 The Difference-in-Differences Saga 387

25 Synthetic Difference-in-Differences 417

Appendices 451

A Debiasing with Orthogonalization 453

B Debiasing with Propensity Score 467

C When Prediction Fails 485

D Why Prediction Metrics are Dangerous for Causal Models 509

E Conformal Inference for Synthetic Controls 515

Index 533

Contents in Detail

Preface xv

PART I · The Yang 1

1 **Introduction to Causality** 3
- Why Bother? 3
- Data Science is Not What it Used to Be (or it Finally Is) 3
- Answering a Different Kind of Question 4
- When Association IS Causation 5
- Bias 10
- Key Ideas 14
- References 14

2 **Randomised Experiments** 17
- The Gold Standard 17
- In a School Far, Far Away 18
- The Ideal Experiment 21
- The Assignment Mechanism 22
- Key Ideas 23
- References 23

3 **Stats Review: The Most Dangerous Equation** 25
- The Standard Error of Our Estimates 29
- Confidence Intervals 30
- Hypothesis Testing 35
- P-values 39
- Key Ideas 40
- References 41

4 Graphical Causal Models **43**
Thinking About Causality . . . 43
Crash Course in Graphical Models . . . 45
Confounding Bias . . . 50
Selection Bias . . . 53
Key Ideas . . . 58
References . . . 58

5 The Unreasonable Effectiveness of Linear Regression **61**
All You Need is Regression . . . 61
Regression Theory . . . 63
Regression For Non-Random Data . . . 65
Omitted Variable or Confounding Bias . . . 69
Key Ideas . . . 73
References . . . 73

6 Grouped and Dummy Regression **75**
Regression With Grouped Data . . . 75
Regression for Dummies . . . 80
Key Ideas . . . 87
References . . . 87

7 Beyond Confounders **89**
Good Controls . . . 89
Mostly Harmful Controls . . . 95
Bad Controls - Selection Bias . . . 100
Key Ideas . . . 108
References . . . 108

8 Instrumental Variables **111**
Going Around Omitted Variable Bias . . . 111
Quarter of Birth and the Effect of Education on Wage . . . 114
The 1st Stage . . . 117
The Reduced Form . . . 119
Instrumental Variables by Hand . . . 121
Multiple Instruments . . . 122
Weakness of Instruments . . . 123
Key Ideas . . . 127
References . . . 127

9 Non Compliance and LATE **129**
Dipping our Toes into a Heterogeneous World . . . 129
Local Average Treatment Effect: LATE . . . 134
Effect on Engagement . . . 136
Key Ideas . . . 138
References . . . 138

10 Matching **141**
What is Regression Doing After All? . . . 141
The Subclassification Estimator . . . 143
Matching Estimator . . . 144
Matching Bias . . . 150
The Curse of Dimensionality . . . 154
Key Ideas . . . 155
References . . . 155

11 Propensity Score **157**
The Psychology of Growth . . . 157
Propensity Score . . . 161
Propensity Weighting . . . 163
Propensity Score Estimation . . . 165
Standard Error . . . 168
Common Issues with Propensity Score . . . 170
Propensity Score Matching . . . 173
Key Ideas . . . 175
References . . . 175

12 Doubly Robust Estimation **177**
Don't Put All your Eggs in One Basket . . . 177
Doubly Robust Estimation . . . 179
Key Ideas . . . 184
References . . . 184

13 Difference-in-Differences **187**
Three Billboards in the South of Brazil . . . 187
DID Estimator . . . 189
Non Parallel Trends . . . 193
Key Ideas . . . 195
References . . . 195

14 Panel Data and Fixed Effects **197**
Parallel Trends . . . 198
Controlling What you Cannot See . . . 199
Fixed Effects . . . 201
Visualizing Fixed Effects . . . 207
Time Effects . . . 209
When Panel Data Won't Help You . . . 210
Key Ideas . . . 211
References . . . 211

15 Synthetic Control **213**
One Amazing Math Trick to Learn What can't be Known . . . 213
We have Time . . . 216
Synthetic Control as Linear Regression . . . 219

Don't Extrapolate . . . 222
Making Inference . . . 226
Key Ideas . . . 232
References . . . 232

16 Regression Discontinuity Design 235
Is Alcohol Killing You? . . . 236
RDD Estimation . . . 238
Sheepskin Effect and Fuzzy RDD . . . 245
Key Ideas . . . 251
References . . . 252

PART II · The Yin 253

17 Predictive Models 101 255
Machine Learning in the Industry . . . 256
Machine Learning Crash Course . . . 261
Cross Validation . . . 263
Predictions and Policies . . . 265
One Feature Policies . . . 265
Machine Learning Models as Policy Inputs . . . 270
Fine Grain Policy . . . 274
Key Ideas . . . 277
References . . . 277

18 Heterogeneous Treatment Effects and Personalization 279
From Predictions to Causal Inference . . . 279
From ATE to CATE . . . 281
Predicting Sensitivity . . . 283
Key Ideas . . . 292
References . . . 293

19 Evaluating Causal Models 295
Sensitivity by Model Band . . . 298
Cumulative Sensitivity Curve . . . 302
Cumulative Gain Curve . . . 305
Taking Variance Into Account . . . 308
Key Ideas . . . 311
References . . . 311

20 Plug-and-Play Estimators 313
Problem Setup . . . 313
Target Transformation . . . 315
The Continuous Treatment Case . . . 319
Key Ideas . . . 325
References . . . 326

21 Meta Learners **327**
S-Learner (aka, the Go-Horse Learner) . . . 328
T-Learner . . . 331
X-Learner . . . 334
Key Ideas . . . 338
References . . . 338

22 Debiased/Orthogonal Machine Learning **341**
ML for Nuisance Parameters . . . 343
Non Parametric Double/Debiased ML . . . 352
Non-Scientific Double/Debiased ML . . . 358
Key Ideas . . . 366
References . . . 367

23 Challenges with Effect Heterogeneity and Nonlinearity **369**
Treatment Effects on Binary Outcomes . . . 369
Simulating Some Data . . . 371
Continuous Treatment and Non-linearity . . . 380
Key Concepts . . . 384
Reference . . . 385

24 The Difference-in-Differences Saga **387**
1) Birth: The Promise of Panel Data . . . 388
2) Death: Failures over Effect Heterogeneity . . . 395
3) Enlightenment: A Flexible Functional Form . . . 408
Key Concepts . . . 414
Reference . . . 415

25 Synthetic Difference-in-Differences **417**
Diff-in-Diff Revisited . . . 420
Synthetic Controls Revisited . . . 424
Synthetic Diff-in-Diff . . . 427
Time Effect Heterogeneity and Staggered Adoption . . . 437
Placebo Variance Estimation . . . 443
Key Concepts . . . 448
References . . . 449

Appendices **451**

A Debiasing with Orthogonalization **453**
Linear Regression Reborn . . . 453
The Intuition Behind Orthogonalization . . . 455
Orthogonalization with Machine Learning . . . 462
Key Ideas . . . 465
References . . . 466

B Debiasing with Propensity Score **467**

Inverse Probability of Treatment Weighting . . . 470
The Weakness of Propensity Score . . . 476
Positivity or Common Support . . . 479
Key Ideas . . . 483
References . . . 484

C When Prediction Fails 485
When all you have is a Hammer… . . . 485
Who Wants a Coupon? . . . 487
Simple Policy . . . 491
Policy With Model . . . 493
Hammering Your Thumb with Predictions . . . 500
When Might Prediction Helps . . . 503
A Graphical Explanation . . . 505
Key Ideas . . . 507
References . . . 508

D Why Prediction Metrics are Dangerous for Causal Models 509
Simulating Data . . . 510
Predictive Metric For Causal Inference . . . 512

E Conformal Inference for Synthetic Controls 515
Synthetic Control Refresher . . . 515
Inference for Grown Ups . . . 520
Reference . . . 532

Index 533

Preface

A light-hearted yet rigorous approach to learning impact estimation and sensitivity analysis. Everything in Python and with as many memes as I could find.

Part I of the book contains core concepts and models for causal inference. You will learn how to represent causal questions with potential outcome notation, learn about causal graphs, what is bias and how to deal with it. Most of the content here is well established. It's an amalgamation of materials I've found on books, university curriculums and online courses. You can think of Part I as the solid and safe foundation to your causal inquiries.

Part II (WIP) contains modern development and applications of causal inference to the (mostly tech) industry. While Part I focuses mostly on identifying average treatment effects, Part II takes a shift to personalization and heterogeneous effect estimating with CATE models. Most of what is written there was taken from my personal experience and is by no means well established science. It's much more experimental and subject to change, afterall, I, too, am learning.

Acknowledgments

I like to think of this entire series as a tribute to Joshua Angrist, Alberto Abadie and Christopher Walters for their amazing Econometrics class. Most of the ideas from Part I are taken from their classes at the American Economic Association. Watching them was what kept me sane during the tough year of 2020.

- Cross-Section Econometrics[1]
- Mastering Mostly Harmless Econometrics[2]

1 https://www.aeaweb.org/conference/cont-ed/2017-webcasts

2 https://www.aeaweb.org/conference/cont-ed/2020-webcasts

I'll also like to reference the amazing books from Angrist. They have shown me that Econometrics, or 'Metrics as they call it, is not only extremely useful but also profoundly fun.

- Mostly Harmless Econometrics[3]
- Mastering 'Metrics[4]

My final reference is Miguel Hernan and Jamie Robins' book. It has been my trustworthy companion in the most thorny causal questions I had to answer.

- Causal Inference Book[5]

Contribute

Causal Inference for the Brave and True is an open-source material on causal inference, the statistics of science. It uses only free software, based in Python. Its goal is to be accessible monetarily and intellectually. If you found this book valuable and you want to support it, please go to Patreon[6]. If you are not ready to contribute financially, you can also help by fixing typos, suggesting edits or giving feedback on passages you didn't understand. Just go to the book's repository and open an issue[7]. Finally, if you liked this content, please share it with others who might find it useful and give it a star on GitHub[8].

3 https://www.mostlyharmlesseconometrics.com/
4 https://www.masteringmetrics.com/
5 https://www.hsph.harvard.edu/miguel-hernan/causal-inference-book/
6 https://www.patreon.com/causal_inference_for_the_brave_and_true
7 https://github.com/matheusfacure/python-causality-handbook/issues
8 https://github.com/matheusfacure/python-causality-handbook/stargazers

PART I

The Yang

CHAPTER
1

Introduction to Causality

Why Bother?

First and foremost, you might be wondering: what's in it for me? Here's the answer:

Data Science is Not What it Used to Be (or it Finally Is)

The Data Scientist role has been labeled The Sexiest Job of the 21st Century[1] by Harvard Business Review. This was no empty statement. For a decade now, Data Scientist has been in the spotlight. AI experts had salaries that rivaled those of sports superstars[2]. In the search for fame and fortune, hundreds of young professionals jumped into what seemed a frenetic gold rush to get the Data Science title as quickly as possible. Entire new industries sprang around the hype. Miraculous teaching methods promised to make you a Data Scientist without ever showing you a single math formula. Consulting specialists promised millions if your company could unlock the potential of data. AI, or Machine Learning, has been dubbed the new electricity and data, the new oil.

During all this time, economists were trying to answer what is the true impact of education on one's earnings, biostatisticians were trying to understand if saturated fat led to a higher chance of heart attack, and psychologists were trying to understand if words of affirmation actually lead to happier marriages. Meanwhile, we forgot about those who have been doing "old-fashioned" science with data all along. If we were to be completely honest, data science is not a recent field. We're only now becoming aware of it thanks to a massive wave of free publicity from the media.

To use a Jim Collins analogy, think about pouring yourself an ice-cold cup of your favorite beer. If you do this the right way, most cups will be beer, but there will be a 1 finger thick layer of foam at the top. This cup is just like Data Science.

1 https://hbr.org/2012/10/data-scientist-the-sexiest-job-of-the-21st-century

2 https://www.economist.com/business/2016/04/02/million-dollar-babies

1. It's the beer. The statistical foundations, the scientific curiosity, the passion for complex problems. All of this was proven very valuable for hundreds of years.
2. It's the foam. The fluffy stuff is built on unrealistic expectations that will eventually go away.

This foam might come down crashing faster than you think. As The Economist puts it

> The same consultants who predict that AI will have a world-altering impact also report that real managers in real companies are finding AI hard to implement, and that enthusiasm for it is cooling. Svetlana Sicular of Gartner, a research firm, says that 2020 could be the year AI falls onto the downslope of her firm's well-publicised "hype cycle". Investors are beginning to wake up to bandwagon-jumping: a survey of European AI startups by MMC, a venture-capital fund, found that 40% did not seem to be using any AI at all.

What should we, as Data Scientists - or better yet, as "just" Scientists - do during all this craze? If you are smart, you will learn to ignore the foam as a starter. We are in it for the beer. Math and statistics have been helpful forever, and it is unlikely it will stop now. Second, learn what makes your work valuable, not the latest shining tool that no one figured out how to use.

Last but not least, remember that there are no shortcuts. Knowledge in Math and Statistics is valuable precisely because they are hard to acquire. If everyone could do it, excess supply would drive its price down. So **toughen up**! Learn them as well as you can. And heck, why not? Have fun along the way as we embark on this quest only **for the Brave and True**.

Answering a Different Kind of Question

Machine Learning is currently very good at answering the type of question of the prediction kind. As Ajay Agrawal, Joshua Gans, and Avi Goldfarb put it in the book Prediction Machines, "the new wave of artificial intelligence does not actually bring us intelligence but instead a critical component of intelligence - prediction". We can do all sorts of beautiful things with machine learning. The only requirement is that we frame our problems as prediction ones. Want to translate from English to Portuguese? Then build an ML model that predicts Portuguese sentences when given English sentences. Want to recognize faces? Then create an ML model that predicts the presence of a face in a subsection of a picture. Want to build a self-driving car? Then make one ML model to

predict the direction of the wheel and the pressure on the brakes and accelerator when presented with images and sensors from the surroundings of a car.

However, ML is not a panacea. It can perform wonders under rigid boundaries and still fail miserably if its data deviates a little from what the model is accustomed to. To give another example from Prediction Machines, "in many industries, low prices are associated with low sales. For example, in the hotel industry, prices are low outside the tourist season, and prices are high when demand is highest and hotels are full. Given that data, a naive prediction might suggest that increasing the price would lead to more rooms sold."

ML is notoriously bad at this inverse causality type of problem. They require us to answer "what if" questions, which economists call counterfactuals. What would happen if I used another price instead of this price I'm currently asking for my merchandise? What would happen if I do a low sugar one instead of this low-fat diet I'm in? If you work in a bank, giving credit, you will have to figure out how changing the customer line changes your revenue. Or, if you work in the local government, you might be asked to figure out how to make the schooling system better. Should you give tablets to every kid because the era of digital knowledge tells you to? Or should you build an old-fashioned library?

At the heart of these questions, there is a causal inquiry we wish to know the answer to. Causal questions permeate everyday problems, like figuring out how to make sales go up. Still, they also play an essential role in dilemmas that are very personal and dear to us: do I have to go to an expensive school to be successful in life (does education cause earnings)? Does immigration lower my chances of getting a job (does immigration causes unemployment to go up)? Does money transfer to the poor lower the crime rate? It doesn't matter the field you are in. It is very likely you had or will have to answer some type of causal question. Unfortunately for ML, we can't rely on correlation-type predictions to tackle them.

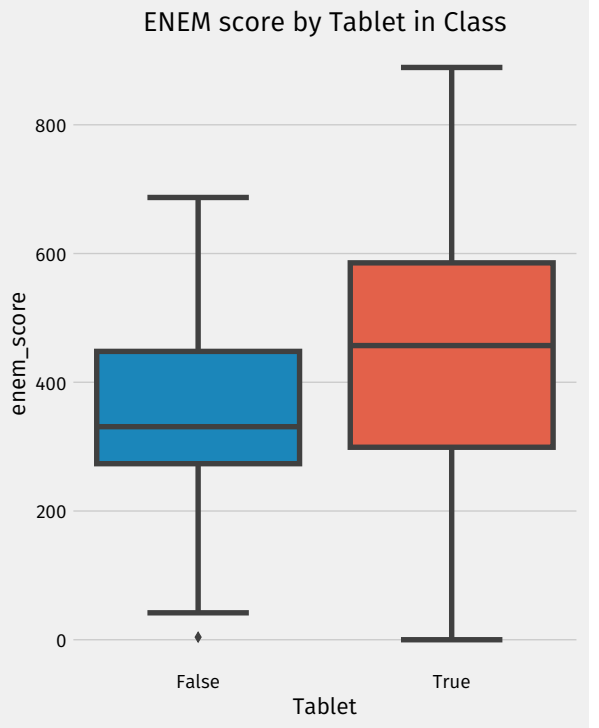
Answering this kind of question is more challenging than most people appreciate. Your parents have probably repeated to you that "association is not causation", "association is not causation". But actually, explaining why that is the case is a bit more involved. This is what this introduction to causal inference is all about. As for the rest of this book, it will be dedicated to **figuring out how to make association be causation**.

When Association IS Causation

Intuitively, we kind of know why the association is not causation. If someone tells you that schools that give tablets to their students perform better than those that don't, you can quickly point out that it is probably the case that those schools with the tablets are wealthier. As such, they would do better than average even without the tablets. Because of this, we can't conclude that giving tablets to kids during classes will cause an increase in their academic performance. We can only say that tablets in school are associated with high academic performance, as measured by ENEM (sort of the SAT in Brazil, which stands for National High School Exam):

```
import pandas as pd
import numpy as np
from scipy.special import expit
import seaborn as sns
from matplotlib import pyplot as plt
from matplotlib import style

style.use("fivethirtyeight")

np.random.seed(123)
n = 100
tuition = np.random.normal(1000, 300, n).round()
tablet = np.random.binomial(1, expit((tuition - tuition.mean()) /
  ↪ tuition.std())).astype(bool)
enem_score = np.random.normal(200 - 50 * tablet + 0.7 * tuition, 200)
enem_score = (enem_score - enem_score.min()) / enem_score.max()
enem_score *= 1000

data = pd.DataFrame(dict(enem_score=enem_score, Tuition=tuition, Tablet=tablet))
```

```
plt.figure(figsize=(6,8))
sns.boxplot(y="enem_score", x="Tablet",
  ↪ data=data).set_title('ENEM score by Tablet in Class')
plt.show()
```



To get beyond simple intuition, let's first establish some notation. This will be our everyday language to speak about causality. Think of it as the common tongue we will use to identify other brave and true causal warriors, and that will compose our cry in the many battles to come.

Let's call T_i the treatment intake for unit i.

$$T_i = \begin{cases} 1 \text{ if unit i received the treatment} \\ 0 \text{ otherwise} \end{cases}$$

The treatment here doesn't need to be a medicine or anything from the medical field. Instead, it is just a term we will use to denote some intervention for which we want to know the effect. In our case, the treatment is giving tablets to students. As a side note, you might sometimes see D instead of T to denote the treatment.

Now, let's call Y_i the observed outcome variable for unit i.

The outcome is our variable of interest. We want to know if the treatment has any influence in it. In our tablet example, it would be the academic performance.

Here is where things get interesting. The **fundamental problem of causal inference** is that we can never observe the same unit with and without treatment. It is as if we have two diverging roads and we can only know what lies ahead of the one we take. As in Robert Frost poem:

> Two roads diverged in a yellow wood,
> And sorry I could not travel both
> And be one traveler, long I stood
> And looked down one as far as I could
> To where it bent in the undergrowth;

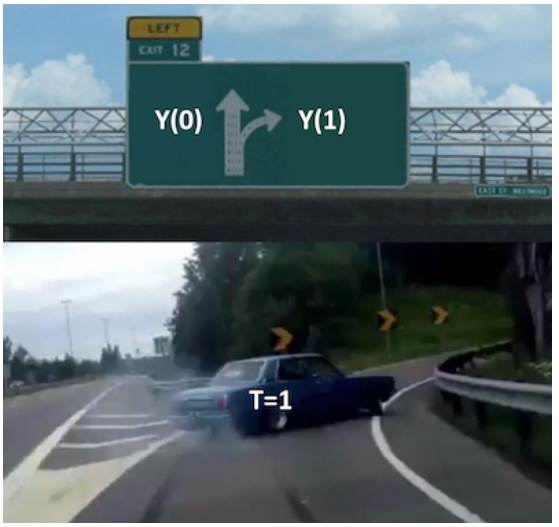
To wrap our heads around this, we will talk a lot in term of **potential outcomes**. They are potential because they didn't actually happen. Instead they denote **what would have happened** in the case some treatment was taken. We sometimes call the potential outcome that happened, factual, and the one that didn't happen, counterfactual.

As for the notation, we use an additional subscript:

Y_{0i} is the potential outcome for unit i without the treatment.

Y_{1i} is the potential outcome for **the same** unit i with the treatment.

Sometimes you might see potential outcomes represented as functions $Y_i(t)$, so beware. Y_{0i} could be $Y_i(0)$ and Y_{1i} could be $Y_i(1)$. Here, we will use the subscript notation most of the time.



Back to our example, Y_{1i} is the academic performance for student i if he or she is in a classroom with tablets. Whether or not this is the case, it doesn't matter for Y_{1i}. It is the same regardless. If student i gets the tablet, we can observe Y_{1i}. If not, we can observe Y_{0i}. Notice how in this last case, Y_{1i} is still defined, we just can't see it. In this case, it is a counterfactual potential outcome.

With potential outcomes, we can define the individual treatment effect:

$$Y_{1i} - Y_{0i}$$

Of course, due to the fundamental problem of causal inference, we can never know the individual treatment effect because we only observe one of the potential outcomes. For the time being, let's focus on something easier than estimating the individual treatment effect. Instead, lets focus on the **average treatment effect**, which is defined as follows.

$$ATE = E[Y_1 - Y_0]$$

where, `E[...]` is the expected value. Another easier quantity to estimate is the **average treatment effect on the treated**:

$$ATT = E[Y_1 - Y_0 | T = 1]$$

Now, I know we can't see both potential outcomes, but just for the sake of argument, let's suppose we could. Pretend that the causal inference deity is pleased with the many statistical battles we fought and has rewarded us with godlike powers to see the potential alternative outcomes. With that power, say we collect data on 4 schools. We know if they gave tablets to its students and their score on some annual academic tests. Here, tablets are the treatment, so $T = 1$ if the school provides tablets to its kids. Y will be the test score.

```
pd.DataFrame(dict(
    i= [1,2,3,4],
    Y0=[500,600,800,700],
    Y1=[450,600,600,750],
    T= [0,0,1,1],
    Y= [500,600,600,750],
    TE=[-50,0,-200,50],
))
```

	i	Y0	Y1	T	Y	TE
0	1	500	450	0	500	-50
1	2	600	600	0	600	0
2	3	800	600	1	600	-200
3	4	700	750	1	750	50

The ATE here would be the mean of the last column, that is, of the treatment effect:

$$ATE = (-50 + 0 - 200 + 50)/4 = -50$$

This would mean that tablets reduced the academic performance of students, on average, by 50 points. The ATT here would be the mean of the last column when $T = 1$:

$$ATT = (-200 + 50)/2 = -75$$

This is saying that, for the schools that were treated, the tablets reduced the academic performance of students, on average, by 75 points. Of course we can never know this. In reality, the table above would look like this:

```
pd.DataFrame(dict(
    i= [1,2,3,4],
    Y0=[500,600,np.nan,np.nan],
    Y1=[np.nan,np.nan,600,750],
    T= [0,0,1,1],
    Y= [500,600,600,750],
    TE=[np.nan,np.nan,np.nan,np.nan],
))
```

	i	Y0	Y1	T	Y	TE
0	1	500.0	NaN	0	500	NaN
1	2	600.0	NaN	0	600	NaN
2	3	NaN	600.0	1	600	NaN
3	4	NaN	750.0	1	750	NaN

This is surely not ideal, you might say, but can't I still take the mean of the treated and compare it to the mean of the untreated? In other words, can't I just do $ATE = (600 + 750)/2 - (500 + 600)/2 = 125$? Well, no! Notice how different the results

are. You've just committed the gravest sin of mistaking association for causation. To understand why let's look into the main enemy of causal inference.

Bias

Bias is what makes association different from causation. Fortunately, it can be easily understood with our intuition. Let's recap our tablets in the classroom example. When confronted with the claim that schools that give tablets to their kids achieve higher test scores, we can refute it by saying those schools will probably achieve higher test scores anyway, even without the tablets. That is because they probably have more money than the other schools; hence they can pay better teachers, afford better classrooms, etc. In other words, it is the case that treated schools (with tablets) are not comparable with untreated schools.

Using potential outcomes notation, we would say that Y_0 of the treated is different from the Y_0 of the untreated. Remember that the Y_0 of the treated **is counterfactual**. We can't observe it, but we can reason about it. In this particular case, we can even leverage our understanding of how the world works to go even further. We can say that, probably, Y_0 of the treated is bigger than Y_0 of the untreated schools. That is because schools that can afford to give tablets to their kids can also afford other factors that contribute to better test scores. Let this sink in for a moment. It takes some time to get used to talking about potential outcomes. Reread this paragraph and make sure you understand it.

With this in mind, we can show with elementary math why it is the case that association is not causation. Association is measured by $E[Y \mid T = 1] - E[Y \mid T = 0]$. In our example, this is the average test score for the schools with tablets minus the average test score for those without them. On the other hand, causation is measured by $E[Y_1 - Y_0]$.

Let's take the association measurement and replace the observed outcomes with the potential outcomes to see how they relate. For the treated, the observed outcome is Y_1. For the untreated, the observed outcome is Y_0.

$$E[Y \mid T = 1] - E[Y \mid T = 0] = E[Y_1 \mid T = 1] - E[Y_0 \mid T = 0]$$

Now, let's add and subtract $E[Y_0 \mid T = 1]$. This is a counterfactual outcome. It tells what would have been the outcome of the treated, had they not received the treatment.

$$E[Y \mid T = 1] - E[Y \mid T = 0] = E[Y_1 \mid T = 1] - E[Y_0 \mid T = 0] + E[Y_0 \mid T = 1] - E[Y_0 \mid T = 1]$$

Finally, we reorder the terms, merge some expectations, and lo and behold:

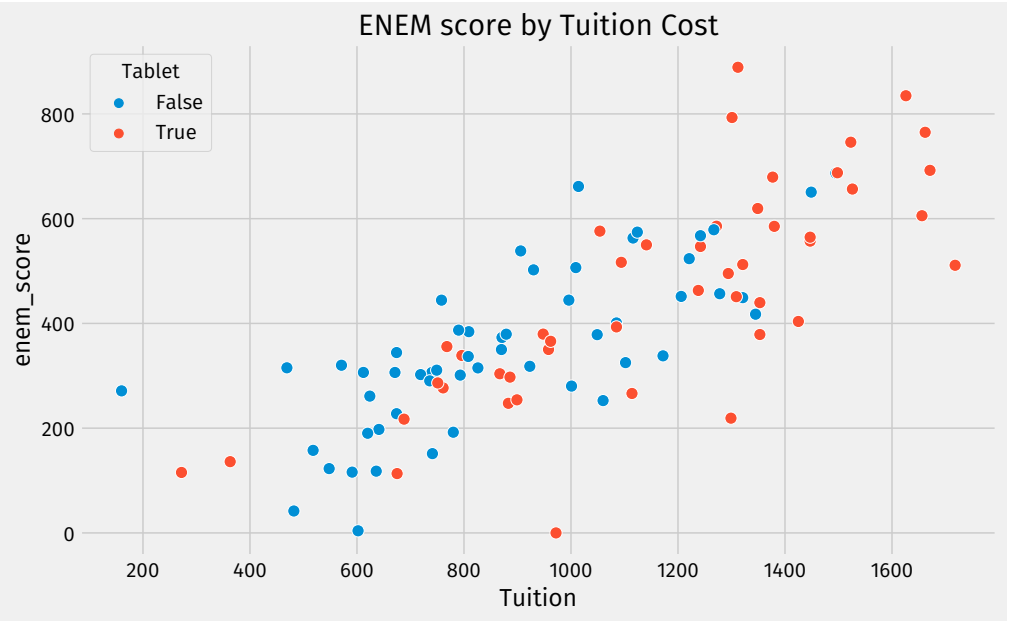
$$E[Y \mid T = 1] - E[Y \mid T = 0] = \underbrace{E[Y_1 - Y_0 \mid T = 1]}_{ATT} + \underbrace{\{E[Y_0 \mid T = 1] - E[Y_0 \mid T = 0]\}}_{BIAS}$$

This simple piece of math encompasses all the problems we will encounter in causal questions. I cannot stress how important it is that you understand every aspect of it. If you're ever forced to tattoo something on your arm, this equation should be a good candidate for it. It's something to hold onto dearly and understand what is telling us,

like some sacred text that can be interpreted 100 different ways. In fact, let's take a deeper look. Let's break it down into some of its implications. First, this equation tells why the association is not causation. As we can see, the association is equal to the treatment effect on the treated plus a bias term. **The bias is given by how the treated and control group differ before the treatment, in case neither of them has received the treatment**. We can now say precisely why we are suspicious when someone tells us that tablets in the classroom boost academic performance. We think that, in this example, $E[Y_0|T=0] < E[Y_0|T=1]$, that is, schools that can afford to give tablets to their kids are better than those that can't, **regardless of the tablets treatment**.

Why does this happen? We will talk more about that once we enter confounding, but for now, you can think of bias arising because many things we can't control are changing together with the treatment. As a result, the treated and untreated schools don't differ only on the tablets. They also differ on the tuition cost, location, teachers... For us to say that tablets in the classroom increase academic performance, we would need for schools with and without them to be, on average, similar to each other.

```
plt.figure(figsize=(10,6))
sns.scatterplot(x="Tuition", y="enem_score", hue="Tablet", data=data,
  ↪ s=70).set_title('ENEM score by Tuition Cost')
plt.show()
```



Now that we understand the problem let's look at the solution. We can also say what would be necessary to make association equal to causation. **If $E[Y_0|T=0] = E[Y_0|T=1]$, then, association IS CAUSATION!** Understanding this is not just remembering the equation. There is a strong intuitive argument here. To say that $E[Y_0|T=0] = E[Y_0|T=1]$ is to say that treatment and control group are comparable before the

treatment. Or, when the treated had not been treated, if we could observe its Y_0, its outcome would be the same as the untreated. Mathematically, the bias term would vanish:

$$E[Y \mid T = 1] - E[Y \mid T = 0] = E[Y_1 - Y_0 \mid T = 1] = ATT$$

Also, if the treated and the untreated only differ on the treatment itself, then, $E[Y_0 \mid T = 0] = E[Y_0 \mid T = 1]$ and we have that the causal impact on the treated is the same as in the untreated (because they are very similar).

$$\begin{aligned} E[Y_1 - Y_0 \mid T = 1] &= E[Y_1 \mid T = 1] - E[Y_0 \mid T = 1] \\ &= E[Y_1 \mid T = 1] - E[Y_0 \mid T = 0] \\ &= E[Y \mid T = 1] - E[Y \mid T = 0] \end{aligned}$$

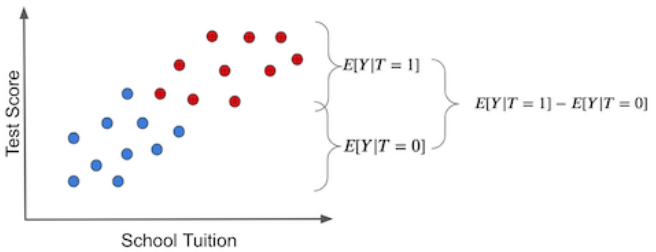
In this case, the **difference in means BECOMES the causal effect**:

$$E[Y \mid T = 1] - E[Y \mid T = 0] = ATT$$

Additionally, if the treated and the untreated only differ on the treatment itself, we also have $E[Y_1 \mid T = 0] = E[Y_1 \mid T = 1]$, that is, we make sure that both treated and control groups respond similarly to the treatment. Now, besides being exchangeable prior to the treatment, treated and untreated are also exchangeable **after the treatment**. In this case, $E[Y_1 - Y_0 \mid T = 1] = E[Y_1 - Y_0 \mid T = 0]$ and

$$E[Y \mid T = 1] - E[Y \mid T = 0] = ATT = ATE$$

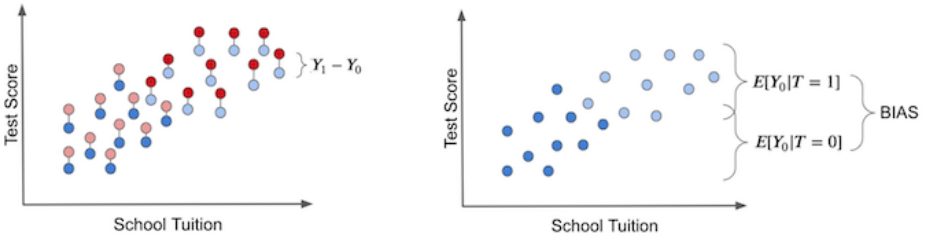
Once again, this is so important that I think it is worth going over it again, now with pretty pictures. If we make a simple average comparison between the treatment and the untreated group, this is what we get (blue dots didn't receive the treatment, that is, the tablet):



Notice how the difference in outcomes between the two groups can have two causes:

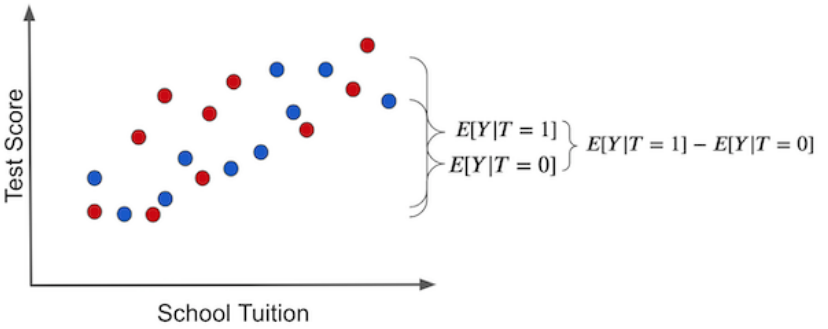
1. The treatment effect. The increase in test scores is caused by giving kids tablets.
2. Some of the differences in test scores can be due to tuition prices for better education. In this case, treated and untreated differ because the treated have a much higher tuition price. Other differences between the treatment and untreated are NOT the treatment itself.

The individual treatment effect is the difference between the unit's outcome and another theoretical outcome that the same unit would have if it got the alternative treatment. The actual treatment effect can only be obtained if we have godlike powers to observe the potential outcome, like the left figure below. These are the counterfactual outcomes and are denoted in a light color.

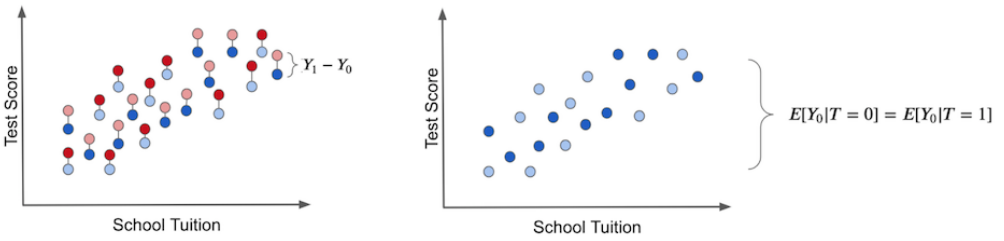


In the right plot, we depicted the bias that we've talked about before. We get the bias if we set everyone to not receive the treatment. In this case, we are only left with the T_0 potential outcome. Then, we see how the treated and untreated groups differ. If they do, something other than the treatment is causing the treated and untreated to be different. This something is the bias and is what shadows the actual treatment effect.

Now, contrast this with a hypothetical situation where there is no bias. Suppose that tablets are randomly assigned to schools. In this situation, rich and poor schools have the same chance of receiving the treatment. Treatment would be well distributed across the tuition spectrum.



In this case, the difference in the outcome between treated and untreated IS the average causal effect. This happens because there is no other source of difference between treatment and untreated other than the treatment itself. All the differences we see must be attributed to it. Another way to say this is that there is no bias.



If we set everyone to not receive the treatment so that we only observe the Y_0s, we would find no difference between the treated and untreated groups.

This is the herculean task causal inference is all about. It is about finding clever ways of removing bias and making the treated and the untreated comparable so that all the difference we see is only the average treatment effect. Ultimately, causal inference is about figuring out how the world works, stripped of all delusions and misinterpretations. And now that we understand this, we can move forward to mastering some of the most powerful methods to remove bias, the weapons of the Brave and True, to identify the causal effect.

Key Ideas

So far, we've seen that association is not causation. Most importantly, we've seen precisely why it isn't and how we can make association be causation. We've also introduced the potential outcome notation as a way to wrap our heads around causal reasoning. With it, we saw statistics as two possible realities: one in which the treatment is given and another in which it is not. But unfortunately, we can only measure one of them, where the fundamental problem of causal inference lies.

We will see some basic techniques to estimate the causal effect, starting with the golden standard of a randomized trial. I'll also review some statistical concepts as we go. I'll end with a quote often used in causal inference classes, taken from a kung-fu series:

> 'What happens in a man's life is already written. A man must move through life as his destiny wills.' -Caine
> 'Yes, yet each man is free to live as he chooses. Though they seem opposite, both are true.' -Old Man

References

I like to think of this book as a tribute to Joshua Angrist, Alberto Abadie and Christopher Walters for their amazing Econometrics class. Most of the ideas here are taken from their classes at the American Economic Association. Watching them is what's keeping me sane during this tough year of 2020.

- Cross-Section Econometrics · https://www.aeaweb.org/conference/cont-ed/2017-webcasts
- Mastering Mostly Harmless Econometrics · https://www.aeaweb.org/conference/cont-ed/2020-webcasts

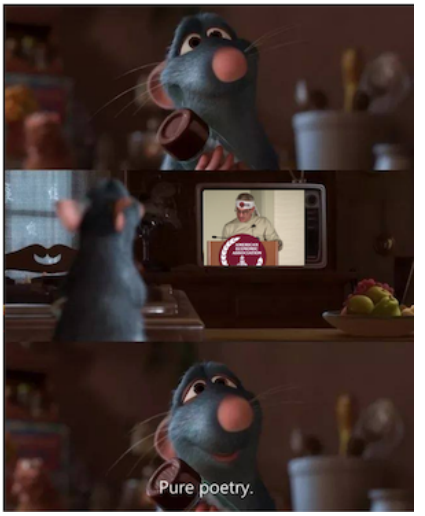
I'll also like to reference the amazing books from Angrist. They have shown me that Econometrics, or 'Metrics as they call it, is not only extremely useful but also profoundly fun.

- Mostly Harmless Econometrics · https://www.mostlyharmlesseconometrics.com/
- Mastering 'Metrics · https://www.masteringmetrics.com/

My final reference is Miguel Hernan and Jamie Robins' book. It has been my trustworthy companion in the most thorny causal questions I had to answer.

- Causal Inference Book · https://miguelhernan.org/whatifbook

The beer analogy was taken from the awesome Stock Series · https://jlcollinsnh.com/2012/04/15/stocks-part-1-theres-a-major-market-crash-coming-and-dr-lo-cant-save-you/, by JL Colins. This is an absolute must read for all of those wanting to learn how to productively invest their money.



CHAPTER

2

Randomised Experiments

The Gold Standard

In the previous session, we saw why and how association is different from causation. We also saw what is required to make association be causation.

$$E[Y \mid T = 1] - E[Y \mid T = 0] = \underbrace{E[Y_1 - Y_0 \mid T = 1]}_{ATT} + \{\underbrace{E[Y_0 \mid T = 1] - E[Y_0 \mid T = 0]}_{BIAS}\}$$

To recap, association becomes causation if there is no bias. There will be no bias if $E[Y_0 \mid T = 0] = E[Y_0 \mid T = 1]$. In words, association will be causation if the treated and control are equal or comparable, except for their treatment. Or, in more technical words, when the outcome of the untreated is equal to the counterfactual outcome of the treated. Remember that this counterfactual outcome is the outcome of the treated group if they had not received the treatment.

I think we did an OK job explaining how to make association equal to causation in math terms. But that was only in theory. Now, we look at the first tool we have to make the bias vanish: **Randomised Experiments**. Randomised experiments randomly assign individuals in a population to a treatment or to a control group. The proportion that receives the treatment doesn't have to be 50%. You could have an experiment where only 10% of your samples get the treatment.

Randomisation annihilates bias by making the potential outcomes independent of the treatment.

$$(Y_0, Y_1) \perp\!\!\!\perp T$$

This can be confusing at first (it was for me). But don't worry, my brave and true fellow, I'll explain it further. If the outcome is independent of the treatment, doesn't this also imply that the treatment has no effect? Well, yes! But notice I'm not talking about the outcomes. Instead, I'm talking about the **potential** outcomes. The potential outcome is how the outcome **would have been** under treatment (Y_1) or under control (Y_0). In

randomized trials, we **don't** want the outcome to be **independent** of the treatment since we think **the treatment causes** the outcome. But we want the **potential** outcomes to be independent of the treatment.

$$(Y_0, Y_1) \perp\!\!\!\perp T$$

$$Y \perp\!\!\!\perp T$$

Saying that the potential outcomes are independent of the treatment is saying that they would be, in expectation, the same in the treatment or the control group. In simpler terms, it means that treatment and control groups are comparable. Or that knowing the treatment assignment doesn't give me any information on how the outcome was previous to the treatment. Consequently, $(Y_0, Y_1) \perp T$ means that the treatment is the only thing generating a difference between the outcome in the treated and in the control group. To see this, notice that independence implies precisely that

$$E[Y_0 \mid T = 0] = E[Y_0 \mid T = 1] = E[Y_0]$$

Which, as we've seen, makes it so that

$$E[Y \mid T = 1] - E[Y \mid T = 0] = E[Y_1 - Y_0] = ATE$$

So, randomization gives us a way to use a simple difference in means between treatment and control and call that the treatment effect.

In a School Far, Far Away

In 2020, the Coronavirus Pandemic forced businesses to adapt to social distancing. Delivery services became widespread, and big corporations shifted to a remote work strategy. With schools, it wasn't different. Many started their own online repository of classes.

Four months into the crisis, many wonder if the introduced changes could be maintained. There is no question that online learning has its benefits. It is cheaper since it can save on real estate and transportation. It can also be more digital, leveraging world-class

content from around the globe, not just from a fixed set of teachers. Despite all of that, we still need to answer if online learning has a negative or positive impact on the student's academic performance.

One way to answer this is to take students from schools that give mostly online classes and compare them with students from schools that provide lectures in traditional classrooms. As we know by now, this is not the best approach. It could be that online schools attract only the well-disciplined students that do better than average even if the class were in-person. In this case, we would have a positive bias, where the treated are academically better than the untreated: $E[Y_0 | T = 1] > E[Y_0 | T = 0]$.

Or on the flip side, it could be that online classes are cheaper and are composed chiefly of less wealthy students, who might have to work besides studying. In this case, these students would do worse than those from the in-person schools even if they took in-person classes. If this was the case, we would have a bias in the other direction, where the treated are academically worse than the untreated: $E[Y_0 | T = 1] < E[Y_0 | T = 0]$.

So, although we could make simple comparisons, it wouldn't be compelling. One way or another, we could never be sure if there wasn't any bias lurking around and masking our causal effect.

To solve that, we need to make the treated and untreated comparable $E[Y_0 | T = 1] = E[Y_0 | T = 0]$. One way to force this is by randomly assigning the online and in-person classes to students. If we managed to do that, the treatment and untreated would be, on average, the same, except for the treatment they receive.

Fortunately, some economists have done that for us. They've randomized classes so that some students were assigned to have face-to-face lectures, others to have only online lessons, and a third group to have a blended format of both online and face-to-face classes. They collected data on a standard exam at the end of the semester.

Here is what the data looks like:

```
import pandas as pd
import numpy as np

data = pd.read_csv("./data/online_classroom.csv")
print(data.shape)
data.head()
```

```
(323, 10)
```

	gender	asian	black	hawaiian	hispanic	unknown
0	0	0.0	0.0	0.0	0.0	0.0
1	1	0.0	0.0	0.0	0.0	0.0
2	1	0.0	0.0	0.0	0.0	0.0
3	1	0.0	0.0	0.0	0.0	0.0
4	1	0.0	0.0	0.0	0.0	0.0

	white	format_ol	format_blended	falsexam
0	1.0	0	0.0	63.29997
1	1.0	0	0.0	79.96000
2	1.0	0	1.0	83.37000
3	1.0	0	1.0	90.01994
4	1.0	1	0.0	83.30000

We can see that we have 323 samples. It's not exactly big data, but something we can work with. To estimate the causal effect, we can simply compute the mean score for each of the treatment groups.

```
(data
 .assign(class_format = np.select(
     [data["format_ol"].astype(bool), data["format_blended"].astype(bool)],
     ["online", "blended"],
     default="face_to_face"
 ))
 .groupby(["class_format"])
 .mean())
```

class_format	gender	asian	black	hawaiian	hispanic
blended	0.550459	0.217949	0.102564	0.025641	0.012821
face_to_face	0.633333	0.202020	0.070707	0.000000	0.010101
online	0.542553	0.228571	0.028571	0.014286	0.028571

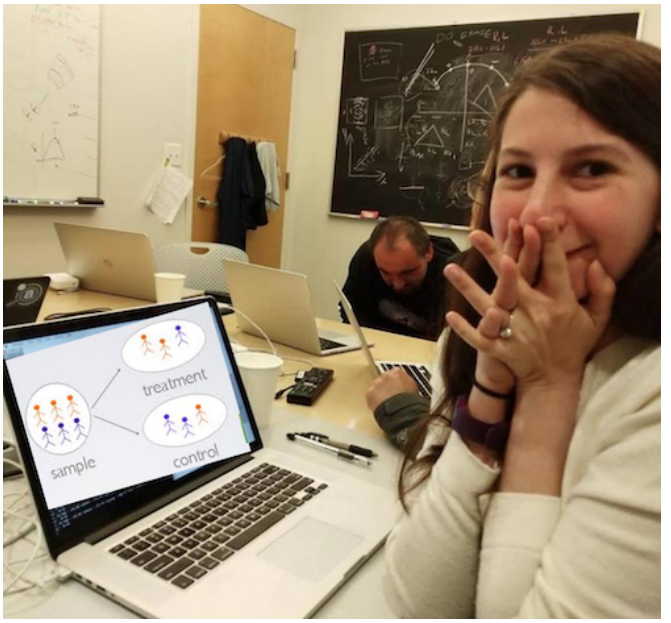
class_format	unknown	white	format_ol	format_blended	falsexam
blended	0.012821	0.628205	0.0	1.0	77.093731
face_to_face	0.000000	0.717172	0.0	0.0	78.547485
online	0.000000	0.700000	1.0	0.0	73.635263

Yup. It's that simple. We can see that face-to-face classes yield a 78.54 average score, while online courses yield a 73.63 average score. Not so good news for the proponents of online learning. The ATE for an online class is thus -4.91. This means that **online classes cause students to perform about 5 points lower**, on average. That's it. You don't need to worry that online courses might have poorer students that can't afford face-to-face classes or, for that matter, you don't have to worry that the students from the different treatments are different in any way other than the treatment they received. By design, the random experiment is made to wipe out those differences.

For this reason, a good sanity check to see if the randomisation was done right (or if you are looking at the correct data) is to check if the treated are equal to the untreated in pre-treatment variables. Our data has information on gender and ethnicity to see if they are similar across groups. We can say that they look pretty similar for the `gender`, `asian`, `hispanic`, and `white` variables. The `black` variable, however, seems a little bit different. This draws attention to what happens with a small dataset. Even under randomisation, it could be that, by chance, one group is different from another. In large samples, this difference tends to disappear.

The Ideal Experiment

Randomised experiments or Randomised Controlled Trials (RCT) are the most reliable way to get causal effects. It's a straightforward technique and absurdly convincing. It is so powerful that most countries have it as a requirement for showing the effectiveness of new medicine. To make a terrible analogy, you can think of RCT as Aang, from Avatar: The Last Airbender, while other techniques are more like Sokka. Sokka is cool and can pull some neat tricks here and there, but Aang can bend the four elements and connect with the spiritual world. Think of it this way, if we could, RCT would be all we would ever do to uncover causality. A well designed RCT is the dream of any scientist.



Unfortunately, they tend to be either very expensive or just plain unethical. Sometimes, we simply can't control the assignment mechanism. Imagine yourself as a physician trying to estimate the effect of smoking during pregnancy on baby weight at birth. You can't simply force a random portion of moms to smoke during pregnancy. Or say you work for a big bank, and you need to estimate the impact of the credit line on customer churn. It would be too expensive to give random credit lines to your customers. Or that you want to understand the impact of increasing the minimum wage on unemployment. You can't simply assign countries to have one or another minimum wage. You get the point.

We will later see how to lower the randomisation cost by using conditional randomisation, but there is nothing we can do about unethical or unfeasible experiments. Still, whenever we deal with causal questions, it is worth thinking about the **ideal experiment**. Always ask yourself, if you could, **what would be the perfect experiment you would run to uncover this causal effect?** This tends to shed some light on the way how we can discover the causal effect even without the ideal experiment.

The Assignment Mechanism

In a randomised experiment, the mechanism that assigns units to one treatment or the other is, well, random. As we will see later, all causal inference techniques will somehow try to identify the assignment mechanisms of the treatments. When we know for sure how this mechanism behaves, causal inference will be much more confident, even if the assignment mechanism isn't random.

Unfortunately, the assignment mechanism can't be discovered by simply looking at the data. For example, if you have a dataset where higher education correlates with wealth, you can't know for sure which one caused which by just looking at the data. You will

have to use your knowledge about how the world works to argue in favor of a plausible assignment mechanism: is it the case that schools educate people, making them more productive and leading them to higher-paying jobs. Or, if you are pessimistic about education, you can say that schools do nothing to increase productivity, and this is just a spurious correlation because only wealthy families can afford to have a kid get a higher degree.

In causal questions, we usually can argue in both ways: that X causes Y, or that it is a third variable Z that causes both X and Y, and hence the X and Y correlation is just spurious. For this reason, knowing the assignment mechanism leads to a much more convincing causal answer. This is also what makes causal inference so exciting. While applied ML is usually just pressing some buttons in the proper order, applied causal inference requires you to seriously think about the mechanism generating that data.

Key Ideas

We looked at how randomised experiments are the simplest and most effective way to uncover causal impact. It does this by making the treatment and control groups comparable. Unfortunately, we can't do randomised experiments all the time, but it is still helpful to think about what is the ideal experiment we would do if we could.

Someone familiar with statistics might be protesting right now that I didn't look at the variance of my causal effect estimate. How can I know that a 4.91 points decrease is not due to chance? In other words, how can I know if the difference is statistically significant? And they would be right. Don't worry. I intend to review some statistical concepts next.

References

I like to think of this entire book as a tribute to Joshua Angrist, Alberto Abadie and Christopher Walters for their amazing Econometrics class. Most of the ideas here are taken from their classes at the American Economic Association. Watching them is what is keeping me sane during this tough year of 2020.

- Cross-Section Econometrics · https://www.aeaweb.org/conference/cont-ed/2017-webcasts
- Mastering Mostly Harmless Econometrics · https://www.aeaweb.org/conference/cont-ed/2020-webcasts

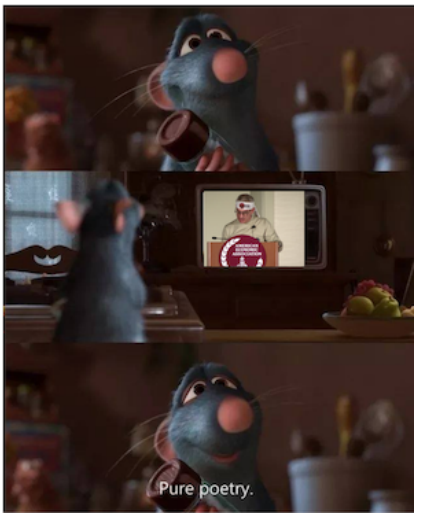
I'll also like to reference the amazing books from Angrist. They have shown me that Econometrics, or 'Metrics as they call it, is not only extremely useful but also profoundly fun.

- Mostly Harmless Econometrics · https://www.mostlyharmlesseconometrics.com/
- Mastering 'Metrics · https://www.masteringmetrics.com/

My final reference is Miguel Hernan and Jamie Robins' book. It has been my trustworthy companion in the most thorny causal questions I had to answer.

- Causal Inference Book · https://miguelhernan.org/whatifbook

The data used here is from a study of Alpert, William T., Kenneth A. Couch, and Oskar R. Harmon. 2016. "A Randomized Assessment of Online Learning" · https://www.aeaweb.org/articles?id=10.1257/aer.p20161057. American Economic Review, 106 (5): 378-82.



CHAPTER

3

Stats Review: The Most Dangerous Equation

In his famous article of 2007, Howard Wainer writes about very dangerous equations:

"Some equations are dangerous if you know them, and others are dangerous if you do not. The first category may pose danger because the secrets within its bounds open doors behind which lies terrible peril. The obvious winner in this is Einstein's iconic equation $E = mc^2$, for it provides a measure of the enormous energy hidden within ordinary matter. [...] Instead I am interested in equations that unleash their danger not when we know about them, but rather when we do not. Kept close at hand, these equations allow us to understand things clearly, but their absence leaves us dangerously ignorant."

The equation he talks about is Moivre's equation:

$$SE = \frac{\sigma}{\sqrt{n}}$$

where SE is the standard error of the mean, σ is the standard deviation, and n is the sample size. Sounds like a piece of math the brave and true should master, so let's get to it.

To see why not knowing this equation is very dangerous, let's look at some education data. I've compiled data on ENEM scores (Brazilian standardised high school scores, similar to SAT) from different schools for 3 years. I also cleaned the data to keep only the information relevant to us. The original data can be downloaded on the Inep website[1].

If we look at the top-performing school, something catches the eye: those schools have a reasonably small number of students.

1 http://portal.inep.gov.br/web/guest/microdados#

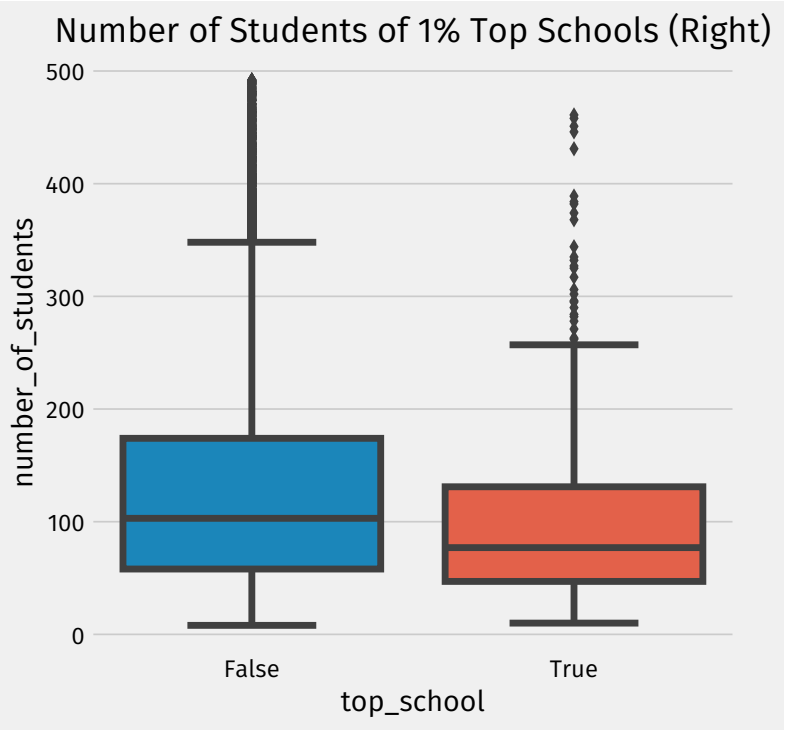
```
import warnings
warnings.filterwarnings('ignore')

import pandas as pd
import numpy as np
from scipy import stats
import seaborn as sns
from matplotlib import pyplot as plt
from matplotlib import style
style.use("fivethirtyeight")
```

```
df = pd.read_csv("./data/enem_scores.csv")
df.sort_values(by="avg_score", ascending=False).head(10)
```

	year	school_id	number_of_students	avg_score
16670	2007	33062633	68	82.97
16796	2007	33065403	172	82.04
16668	2005	33062633	59	81.89
16794	2005	33065403	177	81.66
10043	2007	29342880	43	80.32
18121	2007	33152314	14	79.82
16781	2007	33065250	80	79.67
3026	2007	22025740	144	79.52
14636	2007	31311723	222	79.41
17318	2007	33087679	210	79.38

Looking at it from another angle, we can separate only the 1% of top schools and study them. What are they like? Perhaps we can learn something from the best and replicate it elsewhere. And sure enough, if we look at the top 1% of schools, we figure out they have, on average, fewer students.

```
plot_data = (df
             .assign(top_school = df["avg_score"] >= np.quantile(
               ↪ df["avg_score"], .99))
             [["top_school", "number_of_students"]]
             .query(f"number_of_students<{np.quantile(df['number_of_students'],
      ↪ .98)}")) # remove outliers

plt.figure(figsize=(6,6))
sns.boxplot(x="top_school", y="number_of_students", data=plot_data)
plt.title("Number of Students of 1% Top Schools (Right)");
```



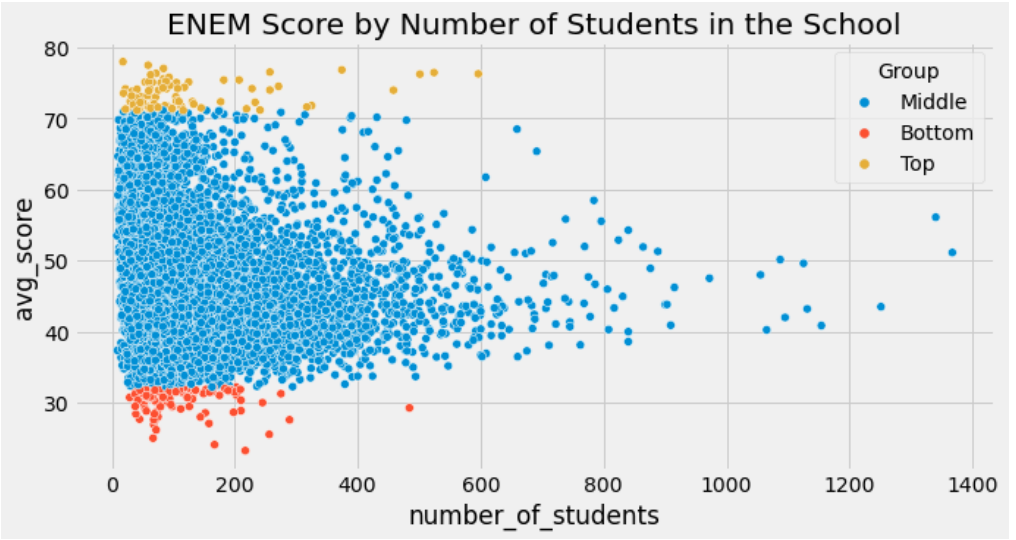
One natural conclusion is that small schools lead to higher academic performance. This makes intuitive sense since we believe that fewer students per teacher allow the teacher to give focused attention to each student. But what does this have to do with Moivre's equation? And why is it dangerous?

Well, it becomes dangerous once people start to make important and expensive decisions based on this information. In his article, Howard continues:

"In the 1990s, it became popular to champion reductions in the size of schools. Numerous philanthropic organisations and government agencies funded the division of larger schools because students at small schools are overrepresented in groups with high test scores."

What people forgot to do was to look also at the bottom 1% of schools. If we do that, lo and behold! They also have very few students!

```
q_99 = np.quantile(df["avg_score"], .99)
q_01 = np.quantile(df["avg_score"], .01)

plot_data = (df
             .sample(10000)
             .assign(Group = lambda d: np.select([d["avg_score"] > q_99,
              ↪ d["avg_score"] < q_01],
                                                 ["Top", "Bottom"], "Middle")))
plt.figure(figsize=(10,5))
sns.scatterplot(y="avg_score", x="number_of_students", hue="Group",
  ↪ data=plot_data)
plt.title("ENEM Score by Number of Students in the School");
```



We are seeing above precisely what is expected according to the Moivre's equation. As the number of students grows, the average score becomes more and more precise. Schools with very few samples can have very high and low scores simply due to chance. This is less likely to occur in large schools. Moivre's equation talks about a fundamental fact about the reality of information and records in the form of data: it is always imprecise. The question then becomes how inaccurate.

Statistics is the science that deals with these imprecisions, so they don't catch us off-guard. As Taleb puts it in his book, Fooled by Randomness:

> Probability is not a mere computation of odds on the dice or more complicated variants; it is the acceptance of the lack of certainty in our knowledge and the development of methods for dealing with our ignorance.

One way to quantify our uncertainty is the **variance of our estimates**. Variance tells us how much observation deviates from its central and most probable value. As Moivre's equation indicates, this uncertainty shrinks as the amount of data we observe increases. This makes sense, right? If we see many students performing excellently at a school, we

can be more confident that this is indeed a good school. However, if we see a school with only 10 students and 8 of them perform well, we need to be more suspicious. By chance, it could be that the school got some above-average students.

The beautiful triangular plot we see above tells precisely this story. It shows us how our estimates of the school performance have a huge variance when the sample sizes are small. It also indicates that variance shrinks as the sample size increases. This is true for the average score in a school, but it is also true about any summary statistics we have, including the ATE we often want to estimate.

The Standard Error of Our Estimates

Since this is just a review of statistics, I'll take the liberty to go a bit faster now. If you are not familiar with distributions, variance, and standard errors, please read on, but keep in mind that you might need some additional resources. I suggest you google any MIT course on introduction to statistics. They are usually quite good.

In the previous section, we estimated the average treatment effect $E[Y_1 - Y_0]$ as the difference in the means between the treated and the untreated $E[Y \mid T = 1] - E[Y \mid T = 0]$. We figured out the ATE for online classes as our motivating example. We also saw a negative impact; online classes made students perform about 5 points worse than the students with face-to-face classes. Now, we get to see if this impact is statistically significant.

To do so, we need to estimate the SE. We already have n, our sample size. To get the estimate for the standard deviation, we can do the following

$$\hat{\sigma} = \sqrt{\frac{1}{N-1}\sum_{i=1}^{N}(x_i - \bar{x})^2}$$

where $\bar{x}$ is the mean of x. Fortunately for us, most programming software already implements this. In Pandas, we can use the method std[2].

```
data = pd.read_csv("./data/online_classroom.csv")
online = data.query("format_ol==1")["falsexam"]
face_to_face = data.query("format_ol==0 & format_blended==0")["falsexam"]

def se(y: pd.Series):
    return y.std() / np.sqrt(len(y))

print("SE for Online:", se(online))
print("SE for Face to Face:", se(face_to_face))
```

```
SE for Online: 1.5371593973041635
SE for Face to Face: 0.8723511456319106
```

2 https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.std.html

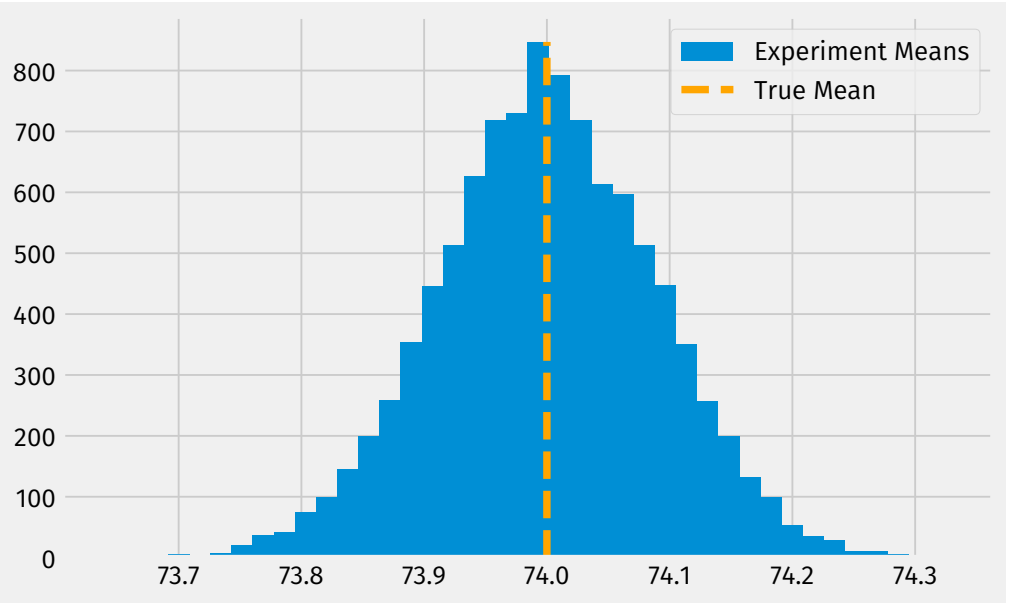
Confidence Intervals

The standard error of our estimate is a measure of confidence. We need to go into turbulent and polemic statistical waters to understand precisely what it means. For one view of statistics, the frequentist view, we would say that our data is nothing more than a manifestation of an accurate data-generating process. This process is abstract and ideal. It is governed by true parameters that are unchanging but also unknown to us. In the context of the students' test, if we could run multiple experiments and collect multiple datasets, all would resemble the true underlying data generating process but wouldn't be exactly like it. This is very much like Plato's writing on the Forms:

> Each [of the essential forms] manifests itself in a great variety of combinations, with actions, with material things, and with one another, and each seems to be many

Let's suppose we have a true abstract distribution of students' test scores to better grasp this. This is a normal distribution with a true mean of 74 and a true standard deviation of 2. From this distribution, we can run 10000 experiments. On each one, we collect 500 samples. If we plot them in a histogram, we can see that means of the experiments are distributed around the true mean. Some experiment data will have a mean lower than the true one, and some will be higher.

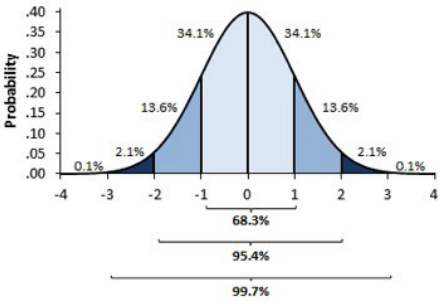
```
true_std = 2
true_mean = 74

n = 500
def run_experiment():
    return np.random.normal(true_mean,true_std, 500)

np.random.seed(42)

plt.figure(figsize=(8,5))
freq, bins, img = plt.hist([run_experiment().mean() for _ in range(10000)],
  ↪ bins=40, label="Experiment Means")
plt.vlines(true_mean, ymin=0, ymax=freq.max(), linestyles="dashed",
  ↪ label="True Mean", color="orange")
plt.legend();
```



Notice that we are talking about the mean of means here. So, by chance, we could have an experiment where the mean is somewhat below or above the true mean. This is to say that we can never be sure that the mean of our experiment matches the true platonic and ideal mean. However, **with the standard error, we can create an interval that will contain the true mean 95% of the time**.

In real life, we don't have the luxury of simulating the same experiment with multiple datasets. We often only have one. But we can draw on the intuition above to construct what we call **confidence intervals**. Confidence intervals come with a probability attached to them. The most common one is 95%. This probability tells us how many hypothetical confidence intervals we would build from different studies contain the true mean. For example, the 95% confidence intervals computed from similar studies would include the true mean 95% of the time.

To calculate the confidence interval, we use the **central limit theorem**. This theorem states that **means of experiments are normally distributed**. From statistical theory, we know that 95% of the mass of a normal distribution is between 2 standard deviations above and below the mean. Technically, 1.96, but 2 is close enough.

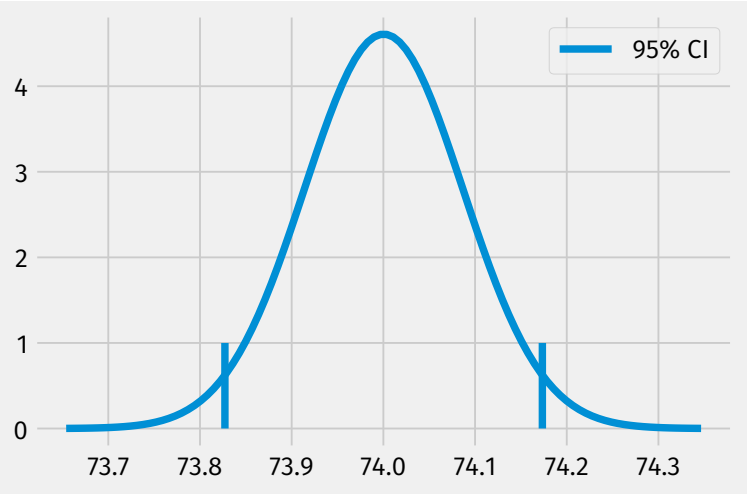


The Standard Error of the mean serves as our estimate of the distribution of the experiment means. So, if we multiply it by 2 and add and subtract it from the mean of one of our experiments, we will construct a 95% confidence interval for the true mean.

```
np.random.seed(321)
exp_data = run_experiment()
exp_se = exp_data.std() / np.sqrt(len(exp_data))
exp_mu = exp_data.mean()
ci = (exp_mu - 2 * exp_se, exp_mu + 2 * exp_se)
print(ci)
```

```
(73.82718114045632, 74.17341543460314)
```

```
x = np.linspace(exp_mu - 4*exp_se, exp_mu + 4*exp_se, 100)
y = stats.norm.pdf(x, exp_mu, exp_se)
plt.plot(x, y)
plt.vlines(ci[1], ymin=0, ymax=1)
plt.vlines(ci[0], ymin=0, ymax=1, label="95% CI")
plt.legend()
plt.show()
```



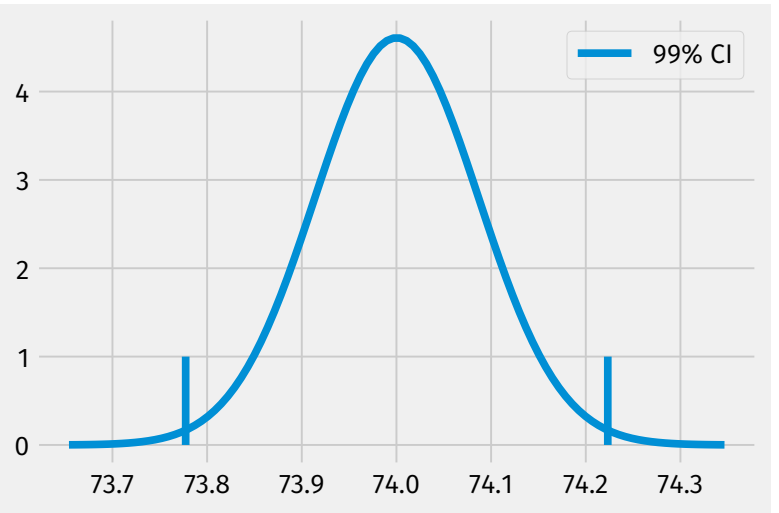
Of course, we don't need to restrict ourselves to the 95% confidence interval. We could generate the 99% interval by finding what we need to multiply the standard deviation by so the interval contains 99% of the mass of a normal distribution.

The function ppf in python gives us the inverse of the CDF. Instead of multiplying the standard error by 2 like we did to find the 95% CI, we will multiply it by z, which will result in the 99% CI. So, `ppf(0.5)` will return 0.0, saying that 50% of the mass of the standard normal distribution (mean 0 and 1 standard deviation) is below 0.0. By the same token, if we plug 99.5%, we will have the value z, such that 99.5% of the distribution mass falls below this value. In other words, 0.5% of the mass falls above this value.

```
from scipy import stats
z = stats.norm.ppf(.995)
print(z)
ci = (exp_mu - z * exp_se, exp_mu + z * exp_se)
ci
```

```
2.5758293035489004
(73.7773381773405, 74.22325839771896)
```

```
x = np.linspace(exp_mu - 4*exp_se, exp_mu + 4*exp_se, 100)
y = stats.norm.pdf(x, exp_mu, exp_se)
plt.plot(x, y)
plt.vlines(ci[1], ymin=0, ymax=1)
plt.vlines(ci[0], ymin=0, ymax=1, label="99% CI")
plt.legend()
plt.show()
```



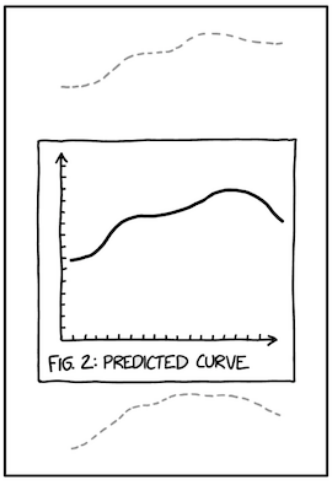
Back to our classroom experiment, we can construct the confidence interval for the mean exam score for both the online and face to face students' group

```
def ci(y: pd.Series):
    return (y.mean() - 2 * se(y), y.mean() + 2 * se(y))

print("95% CI for Online:", ci(online))
print("95% for Face to Face:", ci(face_to_face))
```

```
95% CI for Online: (70.56094429049804, 76.7095818797147)
95% for Face to Face: (76.80278229206951, 80.29218687459715)
```

We can see that the 95% CIs of the groups don't overlap. The lower end of the CI for Face to Face class is above the upper end of the CI for online classes. This is evidence that our result is not by chance and that the true mean for students in face-to-face classes is higher than the true mean for students in online classes. In other words, there is a significant causal decrease in academic performance when switching from face-to-face to online classes.

To recap, confidence intervals are a way to place uncertainty around our estimates. The smaller the sample size, the larger the standard error, and the wider the confidence interval. Since they are super easy to compute, lack of confidence intervals signals either some bad intentions or simply lack of knowledge, which is equally concerning. Finally, you should always be suspicious of measurements without any uncertainty metric.



SCIENCE TIP: IF YOUR MODEL IS BAD ENOUGH, THE CONFIDENCE INTERVALS WILL FALL OUTSIDE THE PRINTABLE AREA.

One final word of caution here. Confidence intervals are trickier to interpret than at first glance. For instance, I **shouldn't** say that this particular 95% confidence interval contains the true population mean with 95% chance. In frequentist statistics that use confidence intervals, the population mean is regarded as a true population constant. So it either is or

isn't in our particular confidence interval. In other words, our specific confidence interval either contains or doesn't contain the true mean. If it does, the chance of containing it would be 100%, not 95%. If it doesn't, the chance would be 0%. Instead, in confidence intervals, the 95% refers to the frequency that such confidence intervals, computed in many studies, contain the true mean. 95% is our confidence in the algorithm used to calculate the 95% CI, not in the particular interval itself.

Now, having said that, as an Economist (statisticians, please look away now), I think this purism is not very useful. In practice, you will see people saying that the particular confidence interval contains the true mean 95% of the time. Although wrong, this is not very harmful, as it still places a precise degree of uncertainty in our estimates. Moreover, if we switch to Bayesian statistics and use probable intervals instead of confidence intervals, we would be able to say that the interval contains the distribution mean 95% of the time. Also, from what I've seen in practice, with decent sample sizes, bayesian probability intervals are more similar to confidence intervals than Bayesian, and frequentists would like to admit. So, if my word counts for anything, feel free to say whatever you want about your confidence interval. I don't care if you say they contain the true mean 95% of the time. Please never forget to place them around your estimates; otherwise, you will look silly.

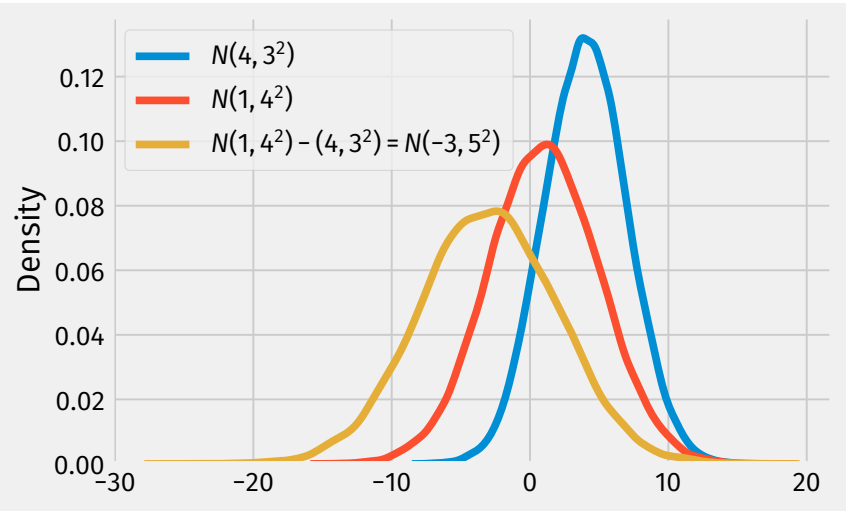
Hypothesis Testing

Another way to incorporate uncertainty is to state a hypothesis test: is the difference in means statistically different from zero (or any other value)? We will recall that the sum or difference of 2 independent normal distributions is also normal distribution. The resulting mean will be the sum or difference between the two distributions, while the variance will always be the sum of the variance:

$$N(\mu_1, \sigma_1^2) - N(\mu_2, \sigma_2^2) = N(\mu_1 - \mu_2, \sigma_1^2 + \sigma_2^2)$$
$$N(\mu_1, \sigma_1^2) + N(\mu_2, \sigma_2^2) = N(\mu_1 + \mu_2, \sigma_1^2 + \sigma_2^2)$$

If you don't recall, it's OK. We can always use code and simulated data to check:

```
np.random.seed(123)
n1 = np.random.normal(4, 3, 30000)
n2 = np.random.normal(1, 4, 30000)
n_diff = n2 - n1
sns.distplot(n1, hist=False, label="$N(4,3^2)$")
sns.distplot(n2, hist=False, label="$N(1,4^2)$")
sns.distplot(n_diff, hist=False, label=f"$N(1,4^2) - (4,3^2) = N(-3, 5^2)$")
plt.legend()
plt.show()
```



If we take the distribution of the means of our 2 groups and subtract one from the other, we will have a third distribution. The mean of this final distribution will be the difference in the means, and the standard deviation of this distribution will be the square root of the sum of the standard deviations.

$$\mu_{diff} = \mu_1 - \mu_2$$

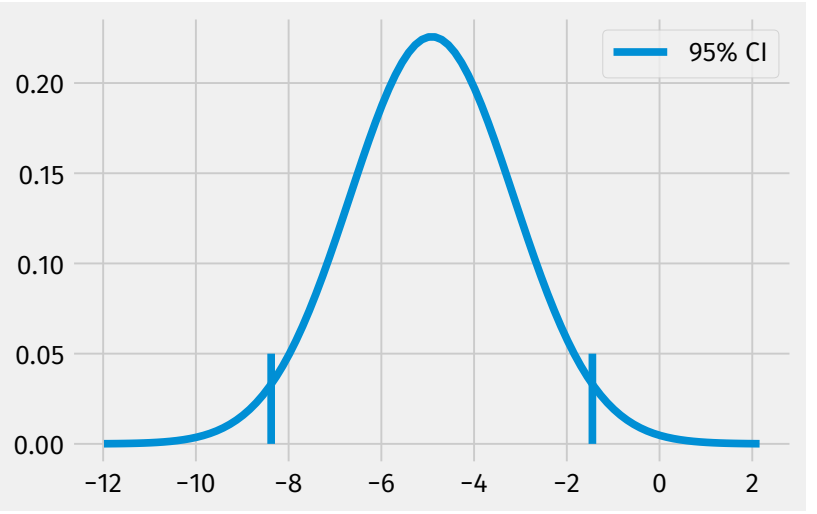
$$SE_{diff} = \sqrt{SE_1^2 + SE_2^2} = \sqrt{\sigma_1^2/n_1 + \sigma_2^2/n_2}$$

Let's return to our classroom example. We will construct this distribution of the difference. Of course, once we have it, building the 95% CI is straightforward.

```
diff_mu = online.mean() - face_to_face.mean()
diff_se = np.sqrt(face_to_face.var()/len(face_to_face) + online.var()/len(
  ↪ online))
ci = (diff_mu - 1.96*diff_se, diff_mu + 1.96*diff_se)
print(ci)
```

```
(-8.376410208363385, -1.4480327880905248)
```

```
x = np.linspace(diff_mu - 4*diff_se, diff_mu + 4*diff_se, 100)
y = stats.norm.pdf(x, diff_mu, diff_se)
plt.plot(x, y)
plt.vlines(ci[1], ymin=0, ymax=.05)
plt.vlines(ci[0], ymin=0, ymax=.05, label="95% CI")
plt.legend()
plt.show()
```



With this at hand, we can say that we are 95% confident that the true difference between the online and face-to-face groups falls between -8.37 and -1.44. We can also construct a **z statistic** by dividing the difference in mean by the SE of the differences.

$$z = \frac{\mu_{diff} - H_0}{SE_{diff}} = \frac{(\mu_1 - \mu_2) - H_0}{\sqrt{\sigma_1^2/n_1 + \sigma_2^2/n_2}}$$

Where H_0 is the value which we want to test our difference against.

The z statistic is a measure of how extreme the observed difference is. We will use contradiction to test our hypothesis that the difference in the means is statistically different from zero. We will assume that the opposite is true; we will assume that the difference is zero. This is called a null hypothesis, or H_0. Then, we will ask ourselves, "is it likely that we would observe such a difference if the true difference were zero?" We can translate this question to checking how far from zero our z statistic is in statistical math terms.

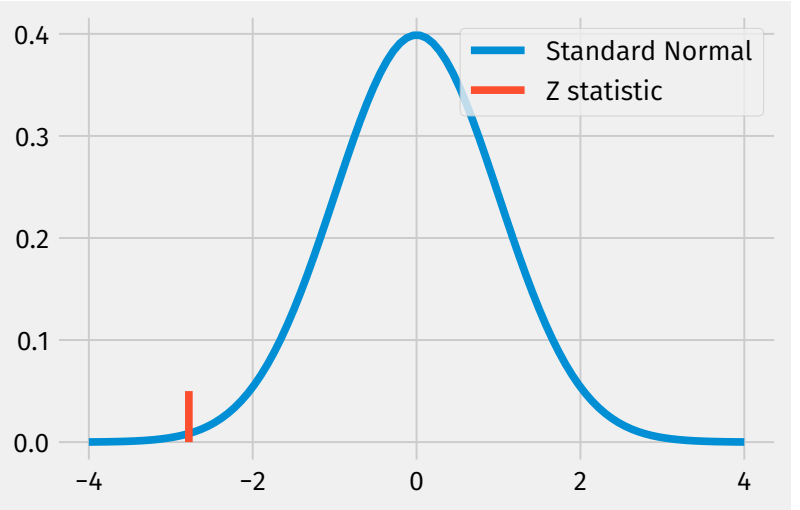
Under H_0, the z statistic follows a standard normal distribution. So, if the difference is indeed zero, we would see the z statistic within 2 standard deviations of the mean 95% of the time. The direct consequence is that if z falls above or below 2 standard deviations, we can reject the null hypothesis with 95% confidence.

Let's see how this looks like in our classroom example.

```
z = diff_mu / diff_se
print(z)
```

```
-2.7792810791031224
```

```
x = np.linspace(-4,4,100)
y = stats.norm.pdf(x, 0, 1)
plt.plot(x, y, label="Standard Normal")
plt.vlines(z, ymin=0, ymax=.05, label="Z statistic", color="C1")
plt.legend()
plt.show()
```



This looks like a pretty extreme value. Indeed, it is above 2, which means there is less than a 5% chance that we would see such an extreme value if there were no difference in the groups. This again leads us to conclude that switching from face-to-face to online classes causes a statistically significant drop in academic performance.

One final interesting thing about hypothesis tests is that it is less conservative than checking if the 95% CI from the treated and untreated group overlaps. In other words, if the confidence intervals in the two groups overlap, it can still be the case that the result is statistically significant. For example, let's pretend that the face-to-face group has an average score of 80 with a standard error of 4, and the online group has an average score of 71 with a standard error of 2.

```
cont_mu, cont_se =  (71, 2)
test_mu, test_se = (80, 4)

diff_mu = test_mu - cont_mu
diff_se = np.sqrt(cont_se**2 + test_se**2)

print("Control 95% CI:", (cont_mu-1.96*cont_se, cont_mu+1.96*cont_se))
print("Test 95% CI:", (test_mu-1.96*test_se, test_mu+1.96*test_se))
print("Diff 95% CI:", (diff_mu-1.96*diff_se, diff_mu+1.96*diff_se))
```

```
Control 95% CI: (67.08, 74.92)
Test 95% CI: (72.16, 87.84)
Diff 95% CI: (0.23461352820082482, 17.765386471799175)
```

If we construct the confidence intervals for these groups, they overlap. The upper bound for the 95% CI of the online group is 74.92, and the lower bound for the face-to-face group is 72.16. However, once we compute the 95% confidence interval for the difference between the groups, we can see that it does not contain zero. Even though the individual confidence intervals overlap, the difference can still be statistically different from zero.

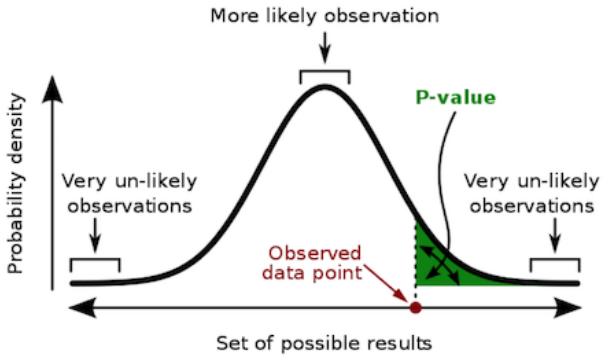
P-values

Previously, I've said that there is less than a 5% chance we would observe such an extreme value if the difference between online and face-to-face groups were actually zero. But can we precisely estimate what that chance is? How likely are we to observe such an extreme value? Enter p-values!

Like with confidence intervals (and most frequentist statistics, as a matter of fact), the true definition of p-values can be very confusing. So, to not take any risks, I'll copy the definition from Wikipedia: "the p-value is the probability of obtaining test results at least as extreme as the results actually observed during the test, assuming that the null hypothesis is correct".

To put it more succinctly, the p-value is the probability of seeing such data, given that the null hypothesis is true. It measures how unlikely it is that you are seeing a measurement if the null hypothesis is true. Naturally, this often gets confused with the probability of the null hypothesis being true. Note the difference here. The p-value is NOT $P(H_0 \mid data)$, but rather $P(data \mid H_0)$.

But don't let this complexity fool you. In practical terms, they are pretty straightforward to use.



A **p-value** (shaded green area) is the probability of an observed (or more extreme) result assuming that the null hypothesis is true.

To get the p-value, we need to compute the area under the standard normal distribution before or after the z statistic. Fortunately, we have a computer to do this calculation for us. We can simply plug the z statistic in the CDF of the standard normal distribution.

```
print("P-value:", stats.norm.cdf(z))
```

```
P-value: 0.0027239680835563383
```

Notice how the p-value is interesting because it avoids us having to specify a confidence level, like 95% or 99%. But, if we wish to report one, from the p-value, we know precisely at which confidence our test will pass or fail. For instance, with a p-value of 0.0027, we see that we have significance up to the 0.2% level. So, while the 95% CI and the 99% CI for the difference will neither contain zero, the 99.9% CI will. This means that there is only a 0.2% chance of observing this extreme z statistic if the difference was zero.

```
diff_mu = online.mean() - face_to_face.mean()
diff_se = np.sqrt(face_to_face.var()/len(face_to_face) + online.var()/len(
  ↪ online))
print("95% CI:", (diff_mu - stats.norm.ppf(.975)*diff_se,
  ↪ diff_mu + stats.norm.ppf(.975)*diff_se))
print("99% CI:", (diff_mu - stats.norm.ppf(.995)*diff_se,
  ↪ diff_mu + stats.norm.ppf(.995)*diff_se))
print("99.9% CI:", (diff_mu - stats.norm.ppf(.9995)*diff_se,
  ↪ diff_mu + stats.norm.ppf(.9995)*diff_se))
```

```
95% CI: (-8.376346553082909, -1.4480964433710017)
99% CI: (-9.46485353526404, -0.3595894611898709)
99.9% CI: (-10.728040658245558, 0.9035976617916459)
```

Key Ideas

We've seen how important it is to know Moivre's equation, and we used it to place a degree of certainty around our estimates. Namely, we figured out that online classes cause a decrease in academic performance compared to face-to-face classes. We also saw that this was a statistically significant result. We did it by comparing the confidence intervals of the means for the 2 groups, looking at the confidence interval for the difference, doing a hypothesis test, and looking at the p-value. Let's wrap everything up in a single function that makes A/B testing comparison like the one we did above

```
def AB_test(test: pd.Series, control: pd.Series, confidence=0.95, h0=0):
    mu1, mu2 = test.mean(), control.mean()
    se1, se2 = test.std() / np.sqrt(len(test)),
      ↪ control.std() / np.sqrt(len(control))

    diff = mu1 - mu2
```

```
    se_diff = np.sqrt(test.var()/len(test) + control.var()/len(control))

    z_stats = (diff-h0)/se_diff
    p_value = stats.norm.cdf(z_stats)

    def critical(se): return -se*stats.norm.ppf((1 - confidence)/2)

    print(f"Test {confidence*100}% CI: {mu1} +- {critical(se1)}")
    print(f"Control {confidence*100}% CI: {mu2} +- {critical(se2)}")
    print(f"Test-Control {confidence*100}% CI: {diff} +- {critical(se_diff)}")
    print(f"Z Statistic {z_stats}")
    print(f"P-Value {p_value}")

AB_test(online, face_to_face)
```

```
Test 95.0% CI: 73.63526308510637 +- 3.0127770572134565
Control 95.0% CI: 78.54748458333333 +- 1.7097768273108005
Test-Control 95.0% CI: -4.912221498226955 +- 3.4641250548559537
Z Statistic -2.7792810791031224
P-Value 0.0027239680835563383
```

Since our function is generic enough, we can test other null hypotheses. For instance, can we try to reject that the difference between online and face-to-face class performance is -1? With the results we get, we can say with 95% confidence that the difference is more significant than -1. But we can't say it with 99% confidence:

```
AB_test(online, face_to_face, h0=-1)
```

```
Test 95.0% CI: 73.63526308510637 +- 3.0127770572134565
Control 95.0% CI: 78.54748458333333 +- 1.7097768273108005
Test-Control 95.0% CI: -4.912221498226955 +- 3.4641250548559537
Z Statistic -2.2134920404560883
P-Value 0.013431870694630114
```

References

I like to think of this entire book as a tribute to Joshua Angrist, Alberto Abadie and Christopher Walters for their amazing Econometrics class. Most of the ideas here are taken from their classes at the American Economic Association. Watching them is what is keeping me sane during this tough year of 2020.

- Cross-Section Econometrics · https://www.aeaweb.org/conference/cont-ed/2017-webcasts
- Mastering Mostly Harmless Econometrics · https://www.aeaweb.org/conference/cont-ed/2020-webcasts

I'll also like to reference the amazing books from Angrist. They have shown me that Econometrics, or 'Metrics as they call it, is not only extremely useful but also profoundly fun.

- Mostly Harmless Econometrics · https://www.mostlyharmlesseconometrics.com/
- Mastering 'Metrics · https://www.masteringmetrics.com/

My final reference is Miguel Hernan and Jamie Robins' book. It has been my trustworthy companion in the most thorny causal questions I had to answer.

- Causal Inference Book · https://miguelhernan.org/whatifbook

The data used here is from a study of Alpert, William T., Kenneth A. Couch, and Oskar R. Harmon. 2016. "A Randomized Assessment of Online Learning" · https://www.aeaweb.org/articles?id=10.1257/aer.p20161057. American Economic Review, 106 (5): 378-82.

CHAPTER

4

Graphical Causal Models

Thinking About Causality

Have you ever noticed how those cooks in YouTube videos are excellent at describing food? "Reduce the sauce until it reaches a velvety consistency". If you are just learning to cook, you have no idea what this even means. Just give me the time I should leave this thing on the stove! With causality, it's the same thing. Suppose you walk into a bar and hear folks discussing causality (probably a bar next to an economics department). In that case, you will hear them say how the confounding of income made it challenging to identify the immigration effect on that neighborhood, so they had to use an instrumental variable. And by now, you might not understand what they are talking about. But I'll fix at least some of this problem right now.

Graphical models are the language of causality. They are not only what you use to talk with other brave and true causality aficionados but also something you use to make your own thoughts more transparent.

As a starting point, let's take conditional independence of the potential outcomes, for example. This is one of the main assumptions that we require to be true when making causal inference:

$$(Y_0, Y_1) \perp T \,|\, X$$

Conditional Independence makes it possible for us to measure an effect on the outcome solely due to the treatment and not any other variable lurking around. The classic example of this is the effect of a medicine on an ill patient. If only highly sick patients get the drug, it might even look like giving the medication decreases the patients' health. That is because the effect of the severity is getting mixed up with the drug's effect. If we break down the patients into severe and not severe cases and analyse the drug impact in each subgroup, we will get a clearer picture of the actual effect. This breaking down the population by its features is what we call controlling for or conditioning on X. By conditioning on the severe cases, the treatment mechanism becomes as good as random.

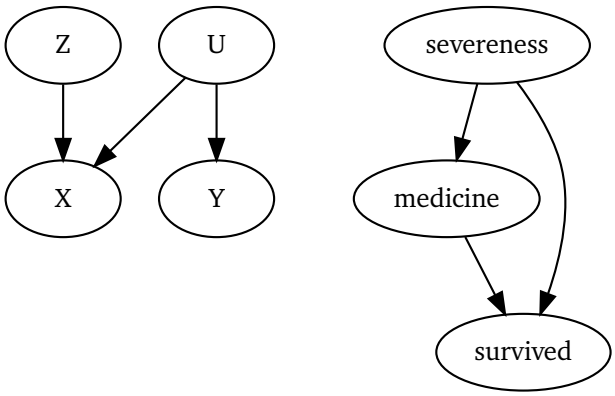
Patients within the severe group may or may not receive the drug only due to chance, not due to a high severity anymore, since all patients are the same on this dimension. And if treatment is randomly assigned within groups, the treatment becomes conditionally independent of the potential outcomes.

Independence and conditional independence are central to causal inference. Yet, it can be pretty challenging to wrap our heads around them. But this can change if we use the correct language to describe this problem. Here is where **causal graphical models** come in. A causal graphical model is a way to represent how causality works in terms of what causes what.

A graphical model looks like this

```
import warnings
warnings.filterwarnings('ignore')

import pandas as pd
import numpy as np
import graphviz as gr
from matplotlib import style
import seaborn as sns
from matplotlib import pyplot as plt
style.use("fivethirtyeight")
```

```
g = gr.Digraph()
g.edge("Z", "X")
g.edge("U", "X")
g.edge("U", "Y")

g.edge("medicine", "survived")
g.edge("severeness", "survived")
g.edge("severeness", "medicine")

g
```



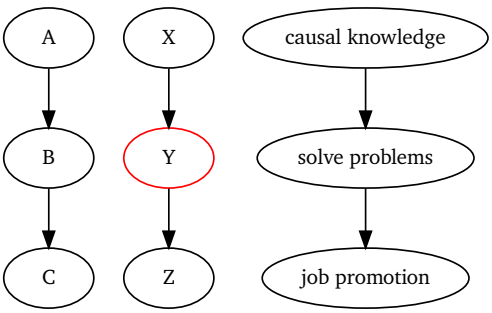
Each node is a random variable. We use arrows, or edges, to show if a variable causes another. In the first graphical model above, we are saying that Z causes X and that U causes X and Y. To give a more concrete example, we can translate our thoughts about the impact of the medicine on patient survival in the second graph above. Severeness causes both medicine and survival, and medicine also causes survival. As we will see, these causal graphical models language will help us make our thinking about causality clearer, as it clarifies our beliefs about how the world works.

Crash Course in Graphical Models

There are whole semesters on graphical models[1]. But, for our purpose, it is just (very) important that we understand what kind of independence and conditional independence assumptions a graphical model entails. As we shall see, independence flows through a graphical model as water flows through a stream. We can stop this flow or enable it, depending on how we treat the variables in it. To understand this, let's examine some common graphical structures and examples. They will be pretty straightforward, but they are sufficient building blocks to understand everything about independence and conditional independence on graphical models.

First, look at this very simple graph. A causes B, B causes C. Or X causes Y which causes Z.

```
g = gr.Digraph()
g.edge("A", "B")
g.edge("B", "C")

g.edge("X", "Y")
g.edge("Y", "Z")
g.node("Y", "Y", color="red")


g.edge("causal knowledge", "solve problems")
g.edge("solve problems", "job promotion")

g
```



1 https://www.coursera.org/specializations/probabilistic-graphical-models

In the first graph, dependence flows in the direction of the arrows. Notice that dependence is symmetric, although it is a little less intuitive. To give a more concrete example, let's say that knowing about causal inference is the only way to solve business problems, and solving those problems is the only way to get a job promotion. So causal knowledge implies solving issues that cause a job promotion. We can say here that job promotion is dependent on causal knowledge. The greater the causal expertise, the greater your chances of getting a promotion. Also, the greater your chances of promotion, the greater your chance of having causal knowledge. Otherwise, it would be difficult to get a promotion.

Now, let's say I condition on the intermediary variable. In this case, the dependence is blocked. So, X and Z are independent given Y. In the graph above, red indicates that Y is a conditioned variable. By the same token, in our example, if I know that you are good at solving problems, knowing that you know causal inference doesn't give any further information about your chances of getting a job promotion. In mathematical terms, $E[Promotion|Solve\ problems, Causal\ knowledge] = E[Promotion|Solve\ problems]$. The inverse is also true; once I know how good you are at solving problems, knowing about your job promotion status gives me no further information about how likely you are to know causal inference.

As a general rule, the dependence flow in the direct path from A to C is blocked when we condition on an intermediary variable B. Or,

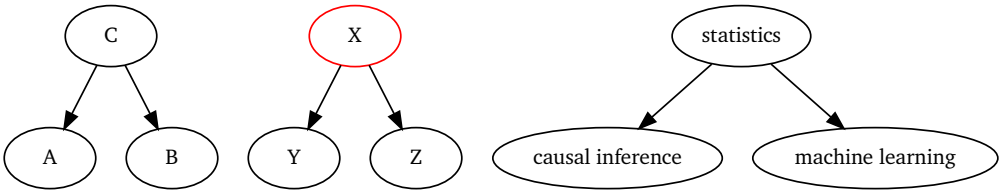
$$A \not\perp C$$

and

$$A \perp C \,|\, B$$

Now, let's consider a fork structure. The same variable causes two other variables down the graph. In this case, the dependence flows backward through the arrows, and we have a **backdoor path**. We can close the backdoor path and shut down dependence by conditioning on the common cause.

```
g = gr.Digraph()
g.edge("C", "A")
g.edge("C", "B")

g.edge("X", "Y")
g.edge("X", "Z")
g.node("X", "X", color="red")

g.edge("statistics", "causal inference")
g.edge("statistics", "machine learning")

g
```



For example, let's say your knowledge of statistics causes you to know more about causal inference and machine learning. If I don't know your level of statistical knowledge, then knowing that you are good at causal inference makes it more likely that you are also good at machine learning. That is because even if I don't know your level of statistical knowledge, I can infer it from your causal inference knowledge. If you are good at causal inference, you are probably good at statistics, making it more likely that you are good at machine learning.

Now, if I condition on your knowledge about statistics, then how much you know about machine learning becomes independent of how much you know about causal inference. Knowing your level of statistics already gives me all the information I need to infer the level of your machine learning skills. You see, knowing your level of causal inference will provide no further information in this case.

As a general rule, two variables that share a common cause are dependent, but independent when we condition on the common cause. Or

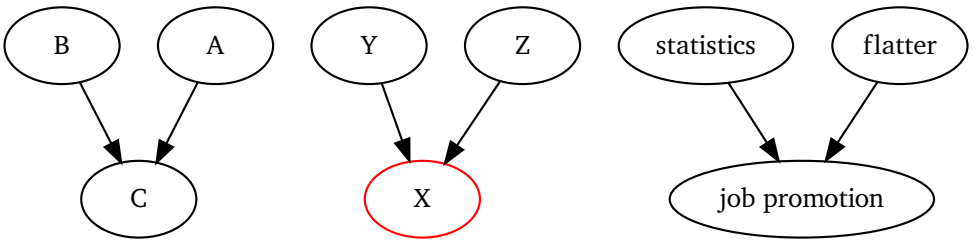
$$A \not\perp B$$

and

$$A \perp B \mid C$$

The only structure that is missing is the collider. A collider is when two arrows collide on a single variable. We can say that in this case, both variables share a common effect.

```
g = gr.Digraph()
g.edge("B", "C")
g.edge("A", "C")

g.edge("Y", "X")
g.edge("Z", "X")
g.node("X", "X", color="red")

g.edge("statistics", "job promotion")
g.edge("flatter", "job promotion")

g
```



For example, consider that there are two ways to get a job promotion. You can either be good at statistics or flatter your boss. If I don't condition on your job promotion, that is, I know nothing if you will or won't get it, then your level of statistics and flattering are independent. In other words, knowing how good you are at statistics tells me nothing about how good you are at flattering your boss. On the other hand, if you did get a job promotion, suddenly, knowing your level of statistics tells me about your flattering level. If you are bad at statistics and did get a promotion, you will likely know how to flatter. Otherwise, you wouldn't get a promotion. Conversely, if you are bad at flattering, it must be the case that you are good at statistics. This phenomenon is sometimes called **explaining away**, because one cause already explains the effect, making the other cause less likely.

As a general rule, conditioning on a collider opens the dependence path. Not conditioning on it leaves it closed. Or

$$A \perp B$$

and

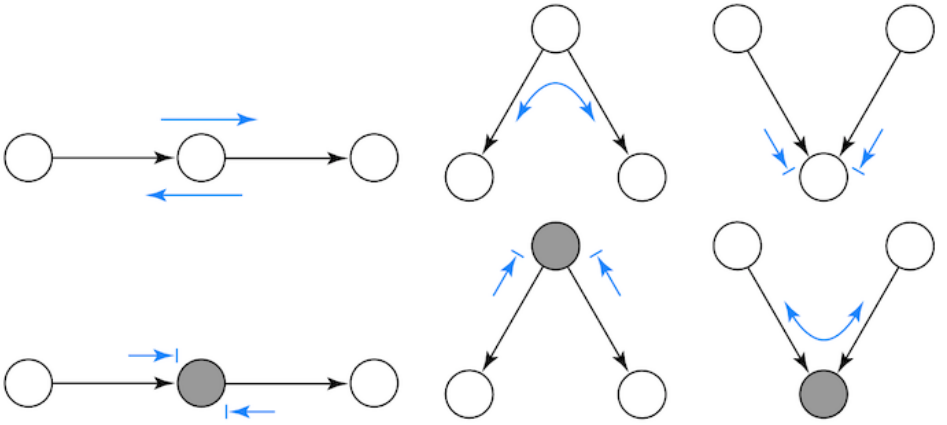
$$A \not\perp B \mid C$$

Knowing the three structures, we can derive an even more general rule. A path is blocked if and only if:

1. It contains a non collider that has been conditioned on
2. It contains a collider that has not been conditioned on and has no descendants that have been conditioned on.

Here is a cheat sheet about how dependence flows in a graph. I've taken from a Stanford presentation[2] by Mark Paskin. The arrows with lines at their tips signify independence, and the arrows without lines at their tips signify dependence.

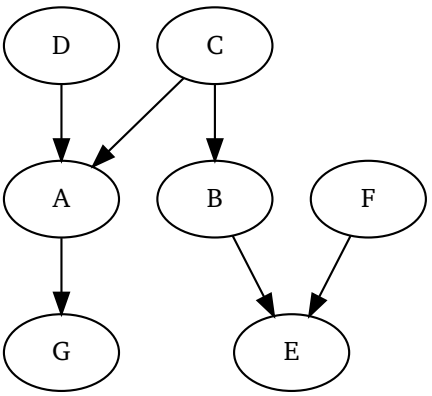
2 http://ai.stanford.edu/~paskin/gm-short-course/lec2.pdf



As a final example, try to figure out some independence and dependence relationship in the following causal graph.

1. Is $D \perp C$?
2. Is $D \perp C \mid A$?
3. Is $D \perp C \mid G$?
4. Is $A \perp F$?
5. Is $A \perp F \mid E$?
6. Is $A \perp F \mid E, C$?

```
g = gr.Digraph()
g.edge("C", "A")
g.edge("C", "B")
g.edge("D", "A")
g.edge("B", "E")
g.edge("F", "E")
g.edge("A", "G")

g
```



Answers:

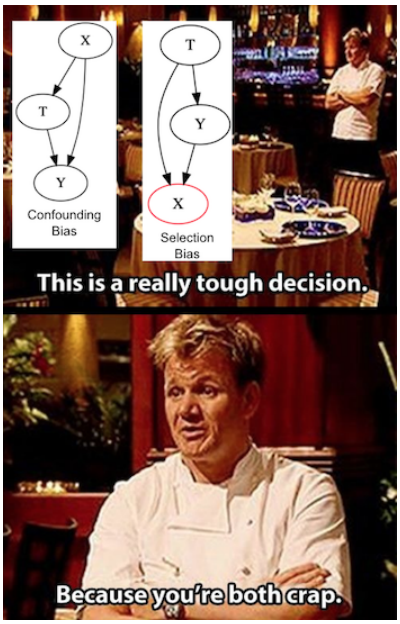
1. $D \perp C$. It contains a collider that it has **not** been conditioned on.
2. $D \not\perp C \mid A$. It contains a collider that it has been conditioned on.
3. $D \not\perp C \mid G$. It contains the descendent of a collider that has been conditioned on. You can see G as some kind of proxy for A here.
4. $A \perp F$. It contains a collider, B->E<-F, that it has **not** been conditioned on.
5. $A \not\perp F \mid E$. It contains a collider, B->E<-F, that it has been conditioned on.
6. $A \perp F \mid E, C$. It contains a collider, B->E<-F, that it has been conditioned on, but it contains a non collider that has been conditioned on. Conditioning on E opens the path, but conditioning on C closes it again.

Knowing about causal graphical models enables us to understand the problems that arise in causal inference. As we've seen, the problem always boils down to bias.

$$E[Y \mid T=1] - E[Y \mid T=0] = \underbrace{E[Y_1 - Y_0 \mid T=1]}_{ATET} + \{\underbrace{E[Y_0 \mid T=1] - E[Y_0 \mid T=0]}_{BIAS}\}$$

Graphical models allow us to diagnose which bias we are dealing with and what are the tools we need to correct for them.

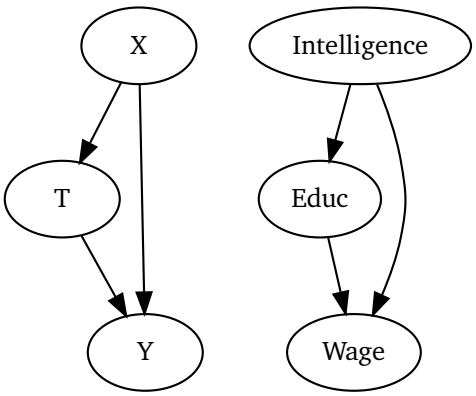
Confounding Bias



The first significant cause of bias is confounding. It happens when the treatment and the outcome share a common cause. For example, let's say that the treatment is education, and the outcome is income. It is hard to know the causal effect of education on wages because both share a common cause: intelligence. So we could argue that more educated

people earn more money simply because they are more intelligent, not because they have more education. We need to close all backdoor paths between the treatment and the outcome to identify the causal effect. If we do so, the only effect that will be left is the direct effect T->Y. In our example, if we control for intelligence, that is, we compare people with the same level of intellect but different levels of education, the difference in the outcome will be only due to the difference in schooling since intelligence will be the same for everyone. To fix confounding bias, we need to control all common causes of the treatment and the outcome.

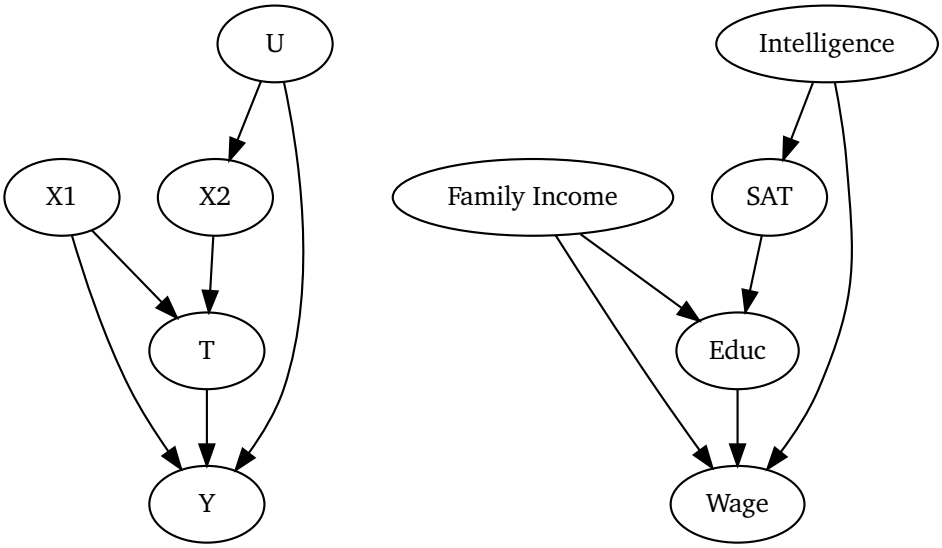
```
g = gr.Digraph()
g.edge("X", "T")
g.edge("X", "Y")
g.edge("T", "Y")

g.edge("Intelligence", "Educ"),
g.edge("Intelligence", "Wage"),
g.edge("Educ", "Wage")
g
```



Unfortunately, it is not always possible to control all common causes. Sometimes, there are unknown causes or known causes that we can't measure. The case of intelligence is one of the latter. Despite all the effort, scientists haven't yet figured out how to measure intelligence well. I'll use U to denote unmeasured variables here. Now, assume for a moment that intelligence can't affect your education directly. It affects how well you do on the SATs, but the SATs determine your level of schooling since it opens the possibility of a good college. Even if we can't control for the unmeasurable intelligence, we can control for SAT and close that backdoor path.

```
g = gr.Digraph()
g.edge("X1", "T")
g.edge("T", "Y")
g.edge("X2", "T")
g.edge("X1", "Y")
```

```
g.edge("U", "X2")
g.edge("U", "Y")

g.edge("Family Income", "Educ")
g.edge("Educ", "Wage")
g.edge("SAT", "Educ")
g.edge("Family Income", "Wage")
g.edge("Intelligence", "SAT")
g.edge("Intelligence", "Wage")
g
```

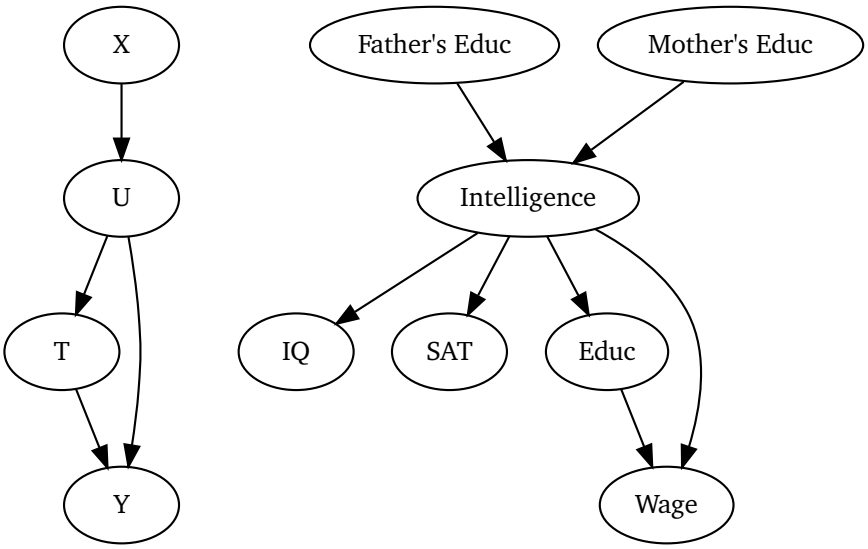


In the following graph, conditioning on X1 and X2, or SAT and family income, is sufficient to close all backdoor paths between the treatment and the outcome. In other words, $(Y_0, Y_1) \perp T \,|\, X1, X2$. So even if we can't measure all common causes, we can still attain conditional independence if we control for measurable variables that mediate the effect of the unmeasured on the treatment. One quick note, we also have $(Y_0, Y_1) \perp T \,|\, X1, U$, but since we can't observe U, we can't condition on it.

But what if that is not the case? What if the unmeasured variable causes the treatment and the outcome directly? In the following example, intelligence causes education and income now. So there is confounding in the relationship between the treatment education and the outcome wage. In this case, we can't control the confounder because it is unmeasurable. However, we have other measured variables that can act as a proxy for the confounder. Those variables are not in the backdoor path, but controlling for them will help lower the bias (but it won't eliminate it). Those variables are sometimes referred to as surrogate confounders.

In our example, we can't measure intelligence, but we can measure some of its causes, like the father's and mother's education, and some of its effects, like IQ or SAT score. Controlling for those surrogate variables is not sufficient to eliminate bias, but it helps.

```
g = gr.Digraph()
g.edge("X", "U")
g.edge("U", "T")
g.edge("T", "Y")
g.edge("U", "Y")

g.edge("Intelligence", "IQ")
g.edge("Intelligence", "SAT")
g.edge("Father's Educ", "Intelligence")
g.edge("Mother's Educ", "Intelligence")

g.edge("Intelligence", "Educ")
g.edge("Educ", "Wage")
g.edge("Intelligence", "Wage")

g
```



Selection Bias

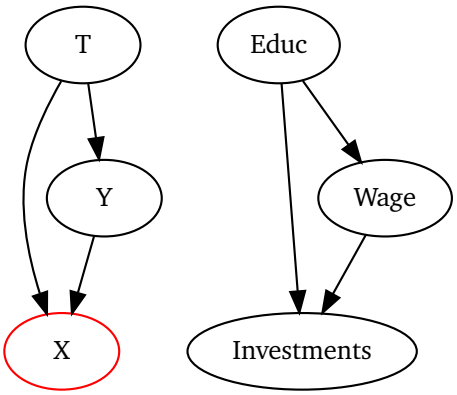
You might think it is a good idea to add everything you can measure to your model just to ensure you don't have confounding bias. Well, think again.

The second significant source of bias is what we will call selection bias. Here, I think the distinction between it and confounding bias is constructive, so I'll stick to it. If confounding bias happens when we don't control for a common cause, selection bias is more related to effects. One word of caution here, economists tend to refer to all sorts of biases as selection bias.

Often, selection bias arises when we control for more variables than we should. It might be the case that the treatment and the potential outcome are marginally independent but become dependent once we condition on a collider.

Imagine that with the help of some miracle, you can finally randomize education to measure its effect on wages. But just to be sure you won't have confounding, you control for many variables. Among them, you control for investments. But investment is not a common cause of education and wage. Instead, it is a consequence of both. More educated people both earn more and invest more. Also, those who make more invest more. Since investment is a collider, by conditioning on it, you are opening a second path between the treatment and the outcome, which will make it harder to measure the direct effect. One way to think about this is that by controlling investments, you look at small groups of the population where investment is the same and then find the effect of education on those groups. But by doing so, you are also indirectly and inadvertently not allowing wages to change much. As a result, you won't be able to see how education changes wages because you are not allowing wages to change as they should.

```
g = gr.Digraph()
g.edge("T", "X")
g.edge("T", "Y")
g.edge("Y", "X")
g.node("X", "X", color="red")

g.edge("Educ", "Investments")
g.edge("Educ", "Wage")
g.edge("Wage", "Investments")

g
```



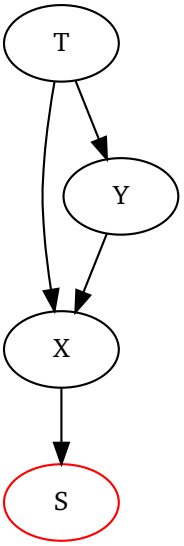
Imagine that investments and education take only 2 values to demonstrate why this is the case. Whether people invest or not. They are either educated or not. Initially, when we don't control for investments, the bias term is zero: $E[Y_0|T=1] - E[Y_0|T=0] = 0$ because the education was randomised. This means that the wage people would have if they didn't receive education $Wage_0$ is the same if they do or don't receive the education treatment. But what happens if we condition on investments?

Looking at those who invest, we probably have the case that $E[Y_0|T=0, I=1] > E[Y_0|T=1, I=1]$. In words, among those who invest, the ones who manage to do so even without education are more likely to achieve high earnings, regardless of their education level. For this reason, the wage those people have, $Wage_0|T=0$, is probably higher than the wage the educated group would have if they didn't have education, $Wage_0|T=1$. A similar reasoning can be applied to those who don't invest, where we also probably have $E[Y_0|T=0, I=0] > E[Y_0|T=1, I=0]$. Those who don't invest even with education probably would have a lower wage, had they not got the education, than those who didn't invest but didn't have an education.

To use a purely graphical argument, if someone invests, knowing that they have high education explains away the second cause, which is wage. Conditioned on investing, higher education is associated with low wages and we have a negative bias $E[Y_0|T=0, I=i] > E[Y_0|T=1, I=i]$.

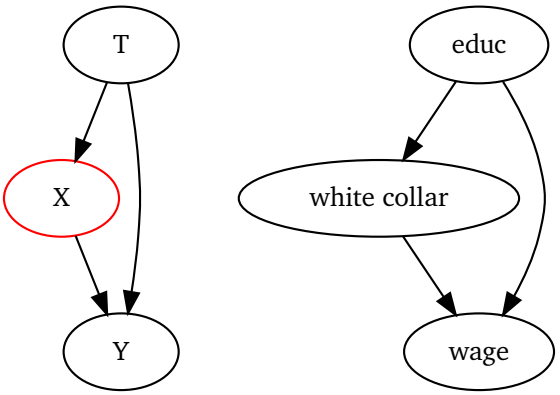
As a side note, all of this we've discussed is true if we condition on any descendent of a common effect.

```
g = gr.Digraph()
g.edge("T", "X")
g.edge("T", "Y")
g.edge("Y", "X")
g.edge("X", "S")
g.node("S", "S", color="red")
g
```



A similar thing happens when we condition on a mediator of the treatment. A mediator is a variable between the treatment and the outcome. It, well, mediates the causal effect. For example, suppose again you can randomize education. But, just to be sure, you decide to control whether or not the person had a white-collar job. Once again, this conditioning biasses the causal effect estimation. This time, not because it opens a front door path with a collider, but because it closes one of the channels through which the treatment operates. In our example, getting a white-collar job is one way more education leads to higher pay. By controlling it, we close this channel and leave open only the direct effect of education on wages.

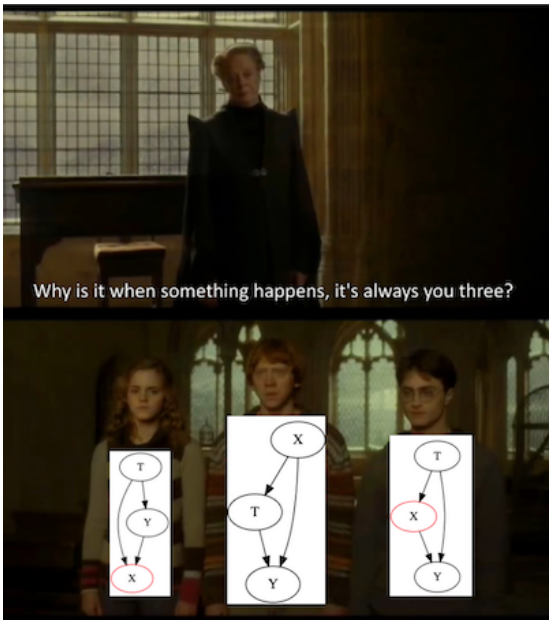
```
g = gr.Digraph()
g.edge("T", "X")
g.edge("T", "Y")
g.edge("X", "Y")
g.node("X", "X", color="red")

g.edge("educ", "white collar")
g.edge("educ", "wage")
g.edge("white collar", "wage")

g
```



To give a potential outcome argument, we know that, due to randomisation, the bias is zero $E[Y_0|T=0] - E[Y_0|T=1] = 0$. However, if we condition on the white-collar individuals, we have that $E[Y_0|T=0, WC=1] > E[Y_0|T=1, WC=1]$. That is because those who manage to get a white-collar job even without education are probably more hard-working than those who require the help of education to get the same job. With the same reasoning, $E[Y_0|T=0, WC=0] > E[Y_0|T=1, WC=0]$ because those that didn't get a white-collar job even with an education are probably less hard-working than those that didn't, but also didn't have any education.

In our case, conditioning on the mediator induces a negative bias. It makes the effect of education seem lower than it actually is. This is the case because the causal effect is positive. If the effect were negative, conditioning on a mediator would have a positive bias. In all cases, this sort of conditioning makes the effect look weaker than it is.

To put it more prosaic way, suppose that you have to choose between two candidates for a job at your company. Both have equally impressive professional achievements, but one does not have a higher education degree. Which one should you choose? Of course, you should go with the one without the higher education because he managed to achieve the same things as the other one but had the odds stacked against him.



Key Ideas

We've studied graphical models as a language to better understand and express causality ideas. We did a quick summary of the rules of conditional independence on a graph. This helped us then explore three structures that can lead to bias.

The first was confounding, which happens when treatment and outcome have a common cause that we don't account for or control for. The second is selection bias due to conditioning on a common effect. The third structure is also a form of selection bias, this time due to excessive controlling of mediator variables. This excessive controlling could lead to bias even if the treatment was randomly assigned. Selection bias can often be fixed by simply doing nothing, which is why it is dangerous. Since we are biased toward action, we tend to see ideas that control things as clever when they can be doing more harm than good.

References

I like to think of this entire book as a tribute to Joshua Angrist, Alberto Abadie and Christopher Walters for their amazing Econometrics class. Most of the ideas here are taken from their classes at the American Economic Association. Watching them is what is keeping me sane during this tough year of 2020.

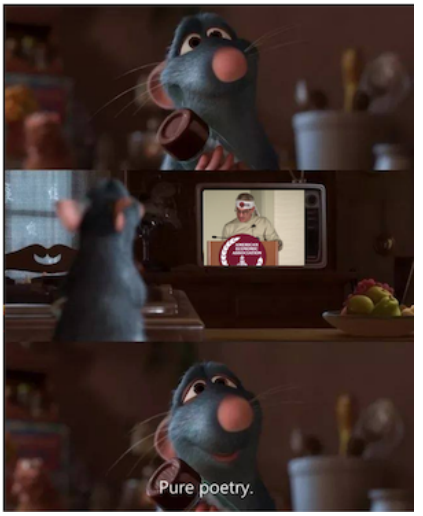
- Cross-Section Econometrics · https://www.aeaweb.org/conference/cont-ed/2017-webcasts
- Mastering Mostly Harmless Econometrics · https://www.aeaweb.org/conference/cont-ed/2020-webcasts

I'll also like to reference the amazing books from Angrist. They have shown me that Econometrics, or 'Metrics as they call it, is not only extremely useful but also profoundly fun.

- Mostly Harmless Econometrics · https://www.mostlyharmlesseconometrics.com/
- Mastering 'Metrics · https://www.masteringmetrics.com/

My final reference is Miguel Hernan and Jamie Robins' book. It has been my trustworthy companion in the most thorny causal questions I had to answer.

- Causal Inference Book · https://miguelhernan.org/whatifbook



CHAPTER

5

The Unreasonable Effectiveness of Linear Regression

All You Need is Regression

When dealing with causal inference, we saw how there are two potential outcomes for each individual: Y_0 is the outcome the individual would have if he or she didn't take the treatment and Y_1 is the outcome if he or she took the treatment. The act of setting the treatment T to 0 or 1 materializes one of the potential outcomes and makes it impossible for us to ever know the other one. This leads to the fact that the individual treatment effect $\tau_i = Y_{1i} - Y_{0i}$ is unknowable.

$$Y_i = Y_{0i} + T_i(Y_{1i} - Y_{0i}) = Y_{0i}(1 - T_i) + T_i Y_{1i}$$

So, for now, let's focus on the simpler task of estimating the average causal effect. With this in mind, we are accepting the fact that some people respond better than others to the treatment, but we are also accepting that we can't know who they are. Instead, we will just try to see if the treatment works, **on average**.

$$ATE = E[Y_1 - Y_0]$$

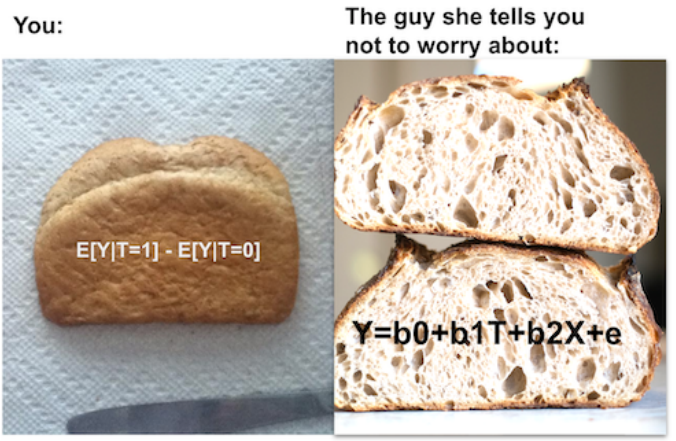
This will give us a simplified model, with a constant treatment effect $Y_{1i} = Y_{0i} + \kappa$. If κ is positive, we will say that the treatment has, on average, a positive effect. Even if some people will respond badly to it, on average, the impact will be positive.

Let's also recall that we can't simply estimate $E[Y_1 - Y_0]$ with the difference in mean $E[Y \mid T = 1] - E[Y \mid T = 0]$ due to bias. Bias often arises when the treated and untreated are different for reasons other than the treatment itself. One way to see this is on how they differ in the potential outcome Y_0

$$E[Y \mid T = 1] - E[Y \mid T = 0] = \underbrace{E[Y_1 - Y_0 \mid T = 1]}_{ATET} + \{\underbrace{E[Y_0 \mid T = 1] - E[Y_0 \mid T = 0]}_{BIAS}\}$$

Previously, we saw how we can eliminate bias with Random Experiments, or **Randomised Controlled Trial** (RCT) as they are sometimes called. RCT forces the treated and the untreated to be equal and that's why the bias vanishes. We also saw how to place uncertainty levels around our estimates for the treatment effect. Namely, we looked at the case of online versus face-to-face classrooms, where $T = 0$ represent face-to-face lectures and $T = 1$ represent online ones. Students were randomly assigned to one of those 2 types of lectures and then their performance on an exam was evaluated. We've built an A/B testing function that could compare both groups, provide the average treatment effect and even place a confidence interval around it.

Now, it's time to see that we can do all of that with the workhorse of causal inference: **Linear Regression**! Think of it this way. If comparing treated and untreated means was an apple for dessert, linear regression would be cold and creamy tiramisu. Or if comparing treated and untreated is a sad and old loaf of white wonder bread, linear regression would be a crusty, soft crumb country loaf sourdough baked by Chad Robertson himself.



Lets see how this beauty works. In the code below, we want to run the exact same analysis of comparing online vs face-to-face classes. But instead of doing all that math of confidence intervals, we just run a regression. More specifically, we estimate the following model:

$$exam_i = \beta_0 + \kappa\, Online_i + u_i$$

This means we are modeling the exam outcome as a baseline β_0 plus κ if the class is online. Of course the exam result is driven by additional variables (like student's mood on the exam day, hours studied and so on). But we don't really care about understanding those relationships. So, instead, we use that u_i term to represent everything else we don't care about. This is called the model error.

Notice that *Online* is our treatment indication and hence, a dummy variable. It is zero when the treatment is face to face and 1 if it's online. With that in mind, we can see that

linear regression will recover $E[Y \mid T = 0] = \beta_0$ and $E[Y \mid T = 1] = \beta_0 + \kappa$. κ will be our ATE.

```
import warnings
warnings.filterwarnings('ignore')

import pandas as pd
import numpy as np
import statsmodels.formula.api as smf
import graphviz as gr
%matplotlib inline
```

```
data = pd.read_csv("data/online_classroom.csv").query("format_blended==0")

result = smf.ols('falsexam ~ format_ol', data=data).fit()
result.summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	78.5475	1.113	70.563	0.000	76.353	80.742
format_ol	-4.9122	1.680	-2.925	0.004	-8.223	-1.601

That's quite amazing. We are not only able to estimate the ATE, but we also get, for free, confidence intervals and P-Values out of it! More than that, we can see that regression is doing exactly what it is supposed to do: comparing $E[Y \mid T = 0]$ and $E[Y \mid T = 1]$. The intercept is exactly the sample mean when $T = 0$, $E[Y \mid T = 0]$, and the coefficient of the online format is exactly the sample difference in means $E[Y \mid T = 1] - E[Y \mid T = 0]$. Don't trust me? No problem. You can see for yourself:

```
(data
 .groupby("format_ol")
 ["falsexam"]
 .mean())
```

```
format_ol
0    78.547485
1    73.635263
Name: falsexam, dtype: float64
```

As expected. If you add to the intercept the ATE, that is, the parameter estimate of online format, you get the sample mean for the treated: $78.5475 + (-4.9122) = 73.635263$.

Regression Theory

I don't intend to dive too deep into how linear regression is constructed and estimated. However, a little bit of theory will go a long way in explaining its power in causal inference.

First of all, regression solves a theoretical best linear prediction problem. Let β^* be a vector of parameters:

$$\beta^* = \underset{\beta}{argmin}\ E[(Y_i - X_i'\beta)^2]$$

Linear regression finds the parameters that minimise the mean squared error (MSE).

If you differentiate it and set it to zero, you will find that the linear solution to this problem is given by

$$\beta^* = E[X_i'X_i]^{-1}E[X_i'Y_i]$$

We can estimate this beta using the sample equivalent:

$$\hat{\beta} = (X'X)^{-1}X'Y$$

But don't take my word for it. If you are one of those that understand code better than formulas, try for yourself:

```
X = data[["format_ol"]].assign(intercep=1)
y = data["falsexam"]

def regress(y, X):
    return np.linalg.inv(X.T.dot(X)).dot(X.T.dot(y))

beta = regress(y, X)
beta
```

```
array([-4.9122215 , 78.54748458])
```

The formulas above are pretty general. However, it pays off to study the case where we only have one regressor. In causal inference, we often want to estimate the causal impact of a variable T on an outcome y. So, we use regression with this single variable to estimate this effect. Even if we include other variables in the model, they are usually just auxiliary. Adding other variables can help us estimate the causal effect of the treatment, but we are not very interested in estimating their parameters.

With a single regressor variable T, the parameter associated to it will be given by

$$\beta_1 = \frac{Cov(Y_i, T_i)}{Var(T_i)}$$

If T is randomly assigned, β_1 is the ATE.

```
kappa = data["falsexam"].cov(data["format_ol"]) / data["format_ol"].var()
kappa
```

```
-4.91222149822695
```

If we have more than one regressor, we can extend the following formula to accommodate that. Let's say those other variables are just auxiliary and that we are truly interested only in estimating the parameter κ associated to T.

$$y_i = \beta_0 + \kappa T_i + \beta_1 X_{1i} + ... + \beta_k X_{ki} + u_i$$

κ can be obtained with the following formula

$$\kappa = \frac{Cov(Y_i, \tilde{T}_i)}{Var(\tilde{T}_i)}$$

where $\tilde{T}_i$ is the residual from a regression of T_i on all other covariates $X_{1i}, ..., X_{ki}$. Now, let's appreciate how cool this is. It means that the coefficient of a multivariate regression is the bivariate coefficient of the same regressor **after accounting for the effect of other variables in the model**. In causal inference terms, κ is the bivariate coefficient of T after having used all other variables to predict it.

This has a nice intuition behind it. If we can predict T using other variables, it means it's not random. However, we can make it so that T is as good as random once we control for other available variables. To do so, we use linear regression to predict it from the other variables and then we take the residuals of that regression $\tilde{T}$. By definition, $\tilde{T}$ cannot be predicted by the other variables X that we've already used to predict T. Quite elegantly, $\tilde{T}$ is a version of the treatment that is not associated with any other variable in X.

By the way, this is also a property of linear regression. The residual are always orthogonal or uncorrelated with any of the variables in the model that created it:

```
e = y - X.dot(beta)
print("Orthogonality imply that the dot product is zero:", np.dot(e, X))
X[["format_ol"]].assign(e=e).corr()
```

```
Orthogonality imply that the dot product is zero: [7.81597009e-13 4.63984406e-12]
```

	format_ol	e
format_ol	1.000000e+00	-9.419033e-16
e	-9.419033e-16	1.000000e+00

And what is even cooler is that these properties don't depend on anything! They are mathematical truths, regardless of what your data looks like.

Regression For Non-Random Data

So far, we worked with random experiment data but, as we know, those are hard to come by. Experiments are very expensive to conduct or simply infeasible. It's very hard to convince McKinsey & Co. to randomly provide their services free of charge so that we

can, once and for all, distinguish the value their consulting services brings from the fact that those firms that can afford to pay them are already very well off.

For this reason, we shall now delve into non random or observational data. In the following example, we will try to estimate the impact of an additional year of education on hourly wage. As you might have guessed, it is extremely hard to conduct an experiment with education. You can't simply randomize people to 4, 8 or 12 years of education. In this case observational data is all we have.

First, let's estimate a very simple model. We will regress log hourly wages on years of education. We use logs here so that our parameter estimates have a percentage interpretation (if you never heard about this amazing properties of the log and want to know why that is, check out this link[1]). With it, we will be able to say that 1 extra year of education yields a wage increase of x%.

$$log(hwage)_i = \beta_0 + \beta_1 educ_i + u_i$$

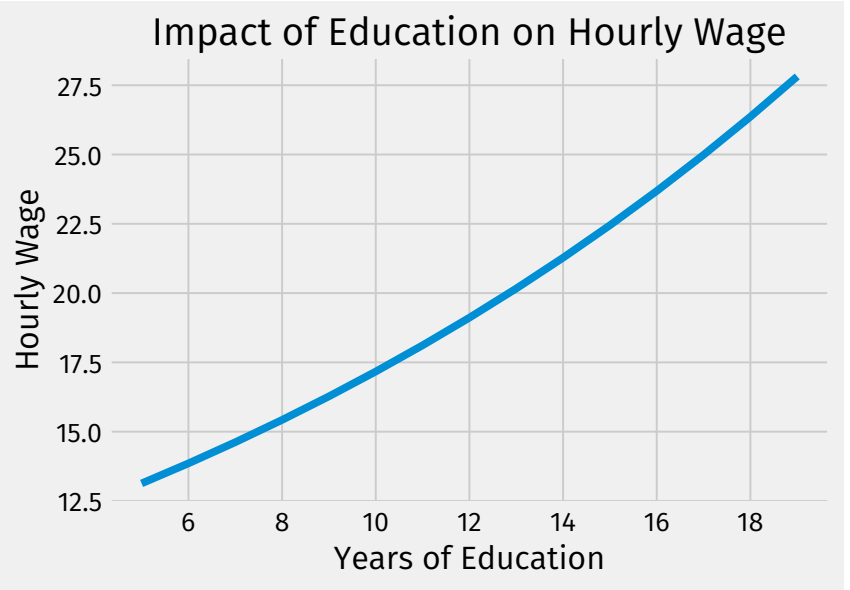
```
wage = pd.read_csv("./data/wage.csv").dropna()
model_1 = smf.ols('np.log(hwage) ~ educ',
  ↪ data=wage.assign(hwage=wage["wage"]/wage["hours"])).fit()
model_1.summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	2.3071	0.104	22.089	0.000	2.102	2.512
educ	0.0536	0.008	7.114	0.000	0.039	0.068

The estimate of β_1 is 0.0536, with a 95% confidence interval of (0.039, 0.068). This means that this model predicts that wages will increase about 5.3% for every additional year of education. This percentage increase is inline with the belief that education impacts wages in an exponential fashion: we expect that going from 11 to 12 years of education (average to graduate high school) to be less rewarding than going from 14 to 16 years (average to graduate college).

```
from matplotlib import pyplot as plt
from matplotlib import style
style.use("fivethirtyeight")

x = np.array(range(5, 20))
plt.plot(x, np.exp(model_1.params["Intercept"] + model_1.params["educ"] * x))
plt.xlabel("Years of Education")
plt.ylabel("Hourly Wage")
plt.title("Impact of Education on Hourly Wage")
plt.show()
```

1 https://stats.stackexchange.com/questions/244199/why-is-it-that-natural-log-changes-are-percentage-changes-what-is-about-logs-th



Of course, it is not because we can estimate this simple model that it's correct. Notice how I was careful with my words saying it **predicts** wage from education. I never said that this prediction was causal. In fact, by now, you probably have very serious reasons to believe this model is biased. Since our data didn't come from a random experiment, we don't know if those that got more education are comparable to those who got less. Going even further, from our understanding of how the world works, we are very certain that they are not comparable. Namely, we can argue that those with more years of education probably have richer parents, and that the increase we are seeing in wages as we increase education is just a reflection of how the family wealth is associated with more years of education. Putting it in math terms, we think that $E[Y_0|T=0] < E[Y_0|T=1]$, that is, those with more education would have higher income anyway, even without so many years of education. If you are really grim about education, you can argue that it can even *reduce* wages by keeping people out of the workforce and lowering their experience.

Fortunately, in our data, we have access to lots of other variables. We can see the parents' education `meduc`, `feduc`, the `IQ` score for that person, the number of years of experience `exper` and the tenure of the person in his or her current company `tenure`. We even have some dummy variables for marriage and black ethnicity.

```
wage.head()
```

	wage	hours	lhwage	IQ	educ	exper	tenure	age
0	769	40	2.956212	93	12	11	2	31
2	825	40	3.026504	108	14	11	9	33
3	650	40	2.788093	96	12	13	7	32
4	562	40	2.642622	74	11	14	5	34
6	600	40	2.708050	91	10	13	0	30

	married	black	south	urban	sibs	brthord	meduc	feduc
0	1	0	0	1	1	2.0	8.0	8.0
2	1	0	0	1	1	2.0	14.0	14.0
3	1	0	0	1	4	3.0	12.0	12.0
4	1	0	0	1	10	6.0	6.0	11.0
6	0	0	0	1	1	2.0	8.0	8.0

We can include all those extra variables in a model and estimate it:

$$log(hwage)_i = \beta_0 + \kappa\, educ_i + \boldsymbol{\beta} X_i + u_i$$

To understand how this helps with the bias problem, let's recap the bivariate breakdown of multivariate linear regression.

$$\kappa = \frac{Cov(Y_i, \tilde{T}_i)}{Var(\tilde{T}_i)}$$

This formula says that we can predict educ from the parents' education, from IQ, from experience and so on. After we do that, we'll be left with a version of educ, $\tilde{educ}$, which is uncorrelated with all the variables included previously. This will break down arguments such as "people that have more years of education have it because they have higher IQ. It is not the case that education leads to higher wages. It is just the case that it is correlated with IQ, which is what drives wages". Well, if we include IQ in our model, then κ becomes the return of an additional year of education while keeping IQ fixed. Pause a little bit to understand what this implies. Even if we can't use randomised controlled trials to keep other factors equal between treated and untreated, regression can do this by including those same factors in the model, even if the data is not random!

```
controls = ['IQ', 'exper', 'tenure', 'age', 'married', 'black',
            'south', 'urban', 'sibs', 'brthord', 'meduc', 'feduc']

X = wage[controls].assign(intercep=1)
t = wage["educ"]
y = wage["lhwage"]

beta_aux = regress(t, X)
t_tilde = t - X.dot(beta_aux)

kappa = t_tilde.cov(y) / t_tilde.var()
kappa
```

```
0.041147191010057635
```

This coefficient we've just estimated tells us that, for people with the same IQ, experience, tenure, age and so on, we should expect an additional year of education to be associated with a 4.11% increase in hourly wage. This confirms our suspicion that the first simple

model with only educ was biased. It also confirms that this bias was overestimating the impact of education. Once we controlled for other factors, the estimated impact of education fell.

If we are wiser and use software that other people wrote instead of coding everything yourself, we can even place a confidence interval around this estimate.

```
model_2 = smf.ols('lhwage ~ educ +' + '+'.join(controls), data=wage).fit()
model_2.summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	1.1156	0.232	4.802	0.000	0.659	1.572
educ	0.0411	0.010	4.075	0.000	0.021	0.061
IQ	0.0038	0.001	2.794	0.005	0.001	0.006
exper	0.0153	0.005	3.032	0.003	0.005	0.025
tenure	0.0094	0.003	2.836	0.005	0.003	0.016
age	0.0086	0.006	1.364	0.173	-0.004	0.021
married	0.1795	0.053	3.415	0.001	0.076	0.283
black	-0.0801	0.063	-1.263	0.207	-0.205	0.044
south	-0.0397	0.035	-1.129	0.259	-0.109	0.029
urban	0.1926	0.036	5.418	0.000	0.123	0.262
sibs	0.0065	0.009	0.722	0.470	-0.011	0.024
brthord	-0.0080	0.013	-0.604	0.546	-0.034	0.018
meduc	0.0089	0.007	1.265	0.206	-0.005	0.023
feduc	0.0069	0.006	1.113	0.266	-0.005	0.019

Omitted Variable or Confounding Bias

The question that remains is: is this parameter we've estimated causal? Unfortunately, we can't say for sure. We can argue that the first simple model that regress wage on education probably isn't. It omits important variables that are correlated both with education and with wages. Without controlling for them, the estimated impact of education is also capturing the impact of those other variables that were not included in the model.

To better understand how this bias works, let's suppose the true model for how education affects wage looks a bit like this

$$Wage_i = \alpha + \kappa\, Educ_i + \beta' A_i + u_i$$

wage is affected by education, which is measured by the size of κ and by an additional ability factor, denoted as A. If we omit ability from our model, our estimate for κ will look like this:

$$\frac{Cov(Wage_i, Educ_i)}{Var(Educ_i)} = \kappa + \beta' \delta_A$$

where δ_A is the coefficient from the regression of A on $Educ$

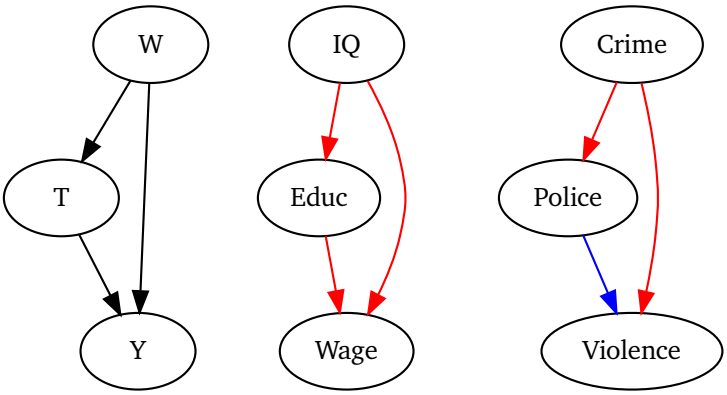
The key point here is that it won't be exactly the κ that we want. Instead, it comes with this extra annoying term $\beta'\delta_A$. This term is the impact of the omitted A on $Wage$, β' times the impact of the omitted on the included $Educ$. This is so important for economists that Joshua Angrist made a mantra out of it so that the students can recite it in meditation:

```
"Short equals long
plus the effect of omitted
times the regression of omitted on included"
```

Here, the short regression is the one that omits variables, while the long is the one that includes them. This formula or mantra gives us further insight into the nature of bias. First, the bias term will be zero if the omitted variables have no impact on the dependent variable Y. This makes total sense. I don't need to control for stuff that is irrelevant for wages when trying to understand the impact of education on it (like how tall the lilies of the field). Second, the bias term will also be zero if the omitted variables have no impact on the treatment variable. This also makes intuitive sense. If everything that impacts education has been included in the model, there is no way the estimated impact of education is mixed with a correlation from education on something else that also impacts wages.

To put it more succinctly, we say that **there is no OVB if all the confounding variables are accounted for in the model**. We can also leverage our knowledge about causal graphs here. A confounding variable is one that **causes both the treatment and the outcome**. In the wage example, IQ is a confounder. People with high IQ tend to complete more years of education because it's easier for them, so we can say that IQ causes education. People with high IQ also tend to be naturally more productive and consequently have higher wages, so IQ also causes wage. Since confounders are variables that affect both the treatment and the outcome, we mark them with an arrow going to T and Y. Here, I've denoted them with W. I've also marked positive causation with red and negative causation with blue.

```
g = gr.Digraph()

g.edge("W", "T"), g.edge("W", "Y"), g.edge("T", "Y")

g.edge("IQ", "Educ", color="red"), g.edge("IQ", "Wage", color="red"),
  ↪ g.edge("Educ", "Wage", color="red")

g.edge("Crime", "Police", color="red"), g.edge("Crime", "Violence",
  ↪ color="red"),
g.edge("Police", "Violence", color="blue")

g
```

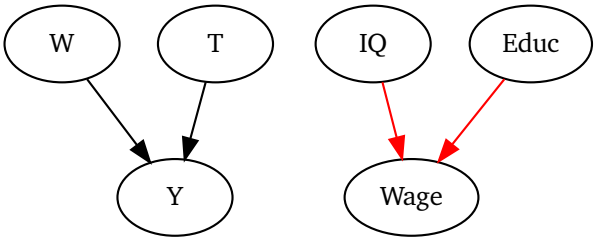


Causal graphs are excellent to depict our understanding of the world and understand how confounding bias works. In our first example, we have a graph where education causes wage: more education leads to higher wages. However, IQ also causes wage and it also causes education: high IQ causes both more education and wage. If we don't account for IQ in our model, some of its effect on wage will flow through the correlation with education. That will make the impact of education look higher than it actually is. This is an example of positive bias.

Just to give another example, but with negative bias, consider the causal graph about the effect of police on city violence. What we usually see in the world is that cities with higher police force also have more violence. Does this mean that the police are causing the violence? Well, it could be, I don't think it's worth getting into that discussion here. But, there is also a strong possibility that there is a confounding variable causing us to see a biased version of the impact of police on violence. It could be that increasing police force decreases violence. But, a third variable crime, causes both more violence and more police force. If we don't account for it, the impact of crime on violence will flow through police force, making it look like it increases violence. This is an example of negative bias.

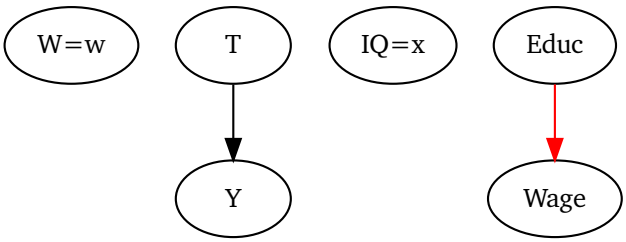
Causal graphs can also show us how both regression and randomized control trials are correct for confounding bias. RCT does so by severing the connection of the confounder to the treatment variable. By making T random, by definition, nothing can cause it.

```
g = gr.Digraph()

g.edge("W", "Y"), g.edge("T", "Y")

g.edge("IQ", "Wage", color="red"), g.edge("Educ", "Wage", color="red")

g
```



Regression, on the other hand, does so by comparing the effect of *T* while maintaining the confounder *W* set to a fixed level. With regression, it is not the case that W ceases to cause T and Y. It is just that it is held fixed, so it can't influence changes on T and Y.

```
g = gr.Digraph()

g.node("W=w"), g.edge("T", "Y")
g.node("IQ=x"), g.edge("Educ", "Wage", color="red")

g
```



Now, back to our question, is the parameter we've estimated for the impact of educ on wage causal? I'm sorry to bring it to you, but that will depend on our ability to argue in favor or against that fact that all confounders have been included in the model. Personally, I think they haven't. For instance, we haven't included family wealth. Even if we included family education, that can only be seen as a proxy for wealth. We've also not accounted for factors like personal ambition. It could be that ambition is what causes both more years of education and higher wage, so it is a confounder. This is to show that **causal inference with non-random or observational data should always be taken with a grain of salt**. We can never be sure that all confounders were accounted for.

Key Ideas

We've covered a lot of ground with regression. We saw how regression can be used to perform A/B testing and how it conveniently gives us confidence intervals. Then, we moved to study how regression solves a prediction problem and is the best linear approximation to the conditional expectation function (CEF). We've also discussed how, in the bivariate case, the regression treatment coefficient is the covariance between the treatment and the outcome divided by the variance of the treatment. Expanding to the multivariate case, we figured out how regression gives us a partialling out interpretation of the treatment coefficient: it can be interpreted as how the outcome would change with the treatment while keeping all other included variables constant. This is what economists love to refer as *ceteris paribus*.

Finally, we took a turn to understanding bias. We saw how `Short equals long plus the effect of omitted times the regression of omitted on included.` This shed some light to how bias comes to be. We discovered that the source of omitted variable bias is confounding: a variable that affects both the treatment and the outcome. Lastly, we used causal graphs to see how RCT and regression fixes confounding.

References

I like to think of this entire book as a tribute to Joshua Angrist, Alberto Abadie and Christopher Walters for their amazing Econometrics class. Most of the ideas here are taken from their classes at the American Economic Association. Watching them is what is keeping me sane during this tough year of 2020.

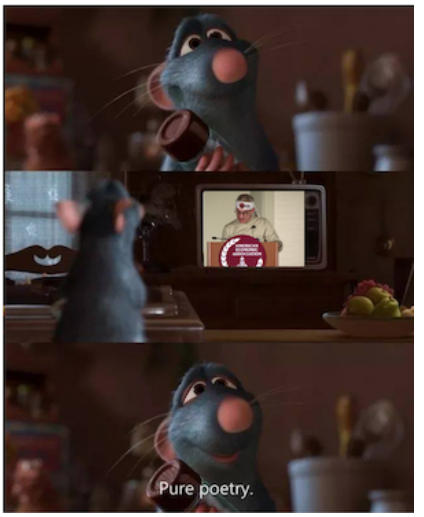
- Cross-Section Econometrics · https://www.aeaweb.org/conference/cont-ed/2017-webcasts
- Mastering Mostly Harmless Econometrics · https://www.aeaweb.org/conference/cont-ed/2020-webcasts

I'll also like to reference the amazing books from Angrist. They have shown me that Econometrics, or 'Metrics as they call it, is not only extremely useful but also profoundly fun.

- Mostly Harmless Econometrics · https://www.mostlyharmlesseconometrics.com/
- Mastering 'Metrics · https://www.masteringmetrics.com/

My final reference is Miguel Hernan and Jamie Robins' book. It has been my trustworthy companion in the most thorny causal questions I had to answer.

- Causal Inference Book · https://miguelhernan.org/whatifbook


Pure poetry.

CHAPTER

6

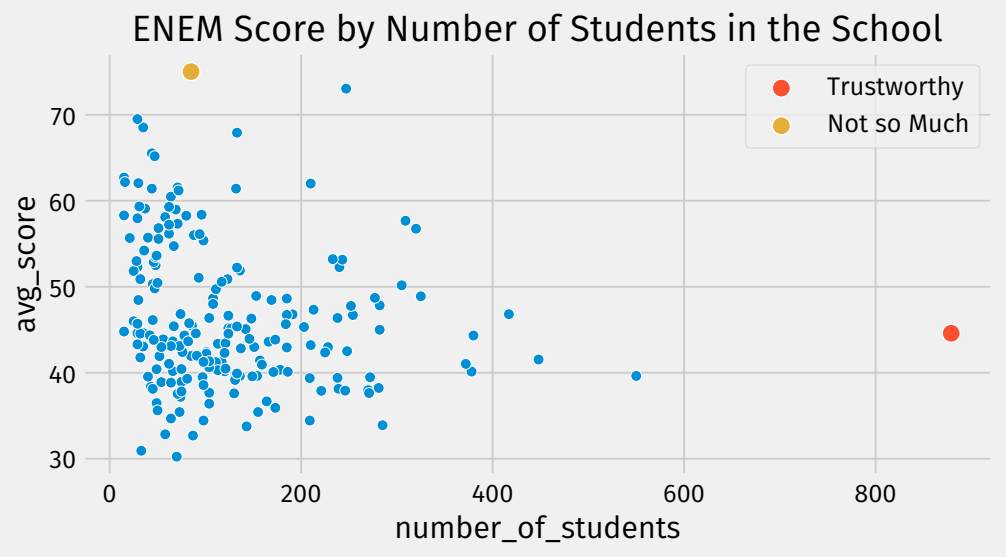
Grouped and Dummy Regression

Regression With Grouped Data

Not all data points are created equal. If we look again at our ENEM dataset, we trust the scores of big schools much more than the scores from small schools. This is not to say that big schools are better or anything. It is just due to the fact that their big size implies less variance.

```
import warnings
warnings.filterwarnings('ignore')

import pandas as pd
import numpy as np
from scipy import stats
from matplotlib import style
import seaborn as sns
from matplotlib import pyplot as plt
import statsmodels.formula.api as smf

style.use("fivethirtyeight")
```

```
np.random.seed(876)
enem = pd.read_csv("./data/enem_scores.csv").sample(200)
plt.figure(figsize=(8,4))
sns.scatterplot(y="avg_score", x="number_of_students", data=enem)
sns.scatterplot(y="avg_score", x="number_of_students", s=100,
  ↪ label="Trustworthy",

      ↪ data=enem.query(f"number_of_students=={enem.number_of_students.max()}"))
sns.scatterplot(y="avg_score", x="number_of_students", s=100,
  ↪ label="Not so Much",
                data=enem.query(f"avg_score=={enem.avg_score.max()}"))
plt.title("ENEM Score by Number of Students in the School");
```



In the data above, intuitively, points to the left should have less impact in my model than points to the right. In essence, points to the right are actually lots of other data points grouped into a single one. If we could unbundle them and run a linear regression on the ungrouped data, they would indeed contribute much more to the model estimation than an unbundled point in the left.

This phenomenon of having a region of low variance and another of high variance is called **heteroskedasticity**. Put it simply, heteroskedasticity is when the variance is not constant across all values of the features. In the case above, we can see that the variance decreases as the feature sample size increases. To give another example of where we have heteroskedasticity, if you plot wage by age, you will see that there is higher wage variance for the old than for the young. But, by far, the most common reason for variance to differ is grouped data.

Grouped data like the one above are extremely common in data analysis. One reason for that is confidentiality. Governments and firms can't give away personal data because that

would violate data privacy requirements they have to follow. If they need to export data to an outside researcher, they can only do it by means of grouping the data. This way, individuals get grouped together and are no longer uniquely identifiable.

Fortunately for us, regression can handle those kinds of data pretty well. To understand how, let's first take some ungrouped data like the one we had on wage and education. It contains one line per worker, so we know the wage for each individual in this dataset and also how many years of education he or she has.

```
wage = pd.read_csv("./data/wage.csv")[["wage", "lhwage", "educ", "IQ"]]

wage.head()
```

	wage	lhwage	educ	IQ
0	769	2.956212	12	93
1	808	2.782539	18	119
2	825	3.026504	14	108
3	650	2.788093	12	96
4	562	2.642622	11	74

If we run a regression model to figure out how education is associated with log hourly wages, we get the following result.

```
model_1 = smf.ols('lhwage ~ educ', data=wage).fit()
model_1.summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	2.2954	0.089	25.754	0.000	2.121	2.470
educ	0.0529	0.007	8.107	0.000	0.040	0.066

Now, let's pretend for a moment that this data was under some confidentiality constraint. The provider of it was not able to give individualised data. So we ask him instead to group everyone by years of education and give us only the mean log hourly wage and the number of individuals in each group. This leaves us with only 10 data points.

```
group_wage = (wage
              .assign(count=1)
              .groupby("educ")
              .agg({"lhwage":"mean", "count":"count"})
              .reset_index())

group_wage
```

	educ	lhwage	count
0	9	2.856475	10
1	10	2.786911	35
2	11	2.855997	43
3	12	2.922168	393
4	13	3.021182	85
5	14	3.042352	77
6	15	3.090766	45
7	16	3.176184	150
8	17	3.246566	40
9	18	3.144257	57

Fear not! Regression doesn't need big data to work! What we can do is provide weights to our linear regression model. This way, it will consider groups with higher sample size more than the small groups. Notice how I've replaced the `smf.ols` with `smf.wls`, for weighted least squares. It's hard to notice, but it will make all the difference.

```
model_2 = smf.wls('lhwage ~ educ', data=group_wage, weights=group_wage["count"])
  ↪ .fit()
model_2.summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	2.2954	0.078	29.327	0.000	2.115	2.476
educ	0.0529	0.006	9.231	0.000	0.040	0.066

Notice how the parameter estimate of educ in the grouped model is very close to the one in the ungrouped data (actually, they are the same in this case). Also, even with only 10 data points, we've managed to get a statistically significant coefficient. That's because, although we have fewer points, grouping also lowers the variance by a lot. Also notice how the standard error is a bit smaller and the t statistics is a bit larger. That's because some information about the variance is lost, so we have to be more conservative. Once we group the data, we don't know how large the variance is within each group. Compare the results above with what we would have with the non weighted model below.

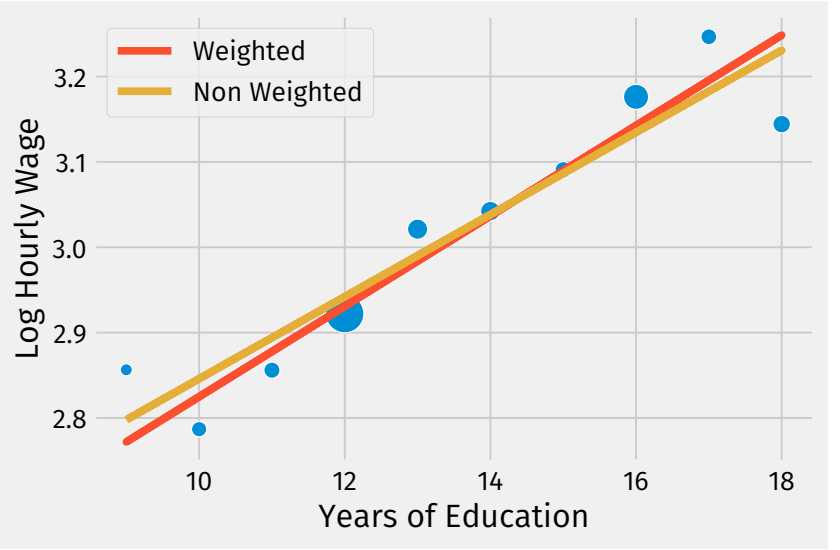
```
model_3 = smf.ols('lhwage ~ educ', data=group_wage).fit()
model_3.summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	2.3650	0.082	28.988	0.000	2.177	2.553
educ	0.0481	0.006	8.136	0.000	0.034	0.062

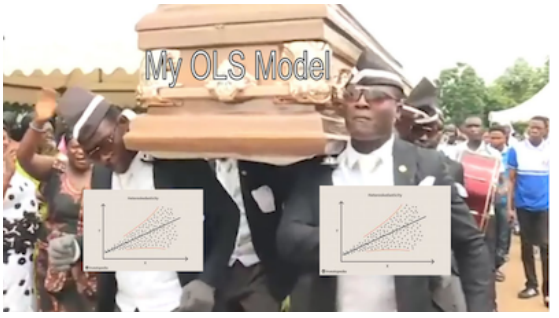
The parameter estimate is smaller. What is happening here is that the regression is placing equal weight for all points. If we plot the model along the grouped points, we see that

the non weighted model is giving more importance to small points in the lower left than it should. As a consequence, the line has a lower slope.

```
sns.scatterplot(x="educ", y = "lhwage", size="count", legend=False,
  ↪ data=group_wage, sizes=(40, 400))
plt.plot(wage["educ"], model_2.predict(wage["educ"]), c="C1",
  ↪ label = "Weighted")
plt.plot(wage["educ"], model_3.predict(wage["educ"]), c="C2",
  ↪ label = "Non Weighted")
plt.xlabel("Years of Education")
plt.ylabel("Log Hourly Wage")
plt.legend();
```



The bottom line is that regression is this marvellous tool that works both with individual or aggregated data, but you have to use weights in this last case. To use weighted regression you need mean statistics. Not sums, not standard deviations, not medians, but means! For both the covariates and the dependent variable. Just keep in mind that the result of weighted regression with grouped data won't match exactly that of regression in ungrouped data, but it will be pretty similar.



I'll finish with a final example using additional covariates in a grouped data model.

```
group_wage = (wage
              .assign(count=1)
              .groupby("educ")
              .agg({"lhwage":"mean", "IQ":"mean", "count":"count"})
              .reset_index())

model_4 = smf.wls('lhwage ~ educ + IQ', data=group_wage,
  ↪ weights=group_wage["count"]).fit()
print("Number of observations:", model_4.nobs)
model_4.summary().tables[1]
```

```
Number of observations: 10.0
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	1.8821	0.324	5.800	0.001	1.115	2.649
educ	0.0257	0.021	1.198	0.270	-0.025	0.077
IQ	0.0077	0.006	1.309	0.232	-0.006	0.022

In this example, we've included IQ as a feature, besides the previously added years of education. The mechanics is pretty much the same: get the means and count, regress the mean and use the count as weights.

Regression for Dummies

Dummy variables are categorical variables we've encoded as binary columns. For example, suppose you have a gender variable that you wish to include in your model. This variable is encoded into 3 categories: male, female and other genders.

gender
male
female
female
other
male

Since our model only accepts numerical values, we need to convert this category to a number. In linear regression, we use dummies for that. We encode each variable as a 0/1 column, denoting the presence of a category. We also leave one of the categories out as the base category. This is necessary since the last category is a linear combination of the others. Put it differently, we can know the last category if someone gives us information on the others. In our example, if someone is neither female nor other genders, we can infer that the person's category is male.

gender	female	other
male	0	0
female	1	0
female	1	0
other	0	1
male	0	0

We've already dealt with a simple form of dummy regression when dealing with A/B testing. More generally, when we are dealing with a binary treatment, we represent it as a dummy variable. In this case, **the regression coefficient for that dummy is the increment for the intercept in the regression line**, or the difference in means between the treated and untreated.

To make this more concrete, consider the problem of estimating the effect of graduating 12th grade on hourly wage (and let's ignore confounding just for now). In the code below, we've created a treatment dummy variable `T` indicating if years of education is greater than 12.

```
wage = (pd.read_csv("./data/wage.csv")
        .assign(hwage=lambda d: d["wage"] / d["hours"])
        .assign(T=lambda d: (d["educ"] > 12).astype(int)))

wage[["hwage", "IQ", "T"]].head()
```

	hwage	IQ	T
0	19.225	93	0
1	16.160	119	1
2	20.625	108	1
3	16.250	96	0
4	14.050	74	0

The dummy works as a kind of switch. In our example, if the dummy is on, the predicted value is the intercept plus the dummy coefficient. If the dummy is off, the predicted value is just the intercept.

```
smf.ols('hwage ~ T', data=wage).fit().summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	19.9405	0.436	45.685	0.000	19.084	20.797
T	4.9044	0.626	7.830	0.000	3.675	6.134

In this case, when the person hasn't completed 12th grade (dummy off), the average income is 19.9. When he or she has completed 12th grade (dummy on), the predicted value

or the average income is 24.8449 (19.9405 + 4.9044). Hence, the dummy coefficient captures the difference in means, which is 4.9044 in our case.

More formally, when the independent variable is binary, as is often the case with treatment indicators, regression captures the ATE perfectly. That is because regression is a linear approximation to the conditional expectation function (CEF) $E[Y|X]$ and, in this particular case, the CEF IS linear. Namely, we can define $E[Y_i|T_i = 0] = \alpha$ and $E[Y_i|T_i = 1] = \alpha + \beta$, which leads to the following CEF

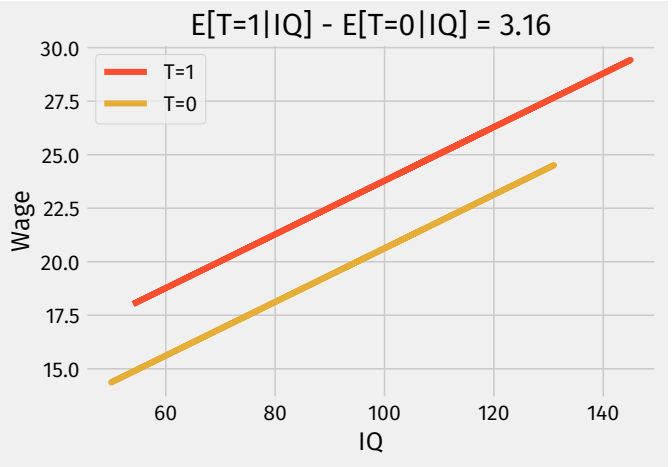
$$E[Y_i|T_i] = E[Y_i|T_i = 0] + \beta T_i = \alpha + \beta T_i$$

and β is the difference in means or the ATE in the case of random data

$$\beta = [Y_i|T_i = 1] - [Y_i|T_i = 0]$$

If we use additional variables, the dummy coefficient becomes the **conditional** difference in means. For instance, let's say we add IQ to the previous model. Now, the dummy coefficient tells us how much increase we should expect from graduating 12th grade **while holding IQ fixed**. If we plot the prediction, we will see two parallel lines. The jump from one line to the next says the amount we should expect for completing 12th grade. They also say that the effect is constant. No matter your IQ, everyone benefits the same from graduating 12th grade.

```
m = smf.ols('hwage ~ T+IQ', data=wage).fit()
plt_df = wage.assign(y_hat = m.fittedvalues)

plt.plot(plt_df.query("T==1")["IQ"], plt_df.query("T==1")["y_hat"], c="C1",
  ↪ label="T=1")
plt.plot(plt_df.query("T==0")["IQ"], plt_df.query("T==0")["y_hat"], c="C2",
  ↪ label="T=0")
plt.title(f"E[T=1|IQ] - E[T=0|IQ] = {round(m.params['T'], 2)}")
plt.ylabel("Wage")
plt.xlabel("IQ")
plt.legend();
```



If we put this model into an equation, we can see why:

$$wage_i = \beta_0 + \beta_1 T_i + \beta_2 IQ_i + e_i$$

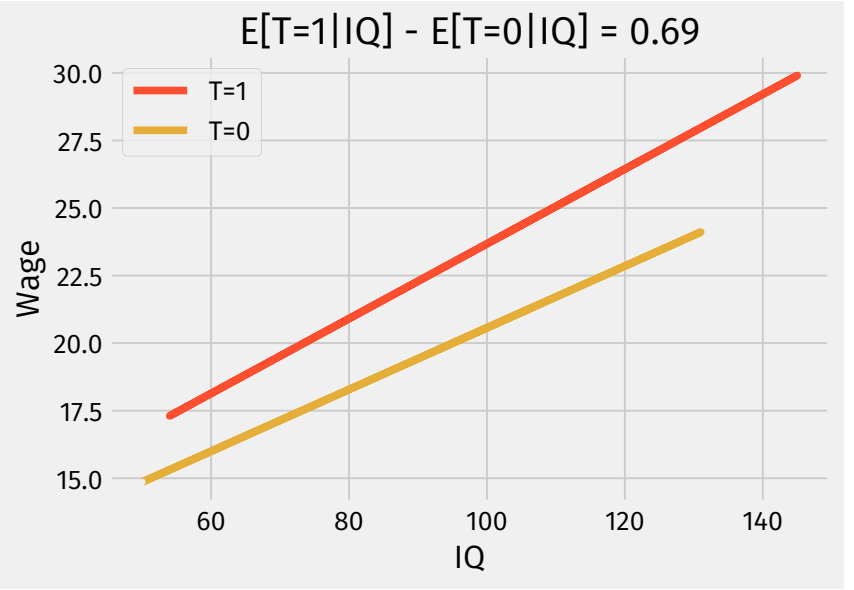
Here, β_1 is the conditional difference in means and it is a constant value, 3.16 in our case. We can make this model more flexible by adding an interaction term.

$$wage_i = \beta_0 + \beta_1 T_i + \beta_2 IQ_i + \beta_3 IQ_i * T_i + e_i$$

Things are getting a little bit more complex, so let's see what each parameter means in this model. First, the intercept β_0. This bad boy doesn't have a particularly interesting interpretation. It's the expected wage when the treatment is zero (the person hasn't graduated from 12th grade) AND the IQ is zero. Since we don't expect IQ to be zero for anyone this parameter is not very meaningful. As for β_1, we have a similar situation. This parameter is how much increase in wage we should expect from completing 12th grade **when IQ is zero**. Once again, since IQ is never zero, it doesn't have a particularly interesting meaning. Now, β_2 is a bit more interesting. It tells us how much IQ increases wages **for the non-treated**. So, in our case, it is something like 0.11. This means that for each 1 extra IQ point, the person that has not completed 12th grade should expect to gain an extra 11 cents per hour. Finally, the most interesting parameter is β_3. It tells us how much IQ increases the effect of graduating 12th grade. In our case, this parameter is 0.024, which means that for each extra IQ point, graduating 12th grade gives 2 extra cents. This might not seem much, but compare someone with 60IQ and with 140IQ. The first one will get an increase of 1.44 in wage (60 * 0.024), while the person with 140 IQ will gain an extra 3.36 dollars (140 * 0.024) when graduating from 12th grade.

In simple modeling jargon, this interaction term allows the treatment effect to change by levels of the features (only IQ, in this example). The result is that if we plot the prediction lines, we will see that they are no longer parallel and that those that graduate 12th grade (T=1) have a higher slope on IQ, higher IQ benefit more from graduating than lower IQ. This is sometimes referenced as effect modification or heterogeneous treatment effect.

```
m = smf.ols('hwage ~ T*IQ', data=wage).fit()
plt_df = wage.assign(y_hat = m.fittedvalues)

plt.plot(plt_df.query("T==1")["IQ"], plt_df.query("T==1")["y_hat"], c="C1",
  ↪ label="T=1")
plt.plot(plt_df.query("T==0")["IQ"], plt_df.query("T==0")["y_hat"], c="C2",
  ↪ label="T=0")
plt.title(f"E[T=1|IQ] - E[T=0|IQ] = {round(m.params['T'], 2)}")
plt.ylabel("Wage")
plt.xlabel("IQ")
plt.legend();
```



Finally, let's look at the case where all the variables in our model are dummies. To do so, we will discretize IQ into 4 bins and treat years of education as a category.

```
wage_ed_bins = (wage
                .assign(IQ_bins = lambda d: pd.qcut(d["IQ"], q=4,
                  ↪ labels=range(4)))
                [["hwage", "educ", "IQ_bins"]])

wage_ed_bins.head()
```

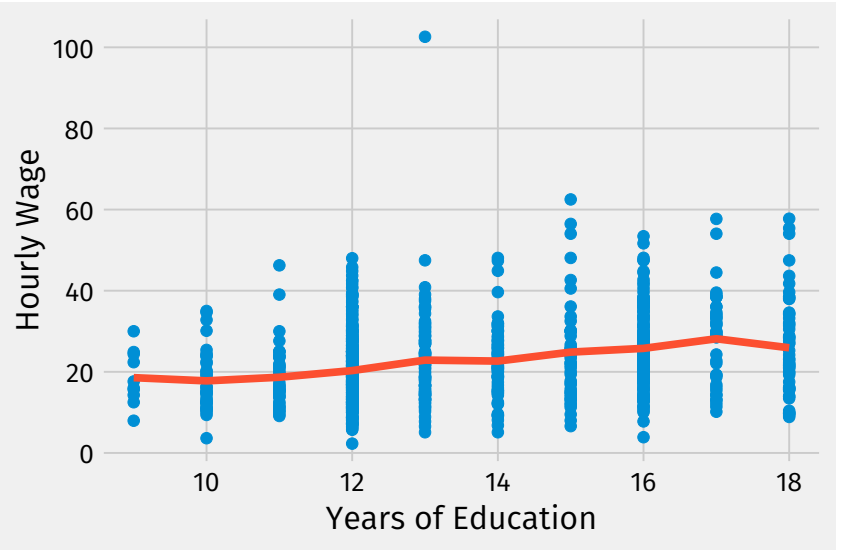
	hwage	educ	IQ_bins
0	19.225	12	1
1	16.160	18	3
2	20.625	14	2
3	16.250	12	1
4	14.050	11	0

Treating education as a category, we no longer restrict the effect of education to a single parameter. Instead, we allow each year of education to have its own distinct impact. By doing so, we gain flexibility, since the effect of education is no longer parametric. This model simply computes the mean wage for each year of education.

```
model_dummy = smf.ols('hwage ~ C(educ)', data=wage).fit()
model_dummy.summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	18.5600	3.011	6.164	0.000	12.651	24.469
C(educ)[T.10]	-0.7874	3.414	-0.231	0.818	-7.488	5.913
C(educ)[T.11]	0.1084	3.343	0.032	0.974	-6.452	6.669
C(educ)[T.12]	1.7479	3.049	0.573	0.567	-4.236	7.732
C(educ)[T.13]	4.3290	3.183	1.360	0.174	-1.918	10.576
C(educ)[T.14]	4.0888	3.200	1.278	0.202	-2.192	10.370
C(educ)[T.15]	6.3013	3.329	1.893	0.059	-0.231	12.834
C(educ)[T.16]	7.2225	3.110	2.323	0.020	1.120	13.325
C(educ)[T.17]	9.5905	3.366	2.849	0.004	2.984	16.197
C(educ)[T.18]	7.3681	3.264	2.257	0.024	0.962	13.775

```
plt.scatter(wage["educ"], wage["hwage"])
plt.plot(wage["educ"].sort_values(), model_dummy.predict(wage["educ"]
  ↪ .sort_values()), c="C1")
plt.xlabel("Years of Education")
plt.ylabel("Hourly Wage");
```



First of all, notice how this removes any assumption about the functional form of how education affects wages. We don't need to worry about logs anymore. In essence, this model is completely non-parametric. All it does is compute sample averages of wage for each year of education. This can be seen in the plot above, where the fitted line doesn't have a particular form. Instead, it is the interpolation of the sample means for each year of education. We can also see that by reconstructing one parameter, for instance, that of 17 years of education. For this model, it's `9.5905`. Below, we can see how it is just the

difference between the baseline years of education (9) and the individuals with 17 years

$$\beta_{17} = E[Y \mid T = 17] - E[Y \mid T = 9]$$

The trade-off is that we lose statistical significance when we allow such flexibility. Notice how big the p-values are for some years.

```
t1 = wage.query("educ==17")["hwage"]
t0 = wage.query("educ==9")["hwage"]
print("E[Y|T=9]:", t0.mean())
print("E[Y|T=17]-E[Y|T=9]:", t1.mean() - t0.mean())
```

```
E[Y|T=9]: 18.56
E[Y|T=17]-E[Y|T=9]: 9.590472362353516
```

If we include more dummy covariates in the model, the parameter on education becomes a weighted average of the effect on each dummy group:

$$E\{ \ (E[Y_i \mid T = 1, Group_i] - E[Y_i \mid T = 0, Group_i])w(Group_i) \ \}$$

$w(Group_i)$ is not exactly, but is proportional to the variance of the treatment in the group $Var(T_i \mid Group_i)$. One natural question that arises from this is why not use the full nonparametric estimator, where the group weight is the sample size? This indeed is a valid estimator, but it is not what regression does. By using the treatment variance, regression is placing more weight on groups where the treatment varies a lot. This makes intuitive sense. If the treatment was almost constant (say 1 treated and everyone else untreated), it doesn't matter its sample size. It wouldn't provide much information about the treatment effect.

```
model_dummy_2 = smf.ols('hwage ~ C(educ) + C(IQ_bins)', data=wage_ed_bins).fit()
model_dummy_2.summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	18.4178	2.991	6.158	0.000	12.548	24.288
C(educ)[T.10]	-1.2149	3.392	-0.358	0.720	-7.872	5.442
C(educ)[T.11]	-0.4687	3.332	-0.141	0.888	-7.008	6.070
C(educ)[T.12]	0.3400	3.059	0.111	0.912	-5.664	6.344
C(educ)[T.13]	2.4103	3.206	0.752	0.452	-3.882	8.702
C(educ)[T.14]	1.8040	3.238	0.557	0.578	-4.551	8.159
C(educ)[T.15]	3.8599	3.369	1.146	0.252	-2.752	10.472
C(educ)[T.16]	4.4060	3.171	1.390	0.165	-1.817	10.629
C(educ)[T.17]	6.7470	3.422	1.971	0.049	0.030	13.464
C(educ)[T.18]	4.3463	3.332	1.304	0.192	-2.194	10.886
C(IQ_bins)[T.1]	1.4216	0.898	1.584	0.114	-0.340	3.183
C(IQ_bins)[T.2]	2.9717	0.930	3.195	0.001	1.146	4.797
C(IQ_bins)[T.3]	3.7879	1.022	3.708	0.000	1.783	5.793

Key Ideas

We started this section by looking at how some data points are more important than others. Namely, those with higher sample size and lower variance should be given more weight when estimating a linear model. Then, we looked at how linear regression can even handle grouped anonymised data with elegance, provided we use sample weights in our model.

Next, we moved to dummy regression. We saw how it can be made a non parametric model that places no assumptions whatsoever on the functional form of how the treatment impacts the outcome. We then explored the intuition behind dummy regression.

References

I like to think of this entire book as a tribute to Joshua Angrist, Alberto Abadie and Christopher Walters for their amazing Econometrics class. Most of the ideas here are taken from their classes at the American Economic Association. Watching them is what is keeping me sane during this tough year of 2020.

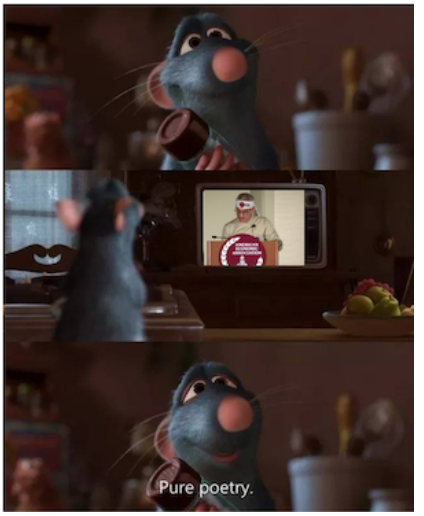
- Cross-Section Econometrics · https://www.aeaweb.org/conference/cont-ed/2017-webcasts
- Mastering Mostly Harmless Econometrics · https://www.aeaweb.org/conference/cont-ed/2020-webcasts

I'll also like to reference the amazing books from Angrist. They have shown me that Econometrics, or 'Metrics as they call it, is not only extremely useful but also profoundly fun.

- Mostly Harmless Econometrics · https://www.mostlyharmlesseconometrics.com/
- Mastering 'Metrics · https://www.masteringmetrics.com/

My final reference is Miguel Hernan and Jamie Robins' book. It has been my trustworthy companion in the most thorny causal questions I had to answer.

- Causal Inference Book · https://miguelhernan.org/whatifbook



CHAPTER 7

Beyond Confounders

Good Controls

We've seen how adding additional controls to our regression model can help identify causal effect. If the control is a confounder, adding it to the model is not just nice to have, but is a requirement. When the unwary see this, a natural response is to throw whatever he can measure into the model. In today's world of big data, this could easily be more than 1000 variables. As it turns out, this is not only unnecessary, but can be detrimental to causal identification. We will now turn our attention to controls that are not confounders. First, let's take a look at the good ones. Then, we will delve into harmful controls.

As a motivating example, let's suppose you are a data scientist in the collections team of a fintech. Your next task is to figure out the impact of sending an email asking people to negotiate their debt. Your response variable is the amount of payments from the late customers.

To answer this question, your team selects 5000 random customers from your late customers base to do a random test. For every customer, you flip a coin, if it's heads, the customer receives the email; otherwise, it is left as a control. With this test, you hope to find out how much extra money the email generates.

```
import warnings
warnings.filterwarnings('ignore')

import pandas as pd
import numpy as np
from scipy import stats
from matplotlib import style
import seaborn as sns
from matplotlib import pyplot as plt
import statsmodels.formula.api as smf
import graphviz as gr

style.use("fivethirtyeight")
```

```
data = pd.read_csv("./data/collections_email.csv")
data.head()
```

	payments	email	opened	agreement	credit_limit	risk_score
0	740	1	1.0	0.0	2348.495260	0.666752
1	580	1	1.0	1.0	334.111969	0.207395
2	600	1	1.0	1.0	1360.660722	0.550479
3	770	0	0.0	0.0	1531.828576	0.560488
4	660	0	0.0	0.0	979.855647	0.455140

Since the data is random, you know that a simple difference in means estimates the Average Treatment Effect. In other words, nothing can have caused the treatment but the randomisation, so the potential outcomes are independent of the treatment: $(Y_0, Y_1) \perp T$.

$$ATE = E[Y \mid T = 1] - E[Y \mid T = 0]$$

Since you are smart and want to place a confidence interval around your estimate, you use a linear regression.

```
print("Difference in means:",
      data.query("email==1")["payments"].mean() -
        ↪ data.query("email==0")["payments"].mean())

model = smf.ols('payments ~ email', data=data).fit()
model.summary().tables[1]
```

```
Difference in means: -0.6202804021329484
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	669.9764	2.061	325.116	0.000	665.937	674.016
email	-0.6203	2.941	-0.211	0.833	-6.387	5.146

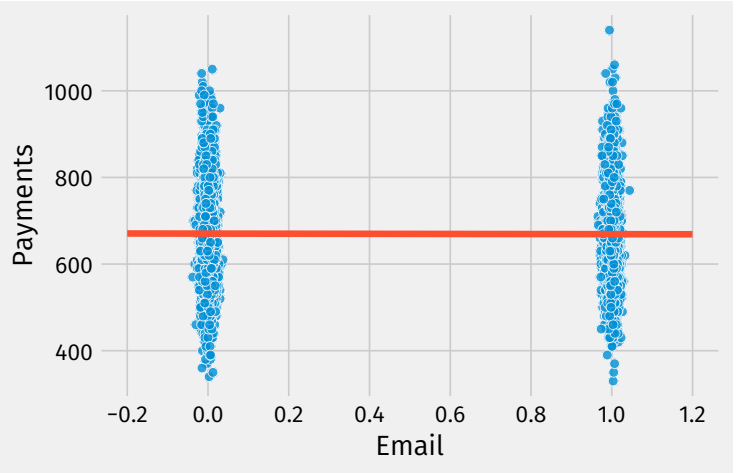
Sadly, the estimated ATE is -0.62, which is pretty weird. How can sending an email make late customers pay less than average? Still, the P-value is so high that this probably doesn't mean anything. What you should do now? Go back to your team with a tail between your legs and say that the test is inconclusive and you need more data? Not so fast.

Notice how your data has some other interesting columns. For example, `credit_limit` represents the customer's credit line prior to he or she getting late. `risk_score` corresponds to the estimated risk of the customer prior to the delivery of the email. It makes sense to think that credit limit and risk are probably very good predictors of payments. But how can that be useful?

First, let's understand why we can fail to find statistical significance in a treatment even when it is there. It could be that, like in this case, the treatment has very little impact on the outcome. If you think about it, what makes people pay their debt is, by and large, factors outside the control of the collections department. People pay their debt because they find a new job, manage their finances, income and so on. In statistical terms, we can say that **the variability of payments is explained much more by other factors other than by the email**.

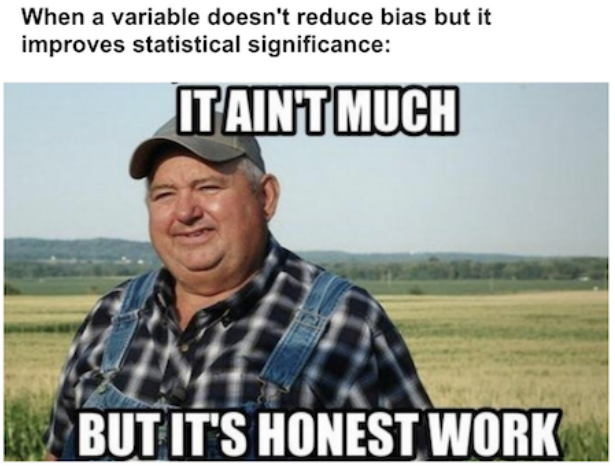
To get a visual understanding of it, we can plot the payments against the treatment variable email. I've also plotted the fitted line of the model above in red. To help visualization, I've added a little bit of noise to the email variable so that it doesn't get smashed at the zero or one.

```
sns.scatterplot(x="email", y="payments",
                alpha=0.8,
                data=data.assign(email=data["email"] + np.random.normal(0, 0.01,
                  ↪ size=len(data["email"]))))
plt.plot(np.linspace(-0.2, 1.2), model.params[0] + np.linspace(-1,
  ↪ 2) * model.params[1], c="C1")
plt.xlabel("Email")
plt.ylabel("Payments");
```



We can see how wildly payments vary in a single treatment group. Visually, it looks like it is going from a little bit under 400 to 1000 in both groups. If the impact of the email is in the order of say 5.00 or 10.00 R$, it is no wonder it will be hard to find it inside all the variability.

Fortunately, regression can help us lower this variability. The trick is to use additional controls. **If a variable is a good predictor of the outcome, it will explain away a lot of its variance**. If risk and credit limit are good predictors of payment, we can control them to make it easier to find the impact of the email on payments. If we remember how regression works, this has an intuitive explanation. Adding extra variables to a regression means keeping them constant while looking at the treatment. So, the reasoning goes, if we look at similar levels of risk and credit limit, the variance of the response variable `payments` should be smaller. Or, in other words, if risk and credit line predicts payments very well, customers with a similar risk and credit line should also have similar payment levels, hence with less variance.



To demonstrate this, let's resort to the partialling out way of breaking regression into 2 steps. First, we will regress the treatment, email, and the outcome, payments, on the additional controls, credit limit and risk score. Second, we will regress the residual of the treatment on the residuals of payments, both obtained in step 1. (This is purely pedagogical, in practice you won't need to go through all the hassle).

```
model_email = smf.ols('email ~ credit_limit + risk_score', data=data).fit()
model_payments = smf.ols('payments ~ credit_limit + risk_score',
  ↪ data=data).fit()

residuals = pd.DataFrame(dict(res_payments=model_payments.resid,
  ↪ res_email=model_email.resid))

model_treatment = smf.ols('res_payments ~ res_email', data=residuals).fit()
```

This lowers the variance of the dependent variable. By regressing payments on credit limit and risk and obtaining the residuals for this model, we are creating a new dependent variable with much less variability than the original one. The last model also uncovers the ATE with valid standard error estimate.

Just out of curiosity, we can also check that the model that predicts the treatment should not be able to lower the variance of it. That's because email is, by design, random, so nothing can predict it.

```
print("Payments Variance", np.var(data["payments"]))
print("Payments Residual Variance", np.var(residuals["res_payments"]))

print("Email Variance", np.var(data["email"]))
print("Email Residual Variance", np.var(residuals["res_email"]))

model_treatment.summary().tables[1]
```

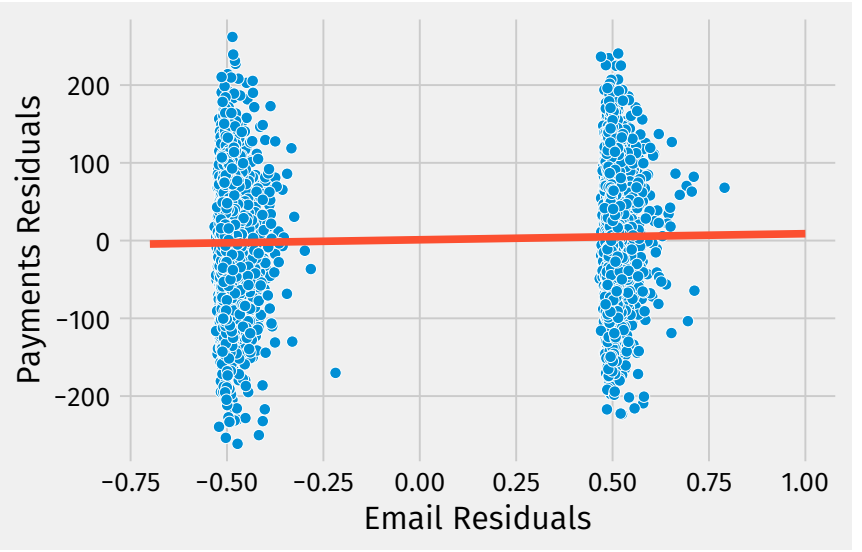
```
Payments Variance 10807.612416
Payments Residual Variance 5652.453558466207
Email Variance 0.24991536
Email Residual Variance 0.24918421069820032
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	-3.71e-13	1.063	-3.49e-13	1.000	-2.084	2.084
res_email	4.4304	2.129	2.080	0.038	0.256	8.605

Notice how the variance of payments went from 10807 to 5652. We've decreased it by almost half once we control for risk and credit limits. Also notice that we didn't manage to reduce the variability of the treatment email. This makes sense, since risk and credit line does not predict email (nothing does, by definition of randomness).

Now, we see something much more reasonable. This new estimate tells us that we should expect customers that received the email to pay, on average, 4.4 reais more than those in the control group. This estimate is now statistically different from zero. We can also visualize how the variance is now lower within each control group.

```
sns.scatterplot(x="res_email", y="res_payments", data=residuals)
plt.plot(np.linspace(-0.7, 1), model_treatment.params[0] + np.linspace(-1,
  ↪ 2) * model_treatment.params[1], c="C1")
plt.xlabel("Email Residuals")
plt.ylabel("Payments Residuals");
```



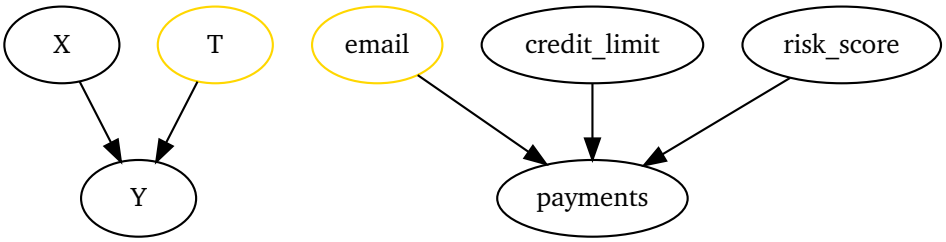
As I've said, we did this for pedagogical reasons. In practice, you can simply add the controls to the regression model together with the treatment and the estimates will be exactly the same.

```
model_2 = smf.ols('payments ~ email + credit_limit + risk_score',
  ↪ data=data).fit()
model_2.summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	490.8653	9.715	50.527	0.000	471.820	509.911
email	4.4304	2.130	2.080	0.038	0.255	8.606
credit_limit	0.1511	0.008	18.833	0.000	0.135	0.167
risk_score	-8.0516	38.424	-0.210	0.834	-83.379	67.276

To wrap it up, anytime we have a control that is a good predictor of the outcome, even if it is not a confounder, adding it to our model is a good idea. It helps lowering the variance of our treatment effect estimates. Here is a picture of what this situation looks like with causal graphs.

```
g = gr.Digraph()
g.edge("X", "Y"), g.edge("T", "Y")
g.node("T", color="gold")

g.node("email", color="gold")
g.edge("credit_limit", "payments")
g.edge("risk_score", "payments")
g.edge("email", "payments")

g
```



Mostly Harmful Controls

As a second motivating example, let's consider a drug test scenario with 2 hospitals. Both of them are conducting randomised trials on a new drug to treat a certain illness. The outcome of interest is days hospitalised. If the treatment is effective, it will lower the amount of days the patient stays in the hospital. For one of the hospitals, the policy regarding the random treatment is to give it to 90% of its patients while 10% get a placebo. The other hospital has a different policy: it gives the drug to a random 10% of its patients and 90% get a placebo. You are also told that the hospital that gives 90% of the true drug and 10% of placebo usually gets more severe cases of the illness to treat.

```
hospital = pd.read_csv("./data/hospital_treatment.csv")
hospital.head()
```

	hospital	treatment	severity	days
0	1	1	29.686618	82
1	1	1	20.050340	57
2	1	1	20.302399	49
3	0	0	10.603118	44
4	0	0	8.332793	15

Since you are dealing with randomized data, your first instinct is to simply run a regression of the outcome on the treatment.

```
hosp_1 = smf.ols('days ~ treatment', data=hospital).fit()
hosp_1.summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	33.2667	2.662	12.498	0.000	27.968	38.566
treatment	14.1533	3.367	4.204	0.000	7.451	20.856

But you find some counterintuitive results. How can the treatment be increasing the number of days in the hospital? The answer lies in the fact that we are running 2 different experiments. Severity is positively linked with more days at the hospital and since the hospital with more severe cases also gives more of the drug, the drug becomes positively correlated with more days at the hospital. When we look at both hospitals together, we have that $E[Y_0 \mid T = 0] < E[Y_0 \mid T = 1]$, that is, the potential outcome of the untreated is, on average, lower than that of the treated because there are more untreated in the hospital with less severe cases. In other words, severity acts as a confounder, determining the hospital the patient goes and, hence, the probability of receiving the drug.

There are 2 ways of fixing that. The first one, which defeats the purpose of using data from both hospitals, is to simply look at the ATE in each hospital individually.

```
hosp_2 = smf.ols('days ~ treatment', data=hospital.query("hospital==0")).fit()
hosp_2.summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	30.4074	2.868	10.602	0.000	24.523	36.292
treatment	-11.4074	10.921	-1.045	0.306	-33.816	11.001

```
hosp_3 = smf.ols('days ~ treatment', data=hospital.query("hospital==1")).fit()
hosp_3.summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	59.0000	6.747	8.745	0.000	45.442	72.558
treatment	-10.3958	6.955	-1.495	0.141	-24.371	3.580

In this case, we did get an intuitive result of the ATE. It looks like now the drug is in fact lowering the amount of days at the hospital. However, since we are looking at each hospital individually, there are not enough data points. As a consequence, we are unable to find statistically significant results.

The other approach, which leverages the power of regression, is to control for severity by including it in the model.

```
hosp_4 = smf.ols('days ~ treatment + severity', data=hospital).fit()
hosp_4.summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	11.6641	2.000	5.832	0.000	7.681	15.647
treatment	-7.5912	2.269	-3.345	0.001	-12.110	-3.073
severity	2.2741	0.154	14.793	0.000	1.968	2.580

The question that arises next is, should we also include hospital in the model? After all, we know that hospitals cause the treatment right? Well, that is true, but once we've controlled for severity, hospital is no longer correlated with the outcome number of days hospitalised. And we know that to be a confounder a variable has to cause both the treatment and the outcome. In this case, we have a variable that only causes the treatment.

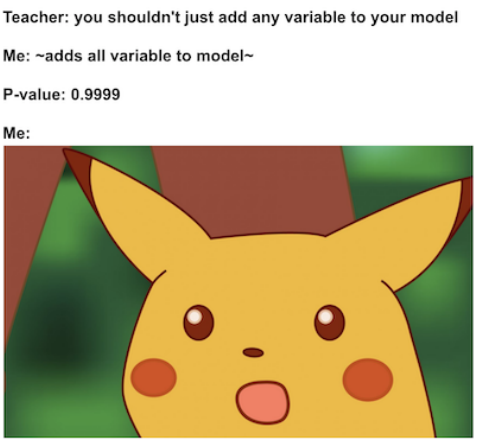
But maybe controlling for it lowers the variance, right? Well, not true again. In order for a control to lower the variance, it has to be a good predictor of the outcome, not of the treatment, which is the case here.

Still, we might want to control it right? It can't hurt… Or can it?

```
hosp_5 = smf.ols('days ~ treatment + severity + hospital', data=hospital).fit()
hosp_5.summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	11.0111	2.118	5.198	0.000	6.792	15.230
treatment	-5.0945	3.492	-1.459	0.149	-12.049	1.861
severity	2.3865	0.195	12.251	0.000	1.999	2.774
hospital	-4.1535	4.413	-0.941	0.350	-12.943	4.636

Surprisingly, it can hurt!



Adding hospital on top of severity as a control introduced MORE variance to our ATE estimator. How can that be? The answer lies in the formula for the standard error of the regression coefficient.

$$\hat{\sigma}^2 = \frac{1}{n-2}\sum(y_i - \hat{y}_i)^2$$

$$\text{Var}(\hat{\beta}_2) = \frac{\sigma^2}{\sum(x_i - \bar{x})^2}$$

From this formula, we can see that the standard error is inversely proportional to the variance of the variable X. This means that, if X doesn't change much, it will be hard to estimate its effect on the outcome. This also makes intuitive sense. Take it to the extreme and pretend you want to estimate the effect of a drug, so you conduct a test with 10000 individuals but only 1 of them get the treatment. This will make finding the ATE very hard, we will have to rely on comparing a single individual with everyone else. Another way to say this is that we need lots of variability in the treatment to make it easier to find its impact.

As to why including hospitals in the model increases the error of our estimate, it is because it is a good predictor of the treatment and not of the outcome (once we control for severity). So, by predicting the treatment, it effectively makes it so that it's variance is lower! Once again, we can resort to partitioning our regression above into it's 2 steps to see this.

```
model_treatment = smf.ols('treatment ~ severity + hospital',
  ↪ data=hospital).fit()
model_days = smf.ols('days ~ severity + hospital', data=hospital).fit()

residuals = pd.DataFrame(dict(res_days=model_days.resid,
  ↪ res_treatment=model_treatment.resid))

model_treatment = smf.ols('res_days ~ res_treatment', data=residuals).fit()

model_treatment.summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	2.817e-14	0.827	3.41e-14	1.000	-1.646	1.646
res_treatment	-5.0945	3.447	-1.478	0.143	-11.957	1.768

```
print("Treatment Variance", np.var(hospital["treatment"]))
print("Treatment Residual Variance", np.var(residuals["res_treatment"]))
```

```
Treatment Variance 0.234375
Treatment Residual Variance 0.05752909187211906
```

Also, don't take my word for it! You can check that the SE formula above is true:

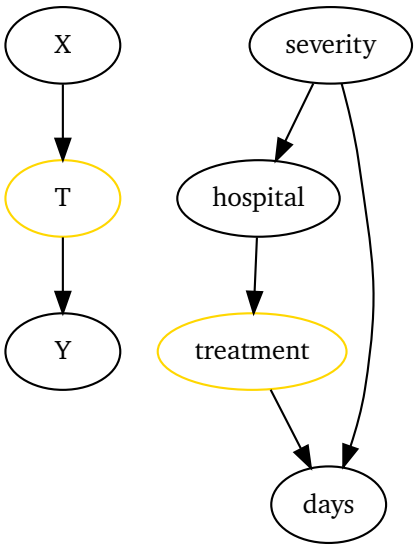
```
sigma_hat = sum(model_treatment.resid**2)/(len(model_treatment.resid)-2)
var = sigma_hat/sum((residuals["res_treatment"] - residuals["res_treatment"]
  ↪ .mean())**2)
print("SE of the Coeficient:", np.sqrt(var))
```

```
SE of the Coeficient: 3.446973767486901
```

So the bottom line is that we should add controls that are both correlated with the treatment and the outcome (confounder), like the severity in the model above. We should also add controls that are good predictors of the outcome, even if they are not confounders, because they lower the variance of our estimates. However, we should **NOT** add controls that are just good predictors of the treatment, because they will increase the variance of our estimates.

Here is a picture of what this situation looks like with causal graphs.

```
g = gr.Digraph()

g.edge("X", "T"), g.edge("T", "Y")
g.node("T", color="gold")

g.node("treatment", color="gold")
g.edge("severity", "hospital")
g.edge("severity", "days")
g.edge("hospital", "treatment")
g.edge("treatment", "days")

g
```



Bad Controls - Selection Bias

Let's go back to the collections email example. Remember that the email was randomly assigned to customers. We've already explained what `credit_limit` and `risk_score` are. Now, let's look at the remaining variables. opened is a dummy variable for the customer opening the email or not. `agreement` is another dummy marking if the customers contacted the collections department to negotiate their debt, after having received the email. Which of the following models do you think is more appropriate? The first is a model with the treatment variable plus `credit_limit` and `risk_score`; the second adds opened and `agreement` dummies.

```
email_1 = smf.ols('payments ~ email + credit_limit + risk_score',
  ↪ data=data).fit()
email_1.summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	490.8653	9.715	50.527	0.000	471.820	509.911
email	4.4304	2.130	2.080	0.038	0.255	8.606
credit_limit	0.1511	0.008	18.833	0.000	0.135	0.167
risk_score	-8.0516	38.424	-0.210	0.834	-83.379	67.276

```
email_2 = smf.ols('payments ~ email + credit_limit + risk_score + opened +
      ↪ agreement', data=data).fit()
email_2.summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	488.4416	9.716	50.272	0.000	469.394	507.489
email	-1.6095	2.724	-0.591	0.555	-6.949	3.730
credit_limit	0.1507	0.008	18.809	0.000	0.135	0.166
risk_score	-2.0929	38.375	-0.055	0.957	-77.325	73.139
opened	3.9808	3.914	1.017	0.309	-3.692	11.654
agreement	11.7093	4.166	2.811	0.005	3.542	19.876

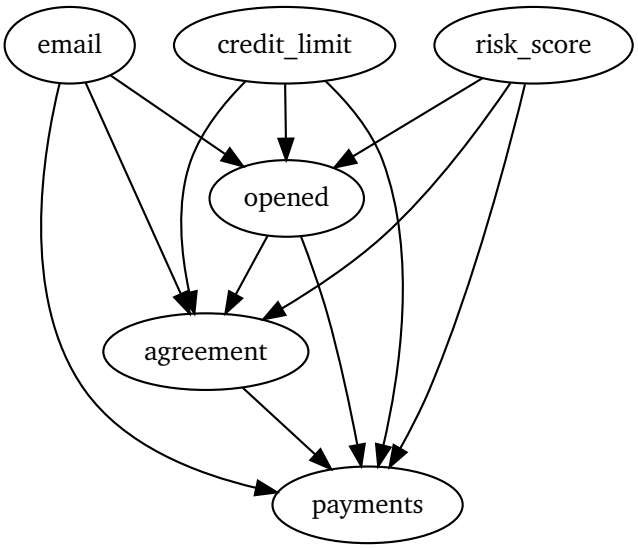
While the first model finds statistically significant results for the email, the second one does not. But maybe the second one is the right model and there's no effect for the email. After all, this model controls for more factors, so it should be more robust right? By now you probably know that this is not the case. What is left is to figure out what is.

We know that we MUST add confounding variables. Variables that cause both the treatment and the outcome. We also know that it is a good idea to add controls that predict the outcome very well. This is not required, but it's nice to have. We also know that it is a bad idea to add controls that predict only the treatment. Again, this is not a deadly sin, but is nice to avoid. So what kind of controls are opened and `agreement`? Turns out, they are neither of the above.

If you think about it, opened and agreement are surely correlated with the email. After all, you can't open the email if you didn't receive it and we've also said that the agreement only considers renegotiation that happened after the email has been sent. But **they don't cause email! Instead, they are caused by it!**

Whenever I need to understand what kind of variables I'm dealing with, I always like to think about their causal graph. Let's do these here

```
g = gr.Digraph()

g.edge("email", "payments")
g.edge("email", "opened")
g.edge("email", "agreement")
g.edge("opened", "payments")
g.edge("opened", "agreement")
g.edge("agreement", "payments")

g.edge("credit_limit", "payments")
g.edge("credit_limit", "opened")
g.edge("credit_limit", "agreement")
g.edge("risk_score", "payments")
g.edge("risk_score", "opened")
g.edge("risk_score", "agreement")

g
```



We know nothing causes email, because it's random by design. And we know (or at least we have strong reasons to believe) that credit limit and risk cause payments. We also think that email causes payments. As for opened, we think that it does cause payments. Intuitively, people that opened the collection email are more willing to negotiate and

pay their debt. We also think that opened causes agreements for the same reasons as it causes payments. Moreover, we know opened is caused by email and we have reasons to believe people with different risk and credit limits have different open rates for the emails, so credit limit and risk also causes opened. As for agreement, we also think that it is caused by opened. If we think about the payments response variable, we can think of is as the result of a funnel:

$$email -> opened -> agreement -> payment$$

We also think that different levels of risk and limit have different propensity of doing an agreement, so we will mark them as also causing agreement. As for email and agreement, we could make an argument that some people just read the subject of the email and that makes them more likely to make an agreement. The point is that email could also cause agreement without passing through open.

What we notice with this graph is that opened and agreement are both in the causal path from email to payments. So, if we control for them with regression, we would be saying "this is the effect of email while keeping opened and `agreement` fixed". However, both are part of the causal effect of the email, so we don't want to hold them fixed. Instead, we could argue that email increases payments precisely because it boosts the agreement rate. If we fix those variables, we are removing some of the true effect from the email variable.

With potential outcomes notation, we can say that, due to randomization $E[Y_0 | T = 0] = E[Y_0 | T = 1]$. However, even with randomization, when we control for agreement, treatment and control are no longer comparable. In fact, with some intuitive thinking, we can even guess how they are different:

$$E[Y_0 | T = 0, Agreement = 0] > E[Y_0 | T = 1, Agreement = 0]$$

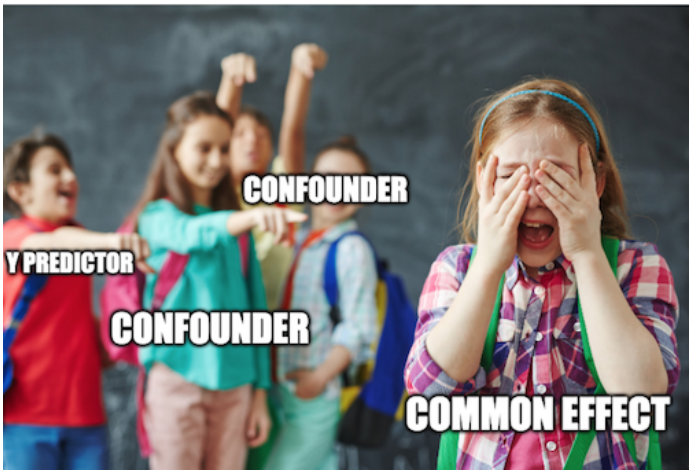
$$E[Y_0 | T = 0, Agreement = 1] > E[Y_0 | T = 1, Agreement = 1]$$

The first equation makes it explicit that we think those without the email and the agreement are better than those with the email and without the agreement. That is because, if the treatment has a positive effect, those that didn't make an agreement **even after having received the email** are probably worse in terms of payments compared to those that also didn't do the agreement but also didn't get the extra incentive of the email. As for the second equation, those that did the agreement even without having received the treatment are probably better than those that did the agreement but had the extra incentive of the email.

This might be very confusing the first time you read it (it was for me), but make sure you understand it. Read it again if necessary. Then, a similar kind of reasoning can be done with the opened variable. Try to make it yourself.

This sort of bias is so pervasive it has its own name. While confounding is the bias from failing to control for a common cause, **selection bias is when we control for a common effect or a variable in between the path from cause to effect.** As a rule of thumb, always include confounders and variables that are good predictors of Y in your model.

Always exclude variables that are good predictors of only T, mediators between the treatment and outcome or common effect of the treatment and outcome.



Selection bias is so pervasive that not even randomization can fix it. Better yet, it is often introduced by the ill advised, even in random data! Spotting and avoiding selection bias requires more practice than skill. Often, they appear underneath some supposedly clever idea, making it even harder to uncover. Here are some examples of selection biases I've encountered:

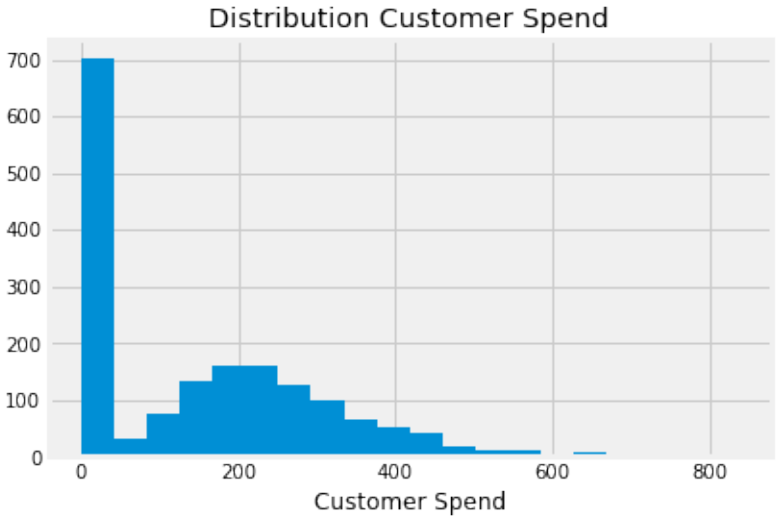
1. Adding a dummy for paying the entire debt when trying to estimate the effect of a collections strategy on payments.
2. Controlling for white vs blue collar jobs when trying to estimate the effect of schooling on earnings
3. Controlling for conversion when estimating the impact of interest rates on loan duration
4. Controlling for marital happiness when estimating the impact of children on extra-marital affairs
5. Breaking up payments modeling $E[Payments]$ into one binary model that predicts if payment will happen and another model that predict how much payment will happen given that some will: $E[Payments|Payments > 0] * P(Payments > 0)$

What is notable about all these ideas is how reasonable they sound. Selection bias often does. Let this be a warning. As a matter of fact, I myself have fallen into the traps above many many times before I learned how bad they were. One in particular, the last one, deserves further explanation because it looks so clever and catches lots of data scientists off guard. It's so pervasive that it has its own name: **The Bad COP**!

Bad COP

The situation goes like this. You have a continuous variable that you want to predict but its distribution is overrepresented at zero. For instance, if you want to model consumer spending, you will have something like a gamma distribution, but with lots of zeros.

```
plt.hist(np.concatenate([
    np.random.gamma(5, 50, 1000),
    np.zeros(700)
]), bins=20)
plt.xlabel("Customer Spend")
plt.title("Distribution Customer Spend");
```



When a data scientist sees this, the first idea that pops into his head is to break up modeling into 2 steps. The first is the participation, that is, the probability that $Y > 0$. In our spend example, this would be modeling if the customer decided to spend or not. The second part models Y for those that decided to participate. It is the Conditional-on-Positives effect. In our case, this would be how much the customer spends after he or she decided they would spend anything. If we would like to estimate the effect of the treatment T on expenditures, it would look something this:

$$E[Y|T] = E[Y|Y > 0, T]P(Y > 0|T)$$

There is nothing wrong with the participation model $P(Y_i > 0|T_i)$. In fact, if T is randomly assigned, it will capture the increase in probability of spending due to the treatment. Also, there is nothing wrong with the decomposition above. It's mathematically true, by the law of total probabilities.

The issue is in estimating the COP part. **It will be biased even under random assignment**. On an intuitive level, it's not crazy to think that some units are zeros only because they didn't get the treatment. The treatment would cause them to **not** be zeros. On the other hand, some units are never zeros. The treatment might increase their outcome, but even without it, they would not be zeros. Now, the key point is understanding that these two types of units are not comparable. Those that are never zeros have high Y_0 compared to those that are zeros if they didn't get the treatment. In fact, for the latter, $Y_0 = 0$.

Knowing this, if we remove the zeros, we will keep the never zeros on both the treated and the control group. But we would remove from the control those that flip from zero to not zero with the treatment. This would make the treatment and control no longer comparable, because the control would only contain those that are never zeros and that have higher Y_0, while the treated would contain both types of units.

Now that we have an intuitive understanding of the problem, let's check it from a mathematical angle. To see it, let's break up the treatment effect. Under random assignment, it is equal to the difference in means

$$
\begin{aligned}
&E[Y \mid T=1] - E[Y \mid T=0] \\
&= E[Y \mid Y>0, T=1]P(Y>0 \mid T=1) - E[Y \mid Y>0, T=0]P(Y>0 \mid T=0) \\
&= \{\underbrace{P(Y>0 \mid T=1) - P(Y>0 \mid T=0)}_{Participation\ Effect}\} * E[Y \mid Y>0, T=1] \\
&+ \{\underbrace{E[Y \mid Y>0, T=1] - E[Y \mid Y>0, T=0]}_{COP\ Effect}\} * P(Y>0 \mid T=0)
\end{aligned}
$$

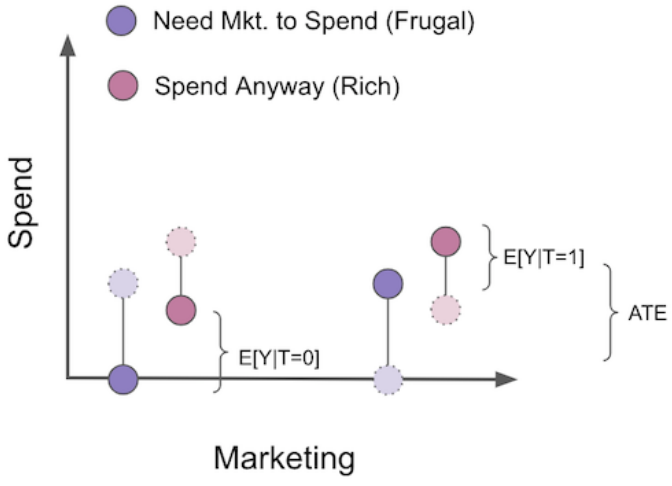
Where the last equality comes from adding and subtracting $E[Y_i \mid Y_i > 0, T_i = 1]P(Y_i > 0 \mid T_i = 0)$ and rearranging the terms. This means that the difference in averages is composed of two parts: first, it's the difference in the probability that the outcome y is positive. This is called the participation effect because it measures the increase in the probability that customers will participate in spending. Second, it's the difference in the outcome conditioned on participation, the COP effect. So far so good. There is nothing wrong with this. It is a mathematical truth. The problem comes when we try to estimate each part separately

This becomes more evident if we analyse the COP effect even further.

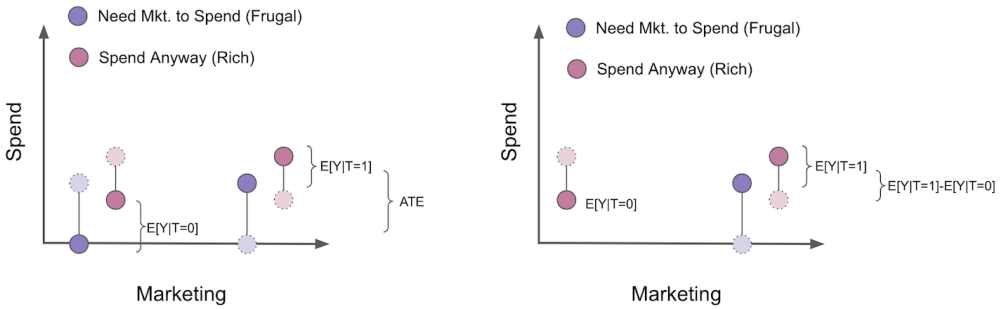
$$
\begin{aligned}
E[Y \mid Y>0, T=1] - E[Y \mid Y>0, T=0] &= E[Y_1 \mid Y_1 > 0] - E[Y_0 \mid Y_0 > 0] \\
&= \underbrace{E[Y_1 - Y_0 \mid Y_1 > 0]}_{Causal\ Effect} + \{\underbrace{E[Y_0 \mid Y_1 > 0] - E[Y_0 \mid Y_0 > 0]}_{Selection\ Bias}\}
\end{aligned}
$$

where the second equality comes after we add and subtract $E[Y_{i0} \mid Y_{i1} > 0]$. When we break up the COP effect, we get first the causal effect on the participant subpopulation. In our example, this would be the causal effect on those that decide to spend something. Second, we get a bias term which is the difference in Y_0 for those that decide to participate when assigned to the treatment ($E[Y_{i0} \mid Y_{i1} > 0]$) and those that would participate even without the treatment ($E[Y_{i0} \mid Y_{i0} > 0]$). In our case, this bias is probably negative, since those that spend when assigned to the treatment, had they not received the treatment, would probably spend less than those that spend even without the treatment $E[Y_{i0} \mid Y_{i1} > 0] < E[Y_{i0} \mid Y_{i0} > 0]$.

Now, I know that COP bias is super counterintuitive at first, so I think it is worth going through a visual example. Let's say that we want to estimate how a marketing campaign increases how much people spend on our product. This marketing campaign has been randomized, so we don't need to worry about confounding. In this example, we can break up the customers into two segments. First, there are those that will only buy our products if they see a marketing campaign. Let's call these customers the frugal ones. They don't spend unless we give them an extra push. Then there are the customers that will spend even without the campaign. The campaign makes them spend more, but they would already spend without seeing it anyway. Let's call them the rich customers. In the figure, I've displayed the counterfactuals in light colors and dashed lines.



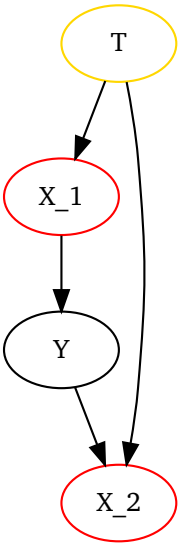
To estimate the ATE of the campaign, since we have randomization, all we need to do is compare the treated with the untreated. But, suppose we use the COP formulation where we break up estimation into two models, a participation model that estimates $P(Y_i > 0|T_i)$ and the COP, which estimates $E[Y_i|Y_i > 0]$. This removes everyone that didn't spend from the analysis.



When we do that, the treated and control are no longer comparable. As we can see, the untreated is now only composed of the segment of customers that will spend even without the campaign. Also notice that we can even know the direction of the bias here. It will be $E[Y_{i0}|Y_{i1} > 0] - E[Y_{i0}|Y_{i0} > 0]$ or $E[Y_{i0}|\text{Frugal and Rich}] - E[Y_{i0}|Rich]$. This is obviously negative, as the rich spend more than the frugal customers. As a result, once we filter only the participant population, our estimate of the ATE becomes biased, even if there was no bias at first due to randomization. I sincerely hope this convinces you to avoid COP like the plague. I see too many Data Scientists doing this separate estimation, unaware of the problems that it brings.

To wrap up selection bias, we need to always remind ourselves to never control for a variable that is either in between the treatment and the outcome or is a common effect of the outcome and the treated. In graphical language, here is what bad control looks like:

```
g = gr.Digraph()

g.edge("T", "X_1"), g.node("T", color="gold"), g.edge("X_1", "Y"), g.node("X_1",
  ↪ color="red")
g.edge("T", "X_2"), g.edge("Y", "X_2"), g.node("X_2", color="red")

g
```



Key Ideas

In this section, we've looked at variables that are not confounders and if we should add them or not to our model for causal identification. We've seen that variables that are good predictors of the outcome y should be added to the model even if they don't predict T (are not confounders). This is because predicting Y lowers its variance and makes it more likely that we will see statistically significant results when estimating the causal effect. Next, we've seen that it is a bad idea to add variables that predict the treatment but not the outcome. Those variables reduce the variability of the treatment, making it harder for us to find the causal effect. Finally, we've looked at selection bias. This is bias that arises when we control for variables in the causal path from the treatment to the outcome or variables that are common effects of the treatment and the outcome.

References

I like to think of this entire book as a tribute to Joshua Angrist, Alberto Abadie and Christopher Walters for their amazing Econometrics class. Most of the ideas here are taken from their classes at the American Economic Association. Watching them is what is keeping me sane during this tough year of 2020.

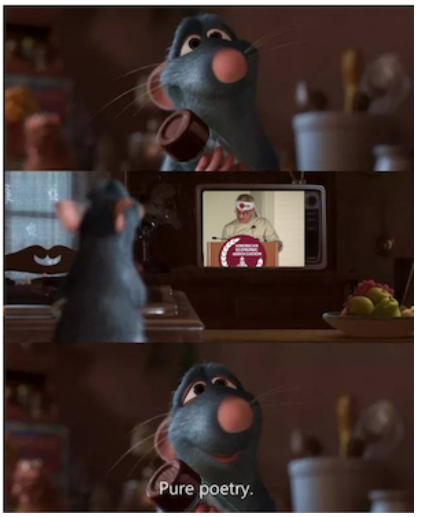
- Cross-Section Econometrics · https://www.aeaweb.org/conference/cont-ed/2017-webcasts
- Mastering Mostly Harmless Econometrics · https://www.aeaweb.org/conference/cont-ed/2020-webcasts

I'll also like to reference the amazing books from Angrist. They have shown me that Econometrics, or 'Metrics as they call it, is not only extremely useful but also profoundly fun.

- Mostly Harmless Econometrics · https://www.mostlyharmlesseconometrics.com/
- Mastering 'Metrics · https://www.masteringmetrics.com/

My final reference is Miguel Hernan and Jamie Robins' book. It has been my trustworthy companion in the most thorny causal questions I had to answer.

- Causal Inference Book · https://miguelhernan.org/whatifbook



CHAPTER

8

Instrumental Variables

Going Around Omitted Variable Bias

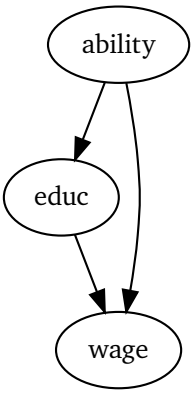
One way to control for OVB is, well, adding the omitted variable into our model. However, that is not always possible, mostly because we simply don't have data on the omitted variables. For instance, let's go back to our model for effect of education on wage:

$$\log(\text{wage})_i = \beta_0 + \kappa\ \text{educ}_i + \boldsymbol{\beta}\text{Ability}_i + u_i$$

To figure out the causal effect of education κ on log (wage) we need to control for ability factors Ability_i. If we don't, we would likely have some bias, after all, ability is probably a confounder, causing both the treatment, education, and the outcome, earnings.

```
import warnings
warnings.filterwarnings('ignore')

import pandas as pd
import numpy as np
from scipy import stats
from matplotlib import style
import seaborn as sns
from matplotlib import pyplot as plt
import statsmodels.formula.api as smf
import graphviz as gr
from linearmodels.iv import IV2SLS

%matplotlib inline

pd.set_option("display.max_columns", 5)
style.use("fivethirtyeight")
```

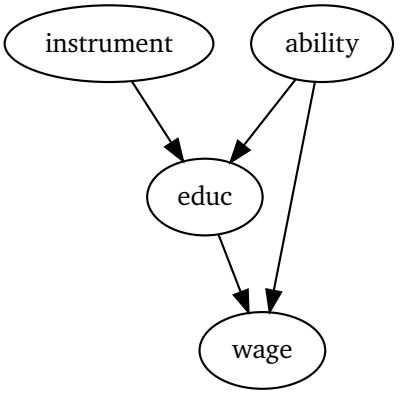
```
g = gr.Digraph()

g.edge("ability", "educ")
g.edge("ability", "wage")
g.edge("educ", "wage")
g
```



One way to avoid this is to control for constant levels of ability when measuring the effect of education on wage. We could do that by including ability in our linear regression model. However, we don't have good measurements of ability. The best we have are some very questionable proxies, like IQ.

But all is not lost. Here is where Instrumental Variables enters the picture. The idea of IV is to find another variable that causes the treatment and it is only correlated with the outcome through the treatment. Another way of saying this is that this instrument Z_i is uncorrelated with Y_0 and Y_1, but it is correlated with T. This is sometimes referred to as the exclusion restriction.

```
g = gr.Digraph()

g.edge("ability", "educ")
g.edge("ability", "wage")
g.edge("educ", "wage")
g.edge("instrument", "educ")
g
```



If we have such a variable, we can recover the causal effect κ with what we will see as the IV formula. To do so, let's think about the ideal equation we want to run. Using more general terms like T for the treatment and W for the confounders, here is what we want:

$$Y_i = \beta_0 + \kappa\, T_i + \boldsymbol{\beta} W_i + u_i$$

However, we don't have data on W, so all we can run is

$$Y_i = \beta_0 + \kappa\, T_i + v_i$$
$$v_i = \boldsymbol{\beta} W_i + u_i$$

Since W is a confounder, $\text{Cov}(T, v) \neq 0$. We have a short, not long equation. In our example, this would be saying that ability is correlated with education. If this is the case, running the short regression would yield a biased estimator for κ due to omitted variables.

Now, behold the magic of IV! Since the instrument Z is only correlated with the outcome through T, this implies that $\text{Cov}(Z, v) = 0$, otherwise there would be a second path from Z to Y through W. With this in mind, we can write

$$\text{Cov}(Z, Y) = \text{Cov}(Z, \beta_0 + \kappa\, T_i + v_i) = \kappa \text{Cov}(Z, T) + \text{Cov}(Z, v) = \kappa \text{Cov}(Z, T)$$

Dividing each side by $V(Z_i)$ and rearranging the terms, we get

$$\kappa = \frac{\text{Cov}(Y_i, Z_i)/V(Z_i)}{\text{Cov}(T_i, Z_i)/V(Z_i)} = \frac{\text{Reduced Form}}{\text{1st Stage}}$$

Notice that both the numerator and the denominator are regression coefficients (covariances divided by variances). The numerator is the result from the regression of Y on Z. In other words, it's the "impact" of Z on Y. Remember that this is not to say that Z causes Y, since we have a requirement that Z impacts Y only through T. Rather, it is only capturing how big this effect of Z on Y through T is. This numerator is so famous it has its own name: the reduced form coefficient.

The denominator is also a regression coefficient. This time, it is the regression of T on Z. This regression captures what is the impact of Z on T and it is also so famous that it is called the 1st Stage coefficient.

Another cool way to look at this equation is in terms of partial derivatives. We can show that the impact of T on Y is equal to the impact of Z on Y, scaled by the impact of Z on T:

$$\kappa = \frac{\frac{\partial y}{\partial z}}{\frac{\partial T}{\partial z}} = \frac{\partial y}{\partial z} * \frac{\partial z}{\partial T} = \frac{\partial y}{\partial T}$$

What this is showing to us is more subtle than most people appreciate. It is also cooler than most people appreciate. By writing IV like this, we are saying, "look, it's hard to find the impact of T on Y due to confounders. But I can easily find the impact of Z on Y, since there is nothing that causes Z and Y (exclusion restriction). However, I'm interested in the impact of T on Y, not Z on Y. So, I'll estimate the easy effect of Z on Y and **scale it by the effect of Z on T**, to convert the effect to T units instead of Z units".

We can also see this in a simplified case where the instrument is a dummy variable. In this case, the IV estimator gets further simplified by the ratio between 2 differences in means.

$$\kappa = \frac{E[Y|Z=1] - E[Y|Z=0]}{E[T|Z=1] - E[T|Z=0]}$$

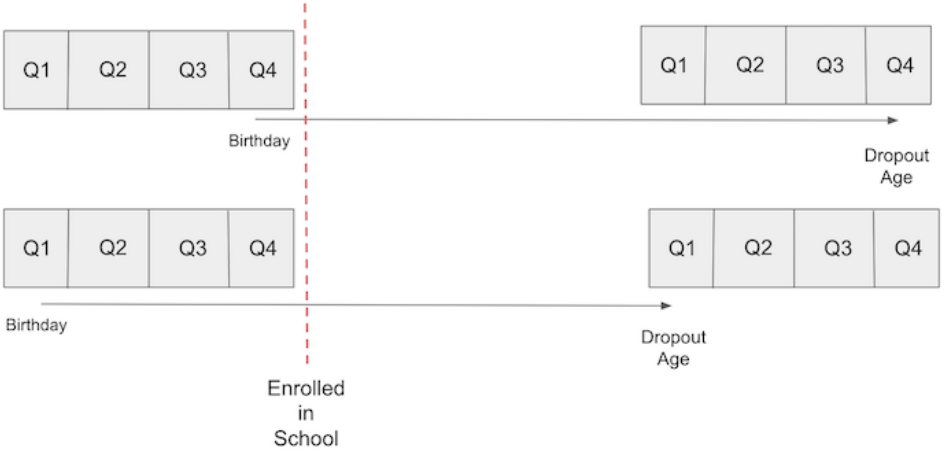
This ratio is sometimes referred to as the **Wald Estimator**. Again, we can tell the IV story where we want the effect of T on Y, which is hard to get. So we focus on the effect of Z on Y, which is easy. By definition, Z only affects Y through T, so we can now convert the impact of Z on Y to the impact of T on Y. We do so by scaling the effect of Z on Y by the effect of Z on T.

Quarter of Birth and the Effect of Education on Wage

So far, we've been treating these instruments as some magical variable Z which have the miraculous property of only affecting the outcome through the treatment. To be honest, good instruments are so hard to come by that we might as well consider them miracles. Let's just say it is not for the faint of heart. Rumor has it that the cool kids at Chicago School of Economics talk about how they come up with this or that instrument at the bar.

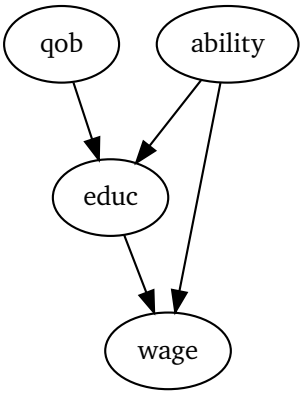
Still, we do have some interesting examples of instruments to make things a little more concrete. We will again try to estimate the effect of education on wage. To do so, we will use the person's quarter of birth as the instrument Z.

This idea takes advantage of US compulsory attendance law. Usually, they state that a kid must have turned 6 by January 1 of the year they enter school. For this reason, kids that are born at the beginning of the year will enter school at an older age. Compulsory attendance law also requires students to be in school until they turn 16, at which point they are legally allowed to drop out. The result is that people born later in the year have, on average, more years of education than those born in the beginning of the year.



If we accept that quarter of birth is independent of the ability factor, that is, it does not confound the impact of education on wage, we can use it as an instrument. In other words, we need to believe that quarter of birth has no impact on wage, other than through its impact on education. If you don't believe in astrology, this is a very compelling argument.

```
g = gr.Digraph()

g.edge("ability", "educ")
g.edge("ability", "wage")
g.edge("educ", "wage")
g.edge("qob", "educ")
g
```



To perform this analysis, we can use data from three decennial census, the same data used by Angrist and Krueger[1] in their article about IV. This dataset has information on log wages, our outcome variable, and years of schooling, our treatment variable. It also has data on quarter of birth, our instrument, and additional controls, such as year of birth and state of birth.

```
data = pd.read_csv("./data/ak91.csv")
data.head()
```

	log_wage	years_of_schooling	year_of_birth	quarter_of_birth	state_of_birth
0	5.790019	12.0	30.0	1.0	45.0
1	5.952494	11.0	30.0	1.0	45.0
2	5.315949	12.0	30.0	1.0	45.0
3	5.595926	12.0	30.0	1.0	45.0
4	6.068915	12.0	30.0	1.0	37.0

1 https://economics.mit.edu/faculty/angrist/data1/data/angkru1991

The 1st Stage

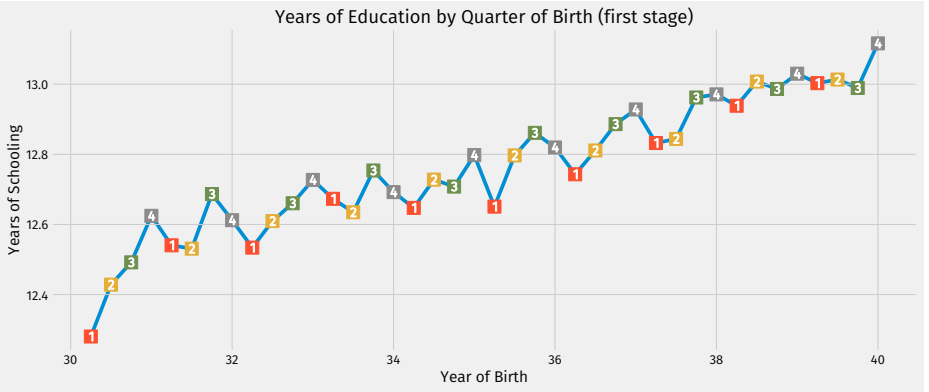
Before we use quarter of birth as an instrument, we need to make sure it is a valid one. This implies arguing in favor of the two Instrumental Variables assumptions:

1. $\text{Cov}(Z,T) \neq 0$. This is saying that we should have a strong 1st stage, or that the instrument indeed impacts the treatment variable.
2. $Y \perp Z|T$. This is the exclusion restriction, stating that the instrument Z only affects the outcome Y through the treatment T.

The first assumption is fortunately verifiable. We can see from data that $\text{Cov}(Z,T)$ is not zero. In our example, if quarter of birth is indeed an instrument like we've said, we should expect individuals born in the last quarter of the year to have slightly more time of education than those born in the beginning of the year. Before running any statistical test to verify this, let's just plot our data and see it with our own eyes.

```
group_data = (data
              .groupby(["year_of_birth", "quarter_of_birth"])
              [["log_wage", "years_of_schooling"]]
              .mean()
              .reset_index()
              .assign(time_of_birth = lambda d: d["year_of_birth"] +
                ↪ (d["quarter_of_birth"])/4))
```

```
plt.figure(figsize=(15,6))
plt.plot(group_data["time_of_birth"], group_data["years_of_schooling"],
  ↪ zorder=-1)
for q in range(1, 5):
    x = group_data.query(f"quarter_of_birth=={q}")["time_of_birth"]
    y = group_data.query(f"quarter_of_birth=={q}")["years_of_schooling"]
    plt.scatter(x, y, marker="s", s=200, c=f"C{q}")
    plt.scatter(x, y, marker=f"${q}$", s=100, c=f"white")

plt.title("Years of Education by Quarter of Birth (first stage)")
plt.xlabel("Year of Birth")
plt.ylabel("Years of Schooling");
```



Remarkably, there is a seasonal pattern on the years of schooling that follows the quarter of the year. Visually, we can see that those born in the first quarter of the year have almost always less education than those born in the last quarter (once we control for the year of birth, after all, those born in later years have more education, in general).

To be a bit more rigorous, we can run the 1st stage as a linear regression. We will first convert the quarter of birth to dummy variables:

```
factor_data = data.assign(**{f"q{int(q)}": (data["quarter_of_birth"] ==
  ↪ q).astype(int)
                             for q in data["quarter_of_birth"].unique()})

factor_data.head()
```

	log_wage	years_of_schooling	...	q3	q4
0	5.790019	12.0	...	0	0
1	5.952494	11.0	...	0	0
2	5.315949	12.0	...	0	0
3	5.595926	12.0	...	0	0
4	6.068915	12.0	...	0	0

5 rows × 9 columns

For simplicity, let's only use the last quarter, q4, as the instrument for now. We will run a regression of years of schooling, the treatment, on quarter of birth, the instrument. This will show us if indeed quarter of birth positively affects time of education like we saw in the plot above. We also need to control for years of birth here and we will add state of birth as an additional control.

```
first_stage = smf.ols("years_of_schooling ~ C(year_of_birth) + C(state_of_birth)
    ↪ + q4", data=factor_data).fit()

print("q4 parameter estimate:, ", first_stage.params["q4"])
print("q4 p-value:, ", first_stage.pvalues["q4"])
```

```
q4 parameter estimate:,  0.10085809272786578
q4 p-value:,  5.4648294166163044e-15
```

It looks like those born in the last quarter of the year have, on average, 0.1 more years of education than those born in other quarters of the year. The p-value is close to zero. This closes the case on whether quarter of birth causes more or less years of schooling.

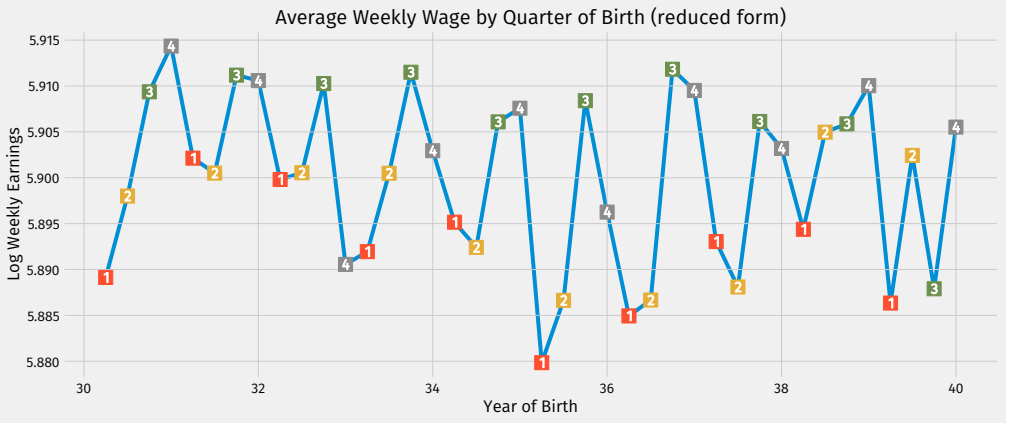
The Reduced Form

Unfortunately, we can't verify the second IV condition. We can only argue in favor of it. We can express our belief that quarter of birth does not influence potential earnings. In other words, the time people are born is not an indication of their personal ability or any other factor that can cause a difference in earnings, other than the effect on education. A good way of doing that is to say that the quarter of birth is as good as randomly assigned when we are thinking about it's impact on earnings. (It isn't random. There is evidence that people tend to conceive around the end of the summer or around some sort of holiday. But I can't think of any good reason that this pattern also affects income in any way other than through education).

Having argued in favor of the exclusion restriction, we can proceed to run the reduced form. The reduced form aims at figuring out how the instrument influences the outcome. Since, by assumption, all this influence is due to the effect on treatment, this will shed

some light into how the treatment affects the outcome. Once again, let's evaluate this visually before getting serious with regression.

```
plt.figure(figsize=(15,6))
plt.plot(group_data["time_of_birth"], group_data["log_wage"], zorder=-1)
for q in range(1, 5):
    x = group_data.query(f"quarter_of_birth=={q}")["time_of_birth"]
    y = group_data.query(f"quarter_of_birth=={q}")["log_wage"]
    plt.scatter(x, y, marker="s", s=200, c=f"C{q}")
    plt.scatter(x, y, marker=f"${q}$", s=100, c=f"white")

plt.title("Average Weekly Wage by Quarter of Birth (reduced form)")
plt.xlabel("Year of Birth")
plt.ylabel("Log Weekly Earnings");
```



Once again, we can see a seasonal pattern on earnings by the quarter of birth. Those born later on the year have slightly higher income than those born in the beginning of the year. To test this hypothesis, we will again regress the instrumental q4 on log wage. We will also add the same additional controls as in the 1st stage:

```
reduced_form = smf.ols("log_wage ~ C(year_of_birth) + C(state_of_birth) + q4",
  ↪ data=factor_data).fit()

print("q4 parameter estimate:, ", reduced_form.params["q4"])
print("q4 p-value:, ", reduced_form.pvalues["q4"])
```

```
q4 parameter estimate:,  0.008603484260140015
q4 p-value:,  0.0014949127183659584
```

Once again, we have a significant result. Those born in the last quarter of the year have, on average, 0.8% higher wages. This time, the p-value is not so close to zero as before, but it's still pretty significant, being just 0.0015.

Instrumental Variables by Hand

Having both our reduced form and our 1st stage, we can now scale the effect of the first stage by the reduced form. Since the first stage coefficient was something like 0.1, this will multiply the effect of the reduced form coefficient by almost 10. This will give us our unbiased IV estimate of the average causal effect:

$$\text{ATE}_{IV} = \frac{\text{Reduced Form}}{\text{1st Stage}}$$

```
reduced_form.params["q4"] / first_stage.params["q4"]
```

```
0.08530286492085315
```

This means that we should expect each additional year of school to increase wages by 8%.

Another way to get the IV estimates is by using 2 stages least squares, **2SLS**. With this procedure, we do the first stage like before and then run a second stage where we replace the treatment variable by the fitted values of the 1st stage

$$\text{educ}_i = \gamma_0 + \gamma_1 \times \text{q4}_i + \gamma_2 \times \text{yob}_i + \gamma_3 \times \text{sob}_i + \nu_i$$

$$\log(\text{wage})_i = \beta_0 + \beta_1 \times \text{educ}_i + \beta_2 \times \text{yob}_i + \beta_3 \times \text{sob}_i + u_i$$

$$\log(\text{wage})_i = \beta_0 + \beta_1[\gamma_0 + \gamma_1 \times \text{q4}_i + \gamma_2 \times \text{yob}_i + \gamma_3 \times \text{sob}_i + \nu_i] + \beta_2 \times \text{yob}_i + \beta_3 \times \text{sob}_i + u_i$$

One thing to notice is that **any additional control we add to the second stage should also be added to the first stage when doing IV**.

```
iv_by_hand = smf.ols("log_wage ~ C(year_of_birth) + C(state_of_birth) +
    ↪ years_of_schooling_fitted",
                     data=factor_data.assign(
                       ↪ years_of_schooling_fitted=first_stage.fittedvalues))
                       ↪ .fit()

iv_by_hand.params["years_of_schooling_fitted"]
```

```
0.0853028649208674
```

As you can see, the parameters are exactly the same. This second way of looking at IV can be useful for the intuition it gives. In 2SLS, the first stage creates a new version of the treatment that is purged from omitted variable bias. We then use this purged version of the treatment, the fitted values of the 1st stage, in a linear regression.

In practice, however, we don't do IV by hand. Not because it is troublesome, but because the standard errors we get from this second stage are a bit off. Instead, we should always

let the machine do the job for us. In Python, we can use the library linearmodels[2] to run 2SLS the right way.

The formula for 2SLS is a bit different. We should add the first stage between [ ] inside the formula. In our case, we add `years_of_schooling ~ q4`. Additional controls don't need to be added to the first stage because the computer will do this automatically if we include them in the second stage. For this reason, we add `year_of_birth` and `state_of_birth` outside the formula of the 1st stage.

```
def parse(model, exog="years_of_schooling"):
    param = model.params[exog]
    se = model.std_errors[exog]
    p_val = model.pvalues[exog]
    print(f"Parameter: {param}")
    print(f"SE: {se}")
    print(f"95 CI: {(-1.96*se,1.96*se) + param}")
    print(f"P-value: {p_val}")

formula = 'log_wage ~ 1 + C(year_of_birth) + C(state_of_birth) +
    ↪ [years_of_schooling ~ q4]'
iv2sls = IV2SLS.from_formula(formula, factor_data).fit()
parse(iv2sls)
```

```
Parameter: 0.0853028649580665
SE: 0.02554081280759766
95 CI: [0.03524287 0.13536286]
P-value: 0.0008381914592261452
```

Once again, we can see that the parameter is exactly the same as the ones we've got before. The extra benefit is that we have valid standard errors now. With this at hand, we can say that we expect 1 extra year of education to increase wages by 8.5%, on average.

Multiple Instruments

Another advantage of using the computers is to run 2SLS is that it is easy to add multiple instruments. In our example, we will use all quarter of birth dummies as instruments for years of schooling.

```
formula = 'log_wage ~ 1 + C(year_of_birth) + C(state_of_birth) +
    ↪ [years_of_schooling ~ q1+q2+q3]'
iv_many_zs = IV2SLS.from_formula(formula, factor_data).fit()
parse(iv_many_zs)
```

2 https://bashtage.github.io/linearmodels/

```
Parameter: 0.10769370488924324
SE: 0.0195571490089523
95 CI: [0.06936169 0.14602572]
P-value: 3.657974678716869e-08
```

With all 3 dummies, the estimated return on education is now 0.1, which means that we should expect a 10% average increase on earnings for every additional year of education. Let's compare this with the traditional OLS estimate. To do that, we can use 2SLS again, but without the 1st stage now.

```
formula = "log_wage ~ years_of_schooling + C(state_of_birth) + C(year_of_birth) +
      ↪ C(quarter_of_birth)"
ols = IV2SLS.from_formula(formula, data=data).fit()
parse(ols)
```

```
Parameter: 0.067325728176578
SE: 0.00038839984390487046
95 CI: [0.06656446 0.06808699]
P-value: 0.0
```

The return on education is estimated to be lower with OLS than with 2SLS. This suggests that OVB might not be as strong as we first though. Also, notice the confidence intervals. 2SLS has a much wider CI than the OLS estimate. Let's explore this further

Weakness of Instruments

When dealing with IV, we need to remember we are estimating the ATE indirectly. Our estimates depend on both the first stage and the second stage. If the impact of the treatment on the outcome is indeed strong, the second stage will also be strong. However, it doesn't matter how strong the second stage is if we have a weak first stage. A weak first stage means that the instrument has only a very small correlation with the treatment. Therefore, we can't learn much about the treatment from the instrument.

The formulas for the IV standard errors are a bit complex and not so intuitive, so we will try something else to grasp this problem. We will simulate data where we have a treatment T with effect 2.0 on the outcome Y, an unobserved confounder U and an additional control X. We will also simulate multiple instruments with different strengths on the 1st stage.

$$
\begin{aligned}
X &\sim N(0, 2^2) \\
U &\sim N(0, 2^2) \\
T &\sim N(1 + 0.5U, 5^2) \\
Y &\sim N(2 + X - 0.5U + 2T, 5^2) \\
Z &\sim N(T, \sigma^2) \text{ for } \sigma^2 \text{ in 0.1 to 100}
\end{aligned}
$$

```
np.random.seed(12)
n = 10000
X = np.random.normal(0, 2, n) # observable variable
U = np.random.normal(0, 2, n) # unobservable (omitted) variable
T = np.random.normal(1 + 0.5*U, 5, n) # treatment
Y = np.random.normal(2 + X - 0.5*U + 2*T, 5, n) # outcome

stddevs = np.linspace(0.1, 100, 50)
Zs = {f"Z_{z}": np.random.normal(T, s, n) for z, s in enumerate(stddevs)}
  ↪ # instruments with decreasing \mathrm{Cov}(Z, T)

sim_data = pd.DataFrame(dict(U=U, T=T, Y=Y)).assign(**Zs)

sim_data.head()
```

	U	T	...	Z_48	Z_49
0	2.696148	8.056988	...	-117.798705	-13.485292
1	2.570240	0.245067	...	-209.727577	-70.792948
2	0.664741	5.597510	...	60.562232	47.619414
3	1.037725	0.493532	...	78.136513	-108.322304
4	-2.590591	-6.263014	...	78.776566	-80.547214

5 rows × 53 columns

Just to double check, we can see that the correlation between Z and T is indeed decreasing.

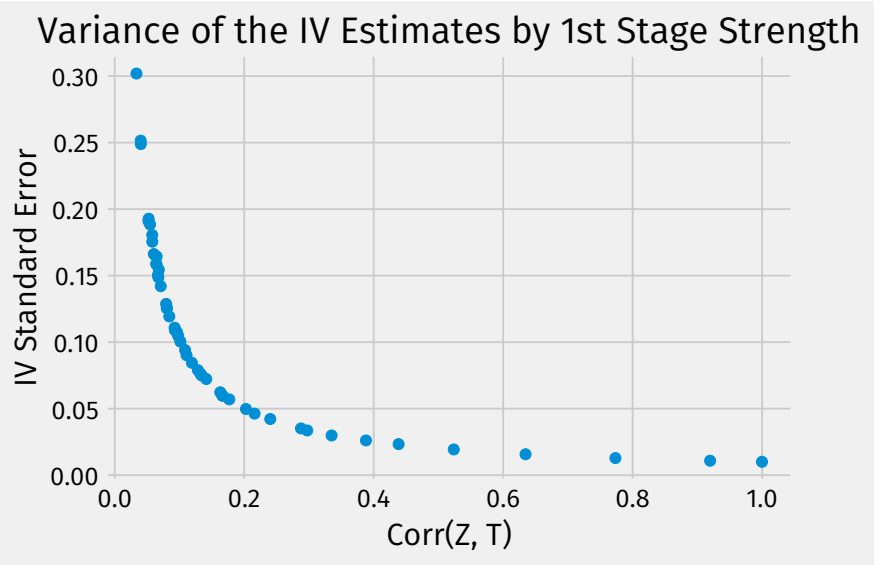
```
corr = (sim_data.corr()["T"]
        [lambda d: d.index.str.startswith("Z")])

corr.head()
```

```
Z_0    0.999807
Z_1    0.919713
Z_2    0.773434
Z_3    0.634614
Z_4    0.523719
Name: T, dtype: float64
```

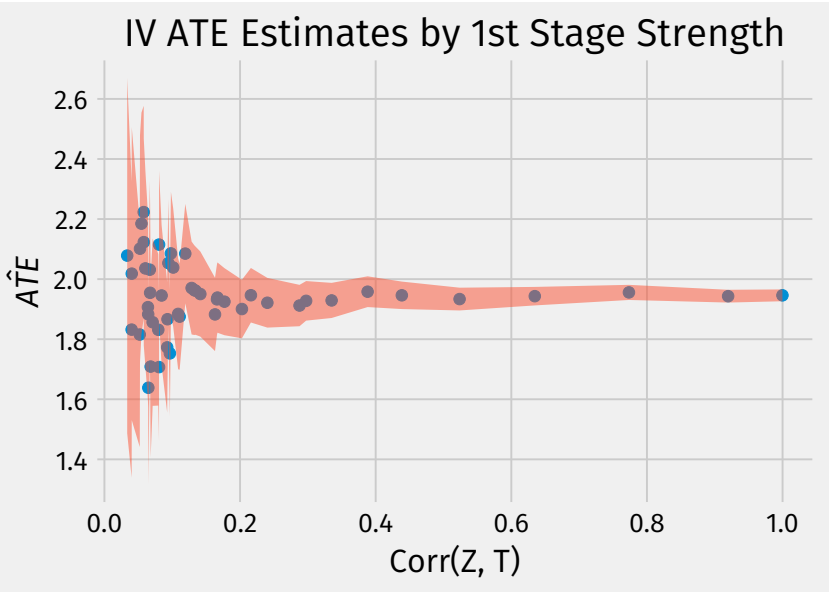
Now, we will run one IV model per instrument we have and collect both the ATE estimate and the standard error.

```
se = []
ate = []
for z in range(len(Zs)):
    formula = f'Y ~ 1 + X + [T ~ Z_{z}]'
    iv = IV2SLS.from_formula(formula, sim_data).fit()
    se.append(iv.std_errors["T"])
    ate.append(iv.params["T"])
```

```
plot_data = pd.DataFrame(dict(se=se, ate=ate, corr=corr)).sort_values(by="corr")

plt.scatter(plot_data["corr"], plot_data["se"])
plt.xlabel("Corr(Z, T)")
plt.ylabel("IV Standard Error");
plt.title("Variance of the IV Estimates by 1st Stage Strength");
```



```
plt.scatter(plot_data["corr"], plot_data["ate"])
plt.fill_between(plot_data["corr"],
                 plot_data["ate"]+1.96*plot_data["se"],
                 plot_data["ate"]-1.96*plot_data["se"], alpha=.5)
plt.xlabel("Corr(Z, T)")
plt.ylabel("$\hat{ATE}$");
plt.title("IV ATE Estimates by 1st Stage Strength");
```



As we can see in the plots above, estimates vary wildly when the correlation between T and Z is weak. This is because the SE also increases a lot when the correlation is low.

Another thing to notice is that **2SLS is biased**! Even with high correlation, the parameter estimate still does not reach the true ATE of 2.0. Actually, 2.0 is not even in the 95% CI! 2SLS is only consistent, which means that it approaches the true parameter value if the sample size is big enough. However, we can't know how big is big enough. We can only stick to some rules of thumb to understand how this bias behaves:

1. 2SLS is biased towards OLS. This means that if OLS has a negative/positive bias, 2SLS will also have it. The advantage of 2SLS is that it is at least consistent, where OLS is not, in the case of omitted variables. In the example above, our unobserved U impacts negatively the outcome but its positively correlated with the treatment, which will result in a negative bias. That is why we are seeing the ATE estimate below the true value (negative bias).
2. The bias will increase with the number of instruments we add. If we add too many instruments, 2SLS becomes more and more like OLS.

Besides knowing how this bias behaves, a final piece of advice is to avoid some **common mistakes when doing IV**:

1. Doing IV by hand. As we've seen, IV by hand will result in wrong standard errors, even if the parameter estimates are right. The SE won't be completely off. Still, why do it if you can use software and get the right SE?
2. Using anything other than OLS on the 1st stage. Lots of Data Scientist encounter IV and think they can do better. For example, they see a dummy treatment and think about replacing the 1st stage by a logistic regression, after all, they are predicting a dummy variable, right?. The problem is that this is plain wrong. The consistency of IV relies on a property that only OLS can give, which is the orthogonality of the residuals, so anything different than OLS on the 1st stage will yield something biased. (OBS: there are some modern techniques that use Machine Learning for IV, but their results have been, at best, questionable).

Key Ideas

We've taken some time here to understand how we can work around omitted variable bias if we have an instrument variable. An instrument is a variable that is correlated with the treatment (has a first stage), but only affects the outcome through the treatment (exclusion restriction). We saw an example of an instrument with quarter of birth to estimate the effect of education on income.

We then delve into the mechanics of estimating the causal effect with IV, namely, using 2SLS. We've also learned that IV is no silver bullet. It can be quite troublesome when we have a weak first stage. Also, although consistent, 2SLS is still a biased method to estimate the causal effect.

References

I like to think of this entire book as a tribute to Joshua Angrist, Alberto Abadie and Christopher Walters for their amazing Econometrics class. Most of the ideas here are taken from their classes at the American Economic Association. Watching them is what is keeping me sane during this tough year of 2020.

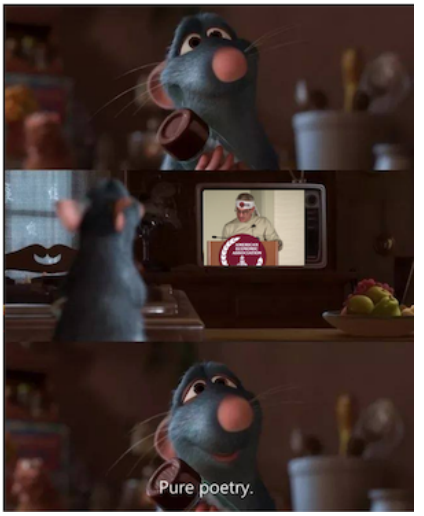
- Cross-Section Econometrics · https://www.aeaweb.org/conference/cont-ed/2017-webcasts
- Mastering Mostly Harmless Econometrics · https://www.aeaweb.org/conference/cont-ed/2020-webcasts

I'll also like to reference the amazing books from Angrist. They have shown me that Econometrics, or 'Metrics as they call it, is not only extremely useful but also profoundly fun.

- Mostly Harmless Econometrics · https://www.mostlyharmlesseconometrics.com/
- Mastering 'Metrics · https://www.masteringmetrics.com/

My final reference is Miguel Hernan and Jamie Robins' book. It has been my trustworthy companion in the most thorny causal questions I had to answer.

- Causal Inference Book · https://miguelhernan.org/whatifbook



CHAPTER
9

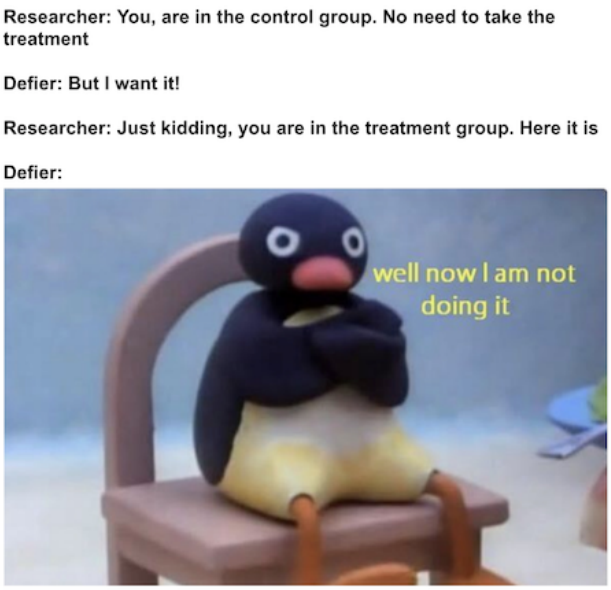
Non Compliance and LATE

Dipping our Toes into a Heterogeneous World

Previously, we've seen Instrumental Variables through a more traditional lens. IV was seen as some sort of natural experiment we can leverage. In contrast, modern IV practice draws a lot of insight from medical sciences. It partitions the world into 4 kinds of subjects, depending on how they respond to the instrumental variable.

1. Compliers
2. Never Takers
3. Always Takers
4. Defiers

This naming comes from pharmaceutical science. Imagine you are conducting an experiment to test the effect of a new drug on some illness. Each subject gets assigned to a treatment: a drug or a placebo. The compliers are the subjects that stick to what was assigned to them. If they get the placebo, they take it; if they get the drug, they also take it. The never takers are those subjects that refuse to take their medicine. Even if they are assigned the new drug, they won't take it. On the other hand, the always takers are those that can somehow get the new drug even if they were assigned to the placebo. Finally, Defiers are the ones that take the treatment if assigned to the control and take the control if assigned the treatment. You can think of them as that annoying child that does the opposite of what it is told. In practice, they are not so common (the defiers, not the kids), so we will often ignore them.



Modern IV treats the instrument as a quasi-experimental design where compliance is not perfect. By doing so, it distinguishes internally from externally valid causal effects. Just as a reminder, an internally valid effect is one we are able to identify. It is valid in that particular setting, with that particular data. In IV, it would be the treatment effect on those where the instrument changes the treatment. External validity, on the other hand, would be concerned with the predictive power of that causal effect. It would ask the question of whether we could generalize the effect we found in this sample for other populations. For example, let's say you conducted an RCT in your university to figure out if people are generous or not, when given an incentive to donate. The experiment is well designed, but you only invite economics students to participate. You then find that all of them are selfish jerks. This is an internally valid conclusion. It's valid for those data points. But can you, from that experiment, infer that humanity is selfish? That's hardly the case. So we would question whether your experiment has external validity to generalize its results. Anyway, back to IV.

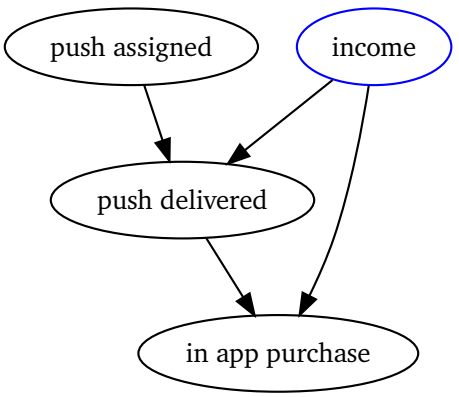
To make it more concrete, let's consider a case where you want to boost user engagement measured by in app purchase. One way to do that, is by asking your marketing department to come up with a push you can use to engage your users. They come up with a brilliant design and very fancy user interactions. With that push, you move on to design a randomised trial. You select 10000 random customers and, for each of them, you assign the push with 50% probability. However, when you execute the test, you notice that some customers who were assigned to receive the push are not receiving it. When you talk to the engineers, they say that it is because they probably have an older phone that doesn't support the kind of push the marketing team designed.

At first, you might think that this is no big deal. Instead of using treatment assigned as your treatment variable, you can just use the treatment received, right? It turns out

it is not that simple. If you plot the causal graph of this whole situation, it would look something like this:

```
import warnings
warnings.filterwarnings('ignore')

import pandas as pd
import numpy as np
from scipy import stats
from matplotlib import style
import seaborn as sns
from matplotlib import pyplot as plt
import statsmodels.formula.api as smf
from linearmodels.iv import IV2SLS
import graphviz as gr

%matplotlib inline

style.use("fivethirtyeight")
```

```
g = gr.Digraph()

g.edge("push assigned", "push delivered")
g.edge("push delivered", "in app purchase")
g.edge("income", "in app purchase")
g.edge("income", "push delivered")
g.node("income", color="blue")
g
```



Up on the causal graph, you have the push assignment. This is random by design, so nothing is causing it. Then, you have a node for if the push was delivered. Not everyone that was assigned to receive the push did it, so you have non compliance here. More specifically, you have some never takers: those that don't receive the treatment even if assigned to it. You also have reasons to suspect that this non compliance is not simply by

chance. Since people with older phones are the ones that don't get the push, you can argue that income is also causing push delivery. The richer the person, the more likely it is that he or she has a nicer phone, which in turn makes it more likely that he or she will receive the push. Finally, you have the outcome variable, in app purchase. Keep in mind that we don't know income, so we can't control it. With this in mind, let's examine what would happen if we simply used the push assigned as the treatment variable and if instead we used push delivered as the treatment.

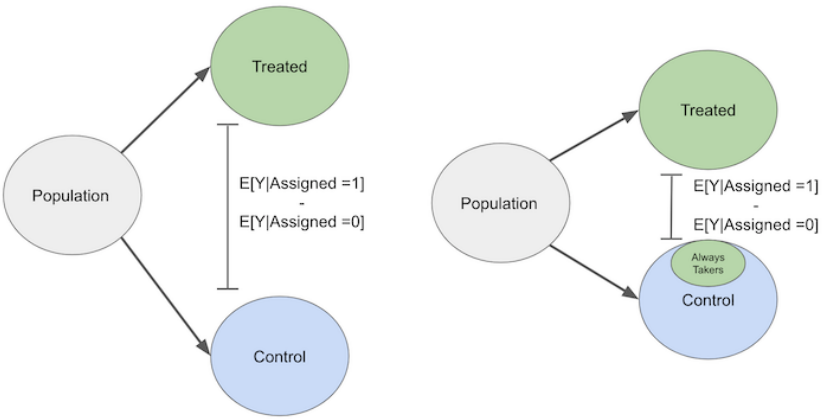
In the first case, we would estimate the causal effect by the following difference in means:

$$ATE = E[Y \mid pushAssigned = 1] - E[Y \mid pushAssigned = 0]$$

As we are very aware by now, this is only unbiased estimate for $E[Y_1] - E[Y_0]$ if the bias $E[Y_0 \mid pushAssigned = 0] - E[Y_0 \mid pushAssigned = 1]$ is zero. Since `pushAssigned` is random, we know that the bias is zero. Does this settle it then? Not quite. You see, if we do this, we are actually answering a different question than the one we intended to. We would be finding the **causal effect of treatment assignment**, not of the treatment itself. But are they different or can we extrapolate the causal effect of treatment assignment to the ATE? In other words, is the causal effect of treatment assignment an unbiased estimate of the ATE?

As it turns out, it is not. Because of non compliance, the outcome of those assigned to the treatment will be pushed in the direction of the outcome of those assigned to the control. Non compliance flips the treatment unintendedly, making the treatment and control more similar in the outcome. Don't confuse this with similarity on the variables. We want treated and control to be similar on the variables. That will make them comparable. What we don't want is for them to be similar in the outcome if there is indeed a treatment effect.

To see this, suppose first we have always takers. Some of them will be assigned to the control by chance. But those that are, will take the treatment anyway. This makes them essentially a treated group that is mixed up with the control. As a result of this mix, the causal effect will be harder to find when we have non compliance.

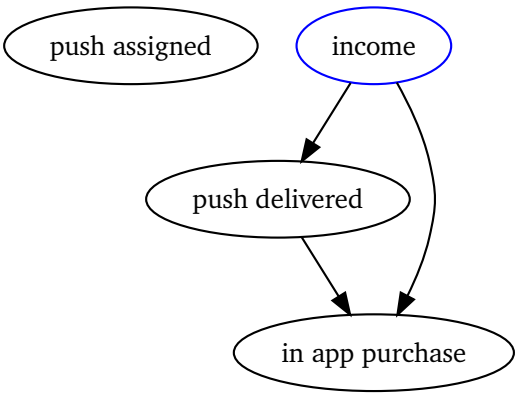


By the same reasoning, never takers will make those assigned to the treatment look a little bit like the untreated, because they don't take the treatment even if assigned to it. In this sense, **the causal effect of treatment assignment is biased towards zero** because non compliance shrinks the detectable impact. Another way to see this is to imagine an extreme case. Let's say that non compliance is really high. Treatment assignment says nothing about treatment received. Treatment received is purely random in this case. In IV language, that would be saying we have a very weak 1st stage. Using Z to denote the treatment assignment, we would have that

$$E[Y \mid Z = 1] - E[Y \mid Z = 0] = 0$$

There would be no more causal link of treatment assignment to the outcome. Z would be just a meaningless random variable floating there.

```
g = gr.Digraph()

g.node("push assigned")
g.edge("push delivered", "in app purchase")
g.edge("income", "in app purchase")
g.edge("income", "push delivered")
g.node("income", color="blue")
g
```



OK, so we've ruled out using the causal effect of the assignment as a way to estimate the causal effect of the treatment. So, how about just using the treatment received?

$$\text{ATE} = E[Y \mid \text{push} = 1] - E[Y \mid \text{push} = 0]$$

Once again, we need to think if this is biased or not, or if $E[Y_0 \mid \text{push} = 0] = E[Y_0 \mid \text{push} = 1]$. Just by looking at the causal graph above, we know it is not the case. We have that unmeasured confounder, income, lurking around, and it is sure to mess things up. As we've said earlier, we know that failure in push delivery, in our case, is caused by customers having older phones. Which means that we probably have $E[Y_0 \mid \text{push} = 0] < E[Y_0 \mid \text{push} = 1]$. We think this is the case because customers with less money both have older phones which will lead to push = 0 and also a lower potential in app purchase Y_0.

Bummer! We can't use treatment assigned nor treatment received to estimate our ATE. But fortunately, we know what we can use: Instrumental Variables. Here, treatment assigned is the perfect instrument for the treatment. It is as good as random and it only causes in-app purchase by the virtue of the treatment.

Local Average Treatment Effect: LATE

Local average treatment effect makes it explicit the population for which we can estimate the causal effect. It is also another way to view IV, which gives other cool intuitions we can use. In modern IV, we see the instrument as starting a causal chain: Z causes T which causes Y. In this context, the exclusion restriction means that Z does not cause Y, unless by its effect on T. The 1st stage is now viewed as the causal effect of Z on T. We also rewrite potential outcomes with a double indexing notation, where the first index denotes the counterfactual for the instrument and the second index, for the treatment

$$\text{Potential Outcome} = \begin{cases} Y_i(1,1) \text{ if } T_i = 1,\ Z_i = 1 \\ Y_i(1,0) \text{ if } T_i = 1,\ Z_i = 0 \\ Y_i(0,1) \text{ if } T_i = 0,\ Z_i = 1 \\ Y_i(0,0) \text{ if } T_i = 0,\ Z_i = 0 \end{cases}$$

In a sense, the treatment becomes the outcome, at least in the first stage. This means we can also write it with potential outcome notation:

$$\text{Potential Treatment} = \begin{cases} T_0 \text{ if } Z_i = 0 \\ T_1 \text{ if } Z_i = 1 \end{cases}$$

Instrumental Variable assumptions can now be rewritten as follows

1. $T_{0i}, T_{1i} \perp Z_i$ and $Y_i(T_{1i}, 1), Y_i(T_{0i}, 0) \perp Z_i$. This is the Independence Assumption. This says that the instrument is as good as randomly assigned. In other words, Z, the instrument, is not correlated with the potential treatments, which is the same as saying that people in different instrument groups are comparable.
2. $Y_i(1,0) = Y_i(1,1) = Y_{i1}$ and $Y_i(0,0) = Y_i(0,1) = Y_{i0}$. This is the exclusion restriction. It says that if I'm looking at the potential outcome for the treated, it is the same for both instrument groups. In other words, the instrument does not affect the potential outcome, which is the same as saying that the instrument only affects the outcome through the treatment.
3. $E[T_{1i} - T_{0i}] \neq 0$. This is the existence of a 1st stage. It is saying that the potential outcome of the 1st stage, that is, the potential treatment, is NOT the same. Another way of saying this is that the instrument does affect the treatment.
4. $T_{i1} \geq T_{i0}$. This is the monotonicity assumption. It is saying that if everyone had the instrument turned on, the treatment level would be equal or higher than if everyone had the instrument turned off.

Now, let's review the Wald estimator to gain some further intuition on IV:

$$ATE = \frac{E[Y|Z=1] - E[Y|Z=0]}{E[T|Z=1] - E[T|Z=0]}$$

Let's take the first bit of it, $E[Y|Z=1]$. Using the exclusion restriction, we can rewrite Y in terms of potential outcome like this.

$$E[Y_i|Z_i=1] = E[Y_{i0} + T_{i1}(Y_{i1} - Y_{i0})|Z_i=1]$$

Using independence, we can take out the conditioning on Z.

$$E[Y_i|Z_i=1] = E[Y_{i0} + T_{i1}(Y_{i1} - Y_{i0})]$$

With a similar argument, we get that

$$E[Y_i|Z_i=0] = E[Y_{i0} + T_{i0}(Y_{i1} - Y_{i0})]$$

We can now rewrite the numerator of the Wald estimator like this

$$E[Y|Z=1] - E[Y|Z=0] = E[(Y_{i1} - Y_{i0})(T_{i1} - T_{i0})]$$

Using monotonicity, we know that $T_{i1} - T_{i0}$ is 0 or 1, so

$$E[(Y_{i1} - Y_{i0})(T_{i1} - T_{i0})] = E[(Y_{i1} - Y_{i0})|T_{i1} > T_{i0}]P(T_{i1} > T_{i0})$$

Using a similar argument to tackle the denominator, we get that

$$E[T|Z=1] - E[T|Z=0] = E[T_{i1} - T_{i0}] = P(T_{i1} > T_{i0})$$

All of this allows us to look at the Wald estimator like this:

$$ATE = \frac{E[(Y_{i1} - Y_{i0})|T_{i1} > T_{i0}]P(T_{i1} > T_{i0})}{P(T_{i1} > T_{i0})} = E[(Y_{i1} - Y_{i0})|T_{i1} > T_{i0}]$$

That is, the ATE as estimated by IV is the ATE on the subpopulation where $T_{i1} > T_{i0}$. And if you think about compliance, which population is this? This is the population where those that get the instrument turned on have the treatment level higher than if they had the instrument turned off. In other words, this is the complier population. Just so we can remember,

1. Compliers means that $T_{i1} > T_{i0}$
2. Never Takers $T_{i1} = T_{i0} = 0$
3. Always Takers $T_{i1} = T_{i0} = 1$

The conclusion of this is that IV says nothing about the effect on never takers, always takers or defiers, because the treatment is unchanged for them! **IV only finds the treatment effect for the compliers**.

Effect on Engagement

Let's see how all of this plays out in the case study where we try to estimate the effect of a push on in-app purchase. The causal graph is the one we depicted above, so I won't repeat it here. The data we have is on both push assigned, the randomised instrument, and push delivered, the treatment variable.

```
data = pd.read_csv("./data/app_engagement_push.csv")

data.head()
```

	in_app_purchase	push_assigned	push_delivered
0	47	1	1
1	43	1	0
2	51	1	1
3	49	0	0
4	79	0	0

First, let's run OLS to see what it would give us.

```
ols = IV2SLS.from_formula("in_app_purchase ~ 1 + push_assigned + push_delivered",
      ↪ data).fit()
ols.summary.tables[1]
```

	Parameter	Std. Err.	T-stat	P-value	Lower CI	Upper CI
Intercept	69.292	0.3624	191.22	0.0000	68.581	70.002
push_assigned	-17.441	0.5702	-30.590	0.0000	-18.559	-16.324
push_delivered	27.600	0.6124	45.069	0.0000	26.399	28.800

OLS is saying that the treatment effect is BRL 27.60, that is, the push increases in-app purchase by 27.6 reais. However, we have reasons to believe this is a biased estimate. We know that older phones are having trouble in receiving the push, so, probably, richer customers, with newer phones, are the compliers. Since the ones that get the treatment also have more money, we believe this bias is positive and the true impact of the push is lower. In other words, we probably have $E[Y_0 | T = 0] < E[Y_0 | T = 1]$.

Now, let's try to estimate this effect with Instrumental Variables. First, let's run the first stage.

```
first_stage = IV2SLS.from_formula("push_delivered ~ 1 + push_assigned",
  ↪ data).fit()
first_stage.summary.tables[1]
```

	Parameter	Std. Err.	T-stat	P-value	Lower CI	Upper CI
Intercept	0.0000	7.408e-11	0.0000	1.0000	-1.452e-10	1.452e-10
push_assigned	0.7176	0.0064	112.07	0.0000	0.7050	0.7301

Looks like we have a strong first stage. Those that get assigned to get the push get it 71.8% of the time. This means that we have something like 28% of never takers. We also have strong reasons to believe there are no always takers, since the intercept parameter is estimated to be zero. This means that no one gets the push if it is not assigned to it. Given the design of our experiment, this is expected.

Let's now run the reduced form:

```
reduced_form = IV2SLS.from_formula("in_app_purchase ~ 1 + push_assigned",
  ↪ data).fit()
reduced_form.summary.tables[1]
```

	Parameter	Std. Err.	T-stat	P-value	Lower CI	Upper CI
Intercept	69.292	0.3624	191.22	0.0000	68.581	70.002
push_assigned	2.3636	0.5209	4.5376	0.0000	1.3427	3.3845

The reduced form shows that the causal effect of treatment assignment is 2.36. This means that assigning someone to receive the push increases the in-app purchase by 2.36 reais.

If we divide the reduced form by the first stage, we scale the effect of the instrument by the units of the treatment, we get $2.3636/0.7176 = 3.29$. Running the 2SLS, we get these same estimates, with the bonus of correct standard errors.

```
iv = IV2SLS.from_formula("in_app_purchase ~ 1 + [push_delivered ~
      ↪ push_assigned]", data).fit()
iv.summary.tables[1]
```

	Parameter	Std. Err.	T-stat	P-value	Lower CI	Upper CI
Intercept	69.292	0.3624	191.22	0.0000	68.581	70.002
push_delivered	3.2938	0.7165	4.5974	0.0000	1.8896	4.6981

This shows that the result with 2SLS is much lower than the one we got with OLS: 3.29 against 27.60. This makes sense, since the causal effect estimated with OLS is positively biased. We also need to remember about LATE. 3.29 is the average causal effect on compliers. Unfortunately, we can't say anything about those never takers. This means that we are estimating the effect on the richer segment of the population that have newer phones.

Key Ideas

Here we've looked at a more modern view of Instrumental Variables. We saw how IV can be seen as a causal chain where the instrument causes the treatment which causes the outcome. In this view, we took a look at compliance to understand ATE in IV estimation and we've figured out that it was the LATE on compliers.

References

I like to think of this entire book as a tribute to Joshua Angrist, Alberto Abadie and Christopher Walters for their amazing Econometrics class. Most of the ideas here are taken from their classes at the American Economic Association. Watching them is what is keeping me sane during this tough year of 2020.

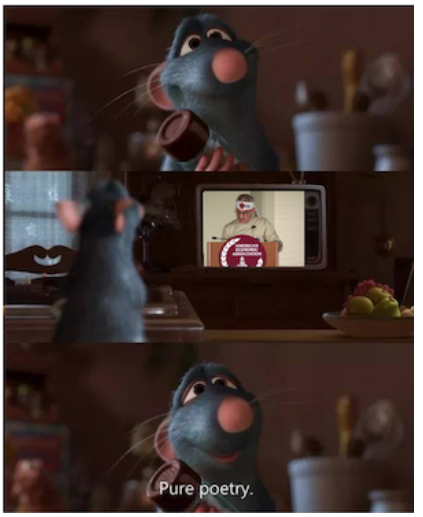
- Cross-Section Econometrics · https://www.aeaweb.org/conference/cont-ed/2017-webcasts
- Mastering Mostly Harmless Econometrics · https://www.aeaweb.org/conference/cont-ed/2020-webcasts

I'll also like to reference the amazing books from Angrist. They have shown me that Econometrics, or 'Metrics as they call it, is not only extremely useful but also profoundly fun.

- Mostly Harmless Econometrics · https://www.mostlyharmlesseconometrics.com/
- Mastering 'Metrics · https://www.masteringmetrics.com/

My final reference is Miguel Hernan and Jamie Robins' book. It has been my trustworthy companion in the most thorny causal questions I had to answer.

- Causal Inference Book · https://miguelhernan.org/whatifbook


Pure poetry.

CHAPTER
10

Matching

What is Regression Doing After All?

As we've seen so far, regression does an amazing job at controlling for additional variables when we do a test vs control comparison. If we have independence, $(Y_0, Y_1) \perp T \,|\, X$, then regression can identify the ATE by controlling for X. The way regression does this is kind of magical. To get some intuition about it, let's remember the case when all variables X are dummy variables. If that is the case, regression partitions the data into the dummy cells and computes the mean difference between test and control. This difference in means keeps the Xs constant, since we are doing it in a fixed cell of X dummy. It is as if we were doing $E[Y \,|\, T = 1] - E[Y \,|\, T = 0] \,|\, X = x$, where x is a dummy cell (all dummies set to 1, for example). Regression then combines the estimate in each of the cells to produce a final ATE. The way it does this is by applying weights to the cell proportional to the variance of the treatment on that group.

```
import warnings
warnings.filterwarnings('ignore')

import pandas as pd
import numpy as np
from matplotlib import style
from matplotlib import pyplot as plt
import statsmodels.formula.api as smf

import graphviz as gr

%matplotlib inline

style.use("fivethirtyeight")
```

To give an example, let's suppose I'm trying to estimate the effect of a drug and I have 6 men and 4 women. My response variable is days hospitalised and I hope my drug can

lower that. On men, the true causal effect is -3, so the drug lowers the stay period by 3 days. On women, it is -2. To make matters more interesting, men are much more affected by this illness and stay longer at the hospital. They also get much more of the drug. Only 1 out of the 6 men does not get the drug. On the other hand, women are more resistant to this illness, so they stay less at the hospital. 50% of the women get the drug.

```
drug_example = pd.DataFrame(dict(
    sex= ["M","M","M","M","M","M", "W","W","W","W"],
    drug=[1,1,1,1,1,0,  1,0,1,0],
    days=[5,5,5,5,5,8,  2,4,2,4]
))
```

Note that simple comparison of treatment and control yields a negatively biased effect, that is, the drug seems less effective than it truly is. This is expected, since we've omitted the sex confounder. In this case, the estimated ATE is smaller than the true one because men get more of the drug and are more affected by the illness.

```
drug_example.query("drug==1")["days"].mean() - drug_example
  ↪ .query("drug==0")["days"].mean()
```

```
-1.1904761904761898
```

Since the true effect for men is -3 and the true effect for women is -2, the ATE should be

$$ATE = \frac{(-3 * 6) + (-2 * 4)}{10} = -2.6$$

This estimate is done by 1) partitioning the data into confounder cells, in this case, men and women, 2) estimating the effect on each cell and 3) combining the estimate with a weighted average, where the weight is the sample size of the cell or covariate group. If we had exactly the same number of men and women in the data, the ATE estimate would be right in the middle of the ATE of the 2 groups, -2.5. Since there are more men than women in our dataset, the ATE estimate is a little bit closer to the men's ATE. This is called a non-parametric estimate, since it places no assumption on how the data was generated.

If we control for sex using regression, we will add the assumption of linearity. Regression will also partition the data into men and women and estimate the effect on both of these groups. So far, so good. However, when it comes to combining the effect on each group, it does not weigh them just by the sample size. Instead, regression uses weights that are also proportional to the variance of the treatment in that group. In our case, the variance of the treatment in men is smaller than in women, since only one man is in the control group. To be exact, the variance of T for men is $0.139 = 1/6 * (1 - 1/6)$ and for women is $0.25 = 2/4 * (1 - 2/4)$. So regression will give a higher weight to women in our example and the ATE will be a bit closer to the women's ATE of -2.

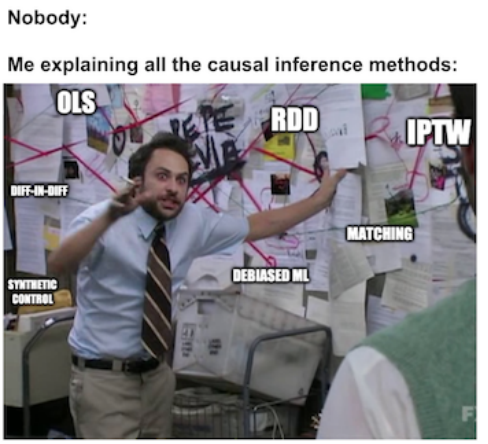
```
smf.ols('days ~ drug + C(sex)', data=drug_example).fit().summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	7.5455	0.188	40.093	0.000	7.100	7.990
C(sex)[T.W]	-3.3182	0.176	-18.849	0.000	-3.734	-2.902
drug	-2.4545	0.188	-13.042	0.000	-2.900	-2.010

This result is more intuitive with dummy variables, but, in its own weird way, regression also keeps continuous variables constant while estimating the effect. Also with continuous variables, the ATE will point in the direction where covariates have more variance.

So we've seen that regression has its idiosyncrasies. It is linear, parametric, likes high variance features… This can be good or bad, depending on the context. Because of this, it's important to be aware of other techniques we can use to control for confounders. Not only are they an extra tool in your causal tool belt, but understanding different ways to deal with confounding expands our understanding of the problem. For this reason, I present you now the **Subclassification Estimator!**

The Subclassification Estimator



If there is some causal effect we want to estimate, like the effect of job training on earnings, **and** the treatment is not randomly assigned, we need to watch out for confounders. It could be that only more motivated people do the training and they would have higher earnings regardless of the training. We need to estimate the effect of the training program within small groups of individuals that are roughly the same in motivation level and any other confounders we may have.

More generally, if there is some causal effect we want to estimate, but it is hard to do so because of confounding of some variables X, what we need to do is make the treatment vs control comparison within small groups where X is the same. If we have conditional independence $(Y_0, Y_1) \perp T \,|\, X$, then we can write the ATE as follows.

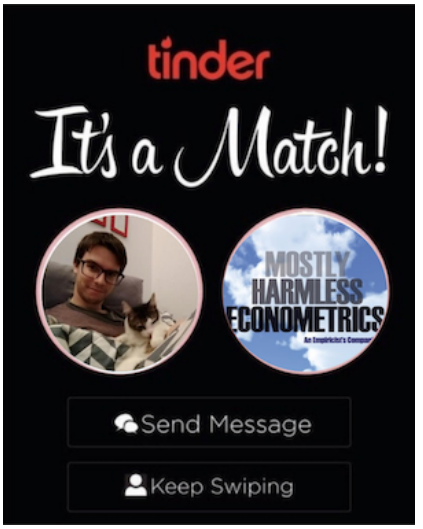
$$ATE = \int (E[Y \,|\, X, T = 1] - E[Y \,|\, X, T = 0]) dP(x)$$

What this integral does is it goes through all the space of the distribution of features X, computes the difference in means for all those tiny spaces and combines everything into the ATE. Another way to see this is to think about a discrete set of features. In this case, we can say that the features X takes on K different cells $\{X_1, X_2, ..., X_k\}$ and what we are doing is computing the treatment effect in each cell and combining them into the ATE. In this discrete case, converting the integral to a sum, we can derive the subclassifications estimator

$$\hat{ATE} = \sum_{k=1}^{K} (\bar{Y}_{k1} - \bar{Y}_{k0}) * \frac{N_k}{N}$$

where the bar represents the mean of the outcome on the treated, Y_{k1}, and non-treated, Y_{k0}, at cell k and N_k is the number of observations in that same cell. As you can see, we are computing a local ATE for each cell and combining them using a weighted average, where the weights are the sample size of the cell. In our medicine example above, this would be the first estimate, which gave us -2.6.

Matching Estimator



The subclassification estimator isn't used much in practice (we will see why shortly, it is because of the curse of dimensionality) but it gives us a nice intuition of what a causal inference estimator should do, how it should control for confounders. This allows us to explore other kinds of estimators, such as the Matching Estimator.

The idea is very similar. Since some sort of confounder X makes it so that treated and untreated are not initially comparable, I can make them so by **matching each treated unit with a similar untreated unit**. It is like I'm finding an untreated twin for every treated unit. By making such comparisons, treated and untreated become again comparable.

As an example, let's suppose we are trying to estimate the effect of a trainee program on earnings. Here is what the trainees looks like

```
trainee = pd.read_csv("./data/trainees.csv")
trainee.query("trainees==1")
```

	unit	trainees	age	earnings
0	1	1	28	17700
1	2	1	34	10200
2	3	1	29	14400
3	4	1	25	20800
4	5	1	29	6100
5	6	1	23	28600
6	7	1	33	21900
7	8	1	27	28800
8	9	1	31	20300
9	10	1	26	28100
10	11	1	25	9400
11	12	1	27	14300
12	13	1	29	12500
13	14	1	24	19700
14	15	1	25	10100
15	16	1	43	10700
16	17	1	28	11500
17	18	1	27	10700
18	19	1	28	16300

And here are the non-trainees:

```
trainee.query("trainees==0")
```

	unit	trainees	age	earnings
19	20	0	43	20900
20	21	0	50	31000
21	22	0	30	21000
22	23	0	27	9300
23	24	0	54	41100
24	25	0	48	29800
25	26	0	39	42000
26	27	0	28	8800
27	28	0	24	25500
28	29	0	33	15500
29	31	0	26	400
30	32	0	31	26600
31	33	0	26	16500
32	34	0	34	24200
33	35	0	25	23300
34	36	0	24	9700
35	37	0	29	6200
36	38	0	35	30200
37	39	0	32	17800
38	40	0	23	9500
39	41	0	32	25900

If I do a simple comparison in means, we get that the trainees earn less money than those that didn't go through the program.

```
trainee.query("trainees==1")["earnings"].mean() -
  ↪ trainee.query("trainees==0")["earnings"].mean()
```

```
-4297.49373433584
```

However, if we look at the table above, we notice that trainees are much younger than non trainees, which indicates that age is probably a confounder. Let's use matching on age to try to correct that. We will take unit 1 from the treated and pair it with unit 27, since both are 28 years old. Unit 2 we will pair it with unit 34, unit 3 with unit 37, unit 4 we will pair it with unit 35… When it comes to unit 5, we need to find someone with age 29 from the non treated, but that is unit 37, which is already paired. This is not a problem, since we can use the same unit multiple times. If more than 1 unit is a match, we can choose randomly between them.

This is what the matched dataset looks like for the first 7 units

```
# make dataset where no one has the same age
unique_on_age = (trainee
                 .query("trainees==0")
                 .drop_duplicates("age"))

matches = (trainee
           .query("trainees==1")
           .merge(unique_on_age, on="age", how="left", suffixes=("_t_1",
             ↪ "_t_0"))
           .assign(t1_minuts_t0 = lambda d: d["earnings_t_1"] -
             ↪ d["earnings_t_0"]))

matches.head(7)
```

	unit_t_1	trainees_t_1	age	earnings_t_1	unit_t_0	trainees_t_0	earnings_t_0	t1_minuts_t0
0	1	1	28	17700	27	0	8800	8900
1	2	1	34	10200	34	0	24200	-14000
2	3	1	29	14400	37	0	6200	8200
3	4	1	25	20800	35	0	23300	-2500
4	5	1	29	6100	37	0	6200	-100
5	6	1	23	28600	40	0	9500	19100
6	7	1	33	21900	29	0	15500	6400

Notice how the last column has the difference in earnings between the treated and its matched untreated unit. If we take the mean of this last column we get the ATET estimate while controlling for age. Notice how the estimate is now very positive, compared to the previous one where we used a simple difference in means.

```
matches["t1_minuts_t0"].mean()
```

```
2457.8947368421054
```

But this was a very contrived example, just to introduce matching. In reality, we usually have more than one feature and units don't match perfectly. In this case, we have to define some measurement of proximity to compare how units are close to each other. One common metric for this is the euclidean norm $||X_i - X_j||$. This difference, however, is not invariant to the scale of the features. This means that features like age, that take values in the tens, will be much less important when computing this norm compared to features like income, which take the order of thousands. For this reason, before applying the norm, we need to scale the features so that they are on roughly the same scale.

Having defined a distance measure, we can now define the match as the nearest neighbour to that sample we wish to match. In math terms, we can write the matching estimator the following way

$$\hat{ATE} = \frac{1}{N}\sum_{i=1}^{N}(2T_i - 1)\big(Y_i - Y_{jm}(i)\big)$$

Where $Y_{jm}(i)$ is the sample from the other treatment group which is most similar to Y_i. We do this $2T_i - 1$ to match both ways: treated with controls and controls with the treatment.

To test this estimator, let's consider a medicine example. Once again, we want to find the effect of a medication on days until recovery. Unfortunately, this effect is confounded by severity, sex and age. We have reasons to believe that patients with more severe conditions have a higher chance of receiving the medicine.

```
med = pd.read_csv("./data/medicine_impact_recovery.csv")
med.head()
```

	sex	age	severity	medication	recovery
0	0	35.049134	0.887658	1	31
1	1	41.580323	0.899784	1	49
2	1	28.127491	0.486349	0	38
3	1	36.375033	0.323091	0	35
4	0	25.091717	0.209006	0	15

If we look at a simple difference in means, $E[Y \mid T = 1] - E[Y \mid T = 0]$, we get that the treatment takes, on average, 16.9 more days to recover than the untreated. This is probably due to confounding, since we don't expect the medicine to cause harm to the patient.

```
med.query("medication==1")["recovery"].mean() -
  ↪ med.query("medication==0")["recovery"].mean()
```

```
16.895799546498726
```

To correct for this bias, we will control for X using matching. First, we need to remember to scale our features, otherwise, features like age will have higher importance than features like severity when we compute the distance between points. To do so, we can standardise the features.

```
# scale features
X = ["severity", "age", "sex"]
y = "recovery"

med = med.assign(**{f: (med[f] - med[f].mean())/med[f].std() for f in X})
med.head()
```

	sex	age	severity	medication	recovery
0	-0.996980	0.280787	1.459800	1	31
1	1.002979	0.865375	1.502164	1	49
2	1.002979	-0.338749	0.057796	0	38
3	1.002979	0.399465	-0.512557	0	35
4	-0.996980	-0.610473	-0.911125	0	15

Now, to the matching itself. Instead of coding a matching function, we will use the K nearest neighbour algorithm from Sklearn[1]. This algorithm makes predictions by finding the nearest data point in an estimation or training set.

For matching, we will need 2 of those. One, `mt0`, will store the untreated points and will find matches in the untreated when asked to do so. The other, `mt1`, will store the treated point and will find matches in the treated when asked to do so. After this fitting step, we can use these KNN models to make predictions, which will be our matches.

```
from sklearn.neighbors import KNeighborsRegressor

treated = med.query("medication==1")
untreated = med.query("medication==0")

mt0 = KNeighborsRegressor(n_neighbors=1).fit(untreated[X], untreated[y])
mt1 = KNeighborsRegressor(n_neighbors=1).fit(treated[X], treated[y])

predicted = pd.concat([
    # find matches for the treated looking at the untreated knn model
    treated.assign(match=mt0.predict(treated[X])),

    # find matches for the untreated looking at the treated knn model
    untreated.assign(match=mt1.predict(untreated[X]))
])

predicted.head()
```

	sex	age	severity	medication	recovery	match
0	-0.996980	0.280787	1.459800	1	31	39.0
1	1.002979	0.865375	1.502164	1	49	52.0
7	-0.996980	1.495134	1.268540	1	38	46.0
10	1.002979	-0.106534	0.545911	1	34	45.0
16	-0.996980	0.043034	1.428732	1	30	39.0

1 https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsRegressor.html

With the matches, we can now apply the matching estimator formula

$$\hat{ATE} = \frac{1}{N}\sum_{i=1}^{N}(2T_i - 1)\big(Y_i - Y_{jm}(i)\big)$$

```
np.mean((2*predicted["medication"] - 1)*(predicted["recovery"] -
  ↪ predicted["match"]))
```

```
-0.9954
```

Using this sort of matching, we can see that the effect of the medicine is not positive anymore. This means that, controlling for X, the medicine reduces the recovery time by about 1 day, on average. This is already a huge improvement on top of the biased estimate that predicted a 16.9 increase in recovery time.

However, we can still do better.

Matching Bias

It turns out the matching estimator as we've designed above is biased. To see this, let's consider the ATET estimator, instead of the ATE, just because it is simpler to write. The intuition will apply to the ATE as well.

$$\hat{ATET} = \frac{1}{N_1}\sum(Y_i - Y_j(i))$$

where N_1 is the number of treated individuals and $Y_j(i)$ is the untreated match of treated unit i. To check for bias, what we do is hope we can apply the Central Limit Theorem so that this down there converges to a normal distribution with mean zero.

$$\sqrt{N_1}(\hat{ATET} - ATET)$$

However, this doesn't always happen. If we define the mean outcome for the untreated given X, $\mu_0(x) = E[Y \mid X = x, T = 0]$, we will have that (btw, I've omitted the proof for that because it's a little beyond the point here).

$$E[\sqrt{N_1}(\hat{ATET} - ATET)] = E[\sqrt{N_1}(\mu_0(X_i) - \mu_0(X_j(i)))]$$

Now, $\mu_0(X_i) - \mu_0(X_j(i))$ is not so simple to understand, so let's look at it more carefully. $\mu_0(X_i)$ is the outcome Y value of a treated unit i had it not been treated. So, it is the counterfactual outcome Y_0 for unit i. $\mu_0(X_j(i))$ is the outcome of the untreated unit j that is the match of unit i. So, it is also the Y_0 , but for unit j now. Only this time, it is a factual outcome, because j is in the non treated group. Now, because j and i are only similar, but not the same, this will likely not be zero. In other words, $X_i \approx X_j$. So, $Y_{0i} \approx Y_{0j}$.

As we increase the sample size, there will be more units to match, so the difference between unit i and its match j will also get smaller. But this difference converges to zero

slowly. As a result $E[\sqrt{N_1}(\mu_0(X_i) - \mu_0(X_j(i)))]$ may not converge to zero, because the $\sqrt{N_1}$ grows faster than $(\mu_0(X_i) - \mu_0(X_j(i)))$ diminishes.

Bias arises when the matching discrepancies are huge. Fortunately, we know how to correct it. Each observation contributes $(\mu_0(X_i) - \mu_0(X_j(i)))$ to the bias so all we need to do is subtract this quantity from each matching comparison in our estimator. To do so, we can replace $\mu_0(X_j(i))$ with some sort of estimate of this quantity $\hat{\mu_0}(X_j(i))$, which can be obtained with models like linear regression. This updates the ATET estimator to the following equation

$$\hat{ATET} = \frac{1}{N_1}\sum\left((Y_i - Y_{j(i)}) - (\hat{\mu_0}(X_i) - \hat{\mu_0}(X_{j(i)}))\right)$$

where $\hat{\mu_0}(x)$ is some estimate of $E[Y \mid X, T = 0]$, like a linear regression fitted only on the untreated sample.

```
from sklearn.linear_model import LinearRegression

# fit the linear regression model to estimate mu_0(x)
ols0 = LinearRegression().fit(untreated[X], untreated[y])
ols1 = LinearRegression().fit(treated[X], treated[y])

# find the units that match to the treated
treated_match_index = mt0.kneighbors(treated[X], n_neighbors=1)[1].ravel()

# find the units that match to the untreatd
untreated_match_index = mt1.kneighbors(untreated[X], n_neighbors=1)[1].ravel()

predicted = pd.concat([
    (treated
     # find the Y match on the other group
     .assign(match=mt0.predict(treated[X]))

     # build the bias correction term
     .assign(bias_correct=ols0.predict(treated[X]) - ols0.predict(untreated
       ↪ .iloc[treated_match_index][X]))),
    (untreated
     .assign(match=mt1.predict(untreated[X]))
     .assign(bias_correct=ols1.predict(untreated[X]) - ols1.predict(treated
       ↪ .iloc[untreated_match_index][X])))
])

predicted.head()
```

	sex	age	severity	medication	recovery	match	bias_correct
0	-0.996980	0.280787	1.459800	1	31	39.0	4.404034
1	1.002979	0.865375	1.502164	1	49	52.0	12.915348
7	-0.996980	1.495134	1.268540	1	38	46.0	1.871428
10	1.002979	-0.106534	0.545911	1	34	45.0	-0.496970
16	-0.996980	0.043034	1.428732	1	30	39.0	2.610159

One immediate question that arises is: doesn't this defeat the point of matching? If I have to run a linear regression anyway, why don't I use only that, instead of this complicated model. That's a fair point, so I should take some time to answer it.

First of all, this linear regression that we are fitting doesn't extrapolate on the treatment dimension to get the treatment effect. Instead, its purpose is just to correct bias. Linear regression here is local, in the sense that it doesn't try to see how the treated would be if it looked like the untreated. It does none of that extrapolation. This is left to the matching part. The meat of the estimator is still the matching component. The point I want to make here is that OLS is secondary to this estimator.

The second point is that matching is a non-parametric estimator. It doesn't assume linearity or any kind of parametric model. As such, it is more flexible than linear regression and can work in situations where linear regression will not, namely, those where non linearity is very strong.

Does this mean that you should only use matching? Well, that's a tough question. Alberto Abadie makes a case that yes, you should. It's more flexible and, once you have the code, equally simple to run. I'm not entirely convinced by that. For example, Abadie spent a

lot of time studying and developing the estimator (yes, he is one of the scientists that contributes to matching being what it is), so he obviously is personally invested in the method. Second, there is something about linear regression's simplicity that you don't see in matching. The partial derivative math of "holding everything else constant" is much easier to grasp with linear regression than with matching. But that's just my preference. To be honest, there is no clear answer to this question. Anyway, back to our example.

With the bias correction formula, I get the following ATE estimation.

```
np.mean((2*predicted["medication"] - 1)*((predicted["recovery"] -
  ↪ predicted["match"])-predicted["bias_correct"]))
```

```
-7.36266090614141
```

Of course, we also need to place a confidence interval around this measurement, but enough of math theory now. In practice, we can simply use someone else's code and just import a matching estimator. Here is one from the library causalinference[2].

```
from causalinference import CausalModel

cm = CausalModel(
    Y=med["recovery"].values,
    D=med["medication"].values,
    X=med[["severity", "age", "sex"]].values
)

cm.est_via_matching(matches=1, bias_adj=True)

print(cm.estimates)
```

```
Treatment Effect Estimates: Matching

                     Est.       S.e.          z      P>|z|      [95% Conf. int.]
--------------------------------------------------------------------------------
           ATE     -7.709      0.609    -12.649      0.000     -8.903     -6.514
           ATC     -6.665      0.246    -27.047      0.000     -7.148     -6.182
           ATT     -9.679      1.693     -5.717      0.000    -12.997     -6.361
```

Finally, we can say with confidence that our medicine does indeed lower the time someone spends at the hospital. The ATE estimate is just a little bit lower than mine, due to the difference in tie breaking of matches of knn sklearn implementation and the causalinference python package.

Before we close this topic, I just wanted to address the cause of bias in matching a little bit more. We saw that matching is biased when the unit and its match are not so similar. But what causes them to be so different?

2 https://github.com/laurencium/causalinference

The Curse of Dimensionality

As it turns out, the answer is quite simple and intuitive. It is easy to find people that match on a few characteristics, like sex. But if we add more characteristics, like age, income, city of birth and so on, it becomes harder and harder to find matches. In more general terms, the more features we have, the higher will be the distance between units and their matches.

This is not something that hurts only the matching estimator. It ties back to the subclassification estimator we saw earlier. Early on, in that contrived medicine example where with man and woman, it was quite easy to build the subclassification estimator. That was because we only had 2 cells: man and woman. But what would happen if we had more? Let's say we have 2 continuous features like age and income and we manage to discretise them into 5 buckets each. This will give us 25 cells, or 5^2. And what if we had 10 covariates with 3 buckets each? Doesn't seem like a lot right? Well, this would give us 59049 cells, or 3^{10}. It's easy to see how this can blow out of proportion pretty quickly. This is a phenomenon pervasive in all data science, which is called the **The Curse of Dimensionality**!!!

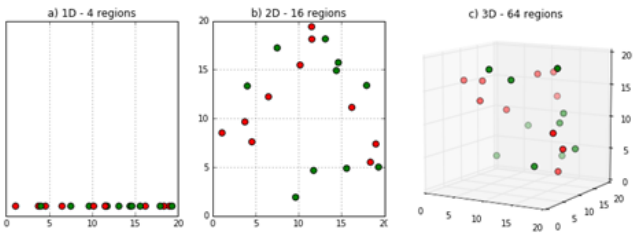


Image Source: https://deepai.org/machine-learning-glossary-and-terms/curse-of-dimensionality

Despite its scary and pretentious name, this only means that the number of data points required to fill a feature space grows exponentially with the number of features, or dimensions. So, if it takes X data points to fill the space of, say, 3 feature spaces, it takes exponentially more points to fill in the space of 4 features.

In the context of the subclassification estimator, the curse of dimensionality means that it will suffer if we have lots of features. Lots of features imply multiple cells in X. If there are multiple cells, some of them will have very few data. Some of them might even have only treated or only control, so it won't be possible to estimate the ATE there, which would break our estimator. In the matching context, this means that the feature space will be very sparse and units will be very far from each other. This will increase the distance between matches and cause bias problems.

As for linear regression, it actually handles this problem quite well. What it does is project all the features X into a single one, the Y dimension. It then makes treatment and control comparison on that projection. So, in some way, linear regression performs some sort of dimensionality reduction to estimate the ATE. It's quite elegant.

Most causal models also have some way to deal with the curse of dimensionality. I won't keep repeating myself, but it is something you should keep in mind when looking at them. For instance, when we deal with propensity scores in the following section, try to see how it solves this problem.

Key Ideas

We've started this section understanding what linear regression does and how it can help us identify causal relationships. Namely, we understood that regression can be seen as partitioning the dataset into cells, computing the ATE in each cell and then combining the cell's ATE into a single ATE for the entire dataset.

From there, we've derived a very general causal inference estimator with subclassification. We saw how that estimator is not very useful in practice but it gave us some interesting insights on how to tackle the problem of causal inference estimation. That gave us the opportunity to talk about the matching estimator.

Matching controls for the confounders by looking at each treated unit and finding an untreated pair that is very similar to it and similarly for the untreated units. We saw how to implement this method using the KNN algorithm and also how to debias it using regression. Finally, we discussed the difference between matching and linear regression. We saw how matching is a non-parametric estimator that doesn't rely on linearity the way linear regression does.

Finally, we've delved into the problem of high dimensional datasets and we saw how causal inference methods can suffer from it.

References

I like to think of this entire book as a tribute to Joshua Angrist, Alberto Abadie and Christopher Walters for their amazing Econometrics class. Most of the ideas here are taken from their classes at the American Economic Association. Watching them is what is keeping me sane during this tough year of 2020.

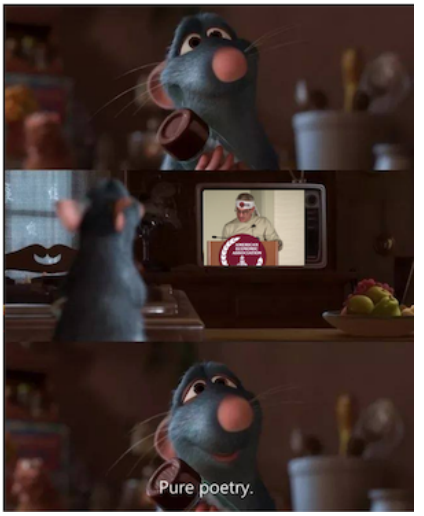
- Cross-Section Econometrics · https://www.aeaweb.org/conference/cont-ed/2017-webcasts
- Mastering Mostly Harmless Econometrics · https://www.aeaweb.org/conference/cont-ed/2020-webcasts

I'll also like to reference the amazing books from Angrist. They have shown me that Econometrics, or 'Metrics as they call it, is not only extremely useful but also profoundly fun.

- Mostly Harmless Econometrics · https://www.mostlyharmlesseconometrics.com/
- Mastering 'Metrics · https://www.masteringmetrics.com/

My final reference is Miguel Hernan and Jamie Robins' book. It has been my trustworthy companion in the most thorny causal questions I had to answer.

- Causal Inference Book · https://miguelhernan.org/whatifbook



CHAPTER
11

Propensity Score

The Psychology of Growth

The field of positive psychology studies what human behaviours lead to a great life. You can think of it as the intersection between self help books with the academic rigor of statistics. One of the famous findings of positive psychology is the **Growth Mindset**. The idea is that people can have a fixed or a growth mindset. If you have a fixed mindset, you believe that abilities are given at birth or in early childhood. As such, intelligence is fixed and can't change throughout life. If you don't have it by now, you can't acquire it. The corollary of this thought is that you should not waste time on areas where you don't excel, since you will never learn how to handle them. On the other hand, if you have a growth mindset, you believe that intelligence can be developed. The direct consequence of this is you see failure not as lack of tenacity, but as part of a learning process.

I don't want to debate which of these mindsets is the correct one (although it's probably somewhere in the middle). For our purpose, it doesn't matter much. What does matter is that psychologists found out that people who have a growth mindset tend to do better in life. They are more likely to achieve what they've set out to do.

As versed as we are with causal inference, we've learned to see those statements with skepticism. Is it that a growth mindset causes people to achieve more? Or is simply the case that people who achieve more are prone to develop a growth mindset as a result of their success? Who came first, the egg or the chicken? In potential outcome notation, we have reasons to believe that there is bias in these statements. $Y_0 \mid T = 1$ is probably larger than $Y_0 \mid T = 0$, which means that those with a growth mindset would have achieved more even if they had a fixed mindset.

To settle things, researchers designed the The National Study of Learning Mindsets[1]. It is a randomised study conducted in U.S. public high schools which aims at finding the impact of a growth mindset. The way it works is that students receive from the school a

1 https://mindsetscholarsnetwork.org/about-the-network/current-initatives/national-mindset-study/#

seminar to instil in them a growth mindset. Then, they follow up the students in their college years to measure how well they've performed academically. This measurement was compiled into an achievement score and standardized. The real data on this study is not publicly available in order to preserve students' privacy. However, we have a simulated dataset with the same statistical properties provided by Athey and Wager[2], so we will use that instead.

```
import warnings
warnings.filterwarnings('ignore')

import pandas as pd
import numpy as np
from matplotlib import style
from matplotlib import pyplot as plt
import seaborn as sns
import statsmodels.formula.api as smf
from causalinference import CausalModel

import graphviz as gr

%matplotlib inline

style.use("fivethirtyeight")
pd.set_option("display.max_columns", 6)
```

Besides the treated and outcome variables, the study also recorded some other features:

- schoolid: identifier of the student's school;
- success_expect: self-reported expectations for success in the future, a proxy for prior achievement, measured prior to random assignment;
- ethnicity: categorical variable for student race/ethnicity;
- gender: categorical variable for student identified gender;
- frst_in_family: categorical variable for student first-generation status, i.e. first in family to go to college;
- school_urbanicity: school-level categorical variable for urbanicity of the school, i.e. rural, suburban, etc;
- school_mindset: school-level mean of students' fixed mindsets, reported prior to random assignment, standardized;
- school_achievement: school achievement level, as measured by test scores and college preparation for the previous 4 cohorts of students, standardized;
- school_ethnic_minority: school racial/ethnic minority composition, i.e., percentage of student body that is Black, Latino, or Native American, standardized;
- school_poverty: school poverty concentration, i.e., percentage of students who are from families whose incomes fall below the federal poverty line, standardized;
- school_size: total number of students in all four grade levels in the school, standardized.

2 https://arxiv.org/pdf/1902.07409.pdf

```
data = pd.read_csv("./data/learning_mindset.csv")
data.sample(5, random_state=5)
```

	schoolid	intervention	achievement_score	...
259	73	1	1.480828	...
3435	76	0	-0.987277	...
9963	4	0	-0.152340	...
4488	67	0	0.358336	...
2637	16	1	1.360920	...

	school_ethnic_minority	school_poverty	school_size
259	-0.515202	-0.169849	0.173954
3435	-1.310927	0.224077	-0.426757
9963	0.875012	-0.724801	0.761781
4488	0.315755	0.054586	1.862187
2637	-0.033161	-0.982274	1.591641

5 rows × 13 columns

Although the study was randomised, it doesn't seem to be the case that this data is free from confounding. If we look at the additional features, we will notice that they vary systematically between treatment and control. One possible reason for this is that the treatment variable is measured by the student's receipt of the seminar. So, although the opportunity to participate was random, participation itself is not. We are dealing with a case of non-compliance here. One evidence of this is how the student's success expectation is correlated with the participation in the seminar. Students with higher self-reported success expectation are more likely to have joined the growth mindset seminar.

```
data.groupby("success_expect")["intervention"].mean()
```

```
success_expect
1    0.271739
2    0.265957
3    0.294118
4    0.271617
5    0.311070
6    0.354287
7    0.362319
Name: intervention, dtype: float64
```

Still, let's see what the difference in means $E[Y \mid T = 1] - E[Y \mid T = 0]$ looks like. This will be a useful baseline to compare against.

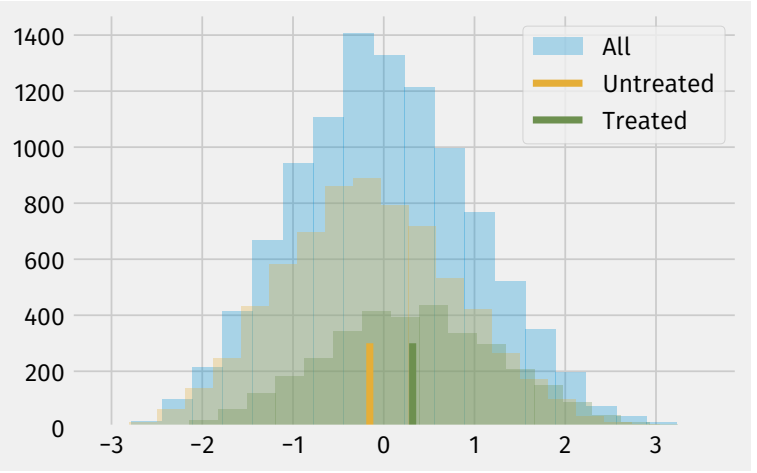
```
smf.ols("achievement_score ~ intervention", data=data).fit().summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	-0.1538	0.012	-13.201	0.000	-0.177	-0.131
intervention	0.4723	0.020	23.133	0.000	0.432	0.512

Simply comparing those with and without the intervention, we can see that the treated have an achievement score that is, on average, 0.3185 (0.4723 - 0.1538) higher than the average score (which is zero, since the score is standardized). But is this big or small? I know that interpreting standardized outcomes can be challenging, but bear with me for a moment. I think it is worth going through this because it won't be the last time you will encounter standardized scores.

The outcome variable being standardized means that it is measured in standard deviations. So, the treated are 0.3185 deviations above the untreated. That is what this means. As for if this is small or big, let's remember some stuff about the normal distribution. We know that 95% of its mass is between 2 standard deviations, leaving 2.5% on one tail and 2.5% on another. This also means that if someone is 2 standard deviations above the mean, 97.5% (95% plus the left 2.5% tail) of all the individuals are below that person. By looking at the normal CDF, we also know that about 85% of its mass is below 1 standard deviation and that 70% of its mass is below 0.5 standard deviations. Since the treated group has an average standardized score of about 0.5, this means that they fall above 70% in terms of individual achievement. Or, in other words, they are in the top 30% who achieve more. Here is what this looks like in a picture.

```
plt.hist(data["achievement_score"], bins=20, alpha=0.3, label="All")
plt.hist(data.query("intervention==0")["achievement_score"], bins=20, alpha=0.3,
  ↪ color="C2")
plt.hist(data.query("intervention==1")["achievement_score"], bins=20, alpha=0.3,
  ↪ color="C3")
plt.vlines(-0.1538, 0, 300, label="Untreated", color="C2")
plt.vlines(-0.1538+0.4723, 0, 300, label="Treated", color="C3")
plt.legend();
```



Of course, we still think this result is biased. The difference between treated and untreated is probably smaller than this, because we think the bias is positive. We've already seen that more ambitious people are more willing to go to the seminar, so they probably would have achieved more even if they had not attended it. To control for this bias, we could use regression or matching, but it's time to learn about a new technique.

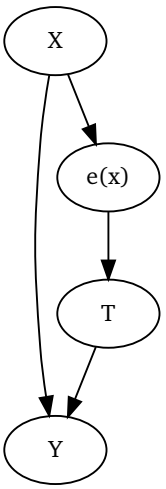
Propensity Score

Propensity score comes from the realisation that you don't need to directly control for confounders X to achieve conditional independence $(Y_1, Y_0) \perp T|X$. Instead, it is sufficient to control for a balancing score $E[T|X]$. This balancing score is often the conditional probability of the treatment, $P(T|X)$, also called the propensity score $e(x)$. The propensity score makes it so that you don't have to condition on the entirety of X to achieve independence of the potential outcomes on the treatment. It is sufficient to condition on this single variable, which is the propensity score:

$$(Y_1, Y_0) \perp T|e(x)$$

There is a formal proof for why this is, but we can forget it for now and approach the matter in a more intuitive way. The propensity score is the conditional probability of receiving the treatment, right? So we can think of it as some sort of function that converts X into the treatment T. The propensity score makes this middle ground between the variable X and the treatment T. If we show this in a causal graph, this is what it would look like.

```
g = gr.Digraph()
g.edge("T", "Y")
g.edge("X", "Y")
g.edge("X", "e(x)")
g.edge("e(x)", "T")
g
```



If I know what e(x) is, X alone tells me nothing more that can help me learn what T would be. Which means that controlling for e(x) acts the same way as controlling for X directly. Think of it in terms of our mindset program. Treated and non treated are initially not comparable because the more ambitious are both more likely to take the treatment and of achieving more in life. However, if I take 2 individuals, one from the treated and one from the control, but with the same probability of receiving the treatment, they are comparable. Think about it. If they have the exact same probability of receiving the treatment, the only reason one of them received it and the other did not is pure chance. Holding the propensity score constant acts in a way of making the data look as good as random.

Now that we got the intuition, let's look at the mathematical proof. We want to show that $(Y_1, Y_0) \perp T \,|\, e(x)$ is equivalent to saying that

$$E[T \,|\, e(x), X] = E[T \,|\, e(x)]$$

This simply says that once I condition on $e(x)$, X can give me no extra information about T. The proof of this is quite weird. We will show that the equation above is true by converting it to a trivial statement. First take a look at the left hand side $E[T \,|\, e(x), X]$.

$$E[T \,|\, e(x), X] = E[T \,|\, X] = e(x)$$

We use the fact that $e(x)$ is just a function of X, so conditioning on it gives no further information after we've conditioned on X itself. Then, we use the definition of the propensity score $E[T \,|\, X]$.

For the right hand side, we will use the law of iterated expectations $E[A] = E[E[A \,|\, B]]$. This law says that we can compute the expected value of A by looking at the value of A broken down by B and then averaging that.

$$E[T \,|\, e(x)] = E[E[T \,|\, e(x), X] \,|\, e(x)] = E[e(x) \,|\, e(x)] = e(x)$$

The first equality comes from the law of iterated expectations. The second comes from what we've figured out when dealing with the left hand side. Since both the left and right hand side equals, $e(x)$, this equation is trivially true.

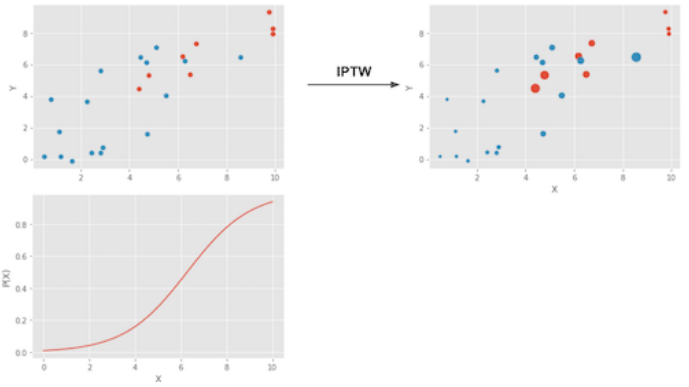
Propensity Weighting

OK, we got the propensity score. Now what? Like I've said, all we need to do is condition on it. For example, we could run a linear regression that conditions only on the propensity score, instead of all the Xs. For now, let's look at a technique that just uses the propensity score and nothing else. The idea is to write the conditional difference in means with the propensity score

$$E[Y \mid X, T = 1] - E[Y \mid X, T = 0] = E\left[\frac{Y}{e(x)} \mid X, T = 1\right]P(T) - E\left[\frac{Y}{(1 - e(x))} \mid X, T = 0\right](1 - P(T))$$

We can simplify this further, but let's take a look at it like this because it gives us some nice intuition of what the propensity score is doing. The first term is estimating Y_1. It is taking all those that are treated and scaling them by the inverse probability of treatment. What this does is it makes those with very low probability of treatment have a high weight. This makes sense, right? If someone has a low probability of treatment, that individual looks like the untreated. However, that same individual was treated. This must be interesting. We have a treated that looks like the untreated, so we will give that entity a high weight. This creates a population with the same size as the original, but where everyone is treated. By the same reasoning, the other term looks at the untreated and gives a high weight to those that look like the treated. This estimator is called the Inverse Probability of Treatment Weighting (IPTW), since it scales each unit by the inverse probability of the treatment it received.

In a picture, here is what this weighting does.



The upper left plot shows the original data. The blue dots are the untreated and the red dots are the treated. The bottom plot shows the propensity score $e(x)$. Notice how it is between 0 and 1 and it grows as X increases. Finally, the upper right plot is the data after weighting. Notice how the red (treated) that are more to the left (lower propensity score) have a higher weight. Similarly, the blue plots that are to the right have also a higher weight.

Now that we got the intuition, we can simplify the terms above to

$$
\begin{aligned}
E[Y\,|\,X,T=1]-E[Y\,|\,X,T=0] &= E\left[\frac{Y}{e(x)}\,|\,X,T=1\right]P(T)-E\left[\frac{Y}{(1-e(x))}\,|\,X,T=0\right](1-P(T)) \\
&= E\left[\frac{YT}{e(x)}\middle|X\right]-E\left[\frac{Y(1-T)}{(1-e(x))}\middle|X\right] \\
&= E\left[\frac{YT}{e(x)}-\frac{Y(1-T)}{(1-e(x))}\middle|X\right] \\
&= E\left[Y\frac{T(1-e(x))-e(x)(1-T)}{e(x)(1-e(x))}\middle|X\right] \\
&= E\left[Y\frac{T-e(x)}{e(x)(1-e(x))}\middle|X\right]
\end{aligned}
$$

which if we integrate over X becomes our propensity score weighting estimator.

$$E\left[Y\frac{T-e(x)}{e(x)(1-e(x))}\right]$$

Notice that this estimator requires that $e(x)$ and $1-e(x)$ are larger than zero. In words, this means that everyone needs to have at least some chance of receiving the treatment and of not receiving it. Another way of stating this is that the treated and untreated distributions need to overlap. This is the **positivity assumption** of causal inference. It also makes intuitive sense. If treated and untreated don't overlap, it means they are very different and I won't be able to extrapolate the effect of one group to the other. This extrapolation is not impossible (regression does it), but it is very dangerous. It is like testing a new drug in an experiment where only men receive the treatment and then assume women will respond to it equally well.

Propensity Score Estimation

In an ideal world, we would have the true propensity score $e(x)$. However, in practice, the mechanism that assigns the treatment is unknown and we need to replace the true propensity score by an estimation of it $\hat{e}(x)$. One common way of doing so is using logistic regression, but other machine learning methods, like gradient boosting, can be used as well (although it requires some additional steps to avoid overfitting).

Here, I'll stick to logistic regression. This means that I'll have to convert the categorical features in the dataset to dummies.

```
categ = ["ethnicity", "gender", "school_urbanicity"]
cont = ["school_mindset", "school_achievement", "school_ethnic_minority",
  ↪ "school_poverty", "school_size"]

data_with_categ = pd.concat([
    data.drop(columns=categ), # dataset without the categorical features
    pd.get_dummies(data[categ], columns=categ,
      ↪ drop_first=False)# categorical features converted to dummies
], axis=1)

print(data_with_categ.shape)
```

```
(10391, 32)
```

Now, let's estimate the propensity score using logistic regression.

```
from sklearn.linear_model import LogisticRegression

T = 'intervention'
Y = 'achievement_score'
X = data_with_categ.columns.drop(['schoolid', T, Y])

ps_model = LogisticRegression(C=1e6).fit(data_with_categ[X], data_with_categ[T])

data_ps = data.assign(propensity_score=ps_model.predict_proba(data_with_categ[
  ↪ X])[:, 1])

data_ps[["intervention", "achievement_score", "propensity_score"]].head()
```

	intervention	achievement_score	propensity_score
0	1	0.277359	0.315486
1	1	-0.449646	0.263801
2	1	0.769703	0.344024
3	1	-0.121763	0.344024
4	1	1.526147	0.367784

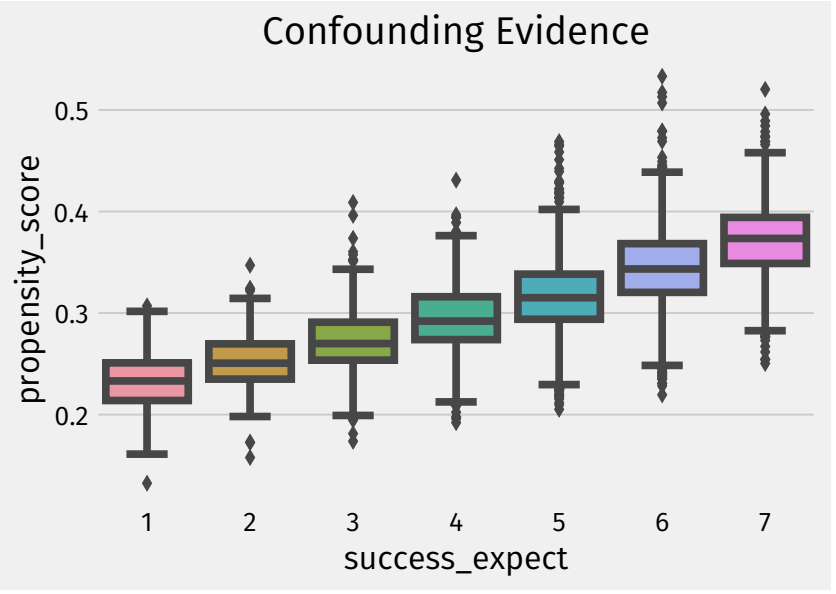
First, we can make sure that the propensity score weight indeed reconstructs a population where everyone is treated. By producing weights $1/e(x)$, it creates the population where everyone is treated and by providing the weights $1/(1-e(x))$ it creates the population where everyone is untreated.

```
weight_t = 1/data_ps.query("intervention==1")["propensity_score"]
weight_nt = 1/(1-data_ps.query("intervention==0")["propensity_score"])
print("Original Sample Size", data.shape[0])
print("Treated Population Sample Size", sum(weight_t))
print("Untreated Population Sample Size", sum(weight_nt))
```

```
Original Sample Size 10391
Treated Population Sample Size 10388.565115367259
Untreated Population Sample Size 10391.440761283782
```

We can also use the propensity score to find evidence of confounding. If a segmentation of the population has a higher propensity score than another, it means that something which is not random is causing the treatment. If that same thing is also causing the outcome, we have confounding. In our case, we can see that students that reported to be more ambitious also have a higher probability of attending the growth mindset seminar.

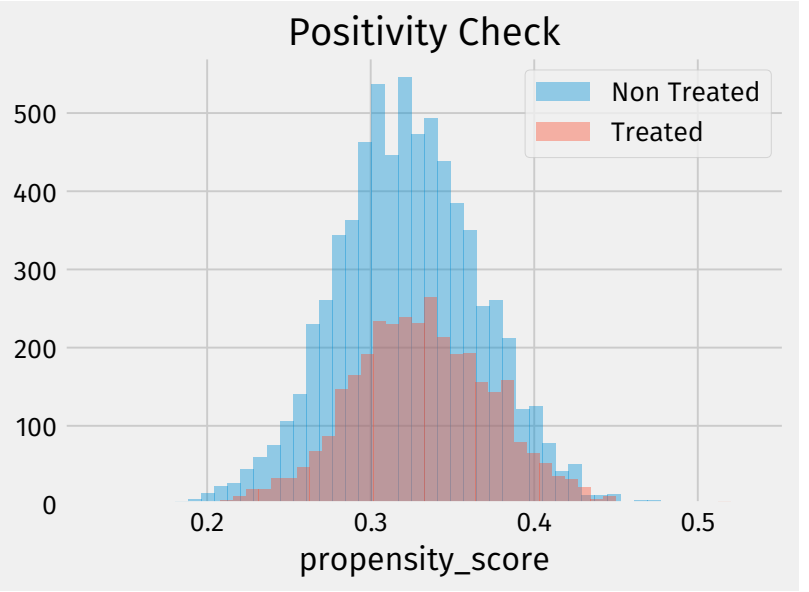
```
sns.boxplot(x="success_expect", y="propensity_score", data=data_ps)
plt.title("Confounding Evidence");
```



We also have to check that there is overlap between the treated and untreated population. To do so, we can see the empirical distribution of the propensity score on the untreated and on the treated. Looking at the image below, we can see that no one has a propensity

score of zero and that even in lower regions of the propensity score we can find both treated and untreated individuals. This is what we call a nicely balanced treated and untreated population.

```
sns.distplot(data_ps.query("intervention==0")["propensity_score"], kde=False,
  ↪ label="Non Treated")
sns.distplot(data_ps.query("intervention==1")["propensity_score"], kde=False,
  ↪ label="Treated")
plt.title("Positivity Check")
plt.legend();
```



Finally, we can use our propensity score weighting estimator to estimate the average treatment effect.

```
weight = ((data_ps["intervention"]-data_ps["propensity_score"]) /
          (data_ps["propensity_score"]*(1-data_ps["propensity_score"])))

y1 = sum(data_ps.query("intervention==1")["achievement_score"]*weight_t) /
  ↪ len(data)
y0 = sum(data_ps.query("intervention==0")["achievement_score"]*weight_nt) /
  ↪ len(data)

ate = np.mean(weight * data_ps["achievement_score"])

print("Y1:", y1)
print("Y0:", y0)
print("ATE", ate)
```

```
Y1: 0.2595823130696891
Y0: -0.1289241686986528
ATE 0.38850648176834174
```

Propensity score weighting is saying that we should expect treated individuals to be 0.38 standard deviations above their untreated fellows, in terms of achievements. We can also see that if no one got the treatment, we should expect the general level of achievements to be 0.12 standard deviation lower than what it is now. By the same reasoning, we should expect the general level of achievement to be 0.25 standard deviations higher if we've given everyone the seminar. Contrast this to the 0.47 ATE estimate we've got by simply comparing treated and untreated. This is evidence that the bias we have is indeed positive and that controlling for X gives us a more modest estimate of the impact of the growth mindset.

Standard Error

To compute the standard error for the IPTW estimator, we can use the formula of the variance of a weighted average.

$$\sigma_w^2 = \frac{\sum_{i=1}^{n} w_i (y_i - \hat{\mu})^2}{\sum_{i=1}^{n} w_i}$$

However, we can only use this if we have the true propensity score. If we are using the estimated version of it, $\hat{P}(x)$, we need to account for the errors in this estimation process. The easiest way of doing this is by bootstrapping the whole procedure. This is achieved

by sampling with replacement from the original data and computing the ATE like we did above. We then repeat this many times to get the distribution of the ATE estimate.

```
from joblib import Parallel, delayed # for parallel processing

# define function that computes the IPTW estimator
def run_ps(df, X, T, y):
    # estimate the propensity score
    ps = LogisticRegression(C=1e6, max_iter=1000).fit(df[X],
      ↪ df[T]).predict_proba(df[X])[:, 1]

    weight = (df[T]-ps) / (ps*(1-ps)) # define the weights
    return np.mean(weight * df[y]) # compute the ATE

np.random.seed(88)
# run 1000 bootstrap samples
bootstrap_sample = 1000
ates = Parallel(n_jobs=4)(delayed(run_ps)(data_with_categ.sample(frac=1,
  ↪ replace=True), X, T, Y)
                          for _ in range(bootstrap_sample))
ates = np.array(ates)
```

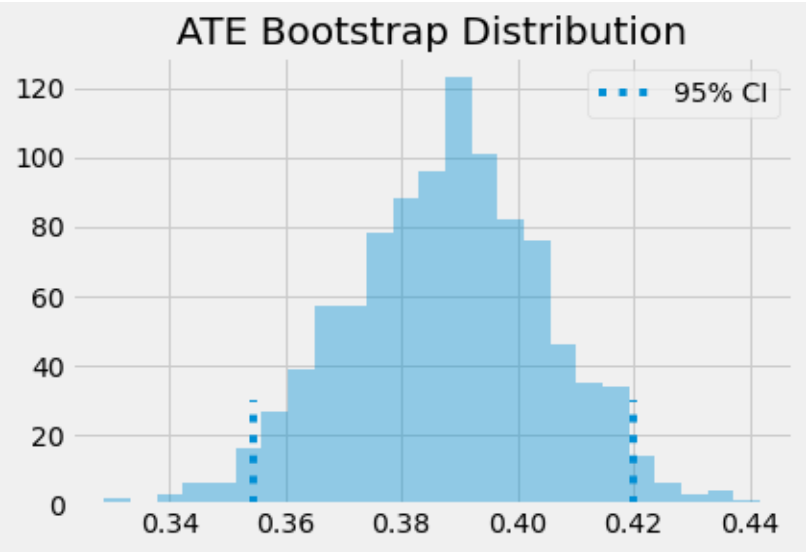
The ATE is then the mean of the bootstrap samples. To get confidence intervals, we can inspect the quantiles of the bootstrap distribution. For the 95% C.I., we use the 2.5 and 97.5 percentiles.

```
print(f"ATE: {ates.mean()}")
print(f"95% C.I.: {(np.percentile(ates, 2.5), np.percentile(ates, 97.5))}")
```

```
ATE: 0.3877446247220722
95% C.I.: (0.3545132414290843, 0.41992560836402076)
```

We can also have a visual on what the bootstrap samples look like, along with the confidence intervals.

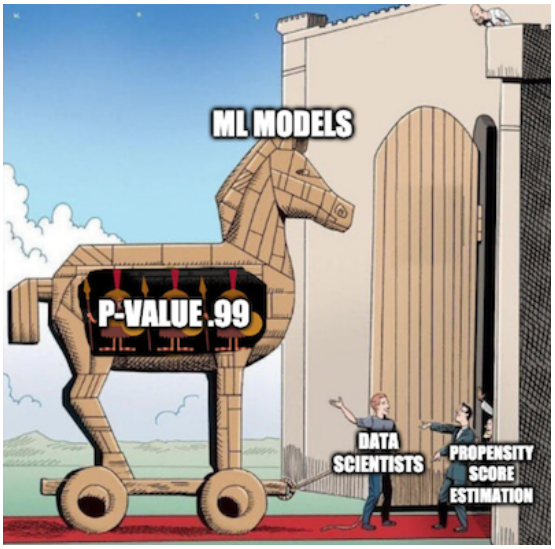
```
sns.distplot(ates, kde=False)
plt.vlines(np.percentile(ates, 2.5), 0, 30, linestyles="dotted")
plt.vlines(np.percentile(ates, 97.5), 0, 30, linestyles="dotted",
  ↪ label="95% CI")
plt.title("ATE Bootstrap Distribution")
plt.legend();
```



Common Issues with Propensity Score

As a data scientist, I know it can be tempting to use all the power of the machine learning toolkit to make propensity score estimation as precise as possible. You can quickly get taken away by the all AUC optimisation, cross validation and bayesian hyper-parameter tuning. Now, I'm not saying you shouldn't do that. In fact, all of the theory about propensity score and machine learning is very recent, so there are lots we don't know yet. But it pays to understand something first.

The first thing is that the predictive quality of the propensity score does not translate into its balancing properties. Coming from the field of machine learning, one of the most challenging aspects of getting acquainted with causal inference is letting go of treating everything as a prediction problem. In fact, maximising the prediction power of the propensity score can even hurt the causal inference goal. **Propensity score doesn't need to predict the treatment very well. It just needs to include all the confounding variables**. If we include variables that are very good in predicting the treatment but have no bearing on the outcome this will actually increase the variance of the propensity score estimator. This is similar to the problem linear regression faces when we include variables correlated with the treatment but not with the outcome.



To see this, consider the following example (adapted from Hernán's Book). You have 2 schools, one of them applies the growth mindset seminar to 99% of its students and the other to 1%. Suppose that the schools have no impact on the treatment effect (except through the treatment), so it's not necessary to control for it. If you add the school variable to the propensity score model, it's going to have a very high predictive power. However, by chance, we could end up with a sample where everyone in school A got the treatment, leading to a propensity score of 1 for that school, which would lead to an infinite variance. This is an extreme example, but let's see how it would work with simulated data.

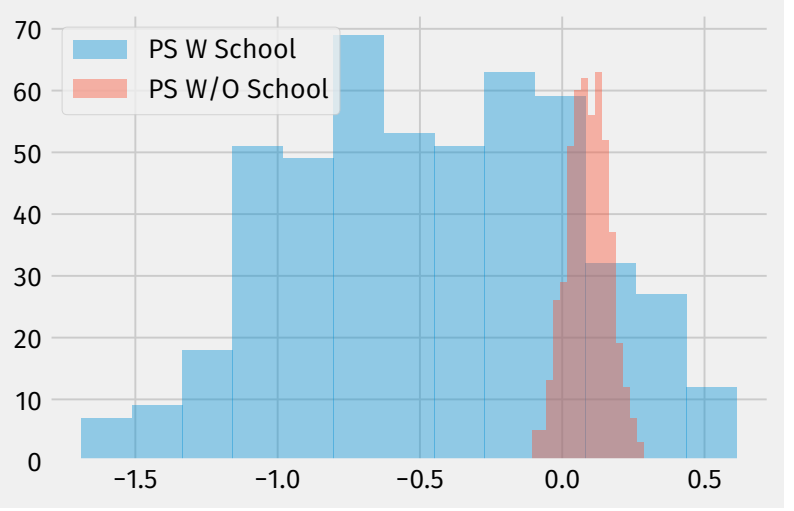
```
np.random.seed(42)
school_a = pd.DataFrame(dict(T=np.random.binomial(1, .99, 400), school=0,
  ↪ intercept=1))
school_b = pd.DataFrame(dict(T=np.random.binomial(1, .01, 400), school=1,
  ↪ intercept=1))
ex_data = pd.concat([school_a, school_b]).assign(y = lambda d: np.random.normal(
  ↪ 1 + 0.1 * d["T"]))
ex_data.head()
```

	T	school	intercept	y
0	1	0	1	0.309526
1	1	0	1	1.571468
2	1	0	1	2.982024
3	1	0	1	2.445420
4	1	0	1	2.693187

Having simulated this data, we run bootstrap with the Propensity Score algorithm twice. The first including school as a feature to the propensity score model. The second time, we don't include school in the model.

```
ate_w_f = np.array([run_ps(ex_data.sample(frac=1, replace=True), ["school"],
  ↪ "T", "y") for _ in range(500)])
ate_wo_f = np.array([run_ps(ex_data.sample(frac=1, replace=True), ["intercept"],
  ↪ "T", "y") for _ in range(500)])
```

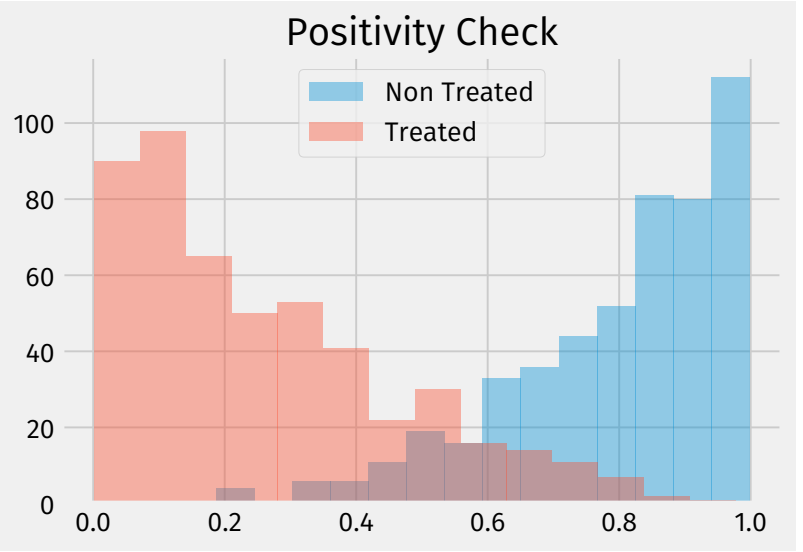
```
sns.distplot(ate_w_f, kde=False, label="PS W School")
sns.distplot(ate_wo_f, kde=False, label="PS W/O School")
plt.legend();
```



As you can see, the propensity score estimator that adds the feature school has a humongous variance, while the one without it is much more well behaved. Also, since school is not a confounder, the model without it is also not biased. As I've said, simply predicting the treatment is not what this is about. We actually need to construct the prediction in a way that controls for confounding, not in a way to predict the treatment.

This leads to another problem often encountered in propensity score methods. In our mindset case, the data turned out to be very balanced. But this is not always the case. In some situations, the treated have a much higher probability of treatment than the untreated and the propensity score distribution doesn't overlap much.

```
sns.distplot(np.random.beta(4,1,500), kde=False, label="Non Treated")
sns.distplot(np.random.beta(1,3,500), kde=False, label="Treated")
plt.title("Positivity Check")
plt.legend();
```



If this happens, it means that positivity is not very strong. If a treated has a propensity score of, say, 0.9 and the maximum propensity score of the untreated is 0.7, we won't have any untreated to compare to the individual with the 0.9 propensity score. This lack of balancing can generate some bias, because we will have to extrapolate the treatment effect to unknown regions. Not only that, entities with very high or very low propensity scores have a very high weight, which increases variance. As a general rule of thumb, you are in trouble if any weight is higher than 20 (which happens with an untreated with propensity score of 0.95 or a treated with a propensity score of 0.05).

An alternative is clipping the weight to be at a maximum size of 20. This will decrease the variance, but it will actually generate more bias. To be honest, although this is a common practice to reduce variance, I don't really like it. You will never know if the bias you are inducing with clipping is too much. Also, if the distributions don't overlap, your data is probably not enough to make a causal conclusion anyway. To gain some further intuition about this, we can look at a technique that combines propensity score and matching

Propensity Score Matching

As I've said before, you don't need to control for X when you have the propensity score. It suffices to control for it. As such, you can think of the propensity score as performing a kind of dimensionality reduction on the feature space. It condenses all the features in X into a single treatment assignment dimension. For this reason, we can treat the propensity score as an input feature for other models. Take a regression model, for instance.

```
smf.ols("achievement_score ~ intervention + propensity_score",
  ↪ data=data_ps).fit().summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	-3.0770	0.065	-47.061	0.000	-3.205	-2.949
intervention	0.3930	0.019	20.974	0.000	0.356	0.430
propensity_score	9.0554	0.200	45.314	0.000	8.664	9.447

If we control for the propensity score, we now estimate an ATE of 0.39, which is lower than the 0.47 we got previously with a regression model without controlling for the propensity score. We can also use matching on the propensity score. This time, instead of trying to find matches that are similar in all the X features, we can find matches that just have the same propensity score.

This is a huge improvement on top of the matching estimator, since it deals with the curse of dimensionality. Also, if a feature is unimportant for the treatment assignment, the propensity score model will learn that and give low importance to it when fitting the treatment mechanism. Matching on the features, on the other hand, would still try to find matches where individuals are similar on this unimportant feature.

```
cm = CausalModel(
    Y=data_ps["achievement_score"].values,
    D=data_ps["intervention"].values,
    X=data_ps[["propensity_score"]].values
)

cm.est_via_matching(matches=1, bias_adj=True)

print(cm.estimates)
```

```
Treatment Effect Estimates: Matching

                    Est.       S.e.          z      P>|z|      [95% Conf. int.]
--------------------------------------------------------------------------------
           ATE      0.389      0.025     15.579      0.000      0.340      0.438
           ATC      0.382      0.027     13.921      0.000      0.328      0.435
           ATT      0.405      0.027     15.175      0.000      0.353      0.457
```

As we can see, we also get an ATE of 0.38, which is more in line with what we've seen before with propensity score weighting. Matching on the propensity score also gives us some intuition about why it is dangerous to have a small overlap in the propensity score between treated and untreated. If this happens, the matching on the propensity score discrepancy will be large, which will lead to bias, as we've seen on the matching chapter.

One final word of caution here is that the above standard errors are wrong, as they don't account for the uncertainty in the estimation of the propensity score. Unfortunately, bootstrap doesn't work with matching[3]. Also, the theory above is so recent that there

3 https://economics.mit.edu/sites/default/files/publications/ON%20THE%20FAILURE%20OF%20THE%20BOOTSTRAP%20FOR.pdf

are no Python implementations of propensity score methods with the correct standard errors. For this reason, we don't see a lot of propensity score matching in Python.

Key Ideas

Here, we've learned that the probability of getting the treatment is called the propensity score and that we can use this as a balancing score. What this means is that, if we have the propensity score, we don't need to control for the confounders directly. It is sufficient to control for the propensity score in order to identify the causal effect. We saw how the propensity scores acts as a dimensionality reduction on the confounder space.

These properties allowed us to derive a weighting estimator for causal inference. Not only that, we saw how the propensity score can be used along other methods to control for confounding bias.

Then, we looked at some common issues that can arise with propensity score and with causal inference in general. The first one is when we get carried away by the task of fitting the treatment mechanism. We saw that, in a very counterintuitive (and hence easy to get it wrong) way, increasing the predictive performance of the treatment does **not** translate into a better causal estimate, as it can increase variance.

Finally, we looked at some extrapolation problems that we might run into if we are unable to have a good overlap between the treated and untreated propensity score distribution.

References

I like to think of this entire book as a tribute to Joshua Angrist, Alberto Abadie and Christopher Walters for their amazing Econometrics class. Most of the ideas here are taken from their classes at the American Economic Association. Watching them is what is keeping me sane during this tough year of 2020.

- Cross-Section Econometrics · https://www.aeaweb.org/conference/cont-ed/2017-webcasts
- Mastering Mostly Harmless Econometrics · https://www.aeaweb.org/conference/cont-ed/2020-webcasts

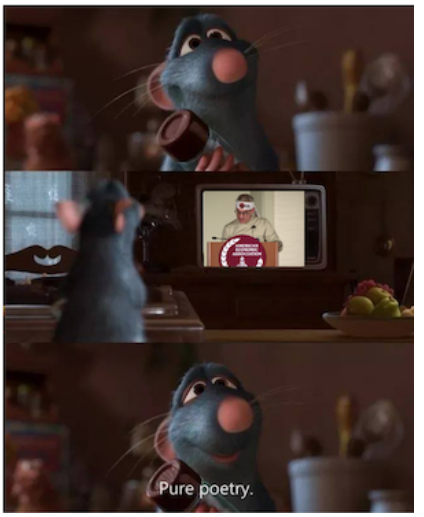
I'll also like to reference the amazing books from Angrist. They have shown me that Econometrics, or 'Metrics as they call it, is not only extremely useful but also profoundly fun.

- Mostly Harmless Econometrics · https://www.mostlyharmlesseconometrics.com/
- Mastering 'Metrics · https://www.masteringmetrics.com/

My final reference is Miguel Hernan and Jamie Robins' book. It has been my trustworthy companion in the most thorny causal questions I had to answer.

- Causal Inference Book · https://miguelhernan.org/whatifbook

The data that we used was taken from the article Estimating Treatment Effects with Causal Forests: An Application · https://arxiv.org/pdf/1902.07409.pdf, by Susan Athey and Stefan Wager.



CHAPTER
12

Doubly Robust Estimation

Don't Put All your Eggs in One Basket

We've learned how to use linear regression and propensity score weighting to estimate $E[Y \mid T = 1] - E[Y \mid T = 0] \mid X$. But which one should we use and when? When in doubt, just use both! Doubly Robust Estimation is a way of combining propensity score and linear regression in a way you don't have to rely on either of them.

To see how this works, let's consider the mindset experiment. It is a randomised study conducted in U.S. public high schools which aims at finding the impact of a growth mindset. The way it works is that students receive from the school a seminar to instil in them a growth mindset. Then, they follow up with the students in their college years to measure how well they performed academically. This measurement was compiled into an achievement score and standardised. The real data on this study is not publicly available in order to preserve students' privacy. However, we have a simulated dataset with the same statistical properties provided by Athey and Wager[1], so we will use that instead.

```
import warnings
warnings.filterwarnings('ignore')

import pandas as pd
import numpy as np
from matplotlib import style
from matplotlib import pyplot as plt
import seaborn as sns

from sklearn.linear_model import LogisticRegression, LinearRegression

%matplotlib inline
```

1 https://arxiv.org/pdf/1902.07409.pdf

```
style.use("fivethirtyeight")
pd.set_option("display.max_columns", 6)
```

```
data = pd.read_csv("./data/learning_mindset.csv")
data.sample(5, random_state=5)
```

	schoolid	intervention	achievement_score	...
259	73	1	1.480828	...
3435	76	0	-0.987277	...
9963	4	0	-0.152340	...
4488	67	0	0.358336	...
2637	16	1	1.360920	...

	school_ethnic_minority	school_poverty	school_size
259	-0.515202	-0.169849	0.173954
3435	-1.310927	0.224077	-0.426757
9963	0.875012	-0.724801	0.761781
4488	0.315755	0.054586	1.862187
2637	-0.033161	-0.982274	1.591641

5 rows × 13 columns

Although the study was randomised, it doesn't seem to be the case that this data is free from confounding. One possible reason for this is that the treatment variable is measured by the student's receipt of the seminar. So, although the opportunity to participate was random, participation is not. We are dealing with a case of non-compliance here. One evidence of this is how the student's success expectation is correlated with the participation in the seminar. Students with higher self-reported high expectations are more likely to have joined the growth mindset seminar.

```
data.groupby("success_expect")["intervention"].mean()
```

```
success_expect
1    0.271739
2    0.265957
3    0.294118
4    0.271617
5    0.311070
6    0.354287
7    0.362319
Name: intervention, dtype: float64
```

As we know by now, we could adjust for this using a linear regression or by estimating a propensity score model with a logistic regression. Before we do that, however, we need to convert the categorical variables to dummies.

```
categ = ["ethnicity", "gender", "school_urbanicity"]
cont = ["school_mindset", "school_achievement", "school_ethnic_minority",
  ↪ "school_poverty", "school_size"]

data_with_categ = pd.concat([
    data.drop(columns=categ), # dataset without the categorical features
    pd.get_dummies(data[categ], columns=categ, drop_first=False)
      ↪ # categorical features converted to dummies
], axis=1)

print(data_with_categ.shape)
```

```
(10391, 32)
```

We are now ready to understand how doubly robust estimation works.

Doubly Robust Estimation

Instead of deriving the estimator, I'll first show it to you and only then tell why it is awesome.

$$A\hat{T}E = \frac{1}{N}\sum\left(\frac{T_i(Y_i - \hat{\mu_1}(X_i))}{\hat{P}(X_i)} + \hat{\mu_1}(X_i)\right) - \frac{1}{N}\sum\left(\frac{(1-T_i)(Y_i - \hat{\mu_0}(X_i))}{1-\hat{P}(X_i)} + \hat{\mu_0}(X_i)\right)$$

where $\hat{P}(x)$ is an estimation of the propensity score (using logistic regression, for example), $\hat{\mu_1}(x)$ is an estimation of $E[Y \mid X, T = 1]$ (using linear regression, for example), and $\hat{\mu_0}(x)$

is an estimation of $E[Y \mid X, T = 0]$. As you might have already guessed, the first part of the doubly robust estimator estimates $E[Y_1]$ and the second part estimates $E[Y_0]$. Let's examine the first part, as all the intuition will also apply to the second part by analogy.

Since I know that this formula is scary at first (but don't worry, you will see it is super simple), I will first show how to code this estimator. I have the feeling that some people are less frightened by code than by formulas. Let's see how this estimator works in practice, shall we?

```
def doubly_robust(df, X, T, Y):
    ps = LogisticRegression(C=1e6, max_iter=1000).fit(df[X],
      ↪ df[T]).predict_proba(df[X])[:, 1]
    mu0 = LinearRegression().fit(df.query(f"{T}==0")[X],
      ↪ df.query(f"{T}==0")[Y]).predict(df[X])
    mu1 = LinearRegression().fit(df.query(f"{T}==1")[X],
      ↪ df.query(f"{T}==1")[Y]).predict(df[X])
    return (
        np.mean(df[T]*(df[Y] - mu1)/ps + mu1) -
        np.mean((1-df[T])*(df[Y] - mu0)/(1-ps) + mu0)
    )
```

```
T = 'intervention'
Y = 'achievement_score'
X = data_with_categ.columns.drop(['schoolid', T, Y])

doubly_robust(data_with_categ, X, T, Y)
```

```
0.38822121767832457
```

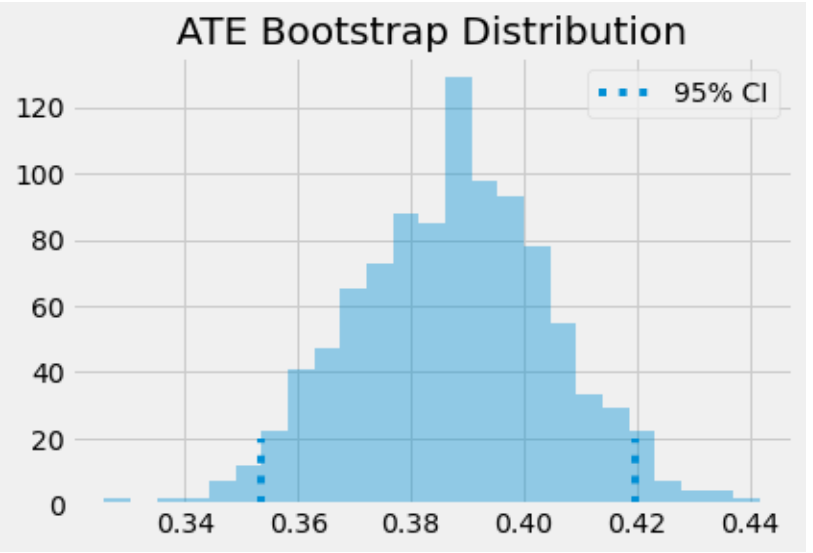
Doubly robust estimator is saying that we should expect individuals who attended the mindset seminar to be 0.388 standard deviations above their untreated fellows, in terms of achievements. Once again, we can use bootstrap to construct confidence intervals.

```
from joblib import Parallel, delayed # for parallel processing

np.random.seed(88)
# run 1000 bootstrap samples
bootstrap_sample = 1000
ates = Parallel(n_jobs=4)(delayed(doubly_robust)(data_with_categ.sample(frac=1,
  ↪ replace=True), X, T, Y)
                          for _ in range(bootstrap_sample))
ates = np.array(ates)
```

```
print(f"ATE 95% CI:", (np.percentile(ates, 2.5), np.percentile(ates, 97.5)))
```

```
ATE 95% CI: (0.35364752721776144, 0.41978441342477585)
```

```
sns.distplot(ates, kde=False)
plt.vlines(np.percentile(ates, 2.5), 0, 20, linestyles="dotted")
plt.vlines(np.percentile(ates, 97.5), 0, 20, linestyles="dotted",
  ↪ label="95% CI")
plt.title("ATE Bootstrap Distribution")
plt.legend();
```



Now that we got a taste of the doubly robust estimator, let's examine why it is so great. First, it is called doubly robust because it only requires one of the models, $\hat{P}(x)$ or $\hat{\mu}(x)$, to be correctly specified. To see this, take the first part that estimates $E[Y_1]$ and take a good look at it.

$$\hat{E}[Y_1] = \frac{1}{N}\sum\left(\frac{T_i(Y_i - \hat{\mu_1}(X_i))}{\hat{P}(X_i)} + \hat{\mu_1}(X_i)\right)$$

Assume that $\hat{\mu_1}(x)$ is correct. If the propensity score model is wrong, we wouldn't need to worry. Because if $\hat{\mu_1}(x)$ is correct, then $E[T_i(Y_i - \hat{\mu_1}(X_i))] = 0$. That is because the multiplication by T_i selects only the treated and the residual of $\hat{\mu_1}$ on the treated have, by definition, mean zero. This causes the whole thing to reduce to $\hat{\mu_1}(X_i)$, which is correctly estimated $E[Y_1]$ by assumption. So, you see, that by being correct, $\hat{\mu_1}(X_i)$ wipes out the relevance of the propensity score model. We can apply the same reasoning to understand the estimator of $E[Y_0]$.

But don't take my word for it. Let the code show you the way! In the following estimator, I've replaced the logistic regression that estimates the propensity score by a random uniform variable that goes from 0.1 to 0.9 (I don't want very small weights to blow up my propensity score variance). Since this is random, there is no way it is a good propensity score model, but we will see that the doubly robust estimator still manages to produce an estimation that is very close to when the propensity score was estimated with logistic regression.

```
from sklearn.linear_model import LogisticRegression, LinearRegression

def doubly_robust_wrong_ps(df, X, T, Y):
    # wrong PS model
    np.random.seed(654)
    ps = np.random.uniform(0.1, 0.9, df.shape[0])
    mu0 = LinearRegression().fit(df.query(f"{T}==0")[X],
      ↪ df.query(f"{T}==0")[Y]).predict(df[X])
    mu1 = LinearRegression().fit(df.query(f"{T}==1")[X],
      ↪ df.query(f"{T}==1")[Y]).predict(df[X])
    return (
        np.mean(df[T]*(df[Y] - mu1)/ps + mu1) -
        np.mean((1-df[T])*(df[Y] - mu0)/(1-ps) + mu0)
    )
```

```
doubly_robust_wrong_ps(data_with_categ, X, T, Y)
```

```
0.3797369830995927
```

If we use bootstrap, we can see that the variance is slightly higher than when the propensity score was estimated with a logistic regression.

```
np.random.seed(88)
parallel_fn = delayed(doubly_robust_wrong_ps)
wrong_ps = Parallel(n_jobs=4)(parallel_fn(data_with_categ.sample(frac=1,
  ↪ replace=True), X, T, Y)
                              for _ in range(bootstrap_sample))
wrong_ps = np.array(wrong_ps)
```

```
print(f"Original ATE 95% CI:", (np.percentile(ates, 2.5), np.percentile(ates,
  ↪ 97.5)))

print(f"Wrong PS ATE 95% CI:", (np.percentile(wrong_ps, 2.5),
  ↪ np.percentile(wrong_ps, 97.5)))
```

```
Original ATE 95% CI: (0.35364752721776144, 0.41978441342477585)
Wrong PS ATE 95% CI: (0.33806443306747086, 0.4335673822553228)
```

As we can see, messing up the propensity score yields slightly different ATEs, but not by much. This covers the case that the propensity model is wrong but the outcome model is correct. What about the other situation? Let's again take a good look at the first part of the estimator, but let's rearrange some terms

$$\hat{E}[Y_1] = \frac{1}{N}\sum\left(\frac{T_i(Y_i - \hat{\mu_1}(X_i))}{\hat{P}(X_i)} + \hat{\mu_1}(X_i)\right)$$

$$\hat{E}[Y_1] = \frac{1}{N}\sum\left(\frac{T_i Y_i}{\hat{P}(X_i)} - \frac{T_i \hat{\mu_1}(X_i)}{\hat{P}(X_i)} + \hat{\mu_1}(X_i)\right)$$

$$\hat{E}[Y_1] = \frac{1}{N}\sum\left(\frac{T_i Y_i}{\hat{P}(X_i)} - \left(\frac{T_i}{\hat{P}(X_i)} - 1\right)\hat{\mu_1}(X_i)\right)$$

$$\hat{E}[Y_1] = \frac{1}{N}\sum\left(\frac{T_i Y_i}{\hat{P}(X_i)} - \left(\frac{T_i - \hat{P}(X_i)}{\hat{P}(X_i)}\right)\hat{\mu_1}(X_i)\right)$$

Now, assume that the propensity score $\hat{P}(X_i)$ is correctly specified. In this case, $E[T_i - \hat{P}(X_i)] = 0$, which wipes out the part dependent on $\hat{\mu_1}(X_i)$. This makes the doubly robust estimator reduce to the propensity score weighting estimator $\frac{T_i Y_i}{\hat{P}(X_i)}$, which is correct by assumption. So, even if the $\hat{\mu_1}(X_i)$ is wrong, the estimator will still be correct, provided that the propensity score is correctly specified.

Once again, if you believe more in code than in formulas, here it is the practical verification. In the code below, I've replaced both regression models with a random normal variable. There is no doubt that $\hat{\mu}(X_i)$ is **not correctly specified**. Still, we will see that doubly robust estimation still manages to recover the same $A\hat{T}E$ of about 0.38 that we've seen before.

```
from sklearn.linear_model import LogisticRegression, LinearRegression

def doubly_robust_wrong_model(df, X, T, Y):
    np.random.seed(654)
    ps = LogisticRegression(C=1e6, max_iter=1000).fit(df[X],
      ↪ df[T]).predict_proba(df[X])[:, 1]

    # wrong mu(x) model
    mu0 = np.random.normal(0, 1, df.shape[0])
    mu1 = np.random.normal(0, 1, df.shape[0])
    return (
        np.mean(df[T]*(df[Y] - mu1)/ps + mu1) -
        np.mean((1-df[T])*(df[Y] - mu0)/(1-ps) + mu0)
    )
```

```
doubly_robust_wrong_model(data_with_categ, X, T, Y)
```

```
0.39811864040982625
```

Once again, we can use bootstrap and see that the variance is just slightly higher.

```
np.random.seed(88)
parallel_fn = delayed(doubly_robust_wrong_model)
wrong_mux = Parallel(n_jobs=4)(parallel_fn(data_with_categ.sample(frac=1,
  ↪ replace=True), X, T, Y)
                               for _ in range(bootstrap_sample))
wrong_mux = np.array(wrong_mux)
```

```
print(f"Original ATE 95% CI:", (np.percentile(ates, 2.5), np.percentile(ates,
  ↪ 97.5)))
print(f"Wrong Mu ATE 95% CI:", (np.percentile(wrong_mux, 2.5),
  ↪ np.percentile(wrong_mux, 97.5)))
```

```
Original ATE 95% CI: (0.35364752721776144, 0.41978441342477585)
Wrong Mu ATE 95% CI: (0.33863822614929406, 0.433173164584189)
```

Once more, messing up the conditional mean model alone yields only slightly different ATE. I hope I've convinced you about the power of doubly robust estimation. Its magic happens because in causal inference, there are two ways to remove bias from our causal estimates: you either model the treatment mechanism or the outcome mechanism. If either of these models are correct, you are good to go.

One caveat is that, in practice, it's very hard to model precisely either of those. More often, what ends up happening is that neither the propensity score nor the outcome model are 100% correct. They are both wrong, but in different ways. When this happens, it is not exactly settled [1][2] [2][3] [3][4] if it's better to use a single model or doubly robust estimation. As for me, I still like using them because at least it gives me two possibilities of being correct.

Key Ideas

Here, we saw a simple way of combining linear regression with the propensity score to produce a doubly robust estimator. This estimator bears that name because it only requires one of the models to be correct. If the propensity score model is correct, we will be able to identify the causal effect even if the outcome model is wrong. On the flip side, if the outcome model is correct, we will also be able to identify the causal effect even if the propensity score model is wrong.

References

I like to think of this entire book as a tribute to Joshua Angrist, Alberto Abadie and Christopher Walters for their amazing Econometrics class. Most of the ideas here are taken from their classes at the American Economic Association. Watching them is what is keeping me sane during this tough year of 2020.

- Cross-Section Econometrics · https://www.aeaweb.org/conference/cont-ed/2017-webcasts
- Mastering Mostly Harmless Econometrics · https://www.aeaweb.org/conference/cont-ed/2020-webcasts

2 https://www.stat.cmu.edu/~ryantibs/journalclub/kang_2007.pdf

3 https://arxiv.org/pdf/0804.2969.pdf

4 https://www.ncbi.nlm.nih.gov/pmc/articles/PMC2798744/

I'll also like to reference the amazing books from Angrist. They have shown me that Econometrics, or 'Metrics as they call it, is not only extremely useful but also profoundly fun.

- Mostly Harmless Econometrics · https://www.mostlyharmlesseconometrics.com/
- Mastering 'Metrics · https://www.masteringmetrics.com/

My final reference is Miguel Hernan and Jamie Robins' book. It has been my trustworthy companion in the most thorny causal questions I had to answer.

- Causal Inference Book · https://miguelhernan.org/whatifbook

The data that we used was taken from the article Estimating Treatment Effects with Causal Forests: An Application · https://arxiv.org/pdf/1902.07409.pdf, by Susan Athey and Stefan Wager.

CHAPTER
13

Difference-in-Differences

Three Billboards in the South of Brazil

I remember when I worked with marketing and a great way to do it was with internet advertisement. Not because it is very efficient (although it is), but because it is very easy to know if it's effective or not. With online marketing, you have a way of knowing which customers saw the ad and you can track them with cookies to see if they ended up on your landing page or clicked some download button. You can also use machine learning to find prospects that are very similar to your customers and present the ad only to them. In this sense, online marketing is very precise: you target only those you want to and you can see if they respond as you would like them to.

But not everyone is susceptible to online marketing. Sometimes you have to resort to less precise techniques, like a TV campaign or placing a billboard down the street. Usually, diversity of marketing channels is something marketing departments look for. But if online marketing is a professional fishing rod to catch that specific type of tuna, billboard and TV are giant nets you throw at a fish shoal and hope to catch at least some big ones. Another problem with billboard and TV ads is that it is much harder to know how effective they are. Sure, you could measure the purchase volume, or whatever you want to drive, before and after placing a billboard somewhere. If there is an increase, there is some evidence that the marketing is effective. But how would you know if this increase is not just some natural trend in the awareness of your product? In other words, how would you know the counterfactual Y_0 of what would have happened if you didn't set up the billboards in the first place?

One technique to answer these types of questions is Difference-in-Difference, or diff-in-diff for close friends. Diff-in-diff is commonly used to assess the effect of macro interventions, like the effect of immigration on unemployment, the effect of gun law changes in crime rates or simply the difference in user engagement due to a marketing campaign. In all these cases, you have a period before and after the intervention and you wish to untangle the impact of the intervention from a general trend. As a motivating example, let's look at a question similar to the one I had to answer.

In order to figure out how good billboards were as a marketing channel, we placed 3 billboards in the city of Porto Alegre, the capital of the state of Rio Grande do Sul. We wanted to see if that boosted deposits into our savings account. As a note for those not very familiar with Brazilian geography, Rio Grande do Sul is part of the south of the country, one of the most developed regions.

Having this in mind, we decided to also look at data from another capital from the south, Florianopolis, the capital city of the state of Santa Catarina. The idea is that we could use Florianopolis as a control sample to estimate the counterfactual Y_0 when compared to Porto Alegre (by the way, this was not the true experiment, which is confidential, but the idea is very similar). We placed the billboard in Porto Alegre for the entire month of June. The data we have looks like this:

```
import warnings
warnings.filterwarnings('ignore')

import pandas as pd
import numpy as np
from matplotlib import style
from matplotlib import pyplot as plt
import seaborn as sns
import statsmodels.formula.api as smf


%matplotlib inline

style.use("fivethirtyeight")
```

```
data = pd.read_csv("data/billboard_impact.csv")
data.head()
```

	deposits	poa	jul
0	42	1	0
1	0	1	0
2	52	1	0
3	119	1	0
4	21	1	0

Remember that deposits are our outcome variable, the one we wish to increase with the billboards. POA is a dummy indicator for the city of Porto Alegre. When it is zero, it means the samples are from Florianopolis. Jul is a dummy for the month of July, or for the post intervention period. When it is zero it refers to samples from May, the pre-intervention period.

DID Estimator

To avoid confusion between Time and Treatment, from now on, I'll use D to denote treatment and T to denote time. Let $Y_D(T)$ be the potential outcome for treatment D on period T. In an ideal world where we have the ability to observe the counterfactual, we would estimate the treatment effect of an intervention the following way:

$$\hat{ATET} = E[Y_1(1) - Y_0(1) \,|\, D = 1]$$

In words, the causal effect is the outcome in the period post intervention in case of a treatment minus the outcome in also in the period after the intervention, but in the case of no treatment. Of course, we can't measure this because $Y_0(1)$ is counterfactual.

One way around this is a before and after comparison.

$$\hat{ATET} = E[Y(1) \,|\, D = 1] - E[Y(0) \,|\, D = 1]$$

In our example, we would compare the average deposits from POA before and after the billboard was placed.

```
poa_before = data.query("poa==1 & jul==0")["deposits"].mean()

poa_after = data.query("poa==1 & jul==1")["deposits"].mean()

poa_after - poa_before
```

```
41.04775
```

This estimator is telling us that we should expect deposits to increase R$ 41,04 after the intervention. But can we trust this?

Notice that $E[Y(0)|D = 1] = E[Y_0(0)|D = 1]$, that is, the observed outcome for the treated unit **before the intervention** is equal to the counterfactual outcome for the treated unit also before the intervention. Since we are using this to estimate the counterfactual **after the intervention** $E[Y_0(1)|D = 1]$, this estimation above assumes that $E[Y_0(1)|D = 1] = E[Y_0(0)|D = 1]$.

It is saying that in the case of no intervention, the outcome in the latter period would be the same as the outcome from the starting period. This would obviously be false if your outcome variable follows any kind of trend. For example, if deposits are going up in POA, $E[Y_0(1)|D = 1] > E[Y_0(0)|D = 1]$, i.e. the outcome of the latter period would be greater than that of the starting period even in the absence of the intervention. With a similar argument, if the trend in Y is going down, $E[Y_0(1)|D = 1] < E[Y_0(0)|D = 1]$. This is to show that this before and after thing is not a great estimator.

Another idea is to compare the treated group with an untreated group that didn't get the intervention:

$$\hat{ATET} = E[Y(1)|D = 1] - E[Y(1)|D = 0]$$

In our example, it would be to compare the deposits from POA to that of Florianopolis in the post intervention period.

```
fl_after = data.query("poa==0 & jul==1")["deposits"].mean()
poa_after - fl_after
```

```
-119.10175000000001
```

This estimator is telling us that the campaign is detrimental and that customers will decrease deposits by R$ 119.10.

Notice that $E[Y(1)|D = 0] = E[Y_0(1)|D = 0]$. And since we are using $E[Y(1)|D = 0]$ to estimate the counterfactual for the treated after the intervention, we are assuming we can replace the missing counterfactual like this: $E[Y_0(1)|D = 0] = E[Y_0(1)|D = 1]$.

But notice that this would only be true if both groups have a very similar baseline level. For instance, if Florianopolis has way more deposits than Porto Alegre, this would not be true because $E[Y_0(1)|D=0] > E[Y_0(1)|D=1]$. On the other hand, if the level of deposits are lower in Florianopolis, we would have $E[Y_0(1)|D=0] < E[Y_0(1)|D=1]$.

Again, this is not a great idea. To solve this, we can use both space and time comparison. This is the idea of the difference in difference approach. It works by replacing the missing counterfactual the following way:

$$E[Y_0(1)|D=1] = E[Y_0(0)|D=1] + (E[Y_0(1)|D=0] - E[Y_0(0)|D=0])$$

What this does is take the treated unit **before the intervention** and adds a trend component to it, which is estimated using the control $E[Y_0(1)|D=0] - E[Y_0(0)|D=0]$. In words, it is saying that the treated **after the intervention**, had it not been treated, would look like the **treated before the treatment** plus a growth factor that is the same as the growth of the control.

It is important to notice that this assumes that the trends in the treatment and control are the same:

$$E[Y_0(1) - Y_0(0)|D=1] = E[Y_0(1) - Y_0(0)|D=0]$$

where the left hand side is the counterfactual trend. Now, we can replace the estimated counterfactual in the treatment effect definition $E[Y_1(1)|D=1] - E[Y_0(1)|D=1]$

$$\hat{ATET} = E[Y(1)|D=1] - (E[Y(0)|D=1] + (E[Y(1)|D=0] - E[Y(0)|D=0])$$

If we rearrange the terms, we get the classical Diff-in-Diff estimator.

$$\hat{ATET} = (E[Y(1)|D=1] - E[Y(1)|D=0]) - (E[Y(0)|D=1] - E[Y(0)|D=0])$$

It gets that name because it gets the difference between the difference between treatment and control after and before the treatment.

Here is what that looks in code.

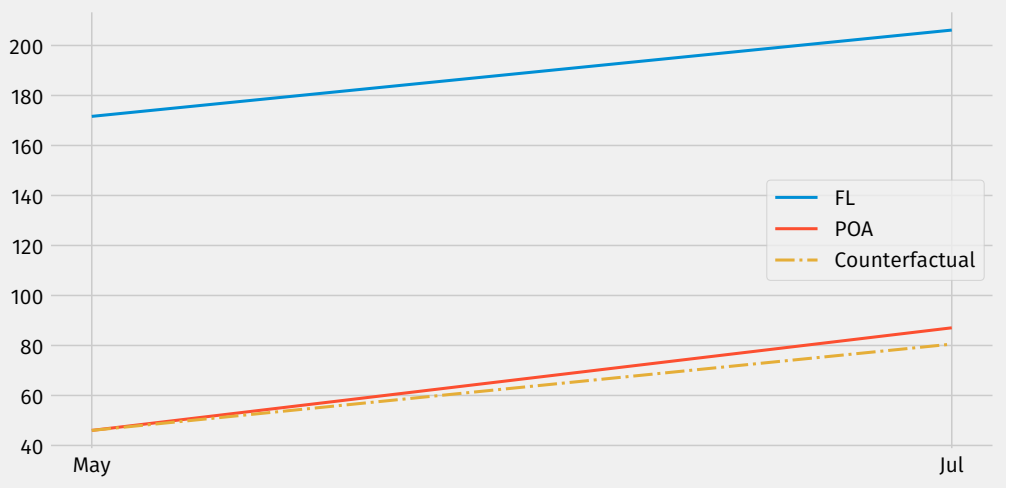
```
fl_before = data.query("poa==0 & jul==0")["deposits"].mean()

diff_in_diff = (poa_after-poa_before)-(fl_after-fl_before)
diff_in_diff
```

```
6.524557692307688
```

Diff-in-Diff is telling us that we should expect deposits to increase by R$ 6.52 per customer. Notice that the assumption that diff-in-diff makes is much more plausible than the other 2 estimators. It just assumes that the growth pattern between the 2 cities are the same. But it doesn't require them to have the same base level nor does it require the trend to be zero.

To visualize what diff-in-diff is doing, we can project the growth trend from the untreated into the treated to see the counterfactual, that is, the number of deposits we should expect if there were no intervention.

```
plt.figure(figsize=(10,5))
plt.plot(["May", "Jul"], [fl_before, fl_after], label="FL", lw=2)
plt.plot(["May", "Jul"], [poa_before, poa_after], label="POA", lw=2)

plt.plot(["May", "Jul"], [poa_before, poa_before+(fl_after-fl_before)],
         label="Counterfactual", lw=2, color="C2", ls="-.")

plt.legend();
```



See that small difference between the red and the yellow dashed lines? If you really focus you can see the small treatment effect on Porto Alegre.

Now, what you might be asking yourself is "how much can I trust this estimator? It is my right to have standard errors reported to me!". Which makes sense, since estimators without them look silly. To do so, we will use a neat trick that uses regression. Specifically, we will estimate the following linear model

$$Y_i = \beta_0 + \beta_1 POA_i + \beta_2 Jul_i + \beta_3 POA_i * Jul_i + e_i$$

Notice that β_0 is the baseline of the control. In our case, is the level of deposits in Florianopolis in the month of May. If we turn on the treated city dummy, we get β_1. So $\beta_0 + \beta_1$ is the baseline of Porto Alegre in May, before the intervention, and β_1 is the increase of Porto Alegre baseline on top of Florianopolis. If we turn the POA dummy off and turn the July dummy on, we get $\beta_0 + \beta_2$, which is the level of Florianópolis in July, after the intervention period. β_2 is then the trend of the control, since we add it on top of the baseline to get the level of the control at the period post intervention. As a recap, β_1 is the increment we get by going from the control to the treated, β_2 is the increment we get by going from the period before to the period after the intervention. Finally, if we turn both dummies on, we get β_3. $\beta_0 + \beta_1 + \beta_2 + \beta_3$ is the level in Porto Alegre after the intervention. So β_3 is the incremental impact when you go from May to July and from Florianopolis to POA. In other words, it is the Difference in Difference estimator.

If you don't believe me, check for yourself. You should get the exact same number we got above. And also notice how we get our much wanted standard errors.

```
smf.ols('deposits ~ poa*jul', data=data).fit().summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	171.6423	2.363	72.625	0.000	167.009	176.276
poa	-125.6263	4.484	-28.015	0.000	-134.418	-116.835
jul	34.5232	3.036	11.372	0.000	28.571	40.475
poa:jul	6.5246	5.729	1.139	0.255	-4.706	17.755

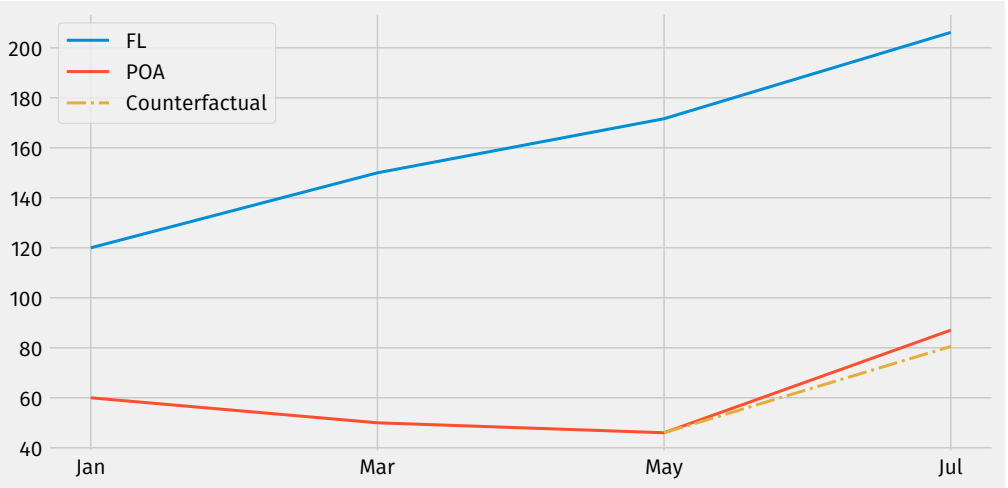
Non Parallel Trends

One obvious problem with Diff-in-Diff is failure to satisfy the parallel trend assumption. If the growth trend from the treated is different from the trend of the control, diff-in-diff will be biased. This is a common problem with non-random data, where the decision to treat a region is based on its potential to respond well to the treatment, or when the treatment is targeted at regions that are not performing very well. Take our marketing example. We decided to test billboards in Porto Alegre not in order to check the effect of billboards in general. The reason is simply because sales perform poorly there. Perhaps online marketing is not working there. In this case, it could be that the growth we would see in Porto Alegre without a billboard would be lower than the growth we observe in other cities. This would cause us to underestimate the effect of the billboard there.

One way to check if this is happening is to plot the trend using past periods. For example, let's suppose POA had a small decreasing trend but Florianopolis was on a steep ascent.

In this case, showing periods from before would reveal those trends and we would know Diff-in-Diff is not a reliable estimator.

```
plt.figure(figsize=(10,5))
x = ["Jan", "Mar", "May", "Jul"]

plt.plot(x, [120, 150, fl_before,  fl_after], label="FL", lw=2)
plt.plot(x, [60, 50, poa_before, poa_after], label="POA", lw=2)

plt.plot(["May", "Jul"], [poa_before, poa_before+(fl_after-fl_before)],
  ↪ label="Counterfactual", lw=2, color="C2", ls="-.")

plt.legend();
```



We will see how to solve this problem with synthetic control. It will use multiple cities to create a synthetic city that closely follows the trend of the city of interest. But for now, remember that you always need to check if you have parallel trends when applying diff-in-diff.

One final issue that it's worth mentioning is that you won't be able to place confidence intervals around your Diff-in-Diff estimator if you only have aggregated data. Say for instance you don't have data on what each of our customers from Florianópolis or Porto Alegre did. Instead, you only have the average deposits before and after the intervention for both cities. In this case, you will still be able to estimate the causal effect by Diff-in-Diff, but you won't know the variance of it. That's because all the variability in your data got squashed out in aggregation.

Key Ideas

We've explored a technique widely applied when we are estimating causal effects at more macro entities (schools, cities, states, countries…). Difference in Difference takes a treated unit before and after the treatment and compares the trend in the outcome to that of a control unit. Here, we've seen how this could be applied at estimating the effect of a city specific marketing campaign.

Finally, we looked at how Diff-in-Diff fails if the trend between the treated and control unit is not the same. We also saw how diff-in-diff will be problematic if we only have aggregated data.

References

I like to think of this entire book as a tribute to Joshua Angrist, Alberto Abadie and Christopher Walters for their amazing Econometrics class. Most of the ideas here are taken from their classes at the American Economic Association. Watching them is what is keeping me sane during this tough year of 2020.

- Cross-Section Econometrics · https://www.aeaweb.org/conference/cont-ed/2017-webcasts
- Mastering Mostly Harmless Econometrics · https://www.aeaweb.org/conference/cont-ed/2020-webcasts

I'll also like to reference the amazing books from Angrist. They have shown me that Econometrics, or 'Metrics as they call it, is not only extremely useful but also profoundly fun.

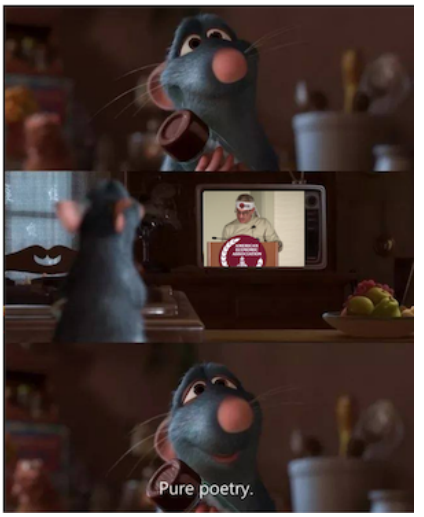
- Mostly Harmless Econometrics · https://www.mostlyharmlesseconometrics.com/
- Mastering 'Metrics · https://www.masteringmetrics.com/

Other important reference is Miguel Hernan and Jamie Robins' book. It has been my trustworthy companion in the most thorny causal questions I had to answer.

- Causal Inference Book · https://miguelhernan.org/whatifbook

Finally, I'd also like to compliment Scott Cunningham and his brilliant work mingling Causal Inference and Rap quotes:

- Causal Inference: The Mixtape · https://www.scunning.com/mixtape.html


Pure poetry.

CHAPTER
14

Panel Data and Fixed Effects

```
import warnings
warnings.filterwarnings('ignore')

import pandas as pd
import numpy as np
from matplotlib import style
from matplotlib import pyplot as plt
import statsmodels.formula.api as smf
import graphviz as gr
from linearmodels.datasets import wage_panel

%matplotlib inline
pd.set_option("display.max_columns", 6)
style.use("fivethirtyeight")
```

In the previous chapter, we explored a very simple Diff-in-Diff setup, where we had a treated and a control group (the city of POA and FLN, respectively) and only two periods, a pre-intervention and a post-intervention period. But what would happen if we had more periods? Or more groups? Turns out this setup is so common and powerful for causal inference that it gets its own name: panel data. A panel is when we have repeated observations of the same unit over multiple periods of time. This happens a lot in government policy evaluation, where we can track data on multiple cities or states over multiple years. But it is also incredibly common in the industry, where companies track user data over multiple weeks and months.

To understand how we can leverage such data structure, let's first continue with our diff-in-diff example, where we wanted to estimate the impact of placing a billboard (treatment) in the city of Porto Alegre (POA). We want to see if that sort of offline marketing strategy can boost the usage of our investment products. Specifically, we want to know how much deposits in our investments account would increase if we placed billboards.

In the previous chapter, we've motivated the DiD estimator as an imputation strategy of what would have happened to Porto Alegre had we not placed the billboards in it. We said that the counterfactual outcome Y_0 for Porto Alegre **after the intervention** (placing a billboard) could be inputted as the number of deposits in Porto Alegre **before the intervention** plus a growth factor. This growth factor was estimated in a control city, Florianopolis (FLN). Just so we can recap some notation, here is how we can estimate this counterfactual outcome

$$\underbrace{\widehat{Y_0(1)|D=1}}_{\substack{\text{POA outcome after intervention} \\ \substack{\text{if no intervention had taken place}}}} = \underbrace{Y_0(0)|D=1}_{\substack{\text{POA outcome} \\ \substack{\text{before intervention}}}}$$

- $\big(\underbrace{Y_0(1)|D=0}_{\substack{\text{FLN outcome after} \\ \substack{\text{intervention in POA}}}}$
- $\underbrace{Y_0(0)|D=0}_{\substack{\text{FLN outcome before} \\ \substack{\text{intervention in POA}}}} \big)$

where t denotes time, D denotes the treatment (since t is taken), $Y_D(t)$ denote the potential outcome for treatment D in period t (for example, $Y_0(1)$ is the outcome under the control in period 1). Now, if we take that imputed potential outcome, we can recover the treatment effect for POA (ATT) as follows

$$\widehat{ATT} = \underbrace{Y_1(1)|D=1}_{\substack{\text{POA outcome} \\ \text{after intervention}}} - \widehat{Y_0(1)|D=1}$$

In other words, the effect of placing a billboard in POA is the outcome we saw on POA after placing the billboard minus our estimate of what would have happened if we hadn't placed the billboard. Also, recall that the power of DiD comes from the fact that estimating the mentioned counterfactual only requires that the growth deposits in POA matches the growth in deposits in FLN. This is the key parallel trends assumption. We should definitely spend some time on it because it is going to become very important later on.

Parallel Trends

One way to see the parallel (or common) trends assumptions is as an independence assumption. If we recall from very early chapters, the independence assumption requires that the treatment assignment is independent from the potential outcomes:

$$Y_d \perp D$$

This means we don't give more treatment to units with higher outcome (which would cause upward bias in the effect estimation) or lower outcome (which would cause downward bias). In less abstract terms, back to our example, let's say that your marketing manager decides to add billboards only to cities that already have very high deposits. That way, he or she can later boast that cities with billboards generate more deposites, so of course the

marketing campaign was a success. Setting aside the moral discussion here, I think you can see that this violates the independence assumption: we are giving the treatment to cities with high Y_0. Also, remember that a natural extension of this assumption is the conditional independence assumption, which allows the potential outcomes to be dependent on the treatment at first, but independent once we condition on the confounders X

$$Y_d \perp D \,|\, X$$

You know all of this already. But how exactly does this tie back to DiD and the parallel trends assumption? If the traditional independence assumption states that the treatment assignment can't be related to **the levels** of potential outcomes, the parallel trends states that the treatment assignment can't be related to **the growth** in potential outcomes over time. In fact, one way to write the parallel trends assumption is as follows

$$\big(Y_d(t) - Y_d(t-1)\big) \perp D$$

In less mathematical terms, this assumption is saying it is fine that we assign the treatment to units that have a higher or lower level of the outcome. What we can't do is assign the treatment to units based on how the outcome is growing. In our billboard example, this means it is OK to place billboards only in cities with originally high deposits level. What we can't do is place billboards only in cities where the deposits are growing the most. That makes a lot of sense if we remember that DiD is inputting the counterfactual growth in the treated unit with the growth in the control unit. If growth in the treated unit under the control is different than the growth in the control unit, then we are in trouble.

Controlling What you Cannot See

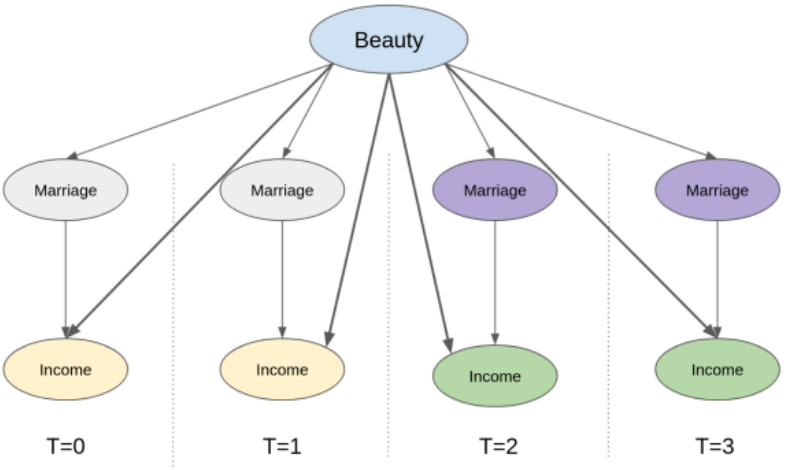
Methods like propensity score, linear regression and matching are very good at controlling for confounding in non-random data, but they rely on a key assumption: conditional unconfoundedness

$$(Y_0, Y_1) \perp T \,|\, X$$

To put it in words, they require that all the confounders are known and measured, so that we can condition on them and make the treatment as good as random. One major issue with this is that sometimes we simply can't measure a confounder. For instance, take a classical labor economics problem of figuring out the impact of marriage on men's earnings. It's a well known fact in economics that married men earn more than single men. However, it is not clear if this relationship is causal or not. It could be that more educated men are both more likely to marry and more likely to have a high earnings job, which would mean that education is a confounder of the effect of marriage on earnings. For this confounder, we could measure the education of the person in the study and run a regression controlling for that. But another confounder could be beauty. It could be that more handsome men are both more likely to get married and more likely to have a high paying job. Unfortunately, beauty is one of those characteristics like intelligence. It's something we can't measure very well.

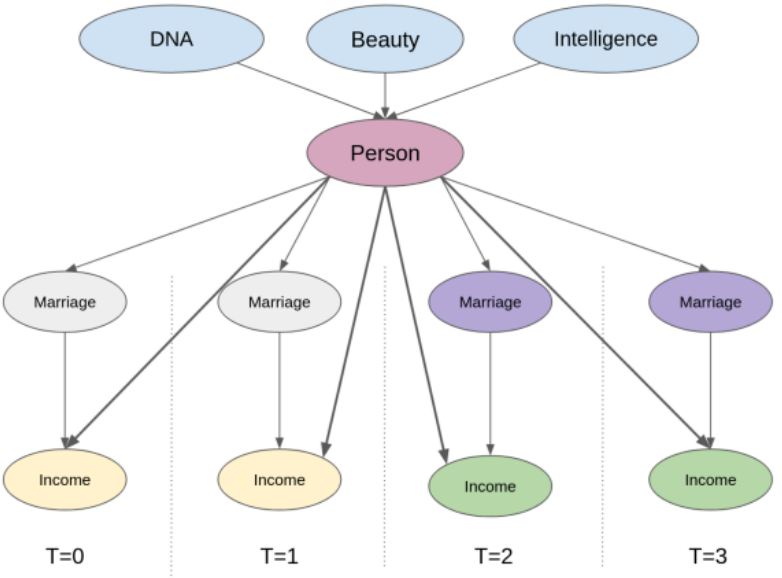
This puts us in a difficult situation, because if we have unmeasured confounders, we have bias. One way to deal with this is with instrumental variables, as we've seen before. But coming up with good instruments it's no easy task and requires a lot of creativity. Here, instead let's take advantage of our panel data structure.

We've already seen how panel data allows us to replace the unconfoundedness assumption with the parallel trends assumption. But how exactly does this help with unmeasured confounders? First, let's take a look at the causal graph which represents this setup where we have repeated observations across time. Here, we track the same observation across 4 time periods. Marriage (the treatment) and Income (the outcome) change over time. Specifically, marriage turns on (from 0 to 1) at period 3 and 4 and income increases in the same periods. Beauty, the unmeasured confounder, is the same across all periods (a bold statement, but reasonable if time is just a few years). So, how can we know that the reason income increases is because of marriage and not simply due to an increase in the beauty confounder? And, more importantly, how can we control that confounder we cannot see?



The trick is to see that, by zooming in a unit and tracking how it evolves over time, we are already controlling for anything that is fixed over time. That includes any time fixed unmeasured confounders. In the graph above, for instance, we can already know that the increase in income over time cannot be due to an increase in beauty, simply because beauty stays the same (it is time fixed after all). The bottom line is that even though we cannot control for beauty, since we can't measure it, we can still use the panel structure so it is not a problem anymore.

Another way of seeing this is to think about these time fixed confounders as attributes that are specific to each unit. This would be equivalent to adding an intermediary unit node to our causal graph. Now, notice how controlling for the unit itself already blocks the backdoor path between the outcome and any of the unobserved but time fixed confounders.



Think about it. We can't measure attributes like beauty and intelligence, but we know that the person who has them is the same individual across time. The mechanics of actually doing this control is very simple. All we need to do is create dummy variables indicating that person and add that to a linear model. This is what we mean when we say we can control for the person itself: we are adding a variable (dummy in this case) that denotes that particular person. When estimating the effect of marriage on income with this person dummy in our model, regression finds the effect of marriage **while keeping the person variable fixed**. Adding this unit dummy is what we call a fixed effect model.

Fixed Effects

To make matters more formal, let's first take a look at the data that we have. Following our example, we will try to estimate the effect of marriage on income. Our data contains those 2 variables, `married` and `lwage`, on multiple individuals (`nr`) for multiple years. Notice that wage is in log form. In addition to this, we have other controls, like number of hours worked that year, years of education and so on.

```
data = wage_panel.load()
data.head()
```

	nr	year	black	...	lwage	expersq	occupation
0	13	1980	0	...	1.197540	1	9
1	13	1981	0	...	1.853060	4	9
2	13	1982	0	...	1.344462	9	9
3	13	1983	0	...	1.433213	16	9
4	13	1984	0	...	1.568125	25	5

5 rows × 12 columns

Generally, the fixed effect model is defined as

$$y_{it} = \beta X_{it} + \gamma U_i + e_{it}$$

where y_{it} is the outcome of individual i at time t, X_{it} is the vector of variables for individual i at time t. U_i is a set of unobservables for individual i. Notice that those unobservables are unchanging through time, hence the lack of the time subscript. Finally, e_{it} is the error term. For the education example, y_{it} is log wages, X_{it} are the observable variables that change in time, like marriage and experience and U_i are the variables that are not observed but constant for each individual, like beauty and intelligence.

Now, remember how I've said that using panel data with a fixed effect model is as simple as adding a dummy for the entities. It's true, but in practice, we don't actually do it. Imagine a dataset where we have 1 million customers. If we add one dummy for each of them, we would end up with 1 million columns, which is probably not a good idea. Instead, we use the trick of partitioning the linear regression into 2 separate models. We've seen this before, but now is a good time to recap it. Suppose you have a linear regression model with a set of features X_1 and another set of features X_2.

$$\hat{Y} = \hat{\beta}_1 X_1 + \hat{\beta}_2 X_2$$

where X_1 and X_2 are feature matrices (one row per feature and one column per observation) and $\hat{\beta}_1$ and $\hat{\beta}_2$ are row vectors. You can get the exact same $\hat{\beta}_1$ parameter by doing

1. regress the outcome y on the second set of features $\hat{y^*} = \hat{\gamma_1} X_2$
2. regress the first set of features on the second $\hat{X_1} = \hat{\gamma_2} X_2$
3. obtain the residuals $\tilde{X_1} = X_1 - \hat{X_1}$ and $\tilde{y}_1 = y_1 - \hat{y^*}$
4. regress the residuals of the outcome on the residuals of the features $\hat{y} = \hat{\beta}_1 \tilde{X}_1$

The parameter from this last regression will be exactly the same as running the regression with all the features. But how exactly does this help us? Well, we can break the estimation of the model with the entity dummies into 2. First, we use the dummies to predict the outcome and the feature. These are steps 1 and 2 above.

Now, remember how running a regression on a dummy variable is as simple as estimating the mean for that dummy? If you don't, let's use our data to show how this is true. Let's run a model where we predict wages as a function of the year dummy.

```
mod = smf.ols("lwage ~ C(year)", data=data).fit()
mod.summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	1.3935	0.022	63.462	0.000	1.350	1.437
C(year)[T.1981]	0.1194	0.031	3.845	0.000	0.059	0.180
C(year)[T.1982]	0.1782	0.031	5.738	0.000	0.117	0.239
C(year)[T.1983]	0.2258	0.031	7.271	0.000	0.165	0.287
C(year)[T.1984]	0.2968	0.031	9.558	0.000	0.236	0.358
C(year)[T.1985]	0.3459	0.031	11.140	0.000	0.285	0.407
C(year)[T.1986]	0.4062	0.031	13.082	0.000	0.345	0.467
C(year)[T.1987]	0.4730	0.031	15.232	0.000	0.412	0.534

Notice how this model is predicting the average income in 1980 to be 1.3935, in 1981 to be 1.5129 (1.3935+0.1194) and so on. Now, if we compute the average by year, we get the exact same result. (Remember that the base year, 1980, is the intercept. So you have to add the intercept to the parameters of the other years to get the mean `lwage` for the year).

```
data.groupby("year")["lwage"].mean()
```

```
year
1980    1.393477
1981    1.512867
1982    1.571667
1983    1.619263
1984    1.690295
1985    1.739410
1986    1.799719
1987    1.866479
Name: lwage, dtype: float64
```

This means that if we get the average for every person in our panel, we are essentially regressing the individual dummy on the other variables. This motivates the following estimation procedure:

1. Create time-demeaned variables by subtracting the mean for the individual:
   $\ddot{Y}_{it} = Y_{it} - \bar{Y}_i$
   $\ddot{X}_{it} = X_{it} - \bar{X}_i$
2. Regress $\ddot{Y}_{it}$ on $\ddot{X}_{it}$

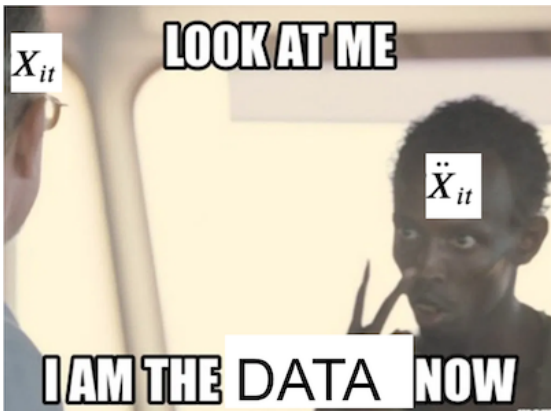
Notice that when we do so, the unobserved U_i vanishes. Since U_i is constant across time, we have that $\bar{U}_i = U_i$. If we have the following system of two equations

$$Y_{it} = \beta X_{it} + \gamma U_i + e_{it}$$
$$\bar{Y}_i = \beta \bar{X}_{it} + \gamma \bar{U}_i + \bar{e}_{it}$$

And we subtract one from the other, we get

$$(Y_{it} - \bar{Y}_i) = (\beta X_{it} - \beta \bar{X}_{it}) + (\gamma U_i - \gamma U_i) + (e_{it} - \bar{e}_{it})$$
$$(Y_{it} - \bar{Y}_i) = \beta (X_{it} - \bar{X}_{it}) + (e_{it} - \bar{e}_{it})$$
$$\ddot{Y}_{it} = \beta \ddot{X}_{it} + \ddot{e}_{it}$$

which wipes out all unobserved that are constant across time. To be honest, not only do the unobserved variables vanish. This happens to all the variables that are constant in time. For this reason, you can't include any variables that are constant across time, as they would be a linear combination of the dummy variables and the model wouldn't run.



To check which variables are those, we can group our data by individual and get the sum of the standard deviations. If it is zero, it means the variable isn't changing across time for any of the individuals.

```
data.groupby("nr").std().sum()
```

```
year            1334.971910
black              0.000000
exper           1334.971910
hisp               0.000000
hours         203098.215649
married          140.372801
educ               0.000000
union            106.512445
lwage            173.929670
expersq        17608.242825
occupation       739.222281
dtype: float64
```

For our data, we need to remove ethnicity dummies, `black` and `hisp`, since they are constant for the individual. Also, we need to remove education. We will also not use occupation, since this is probably mediating the effect of marriage on wage (it could be that single men are able to take more time demanding positions). Having selected the features we will use, it's time to estimate this model.

To run our fixed effect model, first, let's get our mean data. We can achieve this by grouping everything by individuals and taking the mean.

```
Y = "lwage"
T = "married"
X = [T, "expersq", "union", "hours"]

mean_data = data.groupby("nr")[X+[Y]].mean()
mean_data.head()
```

nr	married	expersq	union	hours	lwage
13	0.000	25.5	0.125	2807.625	1.255652
17	0.000	61.5	0.000	2504.125	1.637786
18	1.000	61.5	0.000	2350.500	2.034387
45	0.125	35.5	0.250	2225.875	1.773664
110	0.500	77.5	0.125	2108.000	2.055129

To demean the data, we need to set the index of the original data to be the individual identifier, `nr`. Then, we can simply subtract one data frame from the mean data frame.

```
demeaned_data = (data
                 .set_index("nr") # set the index as the person indicator
                 [X+[Y]]
                 - mean_data) # subtract the mean data

demeaned_data.head()
```

nr	married	expersq	union	hours	lwage
13	0.0	-24.5	-0.125	-135.625	-0.058112
13	0.0	-21.5	0.875	-487.625	0.597408
13	0.0	-16.5	-0.125	132.375	0.088810
13	0.0	-9.5	-0.125	152.375	0.177561
13	0.0	-0.5	-0.125	263.375	0.312473

Finally, we can run our fixed effect model on the time-demeaned data.

```
mod = smf.ols(f"{Y} ~ {'+'.join(X)}", data=demeaned_data).fit()
mod.summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	-6.679e-17	0.005	-1.32e-14	1.000	-0.010	0.010
married	0.1147	0.017	6.756	0.000	0.081	0.148
expersq	0.0040	0.000	21.958	0.000	0.004	0.004
union	0.0784	0.018	4.261	0.000	0.042	0.115
hours	-8.46e-05	1.25e-05	-6.744	0.000	-0.000	-6e-05

If we believe that fixed effect eliminates the all omitted variable bias, this model is telling us that marriage increases a man's wage by 11%. This result is very significant. One detail here is that for fixed effect models, the standard errors need to be clustered. So, instead of doing all our estimation by hand (which is only nice for pedagogical reasons), we can use the library `linearmodels` and set the argument `cluster_entity` to True.

```
from linearmodels.panel import PanelOLS
mod = PanelOLS.from_formula("lwage ~ expersq+union+married+hours+EntityEffects",
                            data=data.set_index(["nr", "year"]))

result = mod.fit(cov_type='clustered', cluster_entity=True)
result.summary.tables[1]
```

	Parameter	Std. Err.	T-stat	P-value	Lower CI	Upper CI
expersq	0.0040	0.0002	16.552	0.0000	0.0035	0.0044
union	0.0784	0.0236	3.3225	0.0009	0.0322	0.1247
married	0.1147	0.0220	5.2213	0.0000	0.0716	0.1577
hours	-8.46e-05	2.22e-05	-3.8105	0.0001	-0.0001	-4.107e-05

Notice how the parameter estimates are identical to the ones we've got with time-demeaned data. The only difference is that the standard errors are a bit larger. Now, compare this to the simple OLS model that doesn't take the time structure of the data into account. For this model, we add back the variables that are constant in time.

```
mod = smf.ols("lwage ~ expersq+union+married+hours+black+hisp+educ",
  ↪ data=data).fit()
mod.summary().tables[1]
```

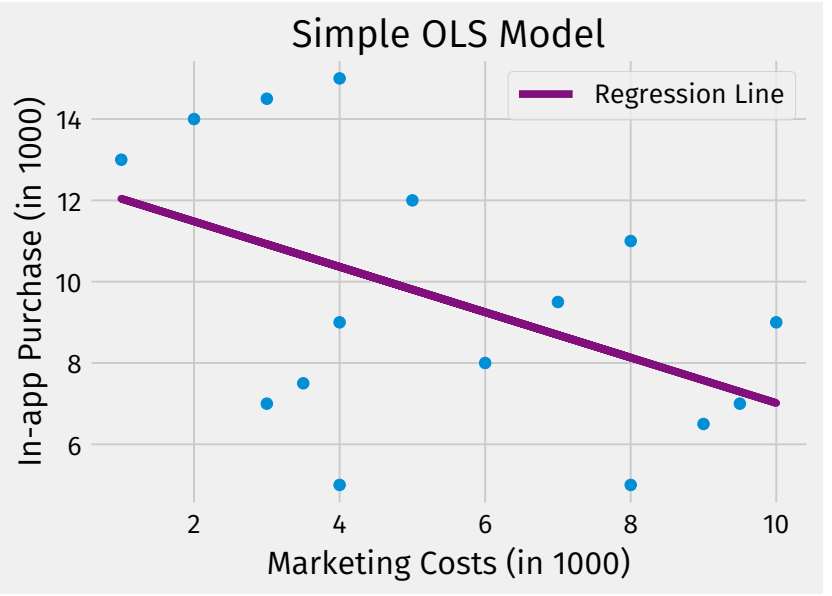
	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	0.2654	0.065	4.103	0.000	0.139	0.392
expersq	0.0032	0.000	15.750	0.000	0.003	0.004
union	0.1829	0.017	10.598	0.000	0.149	0.217
married	0.1410	0.016	8.931	0.000	0.110	0.172
hours	-5.32e-05	1.34e-05	-3.978	0.000	-7.94e-05	-2.7e-05
black	-0.1347	0.024	-5.679	0.000	-0.181	-0.088
hisp	0.0132	0.021	0.632	0.528	-0.028	0.054
educ	0.1057	0.005	22.550	0.000	0.097	0.115

This model is saying that marriage increases the man's wage by 14%. A somewhat larger effect than the one we found with the fixed effect model. This suggests some omitted variable bias due to fixed individual factors, like intelligence and beauty, not being added to the model.

Visualizing Fixed Effects

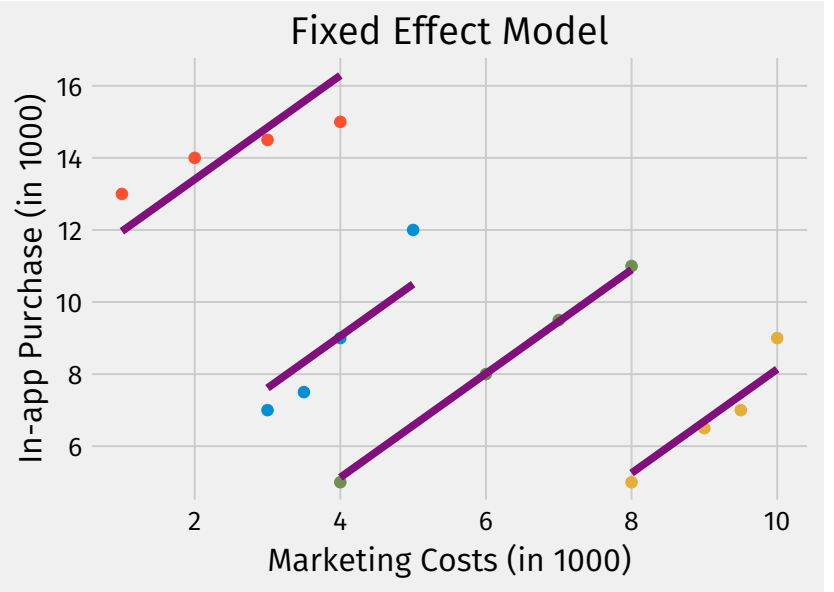
To expand our intuition about how fixed effect models work, let's diverge a little to another example. Suppose you work for a big tech company and you want to estimate the impact of a billboard marketing campaign on in-app purchase. When you look at data from the past, you see that the marketing department tends to spend more to place billboards on cities where the purchase level is lower. This makes sense right? They wouldn't need to do lots of advertisement if sales were skyrocketing. If you run a regression model on this data, it looks like higher cost in marketing leads to less in-app purchase, but only because marketing investments is biased towards low spending regions.

```
toy_panel = pd.DataFrame({
    "mkt_costs":[5,4,3.5,3, 10,9.5,9,8, 4,3,2,1, 8,7,6,4],
    "purchase":[12,9,7.5,7, 9,7,6.5,5, 15,14.5,14,13, 11,9.5,8,5],
    "city":["C0","C0","C0","C0", "C2","C2","C2","C2", "C1","C1","C1","C1",
      ↪ "C3","C3","C3","C3"]
})

m = smf.ols("purchase ~ mkt_costs", data=toy_panel).fit()

plt.scatter(toy_panel.mkt_costs, toy_panel.purchase)
plt.plot(toy_panel.mkt_costs, m.fittedvalues, c="C5", label="Regression Line")
plt.xlabel("Marketing Costs (in 1000)")
plt.ylabel("In-app Purchase (in 1000)")
plt.title("Simple OLS Model")
plt.legend();
```



Knowing a lot about causal inference, you decide to run a fixed effect model, adding the city's indicator as a dummy variable to your model. The fixed effect model controls for city specific characteristics that are constant in time, so if a city is less open to your product, it will capture that. When you run that model, you can finally see that more marketing costs leads to higher in-app purchase.

```
fe = smf.ols("purchase ~ mkt_costs + C(city)", data=toy_panel).fit()

fe_toy = toy_panel.assign(y_hat = fe.fittedvalues)

plt.scatter(toy_panel.mkt_costs, toy_panel.purchase, c=toy_panel.city)
for city in fe_toy["city"].unique():
    plot_df = fe_toy.query(f"city=='{city}'")
    plt.plot(plot_df.mkt_costs, plot_df.y_hat, c="C5")

plt.title("Fixed Effect Model")
plt.xlabel("Marketing Costs (in 1000)")
plt.ylabel("In-app Purchase (in 1000)");
```



Take a minute to appreciate what the image above is telling you about what fixed effect is doing. Notice that fixed effect is fitting **one regression line per city**. Also notice that the lines are parallel. The slope of the line is the effect of marketing costs on in-app purchase. So the **fixed effect is assuming that the causal effect is constant across all entities**, which are cities in this case. This can be a weakness or an advantage, depending on how you see it. It is a weakness if you are interested in finding the causal effect per city. Since the FE model assumes this effect is constant across entities, you won't find any difference in the causal effect. However, if you want to find the overall impact of marketing on in-app purchase, the panel structure of the data is a very useful leverage that fixed effects can explore.

Time Effects

Just like we did a fixed effect for the individual level, we could design a fixed effect for the time level. If adding a dummy for each individual controls for fixed individual characteristics, adding a time dummy would control for variables that are fixed for each time period, but that might change across time. One example of such a variable is inflation. Prices and salary tend to go up with time, but the inflation on each time period is the same for all entities. To give a more concrete example, suppose that marriage is increasing with time. If the wage and marriage proportion also changes with time, we would have time as a confounder. Since inflation also makes salary increase with time, some of the positive association we see between marriage and wage would be simply because both are increasing with time. To correct for that, we can add a dummy variable for each time period. In `linear models`, this is as simple as adding `TimeEffects` to our formula and setting the `cluster_time` to true.

```
mod = PanelOLS.from_formula("lwage ~
      ↪ expersq+union+married+hours+EntityEffects+TimeEffects",
                            data=data.set_index(["nr", "year"]))

result = mod.fit(cov_type='clustered', cluster_entity=True, cluster_time=True)
result.summary.tables[1]
```

	Parameter	Std. Err.	T-stat	P-value	Lower CI	Upper CI
expersq	-0.0062	0.0008	-8.1479	0.0000	-0.0077	-0.0047
union	0.0727	0.0228	3.1858	0.0015	0.0279	0.1174
married	0.0476	0.0177	2.6906	0.0072	0.0129	0.0823
hours	-0.0001	3.546e-05	-3.8258	0.0001	-0.0002	-6.614e-05

In this new model, the effect of marriage on wage decreased significantly from `0.1147` to `0.0476`. Still, this result is significant at a 99% level, so man could still expect an increase in earnings from marriage.

When Panel Data Won't Help You

Using panel data and fixed effects models is an extremely powerful tool for causal inference. When you don't have random data nor good instruments, the fixed effect is as convincing as it gets for causal inference with non experimental data. Still, it is worth mentioning that it is not a panacea. There are situations where even panel data won't help you.

The most obvious one is when you have confounders that are changing in time. Fixed effects can only eliminate bias from attributes that are constant for each individual. For instance, suppose that you can increase your intelligence level by reading books and eating lots of good fats. This causes you to get a higher paying job and a wife. Fixed effect won't be able to remove this bias due to unmeasured intelligence confounding because, in this example, intelligence is changing in time.

Another less obvious case when fixed effect fails is when you have **reversed causality**. For instance, let's say that it isn't marriage that causes you to earn more. Is earning more that increases your chances of getting married. In this case, it will appear that they have a positive correlation but earnings come first. They would change in time and in the same direction, so fixed effects wouldn't be able to control for that.

Key Ideas

Here, we saw how to use panel data, data where we have multiple measurements of the same individuals across multiple time periods. When that is the case, we can use a fixed effect model that controls for the entity, holding all individual, time constant attributes, fixed. This is a powerful and very convincing way of controlling for confounding and it is as good as it gets with non random data.

Finally, we saw that FE is not a panacea. We saw two situations where it doesn't work: when we have reverse causality and when the unmeasured confounding is changing in time.

References

I like to think of this entire book as a tribute to Joshua Angrist, Alberto Abadie and Christopher Walters for their amazing Econometrics class. Most of the ideas here are taken from their classes at the American Economic Association. Watching them is what is keeping me sane during this tough year of 2020.

- Cross-Section Econometrics · https://www.aeaweb.org/conference/cont-ed/2017-webcasts
- Mastering Mostly Harmless Econometrics · https://www.aeaweb.org/conference/cont-ed/2020-webcasts

I'll also like to reference the amazing books from Angrist. They have shown me that Econometrics, or 'Metrics as they call it, is not only extremely useful but also profoundly fun.

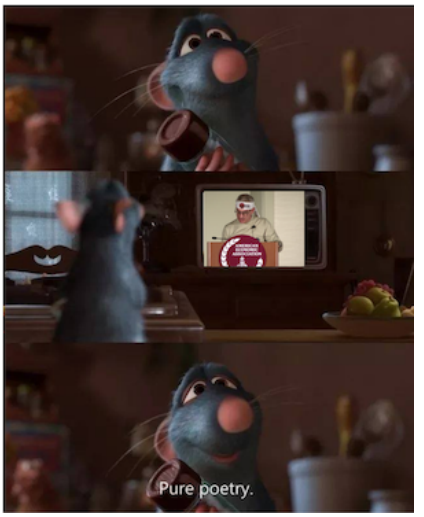
- Mostly Harmless Econometrics · https://www.mostlyharmlesseconometrics.com/
- Mastering 'Metrics · https://www.masteringmetrics.com/

Another important reference is Miguel Hernan and Jamie Robins' book. It has been my trustworthy companion in the most thorny causal questions I had to answer.

- Causal Inference Book · https://miguelhernan.org/whatifbook

Finally, I'd also like to compliment Scott Cunningham and his brilliant work mingling Causal Inference and Rap quotes:

- Causal Inference: The Mixtape · https://www.scunning.com/mixtape.html


Pure poetry.

CHAPTER
15

Synthetic Control

One Amazing Math Trick to Learn What can't be Known

When we looked at difference-in-difference, we had data on multiple customers from 2 different cities: Porto Alegre and Florianopolis. The data span 2 different time periods: before and after a marketing intervention was done in Porto Alegre to boost customer deposits. To estimate the treatment effect, we ran a regression that gave us the difference-in-difference estimator and its standard error.

For that case, we had a lot of samples, because data was disaggregated. But what if all we have is aggregated data on the city level? For instance, let's pretend all we have is the average level of deposits in both cities before and after the intervention.

city	before	after
FL	171.64	206.16
POA	46.01	87.06

We would still be able to compute the Diff-in-Diff estimator

$$(E[Y(1)|D=1] - E[Y(1)|D=0]) - (E[Y(0)|D=1] - E[Y(0)|D=0]) = (87.06 - 206.16) - (46.01 - 171.64) = 6.53$$

However, note that the sample size here is 4, which is also the number of parameters in our Diff-in-Diff models. In this case, the standard error is not well defined, so what should we do? Another problem is that Florianopolis might not be as similar to Porto Alegre as we would want to. For instance, Florianopolis is known for its beautiful beaches and easy going people while Porto Alegre is more famous for its barbecue and prairies. The problem here is that you can't ever know for sure if you are using an appropriate control group.

To work around this, we will use what is known as **"the most important innovation in the policy evaluation literature in the last few years"**[1], Synthetic Controls. It is based on a simple, yet powerful idea. We don't need to find any single unit in the untreated that is very similar to the treated. Instead, we can forge our own as a combination of multiple untreated units, creating what is effectively a synthetic control. Synthetic control is so effective yet so intuitive that it even got an article published, not on a scientific journal, but on the Washington Post[2].

```
import warnings
warnings.filterwarnings('ignore')

import pandas as pd
import numpy as np
from matplotlib import style
from matplotlib import pyplot as plt
import seaborn as sns
import statsmodels.formula.api as smf

%matplotlib inline

pd.set_option("display.max_columns", 6)
style.use("fivethirtyeight")
```

To see it in action, consider the problem of estimating the effect of cigarette taxation on its consumption. To give a bit of context, this is a question that had been debated for a long time in economics. One side of the argument says that taxes will increase the cost of cigars, which will lower its demand. The other side argues that since cigarettes cause addiction, change in their price won't change their demand by much. In economic terms, we would say that the demand for cigarettes is inelastic on price, and an increase in taxation is just a way to increase government income at the cost of smokers. To settle things, we will look at some US data regarding the matter.

In 1988, California passed a famous Tobacco Tax and Health Protection Act, which became known as Proposition 99[3]. "Its primary effect is to impose a 25-cent per pack state excise tax on the sale of tobacco cigarettes within California, with approximately equivalent excise taxes similarly imposed on the retail sale of other commercial tobacco products, such as cigars and chewing tobacco. Additional restrictions placed on the sale of tobacco include a ban on cigarette vending machines in public areas accessible by juveniles, and a ban on the individual sale of single cigarettes. Revenue generated by the act was earmarked for various environmental and health care programs, and anti-tobacco advertisements."

To evaluate its effect, we can gather data on cigarette sales from multiple states and across a number of years. In our case, we got data from the year 1970 to 2000 from 39

1 https://www.aeaweb.org/articles?id=10.1257/jep.31.2.3

2 https://www.washingtonpost.com/news/wonk/wp/2015/10/30/how-to-measure-things-in-a-world-of-competing-claims/

3 https://en.wikipedia.org/wiki/1988_California_Proposition_99

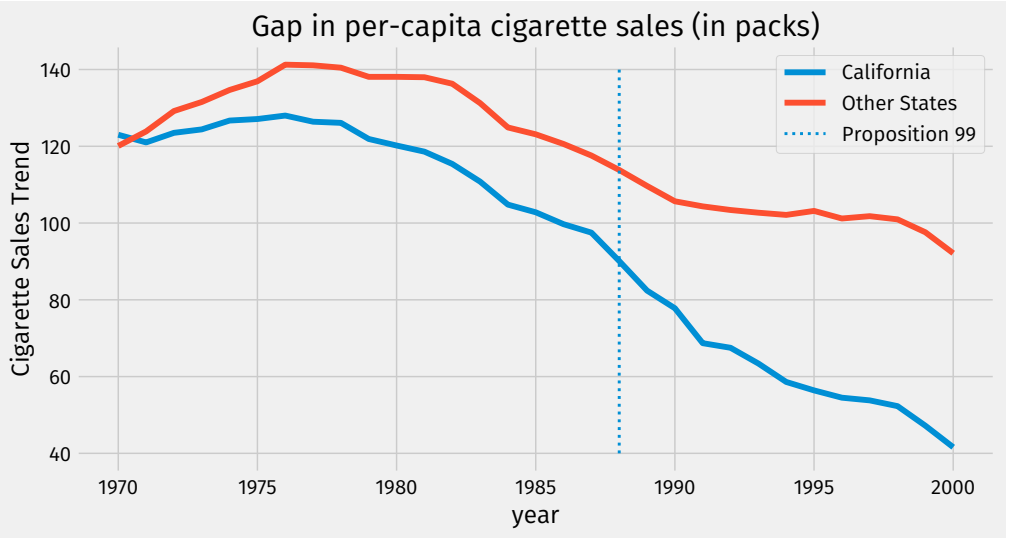
states. Other states had similar Tobacco control programs and were dropped from the analysis. Here is what our data looks like.

```
cigar = (pd.read_csv("data/smoking.csv")
         .drop(columns=["lnincome","beer", "age15to24"]))

cigar.query("california").head()
```

	state	year	cigsale	retprice	california	after_treatment
62	3	1970	123.000000	38.799999	True	False
63	3	1971	121.000000	39.700001	True	False
64	3	1972	123.500000	39.900002	True	False
65	3	1973	124.400002	39.900002	True	False
66	3	1974	126.699997	41.900002	True	False

We have `state` as the state index, where California is the number 3. Our covariates are `retprice`, the cigarette retail price, and `cigsale`, the per-capita sales of cigarettes in packs. Our outcome variable of interest is `cigsale`. Finally, we have boolean helper variables to signal the state of California and the post intervention period. If we plot the sales of cigarettes for California and other states across time, this is what we would get.

```
ax = plt.subplot(1, 1, 1)

(cigar
 .assign(california = np.where(cigar["california"], "California",
  ↪ "Other States"))
 .groupby(["year", "california"])
 ["cigsale"]
 .mean()
 .reset_index()
 .pivot(index="year", columns="california", values="cigsale")
 .plot(ax=ax, figsize=(10,5)))

plt.vlines(x=1988, ymin=40, ymax=140, linestyle=":", lw=2,
 ↪ label="Proposition 99")
plt.ylabel("Cigarette Sales Trend")
plt.title("Gap in per-capita cigarette sales (in packs)")
plt.legend();
```



During the time for which we have data, people in California apparently bought less cigarettes than the national average. Also, it appears to be a decreasing movement in cigarette consumption after the 80s. It looks like after Proposition 99 the decreasing trend accelerated for California, compared to other states, but we can't say that for sure. It is just a guess that we have by examining the plot.

To answer the question of whether Proposition 99 had an effect on cigarette consumption, we will use the pre-intervention period to build a synthetic control. We will combine the other states to **build a fake state that resembles very closely the trend of California**. Then, we will see how this synthetic control behaves after the intervention.

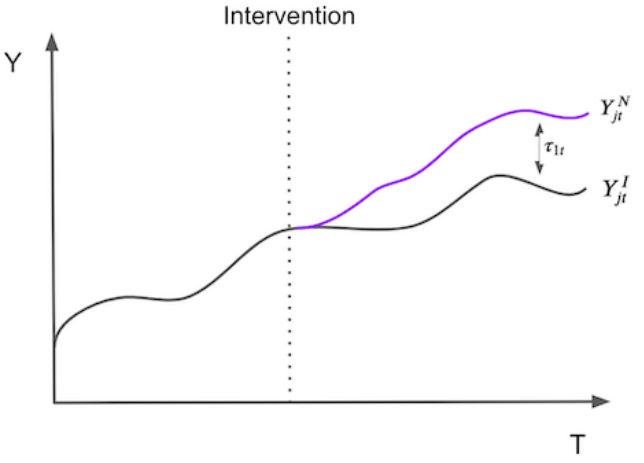
We have Time

To make matters a little bit more formal, suppose that we have $J + 1$ units. Without loss of generality, assume that unit 1 is the unit that gets affected by an intervention. Units $j = 2, ..., J + 1$ are a collection of untreated units that we will refer to as the "donor pool". Also assume that the data we have span T time periods, with T_0 periods before the intervention. For each unit j and each time t, we observe the outcome Y_{jt}. For each unit j and period t, define Y_{jt}^N as the potential outcome without intervention and Y_{jt}^I, the potential outcome with intervention. Then, the effect for the treated unit $j = 1$ at time t, for $t > T_0$ is defined as

$$\tau_{1t} = Y_{1t}^I - Y_{1t}^N$$

Since unit $j = 1$ is the treated one, Y_{1t}^I is factual but Y_{1t}^N is not. The challenge then becomes how do we estimate Y_{1t}^N. Notice how the treatment effect is defined for each period, which means it can change in time. It doesn't need to be instantaneous. It can accumulate or dissipate. To put it in a picture, the problem of estimating the treatment

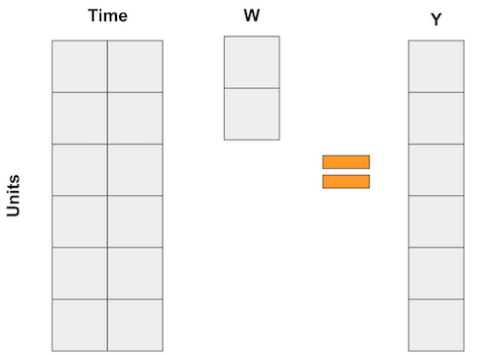
effect boils down to the problem of **estimating what would have happened to the outcome of unit** $j = 1$ **if it had not been treated**.



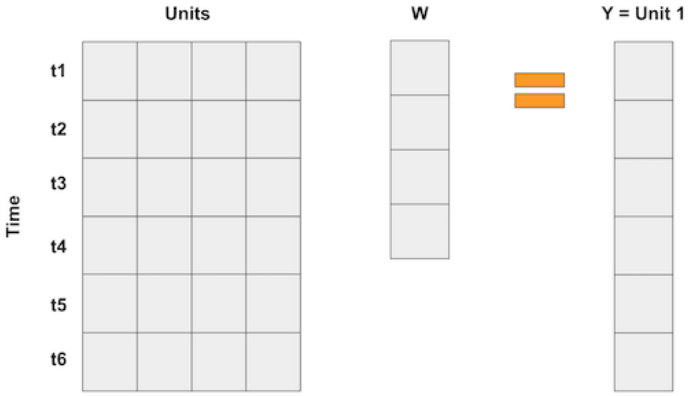
To estimate Y_{1t}^N, we remember that a combination of units in the donor pool may approximate the characteristics of the treated unit much better than any untreated unit alone. Thus, a synthetic control is defined as a weighted average of the units in the control pool. Given the weights $\boldsymbol{W} = (w_2, ..., w_{J+1})$, the synthetic control estimate of Y_{1t}^N is

$$\hat{Y}_{1t}^N = \sum_{j=2}^{J+1} w_j Y_{jt}$$

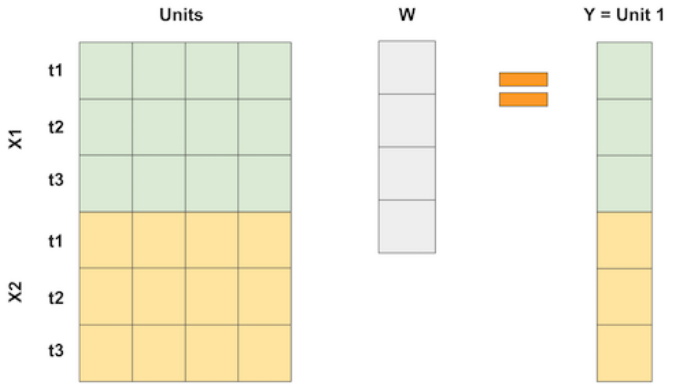
If all this math makes your head hurt, you are not alone. But don't worry, we have lots of examples to make it more intuitive. For once, I like to think about synthetic control as an upside down way of doing regression. As we know, linear regression is also a way of getting the prediction as a weighted average of the variables. Now, think about those regressions like the one in the diff-in-diff example where each variable is a dummy for a time period. In this case, regression can be represented as the following matrix multiplication



On the synthetic control case, we don't have lots of units, but we do have lots of time periods. So what we do is flip the input matrix around. Then, the units become the "variables" and we represent the outcome as a weighted average of the units, like in the following matrix multiplication.



If we have more than one feature per time period, we can pile up the features like this. The important thing is to make it so that the regression is trying to "predict" the treated unit 1 by using the other units. This way, we can choose the weights in some optimal way to achieve this proximity we want. We can even scale features differently to give different importance to them.



So, if synthetic control can be viewed as a linear regression, it also means that we can estimate its weights with OLS right? Yup! In fact, let's do this now.

Synthetic Control as Linear Regression

To estimate the treatment effect with synthetic control, we will try to build a "fake unit" that resembles the treated unit before the intervention period. Then, we will see how this "fake unit" behaves after the intervention. The difference between the synthetic control and the unit that it mimics is the treatment effect.

To do this with linear regression, we will find the weight using OLS. We will minimise the square distance between the weighted average of the units in the donor pool and the treated unit for the pre-intervention period.

To do so, the first thing we need is to convert the units (in our case, the states) into the columns and the time into the rows. Since we have 2 features, `cigsale` and `retprice`, we will pile them on top of each other like we did in the picture above. We will build a synthetic control that looks a lot like California in the pre intervention period and see how it would behave in the post intervention period. For this reason, it is important that we select only the pre-intervention period. Here, the features seem to be on a similar scale, so we won't do anything to them. If features are in different scales, one in the thousands and another in the decimals, the bigger feature will be the most important when minimizing the difference. To avoid this, it's important to scale them first.

```
features = ["cigsale", "retprice"]

inverted = (cigar.query("~after_treatment") # filter pre-intervention period
            .pivot(index='state', columns="year")[features]
              ↪ # make one column per year and one row per state
            .T) # flip the table to have one column per state

inverted.head()
```

	state	1	2	3	...	37	38	39
	year							
cigsale	1970	89.800003	100.300003	123.000000	...	114.500000	106.400002	132.199997
	1971	95.400002	104.099998	121.000000	...	111.500000	105.400002	131.699997
	1972	101.099998	103.900002	123.500000	...	117.500000	108.800003	140.000000
	1973	102.900002	108.000000	124.400002	...	116.599998	109.500000	141.199997
	1974	108.199997	109.699997	126.699997	...	119.900002	111.800003	145.800003

5 rows × 39 columns

Now, we can define our Y variable as the state of California and the X as the other states.

```
y = inverted[3].values # state of california
X = inverted.drop(columns=3).values  # other states
```

Then, we run a regression. Having an intercept is equivalent to adding another state where every row is 1. You can do that, but I think it's more complicated and I'll just leave it out. The regression will return the set of weights that minimize the square difference between the treated unit and the units in the donor pool.

```
from sklearn.linear_model import LinearRegression
weights_lr = LinearRegression(fit_intercept=False).fit(X, y).coef_
weights_lr.round(3)
```

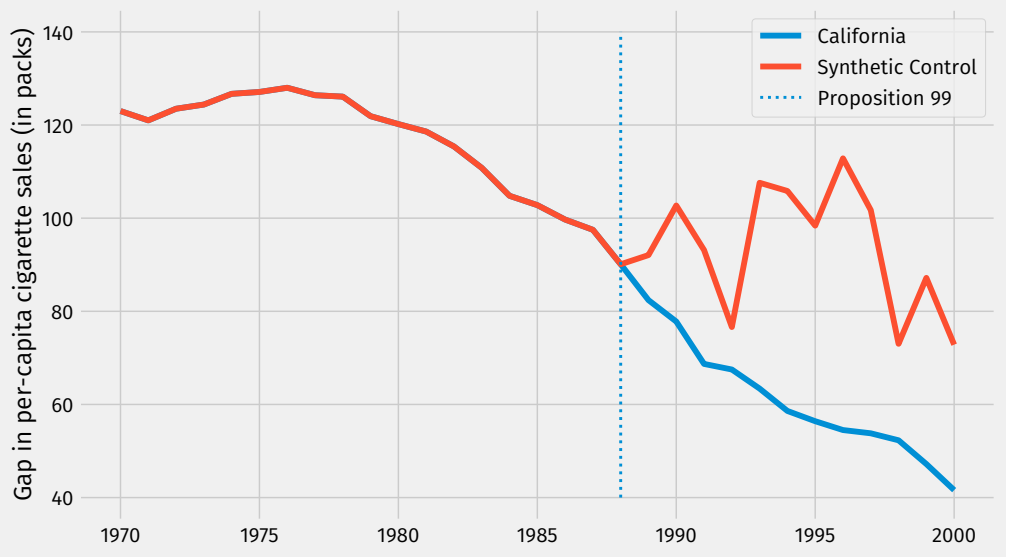
```
array([-0.436, -1.038,  0.679,  0.078,  0.339,  1.213,  0.143,  0.555,
       -0.295,  0.052, -0.529,  1.235, -0.549,  0.437, -0.023, -0.266,
       -0.25 , -0.667, -0.106, -0.145,  0.109,  0.242, -0.328,  0.594,
        0.243, -0.171, -0.02 ,  0.14 , -0.811,  0.362,  0.519, -0.304,
        0.805, -0.318, -1.246,  0.773, -0.055, -0.032])
```

These weights show us how to build the synthetic control. We will multiply the outcome of state 1 by -0.436, of state 2 by -1.038, of state 4 by 0.679 and so on. We can achieve this with a dot product between the matrix from the states in the pool and the weights.

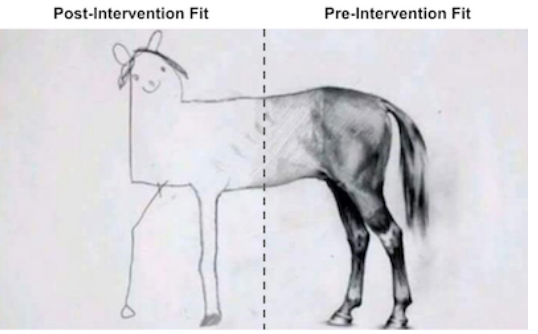
```
calif_synth_lr = (cigar.query("~california")
                  .pivot(index='year', columns="state")["cigsale"]
                  .values.dot(weights_lr))
```

Now that we have our synthetic control, we can plot it with the outcome variable of the State of California.

```
plt.figure(figsize=(10,6))
plt.plot(cigar.query("california")["year"],
  ↪ cigar.query("california")["cigsale"], label="California")
plt.plot(cigar.query("california")["year"], calif_synth_lr,
  ↪ label="Synthetic Control")
plt.vlines(x=1988, ymin=40, ymax=140, linestyle=":", lw=2,
  ↪ label="Proposition 99")
plt.ylabel("Gap in per-capita cigarette sales (in packs)")
plt.legend();
```



OK… Something seems off. What grabs your attention in this picture? First, after the intervention, the synthetic control has more cigarette sales than California. This is an indicative that the intervention was successful in lowering cigarette demand. Second, notice how the pre-intervention period is fitted perfectly. The synthetic control is able to match the state of California exactly. This is a sign that our synthetic control model is probably overfitting the data. Another sign is the huge variance on the outcome variable of the synthetic control after the intervention. Notice how it doesnt follow smooth patterns. Instead, it goes up and down and up and down.



If we think about why this is happening, remember that we have 38 states in our donor pool. So our linear regression has 38 parameters to play with in order to make the pretreatment pool match the treatment as close as it can. This is the case where, even if T is large, N is also large, which gives too much flexibility to our linear regression model. If you are familiar with regularized models, know that you could use Ridge or Lasso regression to fix this. Here, we will look at another more traditional way to avoid overfitting.

Don't Extrapolate

Suppose you have data like in this table below and are asked to build a synthetic control to reproduce the treated unit using any linear combination of the control units.

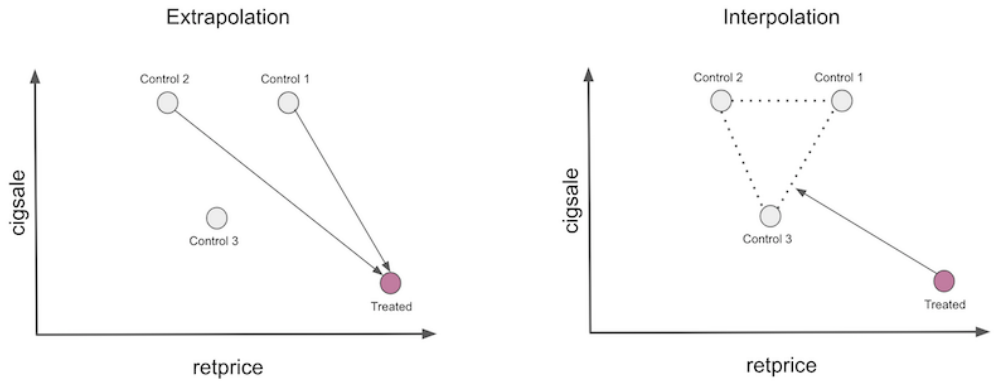
unit	sales	price
control 1	8	8
control 2	8	4
control 3	4	5
treated	2	10

Since there are 3 units and only 2 attributes to match, there are multiple exact solutions to this problem, but a nice one is multiplying the first control by 2.25, multiplying the second by -2 and adding both. Notice how the second multiplication creates a fake unit with sales of -16 and price of -8. This multiplication is extrapolating the control 2 unit to a region of the data that doesn't make a lot of sense, since negative price and sales are almost impossible. The first multiplication is also an extrapolation, since it takes the first unit to a region where sales and price are 18. These numbers are much higher than anything we have in our data, hence the extrapolation.

This is what regression is doing when we ask it to create a synthetic control. Extrapolation is not technically wrong, but it's dangerous in practice. We are making assumptions that the data we have never seen behaves like the data that we have.

One way to play safer is to constrain our synthetic control to only do interpolation. To do so, we will restrict the weights to be positive and sum up to one. Now, the synthetic control will be a convex combination of the units in the donor pool. When doing interpolation,

we will project the treated unit in the convex hull defined by the untreated unit, much like in the picture below.



Notice two things here. First, interpolation won't be able to create a perfect match of the treated unit in this case. This is because the treated is the unit with the smallest number of sales and the highest price. Convex combinations can only replicate exactly features that are in between the control units. Another thing to notice is that interpolation is sparse. We will project the treated unit on a wall of the convex hull and this wall is defined only by a few units. For this reason, interpolation will assign weight zero to many of the units.

This is the general idea, now let's formalize it a little bit. The synthetic control is still defined as

$$\hat{Y}_{jt}^{N} = \sum_{j=2}^{J+1} w_j Y_{jt}$$

but now, we will use weights $\boldsymbol{W} = (w_2, ..., w_{J+1})$ that minimises

$$||\boldsymbol{X}_1 - \boldsymbol{X}_0\boldsymbol{W}|| = \left(\sum_{h=1}^{k} v_h \left(X_{h1} - \sum_{j=2}^{J+1} w_j X_{hj} \right)^2 \right)^{\frac{1}{2}}$$

subject to the restriction that $w_2, ..., w_{J+1}$ are positive and sum to one. Notice that v_h reflect the importance of each variable when minimising the difference between the treated and the synthetic control. Different vs would give different optimal weights. One way to choose V is to make it so that each variable has mean zero and unit variance. A more complex way is to choose V in such a way that variables that help to predict Y better get higher importance. Since we want to keep the code simple, we will simply give the same importance for each variable.

To implement this, first, define the above loss function.

```
from typing import List
from operator import add
from toolz import reduce, partial

def loss_w(W, X, y) -> float:
    return np.sqrt(np.mean((y - X.dot(W))**2))
```

Since we are using the same importance for every feature, we don't need to worry about v.

Now, to get the optimal weights, we will use the quadratic programming optimisation of scipy. We will constrain the weights to sum up to 1 with

```
lambda x: np.sum(x) - 1
```

Also, we will set optimization bounds to be between 0 and 1.

```
from scipy.optimize import fmin_slsqp

def get_w(X, y):

    w_start = [1/X.shape[1]]*X.shape[1]

    weights = fmin_slsqp(partial(loss_w, X=X, y=y),
                         np.array(w_start),
                         f_eqcons=lambda x: np.sum(x) - 1,
                         bounds=[(0.0, 1.0)]*len(w_start),
                         disp=False)
    return weights
```

With this implemented, let's get the weights that define the synthetic control

```
calif_weights = get_w(X, y)
print("Sum:", calif_weights.sum())
np.round(calif_weights, 4)
```

```
Sum: 1.0000000000003093
array([0.    , 0.    , 0.    , 0.0852, 0.    , 0.    , 0.    , 0.    ,
       0.    , 0.    , 0.    , 0.    , 0.    , 0.    , 0.    , 0.    ,
       0.    , 0.    , 0.    , 0.113 , 0.1051, 0.4566, 0.    , 0.    ,
       0.    , 0.    , 0.    , 0.    , 0.    , 0.    , 0.    , 0.    ,
       0.2401, 0.    , 0.    , 0.    , 0.    , 0.    ])
```

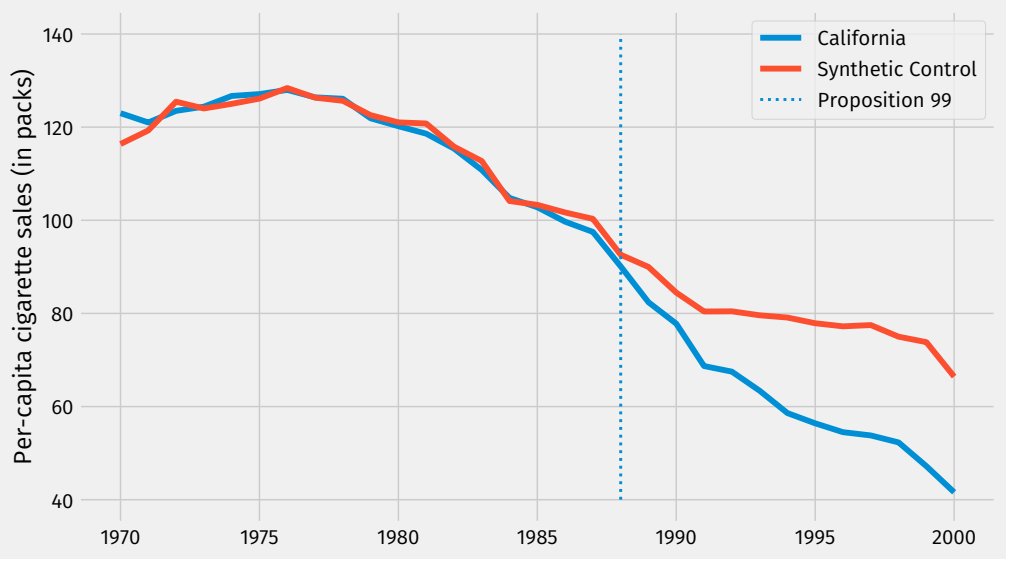
With this weight, we are multiplying states 1,2, and 3 by zero, state 4 by 0.0852 and so on. Notice how the weights are sparse, exactly as we've predicted. Also, all weights sum to one and are between 0 and 1, satisfying our convex combination constraint.

Now, to get the synthetic control, we can multiply those weights by the states exactly as we did before with the regression weights.

```
calif_synth = cigar.query("~california").pivot(index='year',
  ↪ columns="state")["cigsale"].values.dot(calif_weights)
```

If we plot the outcome of the synthetic control now, we get a much smoother trend. Also notice that, in the pre intervention period, the synthetic control doesn't reproduce the treated exactly anymore. This is a good sign, as it indicates that we are not overfitting.

```
plt.figure(figsize=(10,6))
plt.plot(cigar.query("california")["year"],
  ↪ cigar.query("california")["cigsale"], label="California")
plt.plot(cigar.query("california")["year"], calif_synth,
  ↪ label="Synthetic Control")
plt.vlines(x=1988, ymin=40, ymax=140, linestyle=":", lw=2,
  ↪ label="Proposition 99")
plt.ylabel("Per-capita cigarette sales (in packs)")
plt.legend();
```

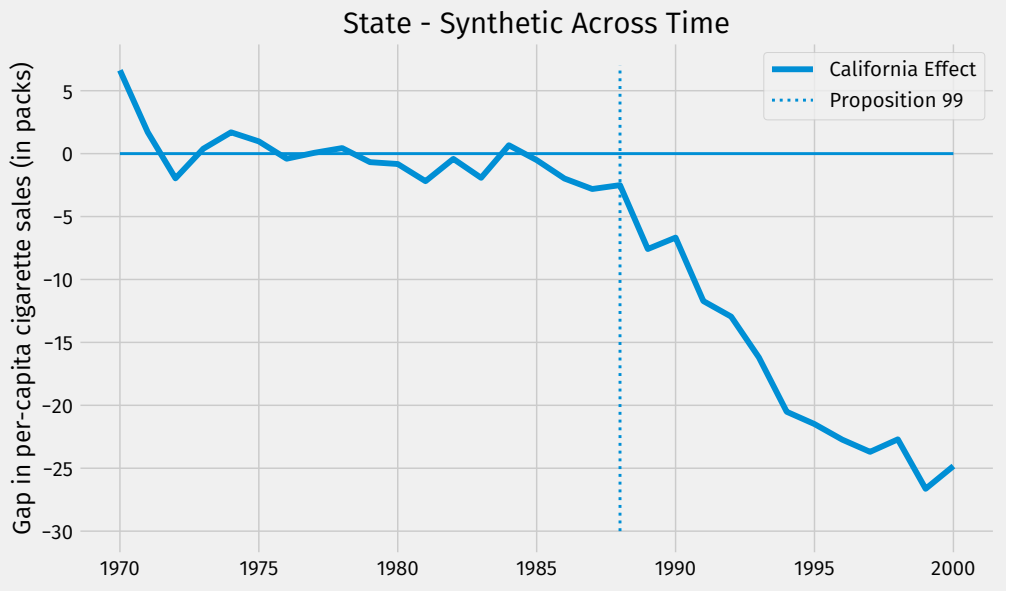


With the synthetic control at hand, we can estimate the treatment effect as the gap between treated and the synthetic control outcomes.

$$\tau_{1t} = Y^I_{jt} - Y^N_{jt}$$

In our particular case, the effect gets bigger and bigger as time passes.

```
plt.figure(figsize=(10,6))
plt.plot(cigar.query("california")["year"], cigar.query("california")["cigsale"]
  ↪ - calif_synth,
         label="California Effect")
plt.vlines(x=1988, ymin=-30, ymax=7, linestyle=":", lw=2,
  ↪ label="Proposition 99")
plt.hlines(y=0, xmin=1970, xmax=2000, lw=2)
plt.title("State - Synthetic Across Time")
plt.ylabel("Gap in per-capita cigarette sales (in packs)")
plt.legend();
```



By the year 2000, it looks like Proposition 99 has reduced the sales in cigarettes by 25 packs. That is very cool and all, but something you might be asking yourself is: how can I know if this is statistically significant?

Making Inference

Since our sample size is very small (39), we will have to be a bit smarter when figuring out if our result is statistically significant and not just due to random luck. Here, we will use the idea of Fisher's Exact Test. Its intuition is very simple. We permute the treated and control exhaustively. Since we only have one treated unit, this would mean that, for each unit, we pretend it is the treated while the others are the control.

iteration	1	2	...	39
1	treated	0	0	0
2	0	treated	0	0
...	0	0	0	0

iteration	1	2	...	39
39	0	0	0	treated

In the end, we will have one synthetic control and effect estimates for each state. So what this does is it pretends that the treatment actually happened for another state, not California, and see what would have been the estimated effect for this treatment that didn't happen. Then, we see if the treatment in California is sufficiently larger when compared to the other fake treatment. The idea is that for states that weren't actually treated, once we pretend they were, we won't be able to find any significant treatment effect.

To implement this, I've built this function that takes as input a state and estimate the synthetic control for that state. This function returns a data frame with one column for the state, one for the year, one for the outcome `cigsale` and the synthetic outcome for that state.

```
def synthetic_control(state: int, data: pd.DataFrame) -> np.array:

    features = ["cigsale", "retprice"]

    inverted = (data.query("~after_treatment")
                .pivot(index='state', columns="year")[features]
                .T)

    y = inverted[state].values # treated
    X = inverted.drop(columns=state).values # donor pool

    weights = get_w(X, y)
    synthetic = (data.query(f"~(state=={state})")
                 .pivot(index='year', columns="state")["cigsale"]
                 .values.dot(weights))

    return (data
            .query(f"state=={state}")[["state", "year", "cigsale",
              ↪ "after_treatment"]]
            .assign(synthetic=synthetic))
```

Here is the result of it when we apply it to the first state.

```
synthetic_control(1, cigar).head()
```

	state	year	cigsale	after_treatment	synthetic
0	1	1970	89.800003	False	95.029419
1	1	1971	95.400002	False	99.118199
2	1	1972	101.099998	False	101.881329
3	1	1973	102.900002	False	103.938655
4	1	1974	108.199997	False	107.038474

To get the result for all the state, we parallelize the computation across 8 processes. If your computer has more or less cores, you can use a different number. This code will return a list of data frames like the one above.

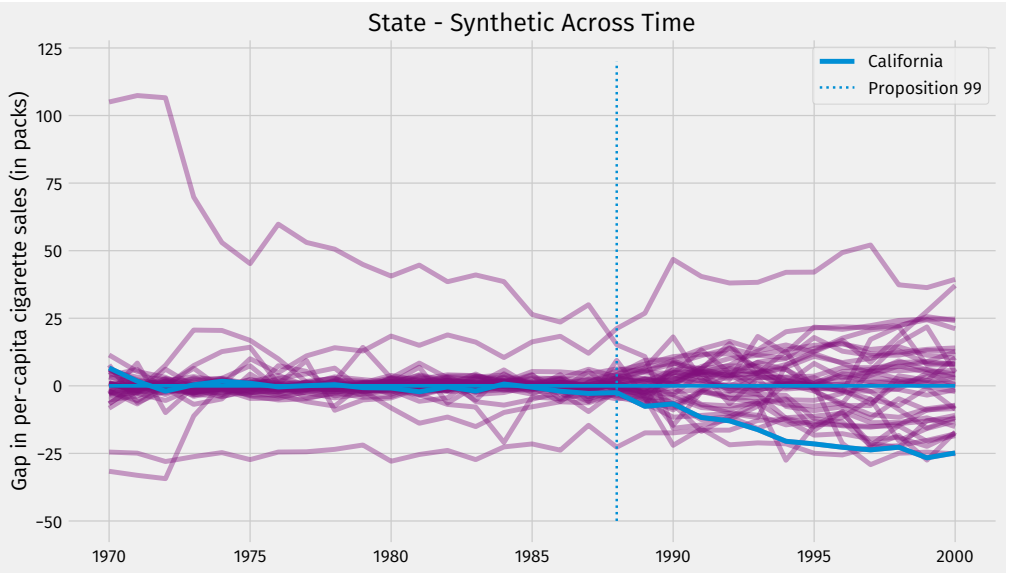
```
from joblib import Parallel, delayed

control_pool = cigar["state"].unique()

parallel_fn = delayed(partial(synthetic_control, data=cigar))

synthetic_states = Parallel(n_jobs=8)(parallel_fn(state) for state in
  ↪ control_pool)
```

```
synthetic_states[0].head()
```

	state	year	cigsale	after_treatment	synthetic
0	1	1970	89.800003	False	95.029419
1	1	1971	95.400002	False	99.118199
2	1	1972	101.099998	False	101.881329
3	1	1973	102.900002	False	103.938655
4	1	1974	108.199997	False	107.038474

With the synthetic control for all the states, we can estimate the gap between the synthetic and the true state for all states. For California, this is the treatment effect. For the other states, this is like a placebo effect, where we estimate the synthetic control treatment effect where the treatment didn't actually happen. If we plot all the placebo effects along with the California treatment effect, we get the following figure.

```
plt.figure(figsize=(12,7))
for state in synthetic_states:
    plt.plot(state["year"], state["cigsale"] - state["synthetic"], color="C5",
      ↪ alpha=0.4)

plt.plot(cigar.query("california")["year"], cigar.query("california")["cigsale"]
  ↪ - calif_synth,
         label="California");

plt.vlines(x=1988, ymin=-50, ymax=120, linestyle=":", lw=2,
  ↪ label="Proposition 99")
plt.hlines(y=0, xmin=1970, xmax=2000, lw=3)
plt.ylabel("Gap in per-capita cigarette sales (in packs)")
plt.title("State - Synthetic Across Time")
plt.legend();
```



Two aspects of this figure jump to the eyes. First, we can see that the variance after the intervention is higher than the variance before the intervention. This is expected, since the synthetic control is designed to minimize the difference in the pre-intervention period. Another interesting aspect is that there are some units we can't fit very well even in the pre-intervention period. This is also to be expected. For example, if some states have very high cigarette consumption, no convex combination of the other states will ever match them.

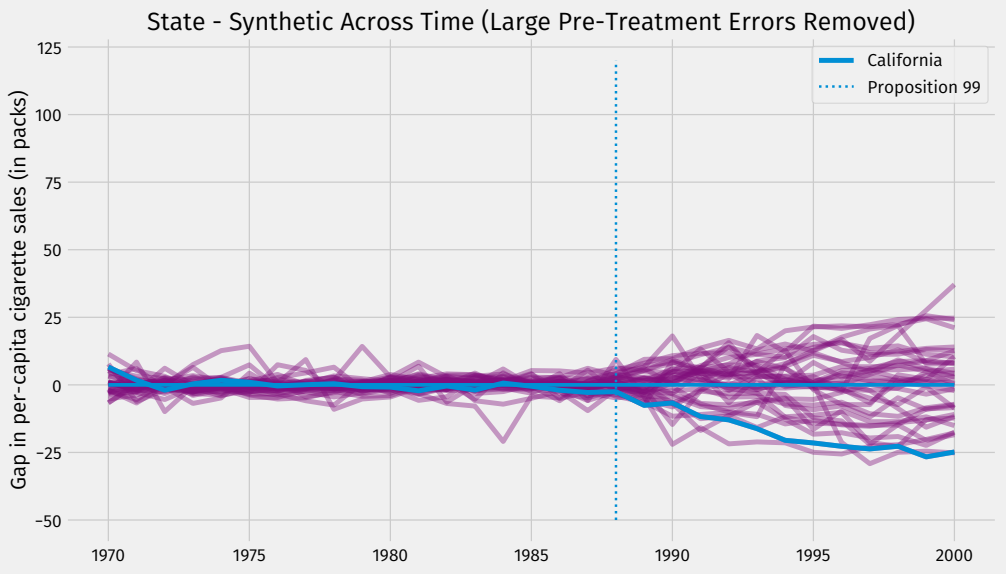
Since those units are so poorly fit, it is a good idea to remove them from the analysis. One way to do it objectively is to set a threshold for pre-intervention error

$$MSE = \frac{1}{N}\sum\left(Y_t - \hat{Y}_t^{Synth}\right)^2$$

and remove those units with high error. If we proceed like this and plot the same figure, this is what we get.

```
def pre_treatment_error(state):
    pre_treat_error = (state.query("~after_treatment")["cigsale"]
                       - state.query("~after_treatment")["synthetic"]) ** 2
    return pre_treat_error.mean()

plt.figure(figsize=(12,7))
for state in synthetic_states:

    # remove units with mean error above 80.
    if pre_treatment_error(state) < 80:
        plt.plot(state["year"], state["cigsale"] - state["synthetic"],
          ↪ color="C5",alpha=0.4)
```

```
plt.plot(cigar.query("california")["year"], cigar.query("california")["cigsale"]
  ↪ - calif_synth,
        label="California");

plt.vlines(x=1988, ymin=-50, ymax=120, linestyle=":", lw=2,
  ↪ label="Proposition 99")
plt.hlines(y=0, xmin=1970, xmax=2000, lw=3)
plt.ylabel("Gap in per-capita cigarette sales (in packs)")
plt.title("Distribution of Effects")
plt.title("State - Synthetic Across Time (Large Pre-Treatment Errors Removed)")
plt.legend();
```



Removing the noise, we can see how extreme of a value is the effect in the state of California. This image shows us that if we pretend the treatment had happened to any other state, we would almost never get an effect so extreme as the one we got with California.

This picture alone is a form of inference, but we can also derive a P-value from these results. All we have to do is see how many times the effects that we've got is below the effect of California.

```
calif_number = 3

effects = [state.query("year==2000").iloc[0]["cigsale"] -
  ↪ state.query("year==2000").iloc[0]["synthetic"]
           for state in synthetic_states
           if pre_treatment_error(state) < 80] # filter out noise

calif_effect = cigar.query("california & year==2000").iloc[0]["cigsale"] -
  ↪ calif_synth[-1]

print("California Treatment Effect for the Year 2000:", calif_effect)
np.array(effects)
```

```
California Treatment Effect for the Year 2000: -24.830159747347786
array([  5.79715884,   0.89458993, -24.83015976,  -7.16628123,
       -10.92204876,  37.11640557, -15.0697173 ,  -0.49805149,
       -18.45795104,  21.13366442,  12.57782757,  -1.47547828,
        10.49627365, -11.67012338,   4.29850835,   8.04811412,
        14.02322417,   8.25002712,   0.32576358,  -8.40826863,
        -2.12402716,  -7.42865018,   2.96157513,  24.10478118,
         4.25211767, -17.75844593,   7.93334017,   2.81640126,
        12.64955984, -17.47677512, -25.16040936, -12.26469139,
        24.69067378,  10.36299576,  -8.59880354])
```

if we want to test the one sided hypothesis that the effect in California is below zero, we can estimate the P-value as the proportion of times the effect in California is bigger than all the estimated effects.

$$PV = \frac{1}{N}\sum 1\{\hat{\tau}_{Calif} > \hat{\tau}_j\}$$

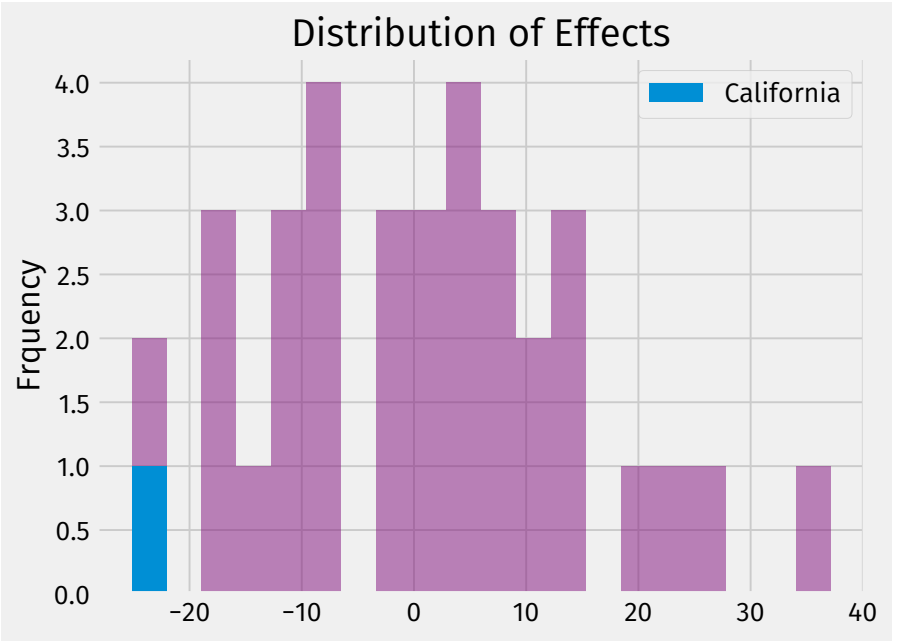
As it turns out, the treatment effect for California in the year 2000 is -24.8, meaning that the intervention reduced the consumption of cigarettes by almost 25 packs. Out of all the other 34 placebo effects that we've estimated, only one is higher than the effect we found in California. So the p-value would be 1/35.

```
np.mean(np.array(effects) < calif_effect)
```

```
np.float64(0.05714285714285714)
```

Finally, we can show the distribution of effects just to get a sense of how extreme the value of the effect in California really is.

```
_, bins, _ = plt.hist(effects, bins=20, color="C5", alpha=0.5);
plt.hist([calif_effect], bins=bins, color="C0", label="California")
plt.ylabel("Frquency")
plt.title("Distribution of Effects")
plt.legend();
```



Key Ideas

We've learned that if we only have aggregated level data on entities like cities or states, diff-in-diff won't allow us to do inference. Also, it has some other limitations, since it has to define a control unit and one single control unit might not be a very good representation of the counterfactual for the treated unit.

To correct for that, we learned that we can build a synthetic control that combines multiple control units to make them resemble the treated unit. With this synthetic control, we were able to see what would have happened to our treated unit in the absence of a treatment.

Finally, we saw how we could use Fisher's Exact Tests to do inference with synthetic control. Namely, we've pretended that the non-treated units were actually the treated and computed their effect. These were the placebo effects: the effects we would observe even without a treatment. We used these to see if the treatment effect we've estimated was statistically significant.

References

I like to think of this entire book as a tribute to Joshua Angrist, Alberto Abadie and Christopher Walters for their amazing Econometrics class. Most of the ideas here are taken from their classes at the American Economic Association. Watching them is what is keeping me sane during this tough year of 2020.

- Cross-Section Econometrics · https://www.aeaweb.org/conference/cont-ed/2017-webcasts

- Mastering Mostly Harmless Econometrics · https://www.aeaweb.org/conference/cont-ed/2020-webcasts

I'll also like to reference the amazing books from Angrist. They have shown me that Econometrics, or 'Metrics as they call it, is not only extremely useful but also profoundly fun.

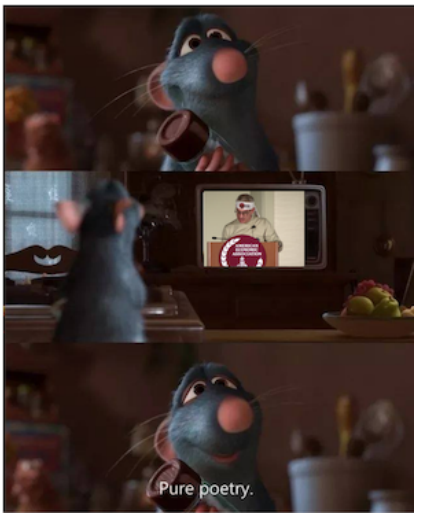
- Mostly Harmless Econometrics · https://www.mostlyharmlesseconometrics.com/
- Mastering 'Metrics · https://www.masteringmetrics.com/

Other important reference is Miguel Hernan and Jamie Robins' book. It has been my trustworthy companion in the most thorny causal questions I had to answer.

- Causal Inference Book · https://miguelhernan.org/whatifbook

Finally, I'd also like to compliment Scott Cunningham and his brilliant work mingling Causal Inference and Rap quotes:

- Causal Inference: The Mixtape · https://www.scunning.com/mixtape.html



CHAPTER
16

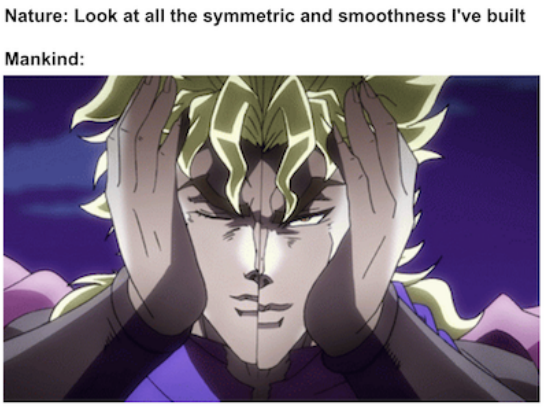
Regression Discontinuity Design

We don't stop to think about it much, but it is impressive how smooth nature is. You can't grow a tree without first getting a bud, you can't teleport from one place to another, a wound takes its time to heal. Even in the social realm, smoothness seems to be the norm. You can't grow a business in one day, consistency and hard work are required to build wealth and it takes years before you learn how linear regression works. Under normal circumstances, nature is very cohesive and doesn't jump around much.

> When the intelligent and animal souls are held together in one embrace, they can be kept from separating.

- Tao Te Ching, Lao Tzu.

Which means that **when we do see jumps and spikes, they are probably artificial** and often man-made situations. These events are usually accompanied by counterfactuals to the normal way of things: if a weird thing happens, this gives us some insight into what would have happened if nature was to work in a different way. Exploring these artificial jumps is at the core of Regression Discontinuity Design.



The basic setup goes like this. Imagine that you have a treatment variable T and potential outcomes Y_0 and Y_1. The treatment T is a discontinuous function of an observed running variable R such that

$$D_i = 1\{R_i > c\}$$

In other words, this is saying that treatment is zero when R is below a threshold c and one otherwise. This means that we get to observe Y_1 when $R > c$ and Y_0 when $R < c$. To wrap our head around this, think about the potential outcomes as 2 functions that we can't observe entirely. Both $Y_0(R)$ and $Y_1(R)$ are there, we just can't see that. The threshold acts as a switch that allows us to see one or the other of those functions, but never both, much like in the image below:

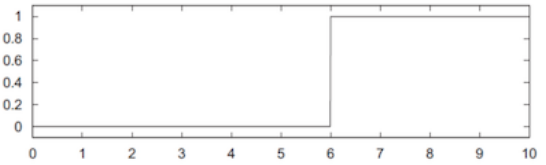


Fig. 1. Assignment probabilities (SRD).

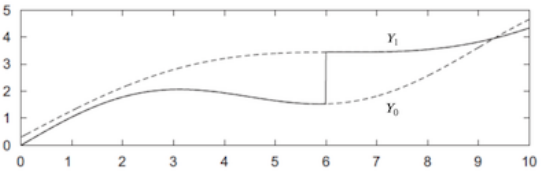


Fig. 2. Potential and observed outcome regression functions.

The idea of regression discontinuity is to compare the outcome just above and just below the threshold to identify the treatment effect at the threshold. This is called a **sharp RD** design, since the probability of getting the treatment jumps from 0 to 1 at the threshold, but we could also think about a **fuzzy RD** design, where the probability also jumps, but is a less dramatic manner.

Is Alcohol Killing You?

A very relevant public policy question is what should be the minimal drinking age. Most countries, Brazil included, set it to be 18 years, but in the US (most states) it is currently 21. So, is it the case that the US is being overly prudent and that they should lower their minimal drinking age? Or is it the case that other countries should make their legal drinking age higher?

One way to look at this question is from a mortality rate perspective (Carpenter and Dobkin, 2009)[1]. From the public policy standpoint, one could argue that we should lower the mortality rate as much as possible. If alcohol consumption increases the mortality rate

1 https://www.aeaweb.org/articles?id=10.1257/app.1.1.164

by a lot, we should avoid lowering the minimum drinking age. This would be consistent with the objective of lowering deaths caused by alcohol consumption.

To estimate the impacts of alcohol on death, we could use the fact that legal drinking age imposes a discontinuity on nature. In the US, those just under 21 years old don't drink (or drink much less) while those just older than 21 do drink. This means that the probability of drinking jumps at 21 years and that is something we can explore with an RDD.

NOTE

You could also argue that turning 21 increases the probability of drinking alcohol, since it is possible to drink it before that age, even though not legally. This would technically make this a case of Fuzzy RD Design, something we'll explore later in this chapter.

```
import warnings
warnings.filterwarnings('ignore')

import pandas as pd
import numpy as np
from matplotlib import style
from matplotlib import pyplot as plt
import seaborn as sns
import statsmodels.formula.api as smf

%matplotlib inline

style.use("fivethirtyeight")
```

To do so we can grab some mortality data aggregated by age. Each row is the average age of a group of people and the average mortality by all causes (`all`), by moving vehicle accident (`mva`) and by suicide (`suicide`).

```
drinking = pd.read_csv("./data/drinking.csv")
drinking.head()[["agecell", "all", "mva", "suicide"]]
```

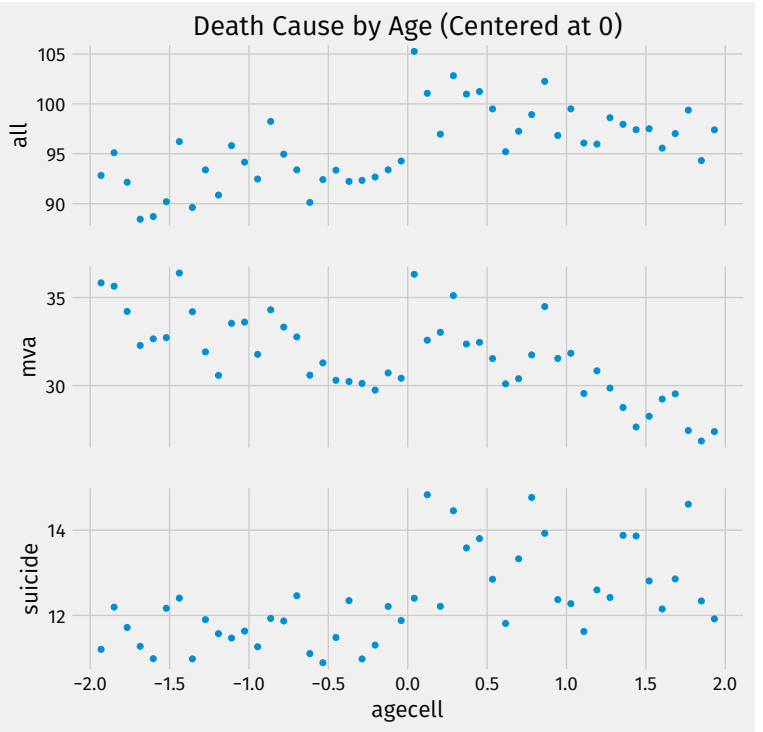
	agecell	all	mva	suicide
0	19.068493	92.825400	35.829327	11.203714
1	19.150684	95.100740	35.639256	12.193368
2	19.232876	92.144295	34.205650	11.715812
3	19.315070	88.427760	32.278957	11.275010
4	19.397260	88.704940	32.650967	10.984314

Just to aid visibility (and for another important reason we will see later) we will centralize the running variable `agecell` at the threshold 21.

```
drinking["agecell"] -= 21
```

If we plot the multiple outcome variables (`all`, `mva`, `suicide`) with the runing variable on the x axis, we get some visual cue about some sort of jump in mortality as we cross the legal drinking age.

```
plt.figure(figsize=(8,8))
ax = plt.subplot(3,1,1)
drinking.plot.scatter(x="agecell", y="all", ax=ax)
plt.title("Death Cause by Age (Centered at 0)")

ax = plt.subplot(3,1,2, sharex=ax)
drinking.plot.scatter(x="agecell", y="mva", ax=ax)

ax = plt.subplot(3,1,3, sharex=ax)
drinking.plot.scatter(x="agecell", y="suicide", ax=ax);
```



There are some cues, but we need more than that. What exactly is the effect of drinking on mortality at the threshold? And what is the standard error on that estimate?

RDD Estimation

The key assumption that RDD relies on is the smoothness of the potential outcome at the threshold. Formally, the limits of the potential outcomes as the running variable

approaches the threshold from the right and from the left should be the same.

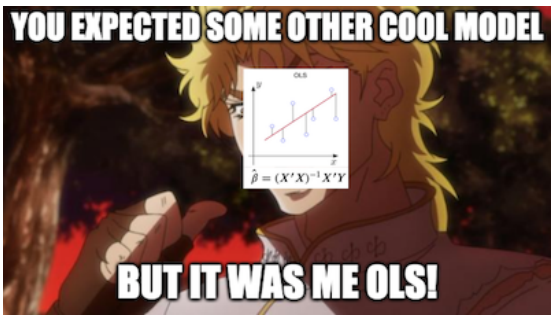
$$\lim_{r \to c^-} E[Y_{ti}|R_i = r] = \lim_{r \to c^+} E[Y_{ti}|R_i = r]$$

If this holds true, we can find the causal effect at the threshold

$$\begin{aligned}\lim_{r \to c^+} E[Y_{ti}|R_i = r] - \lim_{r \to c^-} E[Y_{ti}|R_i = r] &= \lim_{r \to c^+} E[Y_{1i}|R_i = r] - \lim_{r \to c^-} E[Y_{0i}|R_i = r] \\ &= E[Y_{1i}|R_i = c] - E[Y_{0i}|R_i = c] \\ &= E[Y_{1i} - Y_{0i}|R_i = c]\end{aligned}$$

This is, in its own way, a sort of Local Average Treatment Effect (LATE), since we can only know it at the threshold. In this setting, we can think of RDD as a local randomized trial. For those at the threshold, the treatment could have gone either way and, by chance, some people fell below the threshold, and some people fell above. In our example, at the same point in time, some people are just above 21 years and some people are just below 21. What determines this is if someone was born some days later or not, which is pretty random. For this reason, RDD provides a very compelling causal story. It is not the golden standard of RCT, but it is close.

Now, to estimate the treatment effect at the threshold, all we need to do is estimate both of the limits in the formula above and compare them. The simplest way to do that is by running a linear regression



To make it work, we interact a dummy for being above the threshold with the running variable

$$y_i = \beta_0 + \beta_1 r_i + \beta_2 1\{r_i > c\} + \beta_3 1\{r_i > c\} r_i$$

Essentially, this is the same as fitting a linear regression above the threshold and another below it. The parameter β_0 is the intercept of the regression below the threshold and $\beta_0 + \beta_2$ is the intercept for the regression above the threshold.

Here is where the trick of centering the running variable at the threshold comes into play. After this pre-processing step, the threshold becomes zero. This causes the intercept β_0 to be the predicted value at the threshold, for the regression below it. In other words, $\beta_0 = \lim_{r \to c^-} E[Y_{ti}|R_i = r]$. By the same reasoning, $\beta_0 + \beta_2$ is the limit of the outcome from above. Which means, that

$$\lim_{r \to c^+} E[Y_{ti}|R_i = r] - \lim_{r \to c^-} E[Y_{ti}|R_i = r] = \beta_2 = E[ATE|R = c]$$

Here is what this looks like in code for the case where we want to estimate the effect of alcohol consumption on death by all causes at age 21.

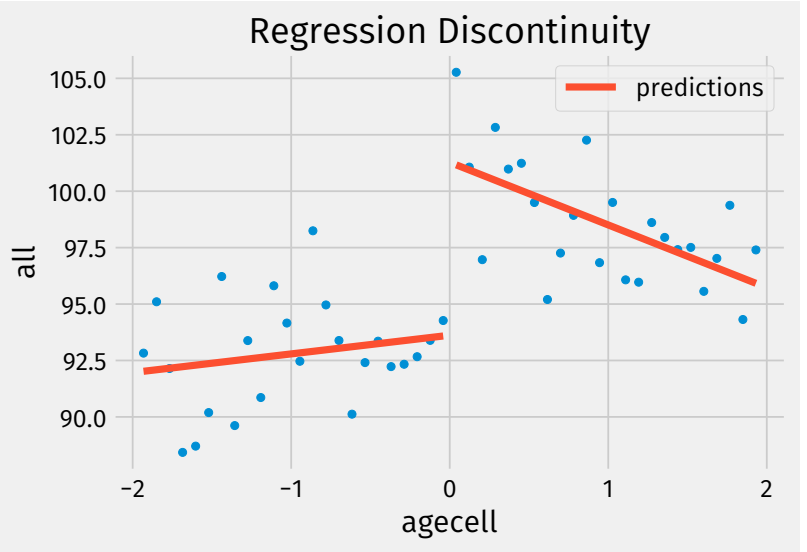
```
rdd_df = drinking.assign(threshold=(drinking["agecell"] > 0).astype(int))

model = smf.wls("all~agecell*threshold", rdd_df).fit()

model.summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	93.6184	0.932	100.399	0.000	91.739	95.498
agecell	0.8270	0.819	1.010	0.318	-0.823	2.477
threshold	7.6627	1.319	5.811	0.000	5.005	10.320
agecell:threshold	-3.6034	1.158	-3.111	0.003	-5.937	-1.269

This model is telling us that mortality increases by 7.6627 points with the consumption of alcohol. Another way of putting this is that alcohol increases the chance of death by all causes by 8% (100*((7.6627+93.6184)/93.6184 - 1)). Notice that this also gives us standard errors for our causal effect estimate. In this case, the effect is statistically significant, since the p-value is below 0.01.

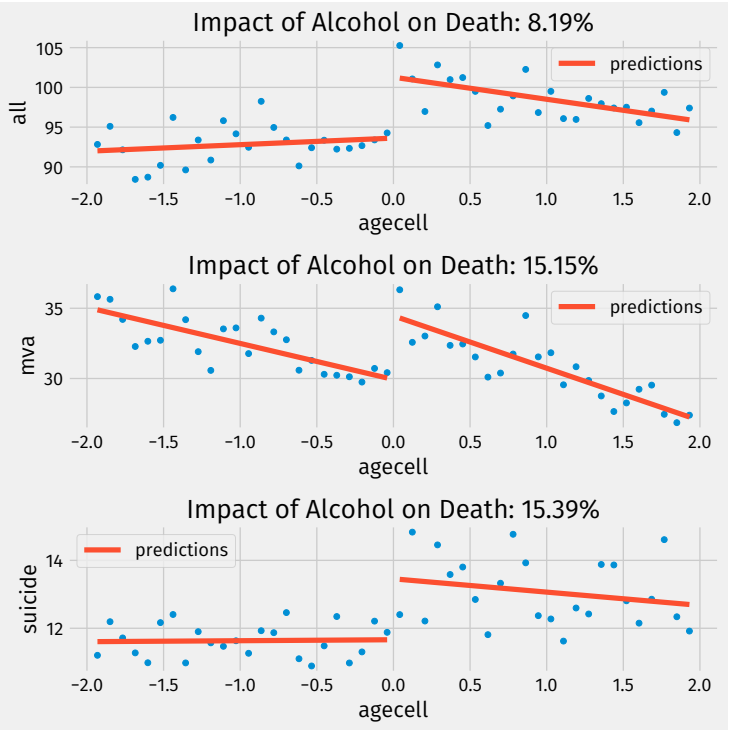
If we want to verify this model visually, we can show the predicted values on the data that we have. You can see that it is as though we had 2 regression models: one for those above the threshold and one for below it.

```
ax = drinking.plot.scatter(x="agecell", y="all", color="C0")
drinking.assign(predictions=model.fittedvalues).plot(x="agecell",
  ↪ y="predictions", ax=ax, color="C1")
plt.title("Regression Discontinuity");
```



If we do the same for the other causes, this is what we get.

```
plt.figure(figsize=(8,8))

for p, cause in enumerate(["all", "mva", "suicide"], 1):
    ax = plt.subplot(3,1,p)
    drinking.plot.scatter(x="agecell", y=cause, ax=ax)
    m = smf.wls(f"{cause}~agecell*threshold", rdd_df).fit()
    ate_pct = 100*((m.params["threshold"] + m.params["Intercept"])/m
      ↪ .params["Intercept"] - 1)
    drinking.assign(predictions=m.fittedvalues).plot(x="agecell",
      ↪ y="predictions", ax=ax, color="C1")
    plt.title(f"Impact of Alcohol on Death: {np.round(ate_pct, 2)}%")

plt.tight_layout()
```



RDD is telling us that alcohol increases the chance of death by suicide and car accidents by 15%, which is a pretty significant amount. These results are compelling arguments to not lower the drinking age, if we want to minimize mortality rates.

Kernel Weighting

Regression Discontinuity relies heavily on the extrapolation properties of linear regression. Since we are looking at the values at the beginning and end of 2 regression lines, we

better get those limits right. What can happen is that regression might focus too much on fitting the other data points at the cost of a poor fit at the threshold. If this happens, we might get the wrong measure of the treatment effect.

One way to solve this is to give higher weights for the points that are closer to the threshold. There are many ways to do this, but a popular one is to reweight the samples with the **triangular kernel**

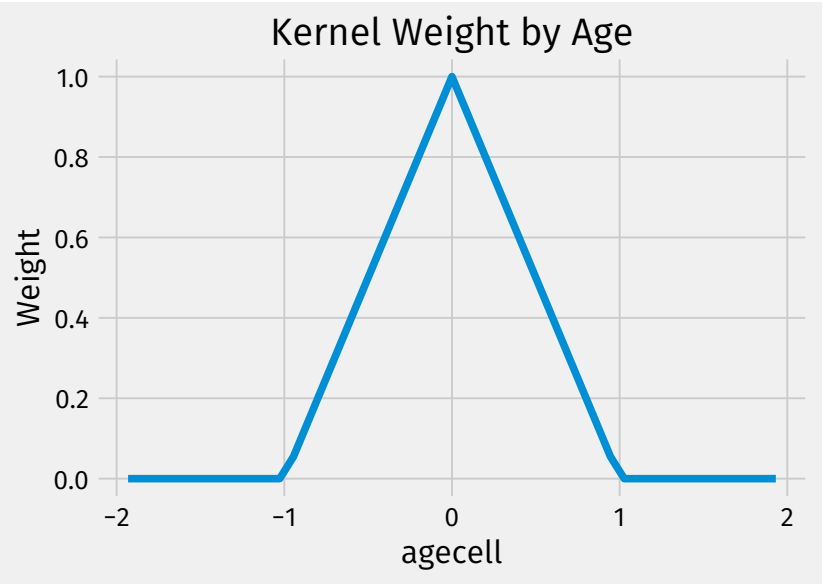
$$K(R, c, h) = 1\{|R - c| \leq h\} * \left(1 - \frac{|R - c|}{h}\right)$$

The first part of this kernel is an indicator function to whether we are close to the threshold. How close? This is determined by a bandwidth parameter h. The second part of this kernel is a weighting function. As we move away from the threshold, the weights get smaller and smaller. These weights are divided by the bandwidth. If the bandwidth is large, the weights get smaller at a slower rate. If the bandwidth is small, the weights quickly go to zero.

To make it easier to understand, here is what the weights look like for this kernel applied to our problem. I've set the bandwidth to be 1 here, meaning we will only consider data from people that are no older than 22 and no younger than 20.

```
def kernel(R, c, h):
    indicator = (np.abs(R-c) <= h).astype(float)
    return indicator * (1 - np.abs(R-c)/h)
```

```
plt.plot(drinking["agecell"], kernel(drinking["agecell"], c=0, h=1))
plt.xlabel("agecell")
plt.ylabel("Weight")
plt.title("Kernel Weight by Age");
```

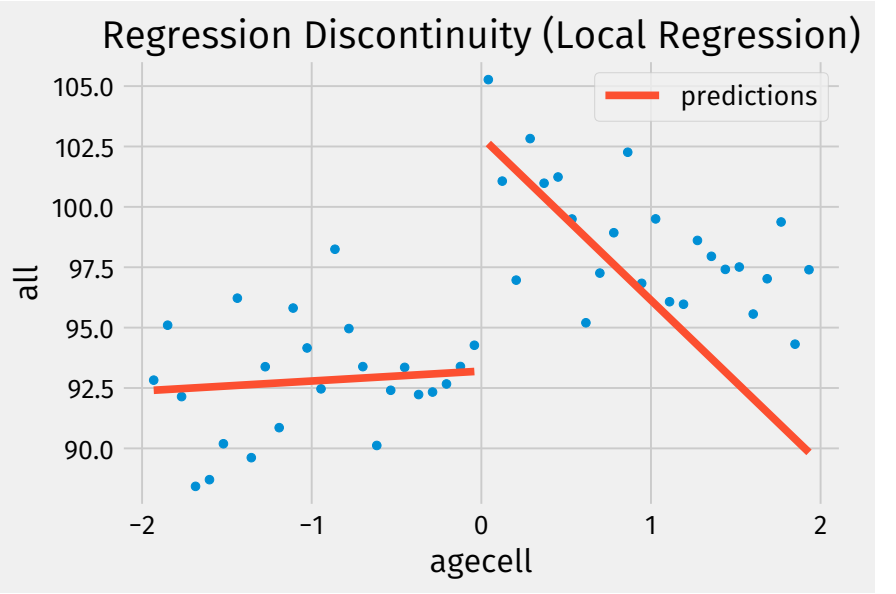


If we apply these weights to our original problem, the impact of alcohol gets bigger, at least for all causes. It jumps from 7.6627 to 9.7004. The result remains very significant. Also, notice that I'm using `wls` instead of `ols`

```
model = smf.wls("all~agecell*threshold", rdd_df,
                weights=kernel(drinking["agecell"], c=0, h=1)).fit()

model.summary().tables[1]
```

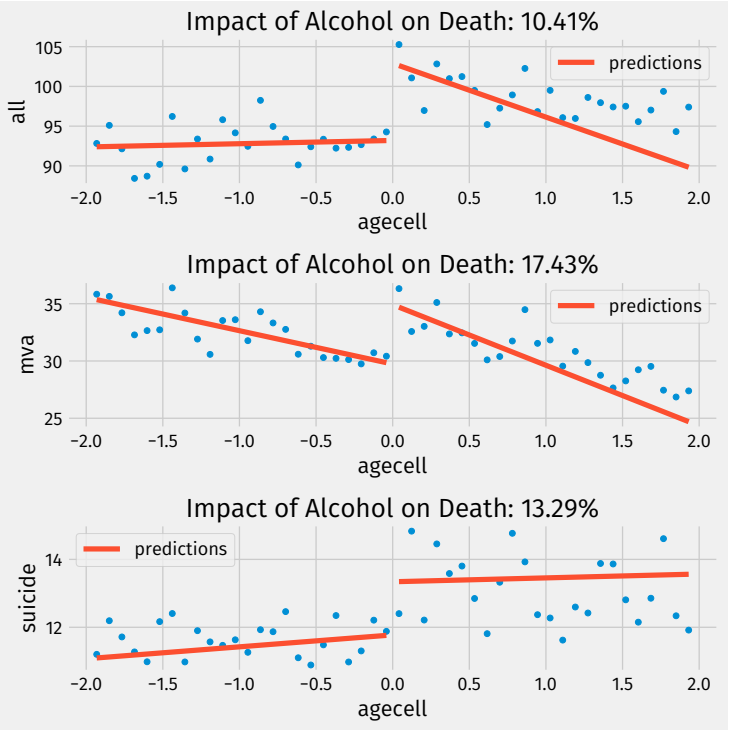
	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	93.2002	0.731	127.429	0.000	91.726	94.674
agecell	0.4109	1.789	0.230	0.819	-3.196	4.017
threshold	9.7004	1.034	9.378	0.000	7.616	11.785
agecell:threshold	-7.1759	2.531	-2.835	0.007	-12.276	-2.075

```
ax = drinking.plot.scatter(x="agecell", y="all", color="C0")
drinking.assign(predictions=model.fittedvalues).plot(x="agecell",
  ↪ y="predictions", ax=ax, color="C1")
plt.title("Regression Discontinuity (Local Regression)");
```



And here is what it looks like for the other causes of death. Notice how the regression on the right is more negatively sloped since it disconsiders the rightmost points.

```
plt.figure(figsize=(8,8))
weights = kernel(drinking["agecell"], c=0, h=1)

for p, cause in enumerate(["all", "mva", "suicide"], 1):
    ax = plt.subplot(3,1,p)
    drinking.plot.scatter(x="agecell", y=cause, ax=ax)
    m = smf.wls(f"{cause}~agecell*threshold", rdd_df, weights=weights).fit()
    ate_pct = 100*((m.params["threshold"] + m.params["Intercept"])/m
      ↪ .params["Intercept"] - 1)
    drinking.assign(predictions=m.fittedvalues).plot(x="agecell",
      ↪ y="predictions", ax=ax, color="C1")
    plt.title(f"Impact of Alcohol on Death: {np.round(ate_pct, 2)}%")

plt.tight_layout()
```



With the exception of suicide, it looks like adding the kernel weight made the negative impact on alcohol bigger. Once again, if we want to minimize the death rate, we should NOT recommend lowering the legal drinking age, since there is a clear impact of alcohol on the death rates.

This simple case covers what happens when regression discontinuity design works perfectly. Next, we will see some diagnostics that we should run in order to check how much we can trust RDD and talk about a topic that is very dear to our heart: the effect of education on earnings.

Sheepskin Effect and Fuzzy RDD

When it comes to the effect of education on earnings, there are two major views in economics. The first one is the widely known argument that education increases human capital, increasing productivity and thus, earnings. In this view, education actually changes you for the better. Another view is that education is simply a signaling mechanism. It just puts you through all these hard tests and academic tasks. If you can make it, it signals to the market that you are a good employee. In this way, education doesn't make you more productive. It only tells the market how productive you have always been. What matters here is the diploma. If you have it, you will be paid more. We refer to this as the **sheepskin effect**, since diplomas were printed in sheepskin in the past.

To test this hypothesis, Clark and Martorell[2] used regression discontinuity to measure the effect of graduating 12th grade on earnings. In order to do that, they had to think about some running variable where students that fall above it graduate and those who fall below it, don't. They found such data in the Texas education system.

In order to graduate in Texas, one has to pass an exam. Testing starts at 10th grade and students can do it multiple times, but eventually, they face a last chance exam at the end of 12th grade. The idea was to get data from students who took those last chance exams and compare those that had barely failed it to those that barely passed it. These students will have very similar human capital, but different signaling credentials. Namely, those that barely passed it, will receive a diploma.

```
sheepskin = pd.read_csv("./data/sheepskin.csv")[["avgearnings", "minscore",
  ↪ "receivehsd", "n"]]
sheepskin.head()
```

	avgearnings	minscore	receivehsd	n
0	11845.086	-30.0	0.416667	12
1	9205.679	-29.0	0.387097	31
2	8407.745	-28.0	0.318182	44
3	11114.087	-27.0	0.377778	45
4	10814.624	-26.0	0.306667	75

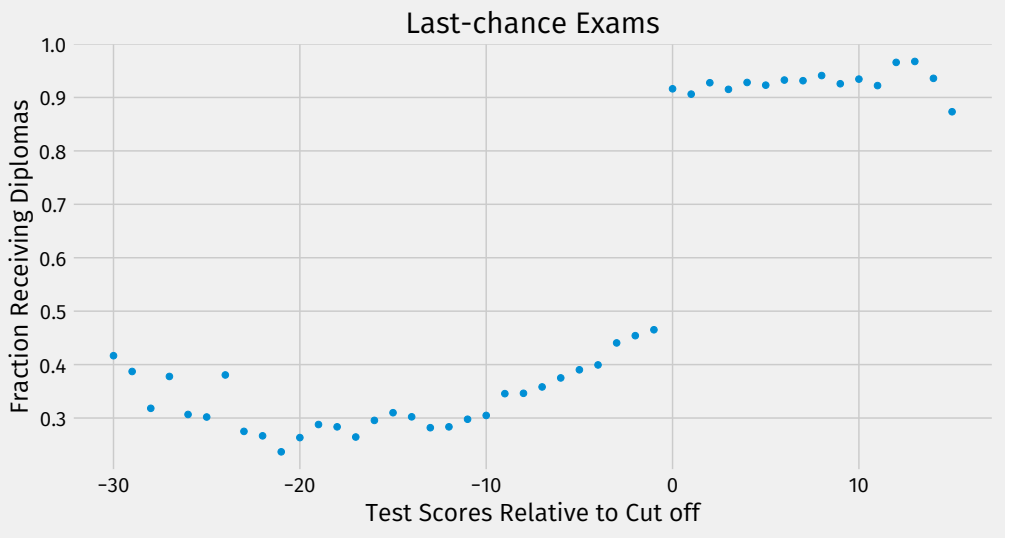
Once again, this data is grouped by the running variable. It contains not only the running variable (minscore, already centered at zero) and the outcome (avgearnings), but it also has the probability of receiving a diploma in that score cell and the size of the call (n). So, for example, out of the 12 students in the cell -30 below the score threshold, only 5 were able to get the diploma (12 * 0,416).

This means that there is some slippage in the treatment assignment. Some students that are below the passing threshold managed to get the diploma anyway. Here, the regression discontinuity is **fuzzy**, rather than sharp. Notice how the probability of getting

2 https://faculty.smu.edu/millimet/classes/eco7321/papers/clark%20martorell%202014.pdf

the diploma doesn't jump from zero to one at the threshold. But it does jump from something like 50% to 90%.

```
sheepskin.plot.scatter(x="minscore", y="receivehsd", figsize=(10,5))
plt.xlabel("Test Scores Relative to Cut off")
plt.ylabel("Fraction Receiving Diplomas")
plt.title("Last-chance Exams");
```



We can think of fuzzy RD as a sort of non compliance. Passing the threshold should make everyone receive the diploma, but some students, the never takers, don't get it. Likewise, being below the threshold should prevent you from getting a diploma, but some students, the always takers, manage to get it anyway.

Just like when we have the potential outcome, we have the potential treatment status in this situation. T_1 is the treatment everyone would have received had they been above the threshold. T_0 is the treatment everyone would have received had they been below the threshold. As you've might have noticed, we can think of the **threshold as an Instrumental Variable**. Just as in IV, if we naively estimate the treatment effect, it will be biased towards zero.

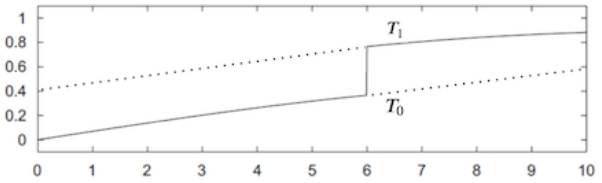


Fig. 3. Assignment probabilities (FRD).

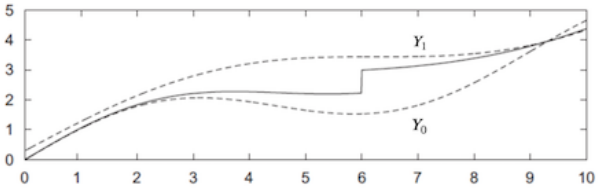


Fig. 4. Potential and observed outcome regression (FRD).

The probability of treatment being less than one, even above the threshold, makes the outcome we observe less than the true potential outcome Y_1. By the same token, the outcome we observe below the threshold is higher than the true potential outcome Y_0. This makes it look like the treatment effect at the threshold is smaller than it actually is and we will have to use IV techniques to correct for that.

Just like when we've assumed smoothness on the potential outcome, we now assume it for the potential treatment. Also, we need to assume monotonicity, just like in IV. In case you don't remember, it states that $T_{i1} \geq T_{i0} \; \forall i$. This means that crossing the threshold from the left to the right only increases your chance of getting a diploma (or that there are no defiers). With these 2 assumptions, we have a Wald Estimator for LATE.

$$\frac{\lim_{r \to c^+} E[Y_i | R_i = r] - \lim_{r \to c^-} E[Y_i | R_i = r]}{\lim_{r \to c^+} E[T_i | R_i = r] - \lim_{r \to c^-} E[T_i | R_i = r]} = E[Y_{1i} - Y_{0i} | T_{1i} > T_{0i}, R_i = c]$$

Notice how this is a local estimate in two senses. First, it is local because it only gives the treatment effect at the threshold c. This is the RD locality. Second, it is local because it only estimates the treatment effect for the compliers. This is the IV locality.

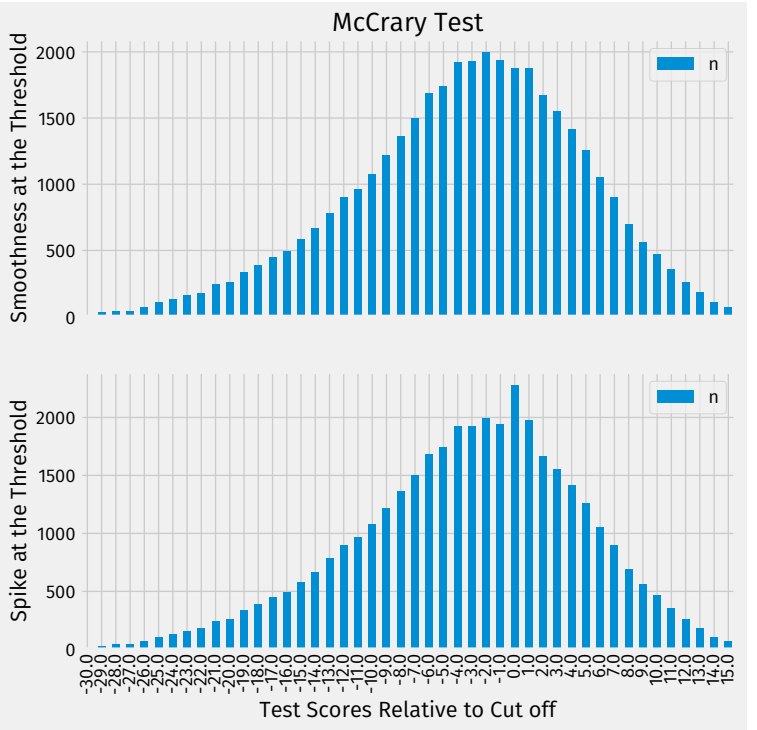
To estimate this, we will use 2 linear regressions. The numerator can be estimated just like we've done before. To get the denominator, we simply replace the outcome with the treatment. But first, let's talk about a sanity check we need to run to make sure we can trust our RDD estimates.

The McCrary Test

One thing that could break our RDD argument is if people can manipulate where they stand at the threshold. In the sheepskin example this could happen if students just below the threshold found a way around the system to increase their test score by just a bit. Another example is when you need to be below a certain income level to get a government benefit. Some families might lower their income on purpose, just to be just eligible for the program.

In these sorts of situations, we tend to see a phenomenon called bunching on the density of the running variable. This means that we will have a lot of entities just above or just below the threshold. To check for that, we can plot the density function of the running variable and see if there are any spikes around the threshold. For our case, the density is given by the n column in our data.

```
plt.figure(figsize=(8,8))

ax = plt.subplot(2,1,1)
sheepskin.plot.bar(x="minscore", y="n", ax=ax)
plt.title("McCrary Test")
plt.ylabel("Smoothness at the Threshold")

ax = plt.subplot(2,1,2, sharex=ax)
sheepskin.replace({1877:1977, 1874:2277}).plot.bar(x="minscore", y="n", ax=ax)
plt.xlabel("Test Scores Relative to Cut off")
plt.ylabel("Spike at the Threshold");
```



The first plot shows how our data density looks like. As we can see, there are no spikes around the threshold, meaning there is no bunching. Students are not manipulating where they fall on the threshold. Just for illustrative purposes, the second plot shows what bunching would look like if students could manipulate where they fall on the threshold. We would see a spike in the density for the cells just above the threshold, since many students would be on that cell, barely passing the exam.

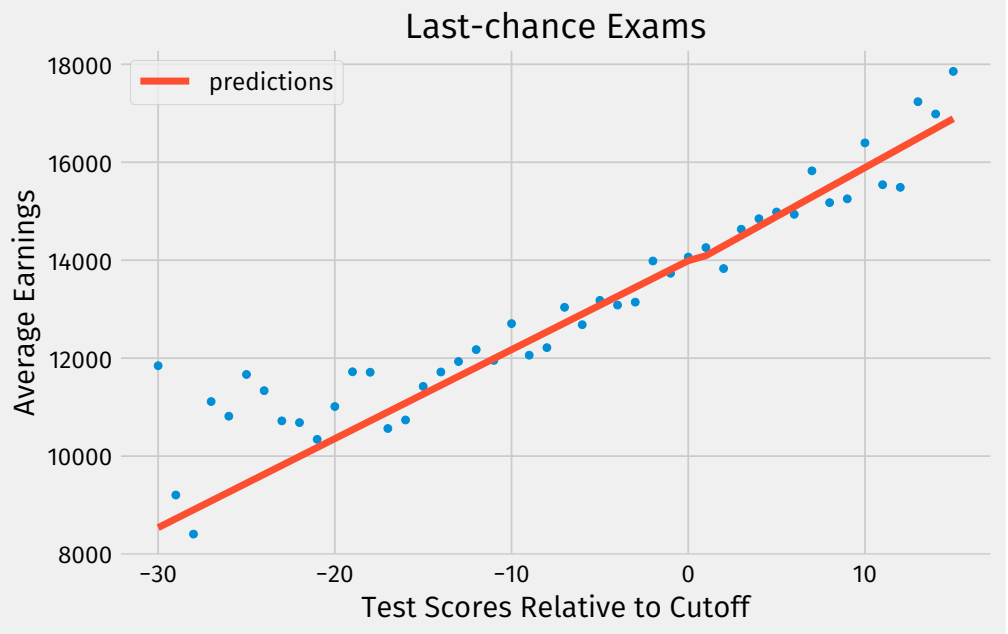
Getting this out of the way, we can go back to estimate the sheepskin effect. As I've said before, the numerator of the Wald estimator can be estimated just like we did in the Sharp RD. Here, we will use as weight the kernel with a bandwidth of 15. Since we also have the cell size, we will multiply the kernel by the sample size to get a final weight for the cell.

```
sheepsking_rdd = sheepskin.assign(threshold=(sheepskin["minscore"]>0).astype(
  ↪ int))
model = smf.wls("avgearnings~minscore*threshold",
                sheepsking_rdd,
                weights=kernel(sheepsking_rdd["minscore"], c=0,
                  ↪ h=15)*sheepsking_rdd["n"]).fit()

model.summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	1.399e+04	83.678	167.181	0.000	1.38e+04	1.42e+04
minscore	181.6636	16.389	11.084	0.000	148.588	214.739
threshold	-97.7571	145.723	-0.671	0.506	-391.839	196.325
minscore:threshold	18.1955	30.311	0.600	0.552	-42.975	79.366

This is telling us that the effect of a diploma is -97.7571, but this is not statistically significant (P-value of 0.5). If we plot these results, we get a very continuous line at the threshold. More educated people indeed make more money, but there isn't a jump at the point where they receive the 12th grade diploma. This is an argument in favor of the view that says that education increases earnings by making people more productive, rather than being just a signal to the marker. In other words, there is no sheepskin effect.

```
ax = sheepskin.plot.scatter(x="minscore", y="avgearnings", color="C0")
sheepskin.assign(predictions=model.fittedvalues).plot(x="minscore",
  ↪ y="predictions", ax=ax, color="C1", figsize=(8,5))
plt.xlabel("Test Scores Relative to Cutoff")
plt.ylabel("Average Earnings")
plt.title("Last-chance Exams");
```



However, as we know from the way non compliance bias works, this result is biased towards zero. To correct for that, we need to scale it by the first stage and get the Wald estimator. Unfortunately, there isn't a good Python implementation for this, so we will have to do it manually and use bootstrap to get the standard errors.

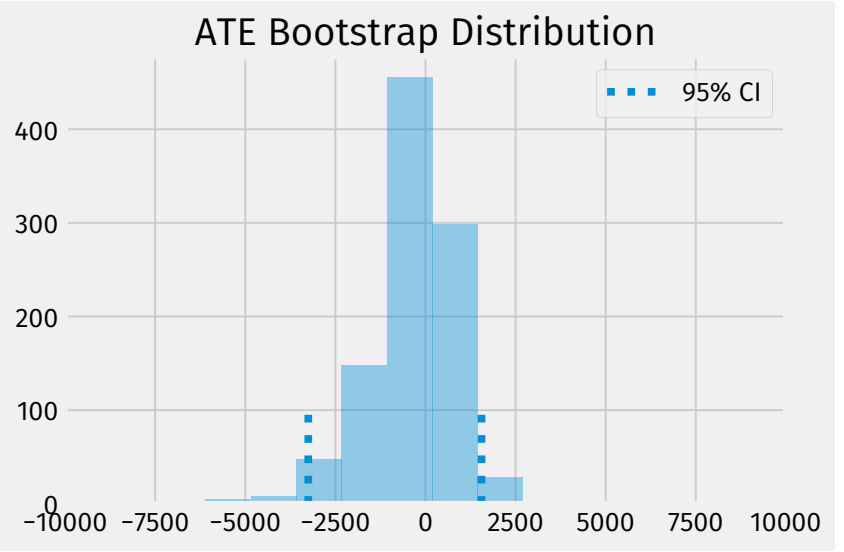
The code below runs the numerator of the Wald estimator just like we did before and also constructs the denominator by replacing the target variable with the treatment variable `receivehsd`. The final step just divides the numerator by the denominator.

```
def wald_rdd(data):
    weights=kernel(data["minscore"], c=0, h=15)*data["n"]
    denominator = smf.wls("receivehsd~minscore*threshold", data,
      ↪ weights=weights).fit()
    numerator = smf.wls("avgearnings~minscore*threshold", data,
      ↪ weights=weights).fit()
    return numerator.params["threshold"]/denominator.params["threshold"]
```

```
from joblib import Parallel, delayed

np.random.seed(45)
bootstrap_sample = 1000
ates = Parallel(n_jobs=4)(delayed(wald_rdd)(sheepsking_rdd.sample(frac=1,
  ↪ replace=True))
                          for _ in range(bootstrap_sample))
ates = np.array(ates)
```

With the bootstrap samples, we can plot the distribution of ATEs and see where the 95% confidence interval is.

```
sns.distplot(ates, kde=False)
plt.vlines(np.percentile(ates, 2.5), 0, 100, linestyles="dotted")
plt.vlines(np.percentile(ates, 97.5), 0, 100, linestyles="dotted",
  ↪ label="95% CI")
plt.title("ATE Bootstrap Distribution")
plt.xlim([-10000, 10000])
plt.legend();
```



As you can see, even when we scale the effect by the first stage, it is still not statistically different from zero. This means that education doesn't increase earnings by a simple sheepskin effect, but rather by increasing one's productivity.

Key Ideas

We learned how to take advantage of artificial discontinuities to estimate causal effects. The idea is that we will have some artificial threshold that makes the probability of treatment jump. One example that we saw was how age makes the probability of drinking jump at 21 years. We could use that to estimate the impact of drinking on mortality rate. We use the fact that very close to the threshold, we have something close to a randomized trial. Entities very close to the threshold could have gone either way and what determines where they've landed is essentially random. With this, we can compare those just above and just below to get the treatment effect. We saw how we could do that with weighted linear regression using a kernel and how this even gave us, for free, standard errors for our ATE.

Then, we look at what would happen in the fuzzy RD design, where we have non compliance. We saw how we could approach the situation much like we did with IV.

References

I like to think of this entire book as a tribute to Joshua Angrist, Alberto Abadie and Christopher Walters for their amazing Econometrics class. Most of the ideas here are taken from their classes at the American Economic Association. Watching them is what is keeping me sane during this tough year of 2020.

- Cross-Section Econometrics · https://www.aeaweb.org/conference/cont-ed/2017-webcasts
- Mastering Mostly Harmless Econometrics · https://www.aeaweb.org/conference/cont-ed/2020-webcasts

I'll also like to reference the amazing books from Angrist. They have shown me that Econometrics, or 'Metrics as they call it, is not only extremely useful but also profoundly fun.

- Mostly Harmless Econometrics · https://www.mostlyharmlesseconometrics.com/
- Mastering 'Metrics · https://www.masteringmetrics.com/

Other important reference is Miguel Hernan and Jamie Robins' book. It has been my trustworthy companion in the most thorny causal questions I had to answer.

- Causal Inference Book · https://miguelhernan.org/whatifbook

PART II

The Yin

CHAPTER
17

Predictive Models 101

We are leaving Part I of this book. That part covered the core about causal inference. Techniques over there are very well known and established. They have survived the test of time. Part I builds the solid foundation we can rely upon. In more technical terms, Part I focuses on defining what is causal inference, what are the biases that prevent correlation from being causation, multiple ways to adjust for those biases (regression, matching and propensity score) and canonical identification strategies (instrumental variables, diff-in-diff and RDD). In summary, Part I focuses on the standard techniques we use to identify the average treatment effect $E[Y_1 - Y_0]$.

As we move to Part II, things will get a bit shaky. We will cover recent developments in the causal inference literature, its relationship with Machine Learning and applications in the industry. In that sense, we trade-off academic rigour for applicability and empiricism. Some methods presented in Part II don't have a solid theory about why they work. Still, when we try them, they seem to work nevertheless. In that sense, Part II might be more useful for industry practitioners that want to use causal inference in their day to day work, rather than scientists who want to research a fundamental causal relationship in the world.

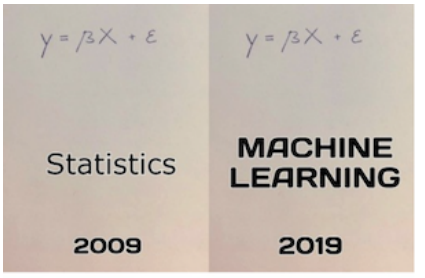
The first few chapters of part two will focus on estimating heterogeneous treatment effects. We will move from a world where all we cared about was the average treatment effect, $E[Y_1 - Y_0]$, to one where we want to know how different units respond differently to the treatment $E[Y_1 - Y_0 | X]$. This is the world where personalisation is paramount. We want to prioritise treating those whose treatment effect will be the most impactful and we don't want to treat those to whom the treatment will be harmful. In a sense, we are also moving from a positive question about what is the average treatment effect to a normative question: who should we treat?

This is the question most businesses ask themselves, albeit in slightly different terms: who should I give discounts to? What interest rate should I charge on a loan? What item should I recommend to this user? What page layout should I show to each customer?

Those are all treatment effect heterogeneity questions that we can answer with the tools presented in Part II. But before we do that, it's only fair that I present what Machine Learning means to the industry, as this will become a fundamental tool we will later use for causal inference.

Machine Learning in the Industry

The focus of this chapter is to talk about how we normally use **machine learning** in the industry. If you are not familiar with machine learning, you can see this chapter as a machine learning crash course. And if you've never worked with ML before, I strongly recommend you learn at least the basics to get the most out of what's to come. But this doesn't mean you should skip this chapter if you are already versed in ML. I still think you will benefit from reading it through. Differently from other machine learning material, this one will **not** discuss the ins and outs of algorithms like decision trees or neural networks. Instead, it will be laser focused on **how machine learning is applied in the real world**.

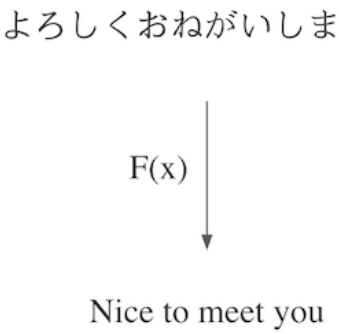


#10yearchallenge

The first thing I want to adress is why are we talking about machine learning in a causal inference book? The short answer is because I think one of the best ways to understand causality is to put it in contrast with the predictive models approach brought by machine learning. The long answer is twofold. First, if you've got to this point in this book, there is a high chance you are already familiar with machine learning. Second, even if you aren't, given the current popularity of these topics, you probably already have some idea on what they are. The only problem is that, with all the hype around machine learning, I might have to bring you back to earth and explain what it really does in very practical terms. Finally, more recent developments in causal inference make heavy use of machine learning algorithms, so there is that too.

Being very direct, **machine learning is a way to make fast, automatic and good predictions**. That's not the entire picture, but it covers 90% of it. It's in the field of supervised machine learning where most of the cool advancements, like computer vision, self-driving cars, language translation and diagnostics, have been made. Notice how, at first, these might not seem like prediction tasks. How is language translation a prediction? And that's the beauty of machine learning. We can solve more problems with prediction than what is initially apparent. In the case of language translation, you can frame it as a

prediction problem where you present a machine with one sentence and it has to predict the same sentence in another language. Notice that I'm **not** using the word prediction in a forecasting or anticipating the future sense. Prediction is simply mapping from one defined input to an initially unknown but equally well defined and observable output.



What machine learning really does is it learns this mapping function, even if it is a very complicated mapping function. The bottom line is that if you can frame a problem as this mapping from an input to an output, then machine learning might be a good candidate to solve it. As for self-driving cars, you can think of it as not one, but multiple complex prediction problems: predicting the correct angle of the wheel from sensors in the front of the car, predicting the pressure in the brakes from cameras around the car, predicting the pressure in the accelerator from GPS data. Solving those (and a ton more) of prediction problems is what makes a self-driving car.

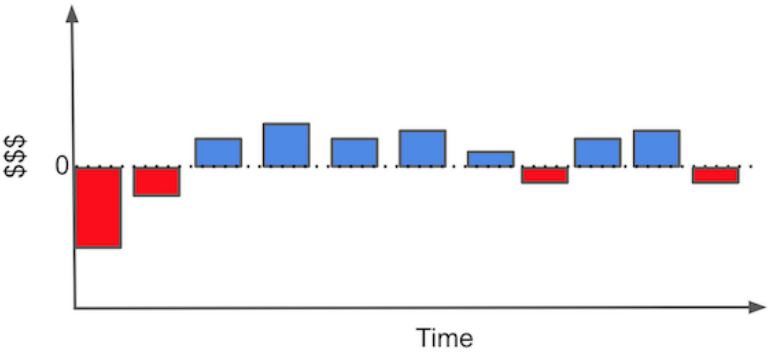
A more technical way of thinking about ML is in term of estimating (possibly very complex) expectation functions:

$$E[Y \mid X]$$

Where Y is what you want to know (translated sentence, diagnostics) and X is what you already know (input sentence, x-ray image). Machine learning is simply a way of estimating that conditional expectation function.

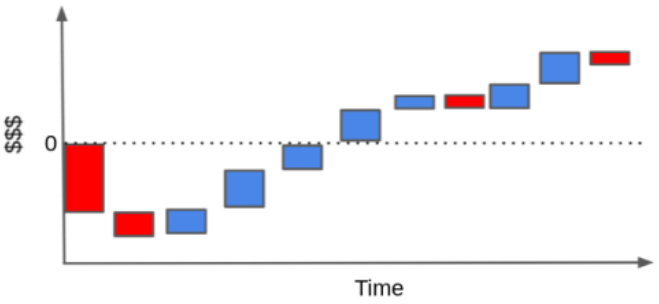
OK… You now understand how prediction can be more powerful than we first thought. Self-driving cars and language translation are cool and all, but they are quite distant, unless you work at a major tech company like Google or Uber. So, to make things more relatable, let's talk in terms of problems almost every company has: customer acquisition (that is getting new customers).

From the customer acquisition perspective, what you often have to do is figure out who the profitable customers are. In this problem, each customer has a cost of acquisition (maybe marketing costs, onboarding costs, shipping costs…) and will hopefully generate a positive cashflow for the company. For example, let's say you are an internet provider or a gas company. Your typical customer might have a cash flow that looks something like this.

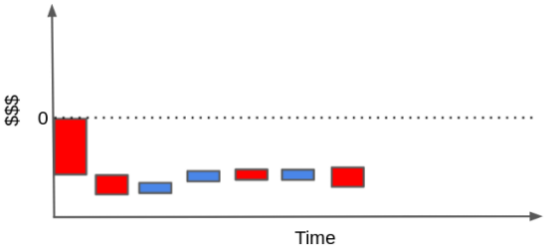


Each bar represents a monetary event in the life of your relationship with the customer. For example, to get a customer, right off the bat, you need to invest in marketing. Then, after someone decides to do business with you, you might incur some sort of onboarding cost (where you have to explain to your customer how to use your product) or installation costs. Only then, the customer starts to generate monthly revenues. At some point, the customer might need some assistance and you will have maintenance costs. Finally, if the customer decides to end the contract, you might have some final costs for that too.

To see if this is a profitable customer, we can rearrange the bar in what is called a cascade plot. Hopefully, the sum of the cash events end up way above the zero line.



In contrast, it could very well be that the customer will generate much more costs than revenues. If he or she uses very little of your product and has high maintenance demands, when we pile up the cash events, they could end up below the zero line.



Of course, this cash flow could be simpler or much more complicated, depending on the type of business. You can do stuff like time discounts with an interest rate and get all crazy about it, but I think the point here is made.

But what can you do about this? Well, if you have many examples of profitable and non profitable customers, you can train a machine learning model to identify them. That way, you can focus your marketing strategies that engage only on the profitable customers. Or, if your contract permits, you can end relations with a customer before he or she generates more costs. Essentially, what you are doing here is **framing the business problem as a prediction problem so that you can solve it with machine learning**: you want to predict or identify profitable and unprofitable customers so that you only engage with the profitable ones.

```
import pandas as pd
import numpy as np
from sklearn import ensemble
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.metrics import r2_score
import seaborn as sns
from matplotlib import pyplot as plt
from matplotlib import style
style.use("ggplot")
```

For instance, suppose you have 30 days of transactional data on 10000 customers. You also have the cost of acquisition `cacq`. This could be the bid you place for them if you are doing online marketing, it could be the cost of shipping or any training you have to do with your customer so they can use your product. Also, for the sake of simplicity (this is a crash course, not a semester on customer valuation), let's pretend you have total control of the customer that you do business with. In other words, you have the power to deny a customer even if he or she wants to do business with you. If that's the case, your task now becomes identifying who will be profitable beforehand, so you can choose to engage only with them.

```
transactions = pd.read_csv("data/customer_transactions.csv")
print(transactions.shape)
transactions.head()
```

```
(10000, 32)
```

	customer_id	cacq	day_0	day_1	day_2	day_3	day_4	day_5	day_6	day_7	...
0	0	-110	6	0	73	10	0	0	0	21	...
1	1	-58	0	0	0	15	0	3	2	0	...
2	2	-7	0	0	0	0	0	0	0	1	...
3	3	-30	0	3	2	0	9	0	0	0	...
4	4	-42	0	0	0	0	0	0	0	0	...

	day_20	day_21	day_22	day_23	day_24	day_25	day_26	day_27	day_28	day_29
0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0	0	0
3	0	0	40	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0

5 rows × 32 columns

What we need to do now is distinguish the good from the bad customers according to this transactional data. For the sake of simplicity, I'll just sum up all transactions and the CACQ. Keep in mind that this throws under the rug a lot of nuances, like distinguishing customers that are churned from those that are in a break between one purchase and the next.

I'll then join this sum, which I call `net_value`, with customer specific features. Since my goal is to figure out which customer will be profitable **before** deciding to engage with them, you can only use data prior to the acquisition period. In our case, these features are age, region and income, which are all available at another `csv` file.

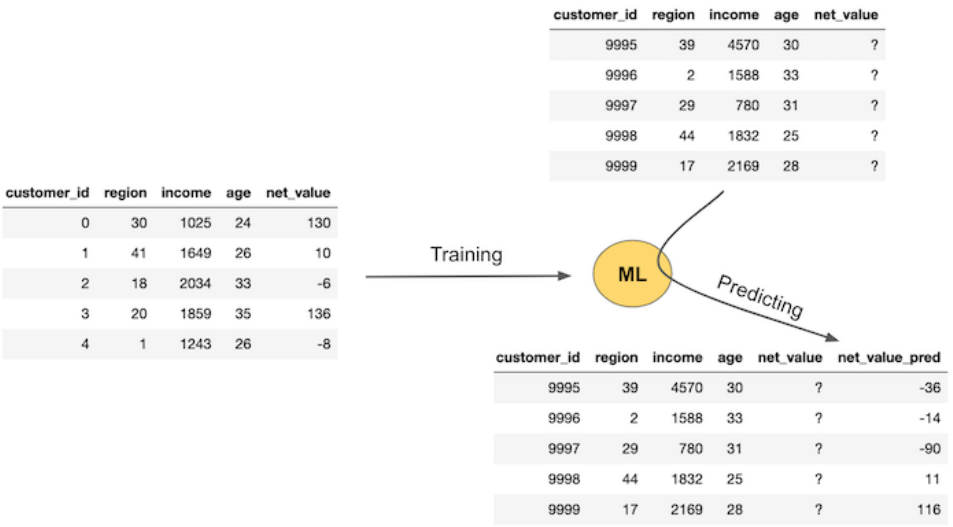
```
profitable = (transactions[["customer_id"]]
              .assign(net_value = transactions
                      .drop(columns="customer_id")
                      .sum(axis=1)))

customer_features = (pd.read_csv("data/customer_features.csv")
                     .merge(profitable, on="customer_id"))

customer_features.head()
```

	customer_id	region	income	age	net_value
0	0	30	1025	24	130
1	1	41	1649	26	10
2	2	18	2034	33	-6
3	3	20	1859	35	136
4	4	1	1243	26	-8

Good! Our task is becoming less abstract. We wish to identify the profitable customers (`net_value > 0`) from the non profitable ones. Let's try different things and see which one works better. But before that, we need to take a quick look into Machine Learning (feel free to skip if you know how ML works)

Machine Learning Crash Course

For our intent and purpose, we can think of ML as an overpowered way of making predictions. For it to work, you need some data with labels or the ground truth of what you are predicting. Then, you can train an ML model on that data and use it to make predictions where the ground truth is not yet known. The image below exemplifies the typical machine learning flow.

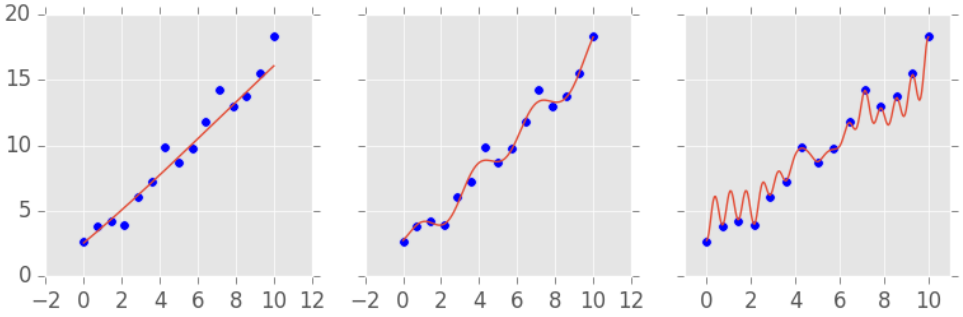


First, you need data where the ground truth, `net_value` here, is known. Then, you train an ML model that will use features - region, income and age in our case - to predict `net_value`. This training or estimating step will produce a machine learning model that can be used to make predictions about `net_value` when you don't yet have the true `net_value`. This is shown in the left part of the image. You have some new data where you have the features (region, income and age) but you don't know the `net_value` yet. So you pass this data through your model and it provides you with `net_value` predictions.

If you are more into technical notation, another way of understanding machine learning is in term of estimating a conditional expectation $E[Y \mid X]$, where Y is called the target variable or outcome and X is called the feature variables. ML is just a powerful way of obtaining $\hat{E}[Y \mid X]$, usually by optimizing some error or loss function.

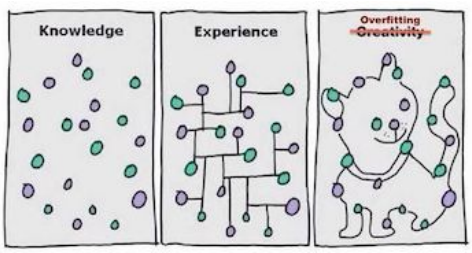
One tricky thing with ML models is that they can approximate almost any function. Another way of saying this is that they can be made so powerful as to perfectly fit the data in the training set. Machine learning models often have what we call complexity

hyperparameters. These things adjust how powerful or complex the model can be. In the image below, you can see examples of a simple model (left), an intermediate model (middle) and a complex and powerful model (right). Notice how the complex model has a perfect fit of the training data.



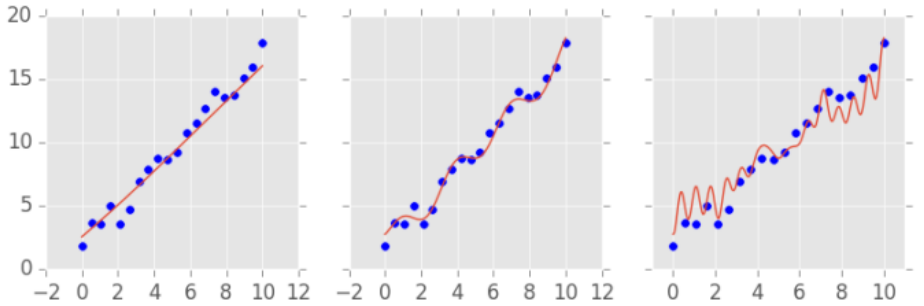
This raises some problems. Namely, how can we know if our model is any good before using it to make predictions in the real world? One way we have is to compare the predictions with the actual values on the dataset where we have the ground truth. These are so-called goodness of fit metrics, like R^2. But remember that the model can be made so powerful as to perfectly fit the data. If this happens, the predictions will perfectly match the ground truth. This is problematic, because it means this validation is misleading, since we can nail it just by making my model more powerful and complex.

Besides, it is generally **not** a good thing to have a very complex model. And you already have some intuition into why that is the case. In the image above, for instance, which model do you prefer? The more complex one that gets all the predictions right? Probably not. You probably prefer the middle one. It's smoother, simpler and yet, it still makes some good predictions, even if it doesn't perfectly fit the data.



Your intuition is in the right place. What happens if you give too much power to your model, is that it will not only learn the patterns in your data, but it also learns the random noise. Since the noise will be different when you use the model to make predictions in the real world (it's random after all), your "perfect" model will make mistakes. In ML terms, we say that models that are too complex are overfitting and don't generalize well. So, what can we do?

We are going to pretend we don't have access to parts of the data. The idea is to split the dataset for which we have the ground truth into two. Then, we can give one part for the model to train on and the other part we can use to validate the model predictions. This is called cross validation.



In the dataset above, which the model didn't see during training, the complex model doesn't do a very good job. The model in the middle, on the other hand, seems to perform better. To choose the right model complexity, we can train different models, each one with a different complexity, and see how they perform on some data that we have the ground truth, but that was not used for training the model. Cross validation is so important we should probably spend more time on it.

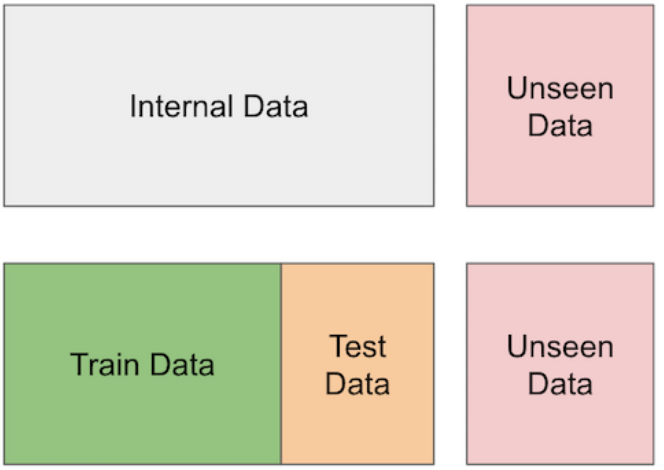
Cross Validation

Cross validation is essential for selecting the model complexity but it's useful beyond that. We can use it whenever we want to try many different things and estimate how they would play out in the real world. The idea being cross validation is to mimic the real world, where we estimate a model on the data that we have, but we make predictions on new, unseen data. The holdout data that we pretend not to have serves as a proxy to what we will encounter in the wild.

Let's see how we can apply cross validation to the whole problem of figuring out which customers are profitable or not. Here is an outline of what we should do:

1. We have data on existing customers. In this data, we know which ones are profitable and which ones are not (we know the ground truth). Let's call our internal data the training set.
2. We will use the internal data to *learn* a rule that tells us which customer is profitable (hence training).
3. We will apply the rule to the holdout data that was **not** used for learning the rule. This should simulate the process of learning a rule in one dataset and applying it to another, a process that will be inevitable when we go to production and score truly unseen data.

Here is a picture of what cross validation looks like. There is the truly unseen data at the rightmost part of the image and then there is data that we only pretend not to have at learning time.



To summarize, we will partition our internal data into a training and a test set. We can use the training set to come up with models or rules that predict if a customer is profitable or not, but we will validate those rules in another partition of the dataset: the test set. This test set will be hidden from our learning procedure.

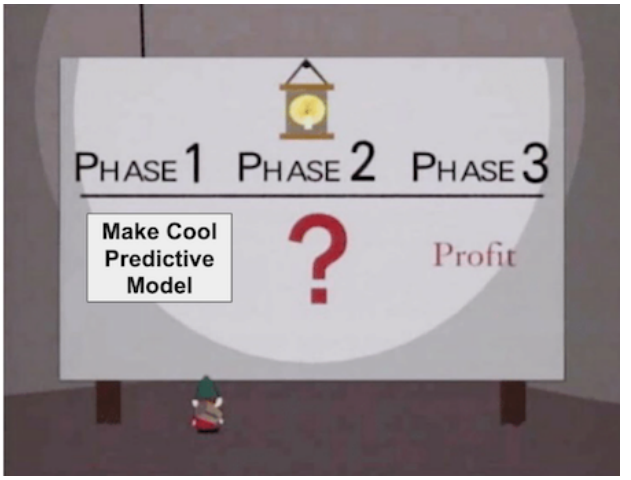
Just as a side note here, there are tons of ways to make cross validation better other than this simple train test split (k-fold cross-validation or temporal cross validation, for instance), but for the sake of what we will do here, this is enough. Remember that the spirit of cross validation is to simulate what would happen once we go to a production environment. By doing that we hope to get more realistic estimates.

For our case, I won't do anything fancy. I'll just divide the dataset into two. 70% will be used to build a method that allows us to identify profitable customers and 30% will be used to evaluate how good that method is.

```
train, test = train_test_split(customer_features, test_size=0.3,
  ↪ random_state=13)
train.shape, test.shape
```

```
((7000, 5), (3000, 5))
```

Predictions and Policies



We've been talking about methods and approaches to identify profitable customers but it is time we get more precise with our concepts. Let's introduce two new ones. A **prediction** is a number that estimates or predicts something. It's the estimation of $\hat{E}[y_i|X_i]$. For example, we can try to predict the profitability of a customer and the prediction would be something like 16 BRL, meaning that we predict this customer will generate 16 BRL in revenue. The point here is that prediction is a simple number.

The second concept is that of a **policy**. A policy is an automatic decision rule. While a prediction is a number, a policy is a decision. For example, we can have a policy that engages with customers with income greater than 1000 and doesn't engage otherwise. We usually build policies on top of predictions: engage with all customers that have profitability predictions above 10 and don't engage otherwise, $\hat{E}[y_i|X_i] > 10$. Machine learning will usually take care of the first concept, that is, of prediction. But notice that predictions alone are useless. We need to attach some decision, or policy to it.

We can do very simple policies and models or very complicated ones. For both policies and predictions, we need to use cross validation, that is, estimate the policy or prediction in one partition of the data and validate its usefulness in another. Since we've already partitioned our data into two, we are good to go.

One Feature Policies

Before we go machine learning crazy on this profitability problem, let's try the simple stuff first. The 80% gain with 20% effort stuff. This often work wonders and, surprisingly, most data scientists forget about it. So, what is the simplest thing we can do? Naturally, **just engage with all the customers!** Instead of figuring out which ones are profitable, let's just do business with everyone and hope the profitable customers more than compensate for the non profitable ones.

To check if this is a good idea, we can see the average net value of the customers. If that turns out to be positive, it means that, on average, we will make money on our customers. Sure, there will be profitable and non profitable ones but, on average, if we have enough customers, we will make money. On the other hand, if this value is negative, it means that we will lose money if we engage with all the customers.

```
train["net_value"].mean()
```

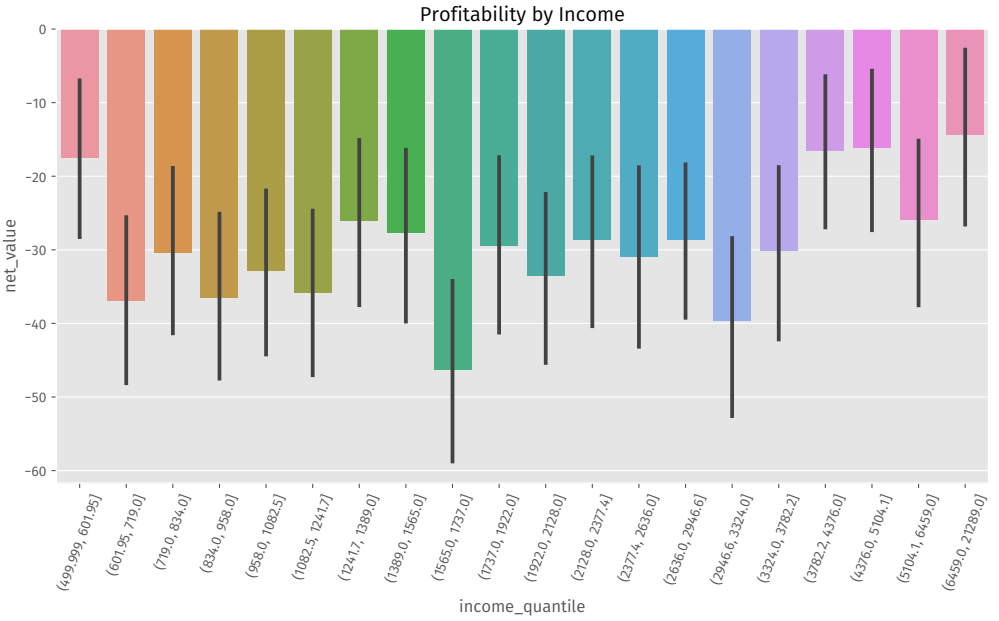
```
-29.169428571428572
```

That's a bummer... If we engage with everyone, we would lose about 30 reais for customers we do business with. Our first, very simple thing didn't work and we better find something more promising if we don't want to go out of business. Just a quick side note here, keep in mind that this is a pedagogical example. Although the very simple, "treat everyone the same" kind of policy didn't work here, they often do in real life. It is usually the case that sending a marketing email to everyone is better than not sending it, or giving discounts coupons to everyone is often better than not giving them.

Moving forward, what is the next simplest thing we can think of? One idea is taking our features and seeing if they alone distinguish the good from the bad customers. Take `income`, for instance. It's intuitive that richer customers should be more profitable, right? What if we do business only with the top richest customers? Would that be a good idea?

To figure this out we can partition our data into income quantiles (a quantile has the property of dividing the data into partitions of equal size, that's why I like them). Then, for each income quantile, let's compute the average net value. The hope here is that, although the average net value in negative, $E[NetValue] < 0$, there might be some subpopulation defined by income where the net value is positive, $E[NetValue|Income = x] > 0$, probably, higher income levels.

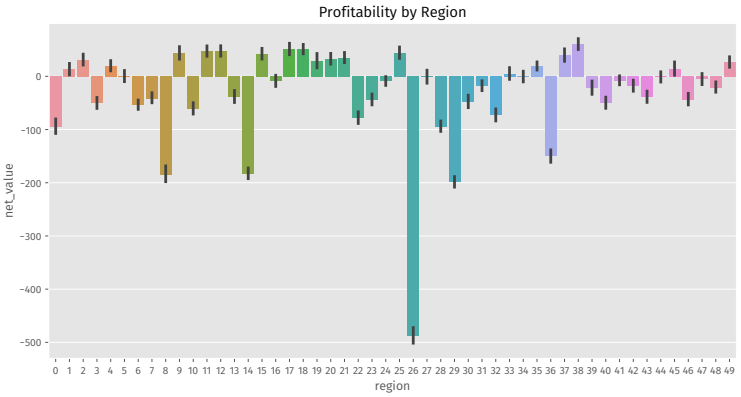
```
plt.figure(figsize=(12,6))
np.random.seed(123) ## seed because the CIs from seaborn uses boostrap

# pd.qcut create quantiles of a column
sns.barplot(data=train.assign(income_quantile=pd.qcut(train["income"], q=20)),
            x="income_quantile", y="net_value")
plt.title("Profitability by Income")
plt.xticks(rotation=70);
```



And, sadly, nope. Yet again, all levels of income have negative average net value. Although it is true that richer customers are "less bad" than non rich customers, they still generate, on average, negative net value. So income didn't help us much here, but what about the other variables, like region? If most of our costs come, say, from having to serve customers in far away places, we should expect that the region distinguishes the profitable from the unprofitable customers.

Since region is already a categorical variable, we don't need to use quantiles here. Let's just see the average net value per region.

```
plt.figure(figsize=(12,6))
np.random.seed(123)
region_plot = sns.barplot(data=train, x="region", y="net_value")
plt.title("Profitability by Region");
```



Bingo! We can clearly see that some regions are profitable, like regions 2, 17, 39, and some are not profitable, like regions 0, 9, 29 and the especially bad region 26. This is looking super promising! We can take this and transform into a policy: only do business with the regions that showed to be profitable *according to the data that we have here*.

One thing to notice is that what we are doing is what ML would do, but in a much simpler way. Namely, we are estimating the expected value of net value in each region: $E[NetValue|Region]$. Now, we need to take this estimation and do something with it.

To construct this policy, we will do something very simple. We will construct a 95% confidence interval around the expected net value of a region. If it is greater than zero, we will do business with that region

The following code builds a dictionary where the key is the region and the value is the lower bound of the 95% CI. Then, the dictionary generator filters only those regions where the expected net value is positive. The result is the regions we will do business with according to our policy.

```
# extract the lower bound of the 95% CI from the plot above
regions_to_net = train.groupby('region')['net_value'].agg(['mean', 'count',
  ↪ 'std'])

regions_to_net = regions_to_net.assign(
    lower_bound=regions_to_net['mean'] - 1.96*regions_to_net['std']/(
      ↪ regions_to_net['count']**0.5)
)

regions_to_net_lower_bound = regions_to_net['lower_bound'].to_dict()
regions_to_net = regions_to_net['mean'].to_dict()

# filters regions where the net value lower bound is > 0.
regions_to_invest = {region: net
                     for region, net in regions_to_net_lower_bound.items()
                     if net > 0}

regions_to_invest
```

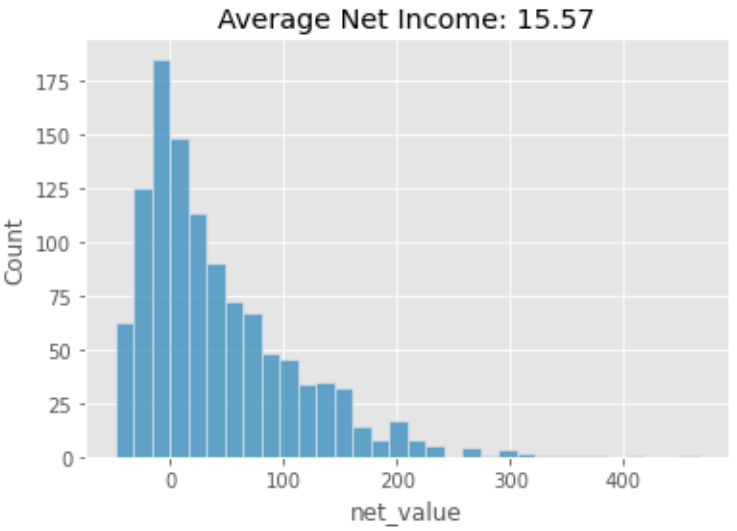
```
{1: 2.9729729729729737,
 2: 20.543302704837856,
 4: 10.051075065003388,
 9: 32.08862469914759,
 11: 37.434210420891255,
 12: 37.44213667009523,
 15: 32.09847683044394,
 17: 39.52753893574483,
 18: 41.86162250217046,
 19: 15.62406327716401,
 20: 22.06654814414531,
 21: 24.621030401718578,
```

```
 25: 33.97022928360584,
 35: 11.68776141117673,
 37: 27.83183541449011,
 38: 49.740709395699994,
 45: 2.286387928016998,
 49: 17.01853709535029}
```

`regions_to_invest` has all the regions we will engage with. Lets now see how this policy would have performed in our test set, the one we pretend not to have. This is a key step in evaluating our policy, because it could very well be that, simply by chance, a region in our training set is appearing to be profitable. If that is only due to randomness, it will be unlikely that we will find that same pattern in the test set.

To do so, we will filter the test set to contain only the customers in the regions defined as profitable (according to our training set). Then, we will plot the distribution of net income for those customers and also show the average net income of our policy.

```
region_policy = (test[test["region"]
                      # filter regions in regions_to_invest
                      .isin(regions_to_invest.keys())])

sns.histplot(data=region_policy, x="net_value")
# average has to be over all customers, not just the one we've filtered with the
    ↪ policy
plt.title("Average Net Income: %.2f" % (region_policy["net_value"].sum() /
  ↪ test.shape[0]));
```



Machine Learning Models as Policy Inputs

If you are willing to do even better, we can now use the power of machine learning. Keep in mind that this might add tones of complexity to the whole thing and usually only marginal gains. But, depending on the circumstances, marginal gains can be translated into huge piles of money and that's why machine learning is so valuable these days.

Here, I'll use a Gradient Boosting model. It's a fairly complicated model to explain, but one that is very simple to use. For our purpose, we don't need to get into the details of how it works. Instead, just remember what we've seen in our ML Crash course: an ML model is a super powerful predictive machine that has some complexity parameters. It's a tool to estimate $E[Y|X]$. The more complex, the more powerful the model becomes. However, if the complexity is too high, the model will overfit, learn noise and not generalize well to unseen data. Hence, we need to use cross validation here to see if the model has the right complexity.

Now, we need to ask, how can good predictions be used to improve upon our simple region policy to identify and engage with profitable customers? I think there are two main improvements that we can make here. First, you will have to agree that going through all the features looking for one that distinguishes good from bad customers is a cumbersome process. Here, we had only 3 of them (age, income and region), so it wasn't that bad, but imagine if we had more than 100. Also, you have to be careful with issues of multiple testing[1] and false positive rates. The second reason is that it is probably the case that you need more than one feature to distinguish between customers. In our example, we believe that features other than region also have some information on customer profitability. Sure, when we looked at income alone it didn't give us much, but what about income in those regions that are just barely unprofitable? Maybe, in those regions, if we focus only on richer customers, we could still get some profit. Technically speaking, we are saying that $E[NetValue|Region, Income, Age]$ is a better predictor of `NetValue` than $E[NetValue|Region]$. This makes a lot of sense. Using more information about income and age on top of region should allow us to predict net value better.

Coming up with these more complicated policies that involve interacting more than one feature can be super complex. The combinations we have to look at grow exponentially with the number of features and it is simply not a practical thing to do. Instead, what we can do is throw all those features into a machine learning model and have it learn those interactions for us. This is precisely what we will do next.

The goal of this model will be to predict `net_value` using `region`, `income`, `age`. To help it, we will take the region feature, which is categorical, and encode it with a numerical value. We will replace each region by the region's average net value on the training set. Remember that we have those stored in the `regions_to_net` dictionary? With this, all we have to do is call the method `.replace()` and pass this dictionary as the argument. I'll create a function for this, because we will do this replacement multiple times. This process of transforming features to facilitate learning is generally called feature engineering.

1 https://en.wikipedia.org/wiki/Multiple_comparisons_problem

```
def encode(df):
    return df.replace({"region": regions_to_net})
```

Next, our model will be imported from Sklearn[2]. All their models have a pretty standard usage. First, you instantiate the model passing in the complexity parameters. For this model, we will set the number of estimators to 400, the max depth to 4 and so on. The deeper the model and the greater the number of estimators, the more powerful the model will be. Of course, we can't let it be too powerful, otherwise it will learn the noise in the training data or overfit to it. Again, you don't need to know the details of what these parameters do. Just keep in mind that this is a very good prediction model. Then, to train our model, we will call the `.fit()` method, passing the features `X` and the variable we want to predict - or target variable - `net_value`.

```
model_params = {'n_estimators': 400,
                'max_depth': 4,
                'min_samples_split': 10,
                'learning_rate': 0.01,
                'loss': 'ls'}

features = ["region", "income", "age"]
target = "net_value"

np.random.seed(123)

reg = ensemble.GradientBoostingRegressor(**model_params)

# fit model on the training set
encoded_train = train[features].pipe(encode)
reg.fit(encoded_train, train[target]);
```

The model is now trained. Next, we need to check if it is any good. To do this, we can look at the predictive performance of this model **on the test set**. There are tons of metrics to evaluate the predictive performance of a machine learning model. Here, I'll use one which is called R^2. We don't need to get into much detail here. It suffices to say that the R^2 is used to evaluate models that predict a continuous variable (like `net_income`). Also, R^2 can go from minus infinity (it will be negative if the prediction is worse than the average) to 1.0. The R^2 tells us how much of the variance in `net_income` is explained by our model.

```
train_pred = (encoded_train
              .assign(predictions=reg.predict(encoded_train[features])))

print("Train R2: ", r2_score(y_true=train[target],
  ↪ y_pred=train_pred["predictions"]))
print("Test R2: ", r2_score(y_true=test[target],
  ↪ y_pred=reg.predict(test[features].pipe(encode))))
```

2 https://scikit-learn.org/stable/

```
Train R2:  0.7108790300152951
Test R2:  0.6938513063048141
```

In this case, the model explains about 71% of the `net_income` variance in the training set but only about 69% of the `net_income` variance in the test set. This is expected. Since the model had access to the training set, the performance there will often be overestimated. Just for fun (and to learn more about overfitting), try setting the 'max_depth' of the model to 14 and see what happens. You will likely see that the train R^2 skyrockets but the test set R^2 gets lower. This is what overfitting looks like.

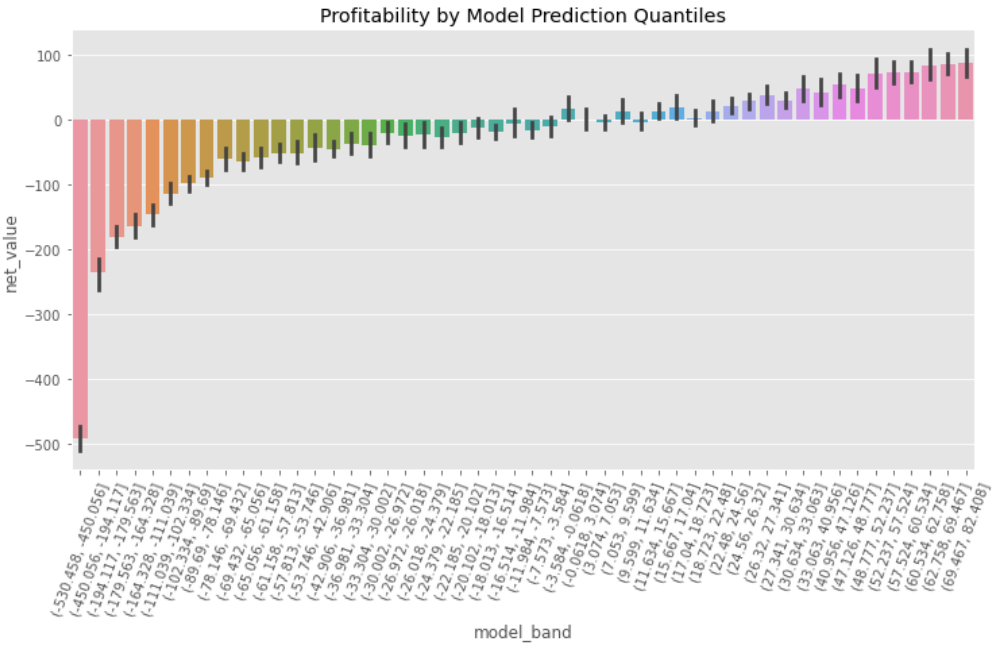
Next, in order to make our policy, we will store the test set predictions in a `prediction` column. This predictions are estimates of $E[NetValue|Age, Income, Region]$.

```
model_policy = test.assign(prediction=reg.predict(test[features].pipe(encode)))

model_policy.head()
```

	customer_id	region	income	age	net_value	prediction
5952	5952	19	1983	23	21	47.734883
1783	1783	31	914	31	-46	-36.026935
4811	4811	33	1349	25	-19	22.553420
145	145	20	1840	26	55	48.306256
7146	7146	19	3032	34	-17	7.039414

Just like we did with the `regions` feature, we can show the average net value by predictions of our model. Since the model is continuous and not categorical, we need to make it discrete first. One way of doing so is using pandas `pd.qcut` (by golly! I love this function!), which partitions the data into quantiles using the model prediction. Let's use 50 quantiles because 50 is the number of regions that we had. And just as a convention, I tend to call these model quantiles **model bands**, because it gives the intuition that this group has model predictions within a band, say, from -10 to 200.

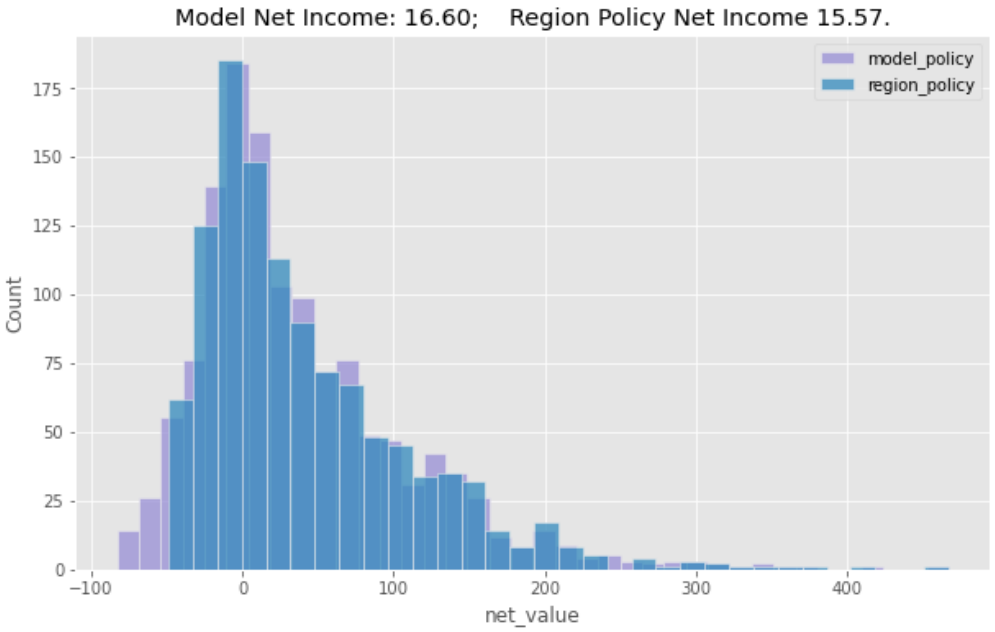
```
plt.figure(figsize=(12,6))

n_bands = 50
bands = [f"band_{b}" for b in range(1,n_bands+1)]

np.random.seed(123)
model_plot = sns.barplot(data=model_policy
                         .assign(model_band =
                           ↪ pd.qcut(model_policy["prediction"], q=n_bands)),
                         x="model_band", y="net_value")
plt.title("Profitability by Model Prediction Quantiles")
plt.xticks(rotation=70);
```



Here, notice how there are model bands where the net value is super negative, while there are also bands where it is very positive. Also, there are bands where we don't know exactly if the net value is negative or positive. Finally, notice how they have an upward trend, from left to right. Since we are predicting net value, it is expected that the prediction will be proportional to what it predicts.

Now, using a model policy that selects customers where $\hat{E}[NetValue|Region, Income, Age] > 0$, and comparing it against the one using only the regions, we can show their histogram of net gains, along with their average net value in the test set.

```
plt.figure(figsize=(10,6))
model_plot_df = (model_policy[model_policy["prediction"]>0])
sns.histplot(data=model_plot_df, x="net_value", color="C2",
  ↪ label="model_policy")

region_plot_df = (model_policy[model_policy["region"].isin(regions_to_invest
  ↪ .keys())])
sns.histplot(data=region_plot_df, x="net_value", label="region_policy")

plt.title("Model Net Income: %.2f;    Region Policy Net Income %.2f." %
          (model_plot_df["net_value"].sum() / test.shape[0],
           region_plot_df["net_value"].sum() / test.shape[0]))
plt.legend();
```



As we can see, the model generates a better policy than just using the `region` feature, but not by much. While the model policy would have made us about 16.6 reais / customer on the test set, the region policy would have made us only 15.5 / customer. It's just slightly better, but if you have tons and tons of customers, this might already justify using a model instead of a simple one feature policy.

Fine Grain Policy

As a recap, so far, we tested the most simple of all policies, which is just engaging with all the customers. This policy can be seen as estimating the marginal net value, $\hat{E}[NetValue] > 0$. Since that didn't work (the average net income per customer was negative), we developed a single feature policy that was based on regions: we would do business in some regions, but not in others, or $\hat{E}[NetValue|Region] > 0$. This already gave us very good results. Next, we went full machine learning, with a predictive model that used all the features: $\hat{E}[NetValue|Region, Income, Age] > 0$. Then, we used that model as an input to a policy and chose to do business with all the customers whose net income predictions were above zero.

Here, the decision which the policy handles is very simple: engage with a customer or don't engage. The policies we had so far dealt with the binary case. They were in the form of

```
if prediction > 0 then do business else don't do business.
```

This is something we call **thresholding**. If the prediction exceeds a certain threshold (zero in our case, but could be something else), we take one decision, if it doesn't, we take

another. One other example of where this could be applied in real life is transactional fraud detection: if the prediction score of a model that detects fraud is above some threshold X, we deny the transaction, otherwise we approve it.

Thresholding works in lots of real case scenarios and it is particularly useful when the nature of the decision is binary. However, we can think of cases where things tend to be more nuanced. For example, you might be willing to spend more on marketing to get the attention of very profitable customers. Or you might want to add them to some prime customers list, where you give special treatment to them, but it also costs you more to do so. Notice that if we include these possibilities, your decision goes from binary (engage vs don't engage) to continuous: how much should you invest in a customer.

Here, for the next example, suppose your decision is not just who to do business with, but how much marketing costs you should invest in each customer. And for the sake of the example, assume that you are competing with other firms and whoever spends more on marketing in a particular customer wins that customer (much like a bidding mechanism). In that case, it makes sense to invest more in highly profitable customers, less in marginally profitable customers and not at all in non profitable customers.

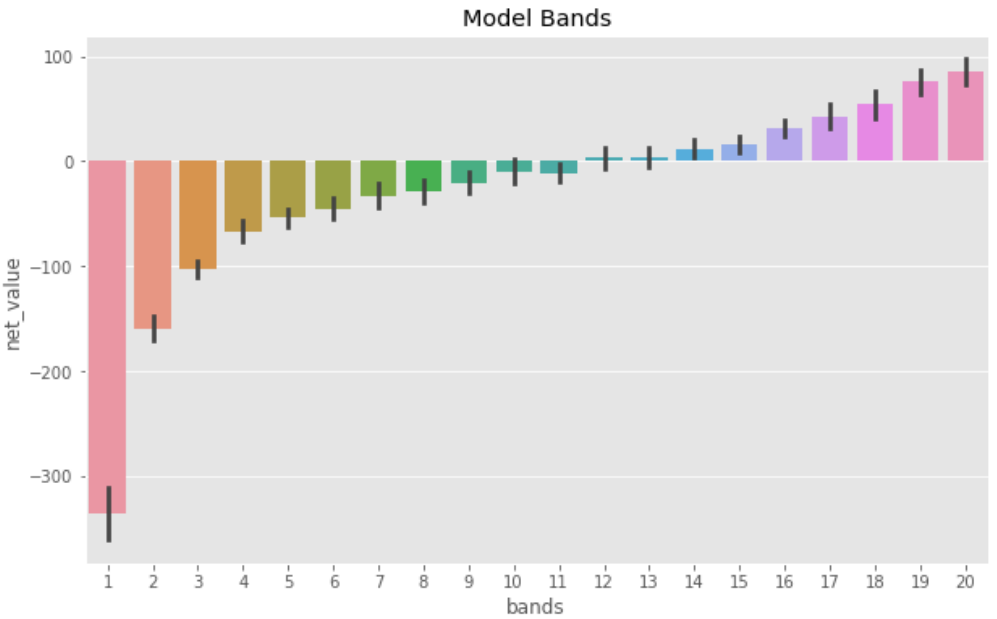
One way to do that is to discretize your predictions into bands. We've done this previously for the purpose of model comparison, but here we'll do it for decision making. Let's create 20 bands. We can think of those as quantiles or equal size groups. The first band will contain the 5% less profitable customers *according to our predictions*, the second band will contain from the 5% to the 10% less profitable and so on. The last band, 20, will contain the most profitable customers.

Notice that the binning too has to be estimated on the training set and applied on the test set! For this reason, we will compute the bins using `pd.qcut` on the training set. To actually do the binning, we will use `np.digitize`, passing the bins that were precomputed on the training set.

```
def model_binner(prediction_column, bins):
    # find the bins according to the training set
    bands = pd.qcut(prediction_column, q=bins, retbins=True)[1]

    def binner_function(prediction_column):
        return np.digitize(prediction_column, bands)

    return binner_function


# train the binning function
binner_fn = model_binner(train_pred["predictions"], 20)

# apply the binning
model_band = model_policy.assign(bands = binner_fn(model_policy["prediction"]))
model_band.head()
```

	customer_id	region	income	age	net_value	prediction	bands
5952	5952	19	1983	23	21	47.734883	18
1783	1783	31	914	31	-46	-36.026935	7
4811	4811	33	1349	25	-19	22.553420	15
145	145	20	1840	26	55	48.306256	18
7146	7146	19	3032	34	-17	7.039414	13

```
plt.figure(figsize=(10,6))
sns.barplot(data=model_band, x="bands", y="net_value")
plt.title("Model Bands");
```



With these bands, we can allocate the bulk of our marketing investments to band 20 and 19. Notice how we went from a binary decision (engage vs not engage), to a continuous one: how much to invest on marketing for each customer. Of course you can fine tune this even more, adding more bands. In the limit, you are not binning at all. Instead, you are using the raw prediction of the model and you can create decision rules like

```
mkt_investments_i = model_prediction_i * 0.3
```

where for each customer *i*, you invest 30% of the net_value predicted by the model (30% was an arbitrary number, but you get the point).

Key Ideas

We've covered A LOT of ground here in a very short time, so I think this recap is extremely relevant for us to see what we accomplished here. First, we learned how the majority of machine learning applications involve nothing more than making good predictions, where prediction is understood as mapping from a known input to an initially unknown, but well defined output. We can also understand prediction as estimating an expectation function $E[Y|X]$. But when I say "nothing more", I'm not being entirely fair. We also saw how good predictions can solve more problems than we might realize at first sight, like language translation and self-driving cars.

Then, we got back down to earth and looked at how good predictions can help us with more common tasks, like figuring out which customer we should bring in and which to avoid. Specifically, we looked at how we could predict customer profit. With that prediction, we built a policy that decides who we should do business with. Notice that this is just an example of where prediction models can be applied. There are sure to be tonnes of other applications, like credit card underwriting, fraud detection, cancer diagnostics and anything else where good predictions might be useful.

The key takeaway here is that **if you can frame your business problem as a prediction problem, then machine learning is probably the right tool for the job**. I really can't emphasize this enough. With all the hype around machine learning, I feel that people forget about this very important point and often end up making models that are very good at predicting something totally useless. Instead of thinking about how to frame a business problem as a prediction problem and *then* solving it with machine learning, they often build a prediction model and try to see what business problem could benefit from that prediction. This might work, but, more often than not, is a shot in the dark that only generates solutions in search of a problem.

References

The things I've written here are mostly stuff from my head. I've learned them through experience. This means there isn't a direct reference I can point you to. It also means that the things I wrote here have **not** passed the academic scrutiny that good science often goes through. Instead, notice how I'm talking about things that work in practice, but I don't spend too much time explaining why that is the case. It's a sort of science from the streets, if you will. However, I am putting this up for public scrutiny, so, by all means, if you find something preposterous, open an issue and I'll address it to the best of my efforts.

Finally, I believe I might have been too quick for those who were hoping for a comprehensive and detailed introduction of machine learning. To be honest, I believe that where I can truly generate value is teaching about causal inference, not machine learning. For the latter, there are tons of amazing online resources, much better than I could ever dream of creating. The classical one is Andrew Ng's course on Machine Learning · https://www.coursera.org/learn/machine-learning and I definitely recommend you take a look into it if you are new to machine learning.

CHAPTER
18

Heterogeneous Treatment Effects and Personalization

From Predictions to Causal Inference

In the last chapter, we briefly covered Machine Learning models. ML models are tools for what I called predictions or, more technically, estimating the conditional expectation function $E[Y|X]$. In other words, ML is incredibly useful when you want to map from a known input X (like an english sentence, sales this month, brain scan images) to an initially unknown but well-defined output Y (like japanese sentences, sales next month or cancer diagnostics). So, if ML deals with predictions, or estimating $E[Y|X]$, for it to be useful, you have to frame whatever problem you want to solve with ML, as a prediction problem, a problem where estimating $E[Y|X]$ is the key. We walked through such an example in the last chapter. There, we had to predict customer profitability from customer specific features: $E[NetValue|Age, Income, Region]$. This information was very useful, because it allowed us to focus our effort on engaging with profitable customers, while not doing business with non-profitable customers. Here, predicting profitability well is key.

Notice that it is a passive approach to estimation in the sense that you remove yourself from the data generating process. In our example, we assumed that customer profitability, `NetValue`, was given. All we had to do was estimate it. In other words, we assumed there was nothing we could do about the customer's profitability other than predict it. We could not increase it, nor decrease it. But that is not always true. In fact, a lot of times, companies have levers they can use to increase customer profitability. These levers can range from a prime or cheaper customer service, to discount, prices or marketing. In the industry, it's often the case that we are inserted in the data generating process. We can affect it. Hence, as data scientists working in the industry, we often have to answer what is the best course of action or what intervention to make in order to optimise some business metric, usually profitability or some other intermediary metric like conversion, costs or sales.

In this world, where we are not passive observers, estimating $E[Y|X]$ is not the full picture. Here is where we enter causal inference. We need to add another piece to our conditional expectation function. That piece is precisely what models our participation in the data generating process. This piece is the treatment:

$$E[Y|X,T]$$

We now have to make the distinction between context or exogenous features X and treatments T. Both impact the outcome Y, but while we have no control over X, we can decide what value T will take or at least intervene on it. To give a concrete example, Y could be sales in a day, X could be context features you can't control, but that gives you information about sales, like average sales in the previous days, and T is the treatment variable you can intervene on in order to increase sales, like price, item stock levels or marketing. Causal inference is then the process of estimating the causal relationship between T and Y under context X. Once we've done that, optimizing Y is just a matter of setting the treatment T in an optimal way

$$\underset{T}{argmax}\ E[Y|X,T]$$

In that sense, beyond the positive aspect of causal inference, we also have a normative motivation.

In Part I, we tried to answer questions like, what is the value of schooling? Can law changes decrease smoking levels? Can we increase academic achievement by having a positive mindset? What is the impact of alcohol on mortality rates? All those questions are interesting from the purely scientific view of understanding how the world works. But there is also practical motivation behind them. If we know the impact of schooling on earnings, we can understand what is a reasonable price to pay for it. In math terms, what we are doing is estimating the causal inference of schooling and optimizing it: $\underset{Educ}{argmax}\ E[Income|X,Educ]$.

Part I was focused on answering if a treatment was overall positive, strong or zero. For example, we wanted to know if investing in education was, in general, a good idea or not. Also in Part I, the role of X was twofold. First, X could contain confounders, in which case, the causal effect was only identifiable if we accounted for or adjusted for X. Or, X could act to reduce the variance of the causal estimation. If X is a good predictor of Y, we can use it to explain away variance in Y, making the causal effect more apparent.

Now, things will become less black and white. We want more than just the average treatment effect. We will allow the treatment to impact positively some people but not others. Context features X will play a role in defining different profiles of units, and each profile might respond differently to the treatment. We now want to personalize the treatment, giving it only to those units that best respond to it. We are going from a world where all we cared about was the average treatment effect to one where we want the heterogeneous treatment effect.

From ATE to CATE

So far, every time we've estimated the causal impact of a treatment, it was the average treatment effect (or, sometimes, the local average treatment effect):

$$E[Y_1 - Y_0]$$

or the continuous treatment equivalent

$$E[y'(t)]$$

where $y'(t)$ is the treatment derivative of the response function or outcome. We've learned techniques to uncover the general effectiveness of a treatment. ATE estimation is the bedrock of causal inference. It's a super useful tool for the decision making problem we refer to as program evaluation. We want to know if we should roll out a treatment to the entire population or not. Don't be confused by the public policy terms. The same technique to estimate the effectiveness of a national education or health program can also be used to know the effect of launching a new product on a company's bottom line. The key thing to note here is that the decision we want to inform is if we should treat or not.

Now, we will try to inform another type of decision: **who** do we treat? Now, we allow the decision to change from one unit to another. It might be beneficial to treat one unit but not another. We want to personalize the treatment. In more technical terms, we want to estimate the Conditional Average Treatment Effect (CATE)

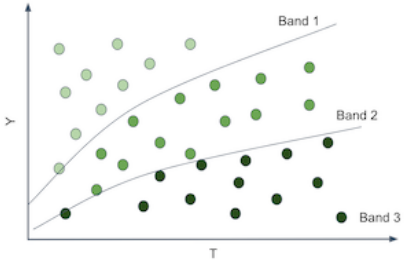
$$E[Y_1 - Y_0 \mid X] \text{ or } E[y'(t) \mid X]$$

The conditioning on X means that we now allow the treatment effect to be different depending on the characteristics of each unit. Again, here, we believe that not all entities respond equally well to the treatment. We want to leverage that heterogeneity. We want to treat only the right units (in the binary case) or figure out what is the optimal treatment dosage for each unit (in the continuous case).

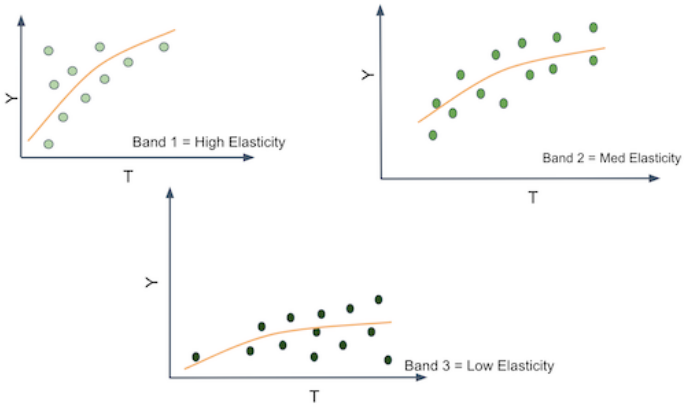
For instance, if you are a bank that has to decide the loan each customer is eligible for, you can be damn sure that it's not a good idea to give loads of money to everyone - although it might be reasonable for some. You will have to be smart with your treatment (loan amount). Perhaps, depending on the customer's credit score (X), you can figure out what is the proper loan dosage. Of course, you don't need to be a big institution to leverage personalization. There's no shortage of examples where it applies. What days of the year should you do sales? How much should you charge for whatever product? How much exercise is too much exercise for each person?

Think of it this way. You have a bunch of customers and a treatment (price, discount, loan,...). You want to personalize the treatment, for example, give different discounts to different customers.

To do that, you have to segment your customers. You have to create groups that respond differently to your treatment. For example, you want to find customers that respond well to discounts and customers who respond poorly to it. Well, the customer's response to a treatment is given by the conditional treatment effect $\frac{\delta Y}{\delta T}$. So, if we could somehow estimate that for each customer, we could group together those that respond great to the treatment (high treatment effect) and those that don't respond very well to it. If we did that, we would split the customers space somewhat like the following image.

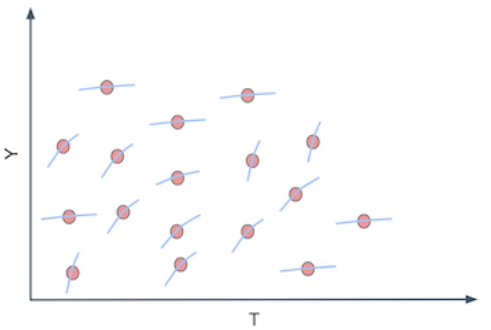


That would be wonderful because now we would be able to estimate different treatment effects or sensitivities on each partition. And notice that the sensitivity is just the slope of the line or function that goes from T to Y. So, if we can produce partitions where the slope or sensitivity differs, it means that entities on those partitions have different responsiveness to the treatment.



In other words, what you want is to move away from predicting Y in its raw form and start to predict the derivative of Y on T, $\frac{\delta Y}{\delta T}$ for each unit. For example, suppose that Y is ice cream sales, T is ice cream price and each unit i is a day. Let's set moral issues aside, for the sake of argument, and pretend that you can change the price of ice cream every day. If you can somehow find the days where $\frac{\delta Sales}{\delta Price}$ **is low**, you can **increase prices** without losing many sales on those days. Perhaps you do this already, say, when you increase them during the holiday season. The point being, it's useful to differentiate days in terms of the price sensitivity because it gives you some basis on how to set prices in an optimal way.

Ok, you might say, but this is kind of tricky. How can I predict sensitivity $\frac{\delta Sales}{\delta Price}$ if I can't see it? That's a very good point. sensitivity is essentially non observable on a unit level. Not only that, it's a strange concept. We are much more accustomed to thinking in terms of raw quantities rather than in terms of change rates of those same quantities. So, to conceptualize sensitivity better, here is a little trick. You can think about each entity as having a Y_i value, sales in our example, but also an individual sensitivity $\frac{\delta Y_i}{\delta T_i}$. The sensitivity is how much Y changes with T, so you can think about each entity also having a slope coefficient associated with it $\frac{\delta Y}{\delta T_i}$. In our example, we would say each day has a slope coefficient of price on sales.



Of course, we can't see those individual slope coefficients. For us to see the individual slopes, we would have to observe each day under two different prices and calculate how the sales change for each of those prices.

$$\frac{\delta Y_i}{\delta T_i} \approx \frac{Y(T_i) - Y(T_i + \epsilon)}{T_i - (T_i + \epsilon)}$$

This is the fundamental problem of causal inference all over again. We can't ever see the same unit under different treatment conditions. So, what can we do?

Predicting Sensitivity

We got ourselves into a complicated situation here. We've agreed that we need to predict $\frac{\delta Y_i}{\delta T_i}$, which is sadly not observable. So it's not like we could use an ML algorithm and plug that as it's target. But maybe we don't need to observe $\frac{\delta Y_i}{\delta T_i}$ in order to predict it

Here is an idea. What if we use linear regression?

Let's say you fit the following linear model to your data.

$$y_i = \beta_0 + \beta_1 t_i + \beta_2 X_i + e_i$$

If you differentiate it on the treatment, you will end up with

$$\frac{\delta y_i}{\delta t_i} = \beta_1$$

And since you can estimate the model above to get $\hat{\beta_1}$, we might even be as bold as to say that **you can predict sensitivity even though you can't observe it**. In the case above, it is a rather simple prediction, that is, we are predicting the constant value $\hat{\beta_1}$ for everyone. That's something, but not yet what we want. That's the ATE, not the CATE. This doesn't help us in our task of grouping entities according to how responsive they are to the treatment, simply because everyone gets the same sensitivity prediction. However, we can do the following simple change

$$y_i = \beta_0 + \beta_1 t_i + \beta_2 X_i + \beta_3 t_i X_i + e_i$$

Which would in turn give us the following sensitivity prediction

$$\widehat{\frac{\delta y_i}{\delta t_i}} = \hat{\beta_1} + \hat{\beta_3} X_i$$

Where β_3 is a vector coefficient for the features in X.

Now each entity defined by a different X_i will have a different sensitivity prediction. In other words, the sensitivity prediction will change as X changes. Alas, regression can give us a way of estimating the CATE $E[y'(t)|X]$.

We are finally getting somewhere. The model above allows us to make a sensitivity prediction for each of our entities. With those predictions we can make more useful

groups. We can take the units with high predicted sensitivity and group them together. We can do the same with the ones that have low predicted sensitivity. Finally, with our sensitivity predictions, we can group entities by how much we think they will respond to the treatment.

Enough of theory for now. It's time to walk through an example of how to make this sort of sensitivity model. Let's consider our ice cream example. Each unit i is a day. For each day, we know if it's a weekday or not, what was the cost we had to make the ice cream (you can think of the cost as a proxy for quality) and the average temperature for that day. Those will make up our feature space X. Then, we have our treatment, price, and our outcome, the number of ice creams sold. For this example, we will consider that the treatment is randomized, just so that we don't have to worry about bias for now.

```
import pandas as pd
import numpy as np
from matplotlib import pyplot as plt

import seaborn as sns
import statsmodels.formula.api as smf
import statsmodels.api as sm

from sklearn.ensemble import GradientBoostingRegressor
from sklearn.model_selection import train_test_split
```

```
prices_rnd = pd.read_csv("./data/ice_cream_sales_rnd.csv")
print(prices_rnd.shape)
prices_rnd.head()
```

```
(5000, 5)
```

	temp	weekday	cost	price	sales
0	25.8	1	0.3	7	230
1	22.7	3	0.5	4	190
2	33.7	7	1.0	5	237
3	23.0	4	0.5	5	193
4	24.4	1	1.0	3	252

Remember our goal here: we need to decide when to charge more and when to charge less depending on the day specific features: `temp` `weekday` and `cost`. If that's the goal, the treatment effect heterogeneity model needs to be evaluated with respect to its usefulness in achieving this goal. We will get to that in a moment (and a lot more in the next chapter). For now, let's just split the dataset into a training and testing set.

```
np.random.seed(123)
train, test = train_test_split(prices_rnd)
```

Now that we have our training data, we need to make a model which will allow us to distinguish days with high price sensitivity from days with low price sensitivity. Our approach to do that will be to simply predict price sensitivity. How exactly? First, let's consider using the following linear model

$$sales_i = \beta_0 + \beta_1 price_i + \boldsymbol{\beta}_2 X_i + e_i$$

If we inspect the parameters of this model, we can see what our predicted sensitivity will look like.

```
m1 = smf.ols("sales ~ price + temp+C(weekday)+cost", data=train).fit()
m1.summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	186.7113	1.770	105.499	0.000	183.241	190.181
C(weekday)[T.2]	-25.0512	0.924	-27.114	0.000	-26.863	-23.240
C(weekday)[T.3]	-24.5834	0.901	-27.282	0.000	-26.350	-22.817
C(weekday)[T.4]	-24.3807	0.897	-27.195	0.000	-26.138	-22.623
C(weekday)[T.5]	-24.9036	0.894	-27.850	0.000	-26.657	-23.150
C(weekday)[T.6]	-24.0921	0.903	-26.693	0.000	-25.862	-22.323
C(weekday)[T.7]	-0.8635	0.888	-0.972	0.331	-2.605	0.878
price	-2.7515	0.106	-25.970	0.000	-2.959	-2.544
temp	1.9848	0.060	33.117	0.000	1.867	2.102
cost	4.4718	0.528	8.462	0.000	3.436	5.508

For $m1$, the predicted price sensitivity $\widehat{\frac{\delta y_i}{\delta t_i}}$ will be given by $\hat{\beta}_1$, which is -2.75, in our case. This means that for each additional BRL we charge for our ice cream, we should expect sales to go down by about 3 units.

Notice how this $m1$ predicts the exact same sensitivity for everyone. Hence, it is not a very good model if we want to know on which days people are less sensitive to ice cream prices. It estimates the ATE when what we need here is the CATE. Remember that our goal is to partition the entities in such a way that we can personalize and optimize our treatment (price) for each individual partition. If every prediction is the same, there is no partitioning we can make. We are not distinguishing sensitive from non sensitive units. To correct for that, consider our second model:

$$sales_i = \beta_0 + \beta_1 price_i + \beta_2 price_i * temp_i * + \boldsymbol{\beta}_3 X_i + e_i$$

This second model includes an **interaction term** between price and temperature. This means that it allows the sensitivity to differ for different temperatures. What we are effectively saying here is that people are more or less sensitive to price increases depending on the temperature.

```
m2 = smf.ols("sales ~ price*temp + C(weekday) + cost", data=train).fit()
m2.summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	192.4767	4.371	44.037	0.000	183.907	201.046
C(weekday)[T.2]	-25.0805	0.924	-27.143	0.000	-26.892	-23.269
C(weekday)[T.3]	-24.5871	0.901	-27.290	0.000	-26.354	-22.821
C(weekday)[T.4]	-24.4225	0.897	-27.231	0.000	-26.181	-22.664
C(weekday)[T.5]	-24.8953	0.894	-27.844	0.000	-26.648	-23.142
C(weekday)[T.6]	-24.1269	0.903	-26.726	0.000	-25.897	-22.357
C(weekday)[T.7]	-0.8581	0.888	-0.966	0.334	-2.599	0.883
price	-3.6299	0.618	-5.873	0.000	-4.842	-2.418
temp	1.7459	0.176	9.912	0.000	1.401	2.091
price:temp	0.0366	0.025	1.443	0.149	-0.013	0.086
cost	4.4558	0.529	8.431	0.000	3.420	5.492

Once we estimate the model, the predicted sensitivity is given by

$$\frac{\widehat{\delta sales_i}}{\delta price_i} = \hat{\beta}_1 + \hat{\beta}_2 temp_i$$

Notice that $\hat{\beta}_2$ is positive 0,03 and the baseline sensitivity β_1 (the sensitivity at $0C^o$) is -3.6. This means that, on average, as we increase price, sales go down, which makes sense. It also means that for each additional degree in temperature, people become less sensitive to price increases on ice cream (although not by much). For example, at $25C^o$, for each additional BRL we charge, our sales go down by 2.8 units ($-3.6+(0.03*25)$). But at $35C^o$, for each additional BRL we charge, they go down only by 2.5 units ($-3.6 + (0.03 * 35)$). This is also sort of intuitive. As the days get hotter and hotter, people are willing to pay more for ice cream.

We can go even further. The next model includes interaction terms on all the feature space. This means that sensitivity will change with temperature, day of the week and cost.

$$sales_i = \beta_0 + \beta_1 price_i + \boldsymbol{\beta}_2 \boldsymbol{X_i} * price_i + \boldsymbol{\beta}_3 X_i + e_i$$

```
m3 = smf.ols("sales ~ price*cost + price*C(weekday) + price*temp",
  ↪ data=train).fit()
```

According to the above model, the unit level sensitivity, or CATE, would be given by

$$\frac{\delta Sales}{\delta Price} = \beta_1 + \boldsymbol{\beta}_2 \boldsymbol{X}_i$$

Where β_1 is the price coefficient and $\boldsymbol{\beta}_2$ is the vector for the interaction coefficients.

Finally, let's see how to actually make those sensitivity predictions. One way is to extract the sensitivity parameters from the model and use the formula above. However, we

will resort to a more general approximation. Since sensitivity is nothing more than the derivative of the outcome on treatment, we can use the definition of the derivative.

$$\frac{\delta y}{\delta t} = \frac{y(t+\epsilon) - y(t)}{(t+\epsilon) - t}$$

with ϵ going to zero. We can approximate this definition by replacing ϵ by 1.

$$\frac{\delta y}{\delta t} \approx \hat{y}(t+1) - \hat{y}(t)$$

where $\hat{y}$ is given by our model's predictions. In words, I'll make two predictions with my models: one passing the original data and another passing the original data but with the treatment incremented by one unit. The difference between those predictions is my CATE prediction.

Below, you can see a function for doing that. Since we've used the train set to estimate our model, we will now make predictions on the test set. First, let's use our first ATE model, $m1$.

```
def pred_sensitivity(m, df, t="price"):
    return df.assign(**{
        "pred_sens": m.predict(df.assign(**{t:df[t]+1})) - m.predict(df)
    })

pred_sensitivity(m1, test).head()
```

	temp	weekday	cost	price	sales	pred_sens
2648	18.6	7	0.5	10	185	-2.751463
2456	26.0	3	0.5	10	200	-2.751463
4557	23.7	3	0.3	8	192	-2.751463
4884	28.9	4	1.5	6	213	-2.751463
92	23.7	1	0.5	8	207	-2.751463

Making sensitivity predictions using $m1$ is not much fun. We can see that it predicts the exact same value for all the days. That's because there are no interaction terms on that model. However, if we make predictions using $m3$, it outputs a different sensitivity prediction for each day. That's because now, the sensitivity or treatment effect depends on the day specific features.

```
pred_sens3 = pred_sensitivity(m3, test)

np.random.seed(1)
pred_sens3.sample(5)
```

	temp	weekday	cost	price	sales	pred_sens
4764	31.1	6	1.0	3	212	1.144309
4324	24.8	7	0.5	10	182	-9.994303
4536	25.0	2	1.5	6	205	0.279273
3466	26.0	3	1.5	3	205	0.308320
115	19.3	3	0.3	9	177	-0.349745

Notice how the predictions are numbers that go from something like -9 to something like 1. Those are not predictions of the sales column, which is in the order of the hundreds. Rather, **it's a prediction of how much sales would change if we increased price by one unit**. Right off of the bat, we can see some strange numbers. For example, take a look at day 4764. It's predicting a positive sensitivity. In other words, we are predicting that sales will increase if we increase the price of ice cream. This doesn't appeal to our economic sense. It's probably the case that the model is doing some weird extrapolation on that prediction. Fortunately, you don't have to worry too much about it. Remember that our ultimate goal is to segment the units by how sensitive they are to the treatment. It's **not** to come up with the most accurate sensitivity prediction ever. For our main goal, it suffices if the sensitivity predictions orders the units according to how sensitive they are. In other words, even if positive sensitivity predictions like 1.1, or 0.5 don't make much sense, all we need is that the ordering is correct, that is, we want the units with prediction 1.1 to be less impacted by price increase than units with predictions 0.5.

Ok, we have our sensitivity or CATE model. But there is still a lurking question: how do they compare to an ML predictive model? Let's try that now. We will use a machine learning algorithm that uses price, temperature, weekday and cost as features X and tries to predict ice cream sales.

```
X = ["temp", "weekday", "cost", "price"]
y = "sales"
ml = GradientBoostingRegressor()
ml.fit(train[X], train[y])

# make sure the model is not overfiting.
ml.score(test[X], test[y])
```

```
0.9124088322890126
```

This model can make predictions about how much sales we will have each day. But is it suited for what we really want? In other words, can this model distinguish between days where people are more sensitive to ice cream prices? Can it help us decide how much to charge depending on that price sensitivity?

To see which model is more useful, let's try using them for segmenting the units. For each model, we will partition the units into 2 groups. Our hope is that one group is highly responsive to price increase while the other not so much. If that is the case,

we can organize our business around those groups: for the days that fall in the high responsiveness group, we better not set prices too high. For the low responsiveness group, we can increase prices without risking too much in sales.

```
bands_df = pred_sens3.assign(
    sens_band = pd.qcut(pred_sens3["pred_sens"], 2),
      ↪ # create two groups based on sensitivity predictions
    pred_sales = ml.predict(pred_sens3[X]),
    pred_band = pd.qcut(ml.predict(pred_sens3[X]), 2),
      ↪ # create two groups based on sales predictions
)

bands_df.head()
```

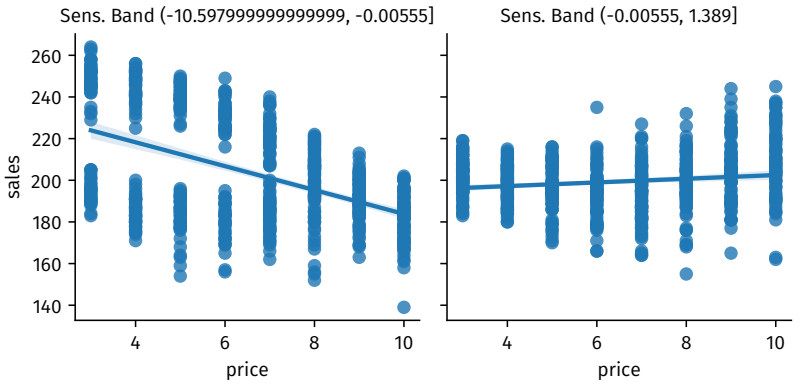
	temp	weekday	cost	price	sales	pred_sens
2648	18.6	7	0.5	10	185	-10.301045
2456	26.0	3	0.5	10	200	0.036165
4557	23.7	3	0.3	8	192	-0.132057
4884	28.9	4	1.5	6	213	0.860663
92	23.7	1	0.5	8	207	-9.953698

	sens_band	pred_sales	pred_band
2648	(-10.597999999999999, -0.00555]	186.878081	(161.089, 198.735]
2456	(-0.00555, 1.389]	203.188327	(198.735, 257.746]
4557	(-10.597999999999999, -0.00555]	188.800637	(161.089, 198.735]
4884	(-0.00555, 1.389]	210.430813	(198.735, 257.746]
92	(-10.597999999999999, -0.00555]	209.044522	(198.735, 257.746]

Next, we need to compare which of these two segmentations is the best one. I might be getting ahead of myself now, since we will only look at CATE model evaluation in the next chapter. But I feel I can give you a taste of what it looks like. One very simple way to check how good those partition schemas are - and by good I mean useful - is to plot a regression line of prices on sales for each partition. We can achieve that easily with Seaborn's `regplot` combined with `FacetGrid`.

Below, we can see the partitions made using the sensitivity predictions. Remember that all of this is done in the test set.

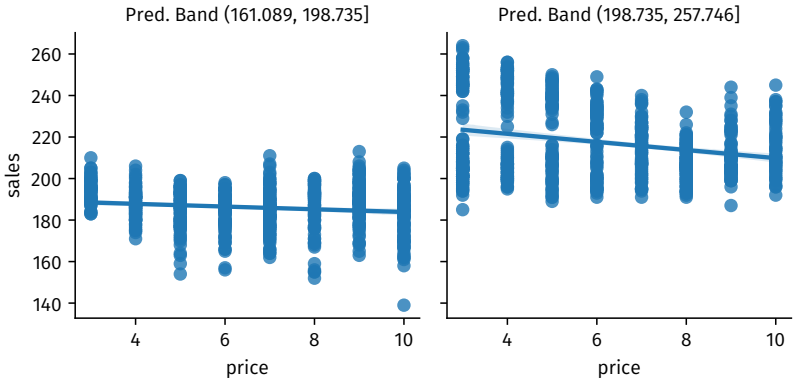
```
g = sns.FacetGrid(bands_df, col="sens_band")
g.map_dataframe(sns.regplot, x="price", y="sales")
g.set_titles(col_template="Sens. Band {col_name}");
```



As we can see, it looks like this partitioning scheme is useful. For the first partition, there is a high price sensitivity. Sales are going down by a lot as prices go up. However, for the second partition, sales remain roughly unchanged as the price goes up. In fact, it even looks like sales are going up as we increase the price, but that's probably noise.

Contrast this with the partitions made using the ML prediction model:

```
g = sns.FacetGrid(bands_df, col="pred_band")
g.map_dataframe(sns.regplot, x="price", y="sales")
g.set_titles(col_template="Pred. Band {col_name}");
```



I really like this plot because it conveys a very important point. As you can see, the predictive model partitions are splitting the units on the y axis. On days like those in the first partition, we don't sell a lot of ice cream, but we do sell more on days like those in the second partition. I find this amazing because the prediction model is doing exactly what it is supposed to do: it predicts sales. It can distinguish between days where there will be low versus high ice cream sales.

The only problem is that prediction is not particularly useful here. Ultimately, we want to know when we can increase prices and when we can't. But once we look at the slopes of the lines in the predictive model partitions, we see that they don't change much. In

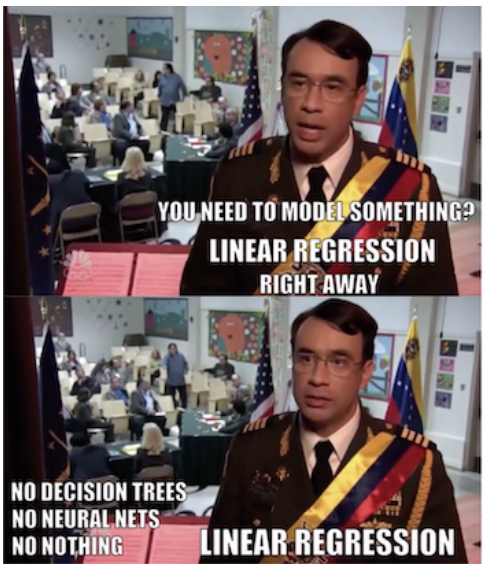
other words, both partitions, as defined by the prediction model, have about the same responsiveness to price increase. This doesn't offer us much insight into which are the days we can increase prices, since it looks like price is not affecting sales at all.

Key Ideas

We finally formalized the concept of Conditional Average Treatment Effect and how it can be useful for personalisation. Namely, if we can understand how each unit responds to a treatment, that is, if we can understand the heterogeneity of the treatment effect, we can give the best treatment depending on the unit's individual characteristics.

We've also contrasted this goal with that of a predictive model. Namely, we are rethinking the estimation task, from predicting Y in it's raw format to predicting how Y changes with T, $\frac{\delta y}{\delta t}$.

Sadly, it's not at all obvious how to build models for that. Since we can't observe sensitivity directly, it's hard to make a model that predicts it. But linear regression came to our rescue. By using a regression model that is fitted to predict Y, we found a way to also predict $\frac{\delta y}{\delta t}$. We also had to include interaction terms of the treatment and the features. This made it so that our sensitivity prediction was different for each customer. In other words, we were now estimating $E[Y'(t)|X]$. Those sensitivity predictions were then used to group our units into more or less sensitive to the treatment, ultimately helping us decide the treatment level for each group.



One natural question that arises from all this is if we can replace the linear regression by a generic machine learning model and use that to predict sensitivity. The answer is yes, but there are some caveats. This chapter used a very simple CATE model because I think it's easier to understand the concept behind them with linear regression. Don't worry,

though. We will see some more sophisticated models in the chapters to come. But before that, I first need to cover a very important topic, which is how can we compare two CATE models and decide which one is better.

References

The things I've written here are mostly stuff from my head. I've learned them through experience. This means there isn't a direct reference I can point you to. It also means that the things I wrote here have **not** passed the academic scrutiny that good science often goes through. Instead, notice how I'm talking about things that work in practice, but I don't spend too much time explaining why that is the case. It's a sort of science from the streets, if you will. However, I am putting this up for public scrutiny, so, by all means, if you find something preposterous, open an issue and I'll address it to the best of my efforts.

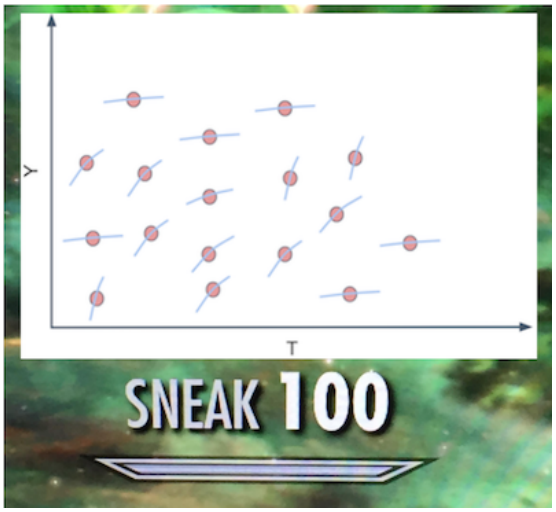
CHAPTER
19

Evaluating Causal Models

In the vast majority of material about causality, researchers use synthetic data to check if their methods are any good. Much like we do in the When Prediction Fails appendix chapter, they generate data on both Y_{0i} and Y_{1i} so that they can check if their model is correctly capturing the treatment effect $Y_{1i} - Y_{0i}$. That's fine for academic purposes, but in the real world, we don't have that luxury. When applying these techniques in the industry, we'll be asked time and again to prove why our model is better, why should it replace the current one in production or why it won't fail miserably. This is so crucial that it's beyond my comprehension why we don't see any material whatsoever explaining how we should evaluate causal inference models with real data.

As a consequence, data scientists that want to apply causal inference models have a really hard time convincing management to trust them. The approach they take is one of showing how sound the theory is and how careful they've been while training the model. Unfortunately, in a world where train-test split paradigm is the norm, that just won't cut it. The quality of your model will have to be grounded on something more concrete than a beautiful theory. Think about it. Machine learning has only achieved its huge success because predictive model validation is very straightforward. There is something reassuring about seeing that the predictions match what really happened.

Unfortunately, it isn't obvious at all how we achieve anything like a train-test paradigm in the case of causal inference. That's because causal inference is interested in estimating an unobservable quantity, $\frac{\delta y}{\delta t}$. Well, if we can't see it, how the hell are we supposed to know if our models are any good at estimating it? Remember, it is as if every entity had an underlying responsiveness, denoted by the slope of the line from treatment to outcome, but we can't measure it.



This is a very very very hard thing to wrap our heads around and it took me years to find something close to an answer. It's not a definitive one, but it works in practice and it has that concreteness, which I hope will approach causal inference from a train-test paradigm similar to the one we have with machine learning. The trick is to use aggregate measurements of sensitivity. Even if you can't estimate sensitivity individually, you can do it for a group and that is what we will leverage here.

```
import pandas as pd
import numpy as np
from matplotlib import pyplot as plt
import seaborn as sns
from toolz import curry

import statsmodels.formula.api as smf
import statsmodels.api as sm

from sklearn.ensemble import GradientBoostingRegressor

import warnings
warnings.filterwarnings("ignore")
```

In this chapter, we'll use non random data to estimate our causal models and random data to evaluate it. Again, we will be talking about how price impacts ice cream sales. As we'll see, random data is very valuable for evaluation purposes. However, in real life, it is often expensive to collect random data (why would you set prices at random if you know some of them are not very good ones and will only make you lose money???). What ends up happening is that we often have an abundance of data where the treatment is **NOT** random and very few, if any, random data. Since evaluating a model with non random data is incredibly tricky, if we have any random data, we tend to leave that for evaluation purposes.

And just in case you forgot, here is what the data looks like.

```
prices = pd.read_csv("./data/ice_cream_sales.csv") # loads non-random data
prices_rnd = pd.read_csv("./data/ice_cream_sales_rnd.csv") # loads random data
print(prices_rnd.shape)
prices.head()
```

```
(5000, 5)
```

	temp	weekday	cost	price	sales
0	17.3	6	1.5	5.6	173
1	25.4	3	0.3	4.9	196
2	23.3	5	1.5	7.6	207
3	26.9	1	0.3	5.3	241
4	20.2	1	1.0	7.2	227

To have something to compare, let's train two models. The first one will be a linear regression with interactions terms so that sensitivity is allowed to vary between units.

$$sales_i = \beta_0 + \beta_1 price_i + \boldsymbol{\beta}_2 \boldsymbol{X}_i + \boldsymbol{\beta}_3 \boldsymbol{X}_i price_i + e_i$$

Once we fit this model, we will be able to make sensitivity predictions

$$\widehat{\frac{\delta sales}{\delta price}} = \hat{\beta}_1 + \hat{\boldsymbol{\beta}}_3 \boldsymbol{X}_i$$

```
m1 = smf.ols("sales ~ price*cost + price*C(weekday) + price*temp",
  ↪ data=prices).fit()
```

The second model will be fully nonparametric, machine learning, predictive model

$$sales_i = G(X_i, price_i) + e_i$$

```
X = ["temp", "weekday", "cost", "price"]
y = "sales"

np.random.seed(1)
m2 = GradientBoostingRegressor()
m2.fit(prices[X], prices[y]);
```

Just to make sure the model is not heavily overfitting, we can check the R^2 on the data we've used to train it and on the new, unseen data. (For those more versed in Machine Learning, notice that some drop in performance is expected, because there is a concept drift. The model was trained in data where price is not random, but the test set has only randomized prices).

```
print("Train Score:", m2.score(prices[X], prices[y]))
print("Test Score:", m2.score(prices_rnd[X], prices_rnd[y]))
```

```
Train Score: 0.9251704824568053
Test Score: 0.7711074163447711
```

After training our models, we will get the sensitivity from the regression model. Again, we will resort to a numerical approximation

$$\frac{\delta y(t)}{\delta t} \approx \frac{y(t+h) - y(t)}{h}$$

Our models were trained on non-random data. Now we turn to the random data to make predictions. Just so we have everything in one place, we will add the predictions from the machine learning model and the sensitivity prediction from the causal model in a single dataframe, `prices_rnd_pred`.

Moreover, let's also include a random model. The idea is that this model just outputs random numbers as predictions. It is obviously not very useful, but it shall serve well as a benchmark. Whenever we are talking about new ways of making evaluations, I always like to think about how a random (useless) model would do. If the random model is able to perform well on the evaluation criterion, that says something about how good the evaluation method really is.

```
def predict_sensitivity(model, price_df, h=0.01):
    return (model.predict(price_df.assign(price=price_df["price"]+h))
            - model.predict(price_df)) / h

np.random.seed(123)
prices_rnd_pred = prices_rnd.assign(**{
    "sensitivity_m_pred": predict_sensitivity(m1, prices_rnd),
     ↪ ## sensitivity model
    "pred_m_pred": m2.predict(prices_rnd[X]), ## predictive model
    "rand_m_pred": np.random.uniform(size=prices_rnd.shape[0]), ## random model
})

prices_rnd_pred.head()
```

	temp	weekday	cost	price	sales	sensitivity_m_pred	pred_m_pred	rand_m_pred
0	25.8	1	0.3	7	230	-13.096964	224.067406	0.696469
1	22.7	3	0.5	4	190	1.054695	189.889147	0.286139
2	33.7	7	1.0	5	237	-17.362642	237.255157	0.226851
3	23.0	4	0.5	5	193	0.564985	186.688619	0.551315
4	24.4	1	1.0	3	252	-13.717946	250.342203	0.719469

Sensitivity by Model Band

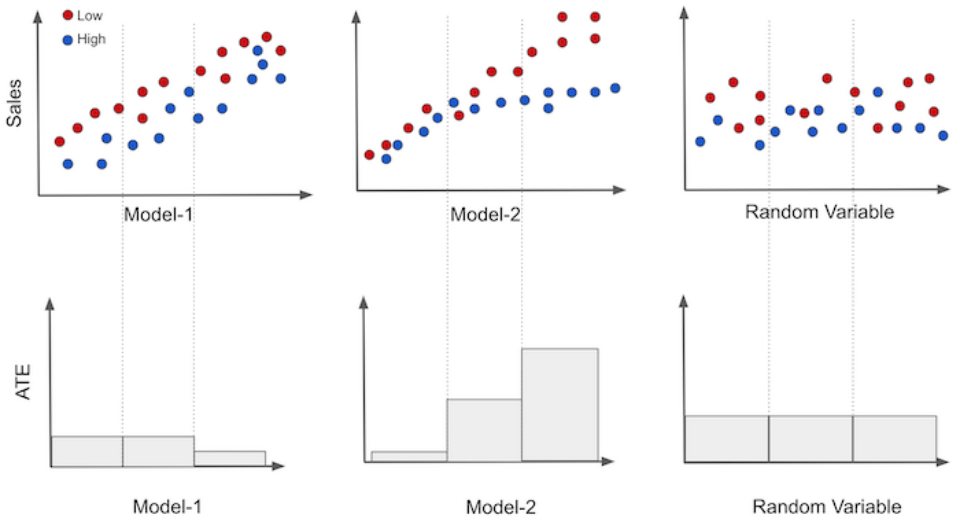
Now that we have our predictions, we need to evaluate how good they are. And remember, we can't observe sensitivity, so there isn't a simple ground truth we can compare against.

Instead, let's think back to what we want from our sensitivity models. Perhaps that will give us some insights into how we should evaluate them.

The idea of making treatment sensitivity models came from the necessity of finding which units are more sensitive to the treatment and which are less. It came from a desire for personalisation. Maybe a marketing campaign is very effective in only one segment of the population. Maybe discounts only work for some type of customers. A good causal model should help us find which customers will respond better and worse to a proposed treatment. They should be able to separate units into how elastic or sensitive they are to the treatment. In our ice cream example, the model should be able to figure out in which days people are willing to spend more on ice cream or, in which days the price sensitivity is less negative.

If that is the goal, it would be very useful if we could somehow order units from more sensitive to less sensitive. Since we have the predicted sensitivity, we can order the units by that prediction and hope it also orders them by the real sensitivity. Sadly, we can't evaluate that ordering on a unit level. But, what if we don't need to? What if, instead, we evaluate groups defined by the ordering? If our treatment is randomly distributed (and here is where randomness enters), estimating sensitivity for a group of units is easy. All we need is to compare the outcome between the treated and untreated.

To understand this better, it's useful to picture the binary treatment case. Let's keep the pricing example, but now the treatment is a discount. In other words, prices can be either high (untreated) or low (treated). Let's plot sales on the Y axis, each of our models in the X axis and price as the color. Then, we can split the data on the model axis into three equal sized groups. **If the treatment was randomly assigned**, we can easily estimate the ATE for each group $E[Y \mid T = 1] - E[Y \mid T = 0]$.



In the image, we can see that the first model is somewhat good at predicting sales (high correlation with sales), but the groups it produces have roughly the same treatment effect,

as shown in the plot on the bottom. Two of the three segments have the same sensitivity and only the last one has a different, lower sensitivity.

On the other hand, each group produced by the second model has a different causal effect. That's a sign this model can indeed be useful for personalisation. Finally, the random model produces groups with the exact same sensitivity. That's not very useful, but it's expected. If the model is random, each segment it produces will be a random and representative sample of the data. So the sensitivity in its groups should be roughly the same as the ATE on the entire dataset.

Just by looking at these plots, you can get a feeling of which model is better. The more ordered the sensitivities look like and the more different they are between bands, the better. Here, model 2 is probably better than model 1, which is probably better than the random model.

To generalize this to the continuous case, we can estimate the sensitivity using a single variable linear regression model.

$$y_i = \beta_0 + \beta_1 t_i + e_i$$

If we run that model with the sample from a group, we will be estimating the sensitivity inside that group.

From the theory on simple linear regression, we know that

$$\hat{\beta}_1 = \frac{\sum(t_i - \bar{t})(y_i - \bar{y})}{\sum(t_i - \bar{t})^2}$$

where $\bar{t}$ is the sample average for the treatment and $\bar{y}$ is the sample average for the outcome. Here is what that looks like in code

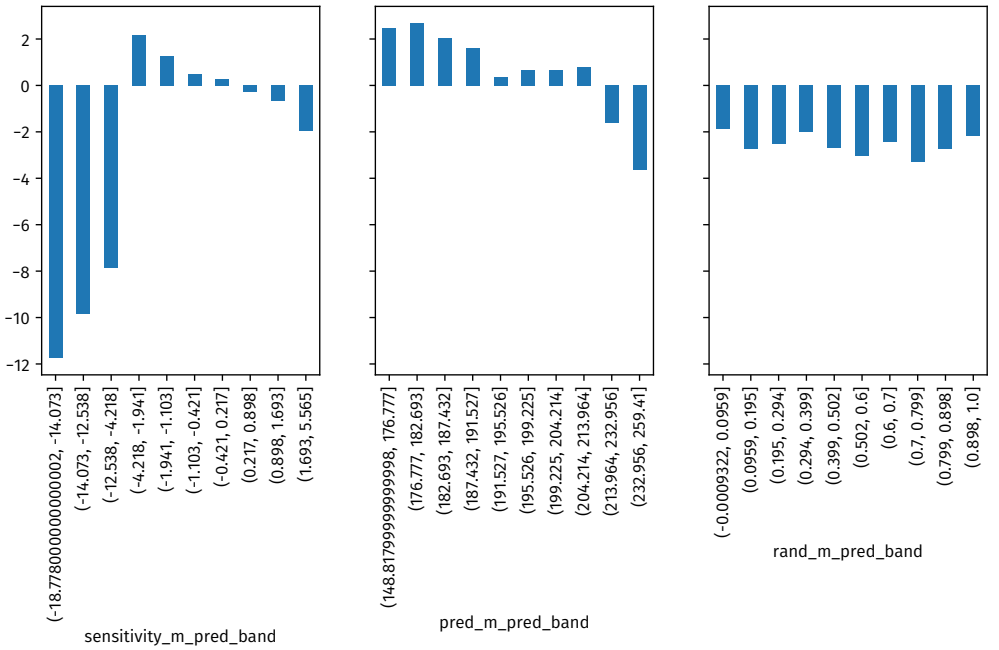
```
@curry
def sensitivity(data, y, t):
        # line coeficient for the one variable linear regression
        return (np.sum((data[t] - data[t].mean())*(data[y] - data[y].mean())) /
                np.sum((data[t] - data[t].mean())**2))
```

Let's now apply this to our ice cream price data. For that, we also need a function that segments the dataset into partitions of equal size and applies the sensitivity to each partition. The following code should handle that.

```
def sensitivity_by_band(df, pred, y, t, bands=10):
    return (df
            .assign(**{f"{pred}_band":pd.qcut(df[pred],
              ↪ q=bands)}) # makes quantile partitions
            .groupby(f"{pred}_band")
            .apply(sensitivity(y=y, t=t)))
             ↪ # estimate the sensitivity on each partition
```

Finally, let's plot the sensitivity by band using the predictions we've made before. Here, we will use each model to construct partitions and then estimate the sensitivity on each partition.

```
fig, axs = plt.subplots(1, 3, sharey=True, figsize=(10, 4))
for m, ax in zip(["sensitivity_m_pred", "pred_m_pred", "rand_m_pred"], axs):
    sensitivity_by_band(prices_rnd_pred, m, "sales", "price").plot.bar(ax=ax)
```



First, look at the random model (`rand_m`). It has roughly the same estimated sensitivity in each of its partitions. We can already see just by looking at the plot that it won't help us much with personalisation since it can't distinguish between the high and low price sensitivity days. Next, consider the predictive model, `pred_m`. That model is actually promising! It manages to construct groups where the sensitivity is high and others where the sensitivity is low. That's exactly what we need.

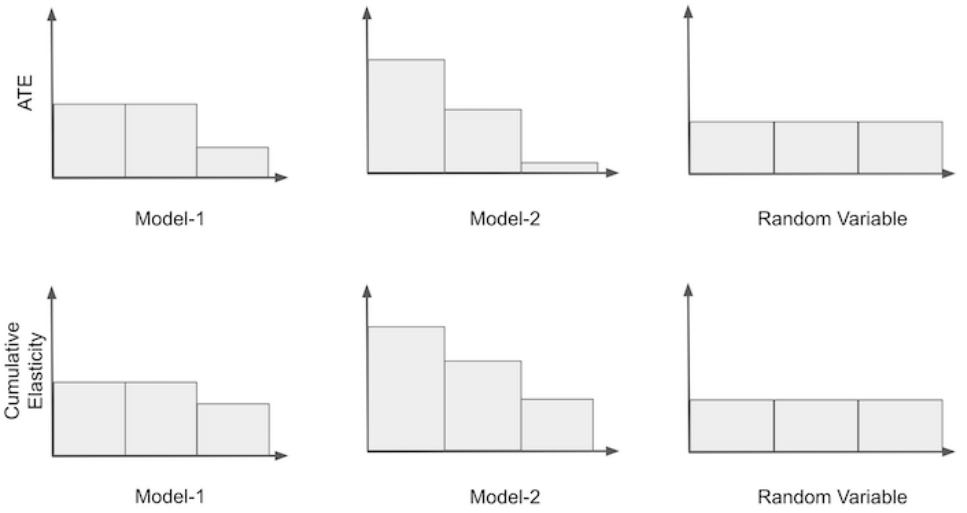
Finally, the causal model `sensitivity_m` looks a bit weird. It identifies groups of really low sensitivity, where low here actually means high price sensitivity (sales will decrease by a lot as we increase prices). Detecting those high price sensitivity days is very useful for us. If we know when they are, we will be careful not to go on increasing prices on those types of days. The causal model also identifies some less sensitive regions, so it can successfully distinguish high from low sensitivities. But the ordering is not as good as that of the predictive model.

So, what should we decide? Which one is more useful? The predictive or the causal model? The predictive model has better ordering, but the causal model can better identify the extremes. The sensitivity by band plot is a good first check, but it can't answer precisely which model is better. We need to move to something more elaborate.

Cumulative Sensitivity Curve

Consider again the illustrative example where price was converted to a binary treatment. We will take it from where we left, so we had the sensitivity of the treatment by band. What we can do next is order the band according to how sensitive they are. That is, we take the most sensitive group and place it in the first place, the second most sensitive group in the second place and so on. For both models 1 and 3, no re-ordering needs to be made, since they are already ordered. For model 2, we have to reverse the ordering.

Once we have the ordered groups, we can construct what we will call the Cumulative Sensitivity Curve. We first compute the sensitivity of the first group; then, of the first and the second and so on, until we've included all the groups. In the end, we will just compute the sensitivity for the entire dataset. Here is what it would look like for our illustrative example.



Notice that the first bin in the cumulative sensitivity is just the ATE from the most sensitive group according to that model. Also, for all models, the cumulative sensitivity will converge to the same point, which is the ATE for the entire dataset.

Mathematically, we can define the cumulative sensitivity as the sensitivity estimated up until unit k.

$$\widehat{y'(t)}_k = \hat{\beta}_{1k} = \frac{\sum_i^k (t_i - \bar{t})(y_i - \bar{y})}{\sum_i^k (t_i - \bar{t})^2}$$

To build the cumulative sensitivity curve, we run the above function iteratively in the dataset to produce the following sequence.

$$(\widehat{y'(t)}_1, \widehat{y'(t)}_2, \widehat{y'(t)}_3, ..., \widehat{y'(t)}_N)$$

This is a very interesting sequence in terms of model evaluation because we can make preference statements about it. First, a model is better to the degree that

$$\hat{y}'(t)_k > \hat{y}'(t)_{k+a}$$

for any k and $a > 0$. In words, if a model is good at ordering sensitivity, the sensitivity observed in the top k samples should be higher than the sensitivity observed in top $k + a$ samples. Or, simply put, if I look at the top units, they should have higher sensitivity than units below them.

Second, a model is better to the degree that

$$\hat{y}'(t)_k - \hat{y}'(t)_{k+a}$$

is the largest, for any k and $a > 0$. The intuition being that not only do we want the sensitivity of the top k units to be higher than the sensitivity of the units below them, but we want that difference to be as large as possible.

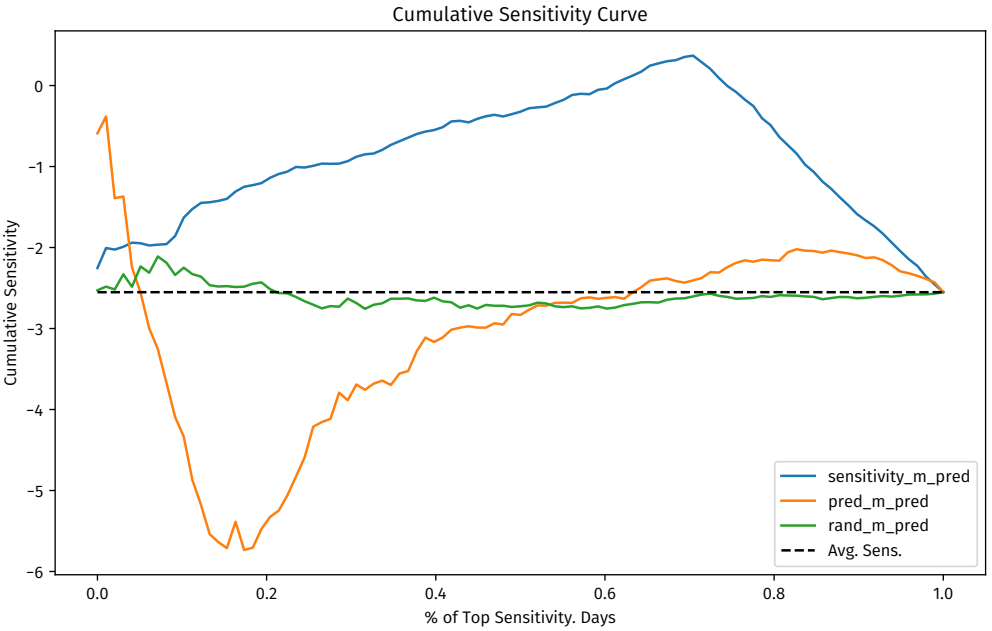
To make it more concrete, here is this idea represented in code.

```
def cumulative_sensitivity_curve(dataset, prediction, y, t, min_periods=30,
  ↪ steps=100):
    size = dataset.shape[0]

    # orders the dataset by the `prediction` column
    ordered_df = dataset.sort_values(prediction, ascending=False).reset_index(
      ↪ drop=True)

    # create a sequence of row numbers that will define our Ks
    # The last item is the sequence is all the rows (the size of the dataset)
    n_rows = list(range(min_periods, size, size // steps)) + [size]

    # cumulative computes the sensitivity. First for the top min_periods units.
    # then for the top (min_periods + step*1), then (min_periods + step*2) and so
          ↪ on
    return np.array([sensitivity(ordered_df.head(rows), y,
      ↪ t) for rows in n_rows])
```

Some things to notice about this function. It assumes that the thing which ordered the sensitivity is stored in the column passed to the `prediction` argument. Also, the first group has `min_periods` units, so it can be different from the others. The reason is that, due to small sample size, the sensitivity can be too noisy at the beginning of the curve. To fix that, we can pass a first group which is already large enough. Finally, the `steps` argument defines how many extra units we include in each subsequent group.

With this function, we can now plot the cumulative sensitivity curve, according to the ordering produced by each of our models.

```
plt.figure(figsize=(10,6))

for m in ["sensitivity_m_pred", "pred_m_pred", "rand_m_pred"]:
    cumu_sens = cumulative_sensitivity_curve(prices_rnd_pred, m, "sales",
      ↪ "price", min_periods=100, steps=100)
    x = np.array(range(len(cumu_sens)))
    plt.plot(x/x.max(), cumu_sens, label=m)

plt.hlines(sensitivity(prices_rnd_pred, "sales", "price"), 0, 1,
  ↪ linestyles="--", color="black", label="Avg. Sens.")
plt.xlabel("% of Top Sensitivity. Days")
plt.ylabel("Cumulative Sensitivity")
plt.title("Cumulative Sensitivity Curve")
plt.legend();
```



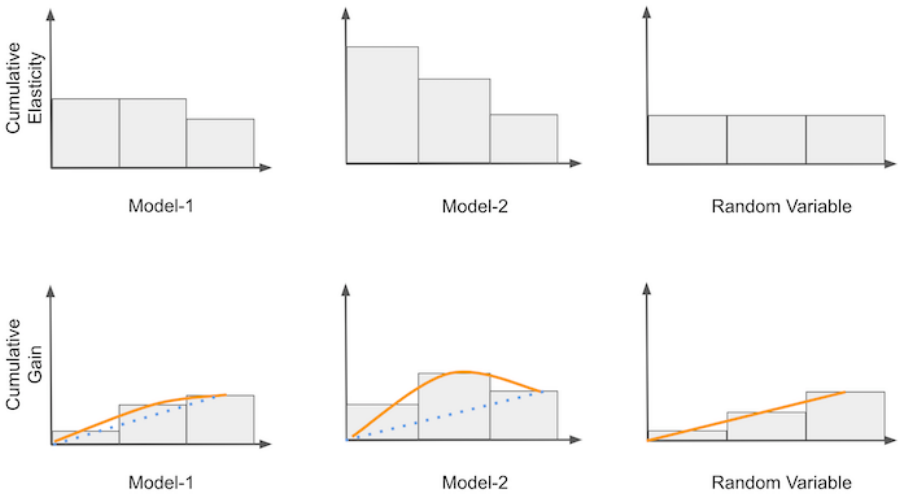
Interpreting a Cumulative Sensitivity Curve can be a bit challenging, but here is how I see it. Again, it might be easier to think about the binary case. The X axis of the curve represents how many samples we are treating. Here, I've normalized the axis to be the proportion of the dataset, so .4 means we are treating 40% of the samples. The Y axis is the sensitivity we should expect at that many samples. So, if a curve has value -1 at 40%, it means that the sensitivity of the top 40% units is -1. Ideally, for this case, we want the least negative sensitivity for the largest possible sample. An ideal curve then would start high up on the Y axis and descend very slowly to the average sensitivity, representing we can treat a high percentage of units while still maintaining a low magnitude sensitivity.

Needless to say, none of our models gets even close to an ideal sensitivity curve. The random model `rand_m` oscillates around the average sensitivity and never goes too far away from it. This means that the model can't find groups where the sensitivity is different from the average one. As for the predictive model `pred_m`, it appears to be reversely ordering sensitivity, because the curve starts below the average sensitivity. Not only that, it also converges to the average sensitivity pretty quickly, at around 50% of the samples. Finally, the causal model `sensitivity_m` seems more interesting. It has this weird behavior at first, where the cumulative sensitivity increases away from the average, but then it reaches a point where we can treat about 75% of the units while keeping a pretty decent sensitivity of almost 0. This is probably happening because this model can identify the very low sensitivity (high price sensitivity) days. Hence, provided we don't increase prices on those days, we are allowed to do it for most of the sample (about 75%), while still having a low price sensitivity.

In terms of model evaluation, the Cumulative Sensitivity Curve is already much better than the simple idea of sensitivity by band. Here, we managed to make preference statements about our models that were much more precise. Still, it's a complicated curve to understand. For this reason, we can do one further improvement.

Cumulative Gain Curve

The next idea is a very simple, yet powerful improvement on top of the cumulative sensitivity. We will multiply the cumulative sensitivity by the proportional sample size. For example, if the cumulative sensitivity is, say -0.5 at 40%, we will end up with -0.2 (-0.5 * 0.4) at that point. Then, we will compare this with the theoretical curve produced by a random model. This curve will actually be a straight line from 0 to the average treatment effect. Think about it this way: every point in the cumulative sensitivity of a random model is the ATE, because the model just produces random representative partitions of the data. If at each point along the (0,1) line we multiply the ATE by that point, we will end up with a straight line between zero and the ATE.



Once we have the theoretic random curve, we can use it as a benchmark and compare our other models against it. All curves will start and end at the same point. However, the better the model at ordering sensitivity, the more the curve will diverge from the random line in the points between zero and one. For example, in the image above, M2 is better than M1 because it diverges more before reaching the ATE at the end point. For those familiar with the ROC curve, you can think about Cumulative Gain as the ROC for causal models.

Mathematically speaking,

$$\widehat{F(t)}_k = \hat{\beta}_{1k} * \frac{k}{N} = \frac{\sum_i^k (t_i - \bar{t})(y_i - \bar{y})}{\sum_i^k (t_i - \bar{t})^2} * \frac{k}{N}$$

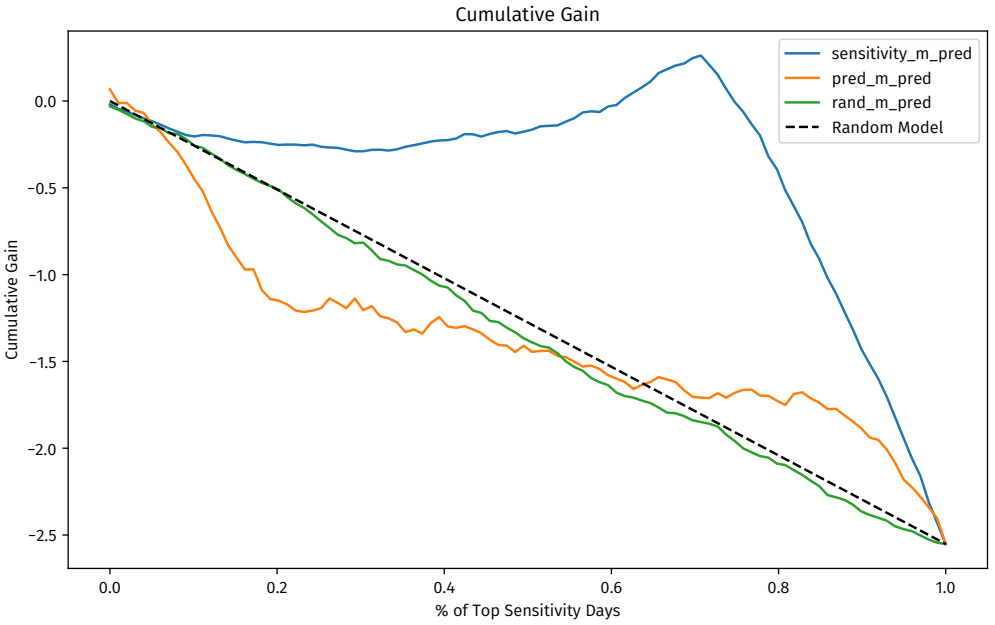
To implement it in code, all we have to do is add the proportional sample size normalization.

```
def cumulative_gain(dataset, prediction, y, t, min_periods=30, steps=100):
    size = dataset.shape[0]
    ordered_df = dataset.sort_values(prediction, ascending=False).reset_index(
      ↪ drop=True)
    n_rows = list(range(min_periods, size, size // steps)) + [size]

    ## add (rows/size) as a normalizer.
    return np.array([sensitivity(ordered_df.head(rows), y, t) * (rows/size) for
      ↪ rows in n_rows])
```

For our ice cream data, we will get the following curves.

```
plt.figure(figsize=(10,6))

for m in ["sensitivity_m_pred", "pred_m_pred", "rand_m_pred"]:
    cumu_gain = cumulative_gain(prices_rnd_pred, m, "sales", "price",
      ↪ min_periods=50, steps=100)
```

```
    x = np.array(range(len(cumu_gain)))
    plt.plot(x/x.max(), cumu_gain, label=m)

plt.plot([0, 1], [0, sensitivity(prices_rnd_pred, "sales", "price")],
  ↪ linestyle="--", label="Random Model", color="black")

plt.xlabel("% of Top Sensitivity Days")
plt.ylabel("Cumulative Gain")
plt.title("Cumulative Gain")
plt.legend();
```



Now it is very clear that the causal model (`sensitivity_m`) is much better than the other two. It diverges much more from the random line than both `rand_m` and `pred_m`. Also, notice how the actual random model follows very closely the theoretical random model. The difference between both is probably just random noise.

With that, we covered some really nice ideas on how to evaluate causal models. That alone is a huge deed. We managed to evaluate how good are models in ordering sensitivity even though we didn't have a ground truth to compare against. There is only one final thing missing, which is to include a confidence interval around those measurements. After all, we are not barbarians, are we?

Taking Variance Into Account

It just feels wrong to not take variance into account when we are dealing with the sensitivity curves. Specially since all of them use linear regression theory, so adding a confidence interval around them should be fairly easy.

To achieve that, we will first create a function that returns the CI for a linear regression parameter. I'm using the formula for the simple linear regression here, but feel free to extract the CI however you want.

$$s_{\hat{\beta}_1} = \sqrt{\frac{\sum_i \hat{\epsilon}_i^2}{(n-2)\sum_i (t_i - \bar{t})^2}}$$

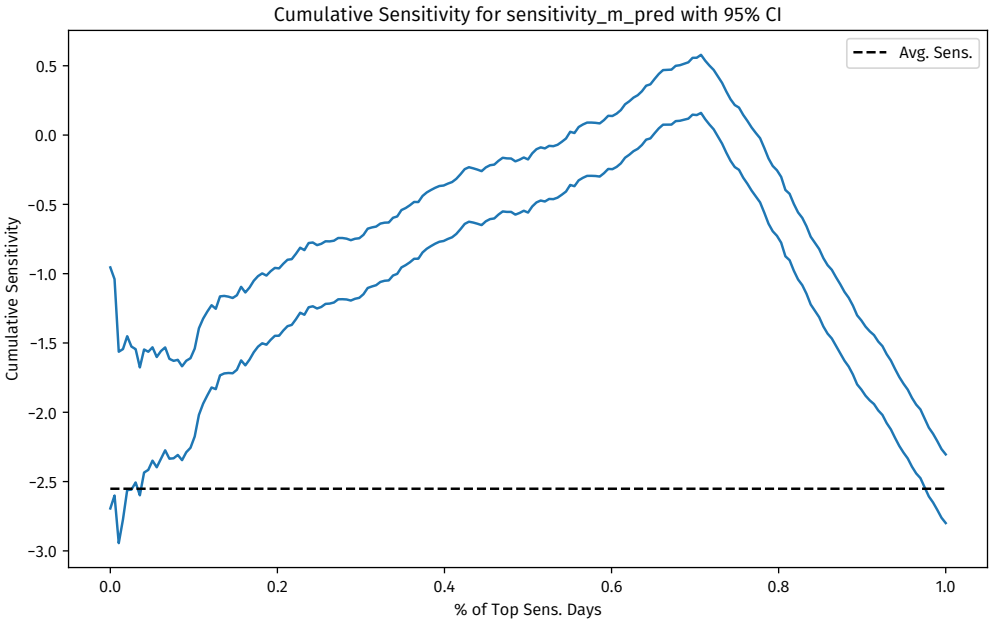
```
def sensitivity_ci(df, y, t, z=1.96):
    n = df.shape[0]
    t_bar = df[t].mean()
    beta1 = sensitivity(df, y, t)
    beta0 = df[y].mean() - beta1 * t_bar
    e = df[y] - (beta0 + beta1*df[t])
    se = np.sqrt(((1/(n-2))*np.sum(e**2))/np.sum((df[t]-t_bar)**2))
    return np.array([beta1 - z*se, beta1 + z*se])
```

With some minor modification on our `cumulative_sensitivity_curve` function, we can output the confidence interval for the sensitivity.

```
def cumulative_sensitivity_curve_ci(dataset, prediction, y, t, min_periods=30,
  ↪ steps=100):
    size = dataset.shape[0]
    ordered_df = dataset.sort_values(prediction, ascending=False).reset_index(
      ↪ drop=True)
    n_rows = list(range(min_periods, size, size // steps)) + [size]

    # just replacing a call to `sensitivity` by a call to `sensitivity_ci`
    return np.array([sensitivity_ci(ordered_df.head(rows), y,
      ↪ t)  for rows in n_rows])
```

And finally, here is the cumulative sensitivity curve with the 95% CI for the causal model.

```
plt.figure(figsize=(10,6))

cumu_gain_ci = cumulative_sensitivity_curve_ci(prices_rnd_pred,
  ↪ "sensitivity_m_pred", "sales", "price", min_periods=50, steps=200)
x = np.array(range(len(cumu_gain_ci)))
plt.plot(x/x.max(), cumu_gain_ci, color="C0")

plt.hlines(sensitivity(prices_rnd_pred, "sales", "price"), 0, 1,
  ↪ linestyles="--", color="black", label="Avg. Sens.")

plt.xlabel("% of Top Sens. Days")
plt.ylabel("Cumulative Sensitivity")
plt.title("Cumulative Sensitivity for sensitivity_m_pred with 95% CI")
plt.legend();
```



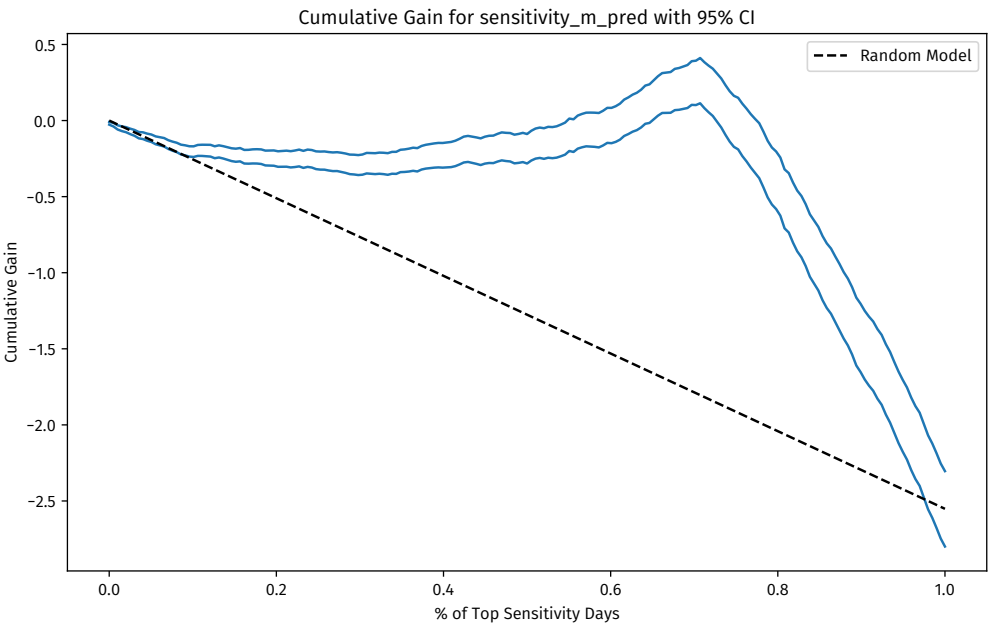
Notice how the CI gets smaller and smaller as we accumulate more of the dataset. That's because the sample size increases.

As for the Cumulative Gain curve, it is also equally simple to get the CI. Again, we just replace a call to the `sensitivity` function with a call to the `sensitivity_ci` function.

```
def cumulative_gain_ci(dataset, prediction, y, t, min_periods=30, steps=100):
    size = dataset.shape[0]
    ordered_df = dataset.sort_values(prediction, ascending=False).reset_index(
      ↪ drop=True)
    n_rows = list(range(min_periods, size, size // steps)) + [size]
    return np.array([sensitivity_ci(ordered_df.head(rows), y, t) * (rows/size)
      ↪ for rows in n_rows])
```

Here is what it looks like for the causal model. Notice that now, the CI starts small, even though the sample size is smaller at the beginning of the curve. The reason is that the normalization factor $\frac{k}{N}$ shrinks the ATE parameter and it's CI along with it. Since this curve should be used to compare models, this shouldn't be a problem, as the curve will apply this shrinking factor equally to all the models being evaluated.

```
plt.figure(figsize=(10,6))

cumu_gain = cumulative_gain_ci(prices_rnd_pred, "sensitivity_m_pred", "sales",
  ↪ "price", min_periods=50, steps=200)
x = np.array(range(len(cumu_gain)))
plt.plot(x/x.max(), cumu_gain, color="C0")

plt.plot([0, 1], [0, sensitivity(prices_rnd_pred, "sales", "price")],
  ↪ linestyle="--", label="Random Model", color="black")

plt.xlabel("% of Top Sensitivity Days")
plt.ylabel("Cumulative Gain")
plt.title("Cumulative Gain for sensitivity_m_pred with 95% CI")
plt.legend();
```



Key Ideas

Here we saw three ways to check how good a model is in terms of ordering sensitivity. We used these methods as a way to compare and decide between models that have a causal purpose. That's a huge deal. We've managed to check if a model is good at identifying groups with different sensitivities even without being able to see sensitivity!

Here, we relied heavily on random data. We trained the model on non-random data, but all the evaluation was done on a sample where the treatment has been randomized. That's because we need some way of confidently estimating sensitivity. Without random data, the simple formulas we used here wouldn't work. As we know very well by now, simple linear regression has omitted variable bias in the presence of confounding variables.

Nonetheless, if we can get our hands on some random data, we already know how to compare random models. In the next chapter, we will address the problem of non random data, but before we go, I wanted to say some last words about model evaluation.

Let's reiterate how important trustworthy model evaluation is. With the cumulative gain curve, we finally have a good way of comparing models that are used for causal inference. We can now decide which model is better for treatment personalisation. That's a major deal. Most materials you will find out there in causal inference don't give us a good way of doing model evaluation. In my opinion, that's the missing piece we need to make causal inference as popular as machine learning. With good evaluation, we can take causal inference closer to the train-test paradigm that has already been so useful for predictive models. That's a bold statement. Which means I'm careful when I say it, but until now, I haven't found any good criticism of it. If you have some, please let me know.

References

The things I've written here are mostly stuff from my head. I've learned them through experience. This means that they have **not** passed the academic scrutiny that good science often goes through. Instead, notice how I'm talking about things that work in practice, but I don't spend too much time explaining why that is the case. It's a sort of science from the streets, if you will. However, I am putting this up for public scrutiny, so, by all means, if you find something preposterous, open an issue and I'll address it to the best of my efforts.

I got the ideas from this chapter from a Pierre Gutierrez and Jean-Yves G'erardy's article called *Causal Inference and Uplift Modeling A review of the literature*. The authors explain the concept of a Qini curve. If you search that, you will find it's a technique used for uplift modeling, which you can think of as causal inference for when the treatment is binary. Here, I took the idea from a Qini curve and generalized it to the continuous treatment case. I think the methods presented here work for both continuous and binary cases, but then again, I've never seen them anywhere else, so keep that in mind.

I also strongly recommend reading the article by Leo Breiman (2001) on the train-test paradigm: Statistical Modeling: The Two Cultures. It's a great resource if you want to understand what makes a statistical technique successful.

CHAPTER

20

Plug-and-Play Estimators

So far, we've seen how to debias our data in the case where the treatment is not randomly assigned, which results in confounding bias. That helps us with the identification problem in causal inference. In other words, once the units are exchangeable, or $Y(0), Y(1) \perp T \mid X$, it becomes possible to learn the treatment effect. But we are far from done.

Identification means that we can find the average treatment effect. In other words, we know how effective a treatment is on average. Of course this is useful, as it helps us to decide if we should roll out a treatment or not. But we want more than that. We want to know if there are subgroups of units that respond better or worse to the treatment. That should allow for a much better policy, one where we only treat the ones that will benefit from it.

Problem Setup

Let's recall our setup of interest. Given the potential outcomes, we can define the individual treatment effect as the difference between the potential outcomes.

$$\tau_i = Y_i(1) - Y_i(0),$$

or, the continuous treatment case, $\tau_i = \partial Y(t)$, where t is the treatment variable. Of course, we can never observe the individual treatment effect, because we only get to see one of potential outcomes

$$Y_i^{obs}(t) = \begin{cases} Y_i(1), & \text{if } t = 1 \\ Y_i(0), & \text{if } t = 0 \end{cases}$$

We can define the average treatment effect (ATE) as

$$\tau = E[Y_i(1) - Y_i(0)] = E[\tau_i]$$

and the conditional average treatment effect (CATE) as

$$\tau(x) = E[Y_i(1) - Y_i(0) \mid X] = E[\tau_i \mid X]$$

In Part I of this book, we've focused mostly on the ATE. Now, we are interested in the CATE. The CATE is useful for personalising a decision making process. For example, if you have a drug as the treatment t, you want to know which type of patients are more responsive to the drug (higher CATE) and if there are some types of patients with a negative response (CATE < 0).

We've seen how to estimate the CATE using a linear regression with interactions between the treatment and the features

$$y_i = \beta_0 + \beta_1 t_i + \beta_2 X_i + \beta_3 t_i X_i + e_i.$$

If we estimate this model, we can get estimates for $\tau(x)$

$$\hat{\tau}(x) = \hat{\beta}_1 + \hat{\beta}_3 X_i$$

Still, the linear models have some drawbacks. The main one being the linearity assumption on X. Notice that you don't even care about β_2 on this model. But if the features X don't have a linear relationship with the outcome, your estimates of the causal parameters β_1 and β_3 will be off.

It would be great if we could replace the linear model by a more flexible machine learning model. We could even plug the treatment as a feature to an ML model, like boosted trees or a neural network

$$y_i = M(X_i, T_i) + e_i$$

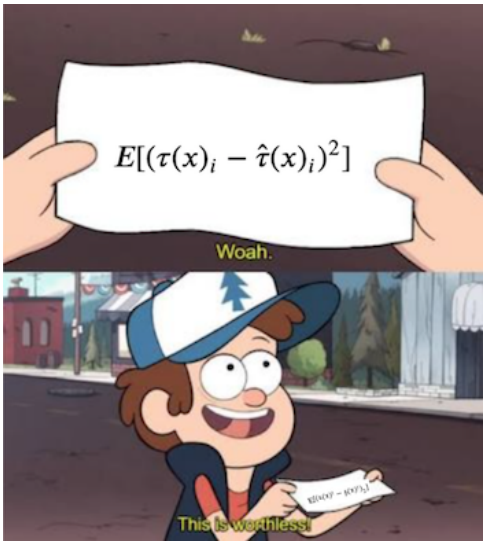
but from there, it is not clear how we can get treatment effect estimates, since this model will output $\hat{y}$ predictions, not $\hat{\tau(x)}$ predictions. Ideally, we would use a machine learning regression model that, instead of minimising the outcome MSE

$$E[(Y_i - \hat{Y}_i)^2]$$

would minimise the treatment effect MSE

$$E[(\tau(x)_i - \hat{\tau}(x)_i)^2] = E[(Y_i(1) - Y_i(0) - \hat{\tau}(x)_i)^2].$$

However, this criterion is what we call infeasible. Again, the problem here is that $\tau(x)_i$ is not observable, so we can't optimize it directly. This puts us in a tough spot… Let's try to simplify it a bit and maybe we can think of something.



Target Transformation

Suppose your treatment is binary. Let's say you are an investment firm testing the effectiveness of sending a financial education email. You hope the email will make people invest more. Also, let's say you did a randomized study where 50% of the customers got the email and the other 50% didn't.

Here is a crazy idea: let's transform the outcome variable by multiplying it with the treatment.

$$Y_i^* = 2Y_i * T_i - 2Y_i * (1 - T_i)$$

So, if the unit was treated, you would take the outcome and multiply it by 2. If it wasn't treated, you would take the outcome and multiply it by -2. For example, if one of your customers invested BRL 2000,00 and got the email, the transformed target would be 4000. However, if he or she didn't get the email, it would be -4000.

This seems very odd, because you are saying that the effect of the email can be a negative number, but bear with me. If we do a little bit of math, we can see that, on average or in expectation, this transformed target will be the treatment effect. This is nothing short of amazing. What I'm saying is that by applying this somewhat wacky transformation, I get to estimate something that I can't even observe.

To understand that, we need a bit of math. Because of random assignment, we have that $T \perp Y(0), Y(1)$, which is our old unconfoundedness friend. That implies that $E[T, Y(t)] = E[T] * E[Y(t)]$, which is a consequence of independence.

Also, we know that

$$Y_i * T_i = Y(1)_i * T_i \text{ and } Y_i * (1 - T_i) = Y(0)_i * (1 - T_i)$$

because the treatment is what materializes one or the other potential outcomes. With that in mind, let's take the expected value of Y_i^* and see what we end up with.

$$\begin{aligned}
E[Y_i^* | X_i = x] &= E[2Y(1)_i * T_i - 2Y(0)_i * (1 - T_i) | X_i = x] \\
&= 2E[Y(1)_i * T_i | X_i = x] - 2E[Y(0)_i * (1 - T_i) | X_i = x] \\
&= 2E[Y(1)_i | X_i = x] * E[T_i | X_i = x] - 2E[Y(0)_i | X_i = x] * E[(1 - T_i) | X_i = x] \\
&= 2E[Y(1)_i | X_i = x] * 0.5 - 2E[Y(0)_i | X_i = x] * 0.5 \\
&= E[Y(1)_i | X_i = x] - E[Y(0)_i | X_i = x] \\
&= \tau(x)_i
\end{aligned}$$

So, this apparently crazy idea ended up being an unbiased estimate of the individual treatment effect $\tau(x)_i$. Now, we can replace our infeasible optimization criterion with

$$E[(Y_i^* - \hat{\tau}(x)_i)^2]$$

In simpler terms, all we have to do is use any regression machine learning model to predict Y_i^* and this model will output treatment effect predictions.

Now that we've solved the simple case, what about the more complicated case, where treatment is not 50% 50%, or not even randomly assigned? As it turns out, the answer is a bit more complicated, but not much. First, if we don't have random assignment, we need at least conditional independence $T \perp Y(1), Y(0) | X$. That is, controlling for X, T is as good as random. With that, we can generalize the transformed target to

$$Y_i^* = Y_i * \frac{T_i - e(X_i)}{e(X_i)(1 - e(X_i))}$$

where $e(X_i)$ is the propensity score. So, if the treatment is not 50% 50%, but randomized with a different probability p, all you have to do is replace the propensity score in the above formula with p. If the treatment is not random, then you have to use the propensity score, either stored or estimated.

If you take the expectation of this, you will see that it also matches the treatment effect. The proof is left as an exercise to the reader. Just kidding, here it is. It's a bit cumbersome, so feel free to skip it.

$$\begin{aligned}
E[Y_i^* | X_i = x] &= E\big[Y_i * \frac{T_i - e(X_i)}{e(X_i)(1 - e(X_i))} | X_i = x\big] \\
&= E\big[Y_i T_i * \frac{T_i - e(X_i)}{e(X_i)(1 - e(X_i))} + Y_i(1 - T_i) * \frac{T_i - e(X_i)}{e(X_i)(1 - e(X_i))} | X_i = x\big] \\
&= E\big[Y(1)_i * \frac{T_i(1 - e(X_i))}{e(X_i)(1 - e(X_i))} | X_i = x\big] - E\big[Y(0)_i * \frac{(1 - T_i)e(X_i)}{e(X_i)(1 - e(X_i))} | X_i = x\big] \\
&= \frac{1}{e(X_i)} E[Y(1)_i * T_i | X_i = x] - \frac{1}{1 - e(X_i)} E[Y(0)_i * (1 - T_i) | X_i = x] \\
&= \frac{1}{e(X_i)} E[Y(1)_i | X_i = x] * E[T_i | X_i = x] - \frac{1}{1 - e(X_i)} E[Y(0)_i | X_i = x] * E[(1 - T_i) | X_i = x] \\
&= E[Y(1)_i | X_i = x] - E[Y(0)_i | X_i = x] \\
&= \tau(x)_i
\end{aligned}$$

As always, I think this will become much more concrete with an example. Again, consider the investment emails we've sent trying to make people invest more. The outcome variable is the binary (invested vs didn't invest) `converted`.

```
import pandas as pd
import numpy as np
from matplotlib import pyplot as plt
import seaborn as sns
from nb21 import cumulative_gain, elast
```

```
email = pd.read_csv("./data/invest_email_rnd.csv")
email.head()
```

	age	income	insurance	invested	em1	em2	em3	converted
0	44.1	5483.80	6155.29	14294.81	0	1	1	0
1	39.8	2737.92	50069.40	7468.15	1	0	0	0
2	49.0	2712.51	5707.08	5095.65	1	0	1	1
3	39.7	2326.37	15657.97	6345.20	1	1	1	0
4	35.3	2787.26	27074.44	14114.86	1	1	1	0

Our goal here is one of personalization. Let's focus on email-1. We wish to send it only to those customers who will respond better to it. In other words, we wish to estimate the conditional average treatment effect of email-1

$$E[Converted(1)_i - Converted(0)_i | X_i = x] = \tau(x)_i$$

so that we can target those customers who will have the best response to the email (higher CATE)

But first, let's break our dataset into a training and a validation set. We will estimate $\tau(x)_i$ on one set and evaluate the estimates on the other.

```
from sklearn.model_selection import train_test_split

np.random.seed(123)
train, test = train_test_split(email, test_size=0.4)
print(train.shape, test.shape)
```

```
(9000, 8) (6000, 8)
```

Now, we will apply the target transformation we've just learned. Since the emails were randomly assigned (although not on a 50% 50% basis), we don't need to worry about the propensity score. Rather, it is constant and equal to the treatment probability.

```
y = "converted"
T = "em1"
X = ["age", "income", "insurance", "invested"]

ps = train[T].mean()

y_star_train = train[y] * (train[T] - ps)/(ps*(1-ps))
```

With the transformed target, we can pick any ML regression algorithm to predict it. Lets use boosted trees here.

```
from lightgbm import LGBMRegressor

np.random.seed(123)
cate_learner = LGBMRegressor(max_depth=3, min_child_samples=300, num_leaves=5)
cate_learner.fit(train[X], y_star_train);
```

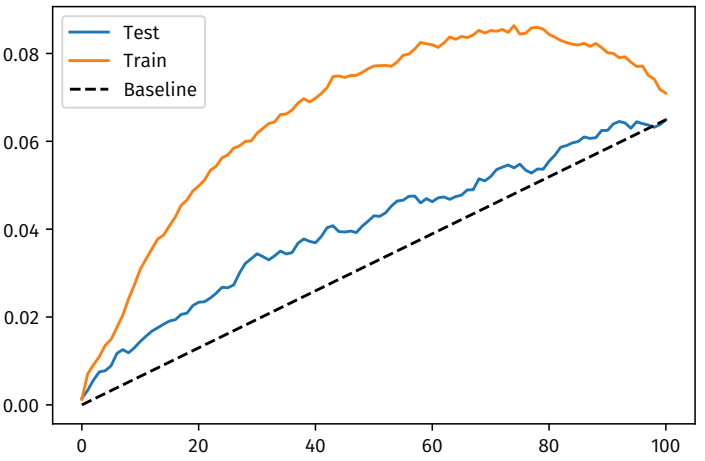
This model can now estimate $\tau(x)_i$. In other words, what it outputs is $\hat{\tau}(x)_i$. For example, if we make predictions on the test set, we will see that some units have higher CATE than others. For example, customer 6958 has a CATE of 0.1, meaning the probability he or she will buy our investment product is predicted to increase by 0.1 if we send the email to this customer. In contrast, for customer 3903, the probability of buying the product is predicted to increase just 0.04.

```
test_pred = test.assign(cate=cate_learner.predict(test[X]))
test_pred.head()
```

	age	income	insurance	invested	em1	em2	em3	converted	cate
6958	40.9	4486.14	37320.33	12559.25	0	0	1	0	0.105665
7534	42.6	6386.19	13270.47	29114.42	0	0	1	0	0.121922
2975	47.6	1900.26	25588.72	2420.39	0	0	1	0	0.034161
3903	41.0	5802.19	57087.37	20182.20	1	0	1	1	0.046805
8437	49.1	2202.96	5050.81	9245.88	1	0	1	0	-0.009099

To evaluate how good this model is, we can show the cumulative gain curves, for both training and testing sets.

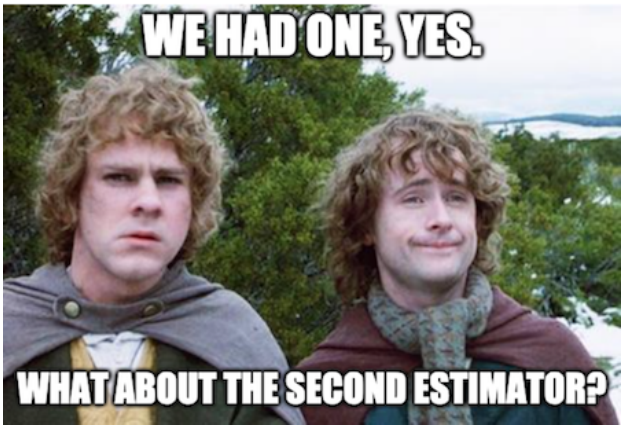
```
gain_curve_test = cumulative_gain(test_pred, "cate", y="converted", t="em1")
gain_curve_train = cumulative_gain(train.assign(cate=cate_learner.predict(train[
  ↪ X])), "cate", y="converted", t="em1")
plt.plot(gain_curve_test, color="C0", label="Test")
plt.plot(gain_curve_train, color="C1", label="Train")
plt.plot([0, 100], [0, elast(test, "converted", "em1")], linestyle="--",
  ↪ color="black", label="Baseline")
plt.legend();
```



As we can see, this plug and play estimator is better than random on the test set. Still, it looks like it is overfitting a lot, since the performance on the training set is much better than that of the test set.

That is actually one of the biggest downsides of this target transformation technique. With this target transformation, you do get a lot of simplicity, since you can just transform the target and use any ML estimator to predict heterogeneous treatment effects. The cost of it is that you get a lot of variance. That's because the transformed target is a very noisy estimate of the individual treatment effect and that variance gets transferred to your estimation. This is a huge problem if you don't have a lot of data, but it should be less of a problem in big data applications, where you are dealing with more than 1MM samples.

The Continuous Treatment Case



Another obvious downside of the target transformation method is that it only works for discrete or binary treatments. This is something you see a lot in the causal inference literature. Most of the research is done for the binary treatment case, but you don't find a lot about continuous treatments. That bothered me a lot, because in the industry, continuous treatments are everywhere, mostly in the form of prices you need to optimize. So, even though I couldn't find anything regarding target transformations for continuous treatment, I came up with something that works in practice. Just keep in mind that I don't have a super solid econometric research around it.

To motivate it, let's go back to the ice cream sales example. There, we were tasked with the problem of estimating demand elasticity to price so that we can better set the ice cream prices to optimize our revenues. Recall that the event sample in the dataset is a day and we wish to know when people are less sensitive to price increases. Also, recall that prices are randomly assigned in this dataset, which means we don't need to worry about confounding bias.

```
prices_rnd = pd.read_csv("./data/ice_cream_sales_rnd.csv")
prices_rnd.head()
```

	temp	weekday	cost	price	sales
0	25.8	1	0.3	7	230
1	22.7	3	0.5	4	190
2	33.7	7	1.0	5	237
3	23.0	4	0.5	5	193
4	24.4	1	1.0	3	252

As before, let's start by separating our data into training and a testing set.

```
np.random.seed(123)
train, test = train_test_split(prices_rnd, test_size=0.3)
train.shape, test.shape
```

```
((3500, 5), (1500, 5))
```

Now is where we need a little bit of creativity. For the discrete case, the conditional average treatment effect is given by how much the outcome changes when we go from untreated to treated, conditioned on unit characteristics X.

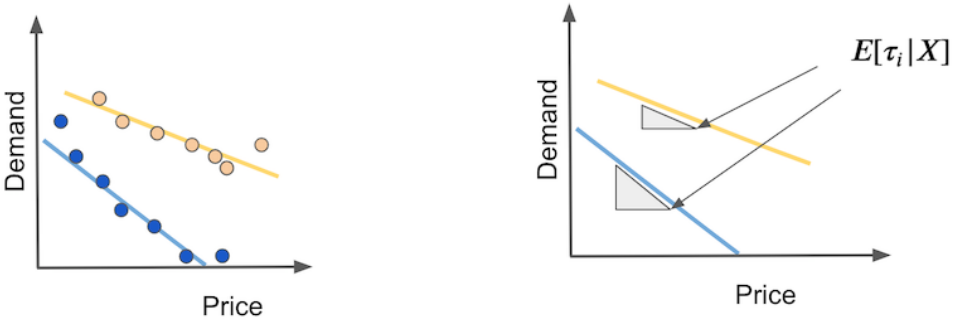
$$\tau(x) = E[Y_i(1) - Y_i(0)|X] = E[\tau_i|X]$$

In plain english, this is estimating the impact of the treatment on different unit profiles, where profiles are defined using the features X. For the continuous case, we don't have that on-off switch. Units are not treated or untreated. Rather, they are all treated, but with different intensities. Therefore, we can't talk about the effect of giving the treatment. Rather, we need to speak in terms of increasing the treatment. In other words, we wish

to know how the outcome would change if we increase the treatment by some amount. This is like estimating the partial derivative of the outcome function Y on the treatment t. And because we wish to know that for each group (the CATE, not the ATE), we condition on the features X

$$\tau(x) = E[\partial Y_i(t) \mid X] = E[\tau_i \mid X]$$

How can we estimate that? First, let's consider the easy case, where the outcome is linear on the treatment. Suppose you have two types of days: hot days (yellow) and cold days (blue). On cold days people are more sensitive to price increases. Also, as price increases, demand falls linearly.



In this case, the CATE will be the slope of each demand line. These slopes will tell us how much demand will fall if we increase price by any amount. If this relationship is indeed linear, we can estimate those elasticities with the coefficient of a simple linear regression estimate on hot days and on cold days separately.

$$\hat{\tau(x)} = Cov(Y_i, T_i)/Var(T_i) = \frac{\sum(Y_i - \bar{Y})(T_i - \bar{T})}{\sum(T_i - \bar{T})^2}$$

We can be inspired by this estimator and think about what it would be like for an individual unit. In other words, what if we have that same thing up there, defined for each day. In my head, it would be something like this:

$$Y_i^* = (Y_i - \bar{Y})\frac{(T_i - \bar{T})}{\sigma_T^2}$$

In plain English, we would transform the original target by subtracting the mean from it, then we would multiply it by the treatment, from which we've also subtracted the mean from. Finally, we would divide it by the treatment variance. Alas, we have a target transformation for the continuous case.

The question now is: does it work? As a matter of fact it does and we can go over a similar proof for why it works, just like we did in the binary case. First, lets call

$$V_i = \frac{(T_i - \bar{T})}{\sigma_T^2}$$

notice that $E[V_i|X_i = x] = 0$ because under random assignment $E[T_i|X_i = x] = \bar{T}$. In other words, for every region of X, $E[T_i] = \bar{T}$. Also $E[T_iV_i|X_i = x] = 1$ because $E[T_i(T_i - \bar{T})|X_i = x] = E[(T_i - \bar{T})^2|X_i = x]$, which is the treatment variance. Finally, under conditional independence (which we get for free in the random treatment assignment case), $E[T_ie_i|X_i = x] = E[T_i|X_i = x]E[e_i|X_i = x]$.

To show that this target transformation works, we need to remember that we are estimating the parameter for a local linear model

$$Y_i = \alpha + \beta T_i + e_i|X_i = x$$

In our example, those would be the linear models for the hot and cold days. Here, we are interested in the β parameter, which is our conditional elasticity or CATE. With all that, we can prove that

$$\begin{aligned}
E[Y_i^*|X_i = X] &= E[(Y_i - \bar{Y})V_i|X_i = x] \\
&= E[(\alpha + \beta T_i + e_i - \bar{Y})V_i|X_i = x] \\
&= \alpha E[V_i|X_i = x] + \beta E[T_iV_i|X_i = x] + E[e_iV_i|X_i = x] \\
&= \beta + E[e_iV_i|X_i = x] \\
&= \beta = \tau(x)
\end{aligned}$$

Bear in mind that this only works when the treatment is randomized. For non randomized treatment, we have to replace $\bar{T}$ by $M(X_i)$, where M is a model that estimates $E[T_i|X_i = x]$.

$$Y_i^* = (Y_i - \bar{Y})\frac{(T_i - M(T_i))}{(T_i - M(T_i))^2}$$

This will make sure that the term $\alpha E[V_i|X_i = x]$ in the third line vanishes to zero and that the term $E[T_iV_i|X_i = x]$ goes to 1. Notice that you don't actually need $E[T_iV_i|X_i = x]$ to go to 1 if you just want to order units in terms of treatment effect. In other words, if

you just want to know in which days demand is more sensitive to price increases but you don't need to know by how much, it doesn't matter if the β estimates are scaled up or down. If that is the case, you can omit the denominator.

$$Y_i^* = (Y_i - \bar{Y})(T_i - M(T_i))$$

If all that math seems tiresome, don't worry. The code is actually very simple. Once again, we transform our training target with the formulas seen above. Here, we have random treatment assignments, so we don't need to build a model that predicts prices. I'm also omitting the denominator, because here I only care about ordering the treatment effect.

```
y_star_cont = (train["price"] - train["price"].mean()) * (train["sales"] -
  ↪ train["sales"].mean())
```

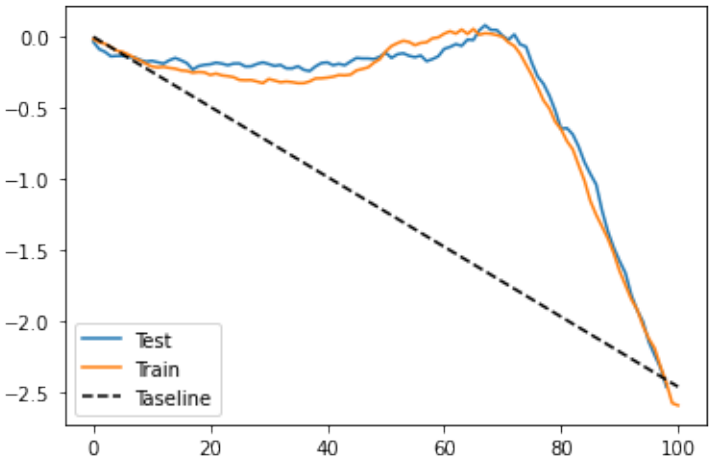
Then, just like before, we fit a regression ML model to predict that target.

```
cate_learner = LGBMRegressor(max_depth=3, min_child_samples=300, num_leaves=5)

np.random.seed(123)
cate_learner.fit(train[["temp", "weekday", "cost"]], y_star_cont)

cate_test_transf_y = cate_learner.predict(test[["temp", "weekday", "cost"]])

test_pred = test.assign(cate=cate_test_transf_y)
test_pred.sample(5)
```

	temp	weekday	cost	price	sales	cate
2815	15.7	4	1.5	3	187	-1395.956278
257	29.4	3	1.0	3	209	-1607.400415
2585	24.6	6	1.0	10	197	-1497.197402
3260	20.2	1	0.5	4	246	-1629.798111
1999	10.0	4	0.5	10	139	-1333.690544

This time, the CATE's interpretation is non intuitive. Since we've removed the denominator from the target transformation, this CATE we are seeing is scaled by $Var(X)$. However, this prediction should still order the treatment effect pretty well. To see that, we can use the cumulative gain curve, just like we did before.

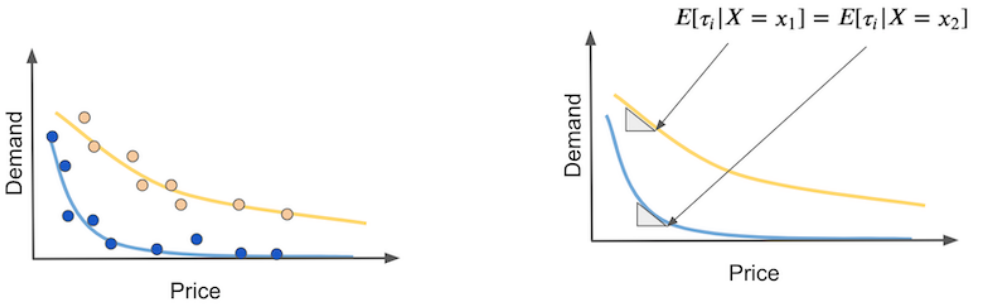
```
gain_curve_test = cumulative_gain(test.assign(cate=cate_test_transf_y),
                                  "cate", y="sales", t="price")

gain_curve_train = cumulative_gain(train.assign(cate=cate_learner.predict(train[
  ↪ ["temp", "weekday", "cost"]])),
                                   "cate", y="sales", t="price")


plt.plot(gain_curve_test, label="Test")
plt.plot(gain_curve_train, label="Train")
plt.plot([0, 100], [0, elast(test, "sales", "price")], linestyle="--",
  ↪ color="black", label="Baseline")
plt.legend();
```



For this data, it looks like the model with transformed target is way better than random. Not only that, train and test results are pretty close, so variance is not an issue here. But this is just a characteristic of this dataset. If you recall, this was not the case when we explored the binary treatment case. There, the model performed not so great.

Non Linear Treatment Effects

Having talked about the continuous case, there is still an elephant in the room we need to adress. We've assumed a linearity on the treatment effect. However, that is very rarely a reasonable assumption. Usually, treatment effects saturate in one form or another. In our example, it's reasonable to think that demand will go down faster at the first units of price increase, but then it will fall more slowly.



The problem here is that **elasticity or treatment effect changes with the treatment itself**. In our example, the treatment effect is more intense at the beginning of the curve and smaller as prices get higher. Again,suppose you have two types of days: hot days (yellow) and cold days (blue) and we want to distinguish between the two with a causal model. The thing is that causal models should predict elasticity, but in the nonlinear case, the elasticity for hot and cold days could be the same, if we look at different price points in the curve (right image).

There is no easy way out of this problem and I confess I'm still investigating what works best. For now, the thing that I do is try to think about the functional form of the treatment effect and somehow linearize it. For example, demand usually has the following functional form, where higher αs means that demand falls faster with each price increase

$$D_i = \frac{1}{P_i^\alpha}$$

So, if I apply the log transformation to both the demand Y and prices T, I should get something that is linear.

$$\begin{aligned} log(D)_i &= log\left(\frac{1}{P_i^\alpha}\right) \\ &= log(1) - log(P_i^\alpha) \\ &= log(1) - log(P_i^\alpha) \\ &= -\alpha * log(P_i) \end{aligned}$$

Linearization is not so easy to do, as it involves some thinking. But you can also try stuff out and see what works best. Often, things like logs and square roots help.

Key Ideas

We are now moving in the direction of estimating conditional average treatment effects using machine learning models. The biggest challenge when doing so is adapting a predictive model to one that estimates causal effects. Another way of thinking about it is that predictive models focus on estimating the outcome Y as a function of features X

and possibly treatment T $Y = M(X, T)$ while causal models need to estimate the partial derivative of this output function on the treatment $\partial Y = \partial M(X, T)$. This is far from trivial, because while we do observe the outcome Y, we can't observe ∂Y, at least not on an individual level. As a consequence, we need to be creative when designing an objective function for our models.

Here, we saw a very simple technique of target transformation. The idea is to combine the original target Y with the treatment T to form a transformed target which is, in expectation, equal to the CATE. With that new target, we can plug any predictive ML model to estimate it and then the model predictions will be CATE estimates. As a side note, besides target transformation, this method also goes by the name of **F-Learner**.

With all that simplicity, there is also a price to pay. The transformed target is a very noisy estimate of the individual treatment effect and that noise will be transferred to the model estimates in the form of variance. This makes target transformation better suited for big data applications, where variance is less of a problem due to sheer sample sizes. Another downside of the target transformation method is that it is only defined for binary or categorical treatments. We did our best to come up with a continuous version of the approach and even ended up with something that seemed to work, but up until now, there is no solid theoretical framework to back it up.

Finally, we've ended with a discussion on non linear treatment effects and the challenges that come with it. Namely, when the treatment effect changes with the treatment itself, we might mistakenly think units have the same treatment response curve because they have the same responsiveness to the treatment, but actually they are just receiving different treatment amounts.

References

The things I've written here are mostly stuff from my head. I've learned them through experience. This means that they have **not** passed the academic scrutiny that good science often goes through. Instead, notice how I'm talking about things that work in practice, but I don't spend too much time explaining why that is the case. It's a sort of science from the streets, if you will. However, I am putting this up for public scrutiny, so, by all means, if you find something preposterous, open an issue and I'll address it to the best of my efforts.

Most of this chapter draws from Susan Atheys' and Guido W. Imbens' paper, *Machine Learning Methods for Estimating Heterogeneous Causal Effects*. Some material about target transformation can also be found on Pierre Gutierrez' and Jean-Yves Gérardy's paper, *Causal Inference and Uplift Modeling: A review of the literature*. Note that these papers only cover the binary treatment case. Another review of causal models for CATE estimation that references the F-Learner is *Meta-learners for Estimating Heterogeneous Treatment Effects using Machine Learning*, by Künzel et al, 2019.

CHAPTER
21

Meta Learners

Just to recap, we are now interested in finding treatment effect heterogeneity, that is, identifying how units respond differently to the treatment. In this framework, we want to estimate

$$\tau(x) = E[Y_i(1) - Y_i(0) \mid X] = E[\tau_i \mid X]$$

or, $E[\delta Y_i(t) \mid X]$ in the continuous case. In other words, we want to know how sensitive the units are to the treatment. This is super useful in the case where we can't treat everyone and need to do some prioritization of the treatment, for example when you want to give discounts but have a limited budget.

Previously, we saw how we could transform the outcome variable Y so that we can plug it in a predictive model and get a Conditional Average Treatment Effect (CATE) estimate. There, we had to pay a price in terms of variance increase. That's something we see a lot in Data Science. There isn't a single best method because each one has its downsides and upsides. For that reason, it is worth learning about many techniques so you can trade-off one for the other depending on the circumstances. In that spirit, this chapter will focus on giving more tools for you to have at your disposal.

Meta learners are a simple way to leverage off-the-shelf predictive machine learning methods in order to solve the same problem we've been looking at so far: estimating the CATE. Again, none of them is the single best one and each one has its weakness. I'll try to go over them, but keep in mind that this stuff is highly dependent on the context. Not only that, meta-learners deploy predictive ML models which can vary from linear regression and boosted decision trees to neural networks and gaussian processes. The success of the meta learner will be also highly dependent on what machine learning method it uses as its components. Oftentimes you just have to try out many different things and see what works best.

```
import pandas as pd
import numpy as np
from matplotlib import pyplot as plt
import seaborn as sns
from nb21 import cumulative_gain, elast
```

Here, we will use the same data we had before, regathing investment advertisement emails. Again, the goal here is to figure out who will respond better to the email. There is a little twist, though. This time, we will use non-random data to train the models and random data to validate them. Dealing with-non random data is a much harder task, because the meta learners will need to debias the data **AND** estimate the CATE.

```
test = pd.read_csv("./data/invest_email_rnd.csv")
train = pd.read_csv("./data/invest_email_biased.csv")
train.head()
```

	age	income	insurance	invested	em1	em2	em3	converted
0	44.1	5483.80	6155.29	14294.81	0	0	1	0
1	39.8	2737.92	50069.40	7468.15	1	0	0	0
2	49.0	2712.51	5707.08	5095.65	0	0	1	1
3	39.7	2326.37	15657.97	6345.20	0	0	0	0
4	35.3	2787.26	27074.44	14114.86	1	1	0	0

Our outcome variable is conversion and our treatment is email-1. Let's create variables to store those along with the features X we will use to search for heterogeneity in the treatment effect.

```
y = "converted"
T = "em1"
X = ["age", "income", "insurance", "invested"]
```

S-Learner (aka, the Go-Horse Learner)

The first learner we will use is the S-Learner. This is the simplest learner we can think of. We will use a single (hence the S) machine learning model M_s to estimate

$$\mu(x) = E[Y \mid T, X]$$

To do so, we will simply include the treatment as a feature in the model that tries to predict the outcome Y.

NOTE

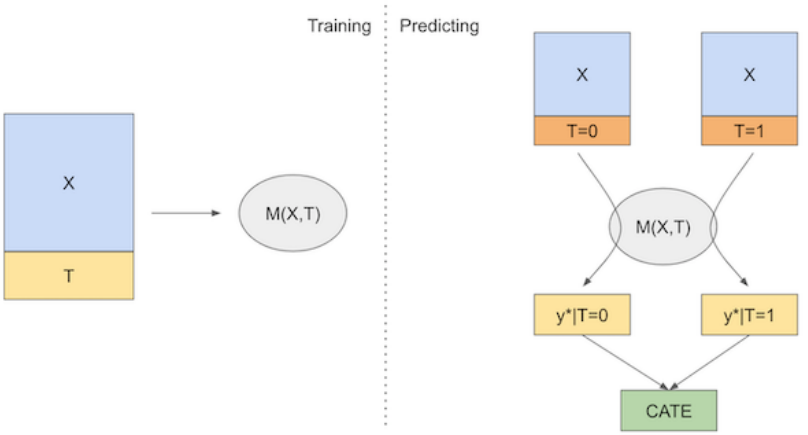
Although I'll use a regressor to estimate $E[Y \mid T, X]$, since the outcome variable is binary, you could also use a classifier. Just be sure to adapt the code so that the model outputs probabilities instead of the binary class, 0, 1.

```
from lightgbm import LGBMRegressor

np.random.seed(123)
s_learner = LGBMRegressor(max_depth=3, min_child_samples=30)
s_learner.fit(train[X+[T]], train[y]);
```

Then, we can make predictions under different treatment regimes. The difference in predictions between the test and control will be our CATE estimate

$$\hat{\tau}(x)_i = M_s(X_i, T = 1) - M_s(X_i, T = 0)$$

If we put that in a diagram, here is what it would look like



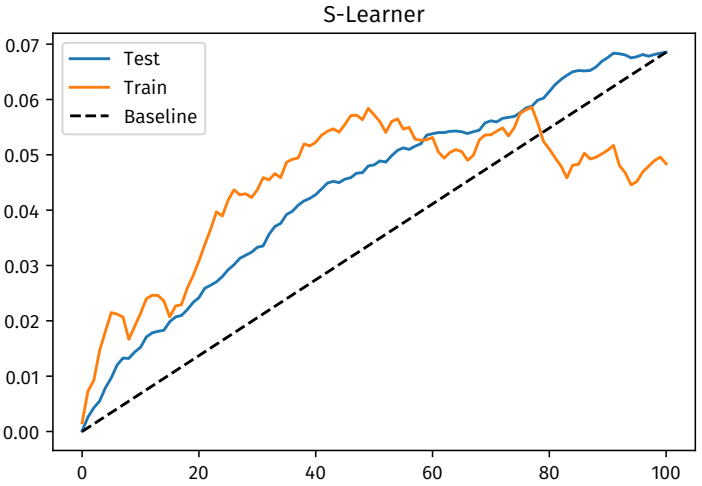
Now, let's see how we can implement this learner in code.

```
s_learner_cate_train = (s_learner.predict(train[X].assign(**{T: 1})) -
                        s_learner.predict(train[X].assign(**{T: 0})))

s_learner_cate_test = test.assign(
    cate=(s_learner.predict(test[X].assign(**{T: 1})) -
      ↪ # predict under treatment
          s_learner.predict(test[X].assign(**{T: 0}))) # predict under control
)
```

To evaluate this model, we will look at the cumulative gain curve in the test set. I'm also plotting the gain curve in the train. Since the train is biased, this curve cannot give any

indication if the model is good, but it can point us out if we are overfitting to the training set. When this happens, the curve in the train set will be super high. If you want to see what that looks like, try replacing the `max_depth` parameter from 3 to 20.

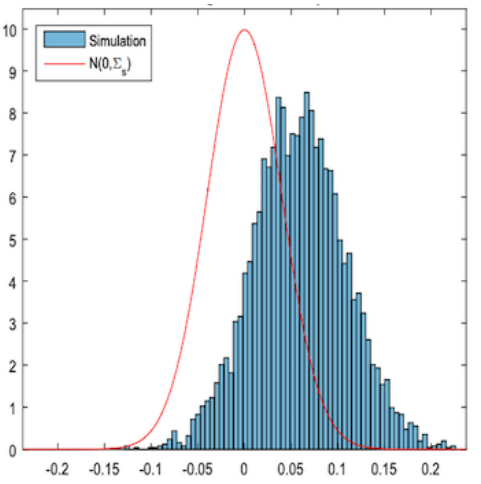
```
gain_curve_test = cumulative_gain(s_learner_cate_test, "cate", y="converted",
  ↪ t="em1")
gain_curve_train = cumulative_gain(train.assign(cate=s_learner_cate_train),
  ↪ "cate", y="converted", t="em1")
plt.plot(gain_curve_test, color="C0", label="Test")
plt.plot(gain_curve_train, color="C1", label="Train")
plt.plot([0, 100], [0, elast(test, "converted", "em1")], linestyle="--",
  ↪ color="black", label="Baseline")
plt.legend()
plt.title("S-Learner");
```



As we can see from the cumulative gain, the S-learner, although simple, can perform okay on this dataset. One thing to keep in mind is that this performance is highly particular to this dataset. Depending on the type of data that you have, the S-learner might do better or worse. In practice, I find that the S-learner is a good first bet for any causal problem, mostly due to its simplicity. Not only that, the S-learner can handle both continuous

and discrete treatments, while the rest of the learners in this chapter can only deal with discrete treatments.

The major disadvantage of the S-learner is that it tends to bias the treatment effect towards zero. Since the S-learner employs what is usually a regularized machine learning model, that regularization can restrict the estimated treatment effect. Chernozhukov et al (2016) outline this problem using simulated data:



Here, they plot the difference between the true causal effect (red outline) and the estimated causal effect, $\tau - \hat{\tau}$, using an S-learner. The estimated causal effect is heavily biased.

Even worse, if the treatment is very weak relative to the impact other covariates play in explaining the outcome, the S-learner can discard the treatment variable completely. Notice that this is highly related to the chosen ML model you employ. The greater the regularization, the greater the problem. One attempt to fix that is the next learner we will see.

T-Learner

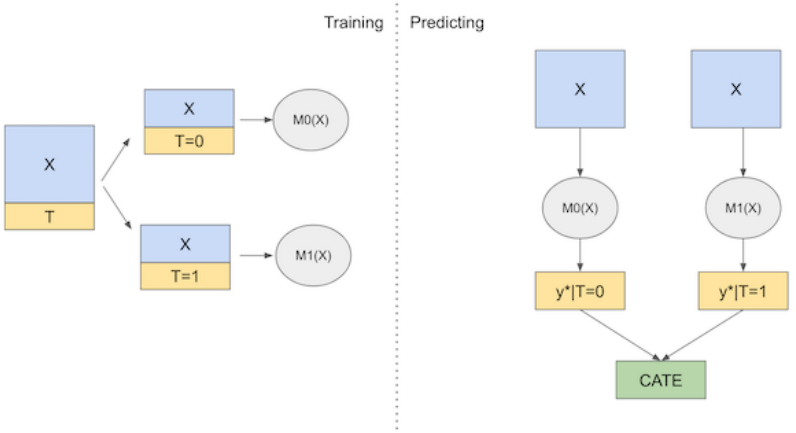
The T-learner tries to solve the problem of discarding the treatment entirely by forcing the learner to first split on it. Instead of using a single model, we will use one model per treatment variable. In the binary case, there are only two models that we need to estimate (hence the name T):

$$\mu_0(x) = E[Y \mid T = 0, X]$$
$$\mu_1(x) = E[Y \mid T = 1, X]$$

Then, at prediction time, we can make counterfactual predictions for each treatment and get the CATE as follows.

$$\hat{\tau}(x)_i = M_1(X_i) - M_0(X_i)$$

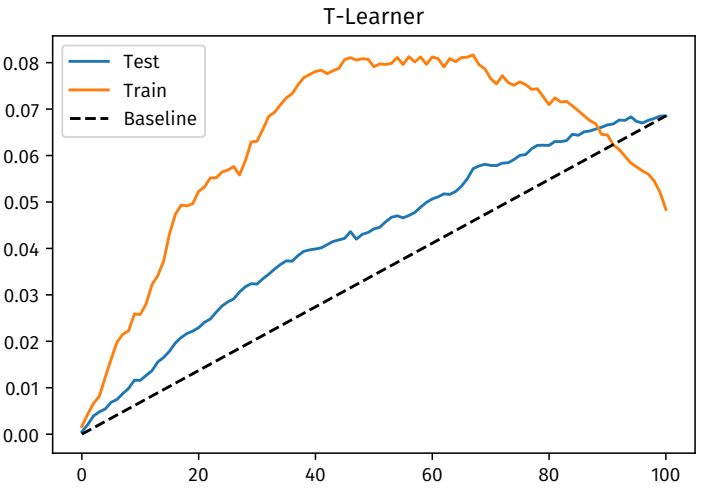
Here is the diagram of this learner



Now, enough about theory. Let's code this up.

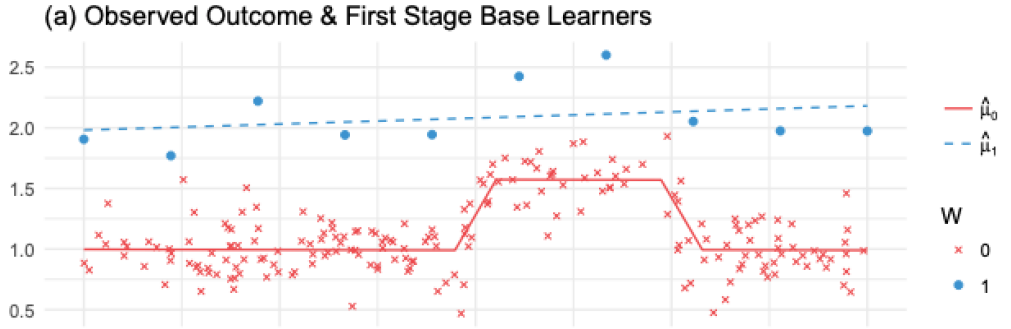
```
np.random.seed(123)

m0 = LGBMRegressor(max_depth=2, min_child_samples=60)
m1 = LGBMRegressor(max_depth=2, min_child_samples=60)

m0.fit(train.query(f"{T}==0")[X], train.query(f"{T}==0")[y])
m1.fit(train.query(f"{T}==1")[X], train.query(f"{T}==1")[y])

# estimate the CATE
t_learner_cate_train = m1.predict(train[X]) - m0.predict(train[X])
t_learner_cate_test = test.assign(cate=m1.predict(test[X]) - m0.predict(test[
  ↪ X]))
```

```
gain_curve_test = cumulative_gain(t_learner_cate_test, "cate", y="converted",
  ↪ t="em1")
gain_curve_train = cumulative_gain(train.assign(cate=t_learner_cate_train),
  ↪ "cate", y="converted", t="em1")
plt.plot(gain_curve_test, color="C0", label="Test")
plt.plot(gain_curve_train, color="C1", label="Train")
plt.plot([0, 100], [0, elast(test, "converted", "em1")], linestyle="--",
  ↪ color="black", label="Baseline")
plt.legend();
plt.title("T-Learner");
```



The T-learner also performs OK on this dataset. The test performance doesn't look much different from what we got with the S-learner. Perhaps because the treatment is not that weak. Also, we can see that the training performance is much higher than the testing performance. This indicates the model is overfitting. This can happen because we are fitting each model on only a subset of the data. With less data points, the model is probably learning some noise.

The T-Learner avoids the problem of not picking up on a weak treatment variable, but it can still suffer from regularization bias. Consider the following situation, taken from Kunzela et al, 2019. You have lots of data for the untreated and very few data for the treated, a pretty common case in many applications, as treatment is often expensive. Now suppose you have some non linearity in the outcome Y, but the **treatment effect is constant**. We can see what happens in the following image



Here, since we have very few treated observations, M_1 will be a very simple model (linear in this case) to avoid overfitting. M_0 will be more complicated, but that's OK because the abundance of data prevents overfitting. This is all reasonable from a Machine Learning perspective. However, if we use these models to compute the cate $\hat{\tau} = M_1(X) - M_0(X)$,

the linearity of $M_1(X)$ minus the non linearity of $M_0(X)$ will result in a nonlinear CATE (blue line minus red line), which is wrong, since the CATE is constant and equal to 1 in this case.

What happens here is that the model for the untreated can pick up the non linearity, but the model for the treated cannot, because we've used regularization to deal with a small sample size. Of course, you could use less regularization on that model, but then the small sample size will cause you to overfit. Seems like we are caught between a rock and a hard place. To solve this problem, we can use an X-learner, proposed in the same paper by Kunzela et al.

X-Learner

The X-learner is significantly more complex to explain than the previous learner, but its implementation is quite simple, so don't worry. The X-Learner has two stages and a propensity score model. The first one is identical to the T-learner. First, we split the samples into treated and untreated and fit a ML model for the treated and for control.

$$\hat{M}_0(X) \approx E[Y \mid T = 0, X]$$
$$\hat{M}_1(X) \approx E[Y \mid T = 1, X]$$

Now, things start to take a turn. For the second stage, we impute the treatment effect for the control and for the treated using the models above

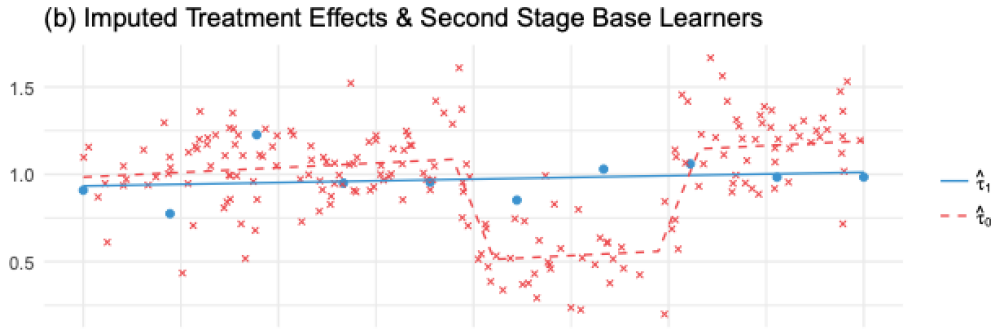
$$\hat{\tau}(X, T = 0) = \hat{M}_1(X, T = 0) - Y_{T=0}$$
$$\hat{\tau}(X, T = 1) = Y_{T=1} - \hat{M}_0(X, T = 1)$$

Then, we fit two more models to predict those effects

$$\hat{M}_{\tau 0}(X) \approx E[\hat{\tau}(X) \mid T = 0]$$
$$\hat{M}_{\tau 1}(X) \approx E[\hat{\tau}(X) \mid T = 1]$$

If we apply this on the image we've shown before, $\hat{\tau}(X, T = 0)$, the imputed treatment effect on the untreated, would be the red crosses and the red dashed line would be $\hat{M}_{\tau 0}(X)$. Notice that this model is wrong. Because $\hat{\tau}(X, T = 0)$ was made using the regularized, simple model, estimated on the treated, $\hat{M}_1$. The treatment effect it imputes is non linear, since it **doesn't** capture the non-linearity in the Y variable.

In contrast, the blue dots are the imputed treatment effect for the treated, $\hat{\tau}(X, T = 1)$. These effects are estimated using the correct model, M_0, trained in the untreated, large, sample. As a result, since its imputed treatment effects are correct, we are able to train a correct second stage model $\hat{M}_{\tau 1}(X)$, shown by the blue line.

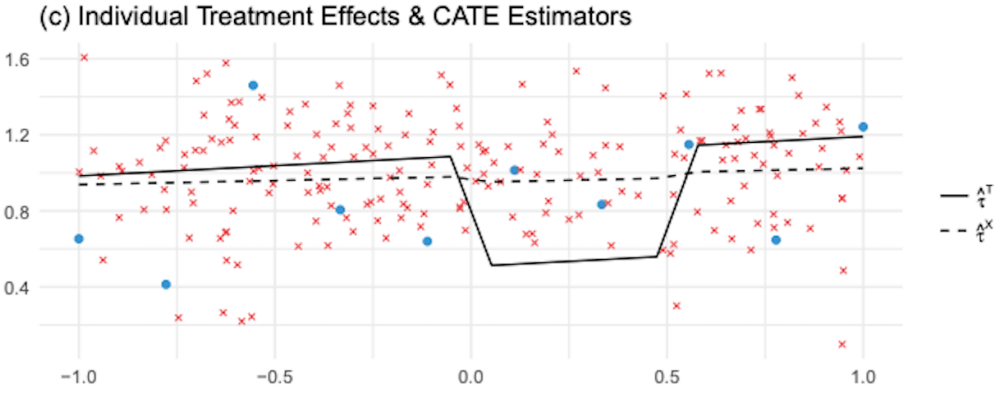


So we have one model that is wrong because we've impute the treatment effects wrongly and another model that is correct because we've imputed those values correctly. Now, we need a way to combine the two in a way that gives more weight to the correct model. Here is where the propensity score model comes to play. Let $\hat{e}(x)$ be the propensity score model, we can combine the two second stage models as follows:

$$\tau(\hat{x}) = \hat{M}_{\tau 0}(X)\hat{e}(x) + \hat{M}_{\tau 1}(X)(1 - \hat{e}(x))$$

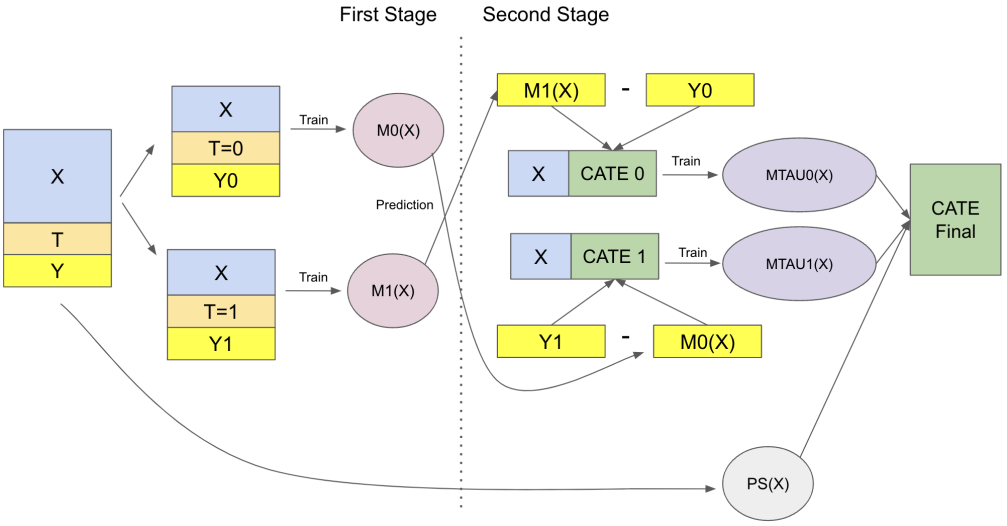
Since there are very few treated units, $\hat{e}(x)$ is very small. This will give a very small weight to the wrong model $\hat{M}_{\tau 0}(X)$.

In contrast, $1 - \hat{e}(x)$ is close to one, so we will give a high weight to the correct model $\hat{M}_{\tau 1}(X)$. More generally, weighted average using the propensity score will make sure we give more weight to the CATE model that was estimated where the assigned treatment was more likely. In other words, we will favor the model that was trained using more data. The following image shows the estimated CATE given by the X-learner and the T-learner.



As we can see, compared to the T-learner, the X-learner does a much better job in correcting the wrong CATE estimated at the non linearity. In general the X-learner performs better when a treatment group is much larger than the other.

I know this might be a mouthful, but it will hopefully be clear once we move to the implementation. Just to summarize everything, here is the diagram of this learner.



To the code at last! First, we have the first stage, which is exactly the same as the T-Learner.

```
from sklearn.linear_model import LogisticRegression

np.random.seed(123)

# first stage models
m0 = LGBMRegressor(max_depth=2, min_child_samples=30)
m1 = LGBMRegressor(max_depth=2, min_child_samples=30)

# propensity score model
g = LogisticRegression(solver="lbfgs", penalty='none')

m0.fit(train.query(f"{T}==0")[X], train.query(f"{T}==0")[y])
m1.fit(train.query(f"{T}==1")[X], train.query(f"{T}==1")[y])

g.fit(train[X], train[T]);
```

Now, we impute the treatment effect and fit the second stage models on them.

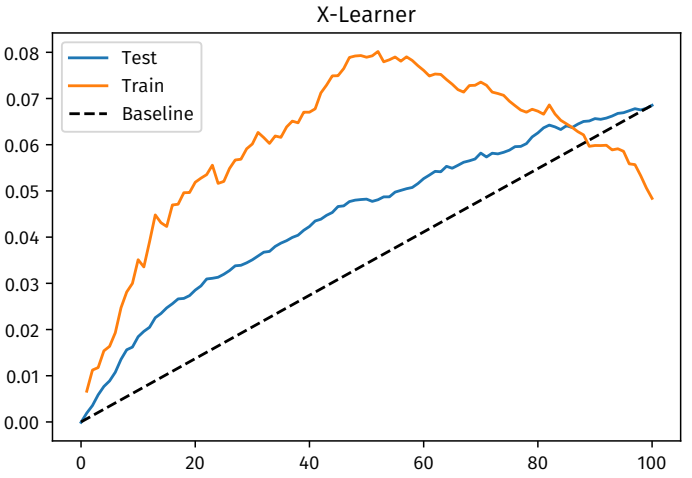
```
d_train = np.where(train[T]==0,
                   m1.predict(train[X]) - train[y],
                   train[y] - m0.predict(train[X]))

# second stage
mx0 = LGBMRegressor(max_depth=2, min_child_samples=30)
mx1 = LGBMRegressor(max_depth=2, min_child_samples=30)

mx0.fit(train.query(f"{T}==0")[X], d_train[train[T]==0])
mx1.fit(train.query(f"{T}==1")[X], d_train[train[T]==1]);
```

Finally, we make corrected predictions using the propensity score model.

```
def ps_predict(df, t):
    return g.predict_proba(df[X])[:, t]


x_cate_train = (ps_predict(train,1)*mx0.predict(train[X]) +
                ps_predict(train,0)*mx1.predict(train[X]))

x_cate_test = test.assign(cate=(ps_predict(test,1)*mx0.predict(test[X]) +
                                ps_predict(test,0)*mx1.predict(test[X])))
```

Lets see how our X-Learner does in the test. Again, let's plot the cumulative gain curve.

```
gain_curve_test = cumulative_gain(x_cate_test, "cate", y="converted", t="em1")
gain_curve_train = cumulative_gain(train.assign(cate=x_cate_train), "cate",
  ↪ y="converted", t="em1")
plt.plot(gain_curve_test, color="C0", label="Test")
plt.plot(gain_curve_train, color="C1", label="Train")
plt.plot([0, 100], [0, elast(test, "converted", "em1")], linestyle="--",
  ↪ color="black", label="Baseline")
plt.legend();
plt.title("X-Learner");
```



Once more, we have an OK performance on this dataset. Here, the S, T and X learners seem to perform pretty similarly. Still, I think it is worth knowing about all these meta learners so you can use whatever works best for you. Keep in mind that the performance is also highly dependent on the base machine learning model we choose. Here, we did everything using Gradient Boosted Trees, but maybe something else or even the same thing with different hyperparameters might work better.

Key Ideas

Again, the simplest thing we can do is using a single or S-learner with the treatment as a feature. This tends to work well when the treatment is not a weak predictor of the outcome. But if that's not the case, the S-learner tends to be biased towards zero or even drop the treatment entirely. Adding a bit more complexity, we can force the learner to pick up the treatment by using a T-learner. Here, we fit one Machine Learning model per treatment level. This works fine when there are enough samples for all treatment levels, but it can fail when one treatment level has a small sample size, forcing a model to be heavily regularized. To fix that, we can add another level of complexity using an X-learner, where we have two fitting stages and we use a propensity score model to correct potential mistakes from models estimated with very few data points.

One big problem of these learners (except the S-learner) is that they assume a binary or categorical treatment. There is one additional learner that we haven't seen yet, which is more general: the R-learner. But don't worry, we will have an entire chapter dedicated to it.

References

The things I've written here are mostly stuff from my head. I've learned them through experience. This means that they have **not** passed the academic scrutiny that good science

often goes through. Instead, notice how I'm talking about things that work in practice, but I don't spend too much time explaining why that is the case. It's a sort of science from the streets, if you will. However, I am putting this up for public scrutiny, so, by all means, if you find something preposterous, open an issue and I'll address it to the best of my efforts.

To write this chapter, I've relied on Uber's *causalml* library and their documentation on meta learners. I've also took a lot of images and concept from Kunzela et al (2019), *Metalearners for estimating heterogeneous treatment effects using machine learning*. Finally, the discussion on the S-learner being biased towards zero was taken from Chernozhukov et al (2017), *Double/Debiased Machine Learning for Treatment and Causal Parameters*.

CHAPTER
22

Debiased/Orthogonal Machine Learning

The next meta-learner we will consider actually came before they were even called meta-learners. As far as I can tell, it came from an awesome 2016 paper that sprung a fruitful field in the causal inference literature. The paper was called *Double Machine Learning for Treatment and Causal Parameters* and it took a lot of people to write it: Victor Chernozhukov, Denis Chetverikov, Mert Demirer, Esther Duflo (which, by the way, won the 2019 Economics Nobel Prize along with Abhijit Banerjee and Michael Kremer "for their experimental approach to alleviating global poverty"), Christian Hansen, Whitney Newey and James Robins. No wonder it was a very good paper and I even took the liberty of representing its authors as the Avengers just because (credits to Paul Goldsmith-Pinkham who had this idea first).

There is only one problem with the paper: it is incredibly difficult to read (which is expected, since it's an econometric paper). And since this book is all about making causal inference mainstream, here we are, trying to make Debiased/Orthogonal Machine Learning intuitive.

But what makes it so special as to deserve it's own chapter, apart from the other meta-learners? The thing that caught my attention was how well justified this Debiased/Orthogonal ML is. The other methods that we've seen so far, the T-learner, S-Learner and X-learner, all seem a bit hackish. We can give an intuitive explanation about why they work, but they didn't seem very general. In contrast, with Debiased/Orthogonal Machine Learning, we have a general framework we can apply, which is both very intuitive and also very rigorous. Another bonus is that Debiased/Orthogonal ML works for both continuous and discrete treatments, something that neither the T, not X learner could do. Not to mention that the papers describing it do an incredible job with the asymptotic analysis of this estimator. So without further ado, let's get down to it.

Once again, as a motivating example, we will resort to our ice cream sales dataset. Just as a reminder, here we are trying to find heterogeneity on the effect of price on sales. Our test set has randomly assigned prices but our training data has only observational prices, which is potentially biased.

```
import pandas as pd
import numpy as np
from matplotlib import pyplot as plt
import seaborn as sns
from nb21 import cumulative_gain, elast
import statsmodels.formula.api as smf
from matplotlib import style
style.use("ggplot")
```

```
test = pd.read_csv("./data/ice_cream_sales_rnd.csv")
train = pd.read_csv("./data/ice_cream_sales.csv")
train.head()
```

	temp	weekday	cost	price	sales
0	17.3	6	1.5	5.6	173
1	25.4	3	0.3	4.9	196
2	23.3	5	1.5	7.6	207
3	26.9	1	0.3	5.3	241
4	20.2	1	1.0	7.2	227

```
np.random.seed(123)
sns.scatterplot(data=train.sample(1000), x="price", y="sales", hue="weekday")
```

```
<AxesSubplot:xlabel='price', ylabel='sales'>
```

One source of bias is very clear. As we can see, prices are much higher on the weekend (weekdays 1 and 7), but we can also have other confounders, like temperature and cost. So, if we want to do any causal inference with it, we will need to correct this bias.

ML for Nuisance Parameters

One way we can try to remove this bias is by using a linear model to estimate the treatment effect of prices on sales while controlling for the confounders.

$$Sales_i = \alpha + \tau price_i + \beta_1 temp_i + \beta_2 cost_i + \boldsymbol{\beta}_3 Weekday_i + e_i$$

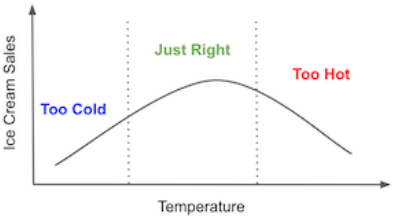
where $\boldsymbol{\beta}_3$ is a vector of parameters associated with each weekday dummy.

Notice that we are only interested in the τ parameter because that's our treatment effect. We are going to call the other parameters nuisance parameters because we don't care about them. But, as it turns out, even if we don't care about them, we have to get them right, because if we don't, our treatment effect will be off. That's sort of annoying.

For instance, if we think about it, the relationship between `temp` and sales is probably not linear. First, as temperature increases, more people will go to the beach and buy ice cream, so sales will increase. But, at some point, it becomes too hot and people decide it is best to stay home. At that point, sales will drop. The relationship between `temp` and sales probably peaks somewhere and then decreases. Which means the above model is probably wrong. It should have been something like

$$Sales_i = \alpha + \tau price_i + \beta_1 temp_i + \beta_2 temp_i^2 + \beta_3 cost_i + \boldsymbol{\beta}_4 Weekday_i + e_i$$

with a quadratic term.



Thinking about how to model nuisance parameters is already boring with just a few covariates. But what if we had tens or hundreds of them? With modern datasets, this is pretty common. So, what can we do about it? The answer lies the coolest Econometric theorem ever derived.

Frisch-Waugh-Lovell

Frisch, Waugh and Lovell were 20th century econometricians who noticed the coolest thing about linear regression. This isn't new to you, as we've talked about it in the context of regression residuals and when talking about fixed effects. But since this theorem is key to understanding Orthogonal-ML, it's very much worth recapping it.

Suppose you have a linear regression model with a set of features X_1 and another set of features X_2. You then estimate that model's parameters.

$$\hat{Y} = \hat{\beta}_1 X_1 + \hat{\beta}_2 X_2$$

where X_1 and X_2 are feature matrices (one column per feature and one row per observation) and $\hat{\beta}_1$ and $\hat{\beta}_2$ are row vectors. You can get the exact same $\hat{\beta}_1$ parameter by doing the following steps

1. regress the outcome y on the second set of features $\hat{y^*} = \hat{\gamma_1} X_2$
2. regress the first set of features on the second $\hat{X_1} = \hat{\gamma_2} X_2$
3. obtain the residuals $\tilde{X}_1 = X_1 - \hat{X_1}$ and $\tilde{y}_1 = y - \hat{y^*}$
4. regress the residuals of the outcome on the residuals of the features $\tilde{y} = \hat{\beta}_1 \tilde{X}_1$

This is insanely cool. Here, we have a generic representation, but notice that one set of features can be just the treatment variable. This means you can estimate all the nuisance parameters separately. First, regress the outcome on the features to get outcome residuals. Then, regress the treatment on the features to get treatment residuals. Finally, regress the outcome residuals on the feature residuals. This will yield the exact same estimate as if we regress the outcome on the features and treatment at the same time.

But don't take my word for it. FWL is something that everyone interested in causal inference should do at least once. In the example below, we estimate the treatment effect by first estimating the effects of the covariates on the outcome (sales) and treatment (price).

```
my = smf.ols("sales~temp+C(weekday)+cost", data=train).fit()
mt = smf.ols("price~temp+C(weekday)+cost", data=train).fit()
```

Then, with the residuals, we estimate the ATE of price on sales.

```
smf.ols("sales_res~price_res",
        data=train.assign(sales_res=my.resid, # sales residuals
                          price_res=mt.resid) # price residuals
       ).fit().summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	-4.96e-13	0.111	-4.49e-12	1.000	-0.217	0.217
price_res	-4.0004	0.110	-36.428	0.000	-4.216	-3.785

We've estimated the ATE to -4, meaning that each unit increase in price will lower sales by 4 units.

Now, let's estimate the same parameter, but this time, we will include the treatment and the covariates in the same model.

```
smf.ols("sales~price+temp+C(weekday)+cost", data=train).fit().params["price"]
```

```
-4.000429145475454
```

As you can see, they are the exact same number! This shows that estimating the treatment effect all at once or separating in the FWL steps is mathematically the same.

Another way of saying this is that the treatment effect can be derived from **regression on residuals**, where we obtain the residuals from regressing Y on X and regress it on the residuals of regressing T on X. Let's say that $\sim$ is the regression operator, so we can summarise FWL theorem as follows.

$$(Y - (Y \sim X)) \sim (T - (T \sim X))$$

which is essentially estimating the causal parameter τ in the following model

$$Y_i - E[Y_i|X_i] = \tau \cdot (T_i - E[T_i|X_i]) + \epsilon$$

As I've said, FWL is so awesome because it allows us to separate the estimation procedure of the causal parameter from that of the nuisance parameters. But we still didn't answer our initial question, which is how can we avoid all the hassle from having to specify the correct functional form on the nuisance parameters? Or, in other words, how can we focus only on the causal parameter without having to worry about the nuisance parameters? Here is where machine learning comes to play.

Frisch-Waugh-Lovell on Steroids

Double/Debiased ML can be seen as Frisch, Waugh and Lovell theorem on steroids. The idea is very simple: use ML models when constructing the outcome and treatment residuals:

$$Y_i - \hat{M}_y(X_i) = \tau \cdot (T_i - \hat{M}_t(X_i)) + \epsilon$$

Where $\hat{M}_y(X_i)$ is estimating $E[Y \mid X]$ and $\hat{M}_t(X_i)$ is estimating $E[T \mid X]$.

The idea is that ML models are super flexible, hence, they can capture interactions and non linearities when estimating the Y and T residuals while still maintaining a FWL style orthogonalisation. This means we don't have to make any parametric assumption about the relationship between the covariates X and the outcome Y nor between the covariates and the treatment in order to get the correct treatment effect. Provided we don't have unobserved confounders, we can recover the ATE with the following orthogonalisation procedure:

1. Estimate the outcome Y with features X using a flexible ML regression model M_y.
2. Estimate the treatment T with features X using a flexible ML regression model M_t.
3. Obtain the residuals $\tilde{Y} = Y - M_y(X)$ and $\tilde{T} = T - M_t(X)$
4. regress the residuals of the outcome on the residuals of the treatment $\tilde{Y} = \alpha + \tau\tilde{T}$

where τ is the causal parameter ATE, which we can estimate, for example, with OLS.

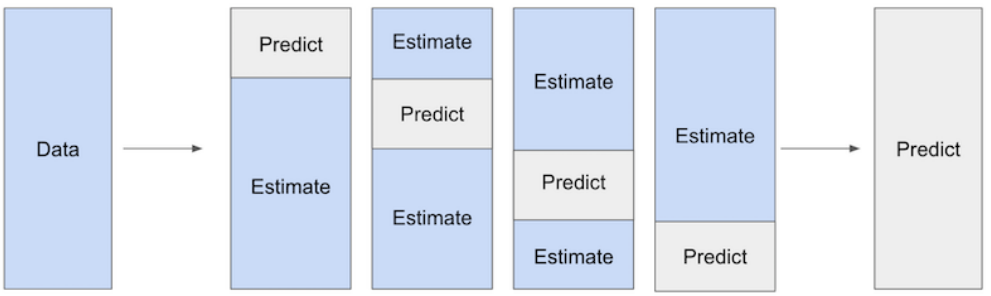
The power you gain with ML is flexibility. ML is so powerful that it can capture complicated functional forms in the nuisance relationships. But that flexibility is also troublesome, because it means we now have to take into account the possibility of overfitting.

Chernozhukov et al (2016) has a much more in depth and rigorous explanation about how overfitting can be troublesome and I definitely recommend you check it out. But here, I'll go on with a more intuition based explanation.

To see the issue, suppose that your M_y model is overfitting. The result is that the residual $\tilde{Y}$ will be smaller than it should be. It also means that M_y is capturing more than only the relationship between X and Y. Part of that something more is the relationship between T and Y, and if M_y is capturing some of that, the residual regression will be biased towards zero. In other words, M_y is capturing the causal relationship and not leaving it to the final residual regression.

Now to see the problem in overfitting M_t, notice that it will explain more of the variance in T than it should. As a result, the treatment residual will have less variance than it should. If there is less variance in the treatment, the variance of the final estimator will be high. It is as if the treatment is the same for almost everyone. And if everyone has the same treatment level, it becomes very difficult to estimate what would happen under different treatment levels. As a side note, this will also happen when T is a deterministic function of X, meaning positivity is being violated.

Those are the problems we have when using ML models, but how can we correct them? The answer lies in what we will call cross prediction and out-of-fold residuals.



We will split out data into K parts of equal size. Then, for each part k, we will estimate the ML models on all the other K-1 samples and make the residuals on the k part. Notice that these residuals are made using out-of-fold prediction. We fit the model on one part of the data, but make the predictions and residuals on another part.

so even if the model does overfit, it won't drive the residuals to zero artificially. Finally, we combine the predictions on all the K parts to estimate the final causal model $\tilde{Y} = \alpha + \tau\tilde{T}$.

OK, we've covered a lot of ground and it might be getting hard to keep up without an example. To go along with all that theory, let's go through a step by step implementation of the Double/Debiased ML. As we do so, I'll take the opportunity to explain what each step is doing.

First, let's estimate the nuisance relationship using ML models. I'll start with the treatment model M_t. We will use a LGBM model to predict prices from the covariates `temp`, `weekday` and `cost`. These predictions will be cross predictions, which we can get from using `sklearn`'s `cross_val_predict` function. I'm also adding the average $\hat{\mu}_t$ to the residuals just for visualization purposes.

```
from lightgbm import LGBMRegressor
from sklearn.model_selection import cross_val_predict

y = "sales"
T = "price"
X = ["temp", "weekday", "cost"]

debias_m = LGBMRegressor(max_depth=3)

train_pred = train.assign(price_res =  train[T] -
                          cross_val_predict(debias_m, train[X], train[T], cv=5)
                          + train[T].mean()) # add mu_t for visualization.
```

Notice that I'm calling the M_t model the debias model. That's because the role this model is playing on the Double/Debias ML is one of debiasing the treatment. The residuals $\tilde{T} = T - M_t(X)$ can be viewed as a version of the treatment where all the confounding bias from X has been removed by the model. In other words, $\tilde{T}$ is orthogonal to X. Intuitively, $\tilde{T}$ can no longer be explained by X, because it already was.

To see that, we can show the same plot we've seen earlier but now replacing price with the price residuals. Remember that, before, weekends had higher prices? Now, that bias is gone. All the weekdays have the same price residual distribution.

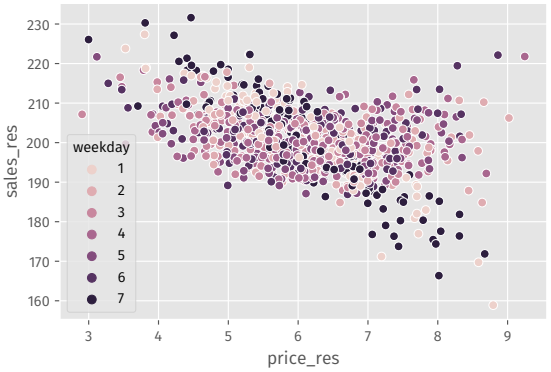
```
np.random.seed(123)
sns.scatterplot(data=train_pred.sample(1000), x="price_res", y="sales",
  ↪ hue="weekday");
```

The role of M_t is to debias the treatment, but what about M_y? Its role is to remove the variance from Y. Hence, I'll call it the denoising model. Intuitively, M_y is creating a version of the outcome where all the variance due to X has been explained away. As a result, it becomes easier to do causal estimation in $\tilde{Y}$. Since it has less noise, the causal relationship becomes easier to see.

```
denoise_m = LGBMRegressor(max_depth=3)

train_pred = train_pred.assign(sales_res =  train[y] -
                               cross_val_predict(denoise_m, train[X], train[y],
                                 ↪ cv=5)
                               + train[y].mean())
```

If we plot the same graph as before, but now replacing sales with sales residual, we can see that the variance in Y is much smaller than it was before.

```
np.random.seed(123)
sns.scatterplot(data=train_pred.sample(1000), x="price_res", y="sales_res",
  ↪ hue="weekday");
```



It is now easy to see the negative relationship between prices and sales.

Finally, to estimate that causal relationship, we can run a regression on the residuals.

```
final_model = smf.ols(formula='sales_res ~ price_res', data=train_pred).fit()
final_model.summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	224.5586	0.443	506.469	0.000	223.689	225.428
price_res	-3.9228	0.071	-54.962	0.000	-4.063	-3.783

As we can see, when we use the residualized or orthogonalised version of sales and price, we can be very confident that the relationship between prices and sales is negative, which makes a lot of sense. As we increase prices, demand for ice cream should fall.

But if we look at the un-residualized or raw relationship between prices and sales, because of bias, we find a positive relationship. That is because, in anticipation to high sales, prices are increased.

```
final_model = smf.ols(formula='sales ~ price', data=train_pred).fit()
final_model.summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	192.9679	1.013	190.414	0.000	190.981	194.954
price	1.2294	0.162	7.575	0.000	0.911	1.547

CATE Estimation with Double-ML

So far, we've seen how Double/Debiased ML allow us to focus on estimating the Average Treatment Effect (ATE), but it can also be used to estimate treatment effect heterogeneity or Conditional Average Treatment Effect (CATE). Essentially, we are now saying that the causal parameter τ changes depending on the unit's covariates.

$$Y_i - M_y(X_i) = \tau(X_i) \cdot (T_i - M_t(X_i)) + \epsilon_i$$

To estimate this model, we will use the same residualised version of price and sales, but now we will interact the price residuals with the other covariates. Then, we can fit a linear CATE model.

$$\tilde{Y}_i = \alpha + \beta_1 \tilde{T}_i + \boldsymbol{\beta}_2 \boldsymbol{X}_i \tilde{T}_i + \epsilon_i$$

Once we've estimated such a model, to make CATE predictions, we will use the randomised test set. Since this final model is linear, we can compute the CATE mechanically:

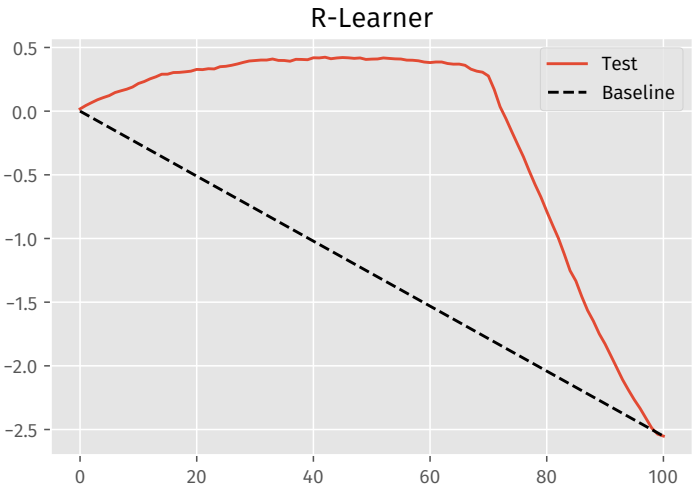
$$\hat{\mu}(\partial Sales_i, X_i) = M(Price = 1, X_i) - M(Price = 0, X_i)$$

where M is our final linear model

```
final_model_cate = smf.ols(formula='sales_res ~ price_res * (temp + C(weekday) +
    ↪ cost)', data=train_pred).fit()

cate_test = test.assign(cate=final_model_cate.predict(test.assign(price_res=1))
                        - final_model_cate.predict(test.assign(price_res=0)))
```

To check how good this model is in terms of differentiating units with high price sensitivity from those with low price sensitivity, we will use the cumulative elasticity curve.

```
gain_curve_test = cumulative_gain(cate_test, "cate", y=y, t=T)
plt.plot(gain_curve_test, color="C0", label="Test")
plt.plot([0, 100], [0, elast(test, y, T)], linestyle="--", color="black",
  ↪ label="Baseline")
plt.legend();
plt.title("R-Learner");
```



The Double/Debiased ML procedure with a final linear model is already very good, as we can see by the curve above. But, maybe we can do even better. As a matter of fact, this is a very general procedure that we can understand just like a meta-learner. Nie and

Wager called it the R-Learner, as a way to recognise the work of Robinson (1988) and to emphasize the role of residualization.

This generalization comes from realizing that the Double/Debiased ML procedure defines a new loss function that we can minimize however we want. Next, we will see how to do that in a very similar fashion to what we've seen before when discussing target transformation method or F-learner.

Non Parametric Double/Debiased ML

The nice thing about Double-ML is that it frees us from all the hassle of learning the nuisance parameters in a causal model. With that, we can focus all our attention to learning the causal parameter of interest, be it the ATE or the CATE. However, with the above specification, we were still using a linear model after the ML residualization, as the final causal model. In our example, this means we are assuming that price impacts sales linearly. That's probably OK for a small range of prices, but we know from microeconomic theory that that's not necessarily the case. It could be that, at low prices, a unit increase in price will lower demand by 2 units. But then, at higher prices, a unit increase in price will lower demand by just 1 unit. That's not a linear relationship.

We could leverage microeconomic theory here to speculate about the functional form of the outcome on the treatment, but maybe we can also delegate that to a ML model. In other words, let the machine learn that complicated function form. As it turns out, that's totally possible if we make a few changes to our original Double/Debiased ML algorithm.

First, we start exactly as before, orthogonalizing the treatment and the outcome with cross predictions from a ML model.

```
y = "sales"
T = "price"
X = ["temp", "weekday", "cost"]

debias_m = LGBMRegressor(max_depth=3)
denoise_m = LGBMRegressor(max_depth=3)

train_pred = train.assign(price_res =  train[T] - cross_val_predict(debias_m,
  ↪ train[X], train[T], cv=5),
                          sales_res =  train[y] - cross_val_predict(denoise_m,
                            ↪ train[X], train[y], cv=5))
```

So far, nothing different. Now is where things get interesting. Recall that Double/Debiased-ML models the data as follows

$$Y_i = \hat{M}_y(X_i) + \tau(X_i)\big(T_i - \hat{M}_t(X)\big) + \hat{\epsilon}_i$$

Where $\hat{M}_y$ and $\hat{M}_t$ are models that, respectively, predicts the outcome and treatment from the features. If we rearrange the terms above, we can isolate the error term

$$\hat{\epsilon}_i = \big(Y_i - \hat{M}_y(X_i)\big) - \tau(X_i)\big(T_i - \hat{M}_t(X)\big)$$

This is nothing short of awesome, because now we can call this a **causal loss function**. Which means that, if we minimize the square of this loss, we will be estimating expected value of $\tau(X_i)$, which is the CATE.

$$\hat{L}_n(\tau(x)) = \frac{1}{n}\sum_{i=1}^{n}\Big(\big(Y_i - \hat{M}_y(X_i)\big) - \tau(X_i)\big(T_i - \hat{M}_t(X)\big)\Big)^2$$

This loss is also called the **R-Loss**, since it's what the R-learner minimizes. Ok, but how do we minimize this loss function? There are multiple ways, actually, but here we will cover the simplest one. First, to declutter the technical notation, let's rewrite the loss function we had before, but using the residualized version of treatment and outcome.

$$\hat{L}_n(\tau(x)) = \frac{1}{n}\sum_{i=1}^{n}\Big(\tilde{Y}_i - \tau(X_i)\tilde{T}_i\Big)^2$$

Finally, we can do some algebraic parkour to take $\tilde{T}_i$ out of the parenthesis and isolate $\tau(X_i)$ in the square part of the loss function.

$$\hat{L}_n(\tau(x)) = \frac{1}{n}\sum_{i=1}^{n}\tilde{T}_i^2\left(\frac{\tilde{Y}_i}{\tilde{T}_i} - \tau(X_i)\right)^2$$

Minimising the above loss is equivalent to minimising what is inside the parenthesis, but weighting each term by $\tilde{T}_i^2$. Minimising what's inside the parenthesis is equivalent to predicting $\frac{\tilde{Y}_i}{\tilde{T}_i}$. This is what's called the weight trick to get the non-parametric causal loss. Notice how similar this is to the target transformation idea we saw earlier. This is, indeed, a target transformation, but with the extra weighting trick.

To summarise before we go to the code, now that we have the nuisance models and residualised versions of treatment and outcome, we will

1. Create weights $\tilde{T}_i^2$
2. Create a target $\frac{\tilde{Y}_i}{\tilde{T}_i}$
3. Use any prediction method to predict the target (2) while using the weights (1).

And here is the code. As you will see, it is incredibly simple.

```
model_final = LGBMRegressor(max_depth=3)

# create the weights
w = train_pred["price_res"] ** 2

# create the transformed target
y_star = (train_pred["sales_res"] / train_pred["price_res"])

# use a weighted regression ML model to predict the target with the weights.
model_final.fit(X=train[X], y=y_star, sample_weight=w);
```

The above ML model, even though it is an off-the-shelf predictive model, is estimating the CATE. That's the power of the non-parametric Double-ML. Before, we were using a

linear regression as the final model for the CATE estimation. Now, since we defined a generic loss, we can use any predictive model at our disposal as the final model.

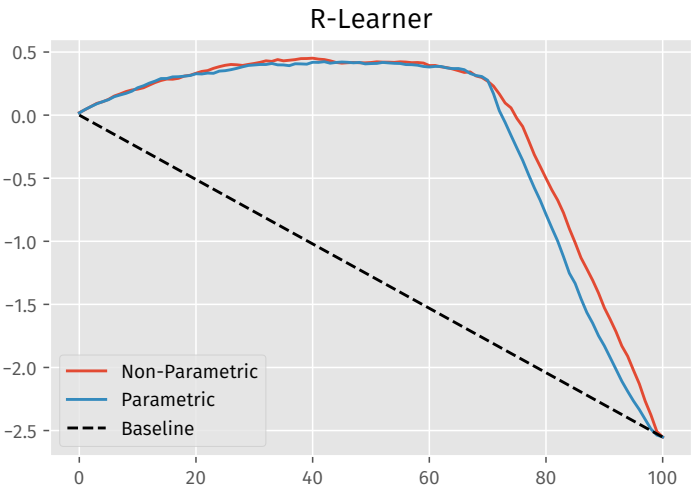
Let's now use the test set to compare this non-parametric version with the linear version we had before.

First, we estimate the individual treatment effect.

```
cate_test_non_param = test.assign(cate=model_final.predict(test[X]))
```

Next, we can plot the non-parametric cumulative elasticity curve side by side with the one we got from the parametric (linear) version of Double/Orthogonal-ML.

```
gain_curve_test_non_param = cumulative_gain(cate_test_non_param, "cate", y=y,
  ↪ t=T)
plt.plot(gain_curve_test_non_param, color="C0", label="Non-Parametric")
plt.plot(gain_curve_test, color="C1", label="Parametric")
plt.plot([0, 100], [0, elast(test, y, T)], linestyle="--", color="black",
  ↪ label="Baseline")
plt.legend();
plt.title("R-Learner");
```



Not a huge improvement here, but it's something. Plus, not having to specify the functional form of the treatment function is already a huge benefit.

What is Non-Parametric About?

Before we move on, I just wanted to highlight a common misconception. When we think about using a non-parametric Double-ML model to estimate the CATE, it looks like we will get a nonlinear treatment effect. For instance, let's assume a very simple data generating process (DGP) where discont affects sales non-linearly, but through a square

root function.

$$Sales_i = 20 + 10 * \sqrt{Discount_i} + e_i$$

The treatment effect is given by the derivative of this Sales function with respect to the treatment.

$$\frac{\partial Sales_i}{\partial Discount_i} = \frac{10}{2\sqrt{Discount_i}}$$

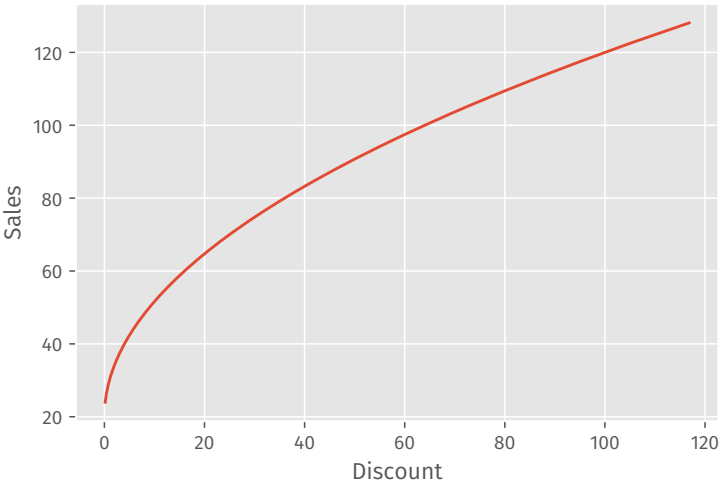
As we can see, the treatment effect is **not** linear. It actually gets weaker as the treatment increases. This makes a lot of sense for this DGP. At first, a little bit of discount increases sales by a lot. But, as we give too much discount, an extra unit of discount will affect sales less and less, because people won't want to buy to infinity. Hence, the discount is only effective up until they point they get satiated.

The question then is, can the non-parametric ML capture this saturating behavior in the treatment effect? Can it extrapolate from a small discount level that, if the discount were higher, the treatment effect would be lower? The answer is… sort of. To better understand that, let's generate data like in the above DGP.

```
np.random.seed(321)
n=5000
discount = np.random.gamma(2,10, n).reshape(-1,1)
discount.sort(axis=0) # for better ploting
sales = np.random.normal(20+10*np.sqrt(discount), 1)
```

If we plot this DGP, we can see the square root relationship between these variables.

```
plt.plot(discount, 20 + 10*np.sqrt(discount))
plt.ylabel("Sales")
plt.xlabel("Discount");
```



Now, let's apply the Non-Parametric Double/Debias ML to this data.

```
debias_m = LGBMRegressor(max_depth=3)
denoise_m = LGBMRegressor(max_depth=3)

# orthogonalising step
discount_res =  discount.ravel() - cross_val_predict(debias_m,
  ↪ np.ones(discount.shape), discount.ravel(), cv=5)
sales_res =  sales.ravel() - cross_val_predict(denoise_m, np.ones(sales.shape),
  ↪ sales.ravel(), cv=5)

# final, non parametric causal model
non_param = LGBMRegressor(max_depth=3)
w = discount_res ** 2
y_star = sales_res / discount_res

non_param.fit(X=discount_res.reshape(-1,1), y=y_star.ravel(),
  ↪ sample_weight=w.ravel());
```

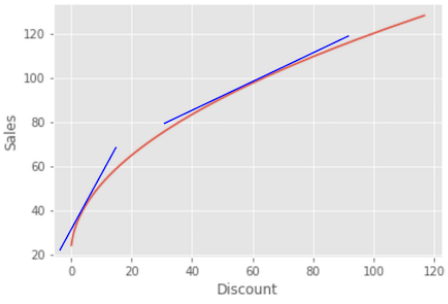
With the above model, we can get the CATE estimate. The issue here is that the CATE is not linear. As the treatment increases, the CATE should decrease. The question we are trying to answer is if the non-parametric model can capture that non linearity.

To answer that properly, let's remember what is the underlying assumption that the Double/Debiased ML makes about the data generating process. These assumptions can be seen in the equation we've laid down before.

$$\tilde{Y}_i = \tau(X_i)\tilde{T}_i + e_i$$

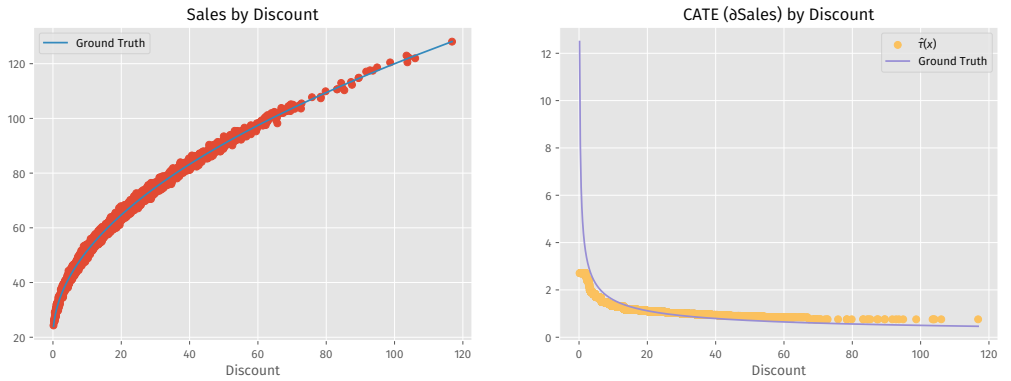
In words, it says that the residualized outcome is equal to the residualized treatment multiplied by the conditional treatment effect. This mean that the **treatment impacts the outcome linearly**. There is no non-linearity here. The above model says that the outcome will increase by a fixed amount $\tau(X_i)$ if we increase the treatment from 1 to 10 or from 100 to 110. It's a simple multiplication.

So, does this mean that the non-parametric model can't capture the non-linearity of the treatment effect? Again, not really… Rather, what is happening is that Double/ML **finds the locally linear approximation to the non-linear CATE**. In other words, it finds the derivative of the outcome with respect to the treatment at that treatment level or around the treatment. This is equivalent to finding the slopes of the lines that are tangential to the outcome function at the treatment point.



This mean that, yes, Non-Parametric Double-ML will figure out that the treatment effect will be smaller as we increase the treatment. But, no, it won't find the non-linear treatment effect, but rather the local linear treatment effect. We can even plot those linear approximations against the ground true non-linear causal effect and indeed, they are good approximations.

```
cate = non_param.predict(X=discount)

plt.figure(figsize=(15,5))
plt.subplot(1,2,1)
plt.scatter(discount, sales)
plt.plot(discount, 20 + 10*np.sqrt(discount), label="Ground Truth", c="C1")
plt.title("Sales by Discount")
plt.xlabel("Discount")
plt.legend()

plt.subplot(1,2,2)
plt.scatter(discount, cate, label="$\hat{\\tau}(x)$", c="C4")
plt.plot(discount, 5/np.sqrt(discount), label="Ground Truth", c="C2")
plt.title("CATE ($\partial$Sales) by Discount")
plt.xlabel("Discount")
plt.legend();
```



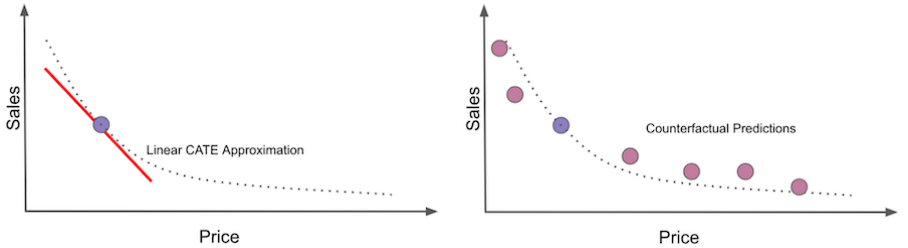
This might sound like technicalities, but it has very practical consequences. For example, let's say you find a treatment effect of 2 for a customer in the example above, meaning that if you increase the discount by 1 unit, your sales to that customer will increase by 2 units. You might look at that and think: "Great! I'll give a lot of discounts to this unit! After all, for every 1 unit in discount, I'll get 2 in sales". However, that's the wrong conclusion. The treatment effect is 2 only at that discount level. As soon as you increase the discount, the effect will fall. For example, say this hypothetical customer got only 5 in discount, which is why her treatment effect is so high. Say you see that huge treatment effect and use it to justify giving 20 in discount to that customer. But as you do so, the effect might go from 2 to something like 0.5. And a 20 discount that made sense at a treatment effect of 2 might no longer be profitable at a treatment effect of 0.5.

This means you have to be extra careful when extrapolating a nonlinear treatment effect to a new treatment level. If you are not, you might end up making very unprofitable decisions. Another way to put is is that, when treatment effect is not linear, even non-parametric Double/Debiased-ML will **struggle to make counterfactuals outcome predictions**. It will try to linearly extrapolate the treatment effect (TE) from a low treatment level to a high treatment level or the other way around. And, due to the non linearity, that extrapolation will likely be off.

To solve that, there is a final idea. Keep in mind that this idea is much less scientific than the things we've seen before. It boils down to using a S-learner after applying the orthogonalization procedure, but I'm getting ahead of myself. Let's look at that next.

Non-Scientific Double/Debiased ML

The final idea we will try is a fundamental shift in mentality. We will no longer try to estimate the linear approximation to the CATE. Instead, we will make counterfactual predictions.



The CATE is the slope of the outcome function at the data point. It is how much we expect the outcome to change if we increase the treatment by a very small amount. More technically, it's the derivative at the point. Counterfactual predictions, on the other hand, are an attempt to recreate the entire outcome curve from a single datapoint. We will predict what the outcome would be if the treatment were at some other level than the one it currently takes, hence the counterfactual.

If we manage to do so, we will be able to simulate different treatments for a unit and predict how it would respond under those different treatment levels. This is very risky business, because we will be extrapolating an entire curve from a single point. Also, although I've used this technique in practice a lot, I've never found any scientific article showing how or why it works. That's why I call it the Non-Scientific Double-ML. Simply put: beware!

Here is how this will go down. First, let's start with the traditional Double/Debiased-ML formulation, where we have the residualized version of the treatment and outcome.

$$\tilde{Y}_i = \tau(X_i)\tilde{T}_i + e_i$$

Now, I'll move the treatment inside the treatment effect function. This allows the treatment effect to be non linear, that is to change with the treatment itself.

$$\tilde{Y}_i = \tau(X_i, \tilde{T}_i) + e_i$$

This is dangerous business, because I have no idea how this treatment functions. For all we know, it could be some weird non-linear function. But, fortunately, we know how to estimate weird functions with Machine Learning. So, that's what we will do. Simply speaking, we will fit a ML model to predict the residualised outcome $\tilde{Y}$ from the residualized treatment $\tilde{T}$ together with the features X. The residualisation is important to remove bias and noise so that this final ML can focus on learning only the treatment effect and how the covariates X impact that treatment effect.

Then, once we have this model, we will make 2 step counterfactual predictions. First we will have to make a prediction for the treatment in order to get $\tilde{T}$, then, we will feed that prediction, along with the features, in our final model $\hat{\tau}(X_i, \tilde{T}_i)$.

Since we will have to make $\tilde{T}$, we first need to implement our own version of the `cross_prediction` function. This function will return not only the cross prediction, but also the models used to make those predictions.

```
from sklearn.model_selection import KFold

def cv_estimate(train_data, n_splits, model, model_params, X, y):
    cv = KFold(n_splits=n_splits)
    models = []
    cv_pred = pd.Series(np.nan, index=train_data.index)
    for train, test in cv.split(train_data):
        m = model(**model_params)
        m.fit(train_data[X].iloc[train], train_data[y].iloc[train])
        cv_pred.iloc[test] = m.predict(train_data[X].iloc[test])
        models += [m]

    return cv_pred, models
```

Now that we have our own cross prediction function that also gives us the models, we can proceed with the orthogonalisation step.

```
y = "sales"
T = "price"
X = ["temp", "weekday", "cost"]

debias_m = LGBMRegressor(max_depth=3)
denoise_m = LGBMRegressor(max_depth=3)

y_hat, models_y = cv_estimate(train, 5, LGBMRegressor, dict(max_depth=3), X, y)
t_hat, models_t = cv_estimate(train, 5, LGBMRegressor, dict(max_depth=3), X, T)

y_res = train[y] - y_hat
t_res = train[T] - t_hat
```

After the orthogonalisation, we will throw $\tilde{T}$ along with X to a ML model that tries to predict $\tilde{Y}$. I'm using a LGBM model here, but you can pick any ML model. One cool thing about LGBM is that I can set monotonic constraints to it. Knowing what we know about prices, sales should decrease as price increases. We can take that into account and constrain our LGBM model **to not increase** it's predictions as prices increase.

```
# -1 on price saying that the predictions should not increase as price increases
monotone_constraints = [-1 if col == T else 0 for col in X+[T]]

model_final = LGBMRegressor(max_depth=3,
  ↪ monotone_constraints=monotone_constraints)
model_final = model_final.fit(X=train[X].assign(**{T: t_res}), y=y_res)
```

Now is where things get a little weird. If we think about it, this final ML model is estimating the following τ treatment function

$$\tilde{Y}_i = \tau(X_i, \tilde{T}_i) + e_i$$

but there isn't a clear way to extract the treatment effect from this function. So, rather than extracting a treatment effect, we will input the counterfactual predictions, just like I've shown in the previous image. We will simulate different price levels for each unit

and use our Double-ML model to predict what would be the sales we would see under those different price levels.

To achieve that, we will 1) cross join the test set with a price table that contains all simulated prices. The end result will be as follows

```
pred_test = (test
             .rename(columns={"price":"factual_price"})
             .assign(jk = 1)
             .reset_index() # create day ID
             .merge(pd.DataFrame(dict(jk=1, price=np.linspace(3, 10, 9))),
               ↪ on="jk")
             .drop(columns=["jk"]))

pred_test.query("index==0")
```

	index	temp	weekday	cost	factual_price	sales	price
0	0	25.8	1	0.3	7	230	3.000
1	0	25.8	1	0.3	7	230	3.875
2	0	25.8	1	0.3	7	230	4.750
3	0	25.8	1	0.3	7	230	5.625
4	0	25.8	1	0.3	7	230	6.500
5	0	25.8	1	0.3	7	230	7.375
6	0	25.8	1	0.3	7	230	8.250
7	0	25.8	1	0.3	7	230	9.125
8	0	25.8	1	0.3	7	230	10.000

Notice that we are showing only the day in index 1, so only a single unit. On that day (unit), the actual or factual price or treatment was 7. But we've simulated different counterfactual treatments, from 3 to 10. Now, we will feed all those counterfactual prices to our causal model, which will make counterfactual sales predictions based on those simulated prices.

Since our model has the following format

$$\widehat{Price_i} = \hat{\tau}(X_i, \tilde{T}_i)$$

Before making the counterfactual predictions, we need to get $\tilde{T}_i$, that is, the price residuals. We will get those residuals by first, making predictions will all our treatment models (remember that we've used a 5 fold cross prediction in the training step), then we will average the predictions from the five models into a single prediction and finally subtract the counterfactual price we've generated earlier from the predicted price using this ensemble of models.

```
def ensamble_pred(df, models, X):
    return np.mean([m.predict(df[X]) for m in models], axis=0)

t_res_test = pred_test[T] - ensamble_pred(pred_test, models_t, X)

pred_test[f"{y}_pred"] = model_final.predict(X=pred_test[X].assign(**{T:
  ↪ t_res_test}))

pred_test.query("index==0")
```

	index	temp	weekday	cost	factual_price	sales	price	sales_pred
0	0	25.8	1	0.3	7	230	3.000	24.302849
1	0	25.8	1	0.3	7	230	3.875	20.862833
2	0	25.8	1	0.3	7	230	4.750	16.093630
3	0	25.8	1	0.3	7	230	5.625	6.274595
4	0	25.8	1	0.3	7	230	6.500	-1.624625
5	0	25.8	1	0.3	7	230	7.375	-10.939867
6	0	25.8	1	0.3	7	230	8.250	-21.655858
7	0	25.8	1	0.3	7	230	9.125	-25.319174
8	0	25.8	1	0.3	7	230	10.000	-25.319174

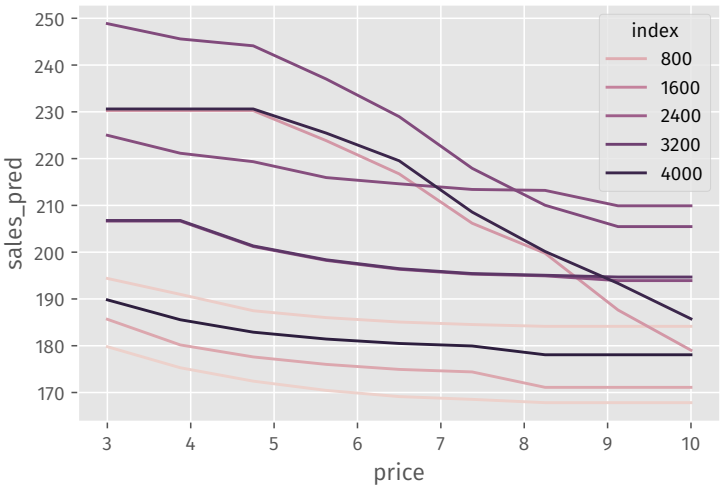
As you can see, we now have a sales prediction for every simulated price. The lower the price, the higher the sales. One interesting thing is that these predictions are off in their level. For instance, they go from about 24 to about -24. That's because the model is predicting the residualized outcome, which is roughly mean zero. This is fine if all you want is to get the slope of the sales curve, which is the price treatment effect. Also, if you want to fix the prediction levels, all you have to do is add the predictions from the denoising model M_y.

```
y_hat_test = ensamble_pred(pred_test, models_y, X)
pred_test[f"{y}_pred"] = (y_hat_test +
                          model_final.predict(X=pred_test[X].assign(**{T:
                            ↪ t_res_test})))

pred_test.query("index==0")
```

	index	temp	weekday	cost	factual_price	sales	price	sales_pred
0	0	25.8	1	0.3	7	230	3.000	254.208352
1	0	25.8	1	0.3	7	230	3.875	250.768336
2	0	25.8	1	0.3	7	230	4.750	245.999133
3	0	25.8	1	0.3	7	230	5.625	236.180099
4	0	25.8	1	0.3	7	230	6.500	228.280878
5	0	25.8	1	0.3	7	230	7.375	218.965636
6	0	25.8	1	0.3	7	230	8.250	208.249645
7	0	25.8	1	0.3	7	230	9.125	204.586329
8	0	25.8	1	0.3	7	230	10.000	204.586329

We can also plot the unit level sales curve. Let's sample ten units and see how they would behave under different prices.

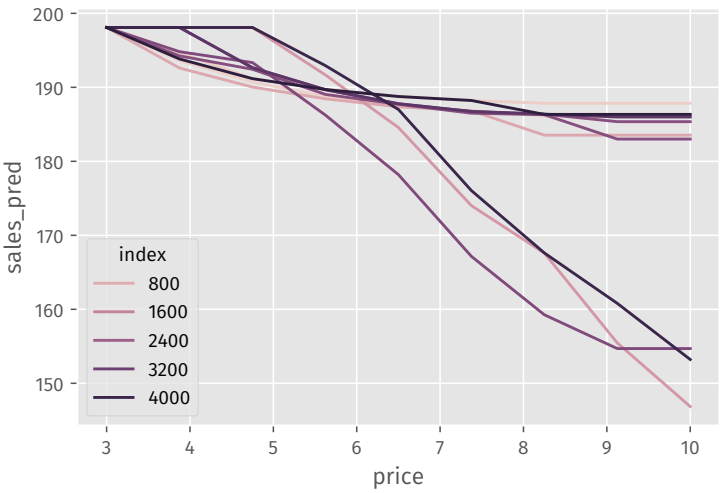
```
np.random.seed(1)
sample_ids = np.random.choice(pred_test["index"].unique(), 10)

sns.lineplot(data=pred_test.query("index in @sample_ids"),
             x="price", y="sales_pred", hue="index");
```



It is interesting to see that some units are very sensitive to price increases. In some cases, we expect sales to drop from 250 to almost 200 as we increase the price from 3 to 10. On the other hand, some units are very inelastic to price: as we increase prices from 3 to 10, we expect the sales to go from about 195 to about 185.

It's hard to see these differences in price sensitivity, so what I like to do is to make all the curves start from the same point (the average sales here). This will make it easier to see that some units have a sharp drop in sales as we increase prices, while others not so much.

```
np.random.seed(1)
sample_ids = np.random.choice(pred_test["index"].unique(), 10)

sns.lineplot(data=(pred_test
                   .query("index in @sample_ids")
                   .assign(max_sales = lambda d: d.groupby("index")[
                    ↪ ["sales_pred"]].transform("max"))
                   .assign(sales_pred = lambda d: d["sales_pred"] -
                    ↪ d["max_sales"] + d["sales_pred"].mean())),
             x="price", y="sales_pred", hue="index");
```



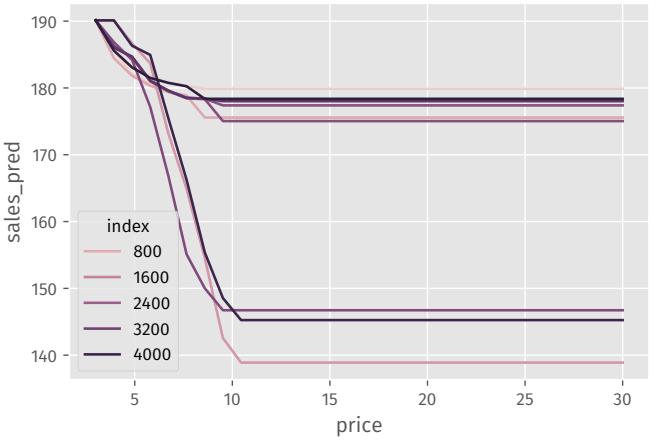
More Econometrics May be Needed!

I want to end this section on the non-scientific Double-ML with a word of caution. I didn't call this approach non scientific for nothing. It is sort of a hack to get non linear counterfactual predictions. And since it's a hack, I feel it is worth talking about its potential downsides.

First and foremost, it has the same problems all ML techniques have when applied naively to causal inference: bias. Since the final model is a regularized ML model, this regularization can bias the causal estimate to zero.

The second problem has to do with the ML algorithm you choose. Here, we choose boosted trees. Trees are not very good at making smooth predictions. As a consequence, we can have discontinuities in the prediction curve. You can see that in the plots above: a stepwise behavior here and there. Also, trees are not very good at extrapolating, so this model might output weird predictions for prices never seen before.

```
pred_test = (test
             .rename(columns={"price":"factual_price"})
             .assign(jk = 1)
             .reset_index() # create day ID
             .merge(pd.DataFrame(dict(jk=1, price=np.linspace(3, 30, 30))),
               ↪ on="jk")
             .drop(columns=["jk"]))

t_res_test = pred_test[T] - ensamble_pred(pred_test, models_t, X)

y_hat_test = ensamble_pred(pred_test, models_y, X)
pred_test[f"{y}_pred"] = model_final.predict(X=pred_test[X].assign(**{T:
  ↪ t_res_test})) + y_hat_test

np.random.seed(1)
sample_ids = np.random.choice(pred_test["index"].unique(), 10)

sns.lineplot(data=(pred_test
                   .query("index in @sample_ids")
                   .assign(max_sales = lambda d: d.groupby("index")[
                     ↪ ["sales_pred"]].transform("max"))
                   .assign(sales_pred = lambda d: d["sales_pred"] -
                     ↪ d["max_sales"] + d["sales_pred"].mean())),
             x="price", y="sales_pred", hue="index");
```



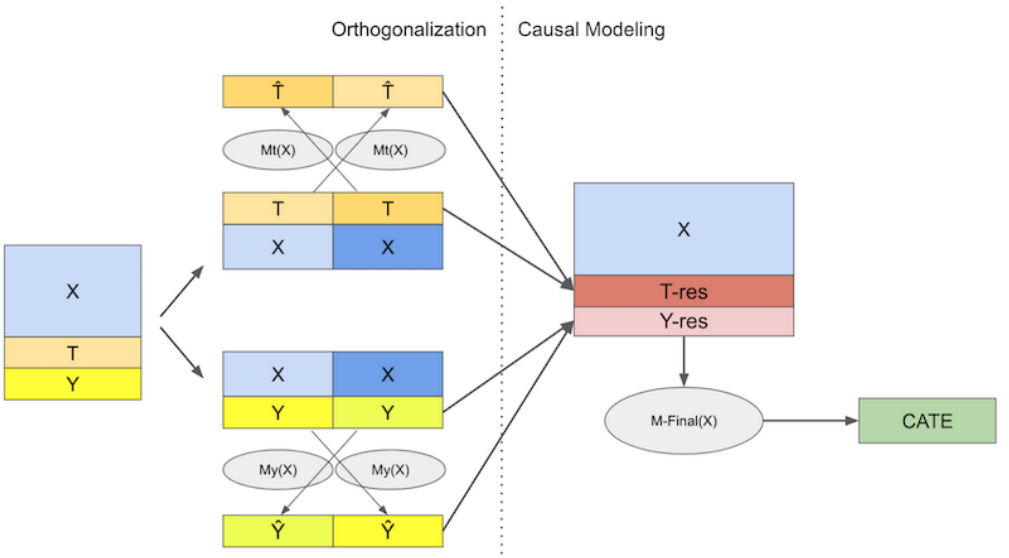
All this to say that this approach is highly dependent on the final ML model. If you regularize it too much, you will bias your causal estimates to zero. If you use this or that ML algorithm, you carry all its limitations to your final counterfactual predictions. Still, if you think this approach is worth a try, by all means give it a shot! Just don't forget the downsides I've outlined here.

Key Ideas

Double/Debiased/Orthogonal ML is a way to delegate estimating nuisance parameters which allows us to focus our attention on the causal parameter of interest. It first employes a two step orthogonalisation procedure:

1. Fit a model $M_t(X)$ to predict the treatment from the covariates X and get out of fold residuals $\tilde{t} = t - M_t(X)$. We called this the debiasing model because the residuals $\tilde{t}$ are, by definition, orthogonal to the features used to construct it.
2. Fit a model $M_y(X)$ to predict the outcome from the covariates X and get out of fold residuals $\tilde{y} = y - M_y(X)$. We called this the denoising model because the residual $\tilde{y}$ can be seen as a version of the outcome where all the variance from the features got explained away.

Once we have those residuals, provided there are no unmeasured confounders, we can regress $\tilde{y}$ on $\tilde{t}$ for a linear approximation to the ATE. We can also interact $\tilde{t}$ with the covariates to estimate the CATE or use a weighting trick to allow for any generic ML model as our final CATE model.



Finally, I've argued that the orthogonalization steps is a general tool to facilitate causal learning. In that spirit, we've tried to feed the treatment and outcome residuals to a S-learner style ML algorithm. With that, we managed to get counterfactual predictions from simulated treatments. Indeed orthogonal ML serves as a pre-processing step in many causal inference applications.

References

The things I've written here are mostly stuff from my head. I've learned them through experience. This means that they have **not** passed the academic scrutiny that good science often goes through. Instead, notice how I'm talking about things that work in practice, but I don't spend too much time explaining why that is the case. It's a sort of science from the streets, if you will. However, I am putting this up for public scrutiny, so, by all means, if you find something preposterous, open an issue and I'll address it to the best of my efforts.

To write this chapter, I've relied on Chernozhukov et al (2016), *Double/Debiased Machine Learning for Treatment and Causal Parameters*, D. Foster and V. Syrgkanis (2019) *Orthogonal Statistical Learning* and on the *econml* library documentation page. Orthogonal ML has gotten a lot of attention lately, so there are many other references on the subject. To name a few, Nie and Wager (Draft, 2020) have a nice discussion on the R-loss, Athey et al (2019) talk about it in the context of causal decision trees and there are many subsequent papers by Chernozhukov, which develop the subject further.

I also stole an image from Pedro Sant'Anna's slides · https://pedrohcgs.github.io/files/Callaway_SantAnna_2020_slides.pdf.

CHAPTER
23

Challenges with Effect Heterogeneity and Nonlinearity

Predicting treatment effects at the unit level is extremely difficult due to the lack of ground truth. Since we only observe one potential outcome $T(t)$, we can't directly estimate it. Rather, we have to rely on target transformations (that can also be viewed as cleverly designed loss function) to estimate conditional treatment effects only in expectation. But that is not the only challenge. Because treatment effects are so slippery, its estimators are often quite noisy. This has huge practical consequences for applications where we want to segment units by their treatment effect, like when we want to do personalized treatment allocation.

We will now see that, sometimes, we can get a better treatment effect segmentation if we don't directly try to estimate CATE, but instead focus on another proxy target, which usually has less variance. A common case when this happens is when the outcome variable of interest Y is binary.

Treatment Effects on Binary Outcomes

```
import pandas as pd
import numpy as np
import seaborn as sns
import statsmodels.formula.api as smf
from matplotlib import pyplot as plt
from matplotlib import style
style.use("ggplot")
```

```
from typing import List

import numpy as np
```

```
import pandas as pd
from toolz import curry, partial

@curry
def avg_treatment_effect(df, treatment, outcome):
    return df.loc[df[treatment] == 1][outcome].mean() - df.loc[df[treatment] ==
     ↪ 0][outcome].mean()



@curry
def cumulative_effect_curve(df: pd.DataFrame,
                            treatment: str,
                            outcome: str,
                            prediction: str,
                            min_rows: int = 30,
                            steps: int = 100,
                            effect_fn = avg_treatment_effect) -> np.ndarray:

    size = df.shape[0]
    ordered_df = df.sort_values(prediction, ascending=False).reset_index(
     ↪ drop=True)
    n_rows = list(range(min_rows, size, size // steps)) + [size]
    return np.array([effect_fn(ordered_df.head(rows), treatment,
     ↪ outcome) for rows in n_rows])


@curry
def cumulative_gain_curve(df: pd.DataFrame,
                          treatment: str,
                          outcome: str,
                          prediction: str,
                          min_rows: int = 30,
                          steps: int = 100,
                          effect_fn = avg_treatment_effect) -> np.ndarray:


    size = df.shape[0]
    n_rows = list(range(min_rows, size, size // steps)) + [size]

    cum_effect = cumulative_effect_curve(df=df, treatment=treatment,
     ↪ outcome=outcome, prediction=prediction,
                                         min_rows=min_rows, steps=steps,
                                          ↪ effect_fn=effect_fn)

    return np.array([effect * (rows / size) for rows, effect in zip(n_rows,
     ↪ cum_effect)])
```

Here is an incredibly common problem you might face if you find yourself working for a tech company: management wants to boost customer conversion to your product by

means of some sort of nudge. For example, they might want to increase the number of app installs by offering a 10 BRL voucher for customers to make in-app purchases. Or offer a free ride the first time you use their ride sharing app. Or decrease transaction fees in the first three months in their investment platform. Since nudges are often expensive, they would love to not have to do it for everyone. Rather, it would be great if we could use the conversion boosting nudge only on those customers who are most sensitive to it.

In causal inference terms, you can probably tell by now that this type of business problem falls under the treatment effect heterogeneity (TEH) umbrella. Specifically, you have a costly nudge as the treatment T, conversion as the binary outcome Y and customer specific pre-treatment features as X. You could then estimate the conditional average treatment effect $E[Y_1 - Y_0|X]$ (or $E[Y'(T)|X]$ if the treatment is continuous) with something like Double/Debiased ML and finally target with the nudge only the customers with the highest estimated treatment effect. In business terms, you would be personalizing your conversion strategy. You would be finding a segment of customers with high conversion incrementality and using nudges only in them.

However, there is one complication here that makes the TEH approach not so obvious. The fact that the outcome is binary complicates things considerably. Because this is a bit counter intuitive, I rather show what happens first and then explain why it happens.

Simulating Some Data

Let's keep this very simple, but still relatable. We will simulate the treatment, `nudge`, as being completely random, drawn from a Bernoulli with $p = 0.5$. This means the treatment is assigned according to a fair coin. It also means there is no confounding we need to watch out for.

$$nudge \sim \mathcal{B}(0.5)$$

Next, let's simulate the customers' covariates `age` and `income` following a Gamma distribution. These are the stuff you know about the customer and hence, you want to personalize based on them. In other words, you wish to find groups of customers defined by age and income in such a way that one group is highly responsive to the `nudge` treatment.

$$age \sim G(10, 4)$$

$$income \sim G(20, 2)$$

Finally, we will simulate the conversion. For that, we will first **create a latent variable that follows a linear model** with random noise. Importantly, notice that `income` is highly predictive of Y_{latent}, but **it does not modify the treatment effect**. Put simply, nudge increases Y_{latent} the same for all levels of `income`. In contrast, `age` only affects Y_{latent} through its interaction with the `nudge` treatment.

$$Y_{latent} \sim N(-4.5 + 0.001\ income + nudge + 0.01\ nudge\ age, 1)$$

Once we have the Y_{latent}, we can simulate `conversion` by setting it to $Y_{latent} > x$. First, let's set `x=0` so that the conversion is roughly 50%. That is, on average 50% of customers

convert to our product.

$$conversion = 1\{Y_{latent} > 0\}$$

```
np.random.seed(123)

n = 100000
nudge = np.random.binomial(1, 0.5, n)
age = np.random.gamma(10, 4, n)
estimated_income = np.random.gamma(20, 2, n)*100

latent_outcome = np.random.normal(-4.5 + estimated_income*0.001 + nudge +
  ↪ nudge*age*0.01)
conversion = (latent_outcome > .1).astype(int)
```

Let's also store everything in a DataFrame for convenience. Also, check that, indeed, the average conversion is close to 50%.

```
df = pd.DataFrame(dict(conversion=conversion,
                       nudge=nudge,
                       age=age,
                       estimated_income=estimated_income,
                       latent_outcome=latent_outcome))

df.mean()
```

```
conversion             0.518260
nudge                  0.500940
age                   40.013487
estimated_income    3995.489527
latent_outcome         0.197076
dtype: float64
```

As for the average treatment effect, since the treatment was randomized, we can estimate it as the simple difference in means between treated and control groups: $E[Y \mid T = 1] - E[Y \mid T = 0]$. So, let's see what those treatment averages look like. We will look at them for both the latent outcome and the conversion perspective. There is something important to see here.

```
df.groupby("nudge")[["latent_outcome", "conversion"]].mean()
```

nudge	latent_outcome	conversion
0	-0.505400	0.320503
1	0.896916	0.715275

```
avg_treatment_effect(df, "nudge", "latent_outcome")
```

```
1.4023163965477656
```

```
avg_treatment_effect(df, "nudge", "conversion")
```

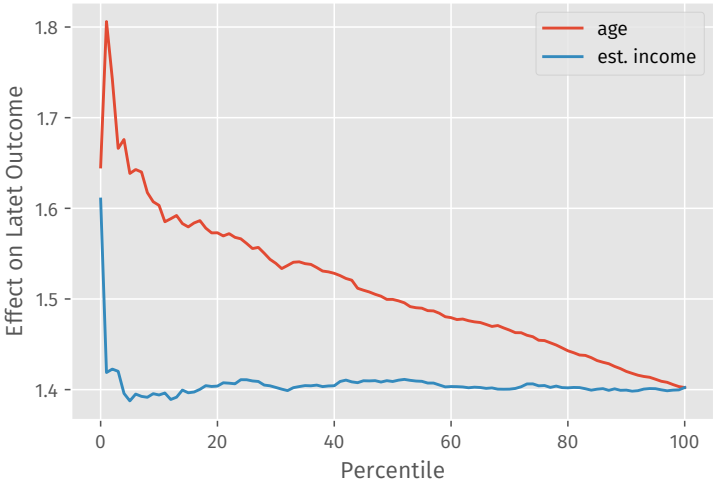
```
0.39477273768476406
```

The ATE for the latent outcome is pretty straightforward. From our data generating model, we know that this effect should be `1 + avg(age)*0.01`. And since the average age is about 40, this gives us an ATE of about 1.4. Where things get a bit more interesting (and complicated) is in the ATE for conversion. **Because conversion is bounded between 0 and 1, its ATE will not be linear**. Hence, we can't deduce it from an easy formula as we did with the latent outcome (there is a formula, but it is quite complicated). Let's just say the effect is smaller. And this makes sense right? I mean, there is no way the treatment could increase conversion by 1.4 points, simply because conversion can't go beyond 100% . Now, I want you to hold on to that fact because it is going to be crucial in understanding what we will see next.

Let's talk about conditional average treatment effects (CATE) now. Looking at our data generating process, we know for a fact that `estimated_income` predicts conversion but does not modify the effect of the nudge on conversion. Therefore, segmenting our customers based on `estimated_income` will generate segments with the same treatment effect. In contrast, `age` only affects conversion by its interaction with the nudge. So, different age segments will respond very differently to the treatment, while different different income segments won't. In other words, `estimated_income` should not be a good personalization variable while age should be.

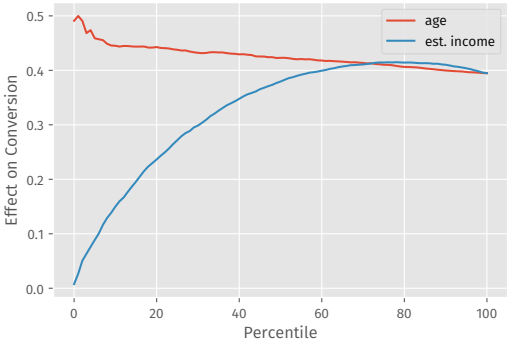
One way to see this is through the cumulative effect curve. The curve for `age` should start very far from the ATE and slowly converge to it, while the curve for `estimated_income` should only fluctuate around the ATE. This is exactly what we see when we plot the cumulative effect curve for the nudge effect on the latent outcome.

```
cumulative_effect_fn = cumulative_effect_curve(df, "nudge", "latent_outcome",
  ↪ min_rows=500)

age_cumm_effect_latent = cumulative_effect_fn(prediction="age")
inc_cumm_effect_latent = cumulative_effect_fn(prediction="estimated_income")

plt.plot(age_cumm_effect_latent, label="age")
plt.plot(inc_cumm_effect_latent, label="est. income")
plt.legend()
plt.xlabel("Percentile")
plt.ylabel("Effect on Latet Outcome");
```



Again, the latent outcome is very nice. Due to its linearity, our expectations match reality pretty closely. But in real life, we don't care about the latent outcome, nor can we observe it. All we have is conversion. And with conversion, the picture looks a lot more complicated. If we plot the cumulative effect curves, age still shows some treatment effect heterogeneity, starting above the ATE and slowly converging to it. This means that the higher the age, the higher the treatment effect. So far so good. This is what we would expect.

```
cumulative_effect_fn = cumulative_effect_curve(df, "nudge", "conversion",
  ↪ min_rows=500)

age_cumm_effect_latent = cumulative_effect_fn(prediction="age")
inc_cumm_effect_latent = cumulative_effect_fn(prediction="estimated_income")

plt.plot(age_cumm_effect_latent, label="age")
plt.plot(inc_cumm_effect_latent, label="est. income")
plt.legend()
plt.xlabel("Percentile")
plt.ylabel("Effect on Conversion");
```

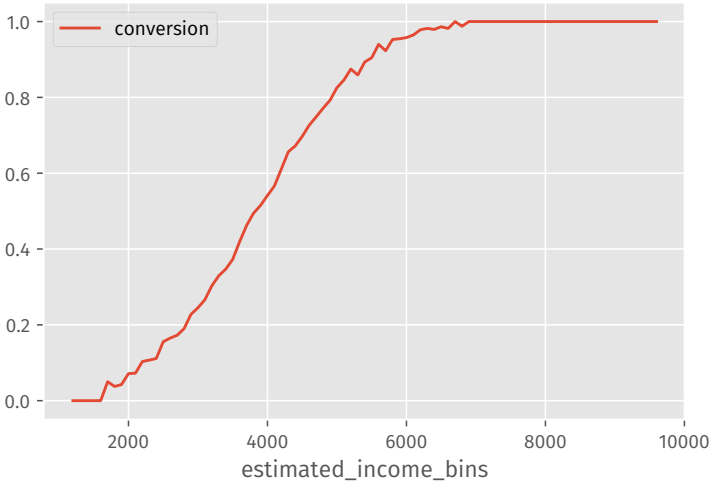


However, we also have A LOT of treatment effect heterogeneity by `estimated_income`. Customers with higher `estimated_income` have a much lower treatment effect, which causes the cumulative effect curve to start all the way at zero, to then overshoot the ATE, and only then to converge to it. This tells us that, as far as personalization is concerned, `estimated_income` will generate segments that have more treatment effect heterogeneity (TEH) compared to the segments we would get with age.

This is inconvenient right? How come the feature we know to drive effect heterogeneity, age, is worse for personalization when compared with a feature (`estimated_income`) we know not to modify the treatment effect? The answer lies in the **non-linearity of the outcome function**. Although `estimated_income` does not modify the effect of the nudge on the latent outcome, it does once we transform that latent outcome to conversion (at least indirectly). Conversion is not linear. This means that **its derivative changes depending on where you are**. Since conversion can only go up to 1, if it is already very high, it will be hard to increase it further. In other words, the derivative at high conversion is very low. The same thing is true at the lower end: Because conversion is also bounded below at zero, it will also have a low derivative if it is already very low. Conversion follows an S-shape, with low derivatives at both ends, which we can see by plotting the average conversion by `estimated_income` bins (bins of width 100).

```
(df
 .assign(estimated_income_bins=(df["estimated_income"]/100).astype(int)*100)
 .groupby("estimated_income_bins")
 [["conversion"]]
 .mean()
 .plot()
);
```

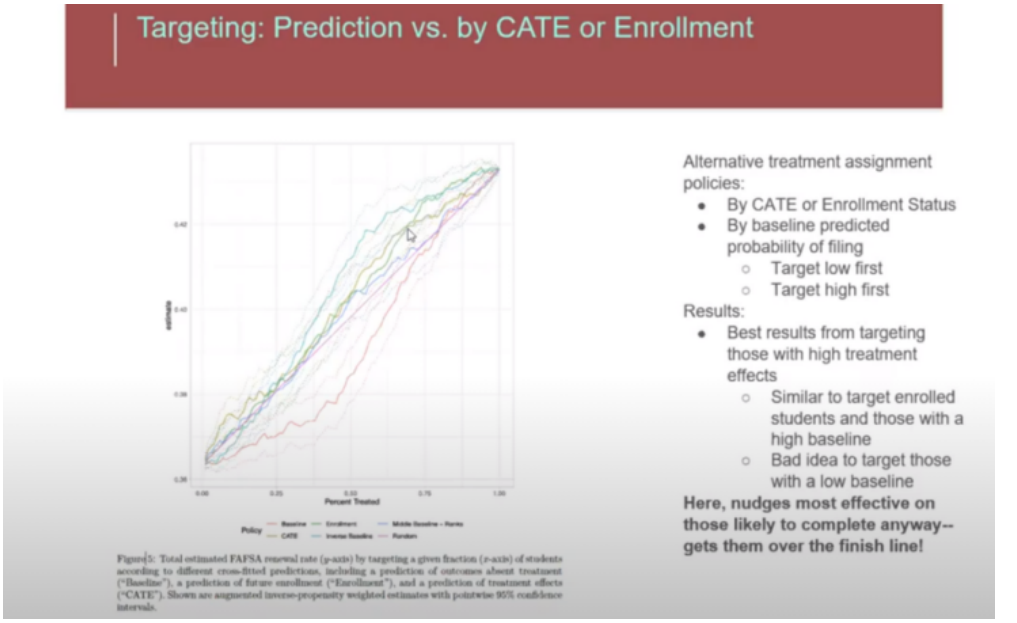
```
<AxesSubplot:xlabel='estimated_income_bins'>
```



Notice how the slope (derivative) of this curve is very small when conversion is very high. It is also small when conversion is very low (although that is harder to see due to the small sample in that region). With this information, we can now explain why `estimated_income` generates segments with high treatment effect heterogeneity.

Since `estimated_income` is highly predictive of conversion, we can say that customers with different `estimated_income` are located in different places along the S-shaped conversion curve. Customers with very high or very low `estimated_income` fall at the extremes of the curve, where the derivative is lower, meaning that increasing conversion is harder, which in turns means that the treatment effect is likely to be small. On the other hand, customers with reported income in the middle of the range also fall in the middle of the conversion curve, where the derivative is higher and, hece, the treatment effect will likely also be higher. I say likely because, in theory, it is possible for a variable to have such a strong effect modification force that it dominates the change in derivative we see as we traverse the conversion curve. However, at least from my experience, the curvature of the S shaped conversion tends to dominate every other effect modification we have.

It's not just me, though. Here is a slide from Susan Atheys' presentation at the Columbia Data Science Institute. Here, she is discussing the effect of a nudge to get students to apply for federal financial aid in order to pay for college. It's also a conversion problem. What she finds is that the best strategy is to target those students that are already likely to convert. She also says it is a bad idea to target those with low probability of conversion



Wait a minute! That is not what you said above! You said that treatment effects were low away from the middle, towards the low and the high ends!

True. However, in real life, conversion rarely spams the entire S-shaped curve. What we usually have is everyone smooshed at one or the other end of the curve. In business terms, your average conversion is rarely 50%. More often than not, it is something like 70 to 90% or something like 1 to 20%. In these more likely situations, targeting those with a high baseline can be a good or a bad idea.

Here is what I mean: Let's take the same latent outcome from before, but now generate a situation where conversion is low on average, by setting it to `latent_outcome > 2`. Next, let's craft a situation where conversion is high by setting `latent_outcome > -2`.

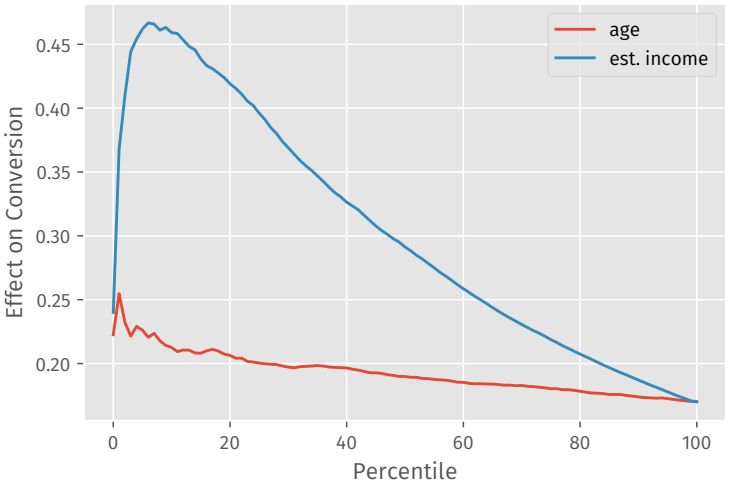
```
df["conversion_low"] = conversion = (latent_outcome > 2).astype(int)
df["conversion_high"] = conversion = (latent_outcome > -2).astype(int)

print("Avg. Low Conversion: ", df["conversion_low"].mean())
print("Avg. High Conversion: ", df["conversion_high"].mean())
```

```
Avg. Low Conversion:  0.12119
Avg. High Conversion:  0.9275
```

Based on what we know about the non linearity of conversion, we can already predict what will happen. For the low conversion situation, targeting those with high baseline conversion (high `estimated_income`) will be much more effective. That's because we are at the left side of the S shaped conversion curve, where the derivative will be smaller the lower the baseline conversion. In this region, **high baseline conversion will translate to**

high treatment effect. Therefore, we should nudge those with high baseline conversion, which will be those with higher `estimated_income`.

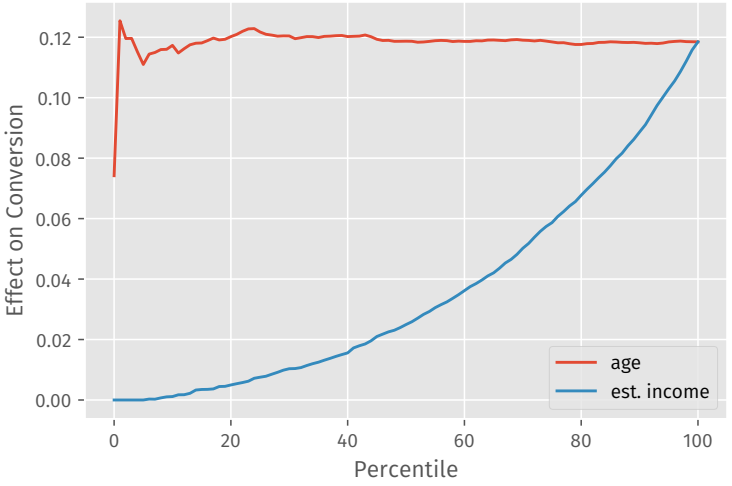
```
cumulative_effect_fn = cumulative_effect_curve(df, "nudge", "conversion_low",
  ↪ min_rows=500)

age_cumm_effect_latent = cumulative_effect_fn(prediction="age")
inc_cumm_effect_latent = cumulative_effect_fn(prediction="estimated_income")

plt.plot(age_cumm_effect_latent, label="age")
plt.plot(inc_cumm_effect_latent, label="est. income")
plt.xlabel("Percentile")
plt.ylabel("Effect on Conversion");
plt.legend();
```



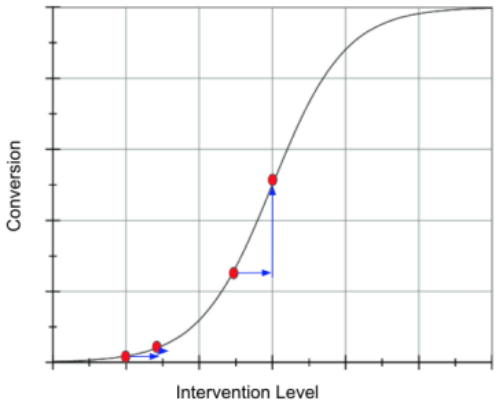
Just like we predicted, those with high `estimated_income`, which translates to high baseline conversion, have a much higher treatment effect.

Now, for the other situation where conversion is high, on average, those with **high baseline conversion will have a lower treatment effect**. Hence, it is a bad idea to target those with high `estimated_income`. We can see this by the inverted cumulative effect curve, which shows those with high `estimated_income` as having a lower treatment effect.

```
cumulative_effect_fn = cumulative_effect_curve(df, "nudge", "conversion_high",
  ↪ min_rows=500)

age_cumm_effect_latent = cumulative_effect_fn(prediction="age")
inc_cumm_effect_latent = cumulative_effect_fn(prediction="estimated_income")

plt.plot(age_cumm_effect_latent, label="age")
plt.plot(inc_cumm_effect_latent, label="est. income")
plt.xlabel("Percentile")
plt.ylabel("Effect on Conversion")
plt.legend();
```



To summarize, what we saw is that when the outcome is binary, the treatment effect tends to be dominated by the curvature (derivative) of the S-shaped function.



For instance, in our conversion problem, if the **average conversion is low**, we are at to the left of the logistic curve and the **treatment effect will be higher at high baseline conversion**. This would translate to a nudge policy that advocates for treating (nudging) those customers with an already high probability of conversion. On the other hand, if the **average conversion is high**, we will be to the right side of the logistic curve, where the derivative (and hence the treatment effect) will be **higher for those customers with lower baseline conversion**.

This is certainly a lot to remember, but we can simplify: **just treat whomever is closer to a baseline conversion of 50%**. The mathematical argument here is pretty solid: the derivative of the logistic function is at its peak at 50%, so just treat units closer to that point.

What is even nicer is that this is one of the rare cases where received wisdom matches the math. In marketing, where conversion problems are very common, there is a belief that we should target neither lost bets (those with very low conversion probability) nor sure wins (those with very high conversion probability). Instead, we should target those in the middle. This is fascinating, since it is the exact same thing we figured using a more formal causal argument.

Continuous Treatment and Non-linearity

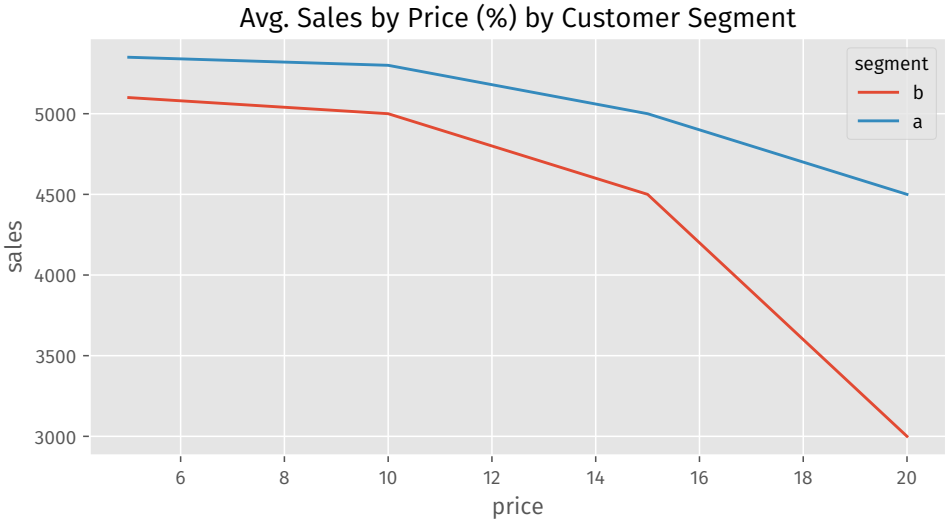
We've explored in depth just one example of binary outcome making Heterogeneous Treatment Effect analysis harder. But this phenomenon goes beyond the conversion problem from marketing. For instance, in 2021, the world managed to deliver the first batch of approved COVID-19 vaccines to the general public. Back then, a crucial question was who should receive the vaccine first. This is, not surprisingly, a Heterogeneous Treatment Effect problem. Policy makers would like to vaccinate those who would benefit the most first. In this situation, the treatment effect is averting death or hospitalization. So, whose death or likelihood to be hospitalized would decrease the most when given a shot? In most countries, it was the elderly and those with pre-existing health conditions (comorbidities). These are people that are **more likely to die when infected with COVID-19**. Also, COVID mortality rate is (thankfully!) much lower than 50%, which puts us to the left of logistic function. In this region, by the same argument we made for marketing, it would make sense to treat those with a high baseline probability of death when getting COVID-19, which are precisely the groups we've mentioned earlier. Is this a coincidence? Maybe. Keep in mind that I'm not a health expert, so I might be very wrong here. But the logic makes a lot of sense to me.

In both cases, marketing nudges and COVID-19 vaccines, **the key complicating factor for Treatment Effect Heterogeneity is the non-linearity of the outcome function** $Y(0)$. This nonlinearity makes it so that, as we go from $Y(0)$ to $Y(1)$, the increase in the outcome is primarily due to the curvature in the outcome function. We saw how this happened with a binary outcome, where $E[Y|X]$ follows a logistic shape. But this is more general. In fact, it is a problem that keeps popping up in business, especially if the

treatment is a continuous variable. Let's go through one last example to drive the point home.

Let's consider the classic pricing problem. You are working for a streaming company like Netflix or HBO. A key question the company wants answered is what price to charge customers. In order to answer the question, they run an experiment in which they randomly assign customers to differently priced deals: 5 BLR/month, 10 BRL/month, 15 BRL/month or 20 BRL/month. By doing so, they hope to answer not just how sensitive customers are to price increases, but also if some types of customers are more sensitive than others. In the plot below, you can see the results from that experiment broken down by two customer segments: A, customers with higher estimated income, and B, customers with lower estimated income.

```
data = pd.DataFrame(dict(
    segment= ["b", "b", "b", "b",  "a", "a", "a", "a",],
    price=[5, 10, 15, 20, ] * 2,
    sales=[5100, 5000, 4500, 3000,  5350, 5300, 5000, 4500]
))

plt.figure(figsize=(8,4))
sns.lineplot(data=data, x="price", y="sales", hue="segment")
plt.title("Avg. Sales by Price (%) by Customer Segment");
```



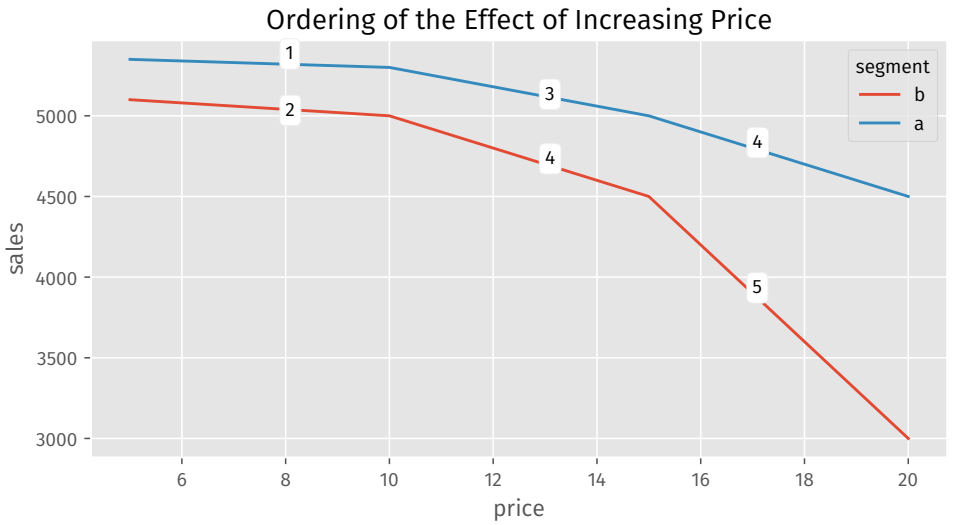
With this data, the company wishes to answer the following question: who is more sensitive to discounts? In other words, how can we **rank customers by their sensitivity to price (price elasticity of sales)**? By looking at the curve, we get a feeling that segment A is overall less sensitive to discount, even though it generates more revenues. However, we can also see that there is some curvature there. In fact, if we take this curvature into account, the ranking of the treatment effect is no longer just between A and B customers.

The treatment effect will also depend on where they are in the treatment curve. For example, the treatment effect of going from 15 BRL to 10 BRL on customers of segment A is higher than the treatment effect of going from 5BRL to 10BRL on customers of segment B:

$$E[Y(10) - Y(5)|Seg = B] < E[Y(15) - Y(10)|Seg = A]$$

If we where to order the resulting treatment effects for this experiment, it would look something like this:

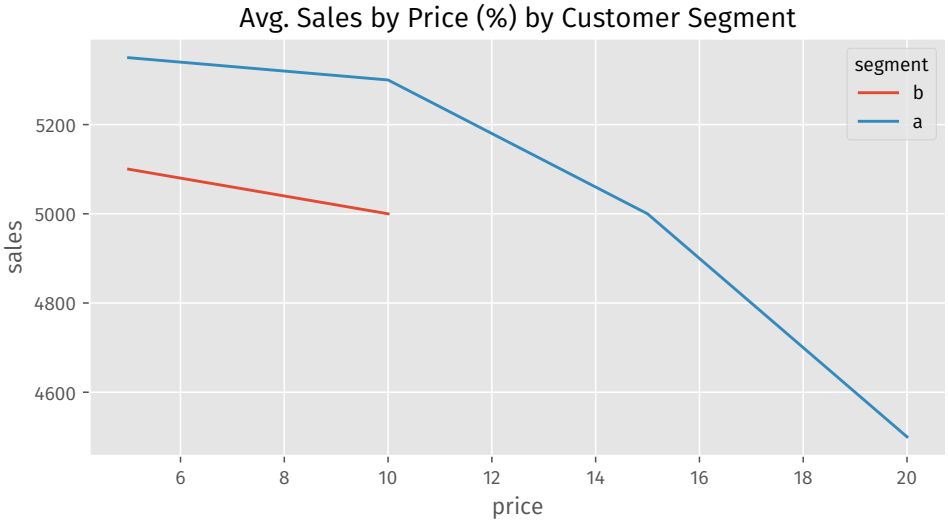
```
plt.figure(figsize=(8,4))
sns.lineplot(data=data, x="price", y="sales", hue="segment")

plt.annotate("1", (8, 5350), bbox=dict(boxstyle="round", fc="1"))
plt.annotate("2", (8, 5000), bbox=dict(boxstyle="round", fc="1"))
plt.annotate("3", (13, 5100), bbox=dict(boxstyle="round", fc="1"))
plt.annotate("4", (13, 4700), bbox=dict(boxstyle="round", fc="1"))
plt.annotate("4", (17, 4800), bbox=dict(boxstyle="round", fc="1"))
plt.annotate("5", (17, 3900), bbox=dict(boxstyle="round", fc="1"))

plt.title("Ordering of the Effect of Increasing Price");
```



Just like in the case where the outcome was binary, in this example, **the treatment effect is correlated with the outcome**. The higher the sales (lower the price), the lower the absolute treatment effect; the lower the sales (higher the price) the lower the absolute treatment effect. But in this case, the situation is even more complicated because the **effect is not only correlated with the outcome, but with the treatment level**. This makes answering counterfactual questions trickier. For example, pretend for a moment that your experimental data actually looks like the following plot, where you test higher prices for the segment A (rich population) but only lower prices for the A population. This

is very common, as firms often want to experiment around the treatment they think is more reasonable.

```
data = pd.DataFrame(dict(
    segment= ["b", "b", "b", "b",  "a", "a", "a", "a",],
    price=[5, 10, 15, 20, ] * 2,
    sales=[5100, 5000, 4500, 3000,  5350, 5300, 5000, 4500]
))

plt.figure(figsize=(8,4))
sns.lineplot(data=data.loc[lambda d: (d["segment"] == "a") | (d["price"] < 12)
  ↪ ], x="price", y="sales", hue="segment")
plt.title("Avg. Sales by Price (%) by Customer Segment");
```



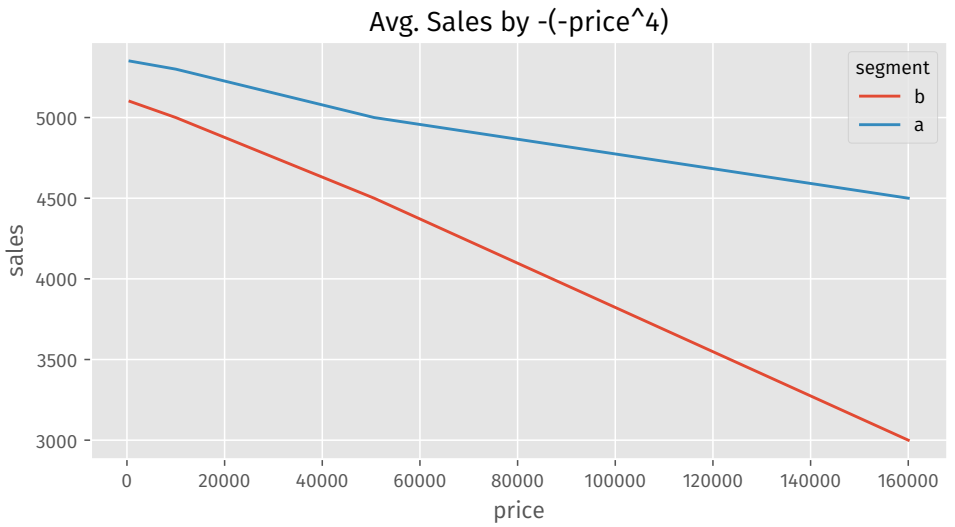
Now, if you naively aggregate the results of the treatment effect, it will look like segment A is much more elastic (has higher absolute treatment effect) to price increase than segment B. But that is only because, for segment B, you've explored the low treatment effect region.

So what can you do when treatment effects change depending on where you are in terms of both treatment and outcome? To be honest, this is still an active area of research. In practical terms, the best thing you can do is to be **very careful** when trying to answer which type of customer is more sensitive to the treatment. Make sure that the compared customer types all had the same treatment distribution. And, if not, be very skeptical of extrapolating the treatment effect. For instance, in the example above, even though customer B looks less sensitive to price increase, you don't know if that will still be the case if you assign higher prices beyond 10BRL to this segment.

Another thing you can try to do is to linearize the response curve. The idea here is to get rid of the curvature by transforming the treatment or the outcome (or both) so that the relationship between them looks like a line. Since a line has constant derivative,

this would get rid of the problem of the treatment effect changing depending on where you are in the curve. As an example, if we take our price variable and transform it by making it negative, exponentiating it by 4 and reversing the sign, we get a somewhat linear(ish) relationship. In this transformed data, the statement that A is less sensitive to price increases then B makes much more sense, since it now does not depend on where we are at the curve.

```
plt.figure(figsize=(8,4))
sns.lineplot(data=data.assign(price = lambda d: -1*(-d["price"]**4)),
             x="price", y="sales", hue="segment")
plt.title("Avg. Sales by -(-price^4)");
```



However, this approach has many drawbacks. First, it is not always possible to linearize a curve. In our example you can clearly see that this linearization is not perfect. But more importantly, sometimes it makes no business sense to discard the curvature. In our pricing example, it may very well be that we are fine in treating customer A at price 15 as more sensitive than customer B at price 5. This would lead us to a sound decision of decreasing price for customer A from 15 to 10, but not doing anything to the price of customer B.

Key Concepts

I realize I may have raised more questions than provide answers. Alas, sometimes the best we can do about a problem is to be very aware of it. In this chapter, I hope I've managed to open your eyes to the complications that arise when the outcome we care about is non-linear.

This is a common and more studied problem with binary outcomes. In this case, the treatment effect tends to be higher the closer we are to an average outcome of 0.5. Since the outcome is bounded at 0 and 1, effects tend to be very small if we are too close to 0 or 1.

Things get more complicated with non-linearities arising in continuous outcome situations. Here, the best you can do is think very carefully about the problem. Try to answer if you care more about treatment effect regardless of treatment baseline or if the baseline is important. That alone will be a valuable guiding principle.

Reference

Most of the things written here are from my own experience with this problem. However, I did find one academic articles that touch on this subject: *Causal Classification: Treatment Effect Estimation vs. Outcome Prediction*, by Fernández-Loría and Provost talk about the case where treatment effect is correlated with the outcome variable.

CHAPTER

24

The Difference-in-Differences Saga

```
from toolz import *

import pandas as pd
import numpy as np
from scipy.special import expit

from linearmodels.panel import PanelOLS
import statsmodels.formula.api as smf

import seaborn as sns
from matplotlib import pyplot as plt
from matplotlib import style

style.use("ggplot")
```

After discussing treatment effect heterogeneity, we will now switch gears a bit, back into average treatment effects. Over the next few chapters, we will cover some recent developments in panel data methods. A panel is a data structure that has repeated observations across time. The fact that we observe the same unit in multiple time periods allows us to see what happens before and after a treatment takes place. This makes panel data a promising alternative to identifying the causal effects when randomization is not possible.

To motivate the use of panel data, we will mostly talk about causal inference applications to marketing. Marketing is particularly interesting for its notorious difficulty in running randomized experiments. In marketing, we often can't control who receives the treatment, that is, who sees our advertisements. When a new user comes to our site or downloads our app, we have no good way of knowing if that user came because he or she saw one of our campaigns or due to some other factor.

(OBS: For those more familiar with marketing attribution, I'm aware of the many attribution tools that aim at solving this problem. But I'm also aware of their many limitations).

The problem is even bigger with offline marketing. How can you know if a TV campaign brought value in excess of its costs? Because of that, a common practice in marketing is Geo-Experiments: we deploy a marketing campaign to some geographical region but not others and compare them. In this design panel data methods are particularly interesting: we can collect data on an entire geography (unit) across multiple periods of time. To make sense of this sort of data and identify the causal effect, perhaps the most popular methods are those in the Diff-in-Diff (DiD) family.

The years 2020 and 2021 have not been easy for most of us. But it was particularly hard on DiD. A lot of recent research highlighted some severe flaws in these methods, flaws which were not well known in the past. So, although we already have a chapter in Part I covering DiD, the content there is rather introductory. It doesn't cover the new findings and effervescent discussions around panel data methods. We should now take a more thorough look at them, beginning with Diff-in-Diff. In this specific chapter, I'll try to summarize the recently discovered problems with diff-in-diff and also show how to fix them. This chapter is divided into three sections:

1. **Birth**: Recap why panel data is so attractive for causal inference and how DiD and Two Way Fixed Effect (TWFE) leverages the temporal structure in its favor.
2. **Death**: Digest one key assumption implied by DiD and TWFE models that has been overlooked. Understand when and how that assumption can fail.
3. **Enlightenment**: Knowing the problem with DiD and TWFE, we can now think about a solution for it. This part shows a simple workaround to the problem explored in section 2.

Let's get right into it!

1) Birth: The Promise of Panel Data

Like I've said, panel data is when we have multiple units `i` over multiple periods of time `t`. Think about a policy evaluation scenario in the US, where you want to check

the effect of cannabis legalization on crime rate. You have crime rate data on multiple states `i` over multiple time periods `t`. You also observe at what point in time each state adopts legislation in the direction of cannabis legalization. I hope you can see why this is incredibly powerful for causal inference. Call cannabis legalization the treatment `D` (since `T` is taken; it represents time). We can follow the trend on crime rates for a particular state that eventually gets treated and see if there are any disruptions in the trend at the treatment time. In a way, a state serves as its own control unit, in a before and after comparison. Furthermore, because we have multiple states, we can also compare treated states to control states. When we put both comparisons together, treated vs control and before vs after treatment, we end up with an incredibly powerful tool to infer counterfactuals and, hence, causal effects.

Panel data methods are often used in government policy evaluation, but we can easily make an argument about why they are also incredibly useful for the (tech) industry. Companies often track user data across multiple periods of time, which results in a rich panel data structure. Not only that, sometimes experimentation is not possible, so we have to rely on other identification strategies. To explore that idea further, let's consider a hypothetical example of a young tech company that tracks the number of people that installed its app across multiple cities. At some point in 2021, the tech company launched a new feature in their app. It now wants to know how many new uses that feature brought to the company. The rollout was gradual. Some cities got the feature in `2021-06-01`. Others, in `2021-07-15`. The full rollout to the rest of the cities only happens in 2022. Since our data only goes up until `2021-07-31`, this last group can be considered the control group. In causal inference terms, rolling out this feature can be seen as the treatment and number of installs can be seen as the outcome. We want to know the treatment effect on the outcome, that is, the effect of the new feature on the number of installs.

Notice how the tech company cannot do an experiment here. They can't control which person gets to know they have a new feature. We say that they have limited control over the treatment assignment. That's because the unit of analysis is **people that are not yet their customers**. They want to know how many of those they can convert to customers by installing their app. Of course they can't randomize anything for those people. So, instead, they changed the unit of analysis to cities. The release in one city versus the other is something they can control, which is not the case for the release for one person versus the other.

The group of cities that got the feature (got treated) at a specific point in time is called a cohort. In our case, we have three cohorts: one that got treated in `2021-06-01`, another that got treated in `2021-07-15` and a control cohort, which only gets treated after the end of our data. To get a sense of what this data looks like, let's plot the average daily installs grouped by cohort.

```
date = pd.date_range("2021-05-01", "2021-07-31", freq="D")
cohorts = pd.to_datetime(["2021-06-01", "2021-07-15", "2022-01-01"]).date
units = range(1, 100+1)
```

```
np.random.seed(1)

df = pd.DataFrame(dict(
    date = np.tile(date, len(units)),
    unit = np.repeat(units, len(date)),
    cohort = np.repeat(np.random.choice(cohorts, len(units)), len(date)),
    unit_fe = np.repeat(np.random.normal(0, 5, size=len(units)), len(date)),
    time_fe = np.tile(np.random.normal(size=len(date)), len(units)),
    week_day = np.tile(date.weekday, len(units)),
    w_seas = np.tile(abs(5-date.weekday) % 7, len(units)),
)).assign(
    trend = lambda d: (d["date"] - d["date"].min()).dt.days/70,
    day = lambda d: (d["date"] - d["date"].min()).dt.days,
    treat = lambda d: (d["date"] >= d["cohort"]).astype(int),
).assign(
    y0 = lambda d: 10 + d["trend"] + d["unit_fe"] + 0.1*d["time_fe"] +
      ↪ d["w_seas"]/10,
).assign(
    y1 = lambda d: d["y0"] + 1
).assign(
    tau = lambda d: d["y1"] - d["y0"],
    installs = lambda d: np.where(d["treat"] == 1, d["y1"], d["y0"])
)
```

```
plt.figure(figsize=(10,4))
[plt.vlines(x=cohort, ymin=9, ymax=15, color=color, ls="dashed") for color,
  ↪ cohort in zip(["C0", "C1"], cohorts[:-1])]
sns.lineplot(
    data=(df
          .groupby(["cohort", "date"])["installs"]
          .mean()
          .reset_index()),
    x="date",
    y = "installs",
    hue="cohort",
);
```

The dashed lines mark the moment a cohort got the treatment (the feature was rolled out). Take a moment to appreciate the richness of the data depicted in the above plot. First, we can see that each cohort has its own baseline level. That's simply because different cities have different population sizes, leading to more or less installs depending on the city size. For instance, it looks like cities in the first cohort (treated in 06/01) have a higher baseline, compared to the cities in the other cohorts. Also, it looks like the control cohort has a lower baseline of intalls. This means that simply comparing treated cohorts against the control cohorts would yield a biased result, since Y_0 for the control is lower than the Y_0 for the treated, or $Y_0 | G = Control < Y_0 | G = Treated$, where we use G to denote the cohort. Fortunately this won't be a problem. Panel data allow us to compare across cities **and** time, which allows it to adjust for different baselines.

Speaking of time, notice how there is an overall upward trend with some wiggles (which looks like weekly seasonality). Focusing on the control cohort, it looks like daily installs went from about 10 in May to about 11 in June, an increase of about 1. In technical terms, latter time periods have higher Y_0 than early time periods. This means that simply comparing the same cities across time would also yield biased results. Once again, we are fortunate that the panel data structure allows us to compare not only across time, but also across cities, adjusting for trends.

Ideally, to infer the effect of the rollout of this new feature on installs, we want to know what would have happened to the cohorts that got the feature, had they not got it. We want to estimate the counterfactual outcome $Y(0)$ in the post treatment periods for the treated cohorts. If we denote each cohort by the time it got treated g (remember that a cohort is just a group of cities that got treated at the same time), we can write this counterfactual as $E[Y_0 | t \geq g]$, which would then allow us to define the treatment effect on the treated (the ATT) for cohort g as follows:

$$E[Y_1 | t \geq g] - E[Y_0 | t \geq g]$$

The next question is how can we estimate this from the data we have. Well, one way is to exploit the power of the panel data structure and estimate those counterfactuals. For instance, we could use linear regression and the Diff-in-Diff formulation to get a Two Way Fixed Effect model. Let's say each city `i` has a base install level γ_i. This ties back to what we saw earlier: maybe a city has more installs because it has a bigger population or

because its culture is more in line with the product from our tech company. Regardless of the reason and even if we don't know why, we say that those unit idiosyncrasies can be captured by a **time fixed parameter** γ_i. Similarly, we can say that each time period `t` has a baseline install level which we can capture by a **unit fixed parameter** θ_t. If that is the case, a good way of modeling install is to say it depends on the city (unit) effect γ and the time effect θ, plus some random noise.

$$Installs_{it} = \gamma_i + \theta_t + e_{it}$$

To include the treatment in this picture, let's define a variable D_{it} which is 1 if the unit is treated and zero otherwise. In our example, this variable would be always zero for the never treated cohort. It would also be zero for all the other cohorts at the beginning, but it would turn into 1 at 06/01 for the cohort treated on 06/01 and stay at 1 after that. Also, it would turn into 1 at 07/15 for the cohort treated on 07/15. We can include those treatment indicators in our model of installs as follows:

$$Installs_{it} = \tau D_{it} + \gamma_i + \theta_t + e_{it}$$

Estimating the above model with OLS is what is called the Two-Way Fixed Effects Model (TWFE). Notice that τ would be the treatment effect, as it tells us how much installs changes once units are treated.

Another way of looking at it is to invoke the "holding things constant" propriety of linear regression. If we estimate the above model, we could read the estimate of τ as how much installs would change if we flip the treatment from 0 to 1 while holding the unit `i` and time `t` fixed.

Notice how bold this is! To say we would hold each unit fixed while seeing how D changes the outcome is to say we are controlling for all unit specific characteristics that are constant through time, known and unknown. For example, we would be controlling for cities' baseline install level, which we can measure, but also stuff we have no idea about, like how much a city culture is in line with our product. The only requirement is that this characteristic is fixed over the time of the analysis. Moreover, to say we would hold each time period fixed is to say we are controlling for all year specific characteristics. For instance, since we are holding the year fixed, while looking at the effect of D, we would be controlling for the trend and seasonalities we saw earlier.

To see all this power in action all we have to do is run an OLS model with the treatment indicator D (`treat` here), plus dummies for the units and time. In our particular example, I've generated data in such a way that the effect of the treatment (new feature) is to increase installs by 1. Notice how TWFE can recover that treatment effect perfectly:

```
formula = f"""installs ~ treat + C(unit) + C(date)"""

twfe_model = smf.ols(formula, data=df).fit()

twfe_model.params["treat"]
```

```
1.0000000000000533
```

Since I've simulated the data above, I know exactly the true individual treatment effect, which is stored in the `tau` column. Since the TWFE should recover the treatment effect on the treated, we can verify that the true ATT matches the one estimated above. All we have to do is filter the treated units and period (`treat==1`) and take the average of the `tau` column.

```
df.query("treat==1")["tau"].mean()
```

```
1.0
```

Before anyone comes and says that generating one dummy column for each unit is impossible with big data, let me come forward and tell you that, yes, that is true. But there is an easy work around. We can use the FWL theorem to partition that single regression into two. In fact, running the above model is numerically equivalent to estimating the following model

$$\tilde{Installs}_{it} = \tau \tilde{D}_{it} + e_{it}$$

where

$$\tilde{Installs}_{it} = Installs_{it} - \underbrace{\frac{1}{T}\sum_{t=0}^{T} Installs_{it}}_{\text{Time Average}} - \underbrace{\frac{1}{N}\sum_{i=0}^{N} Installs_{it}}_{\text{Unit Average}}$$

and

$$\tilde{D}_{it} = D_{it} - \frac{1}{T}\sum_{t=0}^{T} D_{it} - \frac{1}{N}\sum_{i=0}^{N} D_{it}$$

In words now, in case the math is too crowded, we subtract the unit average across time (first term) and the time average across units (second term) from both the treatment indicator and the outcome variable to constrict the residuals. This process is oftentimes called de-meaning, since we subtract the mean from the outcome and treatment. Finally, here is the same exact thing, but in code:

```
@curry
def demean(df, col_to_demean):
    return df.assign(**{col_to_demean: (df[col_to_demean]
                                        - df.groupby("unit")[col_to_demean]
                                          ↪ .transform("mean")
                                        - df.groupby("date")[col_to_demean]
                                          ↪ .transform("mean"))})


formula = f"""installs ~ treat"""
mod = smf.ols(formula,
              data=df
              .pipe(demean(col_to_demean="treat"))
```

```
            .pipe(demean(col_to_demean="installs")))

result = mod.fit()

result.summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	-11.4580	7.98e-16	-1.44e+16	0.000	-11.458	-11.458
treat	1.0000	2.02e-15	4.96e+14	0.000	1.000	1.000

Another thing we can do to understand what TWFE model is doing is to plot the counterfactual predictions $\hat{Y_0}|t \geq g$. This is helpful because what our model sees as the treatment effect $\hat{\tau}$ as simply the estimated difference $Y_1 - \hat{Y_0}$. Looking at this explicit difference can shed some light on what the model is doing. In the plot below, we can see exactly that, $\hat{Y_0}$ represented by dotted lines.

```
df_pred = df.assign(**{"installs_hat_0": twfe_model.predict(df.assign(
  ↪ **{"treat":0}))})


plt.figure(figsize=(10,4))
[plt.vlines(x=cohort, ymin=9, ymax=15, color=color, ls="dashed") for color,
  ↪ cohort in zip(["C0", "C1"], cohorts[:-1])]
sns.lineplot(
    data=(df_pred
          .groupby(["cohort", "date"])["installs_hat_0"]
          .mean()
          .reset_index()),
    x="date",
    y = "installs_hat_0",
    hue="cohort",
    alpha=0.7,
    ls="dotted",
    legend=None
)
sns.lineplot(
    data=(df_pred
          .groupby(["cohort", "date"])["installs"]
          .mean()
          .reset_index()),
    x="date",
    y = "installs",
    hue="cohort",
);
```

This plot shows us how TWFE is projecting the trend it sees in the control units onto the treated units and it is also adjusting the levels. For example, if we look at the red cohort, the counterfactual Y_0 is the average trend from the blue and purple cohorts (trend projection) but shifted to the level of the red cohort (level adjustment). This is why we see TWFE as a Difference-in-Differences method. It also does the trend projection and level adjustment, but it works for multiple time periods and multiple units (in the 2 units by 2 period case, TWFE and DiD are equivalent).

2) Death: Failures over Effect Heterogeneity

As we just saw, DiD and TWFE has its merits. It can estimate counterfactuals quite well, accommodating both time and unit specific variations. This makes it a powerful causal inference technique. But if this was all there was to it, we wouldn't need this chapter, as this is very much covered in Part I of this book. What happened recently is that many academics noticed that extending the 2 by 2 DiD to multiple periods with TWFE is not as straightforward as we first thought. In fact, **TWFE, in its usual formulation, turns out to be biased in many real life applications**. This event caused a wave of revisions in multiple studies in economics that relied on this technique. To understand all of that, the best way to start is by stating the underlying assumption.

For simplicity sake let's consider the FE model without the time effects:

$$y_{it} = \tau D_{it} + \gamma_i + e_{it}$$

We can group the assumption this model makes into two groups

1. Functional Form Assumptions:
   - No heterogeneous effects in time (constant effects);
   - Linearity in the covariates;
   - Additive fixed effects.
2. Strict Exogeneity
   - Parallel trends
   - No anticipation
   - No unobserved time varying confounders
   - Past treatment don't affect current outcome (no carryover)
   - Past outcome don't affect current treatment (no feedback)

Here, we will stick to the functional form assumptions. Linearity in the covariates is very well known and applies to all linear regression models. But, as we saw with the chapter on Double/Debiased Machine Learning, we can easily relax that assumption with a machine learning model. This means that we can relax this assumption if we wish to do so. As for the additive fixed effect, this is not a too restrictive assumption, so it doesn't cause a lot of problems. The one I want to focus on (and which generated a lot of fuss) is the one about no heterogeneous effects in time.

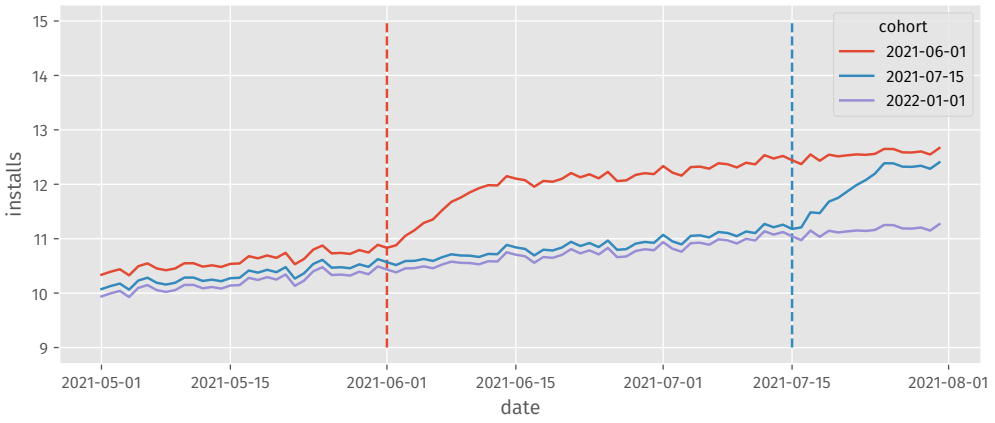
Treatment Effect Heterogeneity in Time

If you ever worked with marketing or tech, you know things take time to mature. If you launch a new feature, it will take time for users to get used to it. Similarly, if you start a marketing campaign, the effect of that campaign won't be instantaneous. It will mature over time and perhaps bring new users even after the campaign is over. This is **not** the pattern that we had in install data we've seen earlier. There, installs jumped up instantaneously, at the moment the cohort is treated. What happens if we change that to be more in line with what we see in reality. Namely, let's make it so that the ATT is still 1,

but now, it takes 10 days to mature (so it will be 0.1 at the first treatment day, 0.2 at the second treatment day and so on, until it reaches 1 on the 10th day). Also, I'll reduce the size of the time and unit effects so that the overall trend is easier to see.

```
date = pd.date_range("2021-05-01", "2021-07-31", freq="D")
cohorts = pd.to_datetime(["2021-06-01", "2021-07-15", "2022-01-01"]).date
units = range(1, 100+1)

np.random.seed(1)

df_heter = pd.DataFrame(dict(
    date = np.tile(date.date, len(units)),
    unit = np.repeat(units, len(date)),
    cohort = np.repeat(np.random.choice(cohorts, len(units)), len(date)),
    unit_fe = np.repeat(np.random.normal(0, 5, size=len(units)), len(date)),
    time_fe = np.tile(np.random.normal(size=len(date)), len(units)),
    week_day = np.tile(date.weekday, len(units)),
    w_seas = np.tile(abs(5-date.weekday) % 7, len(units)),
)).assign(
    trend = lambda d: (d["date"] - d["date"].min()).dt.days/70,
    day = lambda d: (d["date"] - d["date"].min()).dt.days,
    treat = lambda d: (d["date"] >= d["cohort"]).astype(int),
).assign(
    y0 = lambda d: 10 + d["trend"] + 0.2*d["unit_fe"] + 0.05*d["time_fe"] +
      ↪ d["w_seas"]/50,
).assign(
    y1 = lambda d: d["y0"] + np.minimum(0.1*(np.maximum(0,
      ↪ (d["date"] - d["cohort"]).dt.days)), 1)
).assign(
    tau = lambda d: d["y1"] - d["y0"],
    installs = lambda d: np.where(d["treat"] == 1, d["y1"], d["y0"])
)
```

```
plt.figure(figsize=(10,4))
[plt.vlines(x=cohort, ymin=9, ymax=15, color=color, ls="dashed") for color,
  ↪ cohort in zip(["C0", "C1"], cohorts)]
sns.lineplot(
    data=(df_heter
          .groupby(["cohort", "date"])["installs"]
          .mean()
          .reset_index()),
    x="date",
    y = "installs",
    hue="cohort",
);
```



What we see is that the installs still reach the same level as they did before, but it takes some time (10 days) for that. This seems reasonable right? Most of the data we see in real life works like that, with effects taking some time to mature. Ok, so let's run TWFE model on this data to see what happens.

```
formula = f"""installs ~ treat + C(date) + C(unit)"""

twfe_model = smf.ols(formula, data=df_heter).fit()

print("Estimated Effect: ", twfe_model.params["treat"])
print("True Effect: ", df_heter.query("treat==1")["tau"].mean())
```

```
Estimated Effect:  0.7867708225724858
True Effect:  0.8544117647058823
```

First, notice that the true ATT is no longer 1. That is because it will be smaller in the first few periods. Second, and most importantly, we can see is that the **estimated ATT from TWFE is not recovering the true ATT** anymore. To put it simply:TWFE is biased. But why is that? We have parallel trends, no anticipation and all the other strict exogeneity assumptions here. So what is going on?

The first step towards understanding what is happening is to realize that TWFE can actually be decomposed into multiple 2 by 2 Diff-in-Diffs. In our example, that would be one that compares early treated to never treated, late treated against never treated, early treated against late treated (with late treated serving as the control) and late treated against early treated (with early treated being the control):

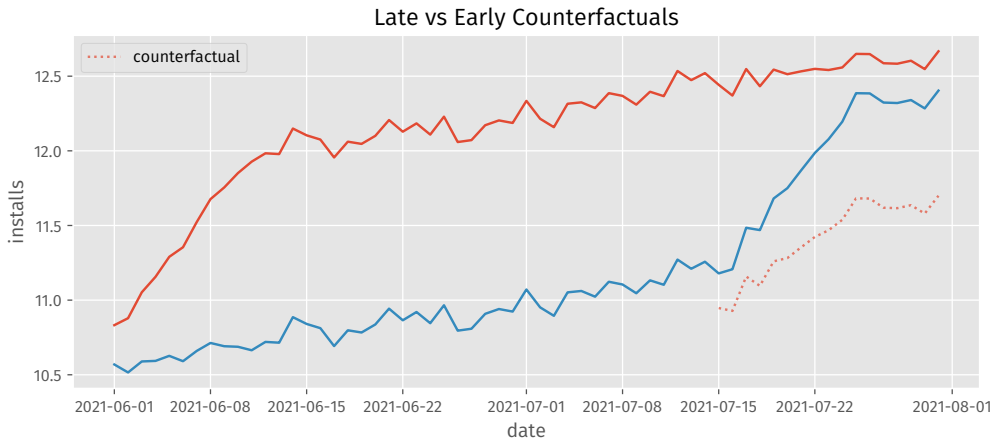
```
g_plot_data = (df_heter
               .groupby(["cohort", "date"])["installs"]
               .mean()
               .reset_index()
               .astype({"cohort":str}))
```

```
fig, axs = plt.subplots(2, 2, figsize=(15,8), sharex=True, sharey=True)

def plot_comp(df, ax, exclude_cohort, name):

    palette=dict(zip(map(str, cohorts), ["C0", "C1", "C2"]))

    sns.lineplot(
        data=df.query(f"cohort != '{exclude_cohort}'"),
        x="date",
        y="installs",
        hue="cohort",
        palette=palette,
        legend=None,
        ax=ax
    )

    sns.lineplot(
        data=df.query(f"cohort == '{exclude_cohort}'"),
        x="date",
        y = "installs",
        hue="cohort",
        palette=palette,
        alpha=0.2,
        legend=None,
        ax=ax
    )

    ax.set_title(name)

plot_comp(g_plot_data, axs[0,0], cohorts[1], "Early vs Never")
plot_comp(g_plot_data, axs[0,1], cohorts[0], "Late vs Never")

plot_comp(g_plot_data[g_plot_data["date"]<=cohorts[1]], axs[1,0], cohorts[-1],
  ↪ "Early vs Late")
plot_comp(g_plot_data[g_plot_data["date"]>cohorts[0]], axs[1,1], cohorts[-1],
  ↪ "Late vs Early")

plt.tight_layout();
```

The first three comparisons are no reason for concern, mostly because what they use as control is very well behaved. However, the fourth comparison, late vs early, is problematic. Notice that this comparison uses the early treated as a control. Also notice that this early treated control has a weird behavior. It climbs up sharply at the beginning. That is a reflection of our ATT not being instantaneous, but instead taking 10 days to mature. Intuitively, we can see that this will mess up the estimation of the counterfactual trend in the DiD, making it steeper than it should be. To visualize that, let's plot the estimated counterfactual Y_0 for the late treated in this 4th group.

```
late_vs_early = (df_heter
                 [df_heter["date"].astype(str)>="2021-06-01"]
                 [lambda d: d["cohort"].astype(str)<="2021-08-01"])


formula = f"""installs ~ treat + C(date) + C(unit)"""

twfe_model = smf.ols(formula, data=late_vs_early).fit()

late_vs_early_pred = (late_vs_early
                      .assign(**{"installs_hat_0": twfe_model.predict(
                        ↪ late_vs_early.assign(**{"treat":0}))})
                      .groupby(["cohort", "date"])
                      [["installs", "installs_hat_0"]]
                      .mean()
                      .reset_index())


plt.figure(figsize=(10,4))
plt.title("Late vs Early Counterfactuals")
sns.lineplot(
```

```
    data=late_vs_early_pred,
    x="date",
    y = "installs",
    hue="cohort",
    legend=None
)

sns.lineplot(
    data=(late_vs_early_pred
          [late_vs_early_pred["cohort"].astype(str) == "2021-07-15"]
          [lambda d: d["date"].astype(str) >= "2021-07-15"]
         ),
    x="date",
    y ="installs_hat_0",
    alpha=0.7,
    color="C0",
    ls="dotted",
    label="counterfactual"
);
```



Just like we said, the counterfactuals have a much steeper trend than it should have. It is capturing that quick increase in the beginning of the early treatment and projecting that trend onto the late treated.

More technically, it can be shown (Goodman-Bacon, 2019) that, even under strict exogeneity (parallel trends, no anticipation…), if the cohorts have the same size, the TWFE estimator will converge to

$$plim_{x\to\infty}\hat{\tau}^{TWFE} = VWATT - \Delta ATT$$

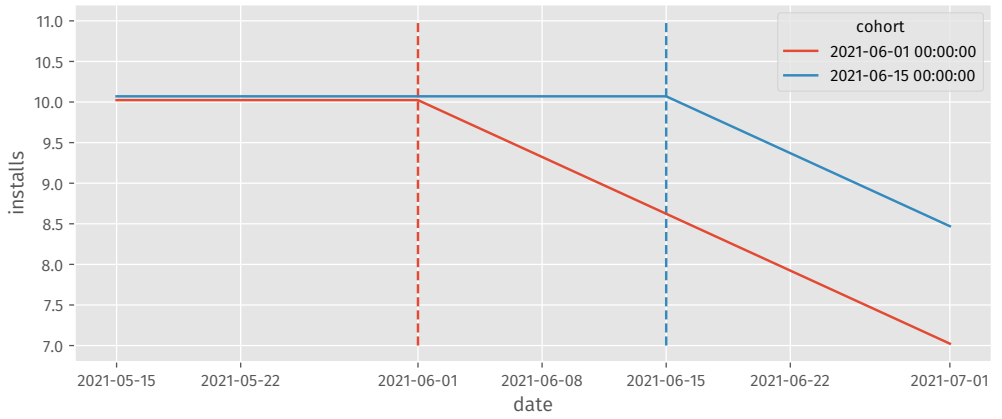
The first term is the variance weighted ATT from multiple DiD comparisons like the ones we saw earlier. This is what we want. However, there is also that extra ΔATT term there. This represents how much the ATT changes over time and it is what bias our estimate.

Looking at this term, we can see that we will have downward bias if the magnitude of the effect increases with time (like in our example) or upward bias if the magnitude of the effect decreases with time.

In the example above, we saw that the effect from TWFE was smaller than the true ATT. But the situation can be even more extreme. I think it is worth going over one last example to see that this bias can be so strong as to even reverse the signal of the true ATT. Let's consider a very simple process, with only two cohorts. Here, the treatment effect will be negative and decreasing by 0.1 every day. I've also removed all time fixed effects and the trend so we can really see what is going on.

```
date = pd.date_range("2021-05-15", "2021-07-01", freq="D")
cohorts = pd.to_datetime(["2021-06-01", "2021-06-15"])
units = range(1, 100+1)

np.random.seed(1)

df_min = pd.DataFrame(dict(
    date = np.tile(date, len(units)),
    unit = np.repeat(units, len(date)),
    cohort = np.repeat(np.random.choice(cohorts, len(units)), len(date)),
    unit_fe = np.repeat(np.random.normal(0, 5, size=len(units)), len(date)),
)).assign(
    trend = 0,
    day = lambda d: (d["date"] - d["date"].min()).dt.days,
    treat = lambda d: (d["date"] >= d["cohort"]).astype(int),
).assign(
    y0 = lambda d: 10 - d["trend"] + 0.1*d["unit_fe"]
).assign(
    y1 = lambda d: d["y0"] - 0.1*(np.maximum(0, (d["date"] - d["cohort"]).dt
      ↪ .days))
).assign(
    tau = lambda d: d["y1"] - d["y0"],
    installs = lambda d: np.where(d["treat"] == 1, d["y1"], d["y0"])
)
```

```
plt.figure(figsize=(10,4))
[plt.vlines(x=cohort, ymin=7, ymax=11, color=color, ls="dashed") for color,
  ↪ cohort in zip(["C0", "C1"], cohorts)]
sns.lineplot(
    data=(df_min
          .groupby(["cohort", "date"])["installs"]
          .mean()
          .reset_index()),
    x="date",
    y = "installs",
    hue="cohort",
);
```



Looking at the plot above, we can clearly see that the ATT is negative right? The correct counterfactual should be a straight line at about 11. However, if we run the TWFE estimator, we get a positive effect!

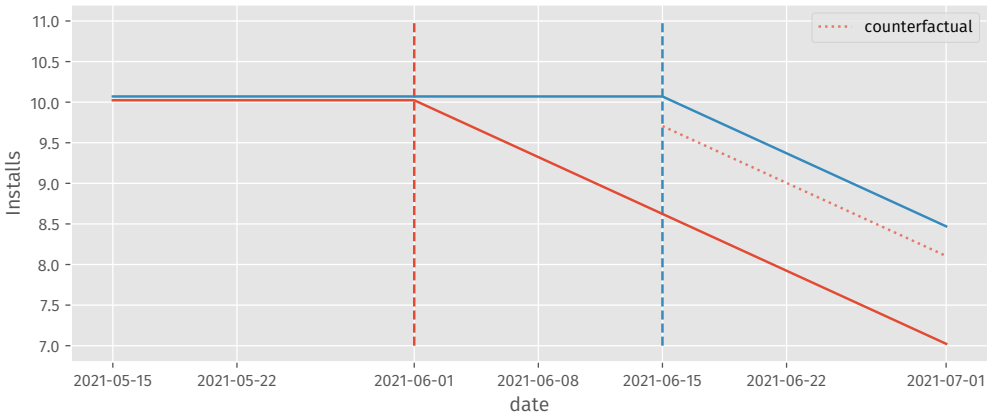
```
formula = f"""installs ~ treat + C(date) + C(unit)"""

twfe_model = smf.ols(formula, data=df_min).fit()

twfe_model.params["treat"]
```

```
0.04999999999998828
```

Once again, to see what is going on, focus your attention on the comparison where the early treated cohort serves as the control for the late treated. Remember that, like DiD, TWFE adjusts the trend from the control group to the level of the treated group, so the counterfactual should reflect that.

```
df_pred = df_min.assign(**{"installs_hat_0": twfe_model.predict(df_min.assign(
  ↪ **{"treat":0}))})

plt.figure(figsize=(10,4))
[plt.vlines(x=cohort, ymin=7, ymax=11, color=color, ls="dashed") for color,
  ↪ cohort in zip(["C0", "C1"], cohorts)]
sns.lineplot(
    data=(df_pred
          [(df_pred["cohort"].astype(str) > "2021-06-01") & (df_pred["date"]
            ↪ .astype(str) >= "2021-06-15")]
          .groupby(["cohort", "date"])["installs_hat_0"]
          .mean()
          .reset_index()),
    x="date",
    y = "installs_hat_0",
    alpha=0.7,
```

```
    ls="dotted",
    color="C0",
    label="counterfactual",
)
sns.lineplot(
    data=(df_pred
          .groupby(["cohort", "date"])["installs"]
          .mean()
          .reset_index()),
    x="date",
    y = "installs",
    hue="cohort",
    legend=None
)
plt.ylabel("Installs");
```



Notice how the counterfactual level is pushed down from where it should be. The fact that the early treated group effect is decreasing pushed this level down from 10 to about 9.5. Not only that, the counterfactual also adjusts for a downward trend, which should not be there. It is pretty clear from the plot that the right counterfactual should be a straight line at 10, but instead, it is a downward sloping line, since that is what it sees in the early treated group it uses as a control.

The end result is that the counterfactual Y_0 is actually below the Y_1 outcome, leading to a positive impact estimation. This is pretty awkward. By simply looking at the plot, we can pretty much see where the right counterfactual should be (the straight line at 10). Still, TWFE cannot recover that simply because it uses the early treated as a control for the late treated.

Event Study Design

Just to get it out of the way, I know someone might think we can easily solve this problem by what is called an event study design, where we add one dummy for each period before

and after the treatment. In this case, we replace the original TWFE model by

$$Y_{i,t} = \tau^{-K} D_{i,t}^{<-K} + \sum_{k=-K}^{-2} \tau^{lead} D_{i,t}^{k} + \sum_{k=0}^{L} \gamma_k^{lag} D_{i,t}^{k} + \gamma_k^{L+} D_{i,t}^{>L} \gamma_i + e_{it}$$

where $D_{i,t}^{k} = 1\{t - \text{Cohort}_i = k\}$ is an event study dummy that is 1 if the unit is k periods away from the treatment and 0 otherwise. I know this looks messy and complicated, but it is actually very simple once we look at it through some code. All we need to do is create a `relative_days` column which measures how far away is the unit from the period where treatment starts.

```
df_min_rel = (df_min
              .assign(relative_days = (df_min["date"] - df_min["cohort"]).dt
                ↪ .days))

df_min_rel.head()
```

	date	unit	cohort	unit_fe	trend	day
0	2021-05-15	1	2021-06-15	-3.435864	0	0
1	2021-05-16	1	2021-06-15	-3.435864	0	1
2	2021-05-17	1	2021-06-15	-3.435864	0	2
3	2021-05-18	1	2021-06-15	-3.435864	0	3
4	2021-05-19	1	2021-06-15	-3.435864	0	4

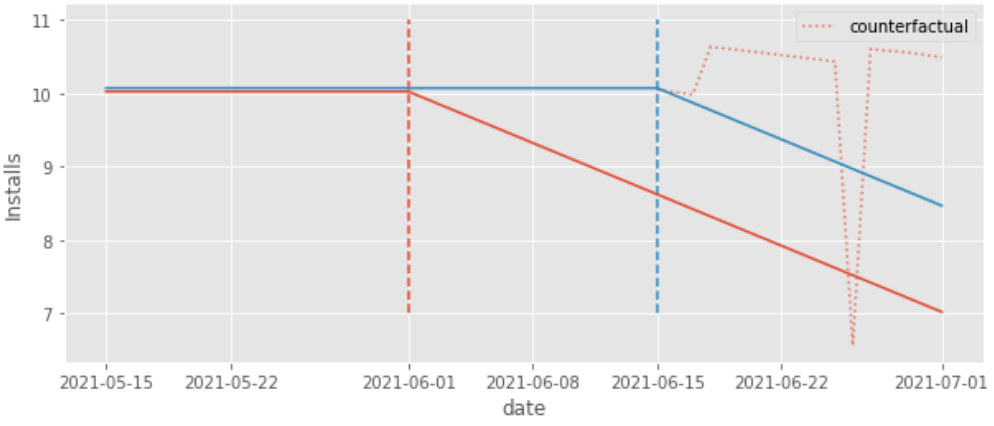
	treat	y0	y1	tau	installs	relative_days
0	0	9.656414	9.656414	0.0	9.656414	-31
1	0	9.656414	9.656414	0.0	9.656414	-30
2	0	9.656414	9.656414	0.0	9.656414	-29
3	0	9.656414	9.656414	0.0	9.656414	-28
4	0	9.656414	9.656414	0.0	9.656414	-27

Then, we can pass that column as a category so our model will estimate the expected number of installs for each period relative to the treatment. We can then define the effect as the extra expected number of installs compared to relative day -1, which is the last day prior to the treatment.

We might think that this formulation would capture the time heterogeneity in the ATT and solve all our issues. Unfortunately, that is not the case. If we try it out and plot the counterfactuals, we can see that they are far from where they should intuitively be (the horizontal line at 11).

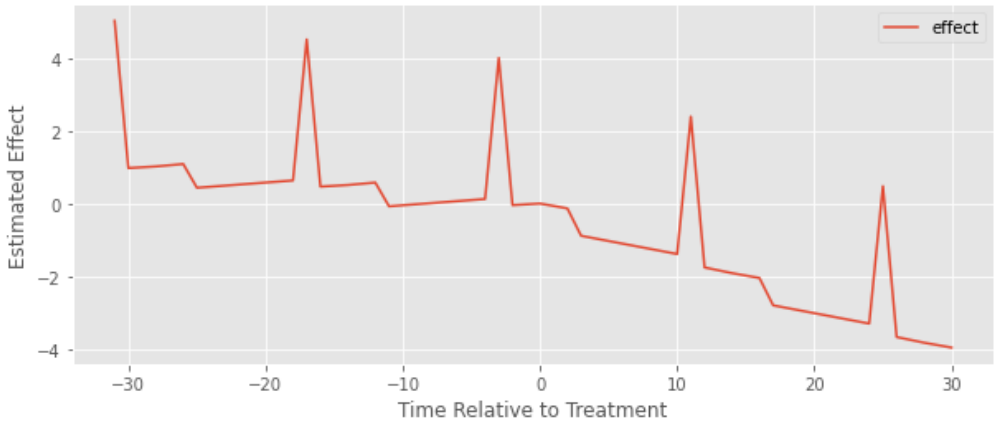
```
# remove the intercept, otherwise effects will be relative to relative day -30
formula = f"installs ~ -1 + C(relative_days) + C(date) + C(unit)"

twfe_model = smf.ols(formula, data=df_min_rel).fit()
```

```
df_pred = df_min_rel.assign(
    installs_hat_0=twfe_model.predict(df_min_rel.assign(relative_days=-1))
)

plt.figure(figsize=(10,4))
[plt.vlines(x=cohort, ymin=7, ymax=11, color=color, ls="dashed") for color,
  ↪ cohort in zip(["C0", "C1"], cohorts)]
sns.lineplot(
    data=(df_pred
          [(df_pred["cohort"].astype(str) > "2021-06-01") & (df_pred["date"]
            ↪ .astype(str) >= "2021-06-15")]
          .groupby(["cohort", "date"])["installs_hat_0"]
          .mean()
          .reset_index()),
    x="date",
    y = "installs_hat_0",
    alpha=0.7,
    ls="dotted",
    color="C0",
    label="counterfactual",
)
sns.lineplot(
    data=(df_pred
          .groupby(["cohort", "date"])["installs"]
          .mean()
          .reset_index()),
    x="date",
    y = "installs",
    hue="cohort",
    legend=None
)
plt.ylabel("Installs");
```



These counterfactuals are a bit better, though. We can see that they fall above the realized Y_1 outcome. As a result, we will at least estimate a negative effect, as we should. To see that, we can plot the estimated effects by first extracting the parameter associated with each dummy and then subtracting from them the parameter associated with relative day -1 (the baseline).

```
effects = (twfe_model.params[twfe_model.params.index.str
  ↪ .contains("relative_days")]
           .reset_index()
           .rename(columns={0:"effect"})
           .assign(relative_day=lambda d: d["index"].str.extract(r'\[(.*)\]')
             ↪ .astype(int))
           # set the baseline to period -1
           .assign(effect = lambda d: d["effect"] -
             ↪ d.query("relative_day==-1")["effect"].iloc[0]))

# effects
effects.plot(x="relative_day", y="effect", figsize=(10,4))
plt.ylabel("Estimated Effect")
plt.xlabel("Time Relative to Treatment");
```



We can kind of see it is a bit better because at least the estimated effect after the treatment is 1) mostly negative and 2) mostly decreasing. But there are these weird spikes and what looks like a positive pre-treatment effect, which obviously doesn't make any sense.

The problem here is the same we've been discussing. Since we have different timing in the treatment, the early treated gets used as a control for the late treated units, which causes the model to estimate a very weird counterfactual trend. The bottom line is that adding time relative to treatment dummies does not solve the problem, but what does, exactly?

3) Enlightenment: A Flexible Functional Form

There are good news and bad news. First, the good news: we've identified the problem as being one related to the functional form, so we can fix it by correcting that form. Namely, we've said over and over that this specific bias in TWFE comes from the time heterogeneous effects. This can happen, among many other reasons, because the effect takes some time to mature (ex: it might take 10 days for a marketing campaign to reach its full potential). In other words, the functional form of traditional TWFE is simply not flexible enough to capture this heterogeneity, leading to the sort of bias we've discussed. Like in most cases, knowing the problem is already a long way in the direction of finding a solution.

In the end of the last section, we saw how simply allowing for a different effect on each time period relative to the treatment (event study design) was not enough. Even though that didn't work, the intention behind it was good. It did make the model more flexible, but not in a way it solved the problem. We need to think of another way to make the model even more flexible than that.

To do that, let's go back to our original example, when we were trying to model the number of incremental installs that rolling out a new feature (treatment) brought us. We saw that the simple TWFE model does not work here:

$$Installs_{it} = \tau D_{it} + \gamma_i + \theta_t + e_{it}$$

More than that, we know it doesn't work because it is too restrictive. It forces the effect to be the same \(\tau_{it}=\tau \ \forall i, t\), that is, it forces time homogeneity. If that is the problem, an easy fix would be to simply allow for a different effect for each time and unit.

$$Installs_{it} = \sum_{i=0}^{N}\sum_{t=0}^{T}\tau_{it}D_{it} + \gamma_i + \theta_t + e_{it}$$

This would be equivalent to running the formula below:

```
installs ~ treat:C(unit):C(date) + C(unit) + C(date)
```

Unfortunately, we can't fit that. It would have more parameters than we have data points. Since we are interacting date and unit, we would have one treatment effect parameter for each unit for each time period $T * N$. But this is exactly the number of samples we have! OLS wouldn't even run here.

Ok, now, we need to reduce the number of treatment effect parameters of the model. To achieve that, we can think about a way of somehow grouping units. If we think about this a bit, we can see a very natural way to group units: by cohort! We know that the effect in an entire cohort follows the same pattern over time. So, a natural improvement on that impractical model above is to allow the effect to change by cohort instead of units:

$$Installs_{it} = \sum_{g=0}^{G}\sum_{t=0}^{T}\tau_{gt}D_{it} + \gamma_i + \theta_t + e_{it}$$

where `G` is the total number of cohorts and `g` marks each individual cohort. That model has a much reasonable number of treatment effect parameters ($T * G$) since G is usually much smaller than N. Now, we can finally run it.

```
formula = f"""installs ~ treat:C(cohort):C(date) + C(unit) + C(date)"""

# for nicer plots latter on
df_heter_str = df_heter.astype({"cohort": str, "date":str})

twfe_model = smf.ols(formula, data=df_heter_str).fit()
```

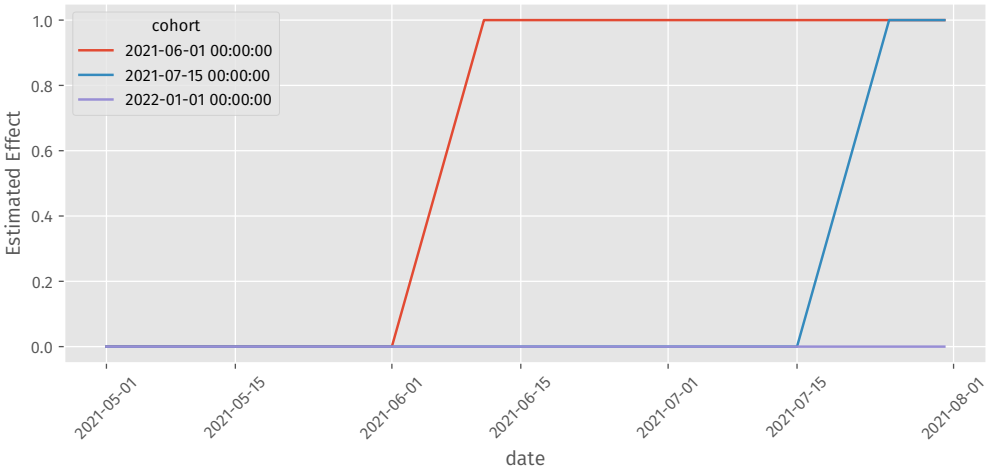
To see if this model works, we can make counterfactual predictions for Y_0 by forcing `treat` to be zero for everyone. Then, we can estimate the effect by taking the observed outcome for the treatment, which is Y_1, and subtract $\hat{Y}_0$ from it. Let's see if that matches the true ATT.

```
df_pred = (df_heter_str
           .assign(**{"installs_hat_0": twfe_model.predict(df_heter_str.assign(
             ↪ **{"treat":0}))})
           .assign(**{"effect_hat": lambda d: d["installs"] -
             ↪ d["installs_hat_0"]}))

print("Number of param.:", len(twfe_model.params))
print("True Effect: ", df_pred.query("treat==1")["tau"].mean())
print("Pred. Effect: ", df_pred.query("treat==1")["effect_hat"].mean())
```

```
Number of param.: 467
True Effect:  0.8544117647058823
Pred. Effect:  0.8544117647058977
```

It does! We finally managed to make a model which is flexible enough to capture the time heterogeneity, which allowed us to estimate the correct treatment effect! Another cool thing we can do is extract the estimated effects by time and cohort and plot them. In this case, because we know how the data was generated, we know what to expect. Namely, the effect for each cohort must be zero before treatment, then 1 10 days after treatment and a line climbing up from zero to 1 in the days between the treatment and 10 days after it.

```
effects = (twfe_model.params[twfe_model.params.index.str.contains("treat")]
           .reset_index()
           .rename(columns={0:"param"})
           .assign(cohort=lambda d: d["index"].str.extract(
             ↪ r'C\(cohort\)\[(.*)\]:'))
           .assign(date=lambda d: d["index"].str.extract(r':C\(date\)\[(.*)\]'))
           .assign(date=lambda d: pd.to_datetime(d["date"]), cohort=lambda d:
             ↪ pd.to_datetime(d["cohort"])))

plt.figure(figsize=(10,4))
sns.lineplot(data=effects, x="date", y="param", hue="cohort")
plt.xticks(rotation=45)
plt.ylabel("Estimated Effect");
```



Once again, the plot above matches our expectations of the effects. They follow the exact pattern we described above.

This is already very good, but we can do even better. First, notice how that model has a huge number of parameters. Since we have 100 units and about 92 days in our data, we

know that 192 of those parameters are the unit and time effects. Still, that leaves us with more than 250 treatment effect parameters.

If we assume a zero effect before the treatment (no anticipation), we can reduce the number of parameters by dropping the dates before treatment from the interaction term.

$$Installs_{it} = \sum_{g=0}^{G} \sum_{t=g}^{T} \tau_{gt} D_{it} + \gamma_i + \theta_t + e_{it}$$

Also, we can drop from the interaction the control cohorts, since its effect before the treatment is always zero

$$Installs_{it} = \sum_{G=q}^{g} \sum_{t=g}^{T} \tau_{gt} D_{it} + \gamma_i + \theta_t + e_{it}$$

where cohorts before $g < q$ are defined as the control cohorts.

Notice however that this is tricky to implement with formulas, so we have to do some feature engineering first. Namely, we will create the cohort dummies by hand, making one column which is 1 when the cohort is `2021-06-01` and 0 otherwise and another column which is 1 when the cohort is `2021-07-15` and 0 otherwise. Also, we will create a date column for cohort `2021-06-01` which collapses all dates before that cohort date to a `control` category. We can do a similar thing with the dates from the `2021-07-15` cohort. Here is what this looks like in code.

```
def feature_eng(df):
    return (
        df
        .assign(date_0601 = np.where(df["date"]>="2021-06-01", df["date"],
          ↪ "control"),
                date_0715 = np.where(df["date"]>="2021-07-15", df["date"],
                  ↪ "control"),)
        .assign(cohort_0601 = (df["cohort"]=="2021-06-01").astype(float),
                cohort_0715 = (df["cohort"]=="2021-07-15").astype(float))
    )

formula = f"""installs ~ treat:cohort_0601:C(date_0601)
                       + treat:cohort_0715:C(date_0715)
                       + C(unit) + C(date)"""

twfe_model = smf.ols(formula, data=df_heter_str.pipe(feature_eng)).fit()
```

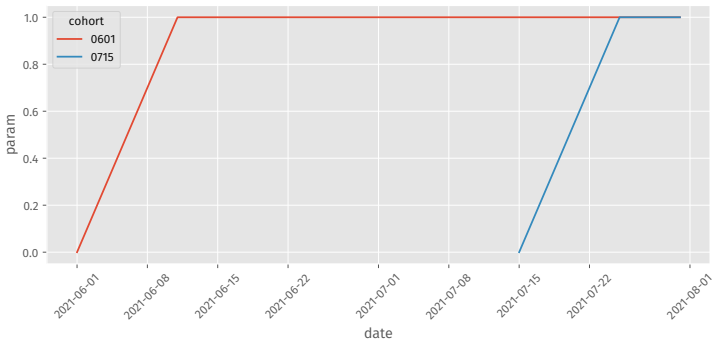
If we now make counterfactual predictions like before, we can see that the estimated effect is still matching the true effect perfectly. The gain here is that we have a much simpler model, with only about 80 treatment effect parameters (remember that 192 of those parameters are the time and unit fixed effect).

```
df_pred = (df_heter
           .assign(**{"installs_hat_0": twfe_model.predict(df_heter_str
                                                           .pipe(feature_eng)

      ↪ .assign(**{"treat":0}))})
           .assign(**{"effect_hat": lambda d: d["installs"] -
            ↪ d["installs_hat_0"]}))


print(len(twfe_model.params))
print("True Effect: ", df_pred.query("treat==1")["tau"].mean())
print("Pred Effect: ", df_pred.query("treat==1")["effect_hat"].mean())
```

```
271
True Effect:  0.8544117647058823
Pred Effect:  0.8544117647059067
```

Plotting the treatment effect parameters, we can see how we've removed those from the control cohort and those from the dates before the cohort is treated.

```
effects = (twfe_model.params[twfe_model.params.index.str.contains("treat")]
           .reset_index()
           .rename(columns={0:"param"})
           .assign(cohort=lambda d: d["index"].str.extract(r':cohort_(.*):'),
                   date_0601=lambda d: d["index"].str.extract(
                     ↪ r':C\(date_0601\)\[(.*)\]'),
                   date_0715=lambda d: d["index"].str.extract(
                     ↪ r':C\(date_0715\)\[(.*)\]'))
           .assign(date=lambda d: pd.to_datetime(d["date_0601"].combine_first(
            ↪ d["date_0715"]), errors="coerce")))


plt.figure(figsize=(10,4))
sns.lineplot(data=effects.dropna(subset=["date"]), x="date", y="param",
  ↪ hue="cohort")
plt.xticks(rotation=45);
```



Notice that we could go even further, since the effect for both cohorts follow the same shape. Namely, we could restrict the model to have the same effect for both cohorts, changing only over time. To do that, we would need to create a column to represent days after the treatment, just like in the event study design. It would be something like this:

```
days_after_treat=1(date>cohort)*(date - cohort)
```

Then, we would interact it with the treatment indicator

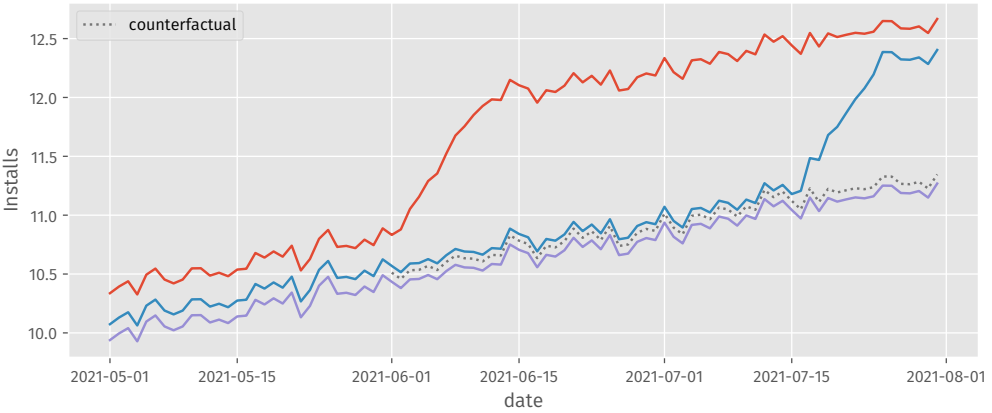
```
installs ~ treat:C(days_after_treat) + C(unit) + C(date)
```

However, I think we can stop here. Not allowing heterogeneity by cohort is, more often than not, a bad idea, since treatment effect tends to change over calendar time, not only over time since the treatment. For example, it could be that, after some time, competitors copy our feature making it no longer a strong customer attractor as it once was. In that case, the effect of the new feature on installs would diminish over time.

One last thing we should do besides showing the effects over time is to plot the counterfactuals to see if they are in a place that feels right. I know this is not a very scientific validation of our model, but trust me, it helps a tone. So here it is.

```
twfe_model_wrong = smf.ols("installs ~ treat + C(date) + C(unit)",
                           data=df_pred).fit()


df_pred = (df_pred
           .assign(**{"installs_hat_0_wrong": twfe_model_wrong.predict(df_pred
             ↪ .assign(**{"treat":0}))}))


plt.figure(figsize=(10,4))
sns.lineplot(
    data=(df_pred
          [(df_pred["cohort"].astype(str) > "2021-06-01") & (df_pred["date"]
            ↪ .astype(str) >= "2021-06-01")]
          .groupby(["date"])["installs_hat_0"]
          .mean()
          .reset_index()),
    x="date",
    y = "installs_hat_0",
    ls="dotted",
    color="C3",
    label="counterfactual",
)

sns.lineplot(
    data=(df_pred
          .groupby(["cohort", "date"])["installs"]
          .mean()
          .reset_index()),
```

```
    x="date",
    y = "installs",
    hue="cohort",
    legend=None
)

plt.ylabel("Installs");
```



As we can see, the counterfactual Y_0 prediction falls right where we think it should fall, that is, pretty close to the control cohort. This is very comforting. We know that the TWFE model is estimating the treatment effect as $Y - \hat{Y}_0$. That is, it is simply comparing the outcome from the treated cohorts to that counterfactual. Since the counterfactual seems OK, we can rest assured that the treatment effect is also probably OK.

That is the good news, but don't think I forgot about the bad news I've promised you. The thing is that, although we've solved the functional form problem with TWFE, there is still an arguably bigger problem with DiD and TWFE, which is related to its independence assumption.

When using DiD and TWFE, we often invoke the parallel trends assumption, without really thinking about what exactly that assumption means. Sadly, the parallel trends assumption is much more restrictive and less plausible than most people realize . But since this chapter is already too big, I think it's fine to end it here, where we can still enjoy the taste of a small victory for DiD.

Key Concepts

I think it is safe to say that we've finally managed not only to understand, but to correct the functional form problem with TWFE. We traced the problem down to its roots (time heterogeneity) and fixed it by allowing for more flexibility. We can now grab ourselves a drink and relax, knowing that TWFE is once again safe to use. Or is it?!

We can never forget that TWFE (and DiD more generally) is a mix of **both functional form and independence assumptions**. In this chapter, we've only tackled the functional form problems, but there is still a big elephant in the room: the parallel trend assumption. Parallel trends is the independence assumption DiD makes. This is very well known. But I feel we don't really understand what this assumption implies. We just invoke it out of thin air as if that would make it true. Unfortunately, the parallel trend assumption requires much more than what most people realize. In the next chapters, we will see why that is and what or if we can do something about it.

Reference

It took a (very) long time to write this chapter. The recent econometric literature has been sprouting with new ideas and insights on the problems related to DiD and how to solve them. There are multiple angles we can see those problems, which in turn leads to multiple approaches we can leverage to solve them. Brace yourselves because the reference list here will be long (and probably not very well organized).

First, I took a lot from *Difference-in-Differences with Variation in Treatment Timing*, by Andrew Goodman-Bacon. His diagnosis of the problem is very neat and intuitive. Not to mention it comes with very nice pictures that gave me a clear understanding on how to understand what was happening with DiD. Some of the pictures in this chapter are almost a direct copy from those made by Goodman-Bacon.

The second major source of inspiration was *Difference-in-Differences with Multiple Time Periods*, by Pedro H. C. Sant'Anna and Brantly Callaway. Notice that the Callaway and Sant'Anna solution to this problem goes on a different route than the one we took. Still, their solution shines a lot of light on the DiD problem, making it easy to understand. The paper aside, Pedro has a very nice blog post showing the problems with TWFE. The Data Generating Process on that blog post heavily inspired the ones that I've used here. I've basically translated Pedro's code from R to Python. Pedro was also very kind in helping

me with some questions I had about the DiD assumptions. He has another very interesting article tackling the problem of adding covariates to the DiD models, which we didn't cover here because the chapter was already too long. The name of the paper is *Doubly Robust Difference-in-Differences Estimators* and I strongly recommend you read it if you plan to add covariates to your model.

Last, but definitely not least, the functional form fix we employed here was inspired on *Estimating dynamic treatment effects in event studies with heterogeneous treatment effects*, by Sun and Abraham and on *Two-Way Fixed Effects, the Two-Way Mundlak Regression, and Difference-in-Differences Estimators*, by Jeffrey Wooldridge. Although I've loved what Callaway and Sant'Anna did on their paper, their solution is a bit more complicated to implement. In contrast, Sun, Abraham and Wooldridge's solution involves nothing more than cleverly playing with interactions in a TWFE regression model, making it super easy to implement using nothing more than `statsmodels` and some formulas.

Aside from the papers mentioned above, I've also relied on presentations given at the DiD Study Group, organized by Taylor Wright on Youtube. It is much easier to understand the articles mentioned above if you first watch the authors talk about them. I'm also very thankful for professor Yiqing Xu for tying all of this together in his amazing Causal Inference with Panel Data lecture series, also available on Youtube.

Finally, keep in mind that I too am also learning, so, by all means, if you find something preposterous, open an issue and I'll address it to the best of my efforts.

CHAPTER

25

Synthetic Difference-in-Differences

In previous chapters, we looked into both Difference-in-Differences and Synthetic Control methods for identifying the treatment effect with panel data (data where we have multiple units observed across multiple time periods). It turns out we can merge both approaches into a single estimator. This new Synthetic Difference-in-Differences estimation procedure manages to exploit advantages of both methods while also increasing the precision (decreasing the error bars) of the treatment effect estimate.

We will discuss Synthetic Difference-in-Differences mostly in the case of **block treatment assignment**. This means we observe multiple units across time and, at the **same** time, some units are treated while other units remain untreated. We can visualize this by a matrix of treatment assignments D, where the **columns of the matrix are units** and **rows of the matrix are time periods**.

$$D = \begin{bmatrix} 0 & 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & 0 & \dots & 0 & 0 \\ \vdots & & & & & \\ 0 & 0 & 0 & \dots & 1 & 1 \\ 0 & 0 & 0 & \dots & 1 & 1 \end{bmatrix}$$

To make things more concrete, let's follow along with the example of estimating the impact of Proposition 99 in California's Cigarette consumption. In this case, we only have one treated unit, California, that gets treated (passed Proposition 99) at some point in time (November 1988, to be precise). If we say California is the last column of the matrix, we get something like this:

$$D = \begin{bmatrix} 0 & 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & 0 & \dots & 0 & 0 \\ \vdots & & & & & \\ 0 & 0 & 0 & \dots & 0 & 1 \\ 0 & 0 & 0 & \dots & 0 & 1 \end{bmatrix}$$

Notice that here we are only talking about the case where all treated units get the treatment **at the same point in time**. In the end, we will discuss how to handle **staggered addoption treatment assignment**, where the treatment is gradually rolled out to units, causing them to get it at different points in time. The only thing we'll require in that design is that, once a unit is treated, it doesn't roll back to being untreated.

Back to the simple case, where we have all units treated at the same time, we can simplify the treatment assignment matrix to four blocks, each represented by another matrix. In general, as we move down the matrix, we are moving forward in time. We will also group the treated units to the right of the matrix. As a result, the first block in our matrix (top left) correspond to the control units prior to the treatment period; the second one (top right) corresponds to the treated units prior to the treatment period; the third block (bottom left) contains the control units after the treatment period and the fourth block (bottom right) is the treated unit after the treatment period. The treatment indicator is zero everywhere except for the block with the treated units after the treatment period.

$$D = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$$

This assignment matrix will lead to the following outcome matrix:

$$Y = \begin{bmatrix} \boldsymbol{Y}_{pre,co} & \boldsymbol{Y}_{pre,tr} \\ \boldsymbol{Y}_{post,co} & \boldsymbol{Y}_{post,tr} \end{bmatrix}$$

Again, notice how the post-treatment period is on the bottom and the treated units are to the right.

In our example of estimating the effect of Proposition 99, the outcome Y is cigarette sales. We use pre and $post$ to represent the period prior and after the treatment respectively and co and tr to represent the control and treated unit respectively.

We will use the above matrix representation whenever we talk about estimating the synthetic control weights, but there is also another data representation which is useful, especially if we are talking about DiD. In this representation, we have a table with 5 columns: one representing the units, one representing the time periods, the outcome column and two boolean columns flagging the treated units and the treatment period. The number of rows in this table is the number of units N times the number of periods T. Here you can see what it looks like for the Proposition 99 data:

```
import numpy as np
import pandas as pd
from toolz import curry, partial
import seaborn as sns
from matplotlib import pyplot as plt
import statsmodels.formula.api as smf
import cvxpy as cp

import warnings
warnings.filterwarnings('ignore')
```

```
from matplotlib import style
style.use("ggplot")

pd.set_option('display.max_columns', 10)
```

```
data = (pd.read_csv("data/smoking.csv")[["state", "year", "cigsale",
  ↪ "california", "after_treatment"]]
        .rename(columns={"california": "treated"})
        .replace({"state": {3: "california"}}))

data.head()
```

	state	year	cigsale	treated	after_treatment
0	1	1970	89.800003	False	False
1	1	1971	95.400002	False	False
2	1	1972	101.099998	False	False
3	1	1973	102.900002	False	False
4	1	1974	108.199997	False	False

```
data.query("state=='california'").query("year.between(1986, 1990)")
```

	state	year	cigsale	treated	after_treatment
78	california	1986	99.699997	True	False
79	california	1987	97.500000	True	False
80	california	1988	90.099998	True	False
81	california	1989	82.400002	True	True
82	california	1990	77.800003	True	True

If we want to go from this table to the matrix representation we discussed earlier, all we have to do is pivot the table by time (year) and unit (state). We'll be going back and forth between these two representations, as one is more convenient for DiD and the other, for Synthetic Controls estimation.

```
data_piv = data.pivot("year", "state", "cigsale")
data_piv = data_piv.rename(columns={c: f"state_{c}" for c in data_piv.columns if
  ↪ c != "california"})

data_piv.head()[["state_1", "state_2", "state_4", "state_38", "state_39",
  ↪ "california"]].round()
```

state year	state_1	state_2	state_4	state_38	state_39	california
1970	90.0	100.0	125.0	106.0	132.0	123.0
1971	95.0	104.0	126.0	105.0	132.0	121.0
1972	101.0	104.0	134.0	109.0	140.0	124.0
1973	103.0	108.0	138.0	110.0	141.0	124.0
1974	108.0	110.0	133.0	112.0	146.0	127.0

In terms of potential outcomes, we can go back to the outcome matrix to review the causal inference goal here. Since the treatment is only rolled out to the treated unit after the treatment period, we observe the potential outcome Y_0 everywhere in the matrix, except for the bottom right block.

$$Y = \begin{bmatrix} \boldsymbol{Y}(0)_{pre,co} & \boldsymbol{Y}(0)_{pre,tr} \\ \boldsymbol{Y}(0)_{post,co} & \boldsymbol{Y}(1)_{post,tr} \end{bmatrix}$$

Our goal is to estimate the $ATT = \boldsymbol{Y}(1)_{post,tr} - \boldsymbol{Y}(0)_{post,tr}$. For that, we need to somehow estimate the missing potential outcome $\boldsymbol{Y}(0)_{post,tr}$. In words, we need to know what would have happened to the treated unit in the post-treatment period had it not been treated. With that in mind, a good place to start is by reviewing both Diff-in-Diff and Synthetic Control. At first, it looks like they are each doing very different things to estimate that missing potential outcome. Combining them sure feels weird. However, both methods have more in common than we might think.

Diff-in-Diff Revisited

In the Diff-in-Diff chapter, we got the treatment effect by estimating the following linear model.

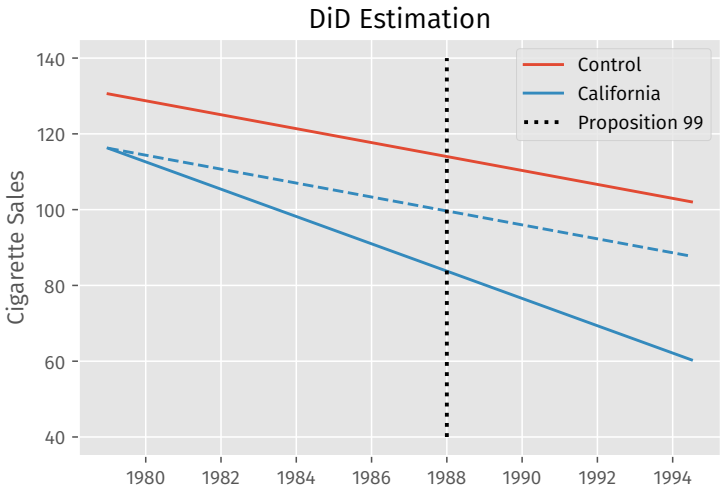
$$Y_{it} = \beta_0 + \beta_1 Post_t + \beta_2 Treated_i + \beta_3 Treated_i Post_t + e_{it}$$

Where `post` is a time dummy indicating that the period is after the treatment and `treated` is a unit dummy marking units as being part of the treated group. If we estimate this model in the California example, we get -27.34 as the estimated ATT, indicating a strong negative effect of Proposition 99. This would mean that per capita consumption of cigarettes fell by 27 packs due to Proposition 99.

```
did_model = smf.ols("cigsale ~ after_treatment*treated", data=data).fit()
att = did_model.params["after_treatment[T.True]:treated[T.True]"]
print("DiD ATT: ", att.round(3))
```

```
DiD ATT:  -27.349
```

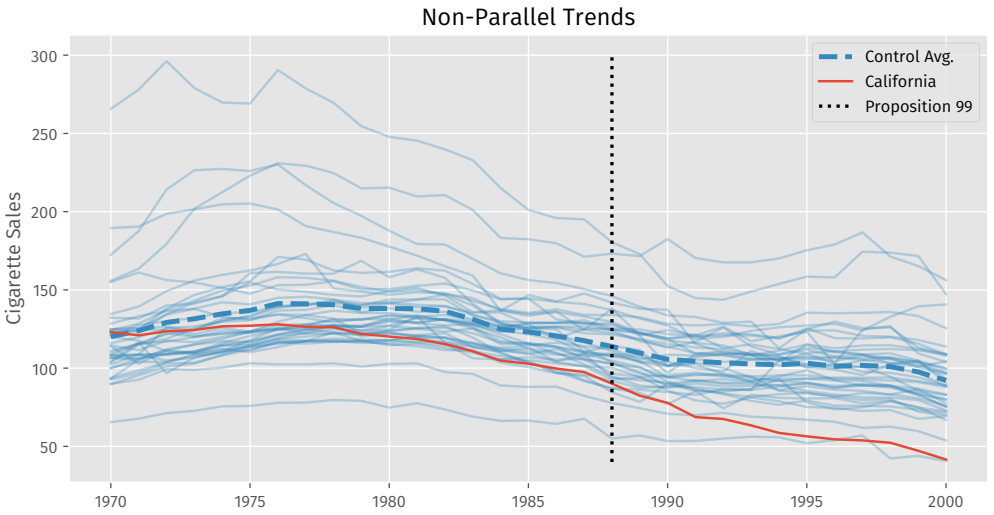
```
pre_year = data.query("~after_treatment")["year"].mean()
post_year = data.query("after_treatment")["year"].mean()

pre_control_y = did_model.params["Intercept"]
post_control_y = did_model.params["Intercept"] + did_model
  ↪ .params["after_treatment[T.True]"]

pre_treat_y = did_model.params["Intercept"] + did_model
  ↪ .params["treated[T.True]"]

post_treat_y0 = post_control_y + did_model.params["treated[T.True]"]

post_treat_y1 = post_treat_y0 + did_model
  ↪ .params["after_treatment[T.True]:treated[T.True]"]

plt.plot([pre_year, post_year], [pre_control_y, post_control_y], color="C0",
  ↪ label="Control")
plt.plot([pre_year, post_year], [pre_treat_y, post_treat_y0], color="C1",
  ↪ ls="dashed")
plt.plot([pre_year, post_year], [pre_treat_y, post_treat_y1], color="C1",
  ↪ label="California")

plt.vlines(x=1988, ymin=40, ymax=140, linestyle=":", lw=2,
  ↪ label="Proposition 99", color="black")
plt.title("DiD Estimation")
plt.ylabel("Cigarette Sales")
plt.legend();
```



This estimate should be taken with a grain of salt, though. We know that Diff-in-Diff requires the trend in the control group to be equal to that of the treated group in the absence of the treatment. Formally, $E[Y(0)_{post,co} - Y(0)_{pre,co}] = E[Y(0)_{post,tr} -$

$Y(0)_{pre,tr}$]. This is an untestable assumption, but looking at the pre-treatment trend of California (the treated unit) and the other states, we can get a feeling for how plausible it is. Specifically, we can see that the trend in `cigsale` for California is not parallel to the other states, at least in the pre-treatment periods. Cigarette sales in California are decreasing faster than the average of the control states, even prior to the treatment. If this trend extends beyond the pre-treatment period, the DiD estimator will be downward biased, meaning that the true effect is actually less extreme than the one we've estimated above.

```
plt.figure(figsize=(10,5))
plt.plot(data_piv.drop(columns=["california"]), color="C1", alpha=0.3)
plt.plot(data_piv.drop(columns=["california"]).mean(axis=1), lw=3, color="C1",
  ↪ ls="dashed", label="Control Avg.")
plt.plot(data_piv["california"], color="C0", label="California")
plt.vlines(x=1988, ymin=40, ymax=300, linestyle=":", lw=2,
  ↪ label="Proposition 99", color="black")
plt.legend()
plt.ylabel("Cigarette Sales")
plt.title("Non-Parallel Trends");
```



The problem of non-parallel trends is where Synthetic Control comes into play in the Synthetic Diff-in-Diff model. But we are getting ahead of ourselves. Regardless of DiD being a valid model for the data above, what is interesting about it is that we can recast it into the Two-Way Fixed-Effects formulation. To frame DiD like this, we fit unit (α_i) and time (β_t) averages, alongside the treatment indicator.

$$\hat{\tau}^{did} = \underset{\mu,\alpha,\beta,\tau}{argmin}\left\{ \sum_{i=1}^{N}\sum_{t=1}^{T}\left(Y_{it} - (\mu + \alpha_i + \beta_t + \tau D_{it}\right)^2 \right\}$$

In this formulation, the unit effects capture the difference in intercepts for each unit while the time effects capture the general trend across both treated and control units. To implement this, we could either add time and unit dummies to the model or demean the data. In this process, we subtract the average across both time and units from both treatment and outcome variables:

$$\ddot{Y}_{it} = Y_{it} - \bar{Y}_i - \bar{Y}_t \ddot{D}_{it} = D_{it} - \bar{D}_i - \bar{D}_t$$

Where, $\bar{X}_i$ is the average across all time periods for unit i and $\bar{X}_t$ is the average across all units for time t:

$$\ddot{Y}_{it} = Y_{it} - T^{-1}\sum_{t=0}^{t=T} Y_{it} - N^{-1}\sum_{i=0}^{i=N} Y_{it} \ddot{D}_{it} = D_{it} - T^{-1}\sum_{t=0}^{t=T} D_{it} - N^{-1}\sum_{i=0}^{i=N} D_{it}$$

After demeaning, a simple regression of the outcome on the treatment indicator (`treat*post`) yields the difference in difference estimator.

```
@curry
def demean(df, col_to_demean):
    return df.assign(**{col_to_demean: (df[col_to_demean]
                                        - df.groupby("state")[col_to_demean]
                                          ↪ .transform("mean")
                                        - df.groupby("year")[col_to_demean]
                                          ↪ .transform("mean"))})

formula = f"""cigsale ~ treat"""
mod = smf.ols(formula,
              data=data
              .assign(treat = data["after_treatment"]*data["treated"])
              .pipe(demean(col_to_demean="treat"))
              .pipe(demean(col_to_demean="cigsale")))

mod.fit().summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	-119.1647	0.333	-358.379	0.000	-119.817	-118.512
treat	-27.3491	4.283	-6.385	0.000	-35.753	-18.945

As you can see, we get the exact same parameter as before. After all, both approaches are simply different ways of looking at the same DiD estimator. However, the reason this formulation is much more interesting for our purpose is that it allows us to see how DiD is actually quite similar to Synthetic Controls. Take a very close look at the TWFE formulation above. Notice that it is a regression problem with time effects and unit effects. But notice how there are no weights in the optimization objective. That is the main difference between Diff-in-Diff and Synthetic Controls, as we will see shortly.

Synthetic Controls Revisited

In the canonical Synthetic Control estimator, we find unit (state) weights that minimize the difference between the pre-treated outcome of the treated unit and the weighted average of the pre-treated outcome of the control units (in a setting with no covariates). We also constrain the weights to be all positive and sum up to one. To find those weights, we solve the following optimization problem:

$$\hat{w}^{sc} = \underset{w}{\text{argmin}} \; ||\bar{y}_{pre,tr} - \boldsymbol{Y}_{pre,co}\boldsymbol{w}_{co}||_2^2 \text{s.t} \;\; \sum w_i = 1 \text{ and } \; w_i > 0 \; \forall \; i$$

where the outcome $\boldsymbol{Y}_{pre,co}$ is a T_{pre} by N_{co} matrix, where the columns are the units and the rows are the time periods. $\boldsymbol{w}_{co}$ is a N_{co} by 1 column vector, with one entry for each unit. Finally, $\bar{\boldsymbol{y}}_{pre,tr}$ is a T_{pre} by 1 column vector, where each entry is the time average of the treated units in the pre-treatment period. This is why we sometimes call Synthetic Control a horizontal regression. In most regression problems, the units are the rows of the matrix, but here they are the columns. Hence, we are regressing the average outcome of the treated units on the control units.

Once we find the weights that solve the problem above, we can multiply them by the control units at all time periods to get a synthetic control for the treated unit:

$$\boldsymbol{y}_{sc} = \boldsymbol{Y}_{co}\hat{\boldsymbol{w}}^{sc}$$

The idea here is that $\boldsymbol{y}_{post,sc}$ is a good estimator for our missing potential outcome $Y(0)_{post,tr}$. If that is the case, the ATT is simply the average of the treated unit in the post-treatment period minus the average of the synthetic control, also in the post treatment period.

$$\hat{\tau} = \bar{y}_{post,tr} - \bar{y}_{post,sc}$$

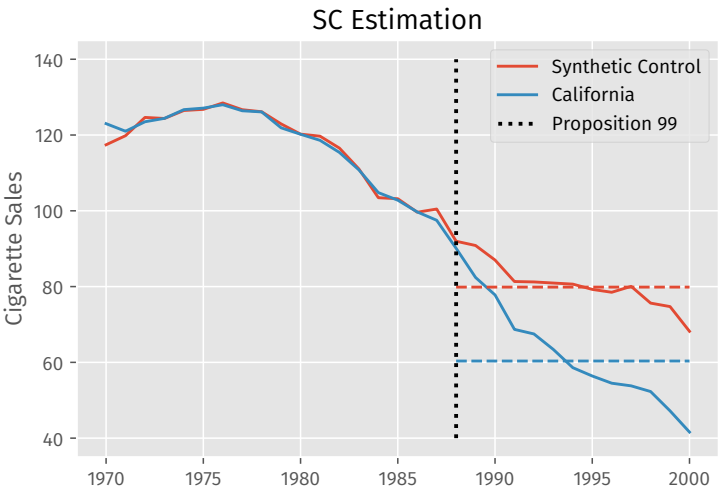
```
from sc import SyntheticControl

sc_model = SyntheticControl()

y_co_pre = data.query("~after_treatment").query("~treated").pivot("year",
  ↪ "state", "cigsale")
y_tr_pre = data.query("~after_treatment").query("treated")["cigsale"]

sc_model.fit(y_co_pre, y_tr_pre)
sc_weights = pd.Series(sc_model.w_, index=y_co_pre.columns, name="sc_w")

sc = data.query("~treated").pivot("year", "state", "cigsale").dot(sc_weights)

att = data.query("treated")["cigsale"][sc.index > 1988].mean() - sc[sc.index >
  ↪ 1988].mean()

print("SC ATT: ", att.round(4))
```

```
SC ATT:  -19.5136
```

This estimate is much smaller than the one we got with Diff-in-Diff. Synthetic Controls can accommodate non-parallel pre-treatment trends much better, so it is not susceptible to the same bias as Diff-in-Diff. Rather, the process of baking a Synthetic Control enforces parallel trends, at least in the pre-treatment period. As a result, the estimate we get is much smaller and much more plausible.

We can visualize this estimation process by plotting the realized outcome for California alongside the outcome of the synthetic control. We also plot as dashed lines the post intervention average of both California and the synthetic control. The difference between these lines is the estimated ATT.

```
plt.plot(sc, label="Synthetic Control")
plt.plot(sc.index, data.query("treated")["cigsale"], label="California",
  ↪ color="C1")

calif_avg = data.query("treated")["cigsale"][sc.index > 1988].mean()
sc_avg = sc[sc.index > 1988].mean()

plt.hlines(calif_avg, 1988, 2000, color="C1", ls="dashed")
plt.hlines(sc_avg, 1988, 2000, color="C0", ls="dashed")

plt.title("SC Estimation")
plt.ylabel("Cigarette Sales")
plt.vlines(x=1988, ymin=40, ymax=140, linestyle=":", lw=2,
  ↪ label="Proposition 99", color="black")
plt.legend();
```



Interestingly enough, we can also recast the Synthetic Control estimator as solving the following optimization problem, which is quite similar to the Two-Way Fixed-Effects

formulation we used for Diff-in-Diff

$$\hat{\tau}^{sc} = \underset{\beta,\tau}{argmin}\left\{ \sum_{i=1}^{N}\sum_{t=1}^{T}(Y_{it} - \beta_t - \tau D_{it})^2 \hat{w}_i^{sc} \right\}$$

where the $\hat{w}_i^{sc}$ weights for the control units are estimated from the optimization problem we saw earlier. For the treated unit, the weights are simply $1/N_{tr}$ (uniform weighting).

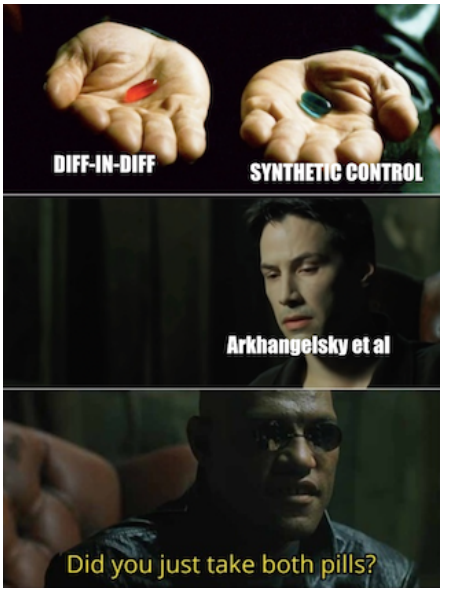
Notice the difference between SC and DiD here. First, Synthetic Control adds unit weights $\hat{w}_i^{sc}$ to the equation. Second, we have time fixed effects β_t but no unit fixed effect α_i, nor an overall intercept μ.

To verify that these two formulations are actually equivalent, here is the code for it, which yields the exact same ATT estimate.

```
@curry
def demean_time(df, col_to_demean):
    return df.assign(**{col_to_demean: (df[col_to_demean]
                                        - df.groupby("year")[col_to_demean]
                                          ↪ .transform("mean"))})

data_w_cs_weights = data.set_index("state").join(sc_weights).fillna(1/len(
  ↪ sc_weights))

formula = f"""cigsale ~ -1 + treat"""

mod = smf.wls(formula,
              data=data_w_cs_weights
              .assign(treat =
      ↪ data_w_cs_weights["after_treatment"]*data_w_cs_weights["treated"])
              .pipe(demean_time(col_to_demean="treat"))
              .pipe(demean_time(col_to_demean="cigsale")),
              weights=data_w_cs_weights["sc_w"]+1e-10)

mod.fit().summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
treat	-19.5136	13.289	-1.468	0.142	-45.586	6.559

We just saw how the two approaches, SC and DiD, are actually closely related. Now, we are ready to talk about Synthetic Diff-in-Diff. As you can probably guess, we will just add weights to the DiD estimator or unit fixed-effects to the Synthetic Control estimator.



Synthetic Diff-in-Diff

Before we jump right into the Synthetic Diff-in-Diff estimator, let me just reproduce the same equations we saw earlier for SC and DiD, which will ease the comparison.

$$\hat{\tau}^{sc} = \underset{\beta,\tau}{argmin}\left\{ \sum_{i=1}^{N}\sum_{t=1}^{T}(Y_{it} - \beta_t - \tau D_{it})^2 \hat{w}_i^{sc} \right\}$$

$$\hat{\tau}^{did} = \underset{\mu,\alpha,\beta,\tau}{argmin}\left\{ \sum_{i=1}^{N}\sum_{t=1}^{T}(Y_{it} - (\mu + \alpha_i + \beta_t + \tau D_{it})^2 \right\}$$

Next, just like I promised, we can easily merge the equations above into one which will contain elements from both of them:

$$\hat{\tau}^{sdid} = \underset{\mu,\alpha,\beta,\tau}{argmin}\left\{ \sum_{i=1}^{N}\sum_{t=1}^{T}(Y_{it} - (\mu + \alpha_i + \beta_t + \tau D_{it})^2 \hat{w}_i^{sdid}\hat{\lambda}_t^{sdid}) \right\}$$

As you can see, we've added back the α_i unit fixed effects. We've also kept the unit weights $\hat{w}_i$. But there is something new, which is the time weights $\hat{\lambda}_t$. Don't worry. There is nothing fancy about them. Remember how the unit weights w_i minimized the difference between the control units and the average of treated units? In other words, we use them to match the pre-trend of the treated and control groups. The time weight does the same thing, but for the periods. That is, it minimizes the difference between the pre and post-treated periods for the controls.

$$\hat{\lambda}^{sdid} = \underset{\lambda}{\text{argmin}} \; ||\bar{y}_{post,co} - (\lambda_{pre}\boldsymbol{Y}_{pre,co} + \lambda_0)||_2^2 \text{s.t} \;\; \sum \lambda_t = 1 \text{ and } \; \lambda_t > 0 \; \forall \; t$$

Again, $\boldsymbol{Y}_{pre,co}$ is a T_{pre} by N_{co} matrix of outcomes where the rows represent time periods and the columns represent the units. But now $\bar{y}_{post,co}$ is a 1 by N_{co} row vector, where

each entry is the time average outcome for that control unit in the post-treatment period. Finally, $\boldsymbol{\lambda}_{pre}$ is a 1 by T_{pre} row vector, with one entry for each pretreatment period. Another way to see this is by noticing that the unit weights w were post-multiplying the outcome matrix $\boldsymbol{Y}_{pre,co}\boldsymbol{w}_{co}$. This means we were regressing the average outcome **for each unit** of the treated group on the outcome of the units in the control group. Now, we are flipping that problem on its head, regressing the average outcome **for each post-treatment time period** of the control group on the outcome of the same control units, but in the pre-treatment period.

As for the time weights in the post-treated periods, we just set them to one over the number of post-treated periods $1/T_{post}$ (again, doing uniform weighting). Notice that we also have an intercept λ_0. We do this to allow the post-treatment period to be above or below all the pre-treatment periods, which is the case in many applications with a clear positive or negative trend.

If all of this seems a bit abstract, maybe code will help you understand what is going on.

```
def fit_time_weights(data, outcome_col, year_col, state_col, treat_col,
  ↪ post_col):

        control = data.query(f"~{treat_col}")

        # pivot the data to the (T_pre, N_co) matrix representation
        y_pre = (control
                 .query(f"~{post_col}")
                 .pivot(year_col, state_col, outcome_col))

        # group post-treatment time period by units to have a (1, N_co) vector.
        y_post_mean = (control
                       .query(f"{post_col}")
                       .groupby(state_col)
                       [outcome_col]
                       .mean()
                       .values)

        # add a (1, N_co) vector of 1 to the top of the matrix, to serve as the
              ↪ intercept.
        X = np.concatenate([np.ones((1, y_pre.shape[1])), y_pre.values], axis=0)

        # estimate time weights
        w = cp.Variable(X.shape[0])
        objective = cp.Minimize(cp.sum_squares(w@X - y_post_mean))
        constraints = [cp.sum(w[1:]) == 1, w[1:] >= 0]
        problem = cp.Problem(objective, constraints)
        problem.solve(verbose=False)

        # print("Intercept: ", w.value[0])
        return pd.Series(w.value[1:], # remove intercept
```

```
                          name="time_weights",
                          index=y_pre.index)
```

The first thing we do in this code is to filter out the treated group. Then, we pivot the pre-treated data so that we have the matrix $\boldsymbol{Y}_{pre,co}$. Next, we group the post-treatment data to get the average outcome for each control unit in the post-treatment period. We then add a row full of ones to the top of $\boldsymbol{Y}_{pre,co}$, which will serve as the intercept. Finally, we regress $\bar{\boldsymbol{y}}_{post,co}$ on the pre-treated periods (the rows of $\boldsymbol{Y}_{pre,co}$) to get the time weights λ_t. Notice how we add the constraints to have the weights sum up to 1 and be non-negative. Finally, we toss the intercept away and store the time weights in a series.

Here is the result we get by running the code above to find the time weights in the Proposition 99 problem. Notice that all periods except for 1986, 87 ans 88 get zero weights. This means that a weighted average of only the last 3 periods is enough to balance pre and post treatment periods.

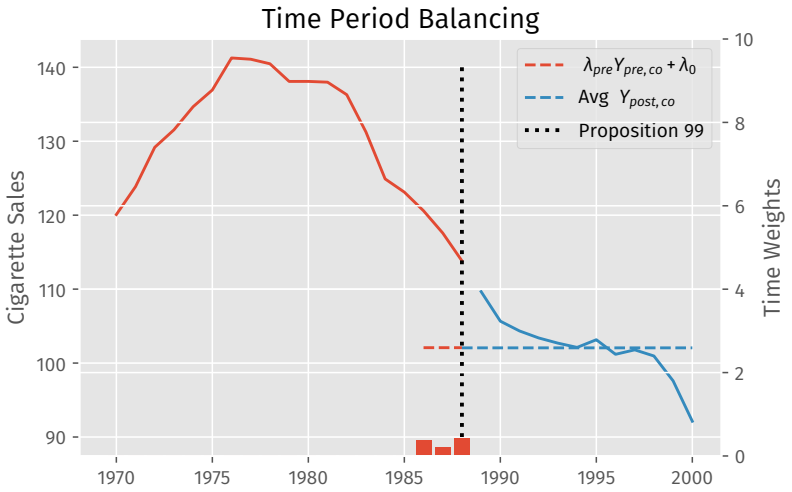
```
time_weights = fit_time_weights(data,
                                outcome_col="cigsale",
                                year_col="year",
                                state_col="state",
                                treat_col="treated",
                                post_col="after_treatment")

time_weights.round(3).tail()
```

```
year
1984   -0.000
1985   -0.000
1986    0.366
1987    0.206
1988    0.427
Name: time_weights, dtype: float64
```

To understand a bit more about the role of these weights, we can plot $\hat{\boldsymbol{\lambda}}_{pre}\boldsymbol{Y}_{pre,co} + \hat{\lambda}_0$ as a horizontal line in the pretreatment period that doesn't get zeroed out. Next to it, we plot the average outcome in the post-treatment period. Notice how they align perfectly. We also show the estimated time weights in red bars and in the secondary axis.

```
fig = plt.figure()
ax = fig.add_subplot(111)

ax.plot(data.query("~treated").query("~after_treatment")
  ↪ .groupby("year")["cigsale"].mean())
ax.plot(data.query("~treated").query("after_treatment")
  ↪ .groupby("year")["cigsale"].mean())

intercept = -15.023877689807628
ax.hlines((data.query("~treated").query("~after_treatment")
```

```
  ↪ .groupby("year")["cigsale"].mean() * time_weights).sum() - 15, 1986, 1988,
          color="C0", ls="dashed", label="""  $\lambda_{pre} Y_{pre, co} +
      ↪ \lambda_0$""")
ax.hlines(data.query("~treated").query("after_treatment")
  ↪ .groupby("year")["cigsale"].mean().mean(), 1988, 2000,
          color="C1", ls="dashed", label="""Avg  $Y_{post, co}$""")
ax.vlines(x=1988, ymin=90, ymax=140, linestyle=":", lw=2,
  ↪ label="Proposition 99", color="black")
plt.legend()

plt.title("Time Period Balancing")
plt.ylabel("Cigarette Sales");

ax2 = ax.twinx()
ax2.bar(time_weights.index, time_weights, label="$\lambda$")
ax2.set_ylim(0,10)
ax2.set_ylabel("Time Weights");
```



Now that we learned what are the time weights λ_t in the Synthetic Diff-in-Diff estimator and how to estimate them, let's turn our attention to the unit weights w_i. And no, unfortunately they are not just like the ones we get when using traditional Synthetic Controls. The first difference between them is that we also allow for an intercept w_0. We do this because we don't need the treated unit and synthetic control to have the same level anymore. Since we will throw DiD into the mix, we only need to make the synthetic control and treated unit have parallel trends.

The next difference is that we add a L_2 penalty to the weights. This helps non-zero weights to be more distributed across the control units, as opposed to having just a few of them contributing to the synthetic control. The L_2 penalty ensures we don't have very

big weights, which forces us to use more units.

$$\hat{w}^{sdid} = \underset{w}{\text{argmin}} \; ||\bar{y}_{pre,tr} - (\boldsymbol{Y}_{pre,co}\boldsymbol{w}_{co} + w_0)||_2^2 + \zeta^2 T_{pre}||\boldsymbol{w}_{co}||_2^2 \text{s.t} \;\; \sum w_i = 1 \text{ and } \; w_i > 0 \; \forall \; i$$

There is also this ζ^2 term, which is theoretically motivated, but very complicated to explain, so I will unfortunately leave it as a bit of a mystery. In the reference, you can check the original article, which explains them. We define it like this:

$$\zeta = (N_{tr} * T_{post})^{1/4} \sigma(\Delta_{it})$$

where Δ_{it} is the first difference in the outcomes $Y_{it} - Y_{i(t-1)}$ and $\sigma(\Delta_{it})$ is the standard deviation of this difference. Here is the code to compute it.

```
def calculate_regularization(data, outcome_col, year_col, state_col, treat_col,
  ↪ post_col):

    n_treated_post = data.query(post_col).query(treat_col).shape[0]

    first_diff_std = (data
                      .query(f"~{post_col}")
                      .query(f"~{treat_col}")
                      .sort_values(year_col)
                      .groupby(state_col)
                      [outcome_col]
                      .diff()
                      .std())

    return n_treated_post**(1/4) * first_diff_std
```

As for the unit weights, there is nothing particularly new in them. We can reuse a lot of the code from the function to estimate the time weights. We only need to be careful about the dimensions, since the problem is now upside down.

```
def fit_unit_weights(data, outcome_col, year_col, state_col, treat_col,
  ↪ post_col):

    zeta = calculate_regularization(data, outcome_col, year_col, state_col,
      ↪ treat_col, post_col)
    pre_data = data.query(f"~{post_col}")

    # pivot the data to the (T_pre, N_co) matrix representation
    y_pre_control = (pre_data
                     .query(f"~{treat_col}")
                     .pivot(year_col, state_col, outcome_col))

    # group treated units by time periods to have a (T_pre, 1) vector.
    y_pre_treat_mean = (pre_data
                        .query(f"{treat_col}")
                        .groupby(year_col)
                        [outcome_col]
                        .mean())
```

```
    # add a (T_pre, 1) column to the begining of the (T_pre, N_co) matrix to
        ↪ serve as intercept
    T_pre = y_pre_control.shape[0]
    X = np.concatenate([np.ones((T_pre, 1)), y_pre_control.values], axis=1)

    # estimate unit weights. Notice the L2 penalty using zeta
    w = cp.Variable(X.shape[1])
    objective = cp.Minimize(cp.sum_squares(X@w - y_pre_treat_mean.values) +
      ↪ T_pre*zeta**2 * cp.sum_squares(w[1:]))
    constraints = [cp.sum(w[1:]) == 1, w[1:] >= 0]

    problem = cp.Problem(objective, constraints)
    problem.solve(verbose=False)

    # print("Intercept:", w.value[0])
    return pd.Series(w.value[1:], # remove intercept
                     name="unit_weights",
                     index=y_pre_control.columns)
```

First, we calculate ζ using the function we defined earlier and filter out the post-treatment period. Next, we pivot the pre-treatment data to get the $\bar{y}_{pre,tr}$ matrix of outcomes. Then, we add a column full of ones to the beginning of the $\bar{y}_{pre,tr}$ matrix. This column will allow us to estimate the intercept. With all of that, we define the optimization objective, which includes the L_2 penalty on the weights. Finally, we toss the intercept away and store the estimated weights in a series.

If we use this code to estimate the unit weights in the Proposition 99 problem, here is the result we get for the first 5 states:

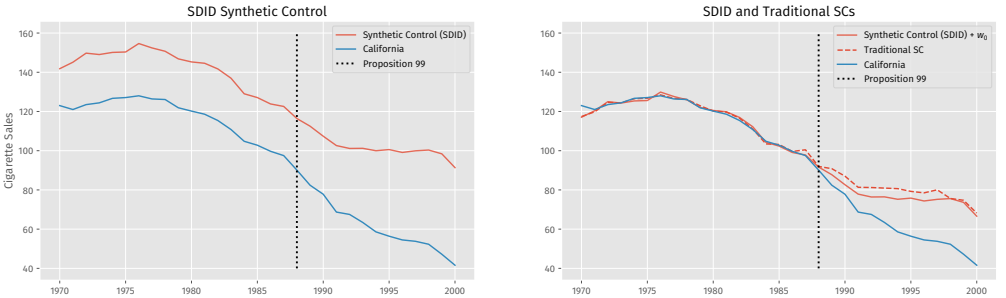
```
unit_weights = fit_unit_weights(data,
                                outcome_col="cigsale",
                                year_col="year",
                                state_col="state",
                                treat_col="treated",
                                post_col="after_treatment")

unit_weights.round(3).head()
```

```
state
1   -0.000
2   -0.000
4    0.057
5    0.078
6    0.070
Name: unit_weights, dtype: float64
```

These unit weights also define a synthetic control that we can plot alongside the outcome of California. We'll also plot the traditional synthetic control we've estimated earlier

alongside the one we've just estimated plus the intercept term. This will give us some intuition on what is going on and the difference between what we just did and traditional Synthetic Control.

```
intercept = -24.75035353644767
sc_did = data_piv.drop(columns="california").values @ unit_weights.values

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(18,5))

ax1.plot(data_piv.index, sc_did, label="Synthetic Control (SDID)", color="C0",
  ↪ alpha=.8)
ax1.plot(data_piv["california"], label="California", color="C1")
ax1.vlines(x=1988, ymin=40, ymax=160, linestyle=":", lw=2,
  ↪ label="Proposition 99", color="black")

ax1.legend()
ax1.set_title("SDID Synthetic Control")
ax1.set_ylabel("Cigarette Sales");

ax2.plot(data_piv.index, sc_did+intercept,
  ↪ label="Synthetic Control (SDID) + $w_0$", color="C0", alpha=.8)
ax2.plot(data_piv.index, sc, label="Traditional SC", color="C0", ls="dashed")
ax2.plot(data_piv["california"], label="California", color="C1")
ax2.vlines(x=1988, ymin=40, ymax=160, linestyle=":", lw=2,
  ↪ label="Proposition 99", color="black")
ax2.legend()
ax2.set_title("SDID and Traditional SCs");
```



As we can see in the first plot, the obvious difference is that this new synthetic control is no longer on top of California. That's because we've included an intercept, which allows the treated unit to be on an arbitrarily different level than its synthetic control. This new Synthetic Control method is built to have the same pretreatment trend as the treated unit, but not necessarily the same level.

In the second plot, we shift this new SC by adding back the intercept we've removed earlier. This puts it on top of the treated unit, California. For comparison, we show the traditional SC we've fitted earlier as the red dashed line. Notice that they are not the

same. This difference comes both from the fact that we allowed for an intercept and from the L_2 penalty, which pushed the weights towards zero.

Now that we have both time $\hat{\lambda}_t$ and unit $\hat{w}_t$ weights, we can proceed to running the Diff-in-Diff part of the Synthetic DiD estimator. For this part, it is better if we work with the data in the format of a table with N by T rows, where we have columns for the states, the years, the outcome, the post-treatment indicator and the treated unit indicator. To that table, we will add the time and unit weights. Since the time weight is in a series with a time index and the unit weights is in another series with unit index, we can simply join everything together.

```
def join_weights(data, unit_w, time_w, year_col, state_col, treat_col,
  ↪ post_col):
    return (
        data
        .set_index([year_col, state_col])
        .join(time_w)
        .join(unit_w)
        .reset_index()
        .fillna({time_w.name: 1 / len(pd.unique(data.query(f"{post_col}")[
          ↪ year_col])),
                 unit_w.name: 1 / len(pd.unique(data.query(f"{treat_col}")[
                   ↪ state_col]))})
        .assign(**{"weights": lambda d: (d[time_w.name] * d[unit_w.name]).round(
          ↪ 10)})
        .astype({treat_col: int, post_col: int}))
```

This joining process will leave `null` for the unit weights in the treated group and for the time weights in the post-treatment period. Fortunately, because we use uniform weighting in both cases, it is pretty easy to fill out those `nulls`. For the time weights, we fill with the average of the post-treatment dummy, which will be $1/T_{post}$; for the unit weights, we fill with the average of the treated dummy, which will be $1/N_{tr}$. Finally, we multiply both weights together.

Here is the result we get by running this code on the Proposition 99 data:

```
did_data = join_weights(data, unit_weights, time_weights,
                        year_col="year",
                        state_col="state",
                        treat_col="treated",
                        post_col="after_treatment")

did_data.head()
```

	year	state	cigsale	treated	after_treatment	time_weights	unit_weights	weights
0	1970	1	89.800003	0	0	-4.600034e-14	-1.360835e-16	0.0
1	1971	1	95.400002	0	0	-4.582315e-14	-1.360835e-16	0.0
2	1972	1	101.099998	0	0	-5.274190e-14	-1.360835e-16	0.0
3	1973	1	102.900002	0	0	-5.766356e-14	-1.360835e-16	0.0
4	1974	1	108.199997	0	0	-5.617979e-14	-1.360835e-16	0.0

```
data["after_treatment"].mean()
```

```
0.3870967741935484
```

```
1/len(data.query("after_treatment==1")["year"].unique())
```

```
0.08333333333333333
```

Finally, all we have to do is estimate a Diff-in-Diff model with the weights we've just defined. The parameter estimate associated with the interaction term for the post-treatment period and treated dummy will be the Synthetic Difference-in-Differences estimate for the ATT.

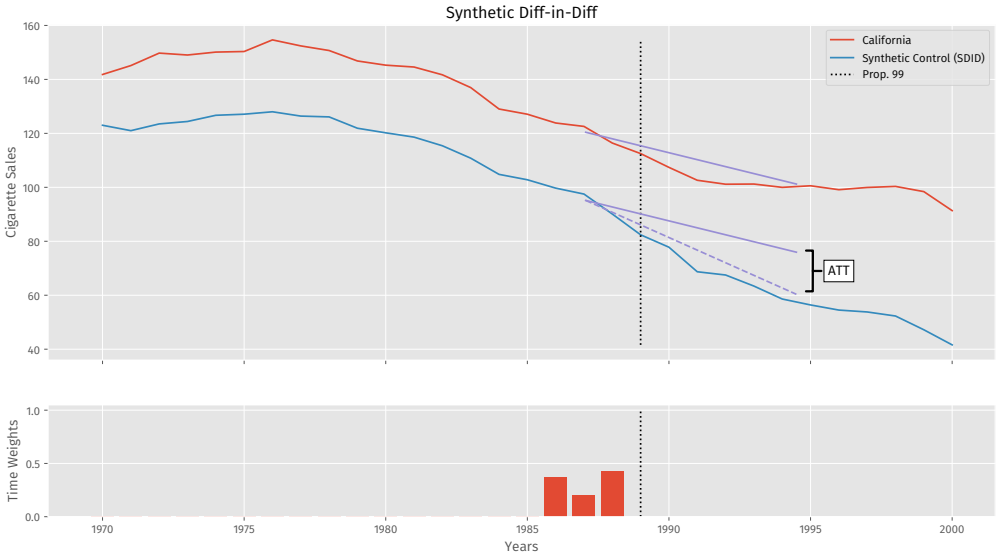
```
did_model = smf.wls("cigsale ~ after_treatment*treated",
                    data=did_data,
                    weights=did_data["weights"]+1e-10).fit()

did_model.summary().tables[1]
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	120.4060	1.272	94.665	0.000	117.911	122.901
after_treatment	-19.1905	1.799	-10.669	0.000	-22.720	-15.661
treated	-25.2601	1.799	-14.043	0.000	-28.789	-21.731
after_treatment:treated	-15.6054	2.544	-6.135	0.000	-20.596	-10.615

This estimate is much smaller than the one we get with Diff-in-Diff, but that is not surprising. As we've already discussed, the Diff-in-Diff estimator is probably biased in this case, since we have pretty good reasons to question the parallel trends assumption. What is perhaps less obvious is why the SDID estimate is smaller than the traditional SC estimate. If we go back and look at the SC plot, we can see that cigarette sales in California started to fall below its synthetic control prior to Proposition 99. This is probably due to the fact that traditional Synthetic Control has to match treated and control units in the entire pre-treatment period, causing it to miss one year or the other. This is less of an issue in SDID, since the time weights allow us to focus just on the periods that are more similar to the post-intervention period. In this case, those were precisely the three years anteceding Proposition 99.

To grasp what SDID is doing, we can plot the Diff-in-Diff lines for the treated (California) and the SDID Synthetic Control. Notice how we are projecting the trend we see in the synthetic control onto the treated unit to get the counterfactual $Y(0)_{tr,post}$. The difference between the dashed purple line and the lower solid purple line is the ATT. We start those lines in 1987 to show how the time weights zero out all periods but 1986, 87 and 88. The time weights are also shown in the small plot down below.

```
avg_pre_period = (time_weights * time_weights.index).sum()
avg_post_period = 1989 + (2000 - 1989) / 2

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(15,8), sharex=True,
  ↪ gridspec_kw={'height_ratios': [3, 1]})

ax1.plot(data_piv.index, sc_did, label="California")
ax1.plot(data_piv.index, data_piv["california"],
  ↪ label="Synthetic Control (SDID)")
ax1.vlines(1989, data_piv["california"].min(), sc_did.max(), color="black",
  ↪ ls="dotted", label="Prop. 99")

pre_sc = did_model.params["Intercept"]
post_sc = pre_sc + did_model.params["after_treatment"]
pre_treat = pre_sc + did_model.params["treated"]
post_treat = post_sc + did_model.params["treated"] + did_model
  ↪ .params["after_treatment:treated"]

sc_did_y0 = pre_treat + (post_sc - pre_sc)

ax1.plot([avg_pre_period, avg_post_period], [pre_sc, post_sc], color="C2")
ax1.plot([avg_pre_period, avg_post_period], [pre_treat, post_treat], color="C2",
  ↪ ls="dashed")
ax1.plot([avg_pre_period, avg_post_period], [pre_treat, sc_did_y0], color="C2")

ax1.annotate('ATT', xy=(1995, 69), xytext=(1996, 66.5),
             fontsize=12, ha='center', va='bottom',
             bbox=dict(boxstyle='square', fc='white', color='k'),
             arrowprops=dict(arrowstyle='-[, widthB=1.5, lengthB=0.5', lw=2.0,
               ↪ color='k'))

ax1.legend()
ax1.set_title("Synthetic Diff-in-Diff")
ax1.set_ylabel("Cigarette Sales")

ax2.bar(time_weights.index, time_weights)
ax2.vlines(1989, 0, 1, color="black", ls="dotted")
ax2.set_ylabel("Time Weights")
ax2.set_xlabel("Years");
```



The above estimator estimates the ATT which is the effect of Propostion 99 on California averaged out across all post-treatment time periods. But, from the above plot, it looks like the effect increases over time. What if we want to take that into account? Fortunately, it is very straightforward to do that.

Before we move on, just a word of caution about the above estimator. You should **not** trust the standard errors or confidence intervals reported by the regression we just ran. They don't reflect the variance in estimating the weights. We will take a look at how to do proper inference briefly, but first, let's see how to deal with effect heterogeneity across time.

Time Effect Heterogeneity and Staggered Adoption

Fortunately, it is incredibly easy to estimate one effect for each time period using SDID. All we have to do is run it multiple times, one for each time period. To be more precise, let's say we have the following treatment assignment matrix, with just 4 time periods and 3 units. The last unit is the treated one.

$$D = \begin{bmatrix} 0_1 & 0_1 & 0_1 \\ 0_2 & 0_2 & 0_2 \\ 0_3 & 0_3 & 1_3 \\ 0_4 & 0_4 & 1_4 \end{bmatrix}$$

Running SDID with the above matrix would give us the average ATT across periods 3 and 4. What we can do to estimate the effect on each period individually is simply to partition the problem into two, one for each post-treatment time period. Then, we run SDID on the data where we only keep post-treatment period 3 and again on the data where we only keep post-treatment period 4. That is, we run SDID on each of the following matrices

individually.

$$D_1 = \begin{bmatrix} 0_1 & 0_1 & 0_1 \\ 0_2 & 0_2 & 0_2 \\ 0_3 & 0_3 & 1_3 \end{bmatrix}$$

$$D_2 = \begin{bmatrix} 0_1 & 0_1 & 0_1 \\ 0_2 & 0_2 & 0_2 \\ 0_4 & 0_4 & 1_4 \end{bmatrix}$$

To do that, it would be best if we first merge all the steps of SDID into a single function. That is, estimating the unit and time weights and running DiD.

```
def synthetic_diff_in_diff(data, outcome_col, year_col, state_col, treat_col,
  ↪ post_col):

    # find the unit weights
    unit_weights = fit_unit_weights(data,
                                    outcome_col=outcome_col,
                                    year_col=year_col,
                                    state_col=state_col,
                                    treat_col=treat_col,
                                    post_col=post_col)

    # find the time weights
    time_weights = fit_time_weights(data,
                                    outcome_col=outcome_col,
                                    year_col=year_col,
                                    state_col=state_col,
                                    treat_col=treat_col,
                                    post_col=post_col)

    # join weights into DiD Data
    did_data = join_weights(data, unit_weights, time_weights,
                            year_col=year_col,
                            state_col=state_col,
                            treat_col=treat_col,
                            post_col=post_col)

    # run DiD
    formula = f"{outcome_col} ~ {post_col}*{treat_col}"
    did_model = smf.wls(formula, data=did_data,
      ↪ weights=did_data["weights"]+1e-10).fit()

    return did_model.params[f"{post_col}:{treat_col}"]


synthetic_diff_in_diff(data,
                       outcome_col="cigsale",
                       year_col="year",
                       state_col="state",
```

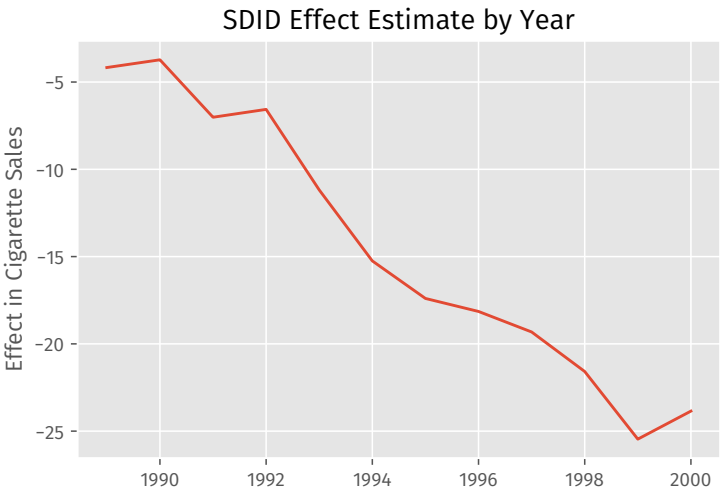
```
                        treat_col="treated",
                        post_col="after_treatment")
```

```
-15.605397234587002
```

Now that we have a way of easily running SDID, we can run it multiple times, filtering out all the post-treatment periods except the one for which we want the effect.

```
effects = {year: synthetic_diff_in_diff(data.query(
  ↪ f"~after_treatment|(year=={year})"),
                                        outcome_col="cigsale",
                                        year_col="year",
                                        state_col="state",
                                        treat_col="treated",
                                        post_col="after_treatment")
           for year in range(1989, 2001)}

effects = pd.Series(effects)
```

```
plt.plot(effects);
plt.ylabel("Effect in Cigarette Sales")
plt.title("SDID Effect Estimate by Year");
```



As expected, the effect gets bigger as time passes. It starts small, but it gradually increases to what seems like a decrease in consumption of 25 cigarette packs per capita in 2020.

Conveniently, running multiple SDID will also be important to deal with the staggered adoption case. With staggered addoption design, we have multiple treated units, which get the treatment at different time periods. For example, going back to our very simple assignment matrix, with 3 units and 4 time periods, let's say unit 1 never gets the

treatment, unit 2 gets the treatment at period 4 and unit 3 gets the treatment at period 3. This would result in the following matrix

$$D = \begin{bmatrix} 0_1 & 0_1 & 0_1 \\ 0_2 & 0_2 & 0_2 \\ 0_3 & 0_3 & 1_3 \\ 0_4 & 1_4 & 1_4 \end{bmatrix}$$

Notice that SDID can't handle that matrix, because we don't have a clear definition about what is a pre-treatment period (before time 4, in the case of the second unit or before period 3, in the case of the second unit) or what is a control unit (unit 2 could be a control for the treatment starting at period 3). The key in solving this is realizing we can delete columns (units) or rows (time periods) in that matrix in order to go back to the block assignment design.

For example, we can create two block matrices from the one above by first deleting the 3rd time period and then another one where we delete the 4th time period.

$$D_1 = \begin{bmatrix} 0_1 & 0_1 & 0_1 \\ 0_2 & 0_2 & 0_2 \\ 0_4 & 1_4 & 1_4 \end{bmatrix}$$

$$D_2 = \begin{bmatrix} 0_1 & 0_1 & 0_1 \\ 0_2 & 0_2 & 0_2 \\ 0_3 & 0_3 & 1_3 \end{bmatrix}$$

The result is two block matrices, which means we can run SDID in both of them. The result will be two ATT estimates, which we can then combine with a weighted average, where the weights are the proportion of treated time and periods in each block. In our example, the weight for ATT_1 would be 2/3 and the weight for ATT_2 would be 1/3.

Alternatively, we could also have 2 block designs by removing columns, which would result in the following matrices

$$D_1 = \begin{bmatrix} 0_1 & 0_1 \\ 0_2 & 0_2 \\ 0_3 & 0_3 \\ 0_4 & 1_4 \end{bmatrix}$$

$$D_2 = \begin{bmatrix} 0_1 & 0_1 \\ 0_2 & 0_2 \\ 0_3 & 1_3 \\ 0_4 & 1_4 \end{bmatrix}$$

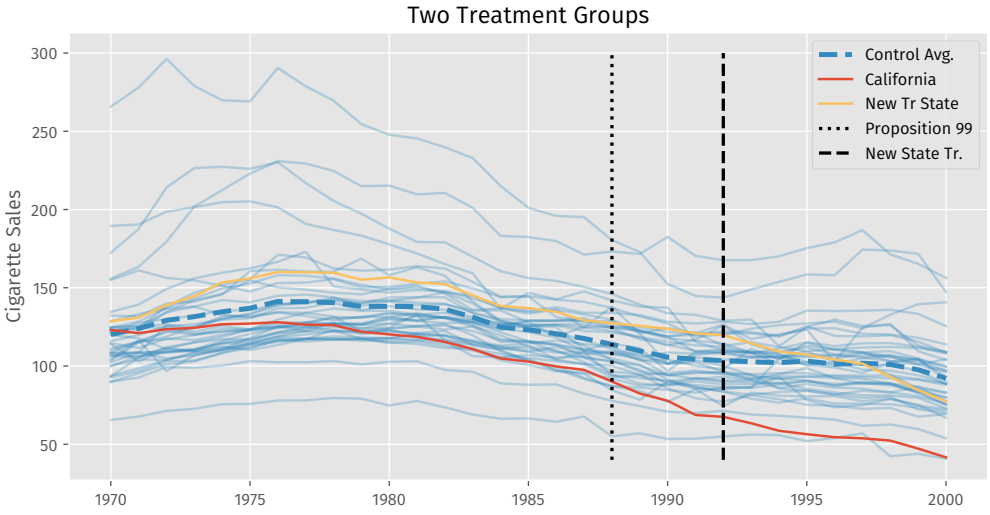
where D_1 has units 1 and 2 and D_2 has units 1 and 4.

Since we already saw how to estimate SDID for different time periods, let's look at this approach where we filter out units. Since we don't originally have a staggered adoption design in our Proposition 99 data, let's instead simulate one. We'll create 3 new states from our data and pretend they pass a law similar to Proposition 99, but in the year 1993.

Maybe they were impressed with the results in California and wanted to try in their states too. Once they do, this law that they pass decreases cigarette consumption by 3% each year. We can visualize the average cigarette consumption for those new states to better understand what is going on. In dashed black, we have the year in which these states pass this anti-tobacco law.

```
np.random.seed(1)
n = 3
tr_state = (data
            .query(f"state.isin({list(np.random.choice(data['state'].unique(),
    ↪ n))})")
            .assign(**{
                "treated": True,
                "state": lambda d: "new_" + d["state"].astype(str),
                "after_treatment": lambda d: d["year"] > 1992
            })
            # effect of 3% / year
            .assign(**{"cigsale": lambda d: np.random.normal(d["cigsale"] -
                                                             d["cigsale"]*(
                                                               ↪ 0.03*(d["year"]
    ↪- 1992))*d["after_treatment"], 1)}))

new_data = pd.concat([data, tr_state]).assign(**{"after_treatment": lambda d:
  ↪ np.where(d["treated"], d["after_treatment"], False)})
```

```
new_data_piv = new_data.pivot("year", "state", "cigsale")

new_tr_states = list(filter(lambda c: str(c).startswith("new"),
  ↪ new_data_piv.columns))

plt.figure(figsize=(10,5))
plt.plot(new_data_piv.drop(columns=["california"]+new_tr_states), color="C1",
  ↪ alpha=0.3)
plt.plot(new_data_piv.drop(columns=["california"]+new_tr_states).mean(axis=1),
  ↪ lw=3, color="C1", ls="dashed", label="Control Avg.")

plt.plot(new_data_piv["california"], color="C0", label="California")
plt.plot(new_data_piv[new_tr_states].mean(axis=1), color="C4",
  ↪ label="New Tr State")

plt.vlines(x=1988, ymin=40, ymax=300, linestyle=":", lw=2,
  ↪ label="Proposition 99", color="black")
plt.vlines(x=1992, ymin=40, ymax=300, linestyle="dashed", lw=2,
  ↪ label="New State Tr.", color="black")
plt.legend()
plt.ylabel("Cigarette Sales")
plt.title("Two Treatment Groups");
```



We finally have this staggered adoption data. Now, we need to figure out how to filter out some states so we can break the problem into multiple block assignment cases. First, we can group states by when they passed the law. The following code does exactly that.

```
assignment_blocks = (new_data.query("treated & after_treatment")
                     .groupby("state")["year"].min()
                     .reset_index()
                     .groupby("year")["state"].apply(list).to_dict())

assignment_blocks
```

```
{1989: ['california'], 1993: ['new_13', 'new_38', 'new_9']}
```

As you can see, we have two groups of states. One with only California, which was treated starting in 1989, and another with the three new states we've created, which were all treated starting in 1993. Now, we need to run SDID for each of those groups. We can easily do that, but keeping just the control units plus one of those groups. There is a catch, though. The `after_treatment` column will have a different meaning, depending on which group we are looking at. If we are looking at the group containing only California, `after_treatment` should be `year >= 1989`; if we are looking at the group with the new states, it should be `year >= 1993`. Fortunately, this is pretty easy to account for. All we need is to recreate the `after_treatment` in each iteration.

```
staggered_effects = {year: synthetic_diff_in_diff(new_data

      ↪ .query(f"~treated|(state.isin({states}))")
                                                  .assign(**{"after_treatment":
                                                   ↪ lambda d: d["year"] >=
      ↪ year}),
                                                 outcome_col="cigsale",
                                                 year_col="year",
                                                 state_col="state",
                                                 treat_col="treated",
                                                 post_col="after_treatment")
                     for year, states in assignment_blocks.items()}

staggered_effects
```

```
{1989: -15.605397234587002, 1993: -17.249435402003648}
```

Not surprisingly, the ATT estimate for the first group, the one with only California, is exactly the same as the one we've seen before. The other ATT refers to the one we get with the new group of states. We have to combine them into a single ATT. This can be done with the weighted average we've explained earlier.

First, we calculate the number of treated entries (`after_treatment & treated`) in each block. Then, we combine the ATTs using those weights.

```
weights = {year: sum((new_data["year"] >= year) & (new_data["state"].isin(
  ↪ states)))
           for year, states in assignment_blocks.items()}

att = sum([effect*weights[year]/sum(weights.values()) for year,
  ↪ effect in staggered_effects.items()])

print("weights: ", weights)
print("ATT: ", att)
```

```
weights:  {1989: 12, 1993: 24}
ATT:  -16.701422679531433
```

Here, we have a total of 36 treatment instances: the usual 12 post-treatment periods for California plus 8 treatment periods (1993-2000) for each of the three new treatment states we've introduced. With that in mind, the weight for the first ATT is 12/36 and for the second ATT, 24/36, which combines to the result above.

Placebo Variance Estimation

This chapter is getting a bit too long, but there is one promise we haven't fulfilled yet. Remember how we said, in the very beginning, that SDID has better precision (lower

error bars) when compared to Synthetic Controls? The reason is that the time and unit fixed effects in SDID capture a ton of the variation in the outcome, which in turn, reduces the variance of the estimator.

Of course I wouldn't ask you to take my word for it, so next, we'll show how to place a confidence interval around the SDID estimate. It turns out there are many solutions to this problem, but only one that fits the case for a single treated unit, which is the case we have here since only California was treated. The idea is to run a series of placebo tests, where we pretend a unit from the control pool is treated, when it actually isn't. Then, we use SDID to estimate the ATT of this placebo test and store its result. We re-run this step multiple times, sampling a control unit each time. In the end, we will have an array of placebo ATTs. The variance of this array is the placebo variance of the SDID effect estimate, which we can use to construct a confidence interval.

$$\hat{V}_{\tau}^{placebo} = B^{-1} \sum_{b=1}^{B} \left(\hat{\tau}^{(b)} - \bar{\hat{\tau}}^{(b)} \right)^2$$

$$\tau \in \hat{\tau}^{sdid} \pm z_{\alpha/2} \sqrt{\hat{V}_{\tau}}$$

In order to implement this, the first thing we need is a function which creates the placebo. This function will filter out the treated units, sample a single control unit and flip the `treated` column for that control unit from 0 to a 1.

```
def make_random_placebo(data, state_col, treat_col):
    control = data.query(f"~{treat_col}")
    states = control[state_col].unique()
    placebo_state = np.random.choice(states)
    return control.assign(**{treat_col: control[state_col] == placebo_state})
```

```
np.random.seed(1)
placebo_data = make_random_placebo(data, state_col="state", treat_col="treated")

placebo_data.query("treated").tail()
```

	state	year	cigsale	treated	after_treatment
1204	39	1996	110.300003	True	True
1205	39	1997	108.800003	True	True
1206	39	1998	102.900002	True	True
1207	39	1999	104.800003	True	True
1208	39	2000	90.500000	True	True

In the example above, we've sampled state 39 and we are now pretending that it was treated. Notice how the `treated` column was flipped to `True`.

The next thing we need is to compute the SDID estimate with this placebo data and repeat that a bunch of times. The next function does that. It runs the `synthetic_diff_in_diff` function to get the SDID estimate, but instead of passing the usual data, we pass the

result of calling `make_random_placebo`. We do that multiple times to get an array of SDID estimates and, finally, compute the square root of the variance of this array, which is just the standard deviation.

```
from joblib import Parallel, delayed # for parallel processing


def estimate_se(data, outcome_col, year_col, state_col, treat_col, post_col,
  ↪ bootstrap_rounds=400, seed=0, njobs=4):
    np.random.seed(seed=seed)

    sdid_fn = partial(synthetic_diff_in_diff,
                      outcome_col=outcome_col,
                      year_col=year_col,
                      state_col=state_col,
                      treat_col=treat_col,
                      post_col=post_col)

    effects = Parallel(n_jobs=njobs)(delayed(sdid_fn)(make_random_placebo(data,
      ↪ state_col=state_col, treat_col=treat_col))
                                     for _ in range(bootstrap_rounds))

    return np.std(effects, axis=0)
```

```
effect = synthetic_diff_in_diff(data,
                                outcome_col="cigsale",
                                year_col="year",
                                state_col="state",
                                treat_col="treated",
                                post_col="after_treatment")


se = estimate_se(data,
                 outcome_col="cigsale",
                 year_col="year",
                 state_col="state",
                 treat_col="treated",
                 post_col="after_treatment")
```

The standard deviation can then be used to construct confidence intervals much like we described in the formula above.

```
print(f"Effect: {effect}")
print(f"Standard Error: {se}")
print(f"90% CI: ({effect-1.65*se}, {effect+1.65*se})")
```

```
Effect: -15.605397234587002
Standard Error: 9.912089736240311
90% CI: (-31.960345299383516, 0.7495508302095111)
```

Notice that the ATT is not significant in this case, but what's more interesting here is to compare the standard error of the SDID estimate with the one we get from traditional Synthetic Control.

```
def synthetic_control(data, outcome_col, year_col, state_col, treat_col,
  ↪ post_col):

    x_pre_control = (data
                     .query(f"~{treat_col}")
                     .query(f"~{post_col}")
                     .pivot(year_col, state_col, outcome_col)
                     .values)

    y_pre_treat_mean = (data
                        .query(f"~{post_col}")
                        .query(f"{treat_col}")
                        .groupby(year_col)
                        [outcome_col]
                        .mean())

    w = cp.Variable(x_pre_control.shape[1])
    objective = cp.Minimize(cp.sum_squares(x_pre_control@w - y_pre_treat_mean
      ↪ .values))
    constraints = [cp.sum(w) == 1, w >= 0]

    problem = cp.Problem(objective, constraints)
    problem.solve(verbose=False)

    sc = (data
          .query(f"~{treat_col}")
          .pivot(year_col, state_col, outcome_col)
          .values) @ w.value

    y1 = data.query(f"{treat_col}").query(f"{post_col}")[outcome_col]
    att = np.mean(y1 - sc[-len(y1):])

    return att


def estimate_se_sc(data, outcome_col, year_col, state_col, treat_col, post_col,
  ↪ bootstrap_rounds=400, seed=0):
    np.random.seed(seed=seed)
    effects = [synthetic_control(make_random_placebo(data, state_col=state_col,
      ↪ treat_col=treat_col),
                                 outcome_col=outcome_col,
                                 year_col=year_col,
                                 state_col=state_col,
                                 treat_col=treat_col,
                                 post_col=post_col)
```

```
            for _ in range(bootstrap_rounds)]

    return np.std(effects, axis=0)


effect_sc = synthetic_control(data,
                              outcome_col="cigsale",
                              year_col="year",
                              state_col="state",
                              treat_col="treated",
                              post_col="after_treatment")


se_sc = estimate_se_sc(data,
                       outcome_col="cigsale",
                       year_col="year",
                       state_col="state",
                       treat_col="treated",
                       post_col="after_treatment")
```

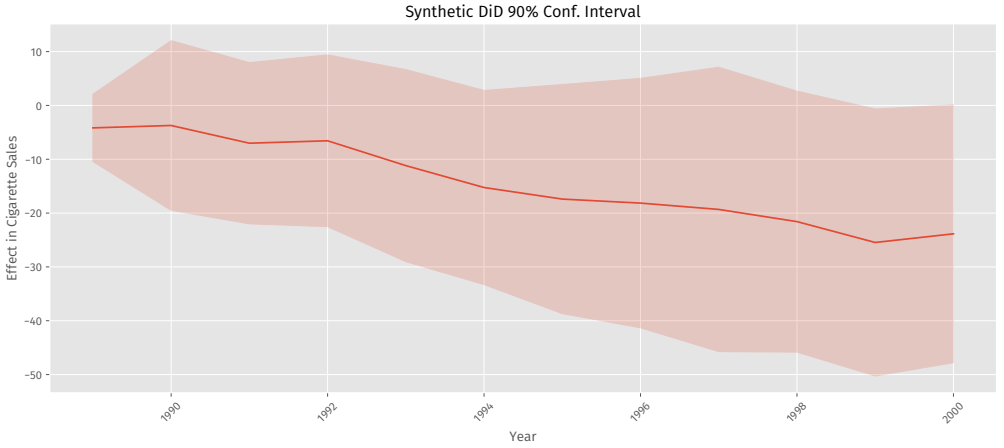
```
print(f"Effect: {effect_sc}")
print(f"Standard Error: {se_sc}")
print(f"90% CI: ({effect_sc-1.65*se_sc}, {effect_sc+1.65*se_sc})")
```

```
Effect: -19.513629763998537
Standard Error: 11.2419349486209
90% CI: (-38.06282242922302, -0.9644370987740523)
```

Notice how the error for synthetic control is higher than for SDID. Again, that is because SDID captures a lot of the variance in the outcome via its time and unit fixed effects. With this, we fulfill the promise we made earlier. But, before we close, it is worth mentioning that we can also use the same procedure to estimate the variance to make a confidence interval around the effect we've estimated for each post-treatment time period. All we need to do is run the code above once for each time period. Just keep in mind that this might take some time to run, even with the parallelization we've implemented.

```
standard_errors = {year: estimate_se(data.query(
  ↪ f"~after_treatment|(year=={year})"),
                                     outcome_col="cigsale",
                                     year_col="year",
                                     state_col="state",
                                     treat_col="treated",
                                     post_col="after_treatment")
                   for year in range(1989, 2001)}

standard_errors = pd.Series(standard_errors)
```

```
plt.figure(figsize=(15,6))

plt.plot(effects, color="C0")
plt.fill_between(effects.index, effects-1.65*standard_errors,
  ↪ effects+1.65*standard_errors, alpha=0.2,  color="C0")

plt.ylabel("Effect in Cigarette Sales")
plt.xlabel("Year")
plt.title("Synthetic DiD 90% Conf. Interval")
plt.xticks(rotation=45);
```



Key Concepts

Synthetic-Diff-in-Diff (SDID) draws inspiration from both Diff-in-Diff and Synthetic Control, which brings advantages from both models. Like SC, SDID still works with multiple periods when pre-treatment trends are not parallel. However, unlike SC, SDID estimates unit weights to build a control unit which is only parallel to the treated group (it doesn't have to match its level). From DID, SDID leverages time and unit fixed effects, which helps to explain a lot of the variance in the outcome, which in turn reduces the variance of the SDID estimator. Synthetic-Diff-in-Diff also introduces some new ideas of its own. First, there is an additional $L2$ penalty in the optimization of the unit weights which makes them more spread out across control units. Second, SDID allows for an intercept (and hence, extrapolation) when building such weights. Third, SDID introduces the use of time weights, which are not present in either DID nor SC. For this reason, I wouldn't say SDID is just merging SC and SDID. It is rather building something new, inspired by these two approaches. I also wouldn't say that SDID is better or worse than traditional Synthetic Control. Each of them have different properties that might be appropriate or not, depending on the situation. For example, you might find yourself in a situation where allowing the extrapolations from SDID is dangerous. In which case, SC might be a good alternative.

References

This chapter is essentially an explainer to the *Synthetic Difference in Differences* (2019) article, by Dmitry Arkhangelsky, Susan Athey, David A. Hirshberg, Guido W. Imbens and Stefan Wager. Additionally, I would love to recognize Masa Asami for his python implementation of SDID, pysynthdid. His code helped me make sure I didn't have any bugs in mine, which was extremely helpful.

Appendices

APPENDIX

A

Debiasing with Orthogonalization

Previously, we saw how to evaluate a causal model. By itself, that's a huge deed. Causal models estimates the elasticity $\frac{\delta y}{\delta t}$, which is an unseen quantity. Hence, since we can't see the ground truth of what our model is estimating, we had to be very creative in how we would go about evaluating them.

The technique shown on the previous chapter relied heavily on data where the treatment was randomly assigned. The idea was to estimate the elasticity $\frac{\delta y}{\delta t}$ as the coefficient of a single variable linear regression of `y ~ t`. However, this only works if the treatment is randomly assigned. If it isn't, we get into trouble due to omitted variable bias.

To workaround this, we need to make the data look as if the treatment is randomly assigned. I would say there are two main techniques to do this. One is using propensity score and the other using orthogonalization. We will cover the latter in this chapter.

One final word of caution before we continue. I would argue that probably the safest way out of non random data is to go out and do some sort of experiment to gather random data. I myself don't trust very much on debiasing techniques because you can never know if you've accounted for every confounder. Having said that, orthogonalization is still very much worth learning. It's an incredibly powerful technique that will be the foundation of many causal models to come.

Linear Regression Reborn

The idea of orthogonalization is based on a theorem designed by three econometricians in 1933, Ragnar Frisch, Frederick V. Waugh, and Michael C. Lovell. Simply put, it states that you can decompose any multivariable linear regression model into three stages or models. Let's say that your features are in an X matrix. Now, you partition that matrix in such a way that you get one part, X_1, with some of the features and another part, X_2, with the rest of the features.

In the first stage, we take the first set of features and estimate the following linear regression model

$$y_i = \theta_0 + \boldsymbol{\theta}_1 \boldsymbol{X}_{1i} + e_i$$

where $\boldsymbol{\theta}_1$ is a vector of parameters. We then take the residuals of that model

$$y^* = y_i - (\hat{\theta}_0 + \hat{\boldsymbol{\theta}}_1 \boldsymbol{X}_{1i})$$

On the second stage, we take the first set of features again, but now we run a model where we estimate the second set of features

$$\boldsymbol{X}_{2i} = \gamma_0 + \boldsymbol{\gamma}_1 \boldsymbol{X}_{1i} + e_i$$

Here, we are using the first set of features to predict the second set of features. Finally, we also take the residuals for this second stage.

$$\boldsymbol{X}^*_{2i} = \boldsymbol{X}_{2i} - (\hat{\gamma}_0 + \hat{\boldsymbol{\gamma}}_1 \boldsymbol{X}_{1i})$$

Lastly, we take the residuals from the first and second stage, and estimate the following model

$$y^*_i = \beta_0 + \boldsymbol{\beta}_2 \boldsymbol{X}^*_{2i} + e_i$$

The Frisch–Waugh–Lovell theorem states that the parameter estimate $\hat{\boldsymbol{\beta}}_2$ from estimating this model is equivalent to the one we get by running the full regression, with all the features:

$$y_i = \beta_0 + \boldsymbol{\beta}_1 \boldsymbol{X}_{1i} + \boldsymbol{\beta}_2 \boldsymbol{X}_{2i} + e_i$$

OK. Let's unpack this a bit further. We know that regression is a very special model. Each of its parameters has the interpretation of a partial derivative: how much would Y increase if I increase one feature **while holding all the others fixed**. This is very nice for causal inference, because it means we can control for variables in the analysis, even if those same variables have not been held fixed during the collection of the data.

We also know that if we omit variables from the regression, we get bias. Specifically, omitted variable bias (or confounding bias). Still, the Frisch–Waugh–Lovell is saying that I can break my regression model into two parts, neither of them containing the full feature set, and still get the same estimate I would get by running the entire regression. Not only that, this theorem also provides some insight into what linear regression is doing. To get

the coefficient of one variable X_k, regression first uses all the other variables to predict X_k and takes the residuals. This "cleans" X_k of any influence from those variables. That way, when we try to understand X_k's impact on Y, it will be free from omitted variable bias. Second, regression uses all the other variables to predict Y and takes the residuals. This "cleans" Y from any influence from those variables, reducing the variance of Y so that it is easier to see how X_k impacts Y.

I know it can be hard to appreciate how awesome this is. But remember what linear regression is doing. It's estimating the impact of X_2 on y while accounting for X_1. This is incredibly powerful for causal inference. It says that I can build a model that predicts my treatment t using my features X, a model that predicts the outcome y using the same features, take the residuals from both models and run a model that estimates how the residual of t affects the residual of y. This last model will tell me how t affects y while controlling for X. In other words, the first two models are controlling for the confounding variables. They are generating data which is as good as random. This is debiasing my data. That's what we use in the final model to estimate the elasticity.

There is a (not so complicated) mathematical proof for why that is the case, but I think the intuition behind this theorem is so straightforward we can go directly into it.

The Intuition Behind Orthogonalization

```
import pandas as pd
import numpy as np
from matplotlib import pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split

import statsmodels.formula.api as smf
import statsmodels.api as sm

from nb21 import cumulative_elast_curve_ci, elast, cumulative_gain_ci
```

Let's take our price data once again. But now, we will only take the sample where prices where **not** randomly assigned. Once again, we separate them into a training and a test set. Since we will use the test set to evaluate our causal model, let's see how we can use orthogonalization to debias it.

```
prices = pd.read_csv("./data/ice_cream_sales.csv")

train, test = train_test_split(prices, test_size=0.5)
train.shape, test.shape
```

```
((5000, 5), (5000, 5))
```

If we show the correlations on the test set, we can see that price is positively correlated with sales, meaning that sales should go up as we increase prices. This is obviously

nonsense. People don't buy more if ice cream is expensive. We probably have some sort of bias here.

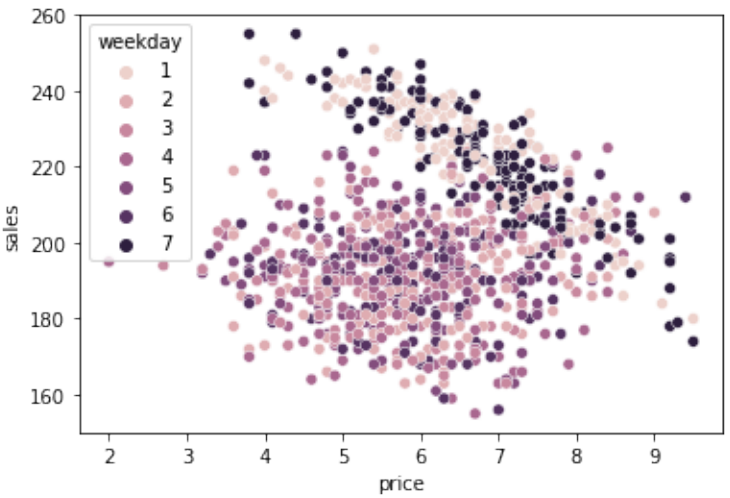
```
test.corr()
```

	temp	weekday	cost	price	sales
temp	1.000000	0.003630	0.006605	-0.011977	0.379108
weekday	0.003630	1.000000	0.011889	0.002610	0.004589
cost	0.006605	0.011889	1.000000	0.388046	-0.009410
price	-0.011977	0.002610	0.388046	1.000000	0.080040
sales	0.379108	0.004589	-0.009410	0.080040	1.000000

If we plot our data, we can see why this is happening. Weekends (Saturday and Sunday) have higher price but also higher sales. We can see that this is the case because the weekend cloud of points seems to be to the upper right part of the plot.

Weekend is probably playing an important role in the bias here. On the weekends, there are more ice cream sales because there is more demand. In response to that demand, prices go up. So it is not that the increase in price causes sales to go up. It is just that both sales and prices are high on weekends.

```
np.random.seed(123)
sns.scatterplot(data=test.sample(1000), x="price", y="sales", hue="weekday");
```



To debias this dataset we will need two models. The first model, let's call it $M_t(X)$, predicts the treatment (price, in our case) using the confounders. It's the one of the stages we've seen above, on the Frisch–Waugh–Lovell theorem.

```
m_t = smf.ols("price ~ cost + C(weekday) + temp", data=test).fit()
debiased_test = test.assign(**{"price-Mt(X)":test["price"] - m_t.predict(test)})
```

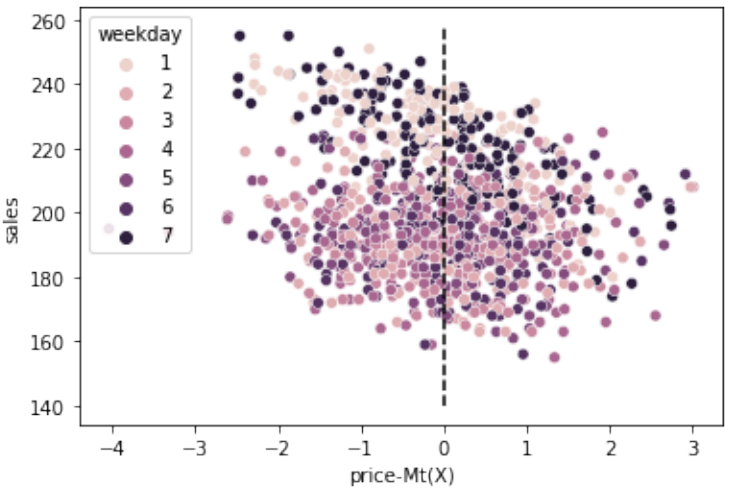
Once we have this model, we will construct the residuals

$$\hat{t}_i = t_i - M_t(X_i)$$

You can think of this residual as a version of the treatment that is unbiased or, better yet, that is impossible to predict from the confounders X. Since the confounders were already used to predict t, the residual is by definition, unpredictable with X. Another way of saying this is that the bias has been explained away by the model $M_t(X_i)$, producing $\hat{t}_i$ which is as good as randomly assigned. Of course this only works if we have in X all the confounders that cause both T and Y.

We can also plot this data to see what it looks like.

```
np.random.seed(123)
sns.scatterplot(data=debiased_test.sample(1000), x="price-Mt(X)", y="sales",
  ↪ hue="weekday")
plt.vlines(0, debiased_test["sales"].min(), debiased_test["sales"].max(),
  ↪ linestyles='--', color="black");
```



We can see that the weekends are no longer to the upper right corner. They got pushed to the center. Moreover, we can no longer differentiate between different price levels (the treatment) using the weekdays. We can say that the residual $price - M_t(X)$, plotted on the x-axis, is a "random" or debiased version of the original treatment.

This alone is sufficient to debias the dataset. This new treatment we've created is as good as randomly assigned. But we can still do one other thing to make the debiased dataset even better. Namely, we can also construct residuals for the outcome.

$$\hat{y}_i = y_i - M_y(X_i)$$

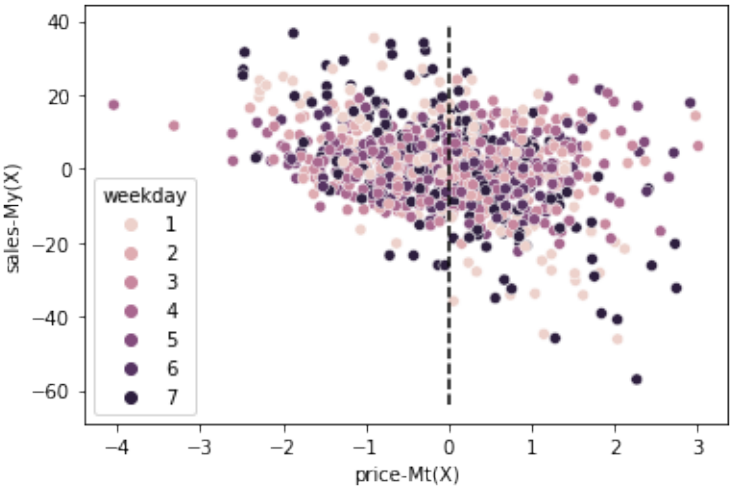
This is another stage from the Frisch–Waugh–Lovell theorem. It doesn't make the set less biased, but it makes it easier to estimate the elasticity by reducing the variance in y. Once again, you can think about $\hat{y}_i$ as a version of y_i that is unpredictable from X or that had all its variances due to X explained away. Think about it. We've already used X to predict y with $M_y(X_i)$. And $\hat{y}_i$ is the error of this prediction. So, by definition, it's not possible to predict it from X. All the information in X to predict y has already been used. If that is the case, the only thing left to explain $\hat{y}_i$ is something we didn't used to construct it (not included in X), which is only the treatment (again, assuming no unmeasured confounders).

```
m_y = smf.ols("sales ~ cost + C(weekday) + temp", data=test).fit()

debiased_test = test.assign(**{"price-Mt(X)":test["price"] - m_t.predict(test),
                               "sales-My(X)":test["sales"] - m_y.predict(test)})
```

Once we do both transformations, not only does weekdays not predict the price residuals, but it also can't predict the residual of sales $\hat{y}$. The only thing left to predict these residuals is the treatment. Also, notice something interesting. In the plot above, it was hard to know the direction of the price elasticity. It looked like sales decreased as prices went up, but there was such a large variance in sales that it was hard to say that for sure.

Now, when we plot the two residuals, it becomes much clear that sales indeed causes prices to go down.

```
np.random.seed(123)
sns.scatterplot(data=debiased_test.sample(1000), x="price-Mt(X)",
  ↪ y="sales-My(X)", hue="weekday")
plt.vlines(0, debiased_test["sales-My(X)"].min(), debiased_test["sales-My(X)"]
  ↪ .max(), linestyles='--', color="black");
```



One small disadvantage of this debiased data is that the residuals have been shifted to a different scale. As a result, it's hard to interpret what they mean (what is a price residual of -3?). Still, I think this is a small price to pay for the convenience of building random data from data that was not initially random.

To summarize, by predicting the treatment, we've constructed $\hat{t}$ which works as an unbiased version of the treatment; by predicting the outcome, we've constructed $\hat{y}$ which is a version of the outcome that can only be further explained if we use the treatment. This data, where we replace y by $\hat{y}$ and t by $\hat{t}$ is the debiased data we wanted. We can use it to evaluate our causal model just like we deed previously using random data.

To see this, let's once again build a causal model for price elasticity using the training data.

```
m3 = smf.ols(f"sales ~ price*cost + price*C(weekday) + price*temp",
  ↪ data=train).fit()
```

Then, we'll make elasticity predictions on the debiased test set.

```
def predict_elast(model, price_df, h=0.01):
    return (model.predict(price_df.assign(price=price_df["price"]+h))
            - model.predict(price_df)) / h

debiased_test_pred = debiased_test.assign(**{
    "m3_pred": predict_elast(m3, debiased_test),
})

debiased_test_pred.head()
```

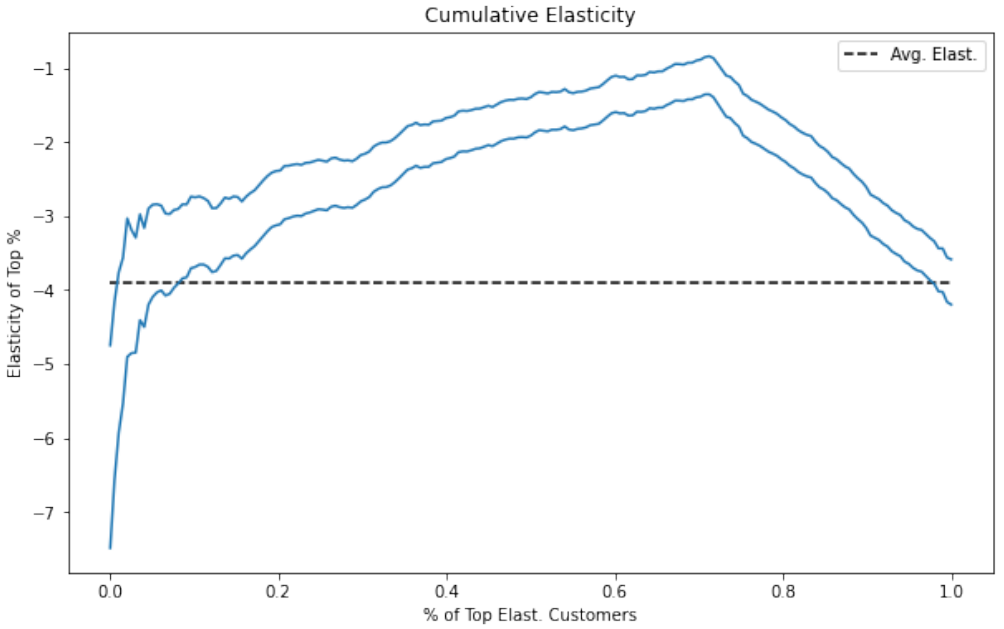
	temp	weekday	cost	price	sales	price-Mt(X)	sales-My(X)	m3_pred
7791	20.8	3	1.5	6.3	187	-0.201769	1.441373	-0.073317
1764	26.6	3	1.5	6.3	201	-0.179506	4.737748	-2.139611
5785	24.0	4	1.0	5.8	186	-0.215107	-5.855171	-0.549798
3542	20.9	3	1.5	5.1	180	-1.401386	-5.743172	-0.108943
9250	26.7	5	1.0	7.0	201	0.978382	4.384885	-1.427230

Now, when it comes to plotting the cumulative elasticity, we still order the dataset by the predictive elasticity, but now we use the debiased versions of the treatment and outcome to get this elasticity. This is equivalent to estimating β_1 in the following regression model

$$\hat{y}_i = \beta_0 + \beta_1 \hat{t}_i + e_i$$

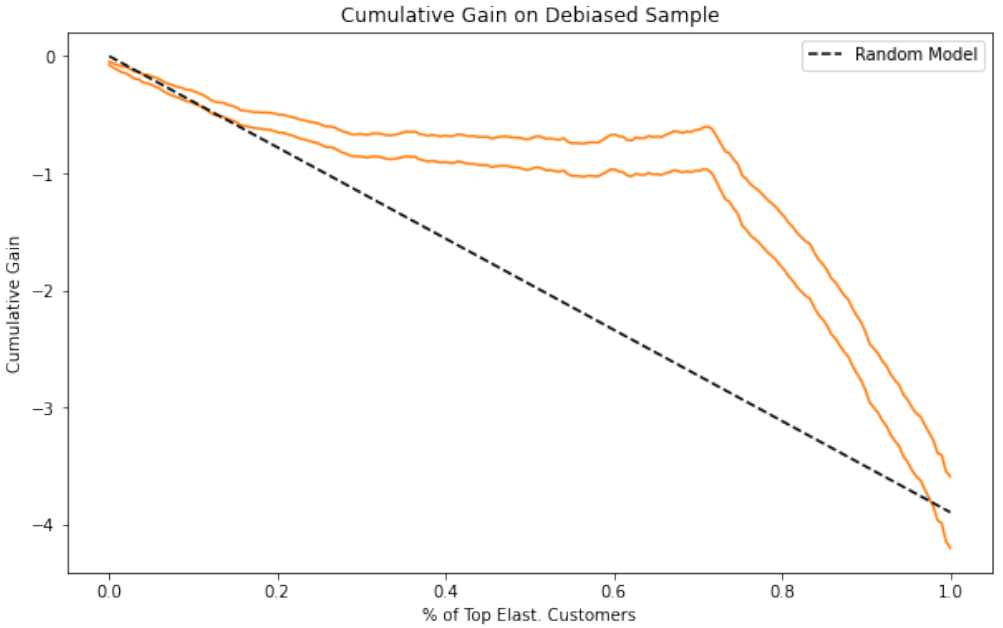
where the residuals are like we've described before.

```
plt.figure(figsize=(10,6))

cumm_elast = cumulative_elast_curve_ci(debiased_test_pred, "m3_pred",
  ↪ "sales-My(X)", "price-Mt(X)", min_periods=50, steps=200)
x = np.array(range(len(cumm_elast)))
plt.plot(x/x.max(), cumm_elast, color="C0")

plt.hlines(elast(debiased_test_pred, "sales-My(X)", "price-Mt(X)"), 0, 1,
  ↪ linestyles="--", color="black", label="Avg. Elast.")
plt.xlabel("% of Top Elast. Customers")
plt.ylabel("Elasticity of Top %")
plt.title("Cumulative Elasticity")
plt.legend();
```



We can do the same thing for the cumulative gain curve, of course.

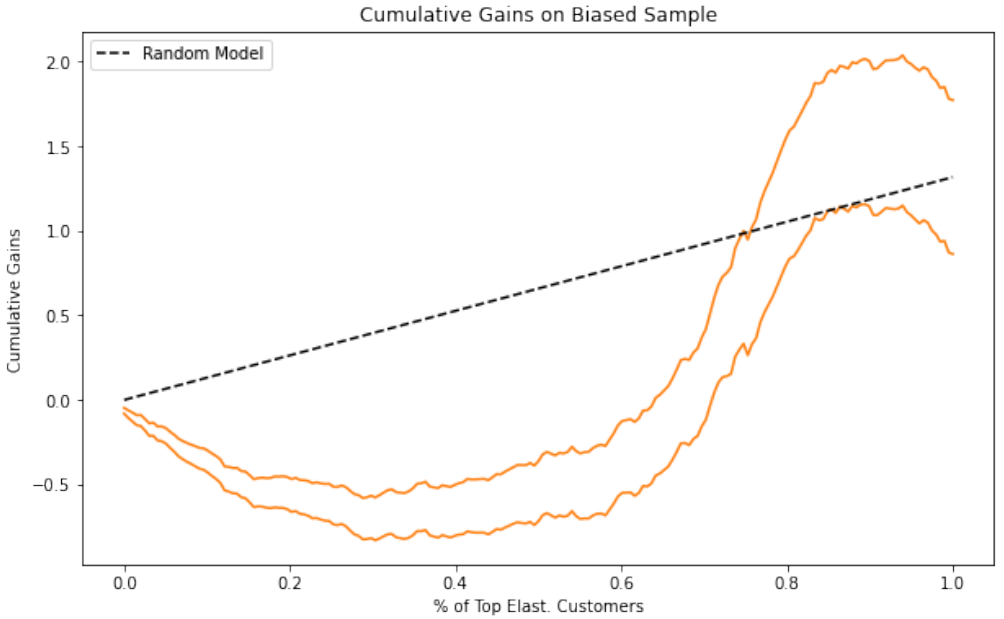
```
plt.figure(figsize=(10,6))

cumm_gain = cumulative_gain_ci(debiased_test_pred, "m3_pred", "sales-My(X)",
  ↪ "price-Mt(X)", min_periods=50, steps=200)
x = np.array(range(len(cumm_gain)))
plt.plot(x/x.max(), cumm_gain, color="C1")

plt.plot([0, 1], [0, elast(debiased_test_pred, "sales-My(X)", "price-Mt(X)")],
  ↪ linestyle="--", label="Random Model", color="black")

plt.xlabel("% of Top Elast. Customers")
plt.ylabel("Cumulative Gain")
plt.title("Cumulative Gain on Debiased Sample")
plt.legend();
```



Notice how similar these plots are to the ones in the previous chapter. This is some indication that the debiasing worked wonders here.

In contrast, let's see what the cumulative gain plot would look like if we used the original, biased data.

```
plt.figure(figsize=(10,6))

cumm_gain = cumulative_gain_ci(debiased_test_pred, "m3_pred", "sales", "price",
  ↪ min_periods=50, steps=200)
x = np.array(range(len(cumm_gain)))
plt.plot(x/x.max(), cumm_gain, color="C1")

plt.plot([0, 1], [0, elast(debiased_test_pred, "sales", "price")],
  ↪ linestyle="--", label="Random Model", color="black")

plt.xlabel("% of Top Elast. Customers")
plt.title("Cumulative Gains on Biased Sample")
plt.ylabel("Cumulative Gains")
plt.legend();
```



First thing you should notice is that the average elasticity goes up, instead of down. We've seen this before. In the biased data, it looks like sales goes up as price increases. As a result, the final point in the cumulative gain plot is positive. This makes little sense, since we now people don't buy more as we increase ice cream prices. If the average price elasticity is already messed up, any ordering in it also makes little sense. The bottom line being that this data should not be used for model evaluation.

Orthogonalization with Machine Learning

In a 2016 paper, Victor Chernozhukov *et all* showed that you can also do orthogonalization with machine learning models. This is obviously very recent science and we still have

much to discover on what we can and can't do with ML models. Still, it's a very interesting idea to know about.

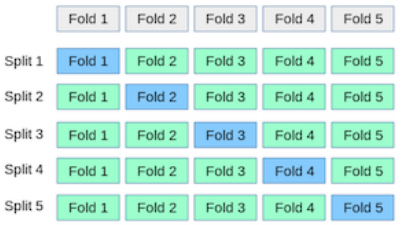
The nuts and bolts are pretty much the same to what we've already covered. The only difference is that now, we use machine learning models for the debiasing.

$$\hat{y}_i = y_i - M_y(X_i)$$
$$\hat{t}_i = t_i - M_t(X_i)$$

There is a catch, though. As we know very well, machine learning models are so powerful that they can fit the data perfectly, or rather, overfit. Just by looking at the equations above, we can know what will happen in that case. If M_y somehow overfits, the residuals will all be very close to zero. If that happens, it will be hard to find how t affects it. Similarly, if M_t somehow overfits, its residuals will also be close to zero. Hence, there won't be variation in the treatment residual to see how it can impact the outcome.

To account for that, we need to do sample splitting. That is, we estimate the model with one part of the dataset and we make predictions in the other part. The simplest way to do this is to split the test sample in half, make two models in such a way that each one is estimated in one half of the dataset and makes predictions in the other half.

A slightly more elegant implementation uses K-fold cross validation. The advantage being that we can train all the models on a sample which is bigger than half the test set.



Fortunately, this sort of cross prediction is very easy to implement using Sklearn's `cross_val_predict` function.

```
from sklearn.model_selection import cross_val_predict
from sklearn.ensemble import RandomForestRegressor

X = ["cost", "weekday", "temp"]
t = "price"
y = "sales"

folds = 5

np.random.seed(123)
m_t = RandomForestRegressor(n_estimators=100)
t_res = test[t] - cross_val_predict(m_t, test[X], test[t], cv=folds)
```

```
m_y = RandomForestRegressor(n_estimators=100)
y_res = test[y] - cross_val_predict(m_y, test[X], test[y], cv=folds)
```

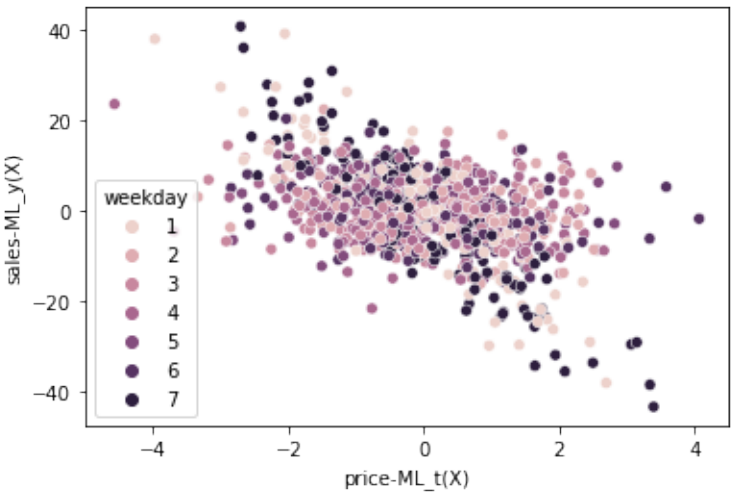
Now that we have the residuals, let's store them as columns on a new dataset.

```
ml_debiased_test = test.assign(**{
    "sales-ML_y(X)": y_res,
    "price-ML_t(X)": t_res,
})
ml_debiased_test.head()
```

	temp	weekday	cost	price	sales	sales-ML_y(X)	price-ML_t(X)
7791	20.8	3	1.5	6.3	187	-3.150833	-0.869267
1764	26.6	3	1.5	6.3	201	-0.418857	-0.192867
5785	24.0	4	1.0	5.8	186	-2.515667	0.790429
3542	20.9	3	1.5	5.1	180	-11.718500	-1.280460
9250	26.7	5	1.0	7.0	201	-1.214167	1.715117

Finally, we can plot the debiased dataset.

```
np.random.seed(123)
sns.scatterplot(data=ml_debiased_test.sample(1000),
                x="price-ML_t(X)", y="sales-ML_y(X)", hue="weekday");
```



Once again, we've uncovered a negative price elasticity on sales. Actually, the plot is incredibly similar to the one we've got when using simple linear regression. But that's probably because this is a very simple dataset. The advantages of machine learning orthogonalization is that it can estimate more complicated functions. It can learn interactions and non linearities in a way that it's hard to encode into linear regression. Also,

there is the advantage that some machine learning models (those bases on decision trees) are much simpler to run than linear regression. They can handle categorical data, outliers and even missing data, stuff that would require some attention if you are just using linear regression.

Finally, before we close, I just need to cover one final common mistake that data scientists often make when they are introduced to this idea (been there, done that). If the treatment or the outcome is binary, one might think it is better to replace the machine learning regression models for their classification versions. However, this does not work. The theory of orthogonalization only functions under regression models, similarly with what we've seen a long time ago when talking about Instrumental Variables. To be honest, it is not that the model will fail miserably if you replace regression by classification, but I would advise against it. If the theory doesn't justify it, why run the risk?

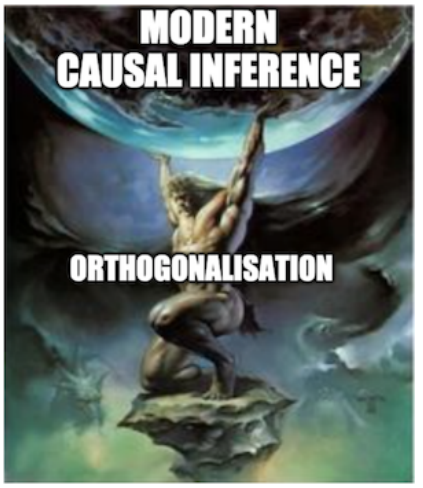
Key Ideas

We've started the chapter by highlighting the necessity of random treatment assignment in order for our causal evaluation methods to work. This poses a problem in the case where random data is not available. To be clear, the safest solution in this case is to go and do some experiments in order to get random data. If that is out of the question, only then, we can rely on a clever alternative: transform our data to look as if the treatment has been randomly assigned.

Here, we've covered how to do that using the principles of orthogonalization. First, we've built a model that uses our features X to predict the treatment t and get it's residuals. The idea being that the treatment residuals is, by definition, independent of the features used to construct it. In other words, the treatment residuals are orthogonal to the features. We can see these residuals as a version of the treatment where all the confounding bias due to X has been removed.

That alone is enough to make our data look as good as random. But we can go one step further. We can build a model that predicts the outcome y using the features X but not the treatment and also get its residuals. Again, the intuition is very similar. These outcome residuals is a version of the outcome where all the variance due to the features has been explained away. That will hopefully explay a lot of the variance, making it easier to see the treatment effect.

Here we are using orthogonalization with the goal of debiasing our data for model evaluation. However, this technique is also used for other purposes. Namely, lot's of causal inference models use orthogonalization as a first pre-processing step to ease the task of the causal inference model. We can say that orthogonalization makes the foundation of many modern causal inference algorithms.



References

The things I've written here are mostly stuff from my head. I've learned them through experience. This means that they have **not** passed the academic scrutiny that good science often goes through. Instead, notice how I'm talking about things that work in practice, but I don't spend too much time explaining why that is the case. It's a sort of science from the streets, if you will. However, I am putting this up for public scrutiny, so, by all means, if you find something preposterous, open an issue and I'll address it to the best of my efforts.

This chapter is based on Victor Chernozhukov *et all* (2016), Double/Debiased Machine Learning for Treatment and Causal Parameters. You can also check Frisch, Ragnar; Waugh, Frederick V. (1933) original article, Partial Time Regressions as Compared with Individual Trends.

APPENDIX

B

Debiasing with Propensity Score

Previously, we saw how to go from a biased dataset to one where the treatment looked as good as randomly assigned. We used orthogonalization for that. That technique was based on predicting the treatment and the outcome and then replacing both with their predictions' residuals.

$$t^* = t - M_t(X)$$

$$y^* = y - M_y(X)$$

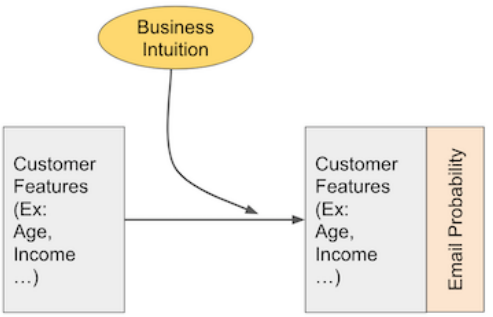
That alone is a powerful technique. It works both when the treatment is continuous or binary. Now, we'll take a look at another method which is based on the propensity score. Since the propensity score is best defined when the treatment is binary or categorical, this debiasing technique will also work only for categorical or binary treatments. Still, in some situations, it can be more reliable than orthogonalization and it is very much worth learning.

But first, we need to change context a little bit. We were talking about ice cream prices, which is a continuous treatment. Now, we will look into marketing emails, a binary treatment. And just as a side note, it is totally fair play to discretize a continuous treatment into buckets so it looks categorial (for example, you can take price, which is continuous, and discretize it in bins of R$ 2.00 like [2.00, 4.00, 6.00, 8.00]). Back to the emails now.

The situation goes like this. You work at a financial company (I'm not very creative with my examples, sorry) that supplies, not surprisingly, financial products such as life insurance, savings account, investment account and so on. The company is coming up with a new financial consulting service and wants to market it to its customers. To do so, the marketing team tried out three different emails: `em1`, `em2` and `em3`.

Because the marketing team is very well educated in statistics and experimental design, they don't simply send the email to everyone and see what happend. Instead, they design an experiment where each customer has a probability of receiving the email. This

probability is based on their business intuition to who will be more responsive to the email. For example, `em1` is targeted to a mass market audience, that don't invest much and that don't have a high income. These people will have a higher probability of receiving `em1`. Finally, after having defined that probability, we randomize the assignment so that customers receive the email according to their assigned probabilities.



As a side note, this is the best way I know of that uses business expertise to target an audience while still allowing you to make valid inference about how effective the marketing strategy is. The marketing team not only assigned the email according to this probability function, but they actually stored each customers' probability, which will be very handy later on.

Here is what the data from this experiment looks like. We have data on four customer attributes: `age`, `income`, how much they have on life `insurance` and how much they have `invested`. These are the attributes which the marketing team used to assign the probability of receiving each email. Those probabilities are stored in columns `em1_ps`, `em2_ps` and `em3_ps`. Then, we have information on whether the customer actually received the email on columns `em1`, `em2` and `em3`. These are our treatment variables. Finally, we have the outcome, `converted`, flags if the customer contracted the financial advisory service.

```
import pandas as pd
import numpy as np
from matplotlib import pyplot as plt
import seaborn as sns
```

```
email = pd.read_csv("./data/invest_email.csv")
email.head()
```

	age	income	insurance	invested	em1_ps
0	44.1	4214.66	14036.41	7988.66	0.118507
1	39.8	1151.27	66622.15	1000.74	0.431123
2	49.0	8047.26	14119.53	29480.22	0.062137
3	39.7	10558.37	14798.26	36373.09	0.047372
4	35.3	802.15	780.84	1643.16	0.616450

	em2_ps	em3_ps	em1	em2	em3	converted
0	0.045174	0.921281	0	0	1	1
1	0.005659	0.000000	1	0	0	0
2	0.166706	0.840049	0	0	1	0
3	0.205684	0.000000	0	0	0	0
4	0.009292	0.000000	1	0	0	0

As we can see, there are customers with very low probability of receiving `em1`. This is the case of customer 2, which has a probability of only 0.062. Also notice that this customer did not receive `em1`. This is not surprising, given that he had a very low probability of receiving this email.

Now, contrast this with the following customers. All of them had a very high chance of getting `em1`, but none of them did.

```
email.query("em1 == 0").query("em1_ps>0.9")
```

	age	income	insurance	invested	em1_ps
2438	48.0	538.02	26276.38	1582.82	0.913562
2692	44.2	545.62	2377.01	743.40	0.901065
3152	44.6	541.42	13272.96	1455.92	0.907929
4175	38.0	543.20	16787.28	1302.20	0.905007

	em2_ps	em3_ps	em1	em2	em3	converted
2438	0.008951	0.999208	0	0	1	1
2692	0.004204	0.999047	0	0	1	0
3152	0.008233	0.999136	0	0	1	1
4175	0.007364	0.000000	0	0	0	1

Inverse Probability of Treatment Weighting

Just to recap the idea behind propensity score debiasing (we've already seen it when we discussed propensity score), we are very interested in customers like the ones above. They look a lot like the ones that get `em1`, after all they had a high probability of getting that email. However, they didn't get it. This makes them awesome candidates to estimate the counterfactual outcome $Y_0|em1 = 1$.

Propensity score debiasing works by encoding this intuitive importance we want to give to customers that look like the treated, but didn't get the treatment. It will also assign a high importance for those that look like the untreated, but got the treatment. The idea is pretty simple: just **weight every unit by the inverse probability of the treatment they got**. So, if a unit got the treatment, weigh it by $1/P(treatment = 1)$. If a unit did not get the treatment, weigh it by $1/P(treatment = 0)$ or $1/P(control)$. In the case of binary treatment, this is simply

$$W_i = \frac{T}{P(T = 1|X)} + \frac{1 - T}{1 - P(T = 1|X)}$$

For multiple treatments, we can generalize it to

$$W_i = \sum_{k=0}^{K} \frac{1_{K=k}}{P(T = k|X)}$$

Now that we had an idea about how to use the propensity score to do debiasing, let's check how this works in practice. But first, let's see what happens when we don't debias our dataset.

Like I've said before, the treatment is confounded by the customer's characteristics: age, income, how much they have on insurance and how muc they have on hinvestment.

```
confounders = ["age", "income", "insurance", "invested"]
```

If we look at their correlation structure, we can see that these variables are indeed confounders. They are correlated both with the treatment `em1` and the outcome `converted`.

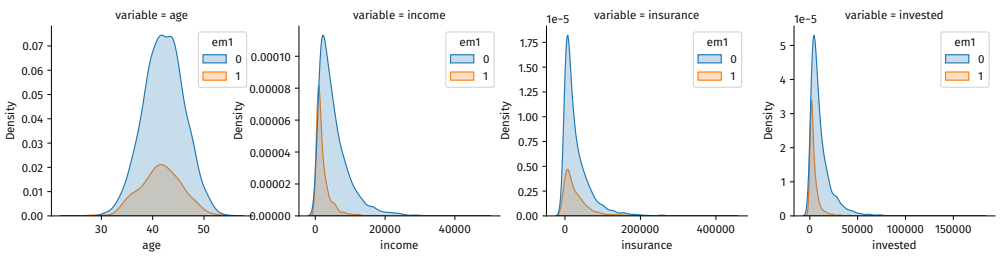
```
email[confounders + ["em1", "converted"]].corr()[["em1", "converted"]]
```

	em1	converted
age	-0.058796	0.401759
income	-0.305610	-0.054009
insurance	-0.023948	-0.076234
invested	-0.242374	-0.030250
em1	1.000000	0.046598
converted	0.046598	1.000000

If we fail to account for this confounding bias, our causal estimates will be wrong. To give an example, consider the `invested` variable. Those that invest **less** are **more** likely to convert **and** also **more** likely to receive the email. Hence, if we don't control for investments, it will look like `em1` increases the chance of conversion. But that could be just due to correlation, not causation, as lower levels of investments leads to both higher conversion and a higher chance of getting `em1`.

Besides looking at the correlation with the confounders, we can also check how the distribution looks like for those that did and didn't get the email `em1`.

```
plt_df = pd.melt(email[confounders + ["em1"]], ["em1"], confounders)

g = sns.FacetGrid(plt_df, col="variable", hue="em1", col_wrap=4, sharey=False,
  ↪ sharex=False)

for i, ax in enumerate(g.axes):
    iter_df = plt_df.loc[lambda df: df["variable"] == confounders[i]]
    sns.kdeplot(x="value", hue="em1", data=iter_df, ax=ax, fill=True)
    ax.set_xlabel(confounders[i])

plt.show()
```



Stored Propensity Score

Now that we've confirmed that the assignment of `em1` is indeed biased, we can work on debiasing it with the propensity score. Now it's a good time to answer a question that might be in your head: why can't I just use orthogonalization? Or, more elegantly, when

should I use the propensity score instead of orthogonalization. That's a good question and one I confess I don't have all the answers for. However, there is one clear case when there is a strong argument for the propensity score.

When you've stored the probabilities of receiving the treatment while conducting your experiment, **propensity score debiasing can be done without having to estimate a model**. If the assignment of the emails was done in a probabilistic way and we've stored those probabilities, then we don't have to rely on models. That's a huge advantage, because models are never perfect, which means that debiasing with them is also never perfect. Here, we have this situation where we've stored the probabilities of the treatments in the columns `em1_ps`, `em2_ps` and `em3_ps`.

Since we are dealing only with `em1`, we only need the probability in `em1_ps` to do the debiasing. Here is what we'll do. First, we will generate the debiasing weights by using the formula we've seen above. Then, we will resample with replacement from this dataset, using the newly created weights. This means a unit with weight 2 will be resampled twice as often as a unit with weight 1.

```
np.random.seed(123)
em1_rnd = email.assign(
    em1_w = email["em1"]/email["em1_ps"] + (1-email["em1"])/(1-email["em1_ps"])
).sample(10000, replace=True, weights="em1_w")
```

```
np.random.seed(5)
em1_rnd.sample(5)
```

	age	income	insurance	invested	em1_ps
2252	44.0	5102.43	42300.28	2294.60	0.097928
4364	44.0	8503.72	751.08	33314.37	0.058805
752	44.4	1778.21	24615.45	3054.08	0.279973
3656	39.8	1752.18	487.22	1893.95	0.284108
4613	43.1	1023.04	64695.01	2220.32	0.484638

	em2_ps	em3_ps	em1	em2	em3	converted	em1_w
2252	0.012976	0.902465	0	0	1	0	1.108559
4364	0.188387	0.830375	1	0	0	1	17.005333
752	0.017270	0.972922	1	0	1	1	3.571777
3656	0.010710	0.000000	0	0	0	0	1.396859
4613	0.012556	0.988928	0	0	1	1	1.940382

This resampling should make a new dataset that is debiased. It should have oversampled units that looked like the treated (high `em1_ps`) but did not get the treatment and those that looked like the control (low `em1_ps`), but got the treatment.

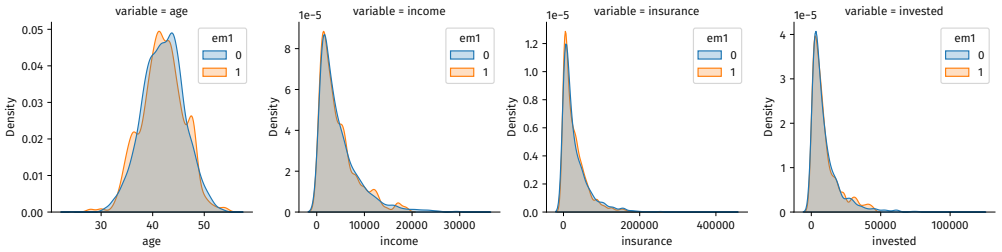
If we look at correlations between the treatment and the confounders, we can see that they essentially vanished.

```
em1_rnd[confounders + ["em1", "converted"]].corr()[["em1", "converted"]]
```

	em1	converted
age	-0.021004	0.404736
income	-0.026685	-0.037946
insurance	-0.037656	-0.071385
invested	-0.012073	-0.047987
em1	1.000000	0.076342
converted	0.076342	1.000000

Moreover, if we look at the confounders distributions by treatment assignment, we can see how nicely they align. This is not 100% proof that the debiasing worked, but it's good evidence of it.

```
plt_df = pd.melt(em1_rnd[confounders + ["em1"]], ["em1"], confounders)

g = sns.FacetGrid(plt_df, col="variable", hue="em1", col_wrap=4, sharey=False,
  ↪ sharex=False)

for i, ax in enumerate(g.axes):
    iter_df = plt_df.loc[lambda df: df["variable"] == confounders[i]]
    sns.kdeplot(x="value", hue="em1", data=iter_df, ax=ax, fill=True)
    ax.set_xlabel(confounders[i])

plt.show()
```



This new dataset we've created is now debiased. We can use it to do model evaluation or any other analys that requires the treatment to be randomly assigned. There is only one thing you need to watch out for. Notice how I've sampled 10000 points but the original dataset had only 5000? With this resampling method, I can make a debiased dataset as big as I wish. This means that confidence intervals that are computed in it are not valid, as they don't take into account the fact that the sample size can be artificially inflated.

Ok, so this technique was super effective because we had the stored probabilities in the first place, but what if we don't? What if we only have access to the confounders and the treatment that got assigned, but not to how likely those units were to get the treatment?

Estimated Propensity Score

If we don't have the propensity score stored, we will have to estimate them. In this situation, it becomes less clear when you should use propensity score or orthogonalisation for debiasing.

Since we don't have the propensity score, we will use a machine learning model to estimate it. The propensity score is closely related to the probability of treatment, so this ML model must be calibrated to output a probability. Not only that, we need to do cross prediction to work around any sort of bias we might have due to overfitting.

```
from sklearn.model_selection import cross_val_predict
from sklearn.ensemble import RandomForestClassifier
from sklearn.calibration import CalibratedClassifierCV

t = "em1"

folds = 5

np.random.seed(123)

# makes calibrated Random Forest.
m_t = CalibratedClassifierCV(
    RandomForestClassifier(n_estimators=100, min_samples_leaf=40, max_depth=3),
    cv=3
)

# estimate PS with cross prediction.
ps_score_m1 = cross_val_predict(m_t, email[confounders], email[t],
                                cv=folds, method="predict_proba")[:, 1]
```

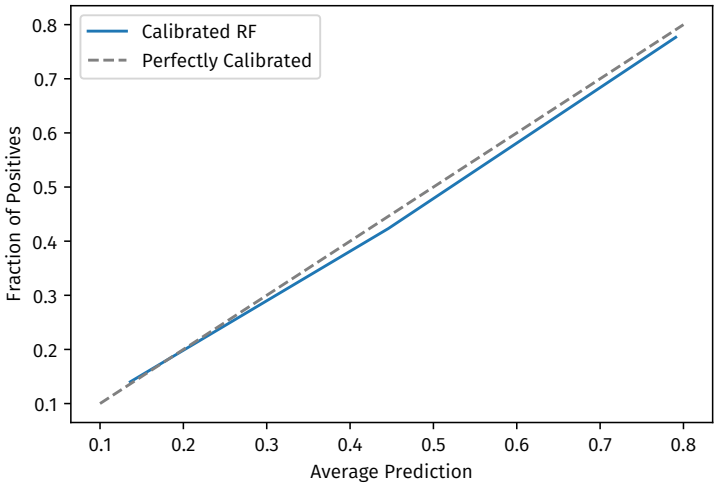
```
email.assign(ps_score_m1_est = ps_score_m1).head()
```

	age	income	insurance	invested	em1_ps	em2_ps
0	44.1	4214.66	14036.41	7988.66	0.118507	0.045174
1	39.8	1151.27	66622.15	1000.74	0.431123	0.005659
2	49.0	8047.26	14119.53	29480.22	0.062137	0.166706
3	39.7	10558.37	14798.26	36373.09	0.047372	0.205684
4	35.3	802.15	780.84	1643.16	0.616450	0.009292

	em3_ps	em1	em2	em3	converted	ps_score_m1_est
0	0.921281	0	0	1	1	0.104727
1	0.000000	1	0	0	0	0.389991
2	0.840049	0	0	1	0	0.083805
3	0.000000	0	0	0	0	0.086290
4	0.000000	1	0	0	0	0.737111

Just out of curiosity, notice that the estimated propensity score, `ps_score_m1_est`, is close to the true one `em1_ps`, but not identical. Those errors in the estimation process will affect the final debiasing, but we hope it won't affect much. We can also check how well calibrated our score is. For that, we can plot the mean score against the mean `em1`. If the score is well calibrated, 20% of those with a score of 0.2 should have received email-1, 30% of those with a score of 0.3 should have received email-1 and so on.

```
from sklearn.calibration import calibration_curve

prob_true, prob_pred = calibration_curve(email["em1"], ps_score_m1, n_bins=3)
plt.plot(prob_pred, prob_true, label="Calibrated RF")
plt.plot([.1,.8], [.1, .8], color="grey", linestyle="dashed",
  ↪ label="Perfectly Calibrated")
plt.ylabel("Fraction of Positives")
plt.xlabel("Average Prediction")
plt.legend();
```



From here, we can proceed as if we had the true propensity score.

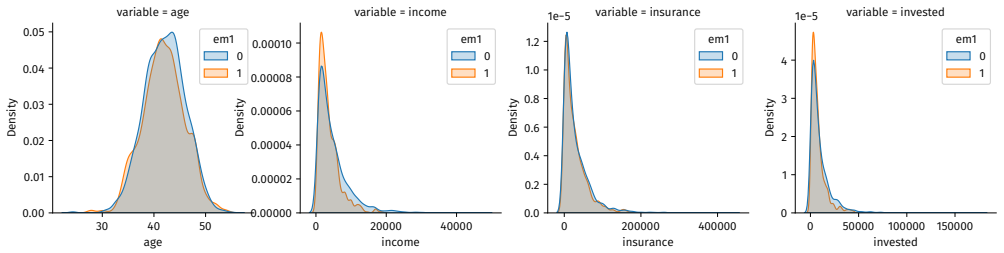
```
np.random.seed(123)
em1_rnd_est = email.assign(
    em1_w = email["em1"]/ps_score_m1 + (1-email["em1"])/(1-ps_score_m1)
).sample(10000, replace=True, weights="em1_w")
```

If we check the correlation structure, we will see that there are still some strong correlations between the treatment and the confounders, even after the debiasing process. For instance, income has a correlation of `-0.18` this is lower than the correlation in the unbiased dataset (`-0.3`), but much higher than what we got in the dataset that was debiased with the original propensity score (`0.01`). The same is true for the `invested` variable, which still shows some correlation.

```
em1_rnd_est[confounders + ["em1"]].corr()["em1"]
```

```
age          -0.030409
income       -0.150937
insurance    -0.037893
invested     -0.111138
em1           1.000000
Name: em1, dtype: float64
```

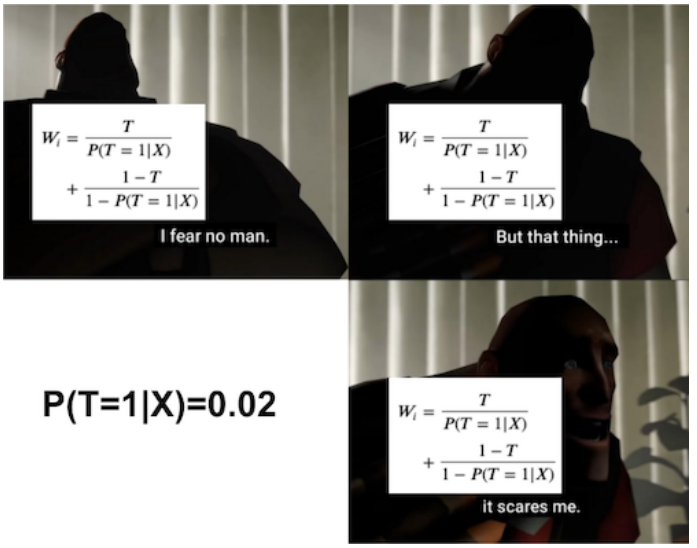
```
plt_df = pd.melt(em1_rnd_est[confounders + ["em1"]], ["em1"], confounders)

g = sns.FacetGrid(plt_df, col="variable", hue="em1", col_wrap=4, sharey=False,
  ↪ sharex=False)

for i, ax in enumerate(g.axes):
    iter_df = plt_df.loc[lambda df: df["variable"] == confounders[i]]
    sns.kdeplot(x="value", hue="em1", data=iter_df, ax=ax, fill=True)
    ax.set_xlabel(confounders[i])

plt.show()
```



As for the distributions, we can see that they don't align that well as before, specially for the variables `invested` and `income`

The Weakness of Propensity Score

We've already talked a lot about this on the propensity score chapter, so I won't spend much time on it, but it is recalling one of the main weaknesses with the propensity score debiasing. It has to do with propensities scores that are two high or too low.



To understand this, take a look at the following unit in our original dataset.

```
email.loc[[1014]]
```

	age	income	insurance	invested	em1_ps
1014	43.2	18628.09	30356.68	28335.68	0.026862

	em2_ps	em3_ps	em1	em2	em3	converted
1014	0.160233	0.615788	1	0	0	1

This unit has a propensity score of 0.027 for `em1`. This means it's weight will be around 37 (1/0.027). This unit will be sampled almost twice as much as a treated unit with a propensity score of 0.05 (weight of 20), which is already a very low propensity score. This unit appears 38 times in the dataset we've resampled using the stored (not estimated) propensity score.

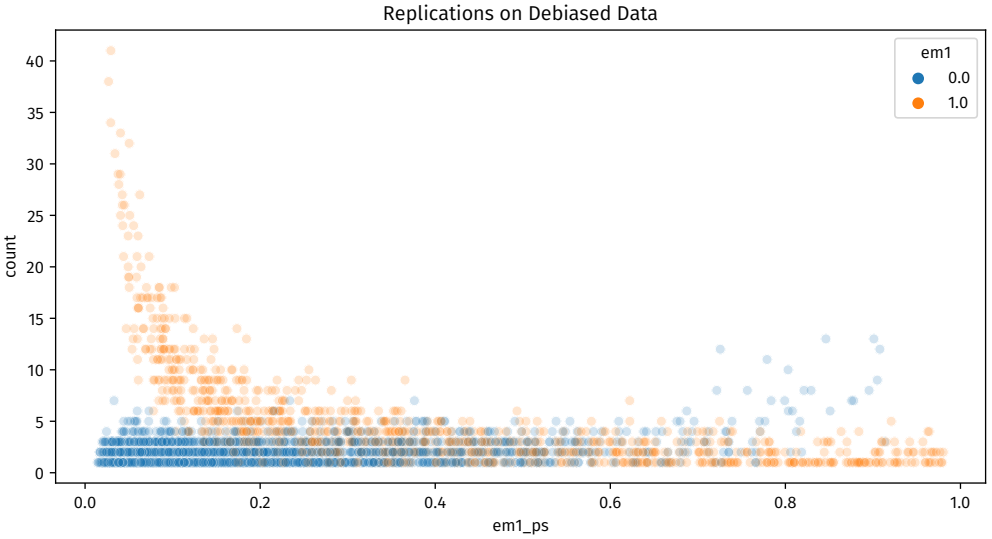
```
em1_rnd.loc[[1014]].shape
```

```
(38, 12)
```

That's a problem, because the debiased dataset is overpopulated with only one unit. If this unit wasn't in the original dataset, the debiased dataset could look totally different. So, removing a single unit can affect a lot how a debiased dataset looks like. This is a problem of high variance.

If we plot the number of replications for each unit in the debiased dataset, we see that a bunch of them appear more than 10 times. Those are treated units with low propensity score or untreated units with high propensity score.

```
plt.figure(figsize=(10,5))
sns.scatterplot(
    data=em1_rnd.assign(count=1).groupby(em1_rnd.index).agg({"count":"count",
      ↪ "em1_ps": "mean", "em1": "mean"}),
    x="em1_ps",
    y="count",
    hue="em1",
    alpha=0.2
)
plt.title("Replications on Debiased Data");
```



To avoid having overly important units, some scientists like to clip the weights, forcing them to be, let's say, at most 20. It is true that this technique will remove some of the variance, but it will add bias back. Since the whole point of this was to remove bias, I feel like clipping the weights is not a good idea.

In my opinion, the best thing you can do is at the experimentation phase, not at the analysis phase. You should make sure none of the units have a weight that is too high. In other words, as much as possible, try to give the treatment with equal probabilities to everyone.

To be fair to the propensity weighting method, all causal inference methods will suffer whenever the probability of receiving either the treatment or the control is too low, for the entire sample or for a subpopulation. Intuitively, this means some units will almost never receive the treatment or the control, which will make it difficult to estimate

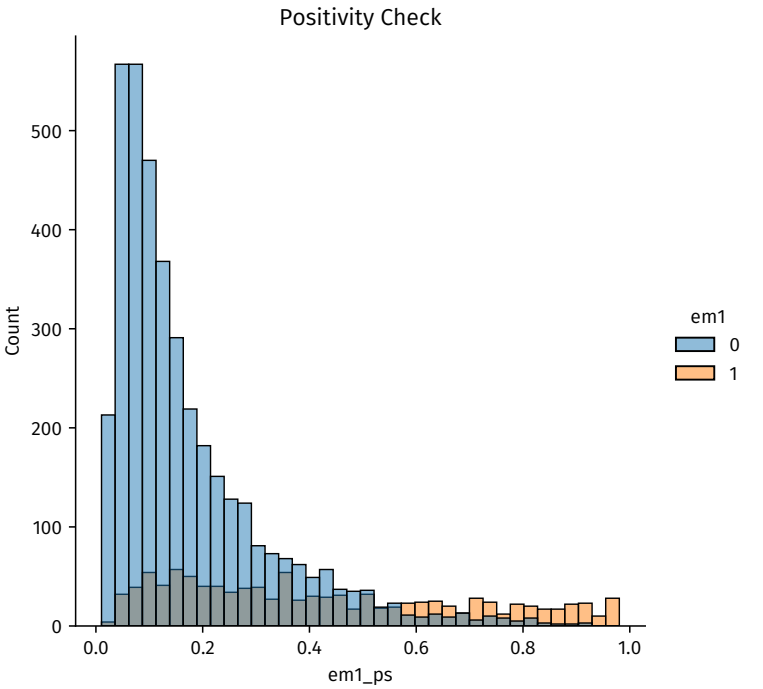
counterfactuals for those units. This problem can be seen as almost violating the common support assumption, which we will explore next.

Positivity or Common Support

Besides high variance, we can also have problems with positivity. Positivity, or common support is a causal inference assumption which states that there must be sufficient overlap between the characteristics of the treated and the control units. Or, in other words, that everyone has a non zero probability of getting the treatment or the control. If this doesn't happen, we won't be able to estimate a causal effect that is valid for the entire population, only for those we have common support.

To be clear, positivity issues are problems of the data itself, not a problem with the propensity score method. Propensity score method just makes it very clear when positivity problems exist. In that sense, this is an argument in favor of the propensity score. While most methods will not warn you when there are positivity problems, propensity score will show it to your face. All you have to do is plot the distribution of the propensity score for the treated and for the untreated.

```
sns.displot(data=email, x="em1_ps", hue="em1")
plt.title("Positivity Check");
```



If they have a nice overlap, like in the plot above, then you have common support.

Up until now, we've only looked at email-1 (`em1`). Let's look at email-3 now.

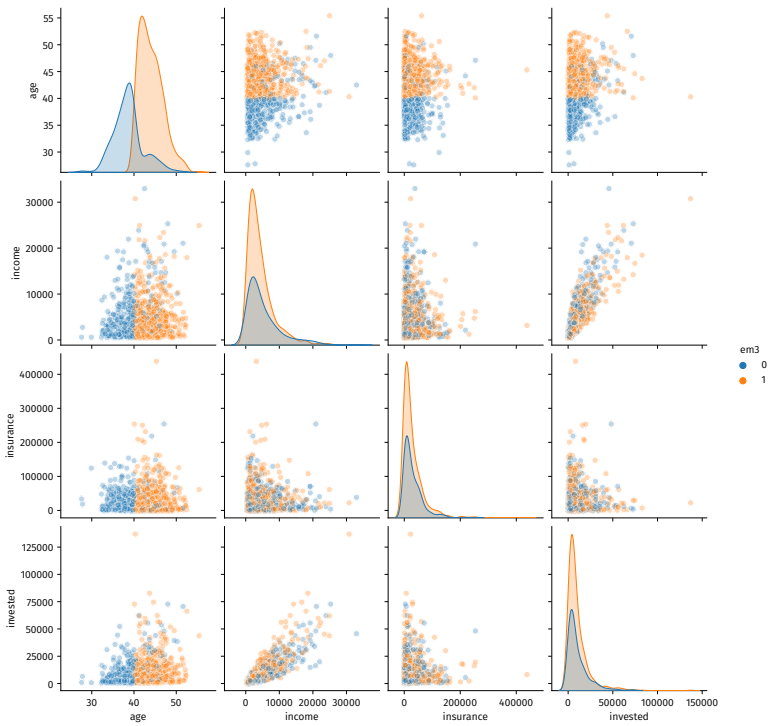
```
email.head()
```

	age	income	insurance	invested	em1_ps
0	44.1	4214.66	14036.41	7988.66	0.118507
1	39.8	1151.27	66622.15	1000.74	0.431123
2	49.0	8047.26	14119.53	29480.22	0.062137
3	39.7	10558.37	14798.26	36373.09	0.047372
4	35.3	802.15	780.84	1643.16	0.616450

	em2_ps	em3_ps	em1	em2	em3	converted
0	0.045174	0.921281	0	0	1	1
1	0.005659	0.000000	1	0	0	0
2	0.166706	0.840049	0	0	1	0
3	0.205684	0.000000	0	0	0	0
4	0.009292	0.000000	1	0	0	0

First thing that jumps the eye is that there are units with zero probability, which already indicates a violation to the positivity assumption. Now, let's look at the features distributions by `em3`

```
sns.pairplot(email.sample(1000)[confounders + ["em3"]], hue="em3",
  ↪ plot_kws=dict(alpha=0.3));
```



It looks like em3 was only sent to customers that are older than 40 years. Thats a huge problem. If the control has younger folks, but the treatment doesn't, there is no way we can estimate the counterfactual $Y_0|T=1, age<40$. Simply because we have no idea how younger customers will respond to this email.

Just so you know what will happen, we can try to make the propensity score debiasing.

```
em3_weight = (email
              # using a different implementation to avoid division by zero
              .assign(em3_w = np.where(email["em3"].astype(bool),
                ↪ 1/email["em3_ps"], 1/(1-email["em3_ps"])))
              .sample(10000, replace=True, weights="em3_w"))
```

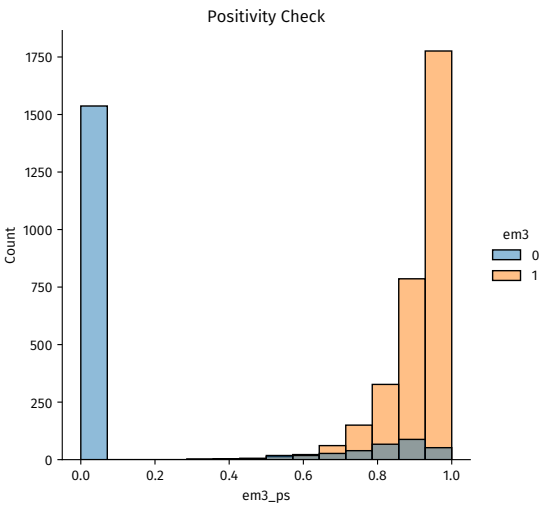
```
em3_weight[confounders + ["em3"]].corr()["em3"]
```

```
age          0.251035
income       0.047607
insurance   -0.000961
invested     0.053524
em3          1.000000
Name: em3, dtype: float64
```

As we can see, there is still a huge correlation between age and the treatment. Not having treated samples that are younger than 40 years made it so that we couldn't remove the bias due to age.

We can also run the positivity diagnostic, plotting the propensity score distribution by treatment.

```
sns.displot(data=email, x="em3_ps", hue="em3")
plt.title("Positivity Check");
```



Notice how poor the overlap is here. Units with propensity score below 0.4 almost never get the treatment. Not to mention that huge peak at zero.

Positivity In Practice

Issues in positivity are very serious in the academic world because they treat the generalization of a conclusion or theory. If you only have treated individuals that are older, you have no way of generalizing whatever treatment effect that you find to a younger population. But in the industry, violations of the positivity assumption might not be so problematic.

To give an example, if you are a lender wishing to estimate the elasticity of loan amount on probability of default, you will probably not give high loans to people with very low credit scores. Sure, this will violate the positivity assumption, but you are not very interested in estimating the loan amount elasticity for risky customers because you are not intending to give them loans anyway. Or, for a somewhat more exaggerated example, if you want to estimate the price elasticity of some product, you will probably not test prices from 0 to 1000000. That's because you have some intuition on where the prices normally are. You will often test around that region.

The point here is that you can (and probably should) use your intuition to exclude some part of the population from receiving the treatment (or the control). Doing random experiments is expensive, so you should focus your efforts on where things are more promising.

In our email example, probably the marketing team though that email-3 should never be sent to younger people. Maybe it contains text explicitly discussing something that only older people can relate to. Regardless of the reason, we can only estimate the effect of email-3 on the older, age>40, population. And that is totally fine. We just shouldn't expect to generalize whatever findings we have to the younger population.

With that in mind, let's debias email-3. Of course, we will remove the younger population from the sample.

```
em3_weight.query("age>40")[confounders + ["em3"]].corr()["em3"]
```

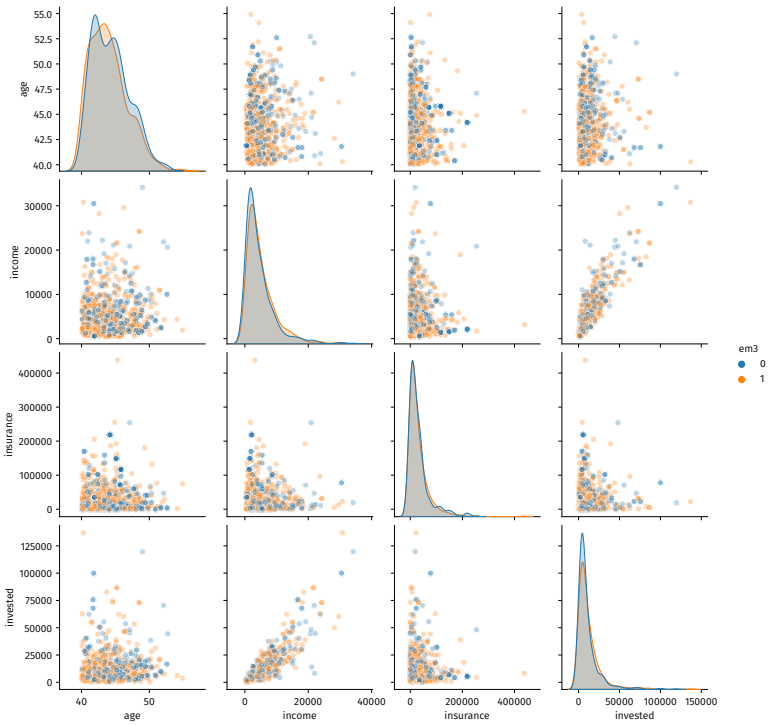
```
age         -0.034455
income       0.024633
insurance   -0.005650
invested     0.020711
em3          1.000000
Name: em3, dtype: float64
```

Once we do that, notice how the correlation between the treatment and the confounders goes away. Remember that in the entire sample, the correlation with age was more than 0.2. Now, it is probably indistinguishable from zero.

We can also explore the distribution of treated and non treated in this filtered sample.

```
sns.pairplot(em3_weight.query("age>40").sample(1000)[confounders + ["em3"]],
  ↪ hue="em3", plot_kws=dict(alpha=0.3))
```

```
<seaborn.axisgrid.PairGrid at 0x132656b70>
```



All of this to say that, yes, due to positivity violations, you can't debias the entire sample with respect to `em3`. But you can do it for the population for which email-3 was designed for in the first place. That is good enough.

Key Ideas

This chapter presented another debiasing idea for when the treatment is binary or discrete. The idea is to resample the units using the inverse propensity score as a weight. This will upsample the units that look like the treated (high propensity score) but where not treated and those that look like the untreated (low propensity score) but where treated.

The main advantage of propensity score debiasing is that it doesn't require estimating a model, if you've stored the probabilities of treatment during the experiment. This doesn't mean you can't apply propensity reweighting if you don't have those probabilities, but having to estimate them will add additional error to your debiasing process.

As for the disadvantages, the propensity score reweighting can have a huge variance if the propensity score is either too high or too low, generating huge weights. Intuitively, this

happens when there are very few untreated units that look like the treated and very few treated units that look like the untreated. To be fair to the propensity score weighting, this is a problem for all causal inference methods, but propensity weighting highlights this almost violation to the positivity assumption more clearly.

Finally, we had a brief discussion about how much of a problem is violation in the positivity assumption. We ended up concluding that it is not a problem **as long as you don't wish to generalize your conclusions to the subset of the population where positivity doesn't hold**.

References

The things I've written here are mostly stuff from my head. I've learned them through experience. This means that they have **not** passed the academic scrutiny that good science often goes through. Instead, notice how I'm talking about things that work in practice, but I don't spend too much time explaining why that is the case. It's a sort of science from the streets, if you will. However, I am putting this up for public scrutiny, so, by all means, if you find something preposterous, open an issue and I'll address it to the best of my efforts.

Most of this chapter draws from Guido W. Imbens, Causal Inference for Statistics, Social, and Biomedical Sciences, where you will find extensive discussions on the propensity score. The discussion on positivity is on me though, from countless debates I had at work with my colleagues.

APPENDIX

C

When Prediction Fails

When all you have is a Hammer...

Between 2015 and 2020, Machine Learning went through a massive surge. Its proven usefulness in the fields of computer vision and natural language understanding, coupled with an initial lack of professionals in the area, provided the perfect opportunity for a machine learning teaching industry. Figures like Andrew Ng and Sebastian Thrun managed to teach machine learning to the world at rock bottom prices. At the same time, on the software side, it became increasingly easier to fit a complex machine learning model (as you've already seen by the very few lines of code it took us to write an ML in the previous chapter). Tutorials about how to make intelligent systems sprung all over the internet. The cost of entry in ML plummeted.

Build a TensorFlow Image Classifier in 5 Min

Building ML became so simple that you didn't even need to know how to code very well (and I'm living evidence of that), nor the math behind the algorithms. In fact, you could build wonders with the following 5 lines of Python.

```
X_train, y_train, X_test, y_test =  train_test_split(X, y)

## instantiate the machine learning model
model = MachineLearningModel()

## Fit the ML model
model.fit(X_train, y_train)

# Make predictions on unseen data
y_pred = model.predict(X_test)

# Evaluate the quality of predictions
print("Performance", metric(y_test, y_pred))
```

For the most part, this is an amazing thing! I'm all in for taking valuable content and making it available. However, there is also a dark side to all of this. This new wave of data scientists were trained mostly in predictive modeling, since that is what ML primarily focuses on solving. As a result, whenever those data scientists encountered a business problem, they tried to tackle it with, not surprisingly, predictive models. When they were indeed prediction problems, like the one we saw in the previous chapter, the data scientist usually succeeded and everyone got happy. However, there is an entire class of problems that are simply not solvable with prediction techniques. And when those appeared, the data scientists usually failed miserably. These are problems that are framed like "how much can I increase Y by changing X".

From my experience, this other type of problem is what management usually cares the most about. They often want to know how to increase sales, decrease cost or bring in more customers. Needless to say, they are not very happy when a data scientist comes up with an answer to how to predict sales instead of how to increase it. Sadly, when everything the data scientist knows is predictive models, this tends to happen a lot. As a boss of mine once told me: "when all you have is a hammer, everything starts to look like a thumb".

Like I've said, I'm all in for lowering the cost of knowledge, but the current Data Scientist curriculum has a huge gap. I think that my job here is to fill in that gap. Is to equip you with tools to solve this other class of problems, which are causal in nature.

What you are trying to do is estimate how something you can control (advertisement, price, customer service) affects or causes something you want to change, but can't control directly (sales, number of customers, PNL). But ,before showing you how to solve these problems, I want to show you what happens when you treat them like prediction tasks and try to solve them with the traditional ML toolkit. The reason for it is that data scientists often come to me and say "OK, but although tackling causal problems with prediction tools is not the best idea, it surely helps something, no? Imean, it couldn't hurt…". Well, as it turns out, it can. And you better understand this before you go on hammering your own thumb.

```
import pandas as pd
import numpy as np
from sklearn import ensemble
from sklearn.model_selection import train_test_split, cross_val_predict
from sklearn.ensemble import gradient_boosting
from sklearn.metrics import r2_score
import seaborn as sns
from matplotlib import pyplot as plt
from matplotlib import style
style.use("ggplot")

# helper functions for this notebook
from nb18 import ltv_with_coupons
```

Who Wants a Coupon?

To make matters more relatable, let's continue with the example we used in the previous chapter, but with a little twist to it. Before, we were trying to distinguish the profitable from the non profitable customers. We framed that as a prediction problem: predicting customer profitability. We could then build a machine learning model for this task and use it to choose who we would do business with: only the customers we predicted to be profitable. In other words, our goal was to separate the profitable from the non-profitable, which we could do with a predictive model.

Now, you have a new task. You suspect that giving coupons to new customers increases their engagement with your business and makes them more profitable in the long run. That is, they spend more and for a longer period. Your new assignment is to figure out how much the coupon value should be (zero included). Notice that, with coupons, you are essentially giving away money for people to spend on your business. For this reason, they enter as a cost in your book account. Notice that if the coupon value is too high, you will probably lose money, since customers will buy all they need using only the coupons. That's another way of saying that they will get your product for free. On the flip side, if coupon value is too low (or zero), you are not even giving coupons. This could be a valid

answer, but it could also be that some discounts upfront, in the form of coupons, will be more profitable in the long run.

For reasons you will see later, we will use a data generating function instead of loading a static dataset. The function `ltv_with_coupons` generates transaction data for us. As you can see, they have the same format as the one we saw previously, with one row per customer, a column for the cost of acquisition and columns for the transactions between day 1 and 30.

```
transactions, customer_features = ltv_with_coupons()

print(transactions.shape)
transactions.head()
```

```
(10000, 32)
```

	customer_id	cacq	day_0	day_1	day_2	day_3	day_4	day_5	day_6	day_7	...
0	0	-110	0	0	0	0	5	0	2	2	...
1	1	-61	2	0	5	2	3	4	1	0	...
2	2	-8	0	0	0	0	0	0	0	0	...
3	3	-30	0	0	0	0	0	0	0	0	...
4	4	-41	0	0	0	2	0	4	0	0	...

	day_20	day_21	day_22	day_23	day_24	day_25	day_26	day_27	day_28	day_29
0	0	3	0	4	0	0	0	0	0	0
1	5	0	1	35	11	0	5	2	4	0
2	0	0	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0

5 rows × 32 columns

As for the other parts of the data, again, we have a customer identifier, the region the customer lives, the customer income and the customer age. In addition, we now have a variable that is coupons, which tells us how much we've given in coupons for that customer.

```
print(customer_features.shape)
customer_features.head()
```

```
(10000, 5)
```

	customer_id	region	income	coupons	age
0	0	18	1025	5	24
1	1	40	1649	5	26
2	2	35	2034	15	33
3	3	29	1859	15	35
4	4	11	1243	5	26

To process this data to a single dataframe, we will sum all the columns in the first table (that is, summing CACQ with the transactions).This will give us the net_value as it was computed in the previous chapter. After that, we will join in the features data and update the net_value to include the coupon cost.

```
def process_data(transactions, customer_data):

    profitable = (transactions[["customer_id"]]
                  .assign(net_value = transactions
                          .drop(columns="customer_id")
                          .sum(axis=1)))

    return (customer_data
            # join net_value and features
            .merge(profitable, on="customer_id")
            # include the coupons cost
            .assign(net_value = lambda d: d["net_value"] - d["coupons"]))

customer_features = process_data(transactions, customer_features)
customer_features.head()
```

	customer_id	region	income	coupons	age	net_value
0	0	18	1025	5	24	-44
1	1	40	1649	5	26	74
2	2	35	2034	15	33	-23
3	3	29	1859	15	35	-45
4	4	11	1243	5	26	-26

This processed data frame has all that we need. It has our target variable net_value, it has our customer features region, income and age, and it has the lever or treatment we want to optimise for: coupons. Just to begin understanding how coupons can increase net_value, let's look at how they were given away.

```
customer_features.groupby("coupons")["customer_id"].count()
```

```
coupons
0      458
5     4749
10    4154
15     639
Name: customer_id, dtype: int64
```

We can see that most of the coupons that were handed out had a value of 5 BRL, followed by the coupons with 10 BRL in value. We gave very few 15 BRL coupons or no coupons at all (zero value). This is indicative that they were **NOT** given randomly. To check that, let's see the correlation between the other variable and coupons.

```
customer_features.corr()[["coupons"]]
```

	coupons
customer_id	0.002239
region	-0.004073
income	0.001236
coupons	1.000000
age	0.897308
net_value	-0.078369

That's interesting. It looks like the older the person, the higher the probability he or she will receive a coupon. This is some indication of bias in our data. We can also see a negative correlation between coupons and `net_value`: the more coupons we give, the smaller the net_value. This is hardly causal, since we already know that coupons were not randomly distributed. It could be that, say, older people spend less on our products and also receive higher coupon values, confounding the relationship between coupons and `net_value` to the point of making it negative.

The point being, we know there is bias. However, since there is already too much packed into this chapter, I'll ignore it for now (actually, I'll bypass the problem with an artifact you will see in just a moment). Just keep in mind it's something we will have to address sometime in the future.

At this point in the analysis, **if this was a prediction problem**, we would probably split the dataset into a training and a test set to build and evaluate some policies, respectively. But this is NOT a prediction problem. The final goal here is not to get a good prediction on customer profitability. Instead, it's to figure out the optimal coupon strategy. To evaluate this optimization, we would have to know how things would have played out if we have given different coupons than the ones that were given. This is the sort of counterfactual "what if'' question we've been studying under causality. Cross validation won't help us here because we simply can't observe counterfactuals. We can only see what happened for the coupons that were actually given, but we can't know what would have happened if customers received a different coupon value. Unless, we have simulated data!

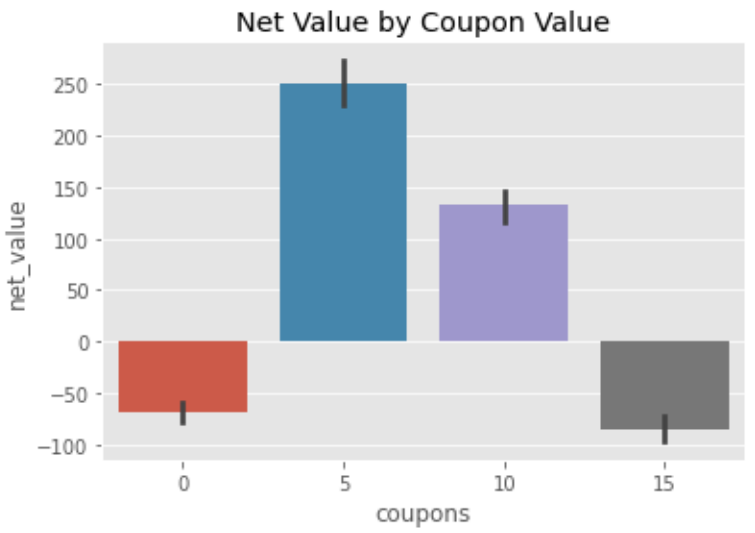
If our data is simulated, we can generate the exact same data, only changing the coupon value parameters. This will allow us to see how `net_value` changes under different coupons strategies. We will then be able to calculate the treatment effect between different strategies $NetValue_{t=a} - NetValue_{t=b}$. With the power of simulated data, understanding this chapter will be much easier. Oh yes, and this will also render the bias problem irrelevant, because we will observe the causal effect directly.

Nevertheless, always remember that this is a pedagogical artifact. In the real world, you don't have simulated data and you certainly can't see what would have happened under different treatment strategies. Individual causal effects remain hidden as they always have been. This poses an interesting problem. How can we evaluate our strategies for identifying causal effect if we can never see the actual causal effect? The real answer is very involved and so important that it deserves its own chapter. Rest assured that we will tackle it. For now, just enjoy the simplicity of simulated data. And speaking of simplicity…

Simple Policy

As always, the first thing we should do whenever we encounter a new data problem is to ask ourselves "what is the simplest thing I can do that will already bring value?". For this specific case, the simplest thing is to look back on the data that we have and estimate the `net_value` for each coupon value. Then, check which coupon value is generating the highest `net_value` and give only that coupon value for every customer.

```
sns.barplot(data=customer_features, x="coupons", y="net_value")
plt.title("Net Value by Coupon Value");
```



Doing that analysis, we can see that, on average, we lose money when the coupon value is 0 or 15 and we gain money for coupons of 5 and 10 BRL. The highest average `net_income`

appears when we have 5 BRL coupons, yielding us about 250 BRL in net_value per customer. Naturally then, the simplest thing we can try is to give everyone 5 BRL in coupons and see how that would play out. This completely disregards the possibility of bias but hey, we are talking simplicity here!

To do evaluate that policy, the function ltv_with_coupons accepts as argument a 10000 array that contains the desired coupon for each of the 10000 customers on our database. To create this array, we will generate an array of ones with np.ones the size of our coupons array (10000) and multiply it by 5. Then, we will pass this array to the ltv_with_coupons. This will generate a new dataset exactly like the one we had previously, but with every coupon value set to 5. We then process this data to get the net value under this newly proposed policy.

```
simple_policy = 5 * np.ones(customer_features["coupons"].shape)

transactions_simple_policy, customer_features_simple_policy = ltv_with_coupons(
  ↪ simple_policy)
customer_features_simple_policy = process_data(transactions_simple_policy,
  ↪ customer_features_simple_policy)

customer_features_simple_policy.head()
```

	customer_id	region	income	coupons	age	net_value
0	0	18	1025	5	24	-44
1	1	40	1649	5	26	74
2	2	35	2034	5	33	63
3	3	29	1859	5	35	63
4	4	11	1243	5	26	-26

Just as a sanity check, let's see if the features are indeed unchanged, considering the first few customers. Take the third one, for example (customer_id 2). For this customer, the region is 35, income is 2034 and the age is 33. If we scroll up a bit, we can see that it matches what we had before, so we are good here. Also, we can check that all the coupons are indeed 5 BRL. Finally, the net_value changes as expected. One reason for this is that the cost associated with coupons will change. For example, that customer had 15 BRL in coupons, but now it's 5. This would decrease the cost from 15 to 5 units. But notice that the net_value goes from -23 to 63, a 86 BRL increase in net_value. This is much larger than the 10 cost difference. Here giving less in the coupons made this particular customer much more profitable than he or she was before. Finally, to evaluate the policy, we can simply take the average net_value.

```
simple_policy_gain = customer_features_simple_policy["net_value"].mean()
simple_policy_gain
```

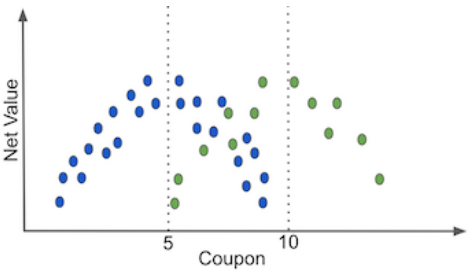
```
252.9268
```

As we can see, this simple policy is telling us we can get, on average, 253 BRL for each customer if we give them all a 5 BRL coupon. This is massive! But can we do better? What if we use our shiny machine learning hammer on this problem? Let's try this next.

Policy With Model

To use ML, we will adapt what we did in the previous chapter. The idea is to build a ML model that predicts `net_value`, just like before, take those predictions and bins them into a defined number of bands. Then, we will partition the data into those bands. Essentially, we are splitting the customer by their predicted `net_value`. Customers that we think will generate roughly the same `net_value` will end up in the same bin or group. Finally, for each group, we will see which coupon value yields the maximum `net_value`. We are doing the same thing as in the simple policy, but now within the groups defined by a prediction band.

The intuition behind this is the following: we know that, on average, 5 BRL coupons performed better. However, it is possible that for some group of customers, another value is even better than 5 BRL. Maybe 5 BRL is the optimal strategy for most of the customers, but not for all of them.



If we can identify the ones where the optimal value is different, we can build a coupon strategy better than the simple one we did above.

This is what we call a personalisation problem. We can leverage personalisation when we have more than one strategy to choose from and at least one of them is not the overall best strategy, but it is the best in a subset of the targeted population. This definition is a bit convoluted, but the intuition is simple. If you have only one strategy, you are not

personalising. You are doing the same thing for every customer. If you have more than one strategy, but one of them is better for every single customer, why will you personalise? You could just do that one best thing. You will only do personalisation if you have one strategy that works better at one subset of the population and another strategy that works best in another subset of the population.

But back to the example. The first thing we need is a function that fits our predictive model and also splits the predictions into the prediction bands. This function will return another function, a prediction function that will take a dataframe and add both predictions and band columns.

```
def model_bands(train_set, features, target, model_params, n_bands, seed=1):

    np.random.seed(seed)

    # train the ML model
    reg = ensemble.GradientBoostingRegressor(**model_params)
    reg.fit(train_set[features], train_set[target])

    # fit the bands
    bands = pd.qcut(reg.predict(train_set[features]), q=n_bands,
      ↪ retbins=True)[1]

    def predict(test_set):
        # make predictions with trained model
        predictions = reg.predict(test_set[features])

        # discretize predictions into bands.
        pred_bands = np.digitize(predictions, bands, right=False)
        return test_set.assign(predictions=predictions,
                               # cliping avoid creating new upper bands
                               pred_bands=np.clip(pred_bands, 1, n_bands))

    return predict
```

To evaluate the quality of our predictions, we will split the dataset into a training and a testing set. Notice here that we are evaluating the quality of the prediction, NOT of the policy. This is just to see if our model is any good at doing what is supposed to do.

```
train, test = train_test_split(customer_features, test_size=0.3, random_state=1)
```

Now, let's train our model and make 10 bands with its predictions.

```
model_params = {'n_estimators': 150,
                'max_depth': 4,
                'min_samples_split': 10,
                'learning_rate': 0.01,
                'loss': 'ls'}

features = ["region", "income", "age"]
target = "net_value"

np.random.seed(1)
model = model_bands(train, features, target, model_params, n_bands=10)
```

After training our model, we can use it to make predictions, passing it a dataframe. The result will also be a dataframe with 2 new columns: `predictions` and `pred_bands`.

```
model(train).head()
```

	customer_id	region	income	coupons	age	net_value	predictions	pred_bands
2228	2228	30	567	5	27	-129	-16.296297	2
5910	5910	32	647	5	25	-55	-16.296297	2
1950	1950	31	2953	15	33	-142	102.237797	7
2119	2119	1	2860	5	27	-23	94.291197	7
5947	5947	49	589	5	26	-91	-3.525593	3

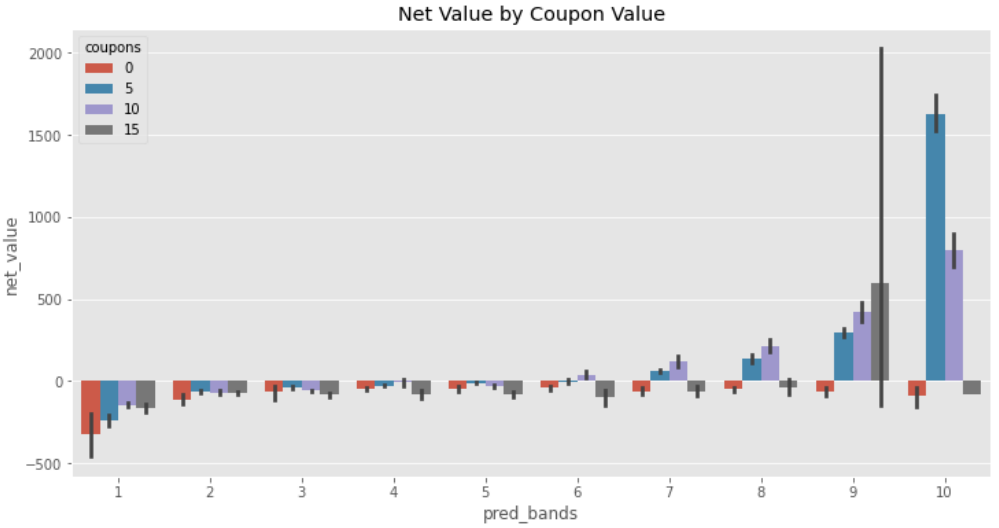
To see the predictive power of our model, we can look at the R^2 for both training and test sets.

```
print("Train Score:, ", r2_score(train["net_value"],
  ↪ model(train)["predictions"]))
print("Test Score:, ", r2_score(test["net_value"], model(test)["predictions"]))
```

```
Train Score:,  0.5382953634651921
Test Score:,  0.504563847410434
```

Remember that this performance is only the predictive performance. What we really want to know is if this model can make us some money. Let's make a policy! The idea here is very similar to what we saw in the previous chapter. We will group the customers by model band. Then, for each type of customer (where type is defined by the bands) we'll see which decision - coupon value in our case - is the best one. To do so, we can group our data by prediction band and coupon value and plot the `net_value`.

```
plt.figure(figsize=(12,6))
sns.barplot(data=model(customer_features), x="pred_bands", y="net_value",
  ↪ hue="coupons")
plt.title("Net Value by Coupon Value");
```



This plot is very interesting. Notice how the optimal decision changes across prediction bands. For instance, on bands like 1, 7 and 8, the best thing to do is to give 10 BRL in coupons. For bands like 3, 5 and 10, the best thing is 5 BRL in coupons. This means that this policy is very much like the simple one, except for the last band. This is evidence that personalisation might be possible, since the optimal decision changes across subpopulations.

We can code that policy with a couple of `if ... then ...` statements, but I'll show a more general approach that leverages dataframe operations.

First, we will group our customers by band and coupon value and take the average `net_value` for each group, much like the plot above.

```
pred_bands = (model(customer_features)
              .groupby(["pred_bands", "coupons"])
              [["net_value"]].mean()
              .reset_index())

pred_bands.head(7)
```

	pred_bands	coupons	net_value
0	1	0	-324.538462
1	1	5	-237.683871
2	1	10	-142.203390
3	1	15	-160.413223
4	2	0	-108.980769
5	2	5	-63.718650
6	2	10	-68.327146

Then, we will group by band and take the `net_value` rank for each row. This will order the rows according to the average `net_value`, where 1 is the best `net_value` in that band.

```
pred_bands["max_net"] = (pred_bands
                         .groupby(['pred_bands'])
                         [["net_value"]]
                         .rank(ascending=False))


pred_bands.head(7)
```

	pred_bands	coupons	net_value	max_net
0	1	0	-324.538462	4.0
1	1	5	-237.683871	3.0
2	1	10	-142.203390	1.0
3	1	15	-160.413223	2.0
4	2	0	-108.980769	4.0
5	2	5	-63.718650	1.0
6	2	10	-68.327146	2.0

For example, for band one, the best coupon strategy is 10 BRL. Next, we will keep only the greatest `net_value` per band.

```
best_coupons_per_band = pred_bands.query("max_net==1")[["pred_bands",
  ↪ "coupons"]]

best_coupons_per_band
```

	pred_bands	coupons
2	1	10
5	2	5
9	3	5
14	4	10
17	5	5
22	6	10
26	7	10
30	8	10
35	9	15
37	10	5

To build our policy, we will take that small table above and join it back on the original table using the band as the key. This will pair each row in the original dataset with what we think is optimal coupon value, according to this policy. Then, we sort the rows according to the `customer_id` so that we keep the same ordering we had previously. This is important for evaluation, since `ltv_with_coupons` takes as argument the coupon value in the order of the original dataframe.

```
coupons_per_id = (model(customer_features)
                  .drop(columns=["coupons"])
                  .merge(best_coupons_per_band, on="pred_bands")
                  [["customer_id", "coupons"]]
                  .sort_values('customer_id'))

coupons_per_id.head()
```

	customer_id	coupons
0	0	10
1	1	10
788	2	5
1982	3	10
2743	4	5

Finally, to evaluate the policy, we pass the `coupons` column as the coupon array to the `ltv_with_coupons` function. This will regenerate the data, now assuming the coupons were given as we defined by this policy.

```
transactions_policy_w_model, customer_features_policy_w_model =
  ↪ ltv_with_coupons(
    coupons_per_id[["coupons"]].values.flatten()
)

customer_features_policy_w_model = process_data(transactions_policy_w_model,
  ↪ customer_features_policy_w_model)

customer_features_policy_w_model.head()
```

	customer_id	region	income	coupons	age	net_value
0	0	18	1025	10	24	-87
1	1	40	1649	10	26	42
2	2	35	2034	5	33	63
3	3	29	1859	10	35	-40
4	4	11	1243	5	26	-26

Just doing a sanity check again, we can see that the third customer is still the one with region 35, income 2034 and age 33. It also has a coupon value of 5 BRL, just like we've established by our policy.

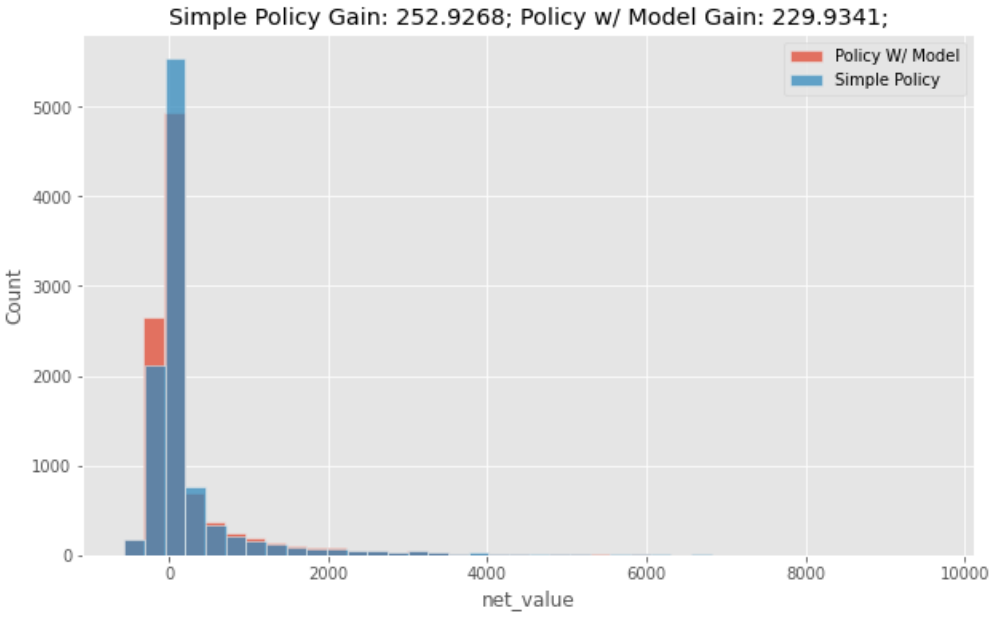
To check how much money this policy is making us, we can compute the average `net_value` mean for this new dataset.

```
policy_w_model_gain = customer_features_policy_w_model["net_value"].mean()
policy_w_model_gain
```

```
229.9341
```

Not bad! We can expect to get about 230 BRL per customer with this model policy. But wait a second! You remember how much we were making with the simple policy? Let's compare both of them side by side.

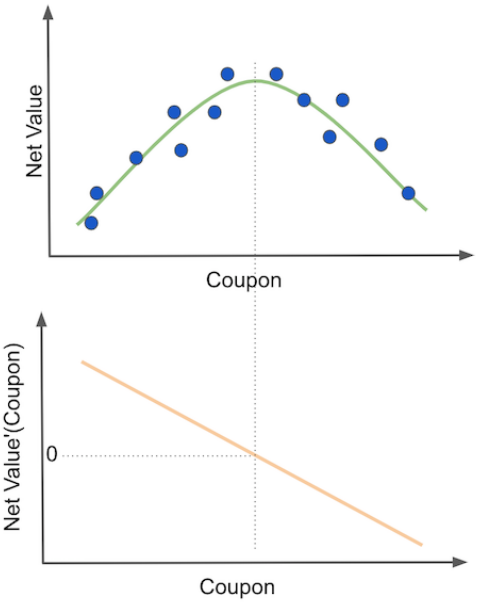
```
plt.figure(figsize=(10,6))
sns.histplot(data=customer_features_policy_w_model, bins=40,
             x="net_value", label="Policy W/ Model", color="C0")
sns.histplot(data=customer_features_simple_policy, bins=40,
             x="net_value", label="Simple Policy", color="C1")
plt.legend()
plt.title(f"Simple Policy Gain: {simple_policy_gain}; Policy w/ Model Gain:
      ↪{policy_w_model_gain};");
```



Here is where most Data Scientists fall off their chair. The policy with the model has an average `net_income` which is 20 BRL worse than a very simple policy. How can a model that is good at predicting `net_value` not be good for a strategy that aims to maximize `net_value`? Surely, there must be some bug in the code. You are clearly mistaken. This cannot be! Well, as it turns out, there is a perfectly reasonable and simple explanation for it. However, the answer to this question is so important that I think it's worth going a bit deeper into it.

Hammering Your Thumb with Predictions

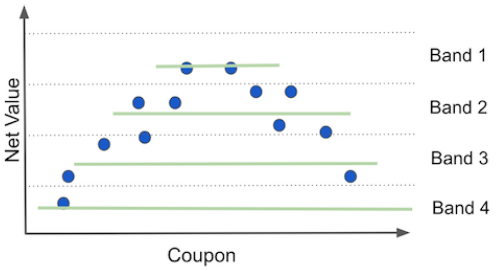
The short answer lies in understanding what we want with this policy, namely, to optimize `net_value` by playing with the coupon values. If we were to put it in an image, it's not crazy to think that `net_value` will have a quadratic shape on coupons: as we increase the coupon value `net_value` first increases, then it reaches a maximum point. After that, any additional coupon value will cost more than the value it brings.



Finding the optimal coupon value is then equivalent to finding the maximum of the `net_value` function. We can do this by differentiating the function and setting it to zero (second plot). Economists might recognize this as a pricing problem. The way they (we) would tackle this problem would be to assume a functional form for `net_value`, differentiate it and optimize it.

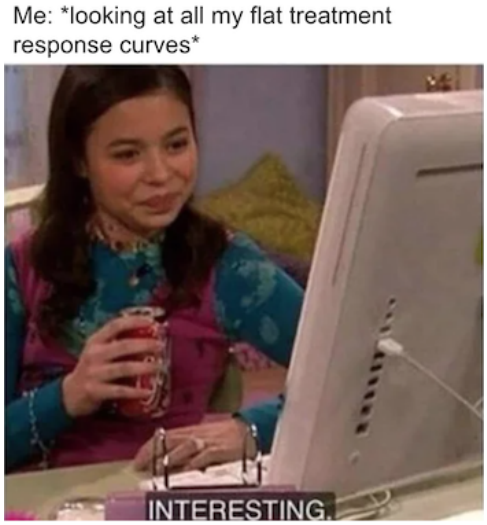
There are some great merits to this approach, but I feel is not that general and it requires a great deal of hypothesizing. Sadly, real world data doesn't come with an underlying function we can differentiate, so guesswork is often involved here. A more practical approach (and perhaps less elegant) is to test multiple coupon values and see which one yields the best `net_value`. This is exactly what our simple policy does. It looks at what happened in the past and repeats what the treatment that was shown to be the most promising one.

Contrast this to what the model based policy does. First, the model based policy fits a machine learning model to predict `net_value`. Then, it partitions the customer space according to the predictions. If the model is good, this is approximately equal to partitioning the space by `net_value` itself, just like in the following plot.



The better the prediction, the more this partitioning of the space approaches partitioning on the target variable, `net_income`. Pay very close attention to what happens when you do that. Essentially, you are splitting the customer into sets where `net_value`, the thing you've predicted, doesn't change! And that makes total sense from a predictive standpoint. If your model is good at predicting, groups of points that have the same prediction will also have the same `net_value`.

So far, so good, but look at what this does to the perceived function of `net_value` on coupons (green lines). It flattens them to have no slope at all. From the predictive point of view, this is awesome. It means that your model has captured all the variation in `net_income`. However, from the policy perspective, this is terrible, because there is no variance left in `net_income` for us to see how it would change given different coupon values. Without this variation in `net_income`, it would look like changing the coupon values has no effect on `net_income` at all, leaving us no room for optimization. By the way, this is a general phenomenon that has nothing to do with the specific quadratic shape we are using here. I'm only using an example to make things more concrete.



To summarize it, whenever we want to optimise some *Y* variable using some *T* variable, predicting *Y* will not only not help, it will hurt our policy, since data partitions defined by

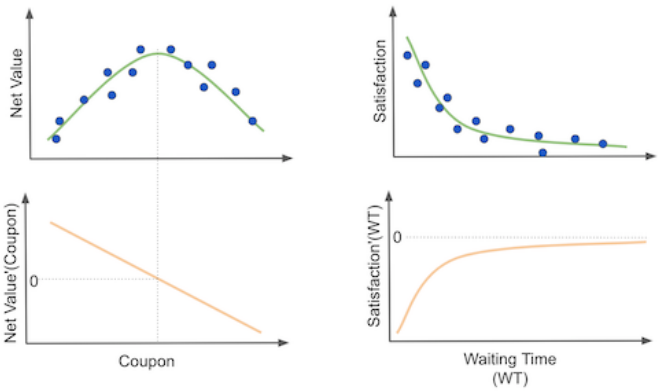
the prediction will have limited Y variance, hindering our capacity to estimate how T changes Y, that is, the elasticity $\frac{\delta Y}{\delta T}$. This is the most important paragraph of this chapter. All of it is summarized here, so reread it if you didn't understood at first.

The key to fixing this mistake lies in adjusting our objective to what we really want. Instead of estimating Y out of X, which is what prediction does, we need to estimate $\frac{\delta Y}{\delta T}$ out of X. Easier said than done. As you might have guessed already, this precisely what causal inference is all about. And, as it is natural of causal problems, we can't observe our quantity of interest $\frac{\delta Y}{\delta T}$. You simply cannot observe how `net_income` would change if we changed the coupon value because we only observe one instance of a coupon per customer. We can never know what would have happened if some different coupon value had taken place (unless we use simulated data, of course. But that's only useful for teaching purposes).

This characteristic of causal problems leads to further questions: how can I know if my model is any good if I can't see what it is supposedly estimating? How can I validate a model like that? Besides, what do we do when data is not random? How can we estimate the best policy under biased data? Those are fair questions and we shall answer them in time. Meanwhile, keep in mind that, as we change our focus from estimating Y to estimating $\frac{\delta Y}{\delta T}$, lots of things will have to change accordingly. The traditional ML toolkit will need some adaptation.

When Might Prediction Helps

All this mess with machine learning hindering our ability to infer causal effects comes from mistaking the real goal. It comes from estimating Y instead of $\frac{\delta Y}{\delta T}$. But sometimes, you might actually get away when using a prediction model to achieve a causal inference goal. But for this to happen, Y and $\frac{\delta Y}{\delta T}$ must be somehow correlated. For example, consider the cases on the image below.

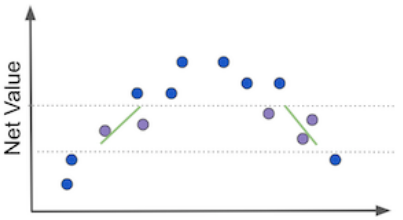


The first one is the problem we've seen before of understanding how coupons impact profitability. First, as we increase coupon value, profitability increases. If they go up together, we can see that they are positively correlated. Moreover, in the derivative plot,

at the beginning, as we increase the coupon value, the elasticity $\frac{\delta Y}{\delta T}$ decreases. If we put both of them together, in the first region of the plot, as net value increases, elasticity will decrease. This means that they are negatively correlated.

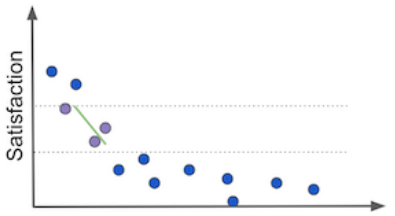
But that's only at the beginning. If we keep increasing the coupon value, profitability will decrease and elasticity profitability will also decrease. This means that $\frac{\delta Y}{\delta T}$ and Y are now positively correlated. So, while outcome and elasticity are negatively correlated at the beginning, this correlation reverses as we increase coupon values. This means that a prediction model won't help us here because there isn't a direct relationship between outcome and elasticity of the outcome.

Another way of thinking about this is that prediction models make slices of the data on the Y axis. If we do those sorts of slices, we will mix units that are both in the positive elasticity region and in the negative elasticity region.



Now, consider a second case where we want to see how waiting in line to get answered by customers services impact customer satisfaction. In this case, we can see that customer satisfaction drops pretty fast in the first few minutes of waiting time. Customers get really pissed off when they go from not having to wait much to having to wait just a little bit. However, as waiting time increases, customer satisfaction is already so low it doesn't drop much afterwards. It sorts of saturates at a lower level.

This case is interesting because the relationship between $\frac{\delta Y}{\delta T}$ and Y doesn't change much. It is always negatively correlated. As waiting time increases, satisfaction decreases and satisfaction elasticity increases. Or, as T increases, $\frac{\delta Y}{\delta T}$ also increases and Y decreases. In this case, a prediction model can be useful. The reason being that, now, if we slice on the Y axis, we will group units that have similar elasticities.



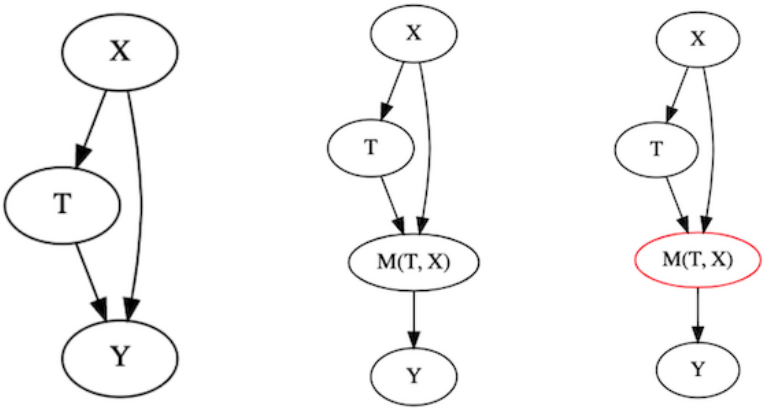
More generally, whenever we have these sorts of functional forms where elasticity changes ruthly in the same direction as the outcome, you might get away with prediction models. Now, be careful here. This doesn't mean they are your best bet nor that you should use

them for causal problems. I'm simply stating this here so that when you come across a prediction model that works for a causal goal, you can understand what is happening.

A Graphical Explanation

The fact that prediction models can be harmful for personalisation is so counterintuitive and strange that I feel I must tackle the problem from multiple perspectives. The following graphical explanation is thus a way of understanding what is happening from a different angle.

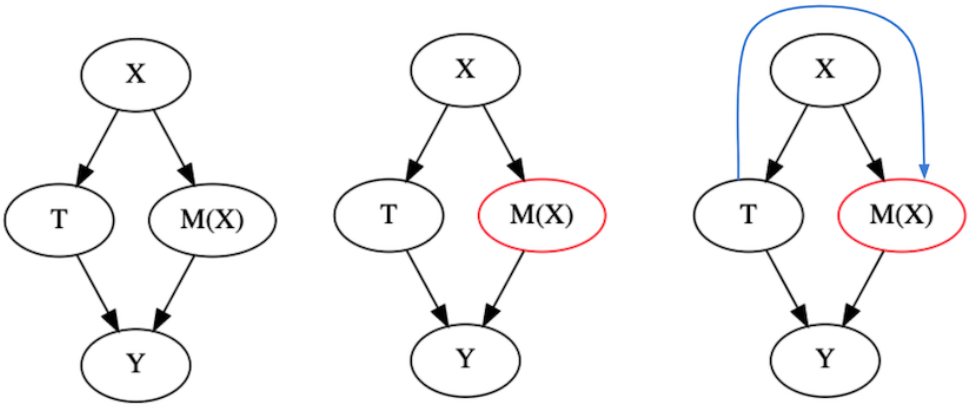
Consider the causal leftmost graph on the image below. It's a typical confounding graph, where we have features X causing both the outcome Y and the treatment T. What happens if you build a predictive model and use it to segment the units by its predictions?



You end up getting something like the graphs on the left (technically, they are not causal anymore, because the model obviously doesn't cause the outcome. It just learns the mapping function from features to outcomes, but that doesn't invalidate the points I'm about to make). Segmenting on this model is equivalent to conditioning on the model node. You are holding $M(T, X)$ fixed on each segment. If you do that and try to learn the relationship between T and Y while conditioning on $M(T, X)$, you will end up with the situation we've discussed in part I, about bad controls. And in case you don't remember, if you control for anything in the path between the treatment and the outcome, you block a path that the treatment is using to affect the outcome. As a consequence, the perceived effect ends up looking smaller than the actual effect. So there you have it. We used another explanation to get to the exact same conclusion as before: segmenting units by a predictive model hinders our ability to identify the causal effect.

Now, if you are smart, you will probably object with the following idea. What if we simply remove T from the model? That's a really good argument. In fact, it's so good it took me years to properly articulate why just trying to fool your model like that is probably not a smart thing to do. In fact, if you look back on the example we've explored in this chapter, notice that I didn't even give the predictive model the coupon treatment. That didn't

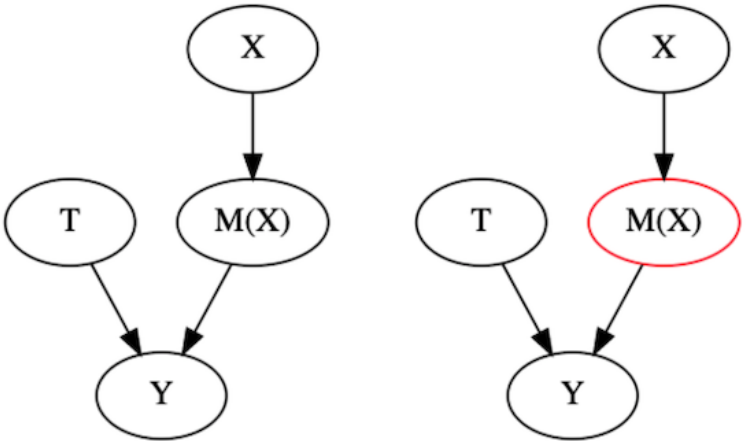
solve the problem that the predictive model partitioning was squashing our measured elasticity to a horizontal line. The answer to why lies in the following image.



Excluding the treatment T from your model $M(X)$ is not enough to make the model ignore the relationship between T and Y. Because X causes T, the model can indirectly learn T through X. Why will it do that? Because if T causes Y and M wants to predict Y as much as possible, it will forcibly explore the pattern in X that links T to Y. In other words, the information of how the treatment impacts the outcomes flows backwards to our model through the features. To give an example, consider that you only give discounts (T) to customers that live in a certain area of town (X). For the sake of the argument, suppose that the area doesn't cause sales (Y) in any way. Even if that is the case, the city area still has predictive power because it causes T, which in turns causes Y. Hence, the model will explore that fact in order to predict Y.

To be fair to this argument, you actually can fool your model a little bit by not giving it T. To see that, I recommend you rerun the code on our examples but this time include `coupons` in the list of features to the predictive model. You will see that the Model Policy will perform even worse. This means that NOT giving T to our model is doing some good, although not much.

Finally, there is one way out of this, which is when T is random. In that very particular case, a predictive model can actually help you with causal inference. But first, let's understand why it doesn't hurt. When T is random, this excludes any confounding that comes from X. In this case, X have no information about how T comes to be.



When that happens, even if you control for $M(X)$, you are still allowing all the effect from T to flow onto Y. In other words, even if you look inside segments where $M(X)$ is fixed, Y will still vary due to T. $M(X)$ can no longer control that variability because it has no way of learning T anymore. Not only that, this sort of condition will actually help you figure out the Average Treatment Effect. We also saw that on Part I. If we control for variables that are good predictors of Y but that don't cause T, we will reduce the variance of our causal estimates. The intuitive reason is that a lot of the variation on Y that is due to X is removed by conditioning on the model. Since X is then explained away, the reason why Y is still changing must be mostly due to T (or other unmeasured things, but you get the point that it helps).

Again, this isn't an argument in favor of using predictive models when the goal is causal inference. In fact, segments defined by a predictive model will help you identify the ATE, but that doesn't mean each partition will have a different ATE (or elasticity). Remember that not only do you want to estimate the elasticity, but you also want to find segments where it is higher or lower than average so that you can personalise the treatment.

Key Ideas

We saw how there is a whole range of problems that are not solvable with traditional, predictive Machine Learning. Those problems are optimization ones, with a causal component, usually framed as "how should I set this thing I can control, T (price, number of phone calls, discount, …), in order to optimize this metric, Y (profits, customer satisfaction, conversion, costs…)". When that is the case, predictive models can do more harm than good.

To prove this point, we walked through an example where we needed to find the best coupon value. The first thing that we tried was to look at past data and see which coupon value led to the highest revenue. Then, we took that supposedly optimal value and gave it to every customer. This approach answers what is the best coupon value (treatment, in

causal terms), on average. That is, if we were to pick only one fixed coupon value, what should it be. But we don't have such a restriction. If it turns out to be the case that some customers respond better to lower valued coupons and some respond better to higher ones, we could personalise the coupon for each customer. To do so, we tried building a predictive ML model, a model that predicted the value each customer would bring. Then, we tried to personalise the coupon value based on the model's predictions. We grouped the customers according to the predictions and for each group, we found which coupon value was the best one. Then, we tried using that learned value as a coupon strategy. However, we found that it performed worse than the simple strategy of giving the same coupon value for everyone. In other words, our ML model failed to separate customers that responded better or worse to coupon value.

Notice that the problem we are trying to solve here is one of **personalisation**. In our case, it was trying to figure out which coupon value is best for each customer. More generally, it is framed as "which is the best decision T I can take for each customer i so that I can maximise Y_i across all customers". This is not an easy problem to solve. But we can solve it with the help of causal inference.

There is one important thing I want you to take out of this chapter. First, notice that causal inference as we saw it in part one helps you design policies like the simple one we've built. With them, we are trying to estimate the average causal effect for the treatment and find the treatment that works best on average. Now, we are getting more ambitious. We don't want the best treatment on average, we want the best treatment for each individual customer. That's what personalisation is all about and it relies heavily on causal inference methods.

References

The things I've written here are mostly stuff from my head. I've learned them through experience. This means there isn't a direct reference I can point you to. It also means that the things I wrote here have **not** passed the academic scrutiny that good science often goes through. Instead, notice how I'm talking about things that work in practice, but I don't spend too much time explaining why that is the case. It's a sort of science from the streets, if you will. However, I am putting this up for public scrutiny, so, by all means, if you find something preposterous, open an issue and I'll address it to the best of my efforts.

For this chapter, there isn't much I can point to. There is only this one great article that formulates the problem of personalisation that we are trying to solve with causal inference: *Recursive Partitioning for Heterogeneous Causal Effects, Susan Athey and Guido Imbens, 2016*. Aslo, the first image is from a YouTube video from Siraj Raval.

APPENDIX

D

Why Prediction Metrics are Dangerous for Causal Models

```
import pandas as pd
import numpy as np
from matplotlib import pyplot as plt
import statsmodels.formula.api as smf
from toolz import *
```

A common misconception I often hear is that, if we have random data, to evaluate a causal model we could just evaluate the predictive performance of such model on the random dataset, using a metric like R^2. Unfortunately things are not that simple and I'll try to explain why.

Generally speaking, we can say any outcome is a function of the treatment and covariates

$$Y = F(x, t)$$

Let's say we can decompose this function into two additive pieces. One piece that doesn't depend on the treatment and another that depends only on the treatment and possible interactions.

$$Y = g(x) + f(t, x)$$

This additive structure places some restriction on the functional form but not much, so we can argue it is a pretty general way of describing a Data Generating Process (DGP).

The point is that if the treatment effect is weaker than the covariates effect, then, **even if we have random data**, we can have a model which has higher predictive power but is bad for causal inference. All that model has to do is approximate $g(x)$ while disregarding $f(t, x)$.

In other words, **predictive performance on a random dataset does not translate our preference for how good a model is for causal inference**.

To show that, let's use some simulated data.

Simulating Data

In the following DGP, we have covariates X that have a high predictive power but don't interact with the treatment. In other words, they don't dictate the treatment effect heterogeneity. We also have features W that only impact the outcome through the treatment (are not confounders). Since the treatment has low predictive power, W also doesn't have much predictive power.

$$Y_i = g(X_i) + f(T_i, W_i) + e_i$$

```
n = 100000
n_features = 20
n_heter = 10

np.random.seed(12321)

X = np.random.normal(1, 10, (n, n_features))
nuisance = np.random.uniform(-1,1, (n_features, 1))

W = np.random.normal(1, 10, (n, n_heter))
heter_y = np.random.uniform(-1,1, (n_heter, 1))

T = np.random.normal(10, 2, (n, 1)) # T is random!
Y = np.random.normal(T + T*W.dot(heter_y) + 20*X.dot(nuisance), 0.1)

df = pd.concat([
    pd.DataFrame(X, columns=[f"f{f}" for f in range(n_features)]),
    pd.DataFrame(W, columns=[f"w{f}" for f in range(n_heter)])
], axis=1).assign(T=T, Y=Y)
```

Now, let's break that dataset into a training and a test set

```
from sklearn.model_selection import train_test_split

train, test = train_test_split(df, test_size=0.5)
train.shape, test.shape
```

```
((50000, 32), (50000, 32))
```

and train two models for treatment effect heterogeneity, $M1$ and $M2$. $M1$ will include the highly predictive features that don't affect the treatment heterogeneity and $M2$ will include the low predictive features that do affect treatment heterogeneity.

```
m1 = smf.ols("Y~T*(" + "+".join([f"f{f}" for f in range(n_features)])+")",
  ↪ data=df).fit()
m2 = smf.ols("Y~T*(" + "+".join([f"w{f}" for f in range(n_heter)])+")",
  ↪ data=df).fit()
```

If we look at the predictive power of both models using the R^2, indeed, $M1$ is much better than $M2$.

```
from sklearn.metrics import r2_score

print("M1:", r2_score(test["Y"], m1.predict(test)))
print("M2:", r2_score(test["Y"], m2.predict(test)))
```

```
M1: 0.9160516511287358
M2: 0.08378351037639298
```

Now, let's calculate the cumulative elasticity curve for both models. For that, we will need Conditional Average Treatment Effect predictions. Since all the models here are linear, we can use the following formula to get CATE predictions

$$\hat{CATE}_i = M(X, W, t) - M(X, W, t-1)$$

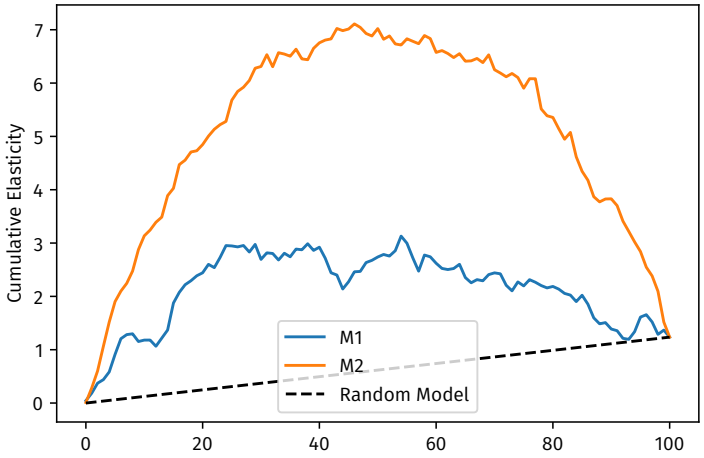
```
@curry
def elast(data, y, t):
        return (np.sum((data[t] - data[t].mean())*(data[y] - data[y].mean())) /
                np.sum((data[t] - data[t].mean())**2))


def cumulative_gain(dataset, prediction, y, t, min_periods=30, steps=100):
    size = dataset.shape[0]
    ordered_df = dataset.sort_values(prediction, ascending=False).reset_index(
      ↪ drop=True)
    n_rows = list(range(min_periods, size, size // steps)) + [size]
    return np.array([elast(ordered_df.head(rows), y, t) * (rows/size) for rows
      ↪ in n_rows])
```

```
test_pred = test.assign(
    cate_1 = m1.predict(test) - m1.predict(test.assign(T=test["T"]-1)),
    cate_2 = m2.predict(test) - m2.predict(test.assign(T=test["T"]-1))
)
```

Once we have those CATE predictions, we can evaluate them using the cumulative elasticity curve

```
cumelast_1 = cumulative_gain(test_pred, "cate_1", "Y", "T", steps=100)
cumelast_2 = cumulative_gain(test_pred, "cate_2", "Y", "T", steps=100)
```

```
plt.plot(range(len(cumelast_1)), cumelast_1, label="M1")
plt.plot(range(len(cumelast_2)), cumelast_2, label="M2")
plt.plot([0, 100], [0, elast(test_pred, "Y", "T")], linestyle="--",
  ↪ label="Random Model", color="black")
plt.legend()
plt.ylabel("Cumulative Elasticity");
```



As we can see, now $M1$ is much worse than $M2$, even though it has a higher R^2 on the test set. This shows that predictive power doesn't translate directly to a good causal model, even if we use random data.

Predictive Metric For Causal Inference

This doesn't mean we can't come up with a way to correctly evaluate a causal model using predictive metrics. In order to do so, let's go back to our additive assumption about de DGP.

$$Y = g(x) + f(t, x)$$

To use a predictive metric, we need to somehow transform the outcome in order to remove the $g(x)$ component from it. That way, all the remaining predictive power will necessarily be used to learn the causal relationship.

$$\tilde{Y} = Y - g(x) = f(t, x)$$

One way of doing that is by orthogonalizing Y. We can use any ML model to estimate g and get out of fold residuals

$$\tilde{Y} = Y - \hat{g}(x)$$

```
denoise_m = smf.ols("Y~"+
                    "+".join([f"w{f}" for f in range(n_heter)])+
                    "+"+
                    "+".join([f"f{f}" for f in range(n_features)]),
                      ↪ data=train).fit()

test_res = test.assign(Y_res = test["Y"] - denoise_m.predict(test) + test["Y"]
  ↪ .mean())
```

Once we do that, we can evaluate the power of each model in predicting the residualized outcome $\tilde{Y}$. The predictive performance on this new outcome will directly translate to a better causal model.

```
print("M1:", r2_score(test_res["Y_res"], m1.predict(test_res)))
print("M2:", r2_score(test_res["Y_res"], m2.predict(test_res)))
```

```
M1: -281.7950185092476
M2: -24.04912481654577
```

The downside of this approach is that it depends on how well you can estimate $g(x)$.

APPENDIX

E

Conformal Inference for Synthetic Controls

```
import numpy as np
import pandas as pd
from toolz import curry
import seaborn as sns
from matplotlib import pyplot as plt
import statsmodels.formula.api as smf
import cvxpy as cp

import toolz as f

from sklearn.linear_model import Lasso
import warnings
warnings.filterwarnings('ignore')

from matplotlib import style
style.use("ggplot")
```

Synthetic Control Refresher

Synthetic Control (SC) is a particularly useful causal inference technique for when you have a single treatment unit and very few control units, but you have repeated observation of each unit through time (although there are plenty of SC extensions in the Big Data world). The canonical use case is when you want to know the impact of the treatment in one geography (like a state) and you use the other untreated states as controls. In our Synthetic Control chapter, we've motivated the technique by trying to estimate the effect of Proposition 99 (a bill passed in 1988 that increased cigarette tax in California) in cigarette sales.

In order to do that, we have to estimate what would have happened to California, had it not passed Proposition 99. This boils down to estimating the counterfactual $Y_t(0)$ so that we can compare it to the observed outcome in the post intervention periods:

$$ATT = Y_t(1) - Y_t(0) = Y_t - Y_t(0) \text{ for } t \geq 1988$$

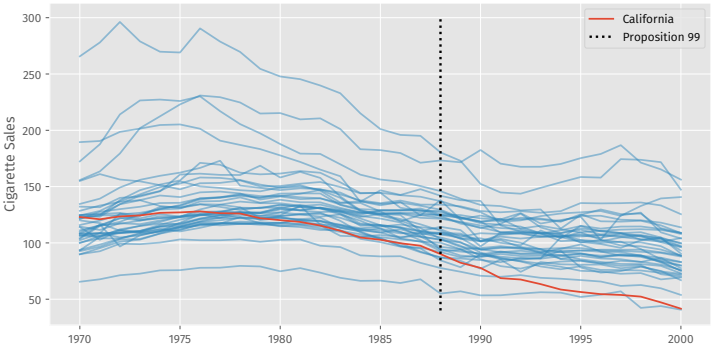
There are many methods to do that, among which, we have Synthetic Controls. Synthetic Controls tries to model $Y(0)$ for the treated unit by combining multiple control units in such a way that they mimic the pre-treatment behavior of the treated unit. In our case, this means finding a combination of states that, together, approximate the cigarette sales trend in California prior to Proposition 99. This is done because we rarely have a control unit that follows the same pattern as the treatment unit. We can see that by plotting the cigarette sales trend for multiple states. Notice none of them have a trend that closely resembles that of California.

```
data = pd.read_csv("data/smoking.csv")

data = data.pivot("year", "state", "cigsale")
data = data.rename(columns={c: f"state_{c}" for c in data.columns}).rename(
  ↪ columns={"state_3": "california"})
data.shape
```

```
(31, 39)
```

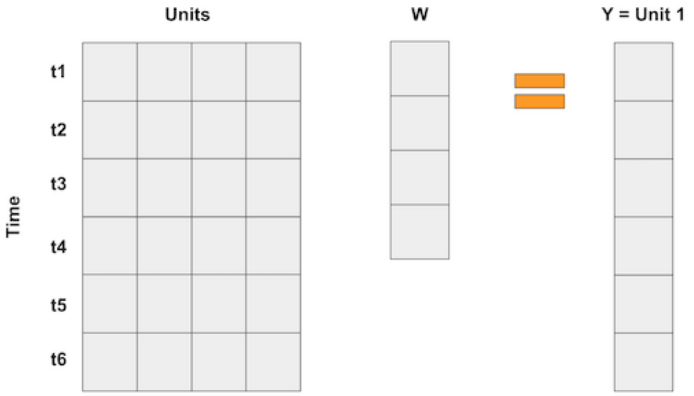
```
plt.figure(figsize=(10,5))
plt.plot(data.drop(columns=["california"]), color="C1", alpha=0.5)
plt.plot(data["california"], color="C0", label="California")
plt.vlines(x=1988, ymin=40, ymax=300, linestyle=":", lw=2,
  ↪ label="Proposition 99", color="black")
plt.legend()
plt.ylabel("Cigarette Sales")
```

```
Text(0, 0.5, 'Cigarette Sales')
```



That is why we combine multiple treated units. The goal is, if we don't have a good enough control, we can craft a synthetic one that resembles the treated unit the way we want.

In order to find the combination of states that better approximate the pretreatment trend of California, the Synthetic Control method runs a horizontal regression, where the rows are the time periods and the columns are the states. It tries to find the weights that, when multiplied by the control states, better approximate the treated state



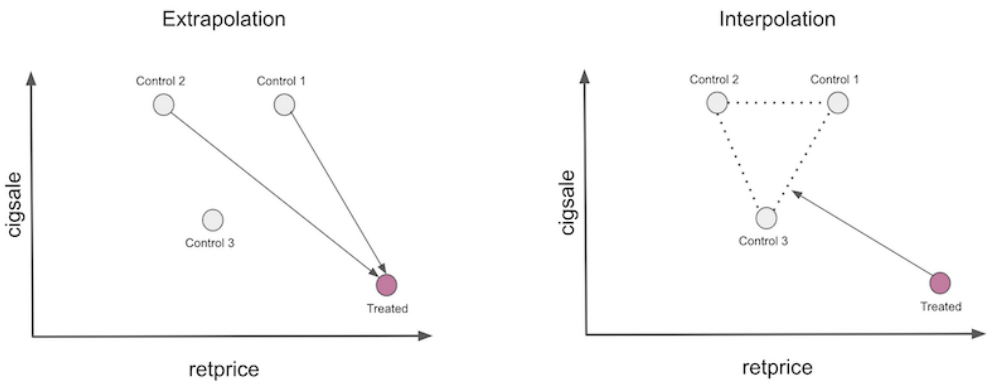
Since we have more states (39, some were discarded from the analysis) than time periods, an unconstrained regression would simply overfit, which is why Synthetic Control imposes two restrictions:

1. Weights must sum to 1;
2. Weights must be non-negative;

Or, in mathematical terms, let $\boldsymbol{y}$ be the vector of outcomes for the treated state in the pre-treated periods, $\boldsymbol{X}$ the J by $T0$ matrix, where each column is a state j and each row is a period t prior to the intervention period, $T1 = T0 + 1$

$$\underset{w}{\text{argmin}} \, ||\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w}|| \text{s.t} \;\; \sum w_j = 1 \text{ and } \; w_j > 0 \; \forall \; j$$

Combined, these constraints means we are defining the synthetic control as a convex combination of the control units. It also means we are not doing any dangerous extrapolation and that our synthetic control will use only a small subset of control units.



Here is what this looks like in code, as an Sklearn estimator:

```
from sklearn.base import BaseEstimator, RegressorMixin
from sklearn.utils.validation import check_X_y, check_array, check_is_fitted
import cvxpy as cp

class SyntheticControl(BaseEstimator, RegressorMixin):

    def __init__(self,):
        pass

    def fit(self, X, y):

        X, y = check_X_y(X, y)

        w = cp.Variable(X.shape[1])
        objective = cp.Minimize(cp.sum_squares(X@w - y))

        constraints = [cp.sum(w) == 1, w >= 0]

        problem = cp.Problem(objective, constraints)
        problem.solve(verbose=False)

        self.X_ = X
        self.y_ = y
        self.w_ = w.value

        self.is_fitted_ = True
        return self


    def predict(self, X):

        check_is_fitted(self)
```

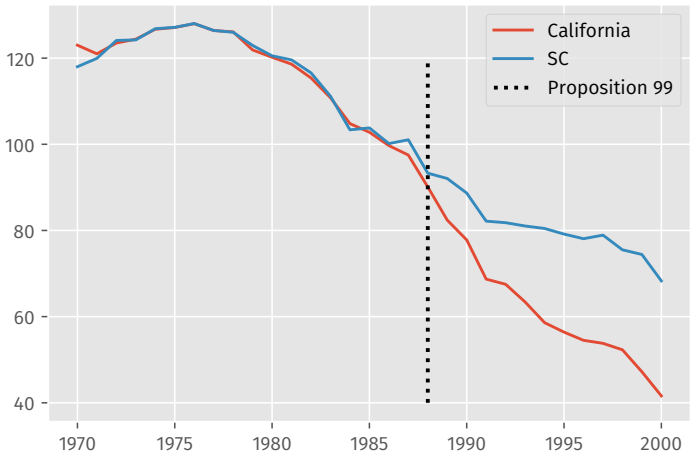
```
        X = check_array(X)

        return X @ self.w_
```

Let's apply this method to our data, fitting it in the pre-intervention period (prior to 1988).

```
model = SyntheticControl()

train = data[data.index < 1988]

model.fit(train.drop(columns=["california"]), train["california"]);
```

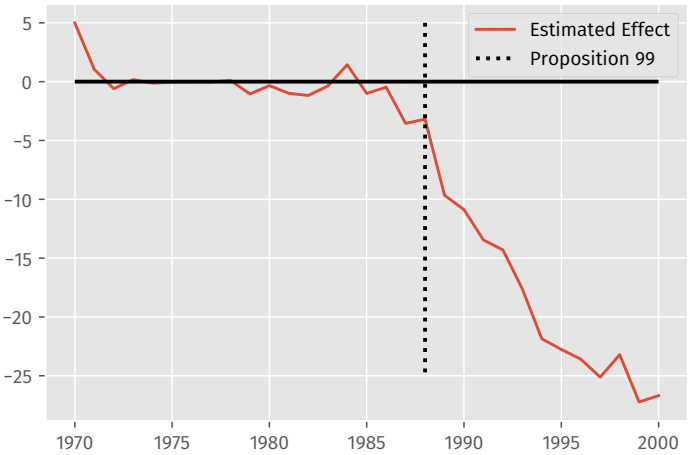
We can now plot, side by side the trend for California and for the synthetic control we've just created. The difference between these two lines is the estimated effect of Proposition 99 in California.

```
plt.plot(data["california"], label="California")
plt.plot(data["california"].index, model.predict(data.drop(
  ↪ columns=["california"])), label="SC")
plt.vlines(x=1988, ymin=40, ymax=120, linestyle=":", lw=2,
  ↪ label="Proposition 99", color="black")

plt.legend();
```



From the look of this plot, it looks like Proposition 99 had a pretty big effect on the reduction of cigarette sales.

```
pred_data = data.assign(**{"residuals": data["california"] - model.predict(data
  ↪ .drop(columns=["california"]))})

plt.plot(pred_data["california"].index, pred_data["residuals"],
  ↪ label="Estimated Effect")
plt.hlines(y=0, xmin=1970, xmax=2000, lw=2, color="Black")
plt.vlines(x=1988, ymin=5, ymax=-25, linestyle=":", lw=2,
  ↪ label="Proposition 99", color="Black")

plt.legend();
```

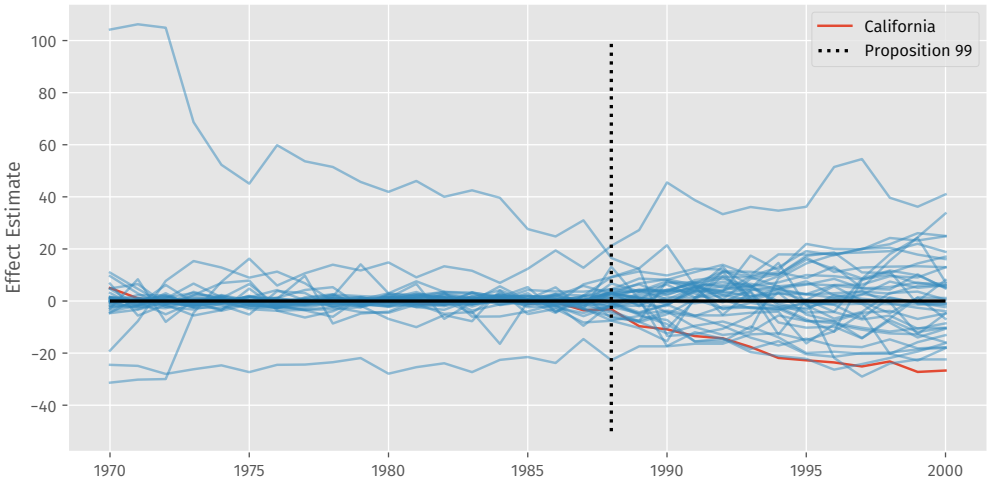


Inference for Grown Ups

In the Synthetic Control chapter, we showed an inference procedure where we've permuted units, pretending control units where treated. This is also referred to as a placebo test, where we check the effect of units that haven't gone through the treatment. If the estimated effect in the treated unit is bigger than most of the placebo effects, we say that this effect estimate is significant.

```
plt.figure(figsize=(10,5))
for state in data.columns:

    model_ier = SyntheticControl()
    train_iter = data[data.index < 1988]
    model_ier.fit(train_iter.drop(columns=[state]), train_iter[state])

    effect = data[state] - model_ier.predict(data.drop(columns=[state]))

    is_california = state == "california"

    plt.plot(effect,
```

```
            color="C0" if is_california else "C1",
            alpha=1 if is_california else 0.5,
            label="California" if is_california else None)

plt.hlines(y=0, xmin=1970, xmax=2000, lw=2, color="Black")
plt.vlines(x=1988, ymin=-50, ymax=100, linestyle=":", lw=2,
  ↪ label="Proposition 99", color="Black")
plt.ylabel("Effect Estimate")
plt.legend();
```



In our example, we can see that the post-treatment difference for California is quite extreme, when compared to the other states. However, there are also some states with terrible pre-treatment fit, which then translates to a huge error in the post-intervention period. The guideline here is to remove units with high pretreatment error, but how high is a bit more complicated. Not only that, this procedure assumes a random assignment of the intervention, which is hard to believe for this kind of policy intervention (see Abadie, 2021)

One alternative method for inference is to recast the problem of effect estimation as counterfactual prediction. If you think about it, all we are trying to do is predict the counterfactual $Y_{i,t}(0)$ where i is the treated unit and $t \geq T1$, that is, in the post intervention period. If we do that, we can leverage the literature on **Conformal Prediction for inference**. Interestingly enough, this method is quite general and applies to other models of $Y_{i,t}(0)$ but let's focus just on Synthetic Controls here.

To understand this procedure, let's first look at how we would do Hypothesis Tests and get P-Values.

Hypothesis Test and P-Values

Let's say we are interested in testing the Hypothesis about the trajectory of effects in the post treatment period $\theta = (\theta_{T0+1}, ..., \theta_T)$

$$H_0 : \theta = \theta^0$$

For instance, if we wish to test for no effect whatsoever, we can set $\theta^0 = (0, ..., 0)$. Notice that this hypothesis fully determines the counterfactual outcome in the absence of treatment:

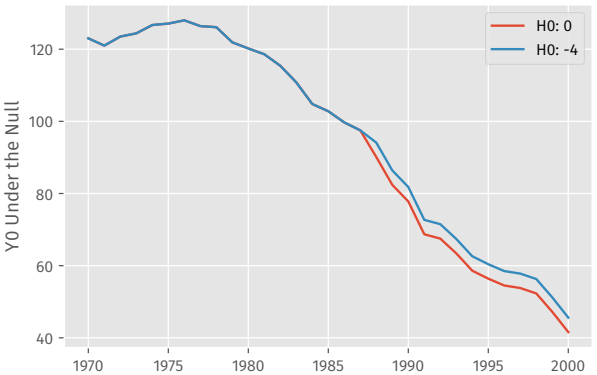
$$Y_t(0) = Y_t(1) - \theta_t = Y_t - \theta_t$$

The key idea is to then generate data following the null hypothesis we want to test and check the residuals of a model for $Y(0)$ in this generated data. If the residuals are too extreme, we say that the data is unlikely to have come from the null hypothesis we've postulated. If this whole procedure sounds obscure at first, don't worry, it will become clear as we walk through a step by step implementation of it.

The first step is to generate data under the null hypothesis. This is achieved by simply subtracting the postulated null from the outcome of the treated unit, just like in the equation above. Here is the code to do that.

```
def with_effect(df, state, null_hypothesis, start_at, window):
    window_mask = (df.index >= start_at) & (df.index < (start_at +window))

    y = np.where(window_mask, df[state] - null_hypothesis, df[state])

    return df.assign(**{state: y})
```

```
plt.plot(with_effect(data, "california", 0, 1988, 2000-1988+1)["california"],
  ↪ label="H0: 0")
plt.plot(with_effect(data, "california", -4, 1988, 2000-1988+1)["california"],
  ↪ label="H0: -4")

plt.ylabel("Y0 Under the Null")
plt.legend();
```



If we postulate the null of no effect, the data under that null means that $Y(0) = Y(1) = Y$, which is just the trajectory of observed outcome we see for the treated state of California. Now, if we postulate that the null is -4, that is, Proposition 99 decreases cigarette sales by 4 packs, then $Y(0) = Y(1) - (-4)$, which shifts the trajectory of the post treatment outcomes by $+4$. This is very intuitive. If we think the bill decreases cigarette sales, then, in the absence of it, we should see higher levels of cigarette sales than the one we have in our observed data.

The next part of the inference procedure is to fit a model for the counterfactual $Y(0)$ (which we get with the function we just created) in the entire data, pre **and** post-treatment period. This is an important distinction between how we usually fit synthetic controls. The idea here is that the model must be estimated with the entire data, under the postulated null hypothesis, to avoid huge post intervention residuals. With this model, we then compute the residuals $\hat{u}_t = Y_t - \hat{Y}_t(0)$ for all time periods t.

The function to do that first uses the `with_effect` function we created earlier to generate data under then null. Then, it fits the model in this data under the null. Next, we estimate $Y(0)$ by making predictions with the recently fit model. Finally, we compute the residuals $\hat{u}_t$ and stores everything in a dataframe.

```
@curry
def residuals(df, state, null, intervention_start, window, model):

    null_data = with_effect(df, state, null, intervention_start, window)

    model.fit(null_data.drop(columns=[state]), null_data[state])

    y0_est = pd.Series(model.predict(null_data.drop(columns=[state])),
      ↪ index=null_data.index)

    residuals = null_data[state] - y0_est

    test_mask = (null_data.index >= intervention_start) & (null_data.index <
      ↪ (intervention_start + window))

    return pd.DataFrame({
        "y0": null_data[state],
        "y0_est": y0_est,
        "residuals": residuals,
        "post_intervention": test_mask
    })[lambda d: d.index < (intervention_start + window)]
      ↪ # just discard  points after the defined effect window
```

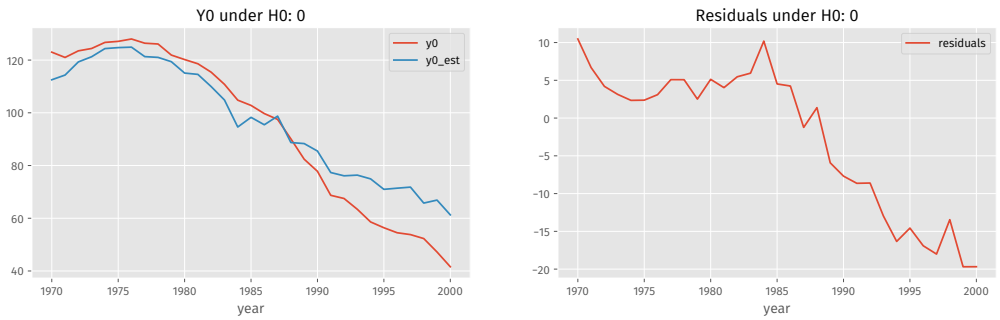
With our data, to get the residuals for $H_0 : 0$, meaning Proposition 99 had no effect, we can simply pass 0 as the null for our function.

```
model = SyntheticControl()

residuals_df = residuals(data,
                         "california",
                         null=0.0,
                         intervention_start=1988,
                         window=2000-1988+1,
                         model=model)

residuals_df.head()
```

year	y0	y0_est	residuals	post_intervention
1970	123.000000	112.529475	10.470525	False
1971	121.000000	114.315723	6.684277	False
1972	123.500000	119.302289	4.197711	False
1973	124.400002	121.265554	3.134447	False
1974	126.699997	124.356696	2.343301	False

The result is a dataframe containing the estimated residuals for each time period, something we will use going forward. Remember that the idea here is to see if that residual, in the post intervention period, is too high. If it is, the data is unlikely to have come from this null, where the effect is zero. To get a visual idea of what we are talking about, we can inspect the error of our model in the post intervention period.

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 4))
residuals_df[["y0", "y0_est"]].plot(ax=ax1)
ax1.set_title("Y0 under H0: 0");
residuals_df[["residuals"]].plot(ax=ax2);
ax2.set_title("Residuals under H0: 0");
```



We can already see that the model fitted under $H_0 : 0$ yields quite large and negative residuals, which is some evidence we might want to reject this null of no effect.

Test Statistic

This visual evidence is interesting for our own understanding, but we need to be a bit more precise here. This is done by the definition of a **Test Statistic S**, which summarizes how big are the residuals and hence, how unikly is the data we saw, under the null.

$$S(\hat{u})_q = \left(\sum_{t=T0+1}^{T} |u_t|^q \right)^{1/q}$$

Here, we focus on $q = 1$, which gives us $S(\hat{u}) = \sum_{t=T0+1}^{T} |u_t|$.

Notice that this statistic is computed using only the post-intervention period, with $t \geq T0 + 1$. So, although we use all the data to fit our model for the counterfactual $Y(0)$, we check the residuals only for the outcome which concerns the formulated null hypothesis, that is, the post-intervention period.

```
def test_statistic(u_hat, q=1, axis=0):
    return (np.abs(u_hat) ** q).mean(axis=axis) ** (1/q)
```

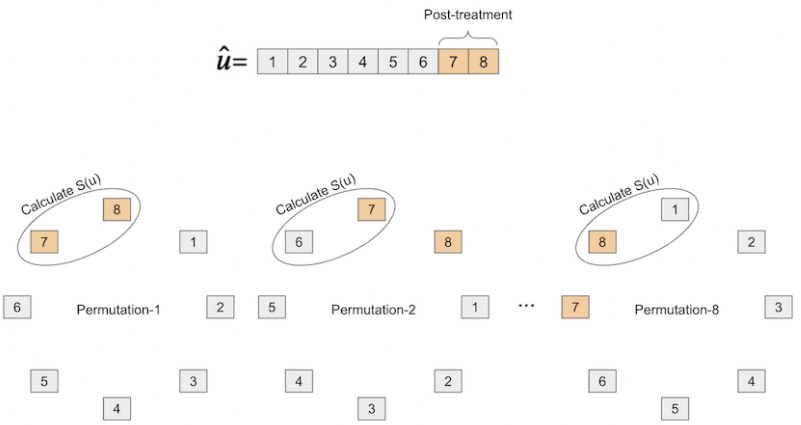
```
print("H0:0 ", test_statistic(residuals_df
  ↪ .query("post_intervention")["residuals"]))
```

```
H0:0  12.602929955114083
```

High values of this test statistic indicate poor post intervention fit and, hence rejection of the null. However, we could have pretty big test statistics in the post-intervention period if our model is poorly fitted, even if H_0 is true. This means we can't define high in absolute terms. Rather, we have to think about how high are the post intervention residuals - and test statistics - in comparison to the pre-intervention residuals.

P-Value

To compute the P-value, we block-permute the residuals, calculating the test statistic in each permutation. This procedure is better understood by the following picture



Once we do that, we will end up with T test statistics, one for each of the block permutations.

Let Π be the set of all block permutations, by the definition of P-value

$$\text{P-value} = \frac{1}{|\Pi|}\sum_{\pi\in\Pi} 1\{S(\hat{u}_{\pi_0}) \leq S(\hat{u}_{\pi})\}$$

and $\hat{u}_{\pi_0}$ is the original (unpermuted) vector or residuals. In plain terms, we are simply finding the proportion of times that the unpermuted test statistic is higher (more extreme) than the test statistics obtained by all possible block permutations.

To implement this, we will make use of the `np.roll` function, which takes an array and circles it, mujustch like we've represented in the image above.

```
def p_value(resid_df, q=1):

    u = resid_df["residuals"].values
    post_intervention = resid_df["post_intervention"].values

    block_permutations = np.stack([np.roll(u, permutation,
      ↪ axis=0)[post_intervention]
                                   for permutation in range(len(u))])

    statistics = test_statistic(block_permutations, q=1, axis=1)

    p_val = np.mean(statistics >= statistics[0])

    return p_val
```

We can now compute the P-value for $H_0 : 0$. As we can see, it is a low P-value, but not extremely low. At $\alpha = 0.1$, we would not reject this null of no effect.

```
p_value(residuals_df)
```

```
0.16129032258064516
```

Remember, this is the P-value for the null hypothesis which states that the effect in all time periods is zero: $\theta = (\theta_{T0+1} = 0, ..., \theta_T = 0)$. From our effect plot from the Synthetic Control, we get the feeling that the effect of Proposition 99 is not a fixed number. We can see that it starts small, around -5, but gradually increases to -25. For this reason, it might be interesting to plot the confidence interval for effect each post treatment period individually, rather than just testing a null hypothesis about an entire affect trajectory.

Confidence Intervals

To understand how we can place a confidence interval around the effect of each post-treatment period, let's first try to understand how we would define the confidence interval for a single time period. If we have a single period, then H_0 is defined in terms of a scalar

value, rather than a trajectory vector θ. This means we can generate a fine line of $H_0 s$ and compute the P-value associated with each null. For example, Let's say we think the effect of Proposition 99 in the year 1988 (the year it passed) is somewhere between -20 and 20. We can then build a table containing a bunch of H_0, from -20 to 20, and each associated P-value:

```
P-value(H_0: -20) = 0.01
P-value(H_0: -19) = 0.01
P-value(H_0: -18) = 0.02
...
P-value(H_0: 18) = 0.03
P-value(H_0: 19) = 0.03
P-value(H_0: 20) = 0.02
```

With the functions we've defined, this can be achieved by first appending the period of interest (1988 in this example) at the end of the pre-intervention period, creating what is called an augmented dataset. Then, we iterate over the fine line of nulls, computing the p-value of a post-intervention window of size 1, which starts at the period of interest

```
def p_val_grid(df, state, nulls, intervention_start, period, model):

    df_aug = df[df.index < intervention_start].append(df.loc[period])

    p_vals =  {null: p_value(residuals(df_aug,
                                       state,
                                       null=null,
                                       intervention_start=period,
                                       window=1,
                                       model=model)) for null in nulls}

    return pd.DataFrame(p_vals, index=[period]).T
```

```
model = SyntheticControl()

nulls = np.linspace(-20, 20, 100)

p_values_df = p_val_grid(
    data,
    "california",
    nulls=nulls,
    intervention_start=1988,
    period=1988,
    model=model
)

p_values_df
```

	1988
-20.000000	0.052632
-19.595960	0.052632
-19.191919	0.052632
-18.787879	0.052632
-18.383838	0.052632
...	...
18.383838	0.052632
18.787879	0.052632
19.191919	0.052632
19.595960	0.052632
20.000000	0.052632

100 rows × 1 columns

As you can see, the result is a table where the row index is the null hypothesis and the row values are the p-values.

To build the confidence interval, all we need to do is filter out the H_0s that gave us a low P-value. Remember that low p-value means that the data we have is unlikely to have come from that null. For instance, if we define the significant level α to be 0.1, we remove H_0s that have P-value lower than 0.1.

```
def confidence_interval_from_p_values(p_values, alpha=0.1):
    big_p_values = p_values[p_values.values >= alpha]
    return pd.DataFrame({
        f"{int(100-alpha*100)}_ci_lower": big_p_values.index.min(),
        f"{int(100-alpha*100)}_ci_upper": big_p_values.index.max(),
    }, index=[p_values.columns[0]])
```

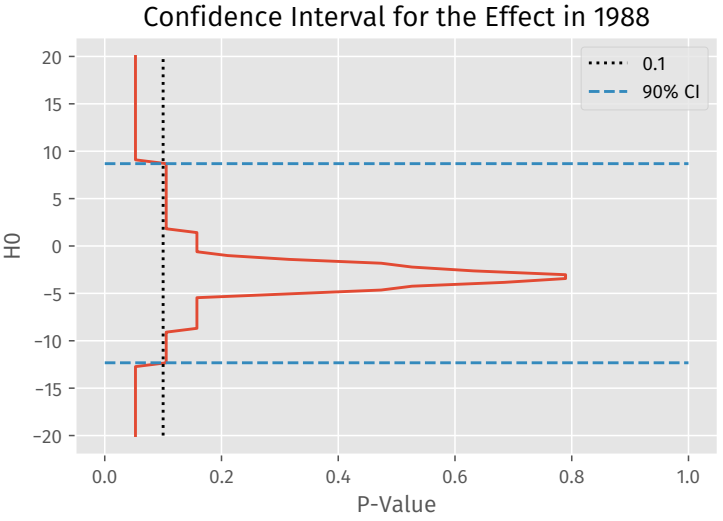
```
confidence_interval_from_p_values(p_values_df)
```

	90_ci_lower	90_ci_upper
1988	-12.323232	8.686869

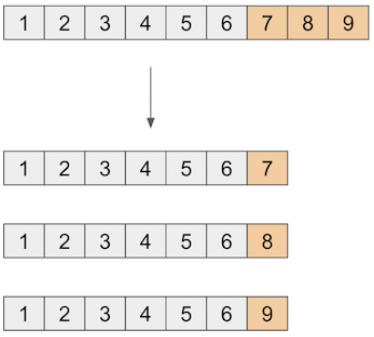
This gives us the confidence interval for the effect in 1988.

We can also plot the H_0 by P-value to better understand how this confidence interval was obtained. In the figure below, the dashed line is the 0.1 line, which is the α we've specified. The blue lines mark the confidence intervals. H_0 outside these lines have a P-value lower than 0.1.

```
plt.plot(p_values_df[1988], p_values_df.index)
plt.xlabel("P-Value")
plt.ylabel("H0")
plt.vlines(0.1, nulls.min(), nulls.max(), color="black", ls="dotted",
  ↪ label="0.1")

plt.hlines(confidence_interval_from_p_values(p_values_df)["90_ci_upper"], 0, 1,
  ↪ color="C1", ls="dashed")
plt.hlines(confidence_interval_from_p_values(p_values_df)["90_ci_lower"], 0, 1,
  ↪ color="C1", ls="dashed", label="90% CI")

plt.legend()
plt.title("Confidence Interval for the Effect in 1988");
```



All there's left to do is repeat the procedure above for each time period. This means that, for each post intervention year, appending it to the end of the pre-intervention period to create the augmented dataset and then computing the confidence interval just like we've done above.



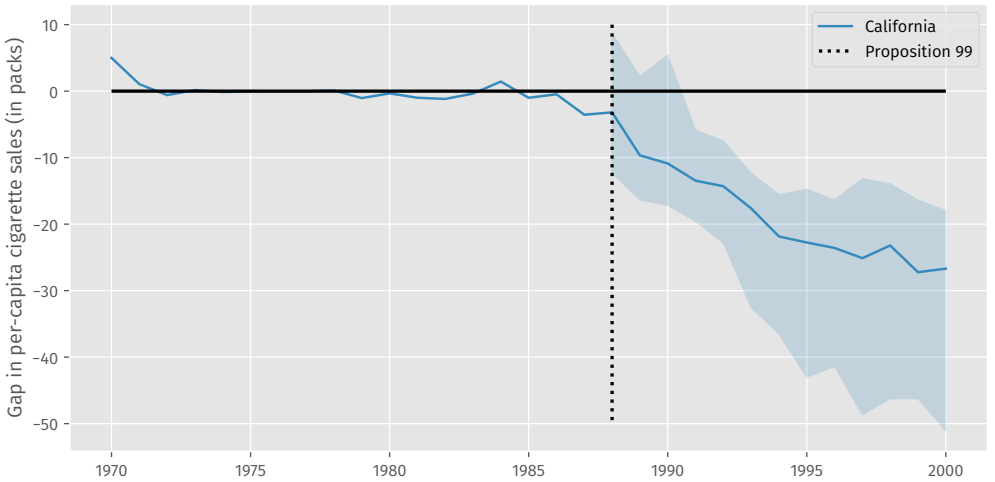

```
def compute_period_ci(df, state, nulls, intervention_start, period, model,
  ↪ alpha=0.1):
    p_vals = p_val_grid(df=df,
                        state=state,
                        nulls=nulls,
                        intervention_start=intervention_start,
                        period=period,
                        model=model)

    return confidence_interval_from_p_values(p_vals, alpha=alpha)


def confidence_interval(df, state, nulls, intervention_start, window, model,
  ↪ alpha=0.1, jobs=4):
    return pd.concat([compute_period_ci(df, state, nulls, intervention_start,
      ↪ period, model, alpha)
                      for period in range(intervention_start,
                        ↪ intervention_start+window)])
```

We are now ready to compute the confidence interval for all the post-intervention periods

```
model = SyntheticControl()

nulls = np.linspace(-60, 20, 100)

ci_df = confidence_interval(
    data,
    "california",
    nulls=nulls,
    intervention_start=1988,
    window=2000 - 1988 + 1,
    model=model
)

ci_df
```

	90_ci_lower	90_ci_upper
1988	-12.323232	8.686869
1989	-16.363636	2.222222
1990	-17.171717	5.454545
1991	-19.595960	-5.858586
1992	-22.828283	-7.474747
1993	-32.525253	-12.323232
1994	-36.565657	-15.555556
1995	-43.030303	-14.747475
1996	-41.414141	-16.363636
1997	-48.686869	-13.131313
1998	-46.262626	-13.939394
1999	-46.262626	-16.363636
2000	-51.111111	-17.979798

```
plt.figure(figsize=(10,5))
plt.fill_between(ci_df.index, ci_df["90_ci_lower"], ci_df["90_ci_upper"],
  ↪ alpha=0.2,  color="C1")
plt.plot(pred_data["california"].index, pred_data["residuals"],
  ↪ label="California", color="C1")
plt.hlines(y=0, xmin=1970, xmax=2000, lw=2, color="Black")
plt.vlines(x=1988, ymin=10, ymax=-50, linestyle=":", color="Black", lw=2,
  ↪ label="Proposition 99")
plt.legend()
plt.ylabel("Gap in per-capita cigarette sales (in packs)");
```



Reference

This Appendix based on the paper *An Exact and Robust Conformal Inference Method for Counterfactual and Synthetic Controls*, by Victor Chernozhukov, Kaspar Wüthrich, Yinchu Zhu. I would like to give special thanks to Kaspar, who clarified a lot of the questions I had.

For additional resources on Synthetic Controls, check out *Using Synthetic Controls: Feasibility, Data Requirements, and Methodological Aspects*, by Alberto Abadie (2021).

Index

Page ranges in the index mark the chapter devoted to a topic; other entries point to where it is put to work elsewhere.

Numbers

2SLS, *see* two-stage least squares (2SLS)

A

A/B test, 40, 62, 81
always takers, 129, 246
association and causation, 5, 17
ATE, *see* average treatment effect (ATE)
ATT, *see* average treatment effect on the treated (ATT)
average treatment effect (ATE), xv, 3–15, 29, 62, 82, 90, 124, 132, 141, 167, 184, 239, 255, 280, 299, 313, 327, 345, 371, 387, 507, 511
average treatment effect on the treated (ATT), 8, 198, 391

B

backdoor path, 46, 200
bias
    confounding, 51, 71, 175, 313, 348, 454, 471
    omitted variable, 73, 121, 206, 311, 453
    selection, 54, 102
bootstrap, 168, 180, 250
bunching, 248

C

CATE, *see* conditional average treatment effect (CATE)
causal graphs, xv, 43–59, 70, 94, 131, 161, 200
central limit theorem, 31, 150
collider, 47
compliers, 129, 247
conditional average treatment effect (CATE), xv, 279–293, 313, 327, 350, 369, 511
conditional independence, 43, 143, 161, 199, 315
confidence interval, 25–42, 62, 90, 123, 153, 169, 180, 195, 250, 268, 307, 437, 473, 526
conformal inference, 515–532
confounding, 11, 43, 70, 81, 89, 111, 133, 142, 159, 178, 199, 280, 311, 343, 371, 396, 453, 470, 490, 510
controls, good and bad, 89–109, 505
counterfactual, 5, 17, 106, 134, 150, 187, 198, 232, 235, 331, 358, 382, 389, 436, 470, 490, 516
cross-validation, 170, 263, 463, 490
cumulative gain curve, 295–311, 318, 329, 460

cumulative sensitivity curve, 295–311, 351, 459, 511
curse of dimensionality, 144, 174

D
debiased machine learning, *see* double/debiased machine learning
decision trees, 256, 328, 465
defiers, 129, 247
diff-in-diff, *see* difference-in-differences
difference-in-differences, 187–197, 213, 255, 387–417
donor pool, 216
double/debiased machine learning, 341–367, 371, 396
doubly robust estimation, 177–185
dummy variables, 62, 75–88, 100, 114, 141, 165, 178, 193, 201, 392, 423

E
elasticity, 320, 351, 381, 453, 482, 503, 511
event study, 404
exchangeability, 12, 313
exclusion restriction, 112, 134

F
first stage, 121, 134, 250, 336, 454
fixed effects, 197–212, 344, 396, 426
Frisch–Waugh–Lovell theorem, 344, 393, 453
fundamental problem of causal inference, 7, 283
FWL theorem, *see* Frisch–Waugh–Lovell theorem

H
heterogeneous treatment effects, 83, 255, 279–293, 319, 327, 350, 369–385, 387, 437, 510
heteroskedasticity, 76
hypothesis testing, 35, 521

I
individual treatment effect, 8, 61, 313, 354, 393
instrumental variables (IV), 43, 111–139, 200, 246, 255, 465
IV, *see* instrumental variables (IV)

K
kernel weighting, 242

L
LATE, *see* local average treatment effect (LATE)
linear regression, 61–88, 90, 112, 151, 163, 177, 199, 217, 235, 284, 297, 314, 328, 344, 391, 453
local average treatment effect (LATE), 129–139, 239, 281
logistic regression, 127, 165, 178

M
matching, 141–156, 161, 199, 255, 411
mediator, 56, 103
meta-learners, 327–339, 341
  S-learner, 328, 342
  T-learner, 331, 342
  X-learner, 334, 342
Moivre's equation, 25
monotonicity, 135, 247

N
nearest neighbours, 147
never takers, 129, 246
non-compliance, 129–139, 159, 178, 246

O
OLS, *see* ordinary least squares (OLS)
omitted variable bias, *see* bias
ordinary least squares (OLS), 123, 136, 152, 206, 218, 346, 392
orthogonalization, 342, 453–467, 512
overfitting, 165, 221, 262, 297, 319, 330, 346, 463, 474, 517

P
p-value, 25–42, 63, 86, 91, 119, 230, 240, 521
panel data, 197–212, 387, 417
parallel trends, 193, 198, 396, 422
permutation test, 525
personalization, xv, 255, 280, 299, 314, 369, 493
placebo test, 95, 129, 228, 444, 520
positivity, 164, 347, 479

potential outcomes, xv, 3–15, 17, 43, 61, 90, 134, 157, 189, 198, 216, 236, 313, 369, 420
propensity score, 155, 157–177, 199, 255, 316, 334, 453, 467–484
propensity weighting, 478
Proposition 99, 214, 417, 515

Q

quarter of birth, 115

R

R-learner, 338, 352
randomized controlled trial (RCT), 17–24, 62, 130, 239, 387, 482
randomized experiment, *see* randomized controlled trial (RCT)
RCT, *see* randomized controlled trial (RCT)
RDD, *see* regression discontinuity design (RDD)
reduced form, 113, 137
regression discontinuity design (RDD), 235–252, 255
regularization, 222, 331, 365
running variable, 236

S

S-learner, *see* meta-learners
SDID, *see* synthetic difference-in-differences (SDID)
selection bias, *see* bias
sheepskin effect, 245
staggered adoption, 418
standard error, 25–42, 78, 93, 121, 137, 168, 193, 206, 213, 238, 437
subclassification, 143
synthetic control, 194, 213–233, 417, 515–532
synthetic difference-in-differences (SDID), 417–449

T

T-learner, *see* meta-learners
target transformation, 313–326, 352, 369
TWFE, *see* two-way fixed effects (TWFE)
two-stage least squares (2SLS), 111–128, 137
two-way fixed effects (TWFE), 387–416, 423

X

X-learner, *see* meta-learners

* * *

This book was typeset with LuaLaTeX and the memoir class,
directly from its Jupyter notebooks by way of pandoc.

The text face is XCharter, Michael Sharpe's extension of Matthew Carter's Charter, with XCharter Math for the mathematics. Headings and tables are set in Fira Sans and code in Raph Levien's Inconsolata.